1POINT3ACRES, PRACHUB, HACKTHEROUNDS

Google interview questions

178 questions in 10 topics, one PDF. Every question starts on its own page.

7	133	38
EASY	MEDIUM	HARD

Topics (tap one to jump to it)

1. Heaps, Stacks and Queues	33 questions | page 7
2. Arrays, Hashing and Two Pointers	30 questions | page 112
3. Trees and Tries	22 questions | page 203
4. Graphs and Search	21 questions | page 276
5. Design, Streaming and Scale	16 questions | page 344
6. Dynamic Programming	15 questions | page 402
7. Binary Search and Sorting	13 questions | page 445
8. Strings, Parsing and Recursion	12 questions | page 487
9. Greedy and Math	10 questions | page 528
10. Grids and Simulation	6 questions | page 562

- The master contents lists every main topic and question; each row is a link.
- Inside a topic, the bar at the top of each page moves Prev / Contents / Next; the link at the bottom returns here.
- Questions run Easy, then Medium, then Hard inside each main topic, newest date first.

Go to Contents >

Contents

178 questions | tap a topic, a main topic or a question to jump to it

1. Heaps, Stacks and Queues (33) Go >

HEAPS AND TOP-K (12)

E 1. Find k pairs with smallest sums — PracHub 2025-11-01
M 2. Active Communication Top Users — 1Point3Acres 2026-09-22
M 3. Sliding-Window Mean After Removing the Largest K Values — PracHub 2026-09-01
M 4. Streaming Median (LC 295) with Follow-ups — 1Point3Acres 2026-08-03
M 5. Batch and Streaming Log Deduplication — 1Point3Acres 2026-07-23
M 6. Top-K Frequent in Large Logs — 1Point3Acres 2026-07-19
M 7. Rearrange Tasks With Cooldown — PracHub 2026-06-03
M 8. Determine if a 14-tile hand is winning — PracHub 2026-03-09
M 9. Implement frequency + distance top-K queries — PracHub 2026-02-12
M 10. Playlist Shuffle No Adjacent Artists — HackTheRounds no date
H 11. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant — 1Point3Acres 2026-10-09
H 12. Return top-k subset sums in descending order — PracHub 2026-01-07

INTERVALS AND SCHEDULING (10)

M 13. Meeting Rooms / Interval Scheduling Variants — 1Point3Acres 2026-09-29
M 14. Union of Sorted Interval Lists — 1Point3Acres 2026-08-16
M 15. Solve Rooms and Top-K Streams — PracHub 2026-06-05
M 16. Compute minimum number of rooms needed — PracHub 2026-02-12
M 17. Simulate room assignments for scheduled meetings — PracHub 2026-01-06
M 18. Compute servers needed for daily recurring jobs — PracHub 2025-11-05
M 19. Meeting Rooms III Variant — HackTheRounds no date
H 20. Compute Turnstile Crossing Times — PracHub 2026-05-04
H 21. Minimize travel time with optional meeting point — PracHub 2026-01-06
H 22. Compute city skyline outline — PracHub 2025-12-08

SHORTEST PATHS WITH A HEAP (5)

M 23. Find A Threshold-Limited Path With Minimum Required Safety — PracHub 2026-07-06
M 24. Compute shortest paths with blocked nodes — PracHub 2025-09-06
H 25. Find minimum threshold enabling grid path — PracHub 2026-02-11
H 26. Compute minimax grid path and network delay — PracHub 2026-01-22
H 27. Minimize maximum height along a grid path — PracHub 2026-01-04

STACKS (5)

M 28. Visible People in a Line (Last Person, Then Everyone) — 1Point3Acres 2026-10-07
M 29. Expand Recursive Key-Value Placeholders in a Template — PracHub 2026-08-04
M 30. Decode String — 1Point3Acres 2026-07-29
M 31. Nested Function Expression Evaluator — 1Point3Acres 2026-06-10
M 32. Find longest increasing contiguous subarray — PracHub 2026-01-06

QUEUES (1)

M 33. Implement Python's itertools.tee — 1Point3Acres 2026-08-20

2. Arrays, Hashing and Two Pointers (30) Go >

HASHING (13)

E 1. Robot Status Message Deduplication Window — 1Point3Acres 2026-09-10
E 2. Parity Alternating Subarray Query — HackTheRounds no date
M 3. Connected Crop Arrangement on a Grid — 1Point3Acres 2026-10-04

Contents

178 questions | tap a topic, a main topic or a question to jump to it

M 4. Next-Word Predictor — 1Point3Acres 2026-09-02
M 5. Filter Repeated Robot Status Messages — PracHub 2026-07-05
M 6. Website Activity Analytics: Unique Users + Session Time — 1Point3Acres 2026-06-26
M 7. Find a Horizontal Cut That Bisects Rectangle Area — PracHub 2026-06-04
M 8. Gold Chain Split into Two Equal Halves — 1Point3Acres 2026-04-21
M 9. Solve array-sum and city-community problems — PracHub 2026-02-20
M 10. Check pivot after removing one element — PracHub 2026-02-12
M 11. Reconstruct original array from doubled shuffle — PracHub 2026-01-04
H 12. Maximum-Area Rectangle from Planar Points — 1Point3Acres 2026-09-29
H 13. Smallest Missing Positive Integer in Linear Time and Constant Extra Space — Space PracHub 2026-09-09

TWO POINTERS AND SLIDING WINDOW (10)

M 14. Range Duplicate Check with Constant-Time Queries — 1Point3Acres 2026-09-29
M 15. Temperature Monitoring Data Structure — 1Point3Acres 2026-09-10
M 16. Piano Hand Movement Grouping — 1Point3Acres 2026-07-14
M 17. Sum of Good Subarrays (Adjacent Diff ±1) — 1Point3Acres 2026-02-08
M 18. Run-Length Vector Storage + Dot Product — 1Point3Acres 2026-01-21
M 19. Maximize 1D deliveries within distance and minimize total distance — PracHub 2025-12-27
M 20. Two-Day Log Intersection under Memory Limit — 1Point3Acres 2025-11-27
M 21. Shortest Subarray with K Distinct Integers — HackTheRounds no date
H 22. Necklace Cut into Two Halves with Equal D/R Counts — 1Point3Acres 2026-05-03
H 23. Find longest duplicated substring — PracHub 2025-12-08

ARRAYS AND PREFIX SUMS (7)

M 24. Longest Same-Color Grid Sequence — 1Point3Acres 2026-09-08
M 25. Solve several coding interview problems — PracHub 2026-03-04
M 26. Min deletions to avoid overlap in first k — PracHub 2026-02-12
M 27. Add two big integers from digit lists — PracHub 2025-11-24
M 28. Minimize calls to find all bad test pairs — PracHub 2025-11-05
M 29. Rotate Any m×n 2D Matrix — 1Point3Acres no date
M 30. Sum of Two Squares in Two Ways — HackTheRounds no date

3. Trees and Tries (22) Go >

TREES (15)

E 1. Find right-side view of binary tree — PracHub 2025-11-01
M 2. Expression Tree Single-Leaf Mutation — 1Point3Acres 2026-09-27
M 3. Compute File System Total Size — 1Point3Acres 2026-09-02
M 4. Nested List Weight Sum (LC 339) with Follow-Ups — 1Point3Acres 2026-08-16
M 5. Longest Consecutive Path in Binary Tree — 1Point3Acres 2026-07-02
M 6. Employee Hierarchy Add / Delete — 1Point3Acres 2026-06-10
M 7. Streaming Insert + Find K-th Largest — 1Point3Acres 2026-02-04
M 8. Find mode in a trinary search tree — PracHub 2025-11-16
M 9. Solve tree, graph, sliding-window problems — PracHub 2025-08-04
M 10. Implement zigzag level-order traversal — PracHub 2025-08-01
M 11. Lowest Common Ancestor of a Binary Tree III — HackTheRounds no date
M 12. Lowest Common Ancestor — HackTheRounds no date
H 13. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree — Tree PracHub 2026-09-16
H 14. Huffman Tree Construction — 1Point3Acres 2026-06-01
H 15. Compute distance-sum from every tree node — PracHub 2026-01-22

Contents

178 questions | tap a topic, a main topic or a question to jump to it

TRIES (7)

- M 16. URL Request Handler Router — 1Point3Acres 2026-10-16
- M 17. Trie Prefix Search / Autocomplete — 1Point3Acres 2026-07-02
- M 18. Greedy Longest-Match Dictionary Tokenizer — 1Point3Acres 2026-06-15
- M 19. Check if all substrings are dictionary words — PracHub 2026-03-11
- M 20. Return dictionary words matching a prefix — PracHub 2026-03-09
- M 21. Solve chat, grid paths, and car rentals — PracHub 2025-12-02
- H 22. Implement Longest-Match Text Replacement — PracHub 2026-01-20

4. Graphs and Search (21) Go >

BFS AND DFS (11)

- M 1. Count Islands in a Grid and Track the Count as Land Cells Are Added — PracHub 2026-10-02
- M 2. Path Minimizing the Maximum Cell (Swim in Rising Water) — 1Point3Acres 2026-09-29
- M 3. Rotten Oranges / Multi-Source BFS (taxis) — 1Point3Acres 2026-09-08
- M 4. Count Clusters of 2D Points Within a Radius — PracHub 2026-06-18
- M 5. Find shortest relationship path using BFS — PracHub 2026-02-12
- M 6. Validate parent array forms a tree — PracHub 2026-02-11
- M 7. Solve matrix groups and recipe inventory — PracHub 2026-01-22
- M 8. Compute distance to nearest taxi in grid — PracHub 2026-01-10
- M 9. Validate course catalog dependencies — PracHub 2025-08-01
- M 10. Solve Grid and Counting Problems — PracHub 2025-01-25
- H 11. Infection Spread on a Grid (Cellular Automaton) — PracHub 2026-06-14

TOPOLOGICAL SORT AND CYCLES (4)

- M 12. Template Variable Expansion with Follow-Ups — 1Point3Acres 2026-10-17
- M 13. Course Scheduling and Dependency Variants — 1Point3Acres 2026-10-04
- M 14. Determine whether two nodes are related — PracHub 2026-01-22
- M 15. Shortest Directed Cycle Through a Node — 1Point3Acres no date

UNION-FIND (3)

- M 16. Number of Islands — 1Point3Acres 2026-09-30
- M 17. 2D Points Connected Components by Radius — 1Point3Acres 2026-08-04
- H 18. Friend Graph with Timeline (Friend / Unfriend Events) — 1Point3Acres 2026-09-29

SHORTEST PATHS (3)

- M 19. Train / Flight Schedule Reachability — 1Point3Acres 2026-10-08
- M 20. Shortest Path Nodes and Escaping a Pursuer — 1Point3Acres 2026-09-29
- M 21. Minimum Direction Violations in a Directed Graph — 1Point3Acres 2026-07-19

5. Design, Streaming and Scale (16) Go >

DATA STRUCTURE DESIGN (14)

- E 1. Implement Batched Undo/Redo Layer — PracHub 2026-02-04
- M 2. Implement a Reusable Locker Allocation System — PracHub 2026-08-14
- M 3. Build Prefix Lookup with a Trie — PracHub 2026-07-03
- M 4. Design a restaurant waitlist system — PracHub 2026-03-01
- M 5. Determine Reporting Relationships — PracHub 2026-02-22
- M 6. Implement Trie Insert and Exact-Word Search — PracHub 2026-02-20
- M 7. Design Tic-Tac-Toe With K Players — PracHub 2026-02-12
- M 8. Restaurant Waitlist: join / delete / find_first_match — 1Point3Acres 2026-02-01

Contents

178 questions | tap a topic, a main topic or a question to jump to it

M 9. Design a logger that returns top-K messages — PracHub 2026-01-06
M 10. Implement iterator merging lists with filtering — PracHub 2026-01-06
M 11. Design set with O(1) random access — PracHub 2026-01-01
M 12. Match people to questions using tags and priorities — PracHub 2025-12-27
M 13. Design server allocation for multi-type nodes — PracHub 2025-09-06
M 14. Hit Counter — HackTheRounds no date

STREAMING AND SCALE (2)

M 15. Find top/bottom-k words in list or stream — PracHub 2026-01-11
M 16. Design an elevator control system — PracHub 2025-09-06

6. Dynamic Programming (15) Go >

M 1. Bounded Knapsack Variant — 1Point3Acres 2026-10-16
M 2. Longest Non-Decreasing Subarray with One Replacement — 1Point3Acres 2026-10-04
M 3. Grid Path Count with Diagonal Moves and Checkpoints — 1Point3Acres 2026-09-29
M 4. Minimum-Cost Tree Disconnection — 1Point3Acres 2026-09-09
M 5. Target Expressions from Ordered Digits — 1Point3Acres 2026-08-13
M 6. Count Word Decompositions into Chemical Element Symbols — 1Point3Acres 2026-07-14
M 7. Fence Painting Minimum Operations (Gemini Team) — 1Point3Acres 2026-04-27
M 8. Maximize sum without choosing adjacent elements — PracHub 2026-01-06
M 9. Jump Game with Profit — 1Point3Acres no date
M 10. Count Same-Color Squares in Matrix — 1Point3Acres no date
M 11. Russian Doll Envelopes — 1Point3Acres no date
M 12. Reverse Coin Change — HackTheRounds no date
H 13. Count Perfect Wake Numbers up to N — 1Point3Acres 2026-05-07
H 14. Tree Distance Sum (Re-root DP) — 1Point3Acres 2026-01-19
H 15. Determine winner in stack-merging game — PracHub 2026-01-06

7. Binary Search and Sorting (13) Go >

BINARY SEARCH (7)

E 1. Check an Interval Against Existing Intervals for Conflicts — PracHub 2026-08-08
M 2. Cutting Ribbons (LC 1891) — 1Point3Acres 2026-08-16
M 3. First Bad Version with Parallel Search Follow-up — 1Point3Acres 2026-06-15
M 4. Find median of merged RLE arrays — PracHub 2026-01-06
M 5. Solve Banana Speed and Interval Merge — PracHub 2025-01-27
H 6. Find Maximum Envelope Nesting — PracHub 2026-03-18
H 7. Count of Smaller Numbers After Self — HackTheRounds no date

SORTING (6)

M 8. Progressive Array Partitioning — 1Point3Acres 2026-08-11
M 9. Streaming Points: Emit K Within Distance — 1Point3Acres 2026-06-23
M 10. Find Containing Range — PracHub 2026-06-04
M 11. Check if all substrings are anagrams of words — PracHub 2026-03-11
M 12. Find shared objects across two log files — PracHub 2025-11-05
H 13. Find safe travel intervals between planet influences — PracHub 2025-11-22

Contents

178 questions | tap a topic, a main topic or a question to jump to it

8. Strings, Parsing and Recursion (12) Go >

STRINGS (8)

M 1. Snake Case to Camel Case with Repository-Wide Refactor — 1Point3Acres 2026-07-14
M 2. Check Whether Digits Are Monotone Increasing — PracHub 2026-07-04
M 3. Can board states be transformed? — PracHub 2026-04-01
M 4. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) — 1Point3Acres 2026-02-01
M 5. Decide string transform with directional tokens and walls — PracHub 2025-07-15
H 6. Find the Longest Straight Run of One Color — PracHub 2026-08-23
H 7. Convert Valid Snake Case to Lower Camel Case — PracHub 2026-04-16
H 8. Find Dictionary Matches for a Jammed String — PracHub 2026-04-09

RECURSION AND BACKTRACKING (3)

M 9. Mahjong Winning Hand Detection (14 Tiles) — 1Point3Acres 2026-10-09
M 10. Count Prefix-Matching Simple Paths in a Labeled Graph — PracHub 2026-08-09
H 11. Matrix Flower Placement with House Adjacency — 1Point3Acres 2026-05-31

PARSING (1)

E 12. Implement string formatting and request tracking tasks — PracHub 2026-02-11

9. Greedy and Math (10) Go >

MATH AND BITS (9)

M 1. Find the Largest Monotone Increasing Number — PracHub 2026-07-04
M 2. Choose optimal guesses for green-only Wordle — PracHub 2025-10-14
M 3. Trip Expense Settlement — HackTheRounds no date
H 4. Check Digits and Combine Ranges — PracHub 2026-03-29
H 5. Find secret word with match-count feedback — PracHub 2026-02-11
H 6. Wordle-Style 5-Letter Minimum-Guess Strategy — 1Point3Acres 2025-12-15
H 7. Shuffle array using random integer API — PracHub 2025-12-08
H 8. Find maximum value in huge text file — PracHub 2025-12-08
H 9. Find All Bad Pairs Using runTest Oracle — 1Point3Acres 2025-11-27

GREEDY (1)

M 10. Min boards to cover roof holes — PracHub 2026-02-12

10. Grids and Simulation (6) Go >

SIMULATION (3)

M 1. Tic-Tac-Toe System with K Players and 3-in-a-Row Win — 1Point3Acres 2026-05-18
M 2. Gate Entry / Exit Time Resolution with Tie-Break Rules — 1Point3Acres 2026-05-04
H 3. Babylon Tile-Stack Merge Game — 1Point3Acres 2025-12-29

GRIDS (3)

H 4. Count Overlapping Rectangle Updates on a Grid — PracHub 2026-07-10
H 5. Simulate In-Place Cellular State Updates — PracHub 2026-05-04
H 6. Count same-color squares in a character grid — PracHub 2025-11-22

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 1 OF 10 | CORE TOPIC

Heaps, Stacks and Queues

33 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	24	8
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Heaps, Stacks and Queues | 33 questions | tap a title to jump to it

HEAPS AND TOP-K (12)

E 1. Find k pairs with smallest sums — PracHub 2025-11-01
M 2. Active Communication Top Users — 1Point3Acres 2026-09-22
M 3. Sliding-Window Mean After Removing the Largest K Values — PracHub 2026-09-01
M 4. Streaming Median (LC 295) with Follow-ups — 1Point3Acres 2026-08-03
M 5. Batch and Streaming Log Deduplication — 1Point3Acres 2026-07-23
M 6. Top-K Frequent in Large Logs — 1Point3Acres 2026-07-19
M 7. Rearrange Tasks With Cooldown — PracHub 2026-06-03
M 8. Determine if a 14-tile hand is winning — PracHub 2026-03-09
M 9. Implement frequency + distance top-K queries — PracHub 2026-02-12
M 10. Playlist Shuffle No Adjacent Artists — HackTheRounds no date
H 11. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant — 1Point3Acres 2026-10-09
H 12. Return top-k subset sums in descending order — PracHub 2026-01-07

INTERVALS AND SCHEDULING (10)

M 13. Meeting Rooms / Interval Scheduling Variants — 1Point3Acres 2026-09-29
M 14. Union of Sorted Interval Lists — 1Point3Acres 2026-08-16
M 15. Solve Rooms and Top-K Streams — PracHub 2026-06-05
M 16. Compute minimum number of rooms needed — PracHub 2026-02-12
M 17. Simulate room assignments for scheduled meetings — PracHub 2026-01-06
M 18. Compute servers needed for daily recurring jobs — PracHub 2025-11-05
M 19. Meeting Rooms III Variant — HackTheRounds no date
H 20. Compute Turnstile Crossing Times — PracHub 2026-05-04
H 21. Minimize travel time with optional meeting point — PracHub 2026-01-06
H 22. Compute city skyline outline — PracHub 2025-12-08

SHORTEST PATHS WITH A HEAP (5)

M 23. Find A Threshold-Limited Path With Minimum Required Safety — PracHub 2026-07-06
M 24. Compute shortest paths with blocked nodes — PracHub 2025-09-06
H 25. Find minimum threshold enabling grid path — PracHub 2026-02-11
H 26. Compute minimax grid path and network delay — PracHub 2026-01-22
H 27. Minimize maximum height along a grid path — PracHub 2026-01-04

STACKS (5)

M 28. Visible People in a Line (Last Person, Then Everyone) — 1Point3Acres 2026-10-07
M 29. Expand Recursive Key-Value Placeholders in a Template — PracHub 2026-08-04
M 30. Decode String — 1Point3Acres 2026-07-29
M 31. Nested Function Expression Evaluator — 1Point3Acres 2026-06-10
M 32. Find longest increasing contiguous subarray — PracHub 2026-01-06

QUEUES (1)

M 33. Implement Python's itertools.tee — 1Point3Acres 2026-08-20

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

EASY | IN APP LC 373 | PracHub | Technical Screen | 1 of 33

Find k pairs with smallest sums

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 119/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #373 Find K Pairs with Smallest Sums

LeetCode match: LC 373 Find K Pairs with Smallest Sums

Posted on PracHub: 2025-11-01 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-k-pairs-with-smallest-sums

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **`k`** pairs **`(arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **`k`** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: in
t) -> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17 The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

APPROACH

Min-heap seeded with (arr1[i]+arr2[0], i, 0) for the first min(k, len1) indices; pop, then push (i, j+1). O(k log k). Return all pairs if fewer than k exist.

HOW TO THINK ABOUT IT

Because both arrays are sorted, the pair (i, j + 1) is never smaller than (i, j), so each row i of the 'matrix of sums' is sorted. The k smallest pairs are the k smallest items of a collection of sorted rows, which is a heap problem: start with the first element of each row (or the first few) and each time you pop a pair, push its successor in the row.

Basis: LeetCode 373, Find K Pairs with Smallest Sums (two sorted arrays).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a, b, k):
    pairs = [(x, y) for x in a for y in b]
    pairs.sort(key=lambda p: p[0] + p[1])
    return pairs[:k]
```

TIME	SPACE
O(m * n log(m * n))	O(m * n)

WHY IT IS SLOW

m * n pairs sorted: with 10^5 elements in each array that is 10^10 pairs. Because both arrays are sorted, a min-heap that starts from each x's first pair and expands only the next pair in that row is O(k log k).

OPTIMAL SOLUTION · Min-heap over the sorted rows

TIME	SPACE
O(k log min(k, m))	O(min(k, m))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The arrays are not sorted.

Sort both first (O(m log m + n log n)); the heap method needs sorted rows. For one huge array you may instead use a heap of size k with a selection of candidates.

FOLLOW-UP 2 Only the k-th smallest sum, with huge arrays.

Binary search the sum value and count how many pairs have sum at most it with two pointers (O(m + n) per check): O((m + n) log(range)) with O(1) space, no heap.

FOLLOW-UP 3 Three arrays, or k pairs with the largest sums.

For largest sums run the same algorithm on the reversed arrays (or negate the values). For three arrays use a heap of triples (i, j, l), pushing the three neighbours with a visited set. **Cost:** O(k log k) with the heap of triples; reversing the arrays gives the largest sums at the same cost.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find k pairs with smallest sums
#
# You are given two **sorted** integer arrays arr1 and
# arr2 (each sorted in non-decreasing order), and an
# integer k. Consider all possible pairs formed by taking
# one element from arr1 and one element from arr2. For
# indices i and j, the pair is (arr1[i], arr2[j]) with sum
# arr1[i] + arr2[j]. Your task is to return the **k
# pairs** (arr1[i], arr2[j]) with the smallest sums. If
# there are fewer than k total possible pairs, return all
# possible pairs. You may return the pairs in any order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The arrays are not sorted.
# 2. Only the k-th smallest sum, with huge arrays.
# 3. Three arrays, or k pairs with the largest sums.
#
# INTERVIEW NOTES
# Because both arrays are sorted, the pair (i, j + 1) is
# never smaller than (i, j), so each row i of the 'matrix
# of sums' is sorted. The k smallest pairs are the k
# smallest items of a collection of sorted rows, which is
# a heap problem: start with the first element of each row
# (or the first few) and each time you pop a pair, push
# its successor in the row.
#
# BRUTE FORCE (say it first): Generate all pairs and sort
# Build all m * n pairs, sort by sum and take k. Correct
# and the oracle; with 10^5 elements in each array it
# would create 10^10 pairs. Time O(m * n log(m * n)).
# Space O(m * n).


import heapq
from bisect import bisect_right


# BRUTE FORCE (say it first): Generate all pairs and sort
# | time O(m * n log(m * n))
def k_pairs_all(a, b, k):
    return sorted(
        ((x, y) for x in a for y in b),
        key=lambda p: (p[0] + p[1], p),
    )[:k]
```

```
# HELPERS for follow-up 2
def count_at_most(a, b, limit):
    # pairs with sum <= limit: for every x binary search
    # in b
    return sum(bisect_right(b, limit - x) for x in a)


# BASE QUESTION (optimal): Min-heap over the sorted rows
# Idea: Push (a[i] + b[0], i, 0) for the first min(k, m)
# values of a. Pop the smallest, record the pair, and push
# the next pair in the same row, (a[i] + b[j + 1], i, j +
# 1). After k pops you have the k smallest sums.
def k_smallest_pairs(a, b, k):
    # a and b sorted: a heap holds the best unused pair of
    # each row i; popping (i, j) offers (i, j + 1)
    if not a or not b or k <= 0:
        return []
    rows = range(min(k, len(a)))
    heap = [(a[i] + b[0], i, 0) for i in rows]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append((a[i], b[j]))
        if j + 1 < len(b):
            nxt = a[i] + b[j + 1]
            heapq.heappush(heap, (nxt, i, j + 1))
    return out


# FOLLOW-UP 1: The arrays are not sorted.
# Answer: Sort both first (O(m log m + n log n)); the heap
# method needs sorted rows. For one huge array you may
# instead use a heap of size k with a selection of
# candidates.
def k_smallest_pairs_unsorted(a, b, k):
    # the heap method needs sorted rows: sort first
    return k_smallest_pairs(sorted(a), sorted(b), k)


# FOLLOW-UP 2: Only the k-th smallest sum, with huge
# arrays.
# Answer: Binary search the sum value and count how many
# pairs have sum at most it with two pointers (O(m + n)
```

```
# per check): O((m + n) log(range)) with O(1) space, no
# heap.
def kth_smallest_sum(a, b, k):
    # huge arrays: binary search the SUM, no heap needed
    lo, hi = a[0] + b[0], a[-1] + b[-1]
    while lo < hi:
        mid = (lo + hi) // 2
        if count_at_most(a, b, mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo


# FOLLOW-UP 3: Three arrays, or k pairs with the largest
# sums.
# Answer: For largest sums run the same algorithm on the
# reversed arrays (or negate the values). For three arrays
# use a heap of triples (i, j, l), pushing the three
# neighbours with a visited set.
def k_largest_pairs(a, b, k):
    # largest sums: negate the values and run the same
    # code
    neg_a = [-x for x in sorted(a, reverse=True)]
    neg_b = [-y for y in sorted(b, reverse=True)]
    pairs = k_smallest_pairs(neg_a, neg_b, k)
    return [(-x, -y) for x, y in pairs]


if __name__ == "__main__":
    a, b = [1, 7, 11], [2, 4, 6]
    three = k_smallest_pairs(a, b, 3)
    assert three == [(1, 2), (1, 4), (1, 6)]
    assert k_smallest_pairs(a, b, 4)[3] == (7, 2)
    assert k_smallest_pairs([1, 1, 2], [1, 2, 3], 2) == [
        (1, 1), (1, 1)]
    assert k_smallest_pairs([], [1], 3) == []
    # k bigger than the number of pairs: all of them
    assert len(k_smallest_pairs([1], [1, 2], 10)) == 2
    fixed = k_smallest_pairs_unsorted([7, 1], [4, 2], 2)
    assert fixed == [(1, 2), (1, 4)]
    assert k_largest_pairs(a, b, 2) == [(11, 6), (11, 4)]
    # the 4th smallest sum is 9 (7 + 2); check by sorting
    # all
    sums = sorted(x + y for x in a for y in b)
```

```
    for k in range(1, 10):
        assert kth_smallest_sum(a, b, k) == sums[k - 1]
```

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

Site tags: hashmap, set, sorting, top-k, streaming, medium

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen | 2 of 33

Active Communication Top Users

ASKED 2026-09-22 | 28 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 228/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-22 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/active-communication-top-users

QUESTION

Process a continuous stream of user pairs such as **`A-B`** and **`B-C`**; each pair creates an active communication edge, and the system should output users ordered by current communication count. Follow-ups add self-communication handling, duplicate communication dedupe, and top-K users by count. An additional API variant exposes register and mostActiveUser(), then asks for constant-time most-active lookup and a changing-K ranking extension.SWEhashmapsetsortingtop-kstreamingmediumFrequencyLowLast asked2026-09-22Stagephone-screen

APPROACH

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a pair A-B counts for both users, whether repeated pairs count once, how self communication is treated, how ties order, and whether k changes over time.

BRUTE FORCE TO SAY OUT LOUD

Recount from all edges and sort users on every query. O(E log U).

OPTIMAL IDEA

Normalize pairs (sorted tuple). Keep an edge set for deduplication, a dict of user to count, and either a sorted structure of (count, user) or a bucket map count to ordered users so an update moves a user by one bucket. Top-K reads the highest buckets. Note the edge set can use quadratic space in the number of users; say so.

DRY-RUN THESE CASES

A-B then B-A (duplicate), a self pair, ties on count, a new user, k larger than the number of users.

LIKELY FOLLOW-UPS

Constant-time most active user (track the maximum count with buckets), a changing k, or expiry of old edges.

PITFALLS

Claiming linear space while storing all pairs. Counting a duplicate edge twice.

HOW TO THINK ABOUT IT

Normalize the pair first (sorted tuple) so A-B and B-A are one edge, then keep an edge set for dedup and a count per user. The ranking part is a separate question: top-K comes from a heap, and the single most active user can be maintained in O(1) because counts only ever go up. Mention that the edge set uses quadratic space in the worst case.

Basis: Written. Assumed rules (confirm them): a pair 'A-B' is an unordered communication edge; repeated pairs count once; a user's activity is the number of distinct partners; order by activity descending, then name; self pairs (A-A) are ignored unless a flag says otherwise.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
pairs = set()
def add(a, b):
    pairs.add((min(a, b), max(a, b)))
def top_users():
    partners = defaultdict(set)
    for a, b in pairs:
        partners[a].add(b)
        partners[b].add(a)
    def rank(u):
        return (-len(partners[u]), u)
    return sorted(partners, key=rank)
```

TIME	SPACE
O(E + U log U) per query	**O(E)**

WHY IT IS SLOW

Each query rebuilds all counts and sorts, O(E + U log U). Maintaining counts incrementally with count buckets gives O(1) updates and an O(1) most-active lookup.

OPTIMAL SOLUTION · Edge set, counts per user, count buckets for the maximum

TIME	SPACE
O(1) per pair, O(U log k) for top-K, O(1) for the most active user	**O(E)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Self communication (A-A) should now count.

Pass a flag to the class: a self pair is one edge that adds one to that user's count (and only once). The dedup set already handles repeated self pairs.

FOLLOW-UP 2 Top-K with a changing K, or a fixed-size priority queue.

A heap of size K can be wrong after counts change, because an element outside it can overtake one inside. Re-run heapq.nsmallest on the count dict per query (O(U log K)) or maintain a sorted structure keyed by (count, name) updated on every increment.

FOLLOW-UP 3 The space claim.

Distinct pairs can number U^2 / 2 in the worst case, so the edge set is quadratic in the users; the counts alone are linear. If duplicates are not a concern, drop the set and keep O(U).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Active Communication Top Users
#
# Process a continuous stream of user pairs such as A-B
# and B-C; each pair creates an active communication edge,
# and the system should output users ordered by current
# communication count. Follow-ups add self-communication
# handling, duplicate communication dedupe, and top-K
# users by count. An additional API variant exposes
# register and mostActiveUser(), then asks for constant-
# time most-active lookup and a changing-K ranking
# extension.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Self communication (A-A) should now count.
# 2. Top-K with a changing K, or a fixed-size priority
# queue.
# 3. The space claim.
#
# INTERVIEW NOTES
# Normalize the pair first (sorted tuple) so A-B and B-A
# are one edge, then keep an edge set for dedup and a
# count per user. The ranking part is a separate question:
# top-K comes from a heap, and the single most active user
# can be maintained in O(1) because counts only ever go
# up. Mention that the edge set uses quadratic space in
# the worst case.
#
# BRUTE FORCE (say it first): Recompute counts from all
# edges on every query
# Keep every edge and, for each query, rebuild the
# distinct-partner counts and sort the users. Fine as an
# oracle; the work is repeated on each call. Time O(E + U
# log U) per query. Space O(E).


import heapq
from collections import Counter, defaultdict
```

```
# BRUTE FORCE (say it first): Recompute counts from all
# edges on every query | time O(E + U log U) per query
def _norm(pair):
    a, b = pair.split("-")
    return (a, b) if a <= b else (b, a)
```

```
class ActiveBrute:
    def __init__(self, count_self=False):
        self.edges, self.count_self = set(), count_self

    def add(self, pair):
        a, b = _norm(pair)
        if a == b and not self.count_self:
            return
        self.edges.add((a, b))

    def counts(self):
        c = Counter()
        for a, b in self.edges:
            c[a] += 1
            if a != b:
                c[b] += 1
        return c

    def top_k(self, k):
        c = self.counts()
        return sorted(c, key=lambda u: (-c[u], u))[:k]

    def most_active(self):
        c = self.counts()
        if not c:
            return set()
        m = max(c.values())
        return {u for u, v in c.items() if v == m}


# BASE QUESTION (optimal): Edge set, counts per user,
# count buckets for the maximum
# Idea: A new edge increments the count of both users.
# Buckets map each count to the set of users with it, so a
# user moves from bucket c to bucket c + 1.
# FOLLOW-UP 1: Self communication (A-A) should now count.
# Answer: Pass a flag to the class: a self pair is one
# edge that adds one to that user's count (and only once).
# The dedup set already handles repeated self pairs.
class Active:
    def __init__(self, count_self=False):
        # distinct edges seen so far (for the dedupe rule)
        self.edges = set()
        # user -> how many different partners
```

```
        self.count = Counter()
        # count -> users having exactly that count
        self.bucket = defaultdict(set)
        # the highest count right now (it never drops)
        self.top = 0
        self.count_self = count_self

    def bump(self, user):
        c = self.count[user]
        # move the user from bucket c to bucket c + 1
        if c:
            self.bucket[c].discard(user)
        self.count[user] = c + 1
        self.bucket[c + 1].add(user)
        self.top = max(self.top, c + 1)

    def add(self, a, b):
        # A-B and B-A are the same edge
        pair = (min(a, b), max(a, b))
        # follow-ups: skip self pairs (unless counted)
        # and skip a repeated pair
```

```
        skip_self = a == b and not self.count_self
        if skip_self or pair in self.edges:
            return
        self.edges.add(pair)
        self.bump(a)
        # a self pair counts once, not twice
        if a != b:
            self.bump(b)

    def most_active(self):
        # O(1): read the top bucket
        if not self.top:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        # k users by count, ties broken by name
        def rank(u):
            return (-self.count[u], u)
        return heapq.nsmallest(k, self.count, key=rank)


if __name__ == "__main__":
    a = Active()
```

```
    for x, y in [("A", "B"), ("B", "C"), ("B", "A")]:
        a.add(x, y)
    # B talks to A and C; the repeat B-A is ignored
    assert a.most_active() == {"B"}
    assert a.count["B"] == 2 and a.count["A"] == 1
    assert a.top_k(2) == ["B", "A"]
    # self pairs are ignored by default
    a.add("C", "C")
    assert a.count["C"] == 1
    # ... but can be switched on, counting once
    s = Active(count_self=True)
    s.add("C", "C")
    s.add("C", "C")
    assert s.count["C"] == 1
    assert Active().most_active() == set()
```

NOTES FROM CANDIDATES

A Java solution used an ordered map / tree structure to keep counts sortable.
The interviewer asked many clarification questions and expected dry runs.
The first version did not require self-communication handling; it was added as a coded follow-up.
Account for the per-user contact sets when explaining space use: the distinct relationship state can take quadratic space in the number of users. The interviewer explicitly challenged an initial linear-space claim.
A fixed-size priority-queue proposal did not finish the changing-K discussion; the interviewer asked for further optimization.
Maintaining the API variant
With registrations only and any tied maximum acceptable, normalize undirected pairs and store the distinct edges. For a new edge, increment each distinct endpoint once; an accepted self-edge changes one count, while an ignored self-edge changes none. Clarify that choice and the empty-state response.
Cache a user with the largest count. Counts never decrease, so only the endpoints of a new edge can displace the cached winner; duplicate edges do not change it. Reading this cache makes mostActiveUser() O(1), with expected O(1) registration work using

hash-based edge membership and counters under unit-cost user identifiers. A stricter tie rule requires separate handling.
For changing K, one option is a balanced ordered index of all users keyed by count and an agreed tie-breaker. Remove and reinsert the affected users when counts change. Updates cost O(log U); starting at the highest-ranked user and walking entries in descending-count order costs O(log U + K) for at most U returned users. This supports different K values without rebuilding a fixed-capacity heap and is a preparation option, not a specified interviewer solution.
Counters, the maximum cache, and the ranking index use O(U) space. Distinct relationship storage adds O(E), giving O(U + E) overall and O(U^2) in a dense graph.

PREPARATION

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

MEDIUM | PracHub | Technical Screen | 3 of 33

Sliding-Window Mean After Removing the Largest K Values

POSTED 2026-09-01 | 49 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 126/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #480 Sliding Window Median

LeetCode match: LC 480 Sliding Window Median (order statistics in a window)

Posted on PracHub: 2026-09-01 (updated 2026-09-23)

Original: https://prachub.com/coding-questions/sliding-window-mean-after-removing-the-largest-k-values

QUESTION

For every contiguous window of a fixed size, remove the largest **k** values by multiplicity and compute the mean of the remaining values. Process sliding updates incrementally rather than sorting each complete window again. Implement **`trimmed_window_means(nums: int[], window: int, k: int) -> string[]`**. Return means in window order as reduced fractions **`numerator/denominator`**, using a positive denominator. This exact fraction encoding is an explicit practice representation that avoids cross-language floating-point rounding differences.

Constraints & Assumptions

- **`1 <= len(nums) <= 200000`**, **`1 <= window <= len(nums)`**, and **`0 <= k < window`**.
- Values range from -1,000,000 through 1,000,000. Use a sufficiently wide integer for window sums.
- Remove exactly **k** occurrences, not **k** distinct values. Equal values on the partition boundary may be represented in either group as long as the multiset counts are correct.
- For **`k == 0`**, retain the whole window. A window that would retain zero values is excluded by the input contract.
- Reduce each fraction by the greatest common divisor of the absolute numerator and denominator. Zero is **`0/1`**; an integer mean such as 5 is **`5/1`**.
- Aim for **`O(n log window)`** processing with **`O(window)`** maintained state, apart from the returned results.

Examples

```
nums = [1,9,2,8], window = 3, k = 1 result = ["3/2","5/1"]
```

```
nums = [-1,1,3], window = 2, k = 0 result = ["0/1","2/1"]
```

QUESTION

Explain how insertions, removals, duplicate values, and rebalancing preserve the retained sum. For the reported large-data/stream follow-up, describe which state must remain in memory and why a new window does not require a full sort.

```
Maintain a size and an ordering invariant Partition the window into the removed largest values and the retained values. Keep their sizes correct and ensure no retained value is larger than a removed value.
```

APPROACH

Keep the window in two structures (top-k multiset and the rest) with running sums, using two heaps with lazy deletion or a balanced BST; mean = rest_sum / (w - k). O(n log w).

HOW TO THINK ABOUT IT

The mean needs two sums: the whole window and the sum of its K largest values. Keep the window in a sorted container; the K largest are its last K elements. When one value enters or leaves, only one element crosses the boundary between 'kept' and 'removed', so the sum of the K largest can be updated in O(1) after an O(log) search.

Basis: LeetCode 480 (Sliding Window Median) style order statistics; the mean of the window after dropping exactly K occurrences of the largest values, returned as reduced fractions.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, w, k):
    means = []
    for i in range(len(nums) - w + 1):
        window = sorted(nums[i:i + w])
        kept = window[:w - k]  # drop k largest
        total = sum(kept)
        means.append(Fraction(total, len(kept)))
    return means
```

TIME	SPACE
O(n * w log w)	O(w)

WHY IT IS SLOW

Each window is sorted from scratch, O(n * w log w). A sorted list updated by one insertion and one removal, with the sum of the top k adjusted at the boundary, avoids it.

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

TIME	SPACE
O(n * (log w + w) in memmove, O(n log w) with a balanced tree)	O(w)

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the update O(log w) in the worst case.

A Python list insert shifts elements (fast in practice, O(w) in theory). A balanced tree or a Fenwick tree over compressed values gives O(log w) insert, delete and 'sum of the top K' via a descent.

FOLLOW-UP 2 Remove the smallest K instead, or both ends (trimmed mean).

Maintain the sum of the K smallest as well (the first K elements of the same sorted list); the mean of the middle values is (total - top - bottom) / (w - 2K).

FOLLOW-UP 3 Duplicates on the boundary.

Removing exactly K occurrences, not K distinct values, is what the positional approach does: elements equal to the boundary value are interchangeable, so which copy sits in the top K does not matter for the sum. **Cost:** unchanged: O(w) per update for list insertion (O(log w) with a balanced tree).

FOLLOW-UP 4 Trim the smallest K as well (a trimmed mean).

Maintain the sum of the K smallest alongside the K largest with the mirror-image boundary updates; the mean of the middle values is (total - top - bottom) / (w - 2K). Checked against sorting every window.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Sliding-Window Mean After Removing the
# Largest K Values
#
# For every contiguous window of a fixed size, remove the
# largest k values by multiplicity and compute the mean of
# the remaining values. Process sliding updates
# incrementally rather than sorting each complete window
# again. Implement trimmed_window_means(nums: int[],
# window: int, k: int) -> string[]. Return means in window
# order as reduced fractions numerator/denominator, using
# a positive denominator. This exact fraction encoding is
# an explicit practice representation that avoids cross-
# language floating-point rounding differences. ###
# Constraints & Assumptions - 1 <= len(nums) <= 200000, 1
# <= window <= len(nums), and 0 <= k < window. - Values
# range from -1,000,000 through 1,000,000. Use a
# sufficiently wide integer for window sums. - Remove
# exactly k occurrences, not k distinct values.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make the update O(log w) in the worst case.
# 2. Remove the smallest K instead, or both ends (trimmed
# mean).
# 3. Duplicates on the boundary.
# 4. Trim the smallest K as well (a trimmed mean).
#
# INTERVIEW NOTES
# The mean needs two sums: the whole window and the sum of
# its K largest values. Keep the window in a sorted
# container; the K largest are its last K elements. When
# one value enters or leaves, only one element crosses the
# boundary between 'kept' and 'removed', so the sum of the
# K largest can be updated in O(1) after an O(log) search.
#
# BRUTE FORCE (say it first): Sort every window and sum
# the kept values
# For each window sort a copy, drop the largest k values,
# and compute the exact fraction. Correct and the oracle;
# every window is sorted from scratch. Time O(n * w log
# w). Space O(w).


from bisect import bisect_left, bisect_right
from fractions import Fraction


# HELPERS for the brute force
def as_text(fr):
    return f"{fr.numerator}/{fr.denominator}"


def brute(nums, window, k, both=False):
    # sort every window: the check used by the demo
    out = []
    for i in range(len(nums) - window + 1):
        part = sorted(nums[i:i + window])
        part = part[k:-k] if both and k else part
        part = part[:len(part) - k] if not both else part
        mean = Fraction(sum(part), len(part))
        out.append(as_text(mean))
    return out


# HELPERS for the base question
# FOLLOW-UP 4: Trim the smallest K as well (a trimmed
# mean).
# Answer: Maintain the sum of the K smallest alongside the
# K largest with the mirror-image boundary updates; the
# mean of the middle values is (total - top - bottom) / (w
# - 2K). Checked against sorting every window.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class Window:
    """A sorted window that keeps the sum of its k largest
    and k smallest values while values come and go."""

    def __init__(self, k):
        self.k, self.s = k, []
        self.total = self.top = self.bottom = 0

    def insert(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_right(s, x)
        s.insert(p, x)
        self.total += x
        # top-k region is the last k places (new list)
        if old_len < k:
            self.top += x
        elif p >= old_len + 1 - k:
            # x entered it: the value at old_len - k left
            # it
            self.top += x - s[old_len - k]
```

```
        # bottom-k region is the first k places
        if old_len < k:
            self.bottom += x
        elif p < k:
            self.bottom += x - s[k]

    def remove(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_left(s, x)
        self.total -= x
        if old_len <= k:
            self.top -= x
            self.bottom -= x
        else:
            if p >= old_len - k:
                # the value just below the region joins it
                self.top += s[old_len - k - 1] - x
            if p < k:
                self.bottom += s[k] - x
        del s[p]


# BASE QUESTION (optimal): Sorted window with an
# incrementally maintained top-K sum
# Idea: Keep the window values in a sorted list, the total
# window sum, and top = the sum of its K largest elements.
def trimmed_window_means(nums, window, k, both=False):
    # mean of each window after dropping the k largest
    # (and, with both=True, the k smallest as well)
    w, out = Window(k), []
    for i, x in enumerate(nums):
        w.insert(x)
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - (2 * k if both else k)
            drop = w.top + (w.bottom if both else 0)
            mean = Fraction(w.total - drop, kept)
            out.append(as_text(mean))
    return out
```

```
if __name__ == "__main__":
    nums = [5, 1, 4, 2, 8, 3, 3, 9]
    fast = trimmed_window_means
    assert fast(nums, 3, 1) == brute(nums, 3, 1)
    assert fast(nums, 4, 2) == brute(nums, 4, 2)
    # a reduced fraction: (1 + 4) / 2 = 5/2
    assert fast([5, 1, 4], 3, 1) == ["5/2"]
    # duplicates at the boundary: remove exactly k copies
    dup = [3, 3, 3, 3]
    assert fast(dup, 3, 1) == ["3/1", "3/1"]
    # trimmed mean: drop k smallest and k largest
    both = fast(nums, 5, 1, True)
    assert both == brute(nums, 5, 1, True)
```

FUNCTION AND TEST CASES

Signature

```
def trimmed_window_means(nums: list[int], window: int, k:
int) -> list[str]:
```

Sample call

```
trimmed_window_means([1, 9, 2, 8], 3, 1)
```

Test cases

Input	Expected
`([1, 9, 2, 8], 3, 1)`	`['3/2', '5/1']`
`([-1, 1, 3], 2, 0)`	`['0/1', '2/1']`

ALSO REPORTED · 2 more

Sliding Window Average After Dropping the Largest Values | PracHub | Technical Screen

https://prachub.com/coding-questions/sliding-window-average-after-dropping-the-largest-values

Compute sliding-window average excluding top k | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-sliding-window-average-excluding-top-k

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

Site tags: heap, streaming, medium

MEDIUM | IN APP LC 295 | DESIGN-STYLE (skippable) | 1Point3Acres | tech-screen | 4 of 33

Streaming Median (LC 295) with Follow-ups

ASKED 2026-08-03 | 78 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 50.4** | **30 DAYS 0** | **PRIORITY 69/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: your app's interview approach (LeetCode #295)

LeetCode match: LC 295 Find Median from Data Stream

Frequency: Single report | Last asked: 2026-08-03 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-median-with-follow-ups

QUESTION

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.SWEheapstreamingmediumFrequencySingle reportLast asked2026-08-03Stagetech-screen

APPROACH

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether values are integers, the range of values, how often median queries happen compared with inserts, and what to return for an even count (average of two middles).

BRUTE FORCE TO SAY OUT LOUD

Keep a sorted list and insert with bisect; median is the middle. O(n) per insert due to shifting.

OPTIMAL IDEA

Two heaps: a max-heap for the lower half and a min-heap for the upper half, sizes within one. Insert by pushing through one heap into the other and rebalancing. Median is the top of the larger heap or the average of both tops. O(log n) insert, O(1) median.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

One element, two elements, duplicates, all equal, alternating very large and very small values.

LIKELY FOLLOW-UPS

Values in a small range (counting array), arbitrary k-th smallest (Fenwick tree or order-statistic tree), or too much data (quantile sketch).

PITFALLS

Rebalancing after the wrong heap push. Integer division when the median should be fractional.

HOW TO THINK ABOUT IT

Keep the data split at the median: a max-heap for the smaller half and a min-heap for the larger half, with sizes equal or differing by one. Then the median is read from the tops. Insert by pushing through one heap into the other to keep every smaller value on the left, and rebalance the sizes.

Basis: LeetCode 295, Find Median from Data Stream.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
nums = []  # always kept sorted
def add(x):
    insort(nums, x)
def median():
    n = len(nums)
    if n % 2:
        return nums[n // 2]
    return (nums[n // 2 - 1] + nums[n // 2]) / 2
```

TIME	SPACE
O(n)	**O(n)**
per insert (list shift), O(1) median	

WHY IT IS SLOW

Each insert shifts up to n elements, O(n) per add (fast in practice, but linear). Two heaps keep the median available with O(log n) per add and O(1) per query.

OPTIMAL SOLUTION · Two heaps

TIME	SPACE
O(log n)	**O(n)**
add, O(1) median	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

FOLLOW-UP 2 Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

FOLLOW-UP 3 What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

FOLLOW-UP 4 The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Median (LC 295) with Follow-ups
#
# A near-canonical LC 295 streaming-median implementation
# using two heaps, followed by two design extensions:
# exploit a very small value domain, then support an
# arbitrary k-th-smallest query instead of only the
# median.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Almost every value is in a small range (say 0 to
# 100). Can you do better than two heaps?
# 2. Now support an arbitrary k-th smallest, not only the
# median.
# 3. What if the data stream is too large to store?
# 4. The median of only the last K values.
#
# INTERVIEW NOTES
# Keep the data split at the median: a max-heap for the
# smaller half and a min-heap for the larger half, with
# sizes equal or differing by one. Then the median is read
# from the tops. Insert by pushing through one heap into
# the other to keep every smaller value on the left, and
# rebalance the sizes.
#
# BRUTE FORCE (say it first): Keep a sorted list and
# insert with bisect
# Insert each value into a sorted list with bisect.insort
# and read the middle. Very simple and the oracle; every
# insert shifts up to n elements, though in practice that
# is a fast memmove. Time O(n) per insert (list shift),
# O(1) median. Space O(n).


import bisect
import heapq
from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Keep a sorted list and
# insert with bisect | time O(n) per insert (list shift),
# O(1) median
class MedianSorted:
    def __init__(self):
        self.a = []
```

```
    def addNum(self, x):
        bisect.insort(self.a, x)

    def findMedian(self):
        n = len(self.a)
        return (
            float(self.a[n // 2])
            if n % 2
            else (self.a[n // 2 - 1] + self.a[n // 2])
            / 2
        )


# BASE QUESTION (optimal): Two heaps
# Idea: Lower half in a max-heap, upper half in a min-
# heap, sizes differing by at most one and every lower
# value <= every upper value. The median is the larger
# heap's root, or the average of both roots.
class MedianFinder:
    def __init__(self):
        # low: max-heap (stored negated); high: min-heap
        self.low, self.high = [], []

    def add(self, x):
        # push through low so the halves stay ordered
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        # keep low the same size as high, or one bigger
        if len(self.high) > len(self.low):
            top = heapq.heappop(self.high)
            heapq.heappush(self.low, -top)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# FOLLOW-UP 1: Almost every value is in a small range (say
# 0 to 100). Can you do better than two heaps?
# Answer: Yes: a counting array gives O(1) inserts, and
# the median needs one pass over at most V counts, so O(V)
# per query with V fixed. Keep a moving pointer to the
# median bucket to make queries amortized O(1).
```

```
class CountingMedian:
    """Follow-up: values are whole numbers in 0..maxv."""

    def __init__(self, maxv):
        self.counts = [0] * (maxv + 1)
        self.n = 0

    def add(self, x):
        self.counts[x] += 1
```

```
        self.n += 1

    def kth(self, k):
        # k-th smallest (1-based): one pass over the
        # counts
        seen = 0
        for value, c in enumerate(self.counts):
            seen += c
            if seen >= k:
                return value

    def median(self):
        lo = self.kth((self.n + 1) // 2)
        hi = self.kth(self.n // 2 + 1)
        return (lo + hi) / 2


# FOLLOW-UP 2: Now support an arbitrary k-th smallest, not
# only the median.
# Answer: Two heaps only work for one fixed rank. Use an
# order-statistics structure: a Fenwick tree over
# (compressed) values, or a balanced BST augmented with
# subtree sizes.
class Fenwick:
    """Follow-up: any k-th smallest in O(log V)."""

    def __init__(self, size):
        self.size, self.tree = size, [0] * (size + 1)

    def add(self, value):
        i = value + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def kth(self, k):
```

```
        # walk down the tree to the k-th smallest value
        pos, step = 0, 1 << self.size.bit_length()
        while step:
            nxt = pos + step
            if nxt <= self.size and self.tree[nxt] < k:
                pos = nxt
                k -= self.tree[nxt]
            step >>= 1
        return pos


# FOLLOW-UP 4: The median of only the last K values.
# Answer: Keep the window in a deque (to know what
# expires) and a sorted list (to read the middle); insert
# and remove with bisect. O(K) per update because of list
# shifting; two heaps with lazy deletion give O(log K).
class WindowMedian:
    """Follow-up: median of only the last k values."""

    def __init__(self, k):
        self.k, self.order, self.sorted = k, [], []

    def add(self, x):
        self.order.append(x)
        insort(self.sorted, x)
        if len(self.order) > self.k:
            old = self.order.pop(0)
            del self.sorted[bisect_left(self.sorted, old)]

    def median(self):
        s, n = self.sorted, len(self.sorted)
        return (s[(n - 1) // 2] + s[n // 2]) / 2


if __name__ == "__main__":
    m = MedianFinder()
```

```
    for x in [5, 2, 8]:
        m.add(x)
    assert m.median() == 5
    m.add(1)
    assert m.median() == 3.5
    c = CountingMedian(100)
    for x in [5, 2, 8, 1]:
        c.add(x)
    assert c.median() == 3.5
```

```
    f = Fenwick(100)
    for x in [5, 2, 8, 1]:
        f.add(x)
    ranks = [f.kth(k) for k in (1, 2, 3, 4)]
    assert ranks == [1, 2, 5, 8]
    w = WindowMedian(3)
    for x in [9, 1, 5, 7]:
        w.add(x)
    # window is [1, 5, 7]
    assert w.median() == 5
```

NOTES FROM CANDIDATES

Keep the lower half in a max-heap and the upper half in a min-heap. Maintain a size difference of at most one and keep every lower-half value no greater than every upper-half value.
For an even-sized stream, average the two heap roots.
Each insertion costs O(log n), a median query costs O(1), and the heaps use O(n) space.
For a small bounded domain, a frequency array gives O(1) updates and an O(U) order-statistic scan, where U is the domain size. For arbitrary k-th-smallest queries, use an order-statistic tree or a Fenwick/segment tree over a known or compressed domain for logarithmic update and selection.
The base problem required complete working code plus a dry run and complexity analysis.
The first follow-up stopped at design and pseudocode; the second was discussion-only because the round ran out of time.

PREPARATION

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

Site tags: dedup, streaming, heap, sorting, medium

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 5 of 33

Batch and Streaming Log Deduplication

ASKED 2026-07-23 | 89 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 226/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-07-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/batch-streaming-log-deduplication

QUESTION

Process **`(timestamp, message)`** logs by deduplicating on message and sorting retained entries by timestamp, first keeping the first-seen entry and then the latest timestamp. The streaming extension adds **`add(timestamp, message)`** and **`get_next()`**, requiring a map plus min-heap, lazy deletion, and a discussion of watermarks and bounded state.SWEdedupstreamingheapsortingmediumFrequencySingle reportLast asked2026-07-23Stageonsite-coding

APPROACH

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which entry to keep in each policy (first seen or latest timestamp), how ties on timestamp break, whether output is sorted by timestamp, and how the streaming interface returns results (get_next).

BRUTE FORCE TO SAY OUT LOUD

Sort all logs and dedupe with a set. Correct for the batch version.

OPTIMAL IDEA

Batch: keep a dict message to chosen entry (keep first-seen, or replace if the new timestamp is later) and sort the values by (timestamp, message). Streaming: a dict for membership plus a min-heap of (timestamp, message); on update of a message push a new entry and mark the old one stale (lazy deletion), and get_next pops until it finds a non-stale one. Amortized O(log n).

DRY-RUN THESE CASES

The same message twice at different times, equal timestamps, an empty stream, get_next on an empty queue.

LIKELY FOLLOW-UPS

Watermarks for late data, bounded memory with eviction by age.

PITFALLS

Returning a stale heap entry. Forgetting deterministic tie-breaking.

HOW TO THINK ABOUT IT

Dedup is a dictionary keyed by the message. Sorting is only for the output order. For a stream, a heap gives the smallest timestamp quickly, but when an entry is replaced you cannot delete from the middle of a heap, so leave the stale entry and skip it when it surfaces (lazy deletion).

Basis: Written. Assumed rules (confirm them): dedupe on the message; policy 'first' keeps the earliest in input order, policy 'latest' keeps the largest timestamp; output sorted by (timestamp, message); get_next returns the retained entry with the smallest timestamp and removes it.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(logs, keep_first=True):
    kept = []
    for entry in logs:
        same = [x for x in logs
                if x[1] == entry[1]]
        pick = max(same)
        if keep_first:
            pick = same[0]
        if entry == pick:
            kept.append(entry)
    return sorted(kept)  # by timestamp
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each line scans the others: O(n^2). A dictionary from message to the kept entry is one pass, and a heap with lazy deletion serves the streaming version.

OPTIMAL SOLUTION · Dictionary on the message, then sort

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Streaming: map + min-heap with lazy deletion

TIME	SPACE
O(log n) amortized per add and get_next	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Late data: how do you decide when it is safe to emit?

Use a watermark: only release entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness). get_next then refuses to return anything newer than the watermark, and anything arriving older than it is dropped or sent to a late-data side output.

FOLLOW-UP 2 Memory must stay bounded.

Evict messages older than a retention window (a second heap or the same heap keyed on timestamp) and accept that duplicates arriving after eviction are no longer detected. **Cost:** O(log n) per eviction with a heap; memory O(entries inside the retention window).

FOLLOW-UP 3 Keep the entry with the most occurrences instead.

Store (count, timestamp) per message and update the count on every duplicate; the heap key changes only if you rank by count, so push a new entry when the rank changes and rely on lazy deletion again.

FOLLOW-UP 4 Only release entries once they are safe (late data may still arrive).

Add a watermark: release only entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness), in order; anything older than the watermark that arrives later goes to a late-data path. The lazy-deletion heap pops stale entries as it goes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Batch and Streaming Log Deduplication
#
# Process (timestamp, message) logs by deduplicating on
# message and sorting retained entries by timestamp, first
# keeping the first-seen entry and then the latest
# timestamp. The streaming extension adds add(timestamp,
# message) and get_next(), requiring a map plus min-heap,
# lazy deletion, and a discussion of watermarks and
# bounded state.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Late data: how do you decide when it is safe to emit?
# 2. Memory must stay bounded.
# 3. Keep the entry with the most occurrences instead.
# 4. Only release entries once they are safe (late data
# may still arrive).
#
# INTERVIEW NOTES
# Dedup is a dictionary keyed by the message. Sorting is
# only for the output order. For a stream, a heap gives
# the smallest timestamp quickly, but when an entry is
# replaced you cannot delete from the middle of a heap, so
# leave the stale entry and skip it when it surfaces (lazy
# deletion).
#
# BRUTE FORCE (say it first): Quadratic scan for earlier
# or later duplicates
# For each log line, scan the rest (or the earlier lines)
# to see if it is the one to keep. Clear logic, quadratic
# cost. Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Quadratic scan for earlier
# or later duplicates | time O(n^2)
def dedupe_brute(logs, policy="first"):
    keep = []
    for i, (ts, msg) in enumerate(logs):
        if policy == "first":
            ok = all(
                logs[j][1] != msg for j in range(i)
            )
        else:
            ok = all(
                logs[j][1] != msg
                or logs[j][0] < ts
                or (logs[j][0] == ts and j < i)
                for j in range(len(logs))
                if j != i
            )
        if ok:
            keep.append((ts, msg))
    return sorted(keep, key=lambda e: (e[0], e[1]))


# BASE QUESTION (optimal): Dictionary on the message, then
# sort
# Idea: One pass with a dict from message to the retained
# timestamp (first seen: keep the first; latest: replace
# when the new timestamp is larger), then sort the
# retained entries for the output.
def dedupe(logs, policy="first"):
    # logs: (timestamp, message) pairs; keep one per
```

```
    # message
    kept = {}
    for ts, msg in logs:
        # "first" keeps the first seen, "latest" the
        # newest
        if msg not in kept or (
                policy == "latest" and ts > kept[msg]):
            kept[msg] = ts
    # retained entries sorted by time, then message
    return sorted((ts, msg) for msg, ts in kept.items())


# FOLLOW-UP 4: Only release entries once they are safe
# (late data may still arrive).
# Answer: Add a watermark: release only entries whose
# timestamp is at most (the largest timestamp seen minus
# the allowed lateness), in order; anything older than the
# watermark that arrives later goes to a late-data path.
class StreamDedup:
    """add(ts, msg) and get_next() with a lazy heap."""

    def __init__(self, policy="first", lateness=0):
        self.policy, self.lateness = policy, lateness
        # message -> the timestamp currently kept
        self.kept = {}
```

```
        # (timestamp, message), may hold outdated entries
        self.heap = []
        self.max_ts = None

    def watermark(self):
        # everything at or below this is safe to release
        if self.max_ts is None:
            return None
        return self.max_ts - self.lateness

    def add(self, ts, msg):
        wm = self.watermark()
        # too late: older than the watermark, drop it
        if wm is not None and ts < wm:
            return False
        if self.max_ts is None or ts > self.max_ts:
            self.max_ts = ts
        old = self.kept.get(msg)
        if old is None or (
                self.policy == "latest" and ts > old):
            self.kept[msg] = ts
            heapq.heappush(self.heap, (ts, msg))
        return True

    def get_next(self):
        wm = self.watermark()
        while self.heap:
            ts, msg = self.heap[0]
            # lazy deletion: skip entries that were
            # replaced
            if self.kept.get(msg) != ts:
                heapq.heappop(self.heap)
                continue
            # not safe yet: later data may still arrive
            if ts > wm:
                return None
            heapq.heappop(self.heap)
            del self.kept[msg]
            return ts, msg
        return None


if __name__ == "__main__":
    logs = [(5, "b"), (1, "a"), (9, "a"), (3, "b")]
    assert dedupe(logs, "first") == [(1, "a"), (5, "b")]
    assert dedupe(logs, "latest") == [(5, "b"), (9, "a")]
    s = StreamDedup("first", lateness=2)
    for ts, msg in [(1, "a"), (3, "c"), (5, "b")]:
        assert s.add(ts, msg)
    # watermark is 5 - 2 = 3: release 1, then 3
    assert s.get_next() == (1, "a")
    assert s.get_next() == (3, "c")
    # 5 is still above the watermark
    assert s.get_next() is None
    # data older than the watermark is dropped
    assert not s.add(2, "late")
    # a "latest" policy replaces the kept entry
    t = StreamDedup("latest")
    t.add(1, "x")
    t.add(4, "x")
    assert t.get_next() == (4, "x")
    assert t.get_next() is None
```

NOTES FROM CANDIDATES

Empty input, identical entries, equal timestamps, out-of-order arrivals, updates, and removals all need explicit semantics.
For keep-latest streaming behavior, stale heap entries must be detected against the current timestamp stored for that message.
Arbitrarily late arrivals make globally final timestamp ordering impossible without a watermark or maximum-lateness guarantee.

PREPARATION

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

Site tags: heap, hashmap, scaling, medium

MEDIUM | IN APP LC 347 | 1Point3Acres | phone-screen, onsite-coding | 6 of 33

★ TOP 75: rank 29 in the Medium and Hard priority list

★ Top-K Frequent in Large Logs

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 58.8** | **30 DAYS 41.1** | **PRIORITY 21/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: your app's interview approach (LeetCode #347)

LeetCode match: LC 347 Top K Frequent Elements

Frequency: Medium | Last asked: 2026-07-19 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/top-k-frequent-from-logs

QUESTION

EXAMPLES

```
Input:  chatLog = ["A says hi", "B says hi", "A says
    bye", "C says hi", "A says yo"], k=2
Counts: A=3, B=1, C=1
Output: [A, B]  // or [A, C] — ties allowed
```

APPROACH

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

HOW TO THINK ABOUT IT

Count the speakers first; a log line is just a way to read the user id. Then you only need the K largest counts: a heap of size K does that without sorting all users. For logs spread over many machines the local top K are not enough on their own, so say what extra information you send.

Basis: LeetCode 347, Top K Frequent Elements (here the key is the speaker in each chat line).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(chat_log, k):
    names = [l.split()[0] for l in chat_log]
    counts = Counter(names)
    def rank(name):
        return (-counts[name], name)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct speakers, O(d log d), when only K are wanted; a size-K heap is O(d log k), and a stream needs a counter sketch instead of exact counts.

OPTIMAL SOLUTION · Count, then keep a size-K heap

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you do it across many machines when the log does not fit on one?

Count per shard and send each shard's local top-K to a coordinator. That is not exact in general (a user can be 11th everywhere yet first overall), so for an exact answer use the threshold algorithm: gather the local top-K, compute a global lower bound, ask every shard for all users above bound / numShards, and merge. If an approximate answer is acceptable, merge count-min sketches (they add) and keep a heap of the K largest estimates.

FOLLOW-UP 2 What if the log arrives as an unbounded stream?

Use the Misra-Gries or Space-Saving summary with m counters. Any user with more than n / (m + 1) messages is guaranteed to be in the summary, and a second pass (or a count-min sketch for the estimate) can confirm the real counts. **Cost:** O(1) per message with O(m) memory for m counters (Misra-Gries), instead of O(d) memory for the exact counts.

FOLLOW-UP 3 Which approach would you choose and why?

A heap is the safe default: simple, O(n + d log k) and streaming-friendly. Quickselect wins when k is a large fraction of d and you need all K but not their order. Sorting is fine only when d is small. Say that out loud, with the complexities, because the interviewer is grading the trade-off discussion.

FOLLOW-UP 4 The log is a stream and memory is limited.

Misra-Gries keeps m counters; any user with more than n / (m + 1) messages is guaranteed to remain in the summary (checked above). A second pass, or a count-min sketch, then gives the real counts of the survivors.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Top-K Frequent in Large Logs
#
# Examples: Input: chatLog = ["A says hi", "B says hi", "A
# says bye", "C says hi", "A says yo"], k=2 Counts: A=3,
# B=1, C=1 Output: [A, B] // or [A, C] — ties allowed
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you do it across many machines when the log
# does not fit on one?
# 2. What if the log arrives as an unbounded stream?
# 3. Which approach would you choose and why?
# 4. The log is a stream and memory is limited.
#
# INTERVIEW NOTES
# Count the speakers first; a log line is just a way to
# read the user id. Then you only need the K largest
# counts: a heap of size K does that without sorting all
# users. For logs spread over many machines the local top
# K are not enough on their own, so say what extra
# information you send.
#
# BRUTE FORCE (say it first): Count everything, then sort
# all users
# Count the speakers and sort every distinct user by
# (count, name). Correct and the oracle; sorting all d
# users does more work than keeping the best K. Time O(n +
# d log d). Space O(d).


import heapq
from collections import Counter


# BRUTE FORCE (say it first): Count everything, then sort
# all users | time O(n + d log d)
def top_k_speakers_sort(chat_log, k):
    c = Counter(
        line.split(" ", 1)[0] for line in chat_log
    )
    return sorted(c, key=lambda u: (-c[u], u))[:k]


# HELPERS for the base question
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def rank_key(item):
    # more messages first; ties by name
    return (-item[1], item[0])


# BASE QUESTION (optimal): Count, then keep a size-K heap
# Idea: Parse the speaker from each line and count in a
# hash map. nsmallest keeps a heap of size k, so it never
# sorts all d distinct users. Ties break by user name so
# the output is deterministic.
def top_k_speakers(chat_log, k):
    counts = Counter(speaker(line) for line in chat_log)
    # a heap of size k: O(n + d log k)
    items = counts.items()
    best = heapq.nsmallest(k, items, key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 1: How do you do it across many machines when
# the log does not fit on one?
# Answer: Count per shard and send each shard's local
# top-K to a coordinator.
def merge_shards(shard_logs, k):
    # exact answer: add the full counts of every shard
    total = Counter()
    for log in shard_logs:
        total.update(speaker(line) for line in log)
    best = heapq.nsmallest(k, total.items(), key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 2: What if the log arrives as an unbounded
# stream?
# Answer: Use the Misra-Gries or Space-Saving summary with
# m counters. Any user with more than n / (m + 1) messages
# is guaranteed to be in the summary, and a second pass
# (or a count-min sketch for the estimate) can confirm the
# real counts.
# FOLLOW-UP 4: The log is a stream and memory is limited.
# Answer: Misra-Gries keeps m counters; any user with more
# than n / (m + 1) messages is guaranteed to remain in the
# summary (checked above). A second pass, or a count-min
# sketch, then gives the real counts of the survivors.
class MisraGries:
    """m counters: a user with over n / (m + 1) of the
    n messages always stays in the summary."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, user):
        if user in self.counts:
            self.counts[user] += 1
        elif len(self.counts) < self.m:
            self.counts[user] = 1
        else:
            # no room: every counter pays one
            for u in list(self.counts):
                self.counts[u] -= 1
                if self.counts[u] == 0:
                    del self.counts[u]

    def candidates(self):
        return set(self.counts)


if __name__ == "__main__":
    log = ["A says hi", "B says hi", "A says bye",
           "C says hi", "A says yo"]
    assert top_k_speakers(log, 2) == ["A", "B"]
    assert top_k_speakers(log, 5) == ["A", "B", "C"]
    assert top_k_speakers([], 3) == []
    # two shards that each rank someone else first
    s1 = ["A says x", "B says x"]
    s2 = ["B says y", "B says z"]
    assert merge_shards([s1, s2], 1) == ["B"]
    mg = MisraGries(2)
    for u in "AAABBC":
        mg.add(u)
    # A has 3 of 6 messages (> 6 / 3), so it must survive
```

```
assert "A" in mg.candidates()
```

NOTES FROM CANDIDATES

The interviewer typically wants a discussion of the three implementations + complexity trade-off, not just one working solution.
For the distributed follow-up, candidates that mention count-min sketch and topK per shard -> global merge got positive feedback.
The same family surfaces as "Top-K IPs", "Top-K chatters", and "Top-K words"; the technique is interchangeable.
A recent variant uses a gigabyte-scale IP list and explicitly requires exact time and space complexity analysis.

PREPARATION

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

MEDIUM | IN APP LC 621 | PracHub | Technical Screen | 7 of 33

Rearrange Tasks With Cooldown

POSTED 2026-06-03 | 139 DAYS AGO | LAST 6 MONTHS

STUDY: TALK ALL TIME 40.3 30 DAYS 0 PRIORITY 92/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #621 Task Scheduler

LeetCode match: LC 621 Task Scheduler (inferred from overview)

Posted on PracHub: 2026-06-03 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/rearrange-tasks-with-cooldown

QUESTION

Overview: This question evaluates task-scheduling and constraint-handling skills, testing algorithmic reasoning about arranging repeated jobs with cooldown constraints along with analysis of schedule optimality and algorithmic complexity.

APPROACH

Greedy with max-heap + cooldown queue, or the formula (maxFreq-1)*(n+1)+countOfMax vs len(tasks). Rearranged output: always take the available task with highest remaining count.

HOW TO THINK ABOUT IT

The most frequent task is the bottleneck. Lay it out with gaps of n between its copies; every other task can fill gaps. Total length is the larger of 'that skeleton' and 'just the number of tasks' (when there are so many tasks the gaps are all filled and no idle time is needed). The skeleton has (maxcount - 1) full blocks of size n + 1 plus one last block holding every task tied for the maximum.

Basis: LeetCode 621, Task Scheduler (minimum time), with LeetCode 358, Rearrange String k Distance Apart as the construction follow-up. Assumed rule: the same task needs n idle-or-other slots between runs.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(tasks, n):
    left = Counter(tasks)
    free_at = {}  # task -> time it is ready
    t = 0
    while any(left.values()):
        ready = [x for x in left if left[x]
                 and free_at.get(x, 0) <= t]
        if ready:
            task = max(ready, key=left.get)
            left[task] -= 1
            free_at[task] = t + n + 1
        t += 1  # idle if none ready
    return t
```

TIME	SPACE
O(T log k) where T is the answer	O(k)

WHY IT IS SLOW

It simulates every slot, including long idle stretches, O(T log k). The answer follows from the most frequent task: max(len(tasks), (maxCount - 1) * (n + 1) + number of tasks with that count).

OPTIMAL SOLUTION · Closed form from the most frequent task

TIME	SPACE
O(len(tasks))	O(1) for a fixed alphabet

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print an actual schedule, not just the time.

Use the heap-with-cooldown-queue construction: run the available task with the most copies left, park it for n slots, and emit an idle marker when nothing is available. Or rearrange a string so equal letters are at least k apart.

FOLLOW-UP 2 Different cooldowns per task.

The closed form no longer applies. Simulate with a heap, storing for each task the time it becomes available; pick the ready task with the most remaining copies, and jump the clock to the earliest availability when nothing is ready.

FOLLOW-UP 3 Tasks have priorities or deadlines.

Then it is a scheduling problem; say what you optimise (lateness, completion time) and use a heap ordered by deadline for the ready tasks.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Rearrange Tasks With Cooldown
#
# This question evaluates task-scheduling and constraint-
# handling skills, testing algorithmic reasoning about
# arranging repeated jobs with cooldown constraints along
# with analysis of schedule optimality and algorithmic
# complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print an actual schedule, not just the time.
# 2. Different cooldowns per task.
# 3. Tasks have priorities or deadlines.
#
# INTERVIEW NOTES
# The most frequent task is the bottleneck. Lay it out
# with gaps of n between its copies; every other task can
# fill gaps. Total length is the larger of 'that skeleton'
# and 'just the number of tasks' (when there are so many
# tasks the gaps are all filled and no idle time is
# needed). The skeleton has (maxcount - 1) full blocks of
# size n + 1 plus one last block holding every task tied
# for the maximum.
#
# BRUTE FORCE (say it first): Simulate each time slot,
# always running the available task with the most copies
# left
# At every time unit pick, among tasks not cooling down,
# the one with the most remaining copies; if none is
# available the slot is idle. A heap for ready tasks and a
# queue of tasks in cooldown make it work. Correct, but it
# simulates every slot, including long idle stretches.
# Time O(T log k) where T is the answer. Space O(k).


import heapq
from collections import Counter, deque


# BRUTE FORCE (say it first): Simulate each time slot,
# always running the available task with the most copies
# left | time O(T log k) where T is the answer
def brute(tasks, n):
    left = Counter(tasks)
    free_at = {}  # task -> time it is ready
    t = 0
```

```
    while any(left.values()):
        ready = [
            x
            for x in left
            if left[x] and free_at.get(x, 0) <= t
        ]
        if ready:
            task = max(ready, key=left.get)
            left[task] -= 1
            free_at[task] = t + n + 1
        t += 1  # idle if none ready
    return t


# BASE QUESTION (optimal): Closed form from the most
# frequent task
# Idea: Let m be the highest frequency and k the number of
# tasks that have that frequency. The skeleton needs (m -
# 1) * (n + 1) + k slots.
def least_interval(tasks, n):
```

```
    # closed form: the most frequent task sets the frame
    counts = Counter(tasks).values()
    most = max(counts, default=0)
    # how many tasks are tied for the most
    tied = sum(1 for c in counts if c == most)
    if not tasks:
        return 0
    # (most - 1) full frames of n + 1 slots, plus the last
    # row
    return max(len(tasks), (most - 1) * (n + 1) + tied)


# FOLLOW-UP 1: Print an actual schedule, not just the
# time.
# Answer: Use the heap-with-cooldown-queue construction:
# run the available task with the most copies left, park
# it for n slots, and emit an idle marker when nothing is
# available. Or rearrange a string so equal letters are at
# least k apart.
def schedule(tasks, n):
    # an actual schedule, with "idle" where nothing may
    # run
    heap = [(-c, t) for t, c in Counter(tasks).items()]
    heapq.heapify(heap)
    # tasks cooling down: (time ready, -left, task)
```

```
    cooling, out, now = deque(), [], 0
    while heap or cooling:
        # bring back tasks whose cooldown is over
        while cooling and cooling[0][0] <= now:
            _, left, task = cooling.popleft()
            heapq.heappush(heap, (left, task))
        if heap:
            # run the task with the most copies left
            left, task = heapq.heappop(heap)
            out.append(task)
            if left + 1 < 0:
                ready = now + n + 1
                cooling.append((ready, left + 1, task))
        else:
            out.append("idle")
        now += 1
    return out


# HELPERS for the checks below
def valid(plan, n):
    # equal tasks must be more than n slots apart
    last = {}
    for i, task in enumerate(plan):
        if task == "idle":
            continue
        if task in last and i - last[task] <= n:
            return False
        last[task] = i
    return True


if __name__ == "__main__":
    tasks = list("AAABBB")
    assert least_interval(tasks, 2) == 8
    assert least_interval(tasks, 0) == 6
    assert least_interval([], 3) == 0
    # many different tasks: no idle time needed
    assert least_interval(list("ABCDEFG"), 2) == 7
    plan = schedule(tasks, 2)
    assert len(plan) == 8 and valid(plan, 2)
    assert plan.count("idle") == 2
    for t, n in ((list("AAAABBC"), 3), (list("AB"), 5)):
        plan = schedule(t, n)
        assert valid(plan, n)
```

```
        assert len(plan) == least_interval(t, n)
```

FUNCTION AND TEST CASES

Signature

```
def solution(tasks: list[str], cooldown: int) -> str:
```

Sample call

```
solution(['A', 'A', 'A', 'B', 'B', 'C'], 2)
```

Test cases

Input	Expected
`(['A', 'A', 'A', 'B', 'B', 'C'], 2)`	`'ABCAB_A'`
`(['A', 'A', 'A', 'B'], 2)`	`'AB_A__A'`

ALSO REPORTED · 1 more

Design a task scheduler in Python. | Exponent
https://www.aced.io/questions/3504/design-task-scheduler-python

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

MEDIUM | PracHub | Technical Screen | 8 of 33

Determine if a 14-tile hand is winning

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 149/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #659 Split Array into Consecutive Subsequences

LeetCode match: No direct match; related LC 659 Split Array into Consecutive Subsequences

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/determine-if-a-14-tile-hand-is-winning

QUESTION

You are given an integer array **`tiles`** of length 14 representing a Mahjong-like hand. Each integer is a tile value from 1 to 9 (single suit). You may reorder the tiles. A hand is **winning** if it can be partitioned into:

- **1 pair**: two identical tiles, and - **4 melds** (each meld uses 3 tiles), where each meld is either: - a **triplet**: three identical tiles (e.g., 7,7,7), or - a **sequence**: three consecutive values (e.g., 2,3,4). Each tile can be used at most once. Return **`true`** if the hand is winning, otherwise return **`false`**. Assumptions/constraints:
- **`tiles.length == 14`**
- **`1 <= tiles[i] <= 9`**
- Each value can appear multiple times (as in a real tile set).

APPROACH

Count array of tiles 1..9; try each value as the pair, then backtrack: take the smallest remaining tile as a triplet or a run of three. Tiny search space.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether sequences are only consecutive in one suit (values 1 to 9), whether a tile can be used only once, and whether other special hands exist (assume not).

BRUTE FORCE TO SAY OUT LOUD

Try every permutation of the 14 tiles and test the grouping. Far too slow.

OPTIMAL IDEA

Keep a count array of tiles 1 to 9. Try each value that has at least two tiles as the pair, remove it, then backtrack on the remaining 12 tiles: take the smallest value that still has tiles and either remove a triplet of it or a run of three consecutive values starting at it; if neither is possible, fail. Always handling the smallest value makes the search small. Time is tiny for 14 tiles.

DRY-RUN THESE CASES

Four triplets and a pair, four runs and a pair, a hand with two possible groupings, a hand with a pair but impossible melds, all 14 tiles equal (invalid since each value has at most four copies).

LIKELY FOLLOW-UPS

More suits, seven pairs, or all possible winning tiles for a 13-tile hand.

PITFALLS

Greedy without backtracking: a run versus a triplet choice can matter. Forgetting to restore counts.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

Work with counts per value (at most nine numbers), not with the tile list. Try every value as the pair, then handle the lowest remaining value: it must belong to a triplet or to a sequence starting at it (nothing smaller is left to pair with it). Always processing the smallest value makes the search tiny and avoids trying the same grouping in different orders.

Basis: Written; a small exhaustive search like the Mahjong 'standard hand' check. Rule: 14 tiles of one suit with values 1..9 split into one pair and four melds, each meld a triplet (x, x, x) or a sequence (x, x + 1, x + 2).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(counts):
    for v in range(1, 10):
        if counts[v] >= 2:
            counts[v] -= 2  # the pair
            ok = remove_melds(counts)
            counts[v] += 2
            if ok:
                return True
    return False
def remove_melds(counts):
    v = first_value_with_tiles(counts)
    if v is None:
        return True
    for meld in (triple(v), run(v)):
        if can_take(counts, meld):
            take(counts, meld)
            ok = remove_melds(counts)
            give_back(counts, meld)
            if ok:
                return True
    return False
```

TIME	SPACE
exponential but small for...	**O(depth)**
exponential but small for 14 tiles	

BRUTE FORCE

WHY IT IS SLOW

It tries the same groupings in many different orders (still tiny for 14 tiles). Always grouping the smallest remaining tile tries each grouping once.

OPTIMAL SOLUTION · Counts, try each pair, then always group the smallest remaining tile

TIME	SPACE
O(9 * small branching) = constant for 14 tiles	**O(1)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Which tile would complete a 13-tile hand?

Try each of the nine values as the 14th tile and run the same check: 9 checks of a tiny search. That is how a player finds the tiles they are waiting for.

FOLLOW-UP 2 More suits or special hands (seven pairs).

Run the single-suit check on each suit separately (melds never cross suits) after splitting the hand; seven pairs is a separate one-line count check (seven values with count 2).

FOLLOW-UP 3 Why is the 'smallest remaining value' rule enough?

The smallest remaining value cannot be the middle or end of a sequence (the smaller values are gone), so it must be in a triplet or the start of a sequence. Trying those two options covers every legal grouping exactly once.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine if a 14-tile hand is winning
#
# You are given an integer array tiles of length 14
# representing a Mahjong-like hand. Each integer is a tile
# value from 1 to 9 (single suit). You may reorder the
# tiles. A hand is **winning** if it can be partitioned
# into: - **1 pair**: two identical tiles, and - **4
# melds** (each meld uses 3 tiles), where each meld is
# either: - a **triplet**: three identical tiles (e.g.,
# 7,7,7), or - a **sequence**: three consecutive values
# (e.g., 2,3,4). Each tile can be used at most once.
# Return true if the hand is winning, otherwise return
# false. Assumptions/constraints: - tiles.length == 14 - 1
# <= tiles[i] <= 9 - Each value can appear multiple times
# (as in a real tile set).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Which tile would complete a 13-tile hand?
# 2. More suits or special hands (seven pairs).
# 3. Why is the 'smallest remaining value' rule enough?
#
# INTERVIEW NOTES
# Work with counts per value (at most nine numbers), not
# with the tile list. Try every value as the pair, then
# handle the lowest remaining value: it must belong to a
# triplet or to a sequence starting at it (nothing smaller
# is left to pair with it). Always processing the smallest
# value makes the search tiny and avoids trying the same
# grouping in different orders.
#
# BRUTE FORCE (say it first): Remove any triplet or
# sequence from anywhere, exhaustively
# For each pair candidate, repeatedly try removing any
# triplet or any sequence available among the counts (in
# any order) and see whether everything can be removed.
# Correct and the oracle; it tries the same groupings in
# many orders, unlike the smallest-first search. Time
# exponential but small for 14 tiles. Space O(depth).


from collections import Counter


# BRUTE FORCE (say it first): Remove any triplet or
# sequence from anywhere, exhaustively | time exponential
# but small for 14 tiles
def winning_brute(tiles):
    cnt = [0] * 12
    for t in tiles:
        cnt[t] += 1

    def melds(c):
        if sum(c) == 0:
            return True
        for v in range(1, 10):
            if c[v] >= 3:
                c[v] -= 3
                ok = melds(c)
                c[v] += 3
                if ok:
                    return True
            if (
                v <= 7
                and c[v]
                and c[v + 1]
                and c[v + 2]
            ):
                c[v] -= 1
                c[v + 1] -= 1
                c[v + 2] -= 1
                ok = melds(c)
                c[v] += 1
                c[v + 1] += 1
                c[v + 2] += 1
                if ok:
                    return True
        return False

    for v in range(1, 10):
        if cnt[v] >= 2:
            cnt[v] -= 2
            ok = melds(cnt)
            cnt[v] += 2
            if ok:
                return True
    return False


# HELPERS for the base question
def melds_ok(cnt, v=1):
```

```
    # can the tiles in cnt (value -> count) form only
    # melds?
    while v <= 9 and cnt[v] == 0:
        v += 1
    if v > 9:
        return True
    # the smallest tile is either in a triplet ...
    if cnt[v] >= 3:
        cnt[v] -= 3
        ok = melds_ok(cnt, v)
        cnt[v] += 3
        if ok:
            return True
    # ... or the start of a run v, v + 1, v + 2
    if v <= 7 and cnt[v + 1] and cnt[v + 2]:
        for t in (v, v + 1, v + 2):
            cnt[t] -= 1
        ok = melds_ok(cnt, v)
        for t in (v, v + 1, v + 2):
            cnt[t] += 1
        return ok
    return False


# HELPERS for follow-up 2
def count_of(values):
    cnt = Counter({v: 0 for v in range(1, 12)})
    for v in values:
        cnt[v] += 1
    return cnt


def is_winning_hand_with_len(values):
    cnt = count_of(values)
    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False


# BASE QUESTION (optimal): Counts, try each pair, then
```

```
# always group the smallest remaining tile
# Idea: Keep counts of 1..9. For each value with at least
# two tiles, take it as the pair.
def is_winning_hand(tiles):
    # one pair plus four melds, single suit, values 1..9
    cnt = Counter({v: 0 for v in range(1, 12)})
    for t in tiles:
        cnt[t] += 1
    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False


# FOLLOW-UP 1: Which tile would complete a 13-tile hand?
# Answer: Try each of the nine values as the 14th tile and
# run the same check: 9 checks of a tiny search. That is
# how a player finds the tiles they are waiting for.
def waiting_tiles(tiles13):
    # follow-up: which 14th tile completes the hand?
    return [t for t in range(1, 10)
            if is_winning_hand(tiles13 + [t])]
```

```
# FOLLOW-UP 2: More suits or special hands (seven pairs).
# Answer: Run the single-suit check on each suit
# separately (melds never cross suits) after splitting the
# hand; seven pairs is a separate one-line count check
# (seven values with count 2).
def is_seven_pairs(tiles):
    # a special hand: seven different pairs
    counts = Counter(tiles)
    pairs = all(c == 2 for c in counts.values())
    return len(tiles) == 14 and pairs


# FOLLOW-UP 2: More suits or special hands (seven pairs).
# Answer: Run the single-suit check on each suit
# separately (melds never cross suits) after splitting the
# hand; seven pairs is a separate one-line count check
# (seven values with count 2).
```

```
def is_winning_multi(hand):
    # several suits: hand is a list of (suit, value);
    # melds
    # never cross suits, and exactly one suit holds the
    # pair
    suits = {}
    for suit, v in hand:
        suits.setdefault(suit, []).append(v)
    pair_suits = [s for s in suits
                  if len(suits[s]) % 3 == 2]
    if len(pair_suits) != 1:
        return False
    for s, vs in suits.items():
        if s == pair_suits[0]:
            if not is_winning_hand_with_len(vs):
                return False
        elif len(vs) % 3 or not melds_ok(count_of(vs)):
            return False
    return True


if __name__ == "__main__":
    win = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]
    assert is_winning_hand(win)
    # 3 and 6 are missing, so the runs cannot be formed
    lose = [1, 2, 4, 5, 7, 8, 1, 2, 4, 5, 7, 8, 9, 9]
    assert not is_winning_hand(lose)
    # thirteen tiles waiting for a 9 (or a 6)
    wait = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9]
    assert 9 in waiting_tiles(wait)
    assert is_seven_pairs([1, 1, 2, 2, 3, 3, 4, 4, 5, 5,
                           6, 6, 7, 7])
    hand = [("m", v) for v in (1, 1, 1, 2, 3)] + [
        ("p", v) for v in (4, 5, 6, 7, 7, 7)] + [
        ("s", v) for v in (2, 2, 2)]
    assert is_winning_multi(hand)
```

FUNCTION AND TEST CASES

Signature

```
def solution(tiles: list[int]) -> bool:
```

Sample call

```
solution([1,1,1,2,3,4,2,3,4,5,6,7,9,9])
```

Test cases

Input	Expected
([1,1,1,2,3,4,2,3,4,5,6,7,9,9],)	True
([2,2,2,3,3,3,4,4,4,5,5,5,9,9],)	True

ALSO REPORTED · 1 more

Mahjong Winning Hand Detection (14 Tiles) | 1Point3Acres | onsite-coding | asked 2026-07-20 | Low

https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:
Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.
For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

Preparation: Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

MEDIUM | PracHub | Technical Screen | 9 of 33

★ TOP 75: rank 28 in the Medium and Hard priority list

★ Implement frequency + distance top-K queries

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58.8 | 30 DAYS 41.1 | PRIORITY 20/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #347 Top K Frequent Elements, LeetCode #973 K Closest Points to Origin

LeetCode match: LC 347 Top K Frequent Elements + LC 973 K Closest Points to Origin

Posted on PracHub: 2026-02-12 (updated 2026-04-28)

Original: https://prachub.com/coding-questions/implement-frequency-plus-distance-top-k-queries

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break) You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **p** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **k distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **x** ascending, then **y** ascending

Input

- **`points`**: list of length **n**, each element **`[x, y]`**
- **k**: integer, **`1 <= k <= (# of distinct points)`**

Output

- A list of **k** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup Design a data structure supporting the following operations:

QUESTION

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.
- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:
- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

Example

If for key **`"a"`** you stored timestamps **`[2, 10, 15]`** with values **`["x", "y", "z"]`**:

- **`getClosest("a", 11)`** returns **`"y"`** (10 is closest)
- **`getClosest("a", 14)`** returns **`"z"`** (15 is closest)
- **`getClosest("a", 12)`** returns **`"y"`** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **`2 * 10^5`** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.

Constraints

1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points

EXAMPLES

```
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1]
    appears 2 times, so they are the top 2 distinct
    points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]],
    3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each
    appear twice. Their squared distances are 4, 5,
    and 5, so [0,2] comes first. Between [1,2] and
    [2,1], x is smaller for [1,2].
```

QUESTION

Hints

First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like `(-frequency, distance, x, y)`.

Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either `['set', key, value, timestamp]` or `['getClosest', key, timestamp]`. For every `getClosest` operation, return the stored value whose timestamp is closest to the query tim

APPROACH

Count frequencies in a hash map; use a size-k heap with key (frequency desc, distance asc) for Q1, and a max-heap of size k on distance for Q2.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two questions. Q1 (top-K points): confirm the full ordering (higher frequency, then smaller distance, then smaller x, then smaller y) and that K is at most the number of distinct points. Q2 (time-based store): timestamps for a key arrive in non-decreasing order, and a tie in distance returns the smaller timestamp.

BRUTE FORCE TO SAY OUT LOUD

Q1: sort all distinct points with the full key. Q2: linear scan of a key's timestamps. Both are correct and easy to state.

OPTIMAL IDEA

Q1: Counter over tuples, then heapq.nsmallest(k, distinct, key=(-freq, x*x + y*y, x, y)), O(n + d log k). Q2: per key keep parallel lists of timestamps and values; bisect_left finds the first timestamp at least the query; compare it with the previous one and return the closer value, the smaller timestamp on a tie. O(log m) per query.

DRY-RUN THESE CASES

Q1: duplicates, ties on frequency and distance, K equal to the number of distinct points. Q2: query before the first timestamp, after the last, exactly on a timestamp, the tie example (a, 12 returns y), an unknown key.

LIKELY FOLLOW-UPS

Q1: top-K in a stream. Q2: an out-of-order insert (then use a sorted container), deletes, and memory limits.

PITFALLS

Q1: sorting by distance only. Q2: off-by-one at the ends of the list.

HOW TO THINK ABOUT IT

Q1 is a ranking: say the full sort key out loud, count with a hash map, and then you only need the K best (heap), not a full sort. Q2 keeps each key's timestamps in a sorted list, so the nearest one is next to the insertion point: binary search, then compare the two neighbours.

Basis: Two tasks: top K points by (frequency desc, squared distance asc, x asc, y asc), a mix of LeetCode 347 and 973; and a time-based store whose getClosest returns the value at the nearest timestamp (ties to the smaller one), a variant of LeetCode 981.

BRUTE FORCE · Q1: sort all distinct points by the full key

BRUTE FORCE · PSEUDO-CODE

```
def top_k_points(points, k):
    counts = Counter(map(tuple, points))
    def rank(p):
        x, y = p
        return (-counts[p], x * x + y * y, x, y)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct points, O(d log d); a size-K heap with the same key is O(d log k).

BRUTE FORCE · Q2: scan every timestamp of the key

BRUTE FORCE · PSEUDO-CODE

```
def get_closest(key, t):
    best = None
    for ts, value in store[key]:
        gap = abs(ts - t)
        if best is None or (gap, ts) < best[:2]:
            best = (gap, ts, value)
    return best[2]
```

TIME	SPACE
O(m) per query	O(total entries)

WHY IT IS SLOW

Linear in the number of stored timestamps per query. Timestamps arrive in order, so binary search on the sorted list finds the neighbours in O(log m).

OPTIMAL SOLUTION · Q1: heap with the full key

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: parallel sorted lists and bisect

TIME	SPACE
O(1) per set, O(log m) per getClosest	**O(total entries)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1 with points arriving over time.

Keep a running Counter and compute the top K on demand with a heap (O(d log k) per query), or maintain a sorted structure updated on every add if queries dominate.

FOLLOW-UP 2 Q2 with timestamps arriving out of order.

The append keeps the list sorted only for in-order input. For arbitrary order keep a sorted list and insert with bisect.insort (O(m) per insert) or a balanced tree (O(log m)); the query is unchanged.

FOLLOW-UP 3 Q2 returns the closest value at or before the timestamp (LeetCode 981).

Use bisect_right(timestamps, ts) - 1 and return that value or an empty string if the index is negative; there is no tie question.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement frequency + distance top-K
# queries
#
# You will implement solutions for two coding interview
# questions. ## Question 1: Return the top-K points by
# frequency (with distance tie-break) You are given an
# array points of 2D integer points, where each point is
# represented as [x, y]. The array may contain duplicates
# (the same [x, y] can appear multiple times). Define: -
# freq(p) = number of times point p appears in the array.
# - dist(p) = squared Euclidean distance to the origin,
# x^2 + y^2. Return **k distinct points** sorted by the
# following priority: 1. Higher frequency first (freq
# descending) 2. If frequencies tie, smaller distance
# first (dist ascending) 3. If still tied, sort by x
# ascending, then y ascending ### Input - points: list of
# length n, each element [x, y] - k: integer, 1 <= k <= (#
# of distinct points) ### Output - A list of k distinct
# points following the ranking above.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1 with points arriving over time.
# 2. Q2 with timestamps arriving out of order.
# 3. Q2 returns the closest value at or before the
# timestamp (LeetCode 981).
#
# INTERVIEW NOTES
# Q1 is a ranking: say the full sort key out loud, count
# with a hash map, and then you only need the K best
# (heap), not a full sort. Q2 keeps each key's timestamps
# in a sorted list, so the nearest one is next to the
# insertion point: binary search, then compare the two
# neighbours.
#
# BRUTE FORCE (say it first): Q1: sort all distinct points
# by the full key
# Count the points, sort the distinct ones by (-frequency,
# distance, x, y), take K. Correct; sorting all d distinct
# points does more work than a heap of size K. Time O(d
# log d). Space O(d).


import heapq
from bisect import bisect_left, bisect_right, insort
from collections import Counter, defaultdict
```

```
# BRUTE FORCE (say it first): Q1: sort all distinct points
# by the full key | time O(d log d)
def top_points_sort(points, k):
    c = Counter(map(tuple, points))
    return [
        list(p)
        for p in sorted(
            c,
            key=lambda p: (
                -c[p],
                p[0] ** 2 + p[1] ** 2,
                p[0],
                p[1],
            ),
        )[:k]
    ]


# BASE QUESTION (optimal): Q1: heap with the full key
# Idea: Count with a Counter, then heapq.nsmallest(k, ...)
# with the key (-frequency, squared distance, x, y).
# Squared distances avoid square roots and are exact for
# integers.
def top_points(points, k):
    # more frequent first, then closer to the origin, then
    # x, y
    counts = Counter(map(tuple, points))

    def rank(p):
        dist = p[0] ** 2 + p[1] ** 2
        return (-counts[p], dist, p[0], p[1])

    best = heapq.nsmallest(k, counts, key=rank)
    return [list(p) for p in best]


# PART 2: closest timestamp value
class TimeStore:
    """set(key, value, ts) and closest-time reads."""

    def __init__(self):
        # key -> sorted [(timestamp, value)]
        self.data = defaultdict(list)
```

```
    def set(self, key, value, ts):
        # insort also handles timestamps arriving out of
        # order
        insort(self.data[key], (ts, value))

    def get_closest(self, key, ts):
        items = self.data.get(key)
        if not items:
            return ""
```

```
        # the first stored time at or after ts, and the
        # one before
        i = bisect_left(items, (ts,))
        options = items[max(0, i - 1):i + 1]
        # closest wins; on a tie the smaller timestamp
        # wins
        def gap(item):
            return (abs(item[0] - ts), item[0])
        best = min(options, key=gap)
        return best[1]

    def get_at_or_before(self, key, ts):
        # LeetCode 981: the latest value not after ts
        items = self.data.get(key, [])
        i = bisect_right(items, (ts, chr(0x10FFFF))) - 1
        return items[i][1] if i >= 0 else ""


# FOLLOW-UP 1: Q1 with points arriving over time.
# Answer: Keep a running Counter and compute the top K on
# demand with a heap (O(d log k) per query), or maintain a
# sorted structure updated on every add if queries
# dominate.
class TopPointsStream:
    # follow-up: points arrive one at a time. The counts
    # are
    # kept as points arrive, so top_k never recounts
    # history.
    def __init__(self):
        self.counts = Counter()

    def add(self, x, y):
        self.counts[(x, y)] += 1

    def top_k(self, k):
```

```
        def rank(p):
            dist = p[0] ** 2 + p[1] ** 2
            return (-self.counts[p], dist, p[0], p[1])

        best = heapq.nsmallest(k, self.counts, key=rank)
        return [list(p) for p in best]


if __name__ == "__main__":
    pts = [[1, 1], [1, 1], [2, 0], [0, 2], [2, 0], [3, 3]]
    # (1,1) and (2,0) appear twice; (2,0) is farther? 4 vs
    # 2
    assert top_points(pts, 2) == [[1, 1], [2, 0]]
    four = top_points(pts, 4)
    assert four == [[1, 1], [2, 0], [0, 2], [3, 3]]
    assert top_points([[5, 5]], 1) == [[5, 5]]
    live = TopPointsStream()
    for x, y in pts:
        live.add(x, y)
    assert live.top_k(2) == [[1, 1], [2, 0]]
    live.add(3, 3)
    live.add(3, 3)
    live.add(3, 3)
    # (3, 3) is now the most frequent point
    assert live.top_k(1) == [[3, 3]]
    s = TimeStore()
    for ts, v in ((2, "x"), (10, "y"), (15, "z")):
        s.set("a", v, ts)
    assert s.get_closest("a", 11) == "y"
    assert s.get_closest("a", 14) == "z"
    # a tie: 10 and 15 are both 2.5 away from 12.5, 12 ->
    # 10
    assert s.get_closest("a", 12) == "y"
    assert s.get_closest("b", 1) == ""
    assert s.get_closest("a", 100) == "z"
    assert s.get_closest("a", -5) == "x"
    assert s.get_at_or_before("a", 14) == "y"
```

```
    assert s.get_at_or_before("a", 1) == ""
    s.set("a", "late", 5)
    assert s.get_closest("a", 6) == "late"
```

FUNCTION AND TEST CASES

Signature

```
def solution(points: list[list[int]], k: int) -> list[list
[int]]:
```

Sample call

```
solution([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
```

Test cases

Input	Expected
([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)	[[2, 2], [1, 1]]
([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]], 3)	[[0, 2], [1, 2], [2, 1]]

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

MEDIUM | HackTheRounds | Phone Screen | 10 of 33

Playlist Shuffle No Adjacent Artists

STUDY: TALK | ALL TIME 34.9 | 30 DAYS 0 | PRIORITY 110/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #767 Reorganize String

LeetCode match: LC 767 / LC 1054

QUESTION

Given a playlist of songs where each song has an artist, implement a shuffle function that minimizes (or eliminates) adjacent songs by the same artist.

EXAMPLES

Example 1	
Input	`[("A", "song1"), ("A", "song2"), ("B", "song3"), ("B", "song4"), ("C", "song5")]`
Output	`[("A", "song1"), ("B", "song3"), ("A", "song2"), ("B", "song4"), ("C", "song5")]`
Why	`No adjacent songs have same artist.`

CONSTRAINTS

- 1 <= songs.length <= 10^4

APPROACH

Max-heap by remaining count per artist

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the shuffle must be random, how to behave when a perfect arrangement is impossible (the most frequent artist has more than half the songs, rounded up), and whether the result must be uniform among valid shuffles.

BRUTE FORCE TO SAY OUT LOUD

Shuffle randomly, then check for adjacent artists and retry. May never finish.

OPTIMAL IDEA

Feasibility: let n be the song count; an arrangement exists if the largest artist group has at most ceil(n / 2) songs. Build it greedily with a max-heap of remaining songs per artist: repeatedly take from the artist with the most remaining songs who is not the previous artist, using random choice among songs of that artist (and optionally among equally big groups) to keep the result varied. If infeasible, place the leftovers at the end or spread them as evenly as possible, and say that adjacency is then unavoidable. O(n log A).

DRY-RUN THESE CASES

One artist (impossible), two artists with equal counts (alternating), the largest artist at exactly half rounded up, three songs with two from the same artist.

LIKELY FOLLOW-UPS

Uniform randomness among all valid shuffles (harder), or a minimum spacing of K songs between the same artist.

PITFALLS

Choosing the largest group even when it was just played. Assuming the greedy result is uniformly random.

HOW TO THINK ABOUT IT

A perfect arrangement exists exactly when the most frequent artist has at most ceil(n / 2) songs. Build it greedily: always play from the artist with the most songs left, except the artist who just played. A max-heap gives the most frequent artist; hold the previous artist back for one step. If it is impossible, the leftover songs of the dominant artist are the only unavoidable adjacencies.

Basis: LeetCode 767 (Reorganize String) and 1054 (Distant Barcodes) with songs and artists. Minimising adjacent same-artist pairs when a perfect arrangement does not exist.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(songs):
    best, best_pairs = None, INF
    for order in permutations(songs):
        pairs = 0
        for a, b in zip(order, order[1:]):
            if a.artist == b.artist:
                pairs += 1
        if pairs < best_pairs:
            best, best_pairs = order, pairs
    return best
```

TIME	SPACE
O(n! * n)	O(n)

WHY IT IS SLOW

n! orderings. A greedy that always plays the artist with the most songs left (not the one that just played), using a heap, is O(n log A).

OPTIMAL SOLUTION · Greedy with a max-heap, holding back the last artist

TIME	SPACE
O(n log A)	O(A)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The shuffle should look random, not always the same arrangement.

Break ties between equally frequent artists randomly and pick the song within the artist at random, as in the code. The result is valid but not uniform among all valid shuffles; a uniform sampler needs counting arrangements.

FOLLOW-UP 2 Songs by the same artist must be at least K apart.

Same idea with a waiting queue of length K: after playing an artist, hold them for K - 1 more songs before they are eligible again (the LeetCode 358 construction).

FOLLOW-UP 3 What is the minimum number of unavoidable repeats?

With n songs and a dominant artist with m songs, the other n - m songs separate at most n - m + 1 groups, so at least m - (n - m + 1) = 2m - n - 1 neighbouring pairs of that artist are forced (if positive).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Playlist Shuffle No Adjacent Artists
#
# Given a playlist of songs where each song has an artist,
# implement a shuffle function that minimizes (or
# eliminates) adjacent songs by the same artist.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The shuffle should look random, not always the same
# arrangement.
# 2. Songs by the same artist must be at least K apart.
# 3. What is the minimum number of unavoidable repeats?
#
# INTERVIEW NOTES
# A perfect arrangement exists exactly when the most
# frequent artist has at most ceil(n / 2) songs. Build it
# greedily: always play from the artist with the most
# songs left, except the artist who just played. A max-
# heap gives the most frequent artist; hold the previous
# artist back for one step. If it is impossible, the
# leftover songs of the dominant artist are the only
# unavoidable adjacencies.
#
# BRUTE FORCE (say it first): Try every ordering and keep
# the fewest adjacent pairs
# Enumerate all orderings of the artists' songs (as a
# multiset) and count neighbours with the same artist.
# Exact and the oracle; hopeless beyond about ten songs.
# Time O(n! * n). Space O(n).


import heapq
import random
from collections import deque
from itertools import permutations


# HELPERS for the brute force
def adjacent_pairs(playlist):
    pairs = zip(playlist, playlist[1:])
    return sum(1 for a, b in pairs if a[0] == b[0])


# BRUTE FORCE (say it first, it also checks the answers)
def brute_min_pairs(songs):
    # oracle: try every order (small playlists only)
    orders = permutations(songs)
    return min(adjacent_pairs(p) for p in orders)


# HELPERS for the base question
def release(waiting, heap):
    # the oldest waiting artist may play again (if it has
    # any song left)
    entry = waiting.popleft()
    if entry[0]:
        heapq.heappush(heap, entry)


def take_oldest(waiting, heap):
    # break the gap on purpose: the first waiting artist
    # that still has songs plays early
    while waiting:
        entry = waiting.popleft()
        if entry[0]:
            heapq.heappush(heap, entry)
            return


# BASE QUESTION (optimal): Greedy with a max-heap, holding
# back the last artist
# Idea: Keep a heap of (remaining songs, artist). Each
# step pop the artist with the most songs left; play one
# of their songs, and push them back only after the next
# artist has played (so they cannot repeat).
# FOLLOW-UP 1: The shuffle should look random, not always
# the same arrangement.
# Answer: Break ties between equally frequent artists
# randomly and pick the song within the artist at random,
# as in the code. The result is valid but not uniform
# among all valid shuffles; a uniform sampler needs
# counting arrangements.
# FOLLOW-UP 2: Songs by the same artist must be at least K
# apart.
# Answer: Same idea with a waiting queue of length K:
# after playing an artist, hold them for K - 1 more songs
# before they are eligible again (the LeetCode 358
# construction).
def shuffle_playlist(songs, gap=2, strict=False,
                     rng=random):
    # songs are (artist, title). The artist with the most
    # songs left plays next and then waits `gap - 1` other
    # songs before it may play again (gap 2 = never twice
    # in
    # a row). Ties between artists and the song picked
    # inside
    # an artist are random, so every run looks different.
```

```
    by_artist = {}
    for artist, title in songs:
        by_artist.setdefault(artist, []).append(title)
    for titles in by_artist.values():
        # FOLLOW-UP 1: random song order inside an artist
        rng.shuffle(titles)
    # FOLLOW-UP 1: random tie-break between artists
    heap = [(-len(t), rng.random(), a)
            for a, t in by_artist.items()]
    heapq.heapify(heap)
    waiting, out = deque(), []
    while len(out) < len(songs):
        if not heap:
            # everyone left is still waiting
            if strict:
                return None
            take_oldest(waiting, heap)
        left, _, artist = heapq.heappop(heap)
        out.append((artist, by_artist[artist].pop()))
        # an artist stays in the queue even with no songs
        # left: it still marks one position of spacing
        waiting.append((left + 1, rng.random(), artist))
        # FOLLOW-UP 2: gap=K keeps an artist out for K - 1
        # songs
        if len(waiting) >= gap:
            release(waiting, heap)
    return out


# FOLLOW-UP 3: What is the minimum number of unavoidable
# repeats?
# Answer: With n songs and a dominant artist with m songs,
# the other n - m songs separate at most n - m + 1 groups,
# so at least m - (n - m + 1) = 2m - n - 1 neighbouring
# pairs of that artist are forced (if positive).
def forced_repeats(counts):
    # follow-up: the fewest unavoidable neighbouring
    # pairs.
    # The other songs split the dominant artist into at
```

```
    # most
    # (n - m + 1) groups, so 2m - n - 1 pairs touch.
    n, m = sum(counts), max(counts)
    return max(0, 2 * m - n - 1)


if __name__ == "__main__":
    rng = random.Random(7)
    songs = [("A", "song1"), ("A", "song2"),
             ("B", "song3"), ("B", "song4"),
             ("C", "song5")]
    mixed = shuffle_playlist(songs, rng=rng)
    assert sorted(mixed) == sorted(songs)
    assert adjacent_pairs(mixed) == 0
    # 4 songs by A and 1 by B: 2 touching pairs are forced
    heavy = [("A", str(i)) for i in range(4)]
    heavy.append(("B", "x"))
    out = shuffle_playlist(heavy, rng=rng)
    assert adjacent_pairs(out) == forced_repeats([4, 1])
    assert adjacent_pairs(out) == 2
    strict = shuffle_playlist(heavy, strict=True, rng=rng)
    assert strict is None
    assert shuffle_playlist([], rng=rng) == []
    # at least 3 apart: three artists, two songs each
    trio = [(a, a + str(i))
            for a in "ABC" for i in range(2)]
    far = shuffle_playlist(trio, 3, True, rng)
    assert sorted(far) == sorted(trio)
    for i, (artist, _) in enumerate(far):
        recent = far[max(0, i - 2):i]
        assert all(other != artist for other, _ in recent)
    # runs differ from each other but are all valid
    seen = {tuple(shuffle_playlist(songs, rng=rng))
```

```
            for _ in range(30)}
    assert len(seen) > 1
    for _ in range(150):
        size = random.randint(1, 7)
        songs = [(random.choice("ABC"), str(i))
                 for i in range(size)]
        counts = [sum(1 for a, _ in songs if a == x)
                  for x in "ABC"]
        got = shuffle_playlist(songs, rng=rng)
        assert sorted(got) == sorted(songs)
        pairs = adjacent_pairs(got)
        assert pairs == forced_repeats(counts)
```

```
        assert pairs == brute_min_pairs(songs)
```

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

Site tags: binary-search, heap, divide-and-conquer, hard

HARD | IN APP LC 4 | 1Point3Acres | tech-screen | 11 of 33

Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 4/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: your app's interview approach (LeetCode #4)

LeetCode match: LC 4 Median of Two Sorted Arrays

Frequency: Single report | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/median-two-sorted-arrays-k-inputs

QUESTION

L5 R1 coding: LC 4 (Median of Two Sorted Arrays). Heading toward the LC 295 streaming two-heap approach drew a redirect from the interviewer. Follow-up: extend to many sorted inputs, in the spirit of LC 23 (Merge k Sorted Lists).

Per the candidate, the base question is LC 4 (Median of Two Sorted Arrays): given two sorted arrays, return the median of their combined elements.
Follow-up: the input becomes many sorted arrays instead of two, in the spirit of LC 23 (Merge k Sorted Lists).

APPROACH

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the median definition when the total is even (the mean of the two middle values), whether the arrays can be empty, whether duplicates are possible, and the sizes.

BRUTE FORCE TO SAY OUT LOUD

Merge and sort, then read the middle. O((m + n) log(m + n)); merging two sorted arrays with two pointers is O(m + n).

OPTIMAL IDEA

Binary search the cut in the shorter array: choose i in a and j = (m + n + 1) // 2 - i in b so the left parts hold half of the elements. The cut is valid when a[i-1] <= b[j] and b[j-1] <= a[i], with minus and plus infinity at the ends. The answer is the larger left value for an odd total, or the mean of the larger left and the smaller right value. O(log min(m, n)). For k arrays: binary search on the value and count elements at most x with a bisect per array, O(k log n log range), or pop from a min-heap until the middle, O(N log k).

DRY-RUN THESE CASES

One array empty, both of equal length, total odd and even, all elements equal, one array entirely smaller than the other, negative numbers.

LIKELY FOLLOW-UPS

Many sorted arrays (the stated follow-up), a stream (two heaps), or the k-th element instead of the median.

PITFALLS

Starting from the two-heap design of the streaming median (wrong family for static sorted input). Searching the longer array and running past its end. Off-by-one in the half size for odd totals.

HOW TO THINK ABOUT IT

Both inputs are already sorted, so the median is a partition problem, not a streaming one (the two-heap design of LeetCode 295 is the wrong family and the interviewer redirects it). Split the small array at i and the large one at j so that the left halves together hold half the elements; the split is right when every left value is at most every right value, which only needs two comparisons across the cut. That condition is monotone in i, so binary search it in O(log min(m, n)). For k arrays the same idea becomes: binary search on the value and count elements at most x with one bisect per array.

Basis: LeetCode 4, Median of Two Sorted Arrays; the follow-up to many sorted inputs is in the spirit of LeetCode 23, Merge k Sorted Lists.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def median(a, b):
    merged = sorted(a + b)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2
```

TIME	SPACE
O((m + n) log(m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that the inputs are sorted and costs O((m + n) log(m + n)) time and O(m + n) memory. A binary search for the partition reads only O(log min(m, n)) values.

OPTIMAL SOLUTION · Binary search on the partition

TIME	SPACE
O(log min(m, n))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Now there are k sorted arrays instead of two. How do you find the median?

Binary search on the answer value. For a candidate x, count the elements at most x with one bisect per array (O(k log n)); the median is the smallest x whose count reaches the middle rank. With integers the search range is the global min to max, so the cost is O(k log n log(range)). The heap alternative pops the smallest element from a k-way merge until it passes the middle, O((N / 2) log k), which is simpler but linear in the total size.

FOLLOW-UP 2 What is the heap version and when would you choose it?

A min-heap holds the front of every array. Pop (N + 1) / 2 times, remembering the one or two middle values. It is O(N log k) at worst, trivial to get right and needs no bound on the values, so choose it when the arrays are small or the values are not integers; choose the value search when N is huge and k is small.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Median of Two Sorted Arrays (LC 4) with a
# Follow-up Variant
#
# L5 R1 coding: LC 4 (Median of Two Sorted Arrays).
# Heading toward the LC 295 streaming two-heap approach
# drew a redirect from the interviewer. Follow-up: extend
# to many sorted inputs, in the spirit of LC 23 (Merge k
# Sorted Lists). Per the candidate, the base question is
# LC 4 (Median of Two Sorted Arrays): given two sorted
# arrays, return the median of their combined elements.
# Follow-up: the input becomes many sorted arrays instead
# of two, in the spirit of LC 23 (Merge k Sorted Lists).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Now there are k sorted arrays instead of two. How do
# you find the median?
# 2. What is the heap version and when would you choose
# it?
#
# INTERVIEW NOTES
# Both inputs are already sorted, so the median is a
# partition problem, not a streaming one (the two-heap
# design of LeetCode 295 is the wrong family and the
# interviewer redirects it). Split the small array at i
# and the large one at j so that the left halves together
# hold half the elements; the split is right when every
# left value is at most every right value, which only
# needs two comparisons across the cut. That condition is
# monotone in i, so binary search it in O(log min(m, n)).
# For k arrays the same idea becomes: binary search on the
# value and count elements at most x with one bisect per
# array.
#
# BRUTE FORCE (say it first): Merge everything and read
# the middle
# Concatenate, sort and take the middle one or two. It is
# the oracle and the baseline to say first. It ignores
# that the inputs are sorted, and it needs the whole
# merged array in memory. Time O((m + n) log(m + n)).
# Space O(m + n).


import bisect
import heapq


# BRUTE FORCE (say it first, it also checks the answers)
def median_brute(*arrays):
    # the oracle: merge, sort, read the middle
    merged = sorted(x for a in arrays for x in a)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Binary search on the partition
# Idea: Make a the shorter array. Binary search i in a,
# set j = half - i in b so the left side holds (m + n + 1)
# // 2 elements.
def median_two(a, b):
    """Both arrays are sorted: binary search the cut. The
    left parts must hold half of all the elements."""
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    if m + n == 0:
        raise ValueError("no elements")
    lo, hi, half = 0, m, (m + n + 1) // 2
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        # values just left and right of each cut
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < m else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < n else INF
        if a_left <= b_right and b_left <= a_right:
            # a valid cut: read the median off its edges
            if (m + n) % 2:
                return max(a_left, b_left)
            return (max(a_left, b_left)
                    + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1


# FOLLOW-UP 1: Now there are k sorted arrays instead of
# two. How do you find the median?
# Answer: Binary search on the answer value. For a
# candidate x, count the elements at most x with one
# bisect per array (O(k log n)); the median is the
# smallest x whose count reaches the middle rank.
```

```python
def median_k_value(arrays):
    """k arrays: binary search the value; one bisect per
    array counts how many elements are at most x."""
    arrays = [a for a in arrays if a]
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")

    def kth(rank):
        # smallest x with at least `rank` elements <= x
        lo = min(a[0] for a in arrays)
        hi = max(a[-1] for a in arrays)
        while lo < hi:
            mid = (lo + hi) // 2
            count = sum(bisect.bisect_right(a, mid)
                        for a in arrays)
            if count >= rank:
                hi = mid
            else:
                lo = mid + 1
        return lo

    if total % 2:
        return kth(total // 2 + 1)
    return (kth(total // 2) + kth(total // 2 + 1)) / 2


# FOLLOW-UP 2: What is the heap version and when would you
# choose it?
# Answer: A min-heap holds the front of every array. Pop
# (N + 1) / 2 times, remembering the one or two middle
# values.
def median_k_heap(arrays):
    """k arrays, simple version: pop from a k-way merge
    until the middle one or two values have been seen."""
    heap = [(a[0], i, 0)
            for i, a in enumerate(arrays) if a]
    heapq.heapify(heap)
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")
    low_rank, high_rank = (total - 1) // 2, total // 2
    seen, middle = 0, []
    while heap:
        value, i, j = heapq.heappop(heap)
        if seen >= low_rank:
            middle.append(value)
        if seen >= high_rank:
            break
        seen += 1
        if j + 1 < len(arrays[i]):
            nxt = (arrays[i][j + 1], i, j + 1)
            heapq.heappush(heap, nxt)
    if total % 2:
        return middle[0]
    return (middle[0] + middle[1]) / 2


if __name__ == "__main__":
    assert median_two([1, 3], [2]) == 2
    assert median_two([1, 2], [3, 4]) == 2.5
    assert median_two([], [5]) == 5
    assert median_k_value([[1, 3], [2], [4, 5, 6]]) == 3.5
    assert median_k_heap([[1, 3], [2], [4, 5, 6]]) == 3.5
    import random
    for _ in range(300):
        arrays = []
        for _ in range(random.randint(1, 5)):
            size = random.randint(0, 6)
            arrays.append(sorted(random.randint(-9, 9)
                                 for _ in range(size)))
        if not any(arrays):
            continue
        want = median_brute(*arrays)
        assert median_k_value(arrays) == want
        assert median_k_heap(arrays) == want
        if len(arrays) >= 2:
            rest = sum(arrays[1:], [])
            rest.sort()
            assert median_two(arrays[0], rest) == want
```

NOTES FROM CANDIDATES

Pitfalls: Starting from the LC 295 (Find Median from Data Stream) two-heap design is the wrong family for the base question; the interviewer stepped in with a hint. Both inputs are already sorted — lean on that.

Follow-ups: Many sorted inputs instead of two

PREPARATION

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Heaps and top-k

HARD | PracHub | Technical Screen | 12 of 33

Return top-k subset sums in descending order

POSTED 2026-01-07 | 286 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 170/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Heaps and top-k

Optimal answer from: the interview site's own solution

LeetCode match: LC 2386 Find the K-Sum of an Array

Posted on PracHub: 2026-01-07 (updated 2026-05-21)

Original: https://prachub.com/coding-questions/return-top-k-subset-sums-in-descending-order

QUESTION

Overview: This question evaluates proficiency in combinatorial reasoning, algorithmic optimization for large search spaces, and efficient extraction of top-k values within the coding & algorithms domain.

APPROACH

Sort values, start from the all-positives sum, and use a max/min-heap of (sum, index) states that either drop the next element or swap it for the following one, popping k times. O(n log n + k log k).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the empty subset counts, whether values can be negative or repeated, how large k is compared with 2^n, and whether equal sums count separately.

BRUTE FORCE TO SAY OUT LOUD

Enumerate all 2^n subsets, sort, take k. Fine for n <= 20; say that out loud.

OPTIMAL IDEA

Reduce negatives: let base be the sum of the positive values, and work with absolute values. Every subset sum of the original equals base minus a subset sum of the absolute values, so the k largest sums are base minus the k smallest subset sums of the sorted absolute values. Generate the k smallest sums with a min-heap of (sum, last index): from a state you either add the next element or replace the last element with the next one. Each pop yields the next smallest sum and creates at most two states, so O(n log n + k log k).

DRY-RUN THESE CASES

All positive values. All negative values (best is the empty subset). A zero in the array. k = 1 and k = 2^n.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Need k sums without storing all of them, subsets themselves (store parent pointers), or a limit on subset size.

PITFALLS

Forgetting to sort first, which breaks the add-next/replace-last move set. Pushing duplicate states.

HOW TO THINK ABOUT IT

k is small compared with 2^n, so never enumerate all subsets. Turn the problem into 'k smallest sums of non-negative numbers': subtract everything from the sum of the positive values. Then grow answers in increasing order with a heap where each state has exactly two successors (extend with the next element, or swap the last element for the next one).

Basis: Written; the classic 'k smallest subset sums' heap technique (also behind LeetCode 2386, K-Sum of an Array).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    sums = []
    for size in range(len(nums) + 1):
        for subset in combinations(nums, size):
            sums.append(sum(subset))
    sums.sort(reverse=True)
    return sums[:k]
```

TIME	SPACE
O(2^n * n)	O(2^n)

WHY IT IS SLOW

2^n subsets and a sort of them: O(2^n * n) time and O(2^n) space, hopeless past about n = 20. A heap that grows the k best sums step by step is O(n log n + k log k).

OPTIMAL SOLUTION · Heap of the k smallest sums of absolute values

TIME	SPACE
O(n log n + k log k)	O(n + k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the subsets themselves, not only the sums.

Store each heap state as a node with (last index, parent node) instead of copying lists, and rebuild a subset by walking parents only for the k answers. That keeps pushes O(1).

FOLLOW-UP 2 **k can be as large as 2^n.**

Then the output itself is that large; the heap approach is still O(k log k) and stops when the heap empties. Check k against 2^n first and fall back to enumeration for tiny n.

FOLLOW-UP 3 **Limit subsets to at most m elements.**

Add the element count to each heap state and refuse to push successors that exceed m. The ordering argument is unchanged because both successors never make the sum smaller.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Return top-k subset sums in descending
# order
#
# This question evaluates proficiency in combinatorial
# reasoning, algorithmic optimization for large search
# spaces, and efficient extraction of top-k values within
# the coding & algorithms domain.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsets themselves, not only the sums.
# 2. k can be as large as 2^n.
# 3. Limit subsets to at most m elements.
#
# INTERVIEW NOTES
# k is small compared with 2^n, so never enumerate all
# subsets. Turn the problem into 'k smallest sums of non-
# negative numbers': subtract everything from the sum of
# the positive values. Then grow answers in increasing
# order with a heap where each state has exactly two
# successors (extend with the next element, or swap the
# last element for the next one).
#
# BRUTE FORCE (say it first): Enumerate all 2^n subsets
# and sort
# Compute every subset sum, sort in descending order and
# take k. Fine for n up to about 20, hopeless beyond that.
# Time O(2^n * n). Space O(2^n).


import heapq
from itertools import combinations


def brute(nums, k):
    # every subset: the check used by the demo
    sums = []
    for r in range(len(nums) + 1):
        for c in combinations(nums, r):
            sums.append(sum(c))
    return sorted(sums, reverse=True)[:k]


# BRUTE FORCE (say it first): Enumerate all 2^n subsets
# and sort | time O(2^n * n)
def top_k_sums_brute(nums, k):
```

```
    sums = []
    for mask in range(1 << len(nums)):
        sums.append(
            sum(
                nums[i]
                for i in range(len(nums))
                if mask >> i & 1
            )
        )
    return sorted(sums, reverse=True)[:k]


# BASE QUESTION (optimal): Heap of the k smallest sums of
# absolute values
# Idea: Every subset sum equals (sum of positives) minus a
# subset sum of the absolute values, so the k largest sums
# are the sum of positives minus the k smallest subset
# sums of the sorted absolute values.
def top_k_sums(nums, k):
    # Idea: start from the best subset (all positives).
    # Every
    # other subset is that one with some elements
    # "flipped",
    # which costs |x|. So the k best subsets are the k
    # cheapest sets of flips: a heap over sorted |x|.
    base = sum(x for x in nums if x > 0)
    cost = sorted(abs(x) for x in nums)
    out = [0]
    heap = [(cost[0], 0)] if cost else []
    while heap and len(out) < k:
        s, i = heapq.heappop(heap)
        out.append(s)
        if i + 1 < len(cost):
            # two successors, neither is ever cheaper:
            # add the next element ...
            heapq.heappush(heap, (s + cost[i + 1], i + 1))
            # ... or move the last flipped one up by one
            moved = s - cost[i] + cost[i + 1]
            heapq.heappush(heap, (moved, i + 1))
    return [base - c for c in out[:k]]


# FOLLOW-UP 1: Return the subsets themselves, not only the
# sums.
# Answer: Store each heap state as a node with (last
```

```
# index, parent node) instead of copying lists, and
# rebuild a subset by walking parents only for the k
# answers. That keeps pushes O(1).
def top_k_subsets(nums, k):
    # also return the subsets: states keep a link to their
    # parent instead of copying lists
    order = sorted(range(len(nums)),
                   key=lambda i: abs(nums[i]))
    cost = [abs(nums[i]) for i in order]
    best = {i for i in range(len(nums)) if nums[i] > 0}

    def subset(chain):
        flipped = set()
        while chain is not None:
            flipped.add(order[chain[0]])
            chain = chain[1]
        return sorted(nums[i] for i in best ^ flipped)

    base = sum(nums[i] for i in best)
    out = [(base, sorted(nums[i] for i in best))]
    heap = [(cost[0], 0, (0, None))] if cost else []
    while heap and len(out) < k:
        s, i, chain = heapq.heappop(heap)
        out.append((base - s, subset(chain)))
        if i + 1 < len(cost):
            heapq.heappush(heap, (s + cost[i + 1], i + 1,
                                  (i + 1, chain)))
            # replace the last flip i by i + 1
            moved = s - cost[i] + cost[i + 1]
            heapq.heappush(heap, (moved, i + 1,
```

```
                                (i + 1, chain[1])))
    return out[:k]


if __name__ == "__main__":
    nums = [3, -1, 2, -4]
    assert top_k_sums(nums, 5) == brute(nums, 5)
    assert top_k_sums(nums, 4) == [5, 4, 3, 2]
    # k bigger than the number of subsets: all 2^n sums
    assert top_k_sums([1, 2], 10) == brute([1, 2], 10)
    assert top_k_sums([], 3) == [0]
    cases = ([5, -2, 7, 1], [-1, -2, -3], [0, 4, -4])
    for sample in cases:
        for k in range(1, 2 ** len(sample) + 1):
            got = top_k_sums(sample, k)
```

```
            assert got == brute(sample, k)
    # the subsets add up to the sums
    for total, sub in top_k_subsets(nums, 4):
        assert sum(sub) == total
    totals = [t for t, _ in top_k_subsets(nums, 4)]
    assert totals == [5, 4, 3, 2]
```

FUNCTION AND TEST CASES

Signature

```
def solution(w: list[int], k: int) -> list[int]:
```

Sample call

```
solution([3, 2, 1], 4)
```

Test cases

Input	Expected
([3, 2, 1], 4)	[6, 5, 4, 3]
([5], 2)	[5, 0]

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

Site tags: interval, heap, greedy, medium

MEDIUM | IN APP LC 253 | 1Point3Acres | phone-screen, tech-screen, onsite-coding | 13 of 33

Meeting Rooms / Interval Scheduling Variants

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 58** | **30 DAYS 0** | **PRIORITY 44/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: your app's interview approach (LeetCode #253)

LeetCode match: LC 253 Meeting Rooms II

Frequency: Very high | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

QUESTION

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

QUESTION

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling). Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example:
    [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40],
    D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap:
    intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

APPROACH

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

HOW TO THINK ABOUT IT

The number of rooms needed is the largest number of intervals overlapping at any moment. Turn each interval into +1 at its start and -1 at its end, sort the events (ends before starts at the same time) and keep a running count. To say which room each meeting gets, keep a min-heap of (end time, room) and reuse the room that frees up first.

Basis: LeetCode 253, Meeting Rooms II, with the common variants: room assignment and cyclic (24-hour) time.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(intervals):
    best = 0
    for start, _ in intervals:
        busy = 0
        for s, e in intervals:
            if s <= start < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(n^2)	O(1)

BRUTE FORCE

WHY IT IS SLOW

Every interval is compared with every other, O(n^2). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Sweep line over start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Meetings wrap around midnight (cyclic time).

Split every wrapped meeting [s, e) with s >= e into [s, 24h) and [0, e) and run the same sweep: the overlap at any clock time is unchanged. A meeting that is exactly 24 hours long covers the whole day.

FOLLOW-UP 2 The input is huge and spread over many machines.

Each shard sorts its events and produces a compressed timeline of (time, delta) changes; merge the sorted timelines with a k-way merge and take the running maximum, or add the per-shard counts at fixed time buckets. The answer is a max over time of a sum over shards.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Meeting Rooms / Interval Scheduling
# Variants
#
# LC 253 (Meeting Rooms II) appears in many forms across
# phone screen and onsite coding rounds — minimum number
# of rooms, room assignment, cyclic time, daily server
# jobs, gate entry/exit. Common follow-ups push toward
# sweep line, min-heap, or distributed processing. The
# base task in every variant: given a list of intervals
# (start, end) — or pickup/return pairs, meeting time
# pairs, server job runs — answer one or more of: Min
# rooms / cars / servers needed to honor all intervals
# concurrently (the canonical min-meeting-rooms problem).
# Assignment: which interval goes to which room/car, so
# the caller can use the rental record. Single-interval
# overlap test between two intervals. Variant: parking-lot
# capacity Arrival and departure intervals represent cars
# occupying a parking lot. Compute the peak simultaneous
# occupancy using the same interval-overlap model.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Meetings wrap around midnight (cyclic time).
# 2. The input is huge and spread over many machines.
#
# INTERVIEW NOTES
# The number of rooms needed is the largest number of
```

```
# intervals overlapping at any moment. Turn each interval
# into +1 at its start and -1 at its end, sort the events
# (ends before starts at the same time) and keep a running
# count. To say which room each meeting gets, keep a min-
# heap of (end time, room) and reuse the room that frees
# up first.
#
# BRUTE FORCE (say it first): Count the overlap at every
# start time
# For every interval count how many intervals contain its
# start point and take the maximum. Correct and simple;
# quadratic. Time O(n^2). Space O(1).


import heapq
from collections import Counter


# BRUTE FORCE (say it first): Count the overlap at every
```

```
# start time | time O(n^2)
def min_rooms_brute(intervals):
    return max(
        (
            sum(
                1
                for s2, e2 in intervals
                if s2 <= s < e2
            )
            for s, e in intervals
        ),
        default=0,
    )


# BASE QUESTION (optimal): Sweep line over start and end
# events
# Idea: Create (time, +1) for each start and (time, -1)
# for each end, sort so that an end at time t comes before
# a start at t (rooms free up the moment a meeting ends)
# and track the running maximum.
def min_rooms(intervals):
    # +1 at every start, -1 at every end
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e, -1))
    # at equal times ends (-1) sort before starts (+1)
    events.sort()
    cur = best = 0
    for _, delta in events:
        cur += delta
        best = max(best, cur)
    return best


# VARIANT: which room for each meeting
def assign_rooms(intervals):
    # which room does each meeting get?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        # reuse the room that frees first, if it is free
```

```
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms
```

```
# VARIANT: on-call consolidation
def oncall_segments(rotations):
    # rotations: (name, start, end), half-open intervals
    delta = {}
    for name, s, e in rotations:
        delta.setdefault(s, []).append((name, 1))
        delta.setdefault(e, []).append((name, -1))
    active, out, times = Counter(), [], sorted(delta)
    for t, nxt in zip(times, times[1:]):
        # apply every change that happens at time t
        for name, d in delta[t]:
            active[name] += d
        who = frozenset(n for n in active if active[n])
        # skip gaps; merge with the previous equal segment
        if not who:
            continue
        if out and out[-1][1] == t and out[-1][2] == who:
            out[-1] = (out[-1][0], nxt, who)
        else:
            out.append((t, nxt, who))
    return out


# FOLLOW-UP 1: Meetings wrap around midnight (cyclic
# time).
# Answer: Split every wrapped meeting [s, e) with s >= e
# into [s, 24h) and [0, e) and run the same sweep: the
# overlap at any clock time is unchanged. A meeting that
# is exactly 24 hours long covers the whole day.
def min_rooms_cyclic(intervals, day=24 * 60):
    parts = []
    for s, e in intervals:
        if s < e:
            parts.append((s, e))
        else:
            # wraps past midnight: split at the boundary
```

```
            parts += [(s, day), (0, e)]
    return min_rooms(parts)


if __name__ == "__main__":
    base = [(0, 30), (5, 10), (15, 20)]
    assert min_rooms(base) == 2
    # touching meetings share a room
    assert min_rooms([(5, 8), (8, 9)]) == 1
    rooms = assign_rooms(base)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    wrap = [(22 * 60, 2 * 60), (23 * 60, 60), (0, 3 * 60)]
    assert min_rooms_cyclic(wrap) == 3
    rot = [("A", 1, 5), ("B", 3, 8)]
    assert oncall_segments(rot) == [
        (1, 3, frozenset("A")),
        (3, 5, frozenset("AB")),
        (5, 8, frozenset("B"))]
    # a gap with nobody on call is left out
    gap = [("A", 1, 2), ("B", 4, 5)]
    assert len(oncall_segments(gap)) == 2
```

NOTES FROM CANDIDATES

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.

The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

PREPARATION

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

Site tags: interval, two-pointer, heap, streaming, medium

MEDIUM | 1Point3Acres | onsite-coding | 14 of 33

Union of Sorted Interval Lists

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 25.5** **30 DAYS 0** **PRIORITY 139/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #986 Interval List Intersections

LeetCode match: LC 986 Interval List Intersections (related)

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/sorted-interval-list-union

QUESTION

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union. Follow-ups extend to K lists with a heap, inputs too large for memory, and lists that may overlap internally.SWEintervaltwo-pointerheapstreamingmediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether intervals are closed or half-open, whether touching intervals merge, whether each list is sorted and non-overlapping internally (later follow-up removes this), and how large the inputs are.

BRUTE FORCE TO SAY OUT LOUD

Concatenate both lists, sort, and merge. O((m + n) log (m + n)).

OPTIMAL IDEA

Two-pointer merge: repeatedly take the interval with the smaller start from either list and fold it into the last output interval if it overlaps (start at most the previous end), else append it. O(m + n). For K lists use heapq.merge (a heap of K heads), and since it is a generator the inputs never need to fit in memory.

DRY-RUN THESE CASES

One list empty, no overlaps, one interval swallowing many, touching endpoints, identical lists.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

K lists, lists too big for memory, lists that overlap internally (sort each first or heap everything).

PITFALLS

Using end instead of start to pick the next interval. Not taking the maximum of ends when merging.

HOW TO THINK ABOUT IT

Each list is already sorted, so you can walk both with two pointers like the merge step of merge sort: always take the interval with the smaller start next. Once the intervals come out in start order, merging is the usual one-pass fold: extend the last output interval if the next one overlaps or touches it, otherwise start a new one.

Basis: The reverse of LeetCode 986 (Interval List Intersections): union instead of intersection.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(list1, list2):
    merged = []
    for s, e in sorted(list1 + list2):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O((m + n) log (m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that both lists are already sorted: O((m + n) log (m + n)) instead of O(m + n) for a two-pointer merge.

OPTIMAL SOLUTION · Two-pointer merge

TIME	SPACE
O(m + n)	O(m + n) for the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

FOLLOW-UP 2 What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

FOLLOW-UP 3 Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

FOLLOW-UP 4 Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Union of Sorted Interval Lists
#
# Reverse of LC 986: given two individually sorted,
# internally non-overlapping interval lists, return their
# union. Follow-ups extend to K lists with a heap, inputs
# too large for memory, and lists that may overlap
# internally.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the lists are too big for memory?
# 2. What if one list may overlap itself?
# 3. Do touching intervals merge?
# 4. Each list may overlap itself or be unsorted.
#
# INTERVIEW NOTES
# Each list is already sorted, so you can walk both with
# two pointers like the merge step of merge sort: always
# take the interval with the smaller start next. Once the
# intervals come out in start order, merging is the usual
# one-pass fold: extend the last output interval if the
# next one overlaps or touches it, otherwise start a new
# one.
#
# BRUTE FORCE (say it first): Concatenate, sort and merge
# Put all intervals in one list, sort by start, and fold
# overlapping ones. Correct and short; the sort ignores
# that both inputs were already sorted. Time O((m + n) log
# (m + n)). Space O(m + n).


import heapq


# BRUTE FORCE (say it first): Concatenate, sort and merge
# | time O((m + n) log (m + n))
def union_sort(a, b):
    out = []
    for s, e in sorted(a + b):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

```
# HELPERS for the base question
# FOLLOW-UP 1: What if the lists are too big for memory?
# Answer: Read each list through an iterator (a file or a
# paged API). union_k only ever holds one interval per
# list plus the current merged interval, so memory is O(K)
# and it can emit results as it goes.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def union_stream(lists, touching=True):
    # lists: sorted iterables of (start, end)
    merged = heapq.merge(*lists)
    cur = None
    for start, end in merged:
        if cur is None:
            cur = [start, end]
            continue
        # touching: [1, 3] and [3, 5] share the point 3
        joined = start < cur[1]
        if touching:
            joined = start <= cur[1]
        if joined:
            cur[1] = max(cur[1], end)
        else:
            # finished one block: emit it as we go
            yield tuple(cur)
            cur = [start, end]
    if cur is not None:
        yield tuple(cur)


# BASE QUESTION (optimal): Two-pointer merge
# Idea: Always take the interval with the smaller start
# next. If it touches or overlaps the last output interval
# (start <= last end, closed intervals) extend that one,
# otherwise append it.
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# VARIANT: K lists with a heap
def union_k(lists, touching=True):
    # same code, any number of lists, O(K) memory
    return list(union_stream(lists, touching))
```

```
# FOLLOW-UP 2: What if one list may overlap itself?
# Answer: The two-pointer merge already works, because it
# folds every incoming interval into the last output
# interval.
def normalise(intervals):
    # a list that is unsorted or overlaps itself
    ordered = sorted(intervals)
    return list(union_stream([ordered]))


# FOLLOW-UP 4: Each list may overlap itself or be
# unsorted.
# Answer: Normalise each list first (sort and merge it by
# itself, O(n log n)), after which the two-pointer union
# applies. Or sort everything together if memory allows.
def union_unsorted(a, b):
    # fix each list first, then use the same union
    return union_two(normalise(a), normalise(b))


if __name__ == "__main__":
```

```
a = [(1, 3), (6, 8)]
b = [(2, 4), (9, 10)]
assert union_two(a, b) == [(1, 4), (6, 8), (9, 10)]
# closed intervals that touch merge
assert union_two([(1, 3)], [(3, 5)]) == [(1, 5)]
# half-open intervals that only touch stay apart
assert union_two([(1, 3)], [(3, 5)], False) == [
    (1, 3), (3, 5)]
# three lists, one of them empty
three = union_k([a, b, []])
assert three == [(1, 4), (6, 8), (9, 10)]
# a list that overlaps itself, and an unsorted one
merged = union_unsorted([(5, 9), (1, 6)], [(8, 12)])
assert merged == [(1, 12)]
assert union_unsorted([(9, 10), (1, 2)], []) == [
    (1, 2), (9, 10)]
```

NOTES FROM CANDIDATES

The round intentionally layers one constraint at a time. Keep the base invariant explicit—every interval before the current pointer has already been absorbed into a canonical output prefix—so the streaming and K-way versions remain easy to justify.
Agree on endpoint semantics before coding: touching closed intervals merge, while touching half-open intervals do not overlap. For K streaming inputs, the heap plus one current interval per input uses O(K) working memory, excluding the output.

PREPARATION

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

MEDIUM | PracHub | Technical Screen | 15 of 33

★ TOP 75: rank 69 in the Medium and Hard priority list

★ Solve Rooms and Top-K Streams

POSTED 2026-06-05 | 137 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 58** | **30 DAYS 0** | **PRIORITY 40/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II, LeetCode #703 Kth Largest Element in a Stream

LeetCode match: LC 253 Meeting Rooms II + LC 703 Kth Largest Element in a Stream (inferred from overview)

Posted on PracHub: 2026-06-05 (updated 2026-06-14)

Original: https://prachub.com/coding-questions/solve-rooms-and-top-k-streams

QUESTION

Overview: This question evaluates algorithmic problem-solving skills across interval scheduling/resource allocation and streaming top-k maintenance, testing competency in data structures for ordered queries, tie-breaking logic, stateful stream processing, and analysis of time and space trade-offs.

APPROACH

Rooms: min-heap of end times. Stream: size-k min-heap, push and pop when over k.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Rooms: is a meeting that starts exactly when another ends a conflict (the scraped solution treats it as no conflict)? Stream: top-K means the K largest values or K most frequent, and what ties do. The scraped solution covers only the rooms part.

BRUTE FORCE TO SAY OUT LOUD

Rooms: assign meetings to rooms and try all orders. Stream: store everything and sort on each query.

OPTIMAL IDEA

Rooms: sort starts and ends separately and sweep with two pointers: if the next start is at least the earliest end, reuse that room, otherwise open a new one; or keep a min-heap of room end times. O(n log n). Stream of the K largest (LeetCode 703): a min-heap of size K, push each value and pop when the size exceeds K, so the root is the K-th largest. O(log K) per value.

DRY-RUN THESE CASES

Rooms: [[0,30],[5,10],[15,20]] gives 2, an empty list gives 0, back-to-back meetings. Stream: fewer than K values so far, duplicates, K = 1.

LIKELY FOLLOW-UPS

Return the room assignment, or a sliding window of the last N values for the stream.

PITFALLS

Sorting starts and ends together by start. Returning from the heap before it holds K items.

INTERVIEW-DAY PLAN

Rooms: [[0,30],[5,10],[15,20]] gives 2, an empty list gives 0, back-to-back meetings. Stream: fewer than K values so far, duplicates, K = 1.

LIKELY FOLLOW-UPS

Return the room assignment, or a sliding window of the last N values for the stream.

PITFALLS

Sorting starts and ends together by start. Returning from the heap before it holds K items.

HOW TO THINK ABOUT IT

Rooms: the number of rooms needed is the largest number of meetings happening at the same time, so sort the starts and the ends separately and sweep: a start that comes before the earliest end needs a new room. Stream: only the K largest values can matter, so keep exactly those in a min-heap; the root is the K-th largest.

Basis: Two tasks: LeetCode 253 (minimum meeting rooms; back-to-back meetings may share a room) and LeetCode 703 (Kth Largest Element in a Stream).

BRUTE FORCE · Rooms: count the overlap at every time unit

BRUTE FORCE · PSEUDO-CODE

```
def rooms(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, not only on n: O(range * n). Sorted starts and ends with two pointers (or a heap of end times) are O(n log n).

BRUTE FORCE · Stream: sort everything on every query

BRUTE FORCE · PSEUDO-CODE

```
values = []
def add(x):
    values.append(x)
    values.sort(reverse=True)
    return values[K - 1]
```

BRUTE FORCE · Stream: sort everything on every query

TIME	SPACE
O(n log n) per add	O(n)

WHY IT IS SLOW

Every add re-sorts the whole history, O(n log n). A min-heap of size K keeps the K-th largest at its root in O(log K) per add.

OPTIMAL SOLUTION · Rooms: sorted starts and ends with two pointers

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Stream: min-heap of size K

TIME	SPACE
O(log k) per add	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rooms: return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room). If the earliest-ending room is free at the next start, reuse it, otherwise open a new room. The heap size never exceeds the answer.

FOLLOW-UP 2 Stream: K changes over time, or you need the median.

A fixed-size heap cannot answer a different K. Use a sorted structure with order statistics (balanced tree or Fenwick tree over values), or two heaps for the median. **Cost:** a balanced tree or Fenwick tree gives O(log n) per insert and per k-th query; two heaps give O(log n) per insert and O(1) per median.

FOLLOW-UP 3 Rooms: meetings on several floors with capacities.

Group meetings by required capacity, schedule each group separately with the sweep, and assign rooms from the smallest sufficient capacity (best fit) when capacities differ.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve Rooms and Top-K Streams
#
# This question evaluates algorithmic problem-solving
# skills across interval scheduling/resource allocation
# and streaming top-k maintenance, testing competency in
# data structures for ordered queries, tie-breaking logic,
# stateful stream processing, and analysis of time and
# space trade-offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Rooms: return which room each meeting uses.
# 2. Stream: K changes over time, or you need the median.
# 3. Rooms: meetings on several floors with capacities.
#
# INTERVIEW NOTES
# Rooms: the number of rooms needed is the largest number
# of meetings happening at the same time, so sort the
# starts and the ends separately and sweep: a start that
# comes before the earliest end needs a new room. Stream:
# only the K largest values can matter, so keep exactly
# those in a min-heap; the root is the K-th largest.
#
# BRUTE FORCE (say it first): Rooms: count the overlap at
# every time unit
# For each integer time t count the meetings with start <=
# t < end and take the maximum. Correct and the oracle
# (integer times only); it depends on the length of the
# time range, not just n. Time O(range * n). Space O(1).


import heapq


# BRUTE FORCE (say it first): Rooms: count the overlap at
# every time unit | time O(range * n)
def rooms_by_time(intervals):
    if not intervals:
        return 0
    return max(
        sum(1 for s, e in intervals if s <= t < e)
        for t in range(
            min(s for s, _ in intervals),
            max(e for _, e in intervals),
        )
    )
```

```
# BASE QUESTION (optimal): Rooms: sorted starts and ends
# with two pointers
# Idea: Sort starts and ends separately. For each start:
# if it is at or after the earliest unfinished end, that
# room is reused (advance the end pointer); otherwise a
# new room is needed.
def min_rooms(intervals):
    # two sorted lists: every start with no room free
    # needs
    # a new room, otherwise it reuses one that just ended
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = j = 0
    for s in starts:
        if s >= ends[j]:
            # the earliest-ending meeting is over: reuse
            # it
            j += 1
```

```
        else:
            rooms += 1
    return rooms


# PART 2: top K values of a stream
class TopKStream:
    """The k largest values seen so far (fixed k)."""

    def __init__(self, k):
        self.k, self.heap = k, []

    def add(self, x):
        # a min-heap of size k: the root is the k-th
        # largest
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, x)
        elif x > self.heap[0]:
            heapq.heapreplace(self.heap, x)

    def top(self):
        return sorted(self.heap, reverse=True)


# VARIANT: median of the stream
```

```
class MedianStream:
    """Follow-up: the median instead of a fixed top-k."""

    def __init__(self):
        # low is a max-heap (negated), high is a min-heap
        self.low, self.high = [], []

    def add(self, x):
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        if len(self.high) > len(self.low):
            back = heapq.heappop(self.high)
            heapq.heappush(self.low, -back)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# FOLLOW-UP 1: Rooms: return which room each meeting uses.
# Answer: Sort by start and keep a min-heap of (end time,
# room). If the earliest-ending room is free at the next
# start, reuse it, otherwise open a new room. The heap
# size never exceeds the answer.
def assign_rooms(intervals):
    # which room does each meeting use? a heap of (end,
    # room)
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


if __name__ == "__main__":
    meetings = [(0, 30), (5, 10), (15, 20)]
```

```
    assert min_rooms(meetings) == 2
    # a meeting may start the moment another ends
    assert min_rooms([(1, 3), (3, 5)]) == 1
    assert min_rooms([]) == 0
    rooms = assign_rooms(meetings)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    t = TopKStream(3)
    for x in [5, 1, 9, 7, 3, 8]:
        t.add(x)
    assert t.top() == [9, 8, 7]
    m = MedianStream()
    for x in [5, 2, 8, 1]:
        m.add(x)
    assert m.median() == 3.5
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 30], [5, 10], [15, 20]])
```

Test cases

Input	Expected
`([[0, 30], [5, 10], [15, 20]],)`	`2`
`([],)`	`0`

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 16 of 33

★ TOP 75: rank 50 in the Medium and Hard priority list

★ Compute minimum number of rooms needed

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 42/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/compute-minimum-number-of-rooms-needed

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time **`t`** does not conflict with another starting at time **`t`**).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- **`intervals`**: an array of **`n`** pairs **`[start, end]`** where **`0 <= start <= end`**.

Output

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- **`1 <= n <= 200000`**
- Times are integers in a reasonable range (e.g., **`0 . 10^9`**).

Example

- Input: **`[[0,30],[5,10],[15,20]]`**
- Output: **`2`** (One room can host **`[0,30]`**, the other can host **`[5,10]`** then **`[15,20]`**.)

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.
Constraints
0 <= n <= 2000000 <= start <= end <= 10^9

QUESTION

EXAMPLES

```
Input: ([[0,30],[5,10],[15,20]],)
Expected Output: 2
Explanation: [0,30] overlaps with both [5,10] and
    [15,20], so at most 2 rooms are needed at the
    same time.
Input: ([[7,10],[2,4],[4,7],[10,12]],)
Expected Output: 1
```

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

APPROACH

Sort by start, min-heap of end times (reuse a room when earliest end <= start), or two-pointer sweep on sorted starts/ends. O(n log n).

HOW TO THINK ABOUT IT

The rooms needed equal the largest number of meetings that overlap at one moment. Two ways to find that without checking every moment: sort by start and keep a min-heap of the end times of rooms in use (reuse the room that frees up first), or sort starts and ends separately and sweep with two pointers.

Basis: LeetCode 253, Meeting Rooms II (half-open intervals [start, end): a meeting ending at t does not conflict with one starting at t).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, O(range * n). A min-heap of room end times, or sorted starts and ends with two pointers, is O(n log n).

OPTIMAL SOLUTION · Sort by start with a min-heap of end times

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can one person attend all meetings (LeetCode 252)?

Sort by start and check that each meeting ends no later than the next one starts. O(n log n), O(1) extra.

FOLLOW-UP 2 Return which room each meeting uses.

Keep a heap of (end time, room id): reuse the room that frees first, otherwise open the next id, and record the id per meeting.

FOLLOW-UP 3 Meetings arrive online.

Keep the end-time heap between calls: pop every end at or before the new start (those rooms are free), then push the new end. The heap size is the number of rooms in use right now; the peak is the answer so far.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute minimum number of rooms needed
#
# ## Problem You are given a list of meetings, each with a
# start time and end time. A single room can host only one
# meeting at a time. Two meetings **overlap** if one
# starts before the other ends (treat meetings as half-
# open intervals **[start, end)** so a meeting ending at
# time t does not conflict with another starting at time
# t). ### Task Return the **minimum number of rooms**
# required to host all meetings. ### Input - intervals: an
# array of n pairs [start, end] where 0 <= start <= end.
# ### Output - An integer: the minimum number of rooms
# needed. ### Constraints (typical interview bounds) - 1
# <= n <= 200000 - Times are integers in a reasonable
# range (e.g., 0 . 10^9).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can one person attend all meetings (LeetCode 252)?
# 2. Return which room each meeting uses.
# 3. Meetings arrive online.
#
# INTERVIEW NOTES
# The rooms needed equal the largest number of meetings
# that overlap at one moment. Two ways to find that
# without checking every moment: sort by start and keep a
# min-heap of the end times of rooms in use (reuse the
# room that frees up first), or sort starts and ends
# separately and sweep with two pointers.
#
# BRUTE FORCE (say it first): Count the overlap at every
# time unit
# For each integer time count the meetings with start <= t
# < end and take the maximum. Correct and the oracle
# (integer times); it depends on the length of the time
# range, not only on n. Time O(range * n). Space O(1).


import heapq


# BRUTE FORCE (say it first): Count the overlap at every
# time unit | time O(range * n)
def rooms_by_time(intervals):
    if not intervals:
        return 0
    return max(
        sum(1 for s, e in intervals if s <= t < e)
        for t in range(
            min(s for s, _ in intervals),
            max(e for _, e in intervals) + 1,
        )
    )


# BASE QUESTION (optimal): Sort by start with a min-heap
# of end times
# Idea: Process meetings by start time. If the earliest-
# ending room has finished by this meeting's start, reuse
# it (pop and push the new end); otherwise open a new room
# (push).
def min_meeting_rooms(intervals):
    # half-open [start, end): a meeting ending at t does
    # not
    # clash with one starting at t. A heap holds the end
    # times
    # of the rooms in use.
    ends = []
    meetings = sorted(m for m in intervals if m[0] < m[1])
    for start, end in meetings:
        if ends and ends[0] <= start:
            # the earliest room is free: reuse it
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)


# FOLLOW-UP 1: Can one person attend all meetings
# (LeetCode 252)?
# Answer: Sort by start and check that each meeting ends
# no later than the next one starts. O(n log n), O(1)
# extra.
def can_attend_all(intervals):
    # LeetCode 252: one person, no overlap at all
    ordered = sorted(intervals)
    pairs = zip(ordered, ordered[1:])
    return all(a[1] <= b[0] for a, b in pairs)


# FOLLOW-UP 2: Return which room each meeting uses.
# Answer: Keep a heap of (end time, room id): reuse the
# room that frees first, otherwise open the next id, and
# record the id per meeting.
def assign_rooms(intervals):
    # which room does each meeting use?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [None] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        if heap and heap[0][0] <= start:
```

```
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# FOLLOW-UP 3: Meetings arrive online.
# Answer: Keep the end-time heap between calls: pop every
# end at or before the new start (those rooms are free),
# then push the new end. The heap size is the number of
# rooms in use right now; the peak is the answer so far.
class RoomTracker:
    """Meetings arrive online (in order of start time)."""

    def __init__(self):
        self.ends, self.peak = [], 0

    def book(self, start, end):
        # rooms whose meeting already ended are free again
        while self.ends and self.ends[0] <= start:
            heapq.heappop(self.ends)
        heapq.heappush(self.ends, end)
        # the heap size is the rooms in use right now
        self.peak = max(self.peak, len(self.ends))
        return len(self.ends)


if __name__ == "__main__":
    meetings = [(0, 30), (5, 10), (15, 20)]
    assert min_meeting_rooms(meetings) == 2
    assert min_meeting_rooms([(7, 10), (2, 4)]) == 1
    # touching meetings share a room; empty ones need none
```

```
    touching = [(1, 3), (3, 5), (4, 4)]
    assert min_meeting_rooms(touching) == 1
    assert min_meeting_rooms([]) == 0
    assert not can_attend_all(meetings)
    assert can_attend_all([(7, 10), (2, 4)])
    rooms = assign_rooms(meetings)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    t = RoomTracker()
    in_use = [t.book(s, e) for s, e in meetings]
    assert in_use == [1, 2, 2]
    assert t.peak == 2
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0,30],[5,10],[15,20]])
```

Test cases

Input	Expected
`([[0,30],[5,10],[15,20]],)`	`2`
`([[7,10],[2,4],[4,7],[10,12]],)`	`1`

ALSO REPORTED · 4 more

Find Minimum Rooms Needed | PracHub | Technical Screen

https://prachub.com/coding-questions/find-minimum-rooms-needed

Compute minimum rooms for time intervals | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-rooms-for-time-intervals

Given a list of car rental requests with start and end times, assign cars to rentals such | Exponent

https://www.aced.io/questions/5425/assign-cars-minimize-total-cars-rented

Meeting Rooms / Interval Scheduling Variants | 1Point3Acres | phone-screen, tech-screen, onsite-coding | asked 2026-09-20 | Very high

https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.
The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

Preparation: Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

MEDIUM | IN APP LC 2402 | PracHub | Technical Screen | 17 of 33

Simulate room assignments for scheduled meetings

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40 | 30 DAYS 0 | PRIORITY 95/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: your app's interview approach (LeetCode #2402)

LeetCode match: LC 2402 Meeting Rooms III (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-03-29)

Original: https://prachub.com/interview-questions/simulate-room-assignments-for-scheduled-meetings

QUESTION

Overview: This question evaluates scheduling and event-simulation skills, including time-ordered processing, resource allocation with tie-breaking, and efficient tracking of room availability and usage counts using appropriate data structures.

APPROACH

Sort meetings by start, min-heap of free rooms (by index) and a heap of busy rooms (end time, room); assign the lowest free room, else wait for the earliest finish and delay.

HOW TO THINK ABOUT IT

Simulate in start order with two questions per meeting: which rooms are free now (lowest index first), and if none, which room frees up first (the meeting is delayed but keeps its length). Two heaps answer both quickly: one of free room ids, one of (end time, room id) for busy rooms.

Basis: LeetCode 2402, Meeting Rooms III.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(n, meetings):
    end = [0] * n
    count = [0] * n
    for start, finish in sorted(meetings):
        free = [r for r in range(n)
                if end[r] <= start]
        if free:
            r = min(free)
            end[r] = finish
        else:
            r = end.index(min(end))  # earliest
            wait = end[r] - start
            end[r] = finish + wait
        count[r] += 1
    return count.index(max(count))
```

TIME	SPACE
O(M * N)	O(N)

WHY IT IS SLOW

Every meeting scans all N rooms, O(M * N). Two heaps (free rooms and busy rooms by end time) make each meeting O(log N).

OPTIMAL SOLUTION · Two heaps: free rooms and busy rooms

TIME	SPACE
O(M log M + M log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if meetings arrive online and cannot be reordered?

The same two heaps work if you process meetings in arrival order and treat the arrival time as the earliest allowed start, but you can no longer rely on sorted starts, so you must first move every room whose end time is <= the arrival into the free heap. Each call stays O(log N).

FOLLOW-UP 2 How do you report the busiest room by total busy time instead of meeting count?

Add the meeting's actual duration to used[room] instead of 1 when you assign it. Rooms are chosen the same way; only the tally changes, and ties still break toward the lowest index.

FOLLOW-UP 3 Rank rooms by total busy time instead of the number of meetings.

Add each meeting's duration (its original length, not the delay) to the room's tally; the choice of room is unchanged.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Simulate room assignments for scheduled
# meetings
#
# This question evaluates scheduling and event-simulation
# skills, including time-ordered processing, resource
# allocation with tie-breaking, and efficient tracking of
# room availability and usage counts using appropriate
# data structures.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if meetings arrive online and cannot be
# reordered?
# 2. How do you report the busiest room by total busy time
# instead of meeting count?
# 3. Rank rooms by total busy time instead of the number
# of meetings.
#
# INTERVIEW NOTES
# Simulate in start order with two questions per meeting:
# which rooms are free now (lowest index first), and if
# none, which room frees up first (the meeting is delayed
# but keeps its length). Two heaps answer both quickly:
# one of free room ids, one of (end time, room id) for
# busy rooms.
#
# BRUTE FORCE (say it first): Scan all rooms for every
# meeting
# Keep the end time of every room in an array. For each
# meeting in start order, take the lowest-index room that
# has finished by the start; if none, take the room with
# the earliest end (lowest index on ties) and delay the
# meeting. Correct and the oracle; each meeting scans all
# N rooms. Time O(M * N). Space O(N).


import heapq


# BRUTE FORCE (say it first): Scan all rooms for every
# meeting | time O(M * N)
def most_booked_scan(n, meetings):
    end, used = [0] * n, [0] * n
    for start, finish in sorted(meetings):
        free = [r for r in range(n) if end[r] <= start]
        if free:
```

```
            r = free[0]
            end[r] = finish
        else:
            r = min(range(n), key=lambda x: (end[x], x))
            end[r] += finish - start
        used[r] += 1
    return used.index(max(used))


# BASE QUESTION (optimal): Two heaps: free rooms and busy
# rooms
# Idea: Process meetings in start order. Move rooms that
# finished by the meeting's start into the free heap
# (lowest index first). If a room is free, use it.
# FOLLOW-UP 3: Rank rooms by total busy time instead of
# the number of meetings.
# Answer: Add each meeting's duration (its original
# length, not the delay) to the room's tally; the choice
# of room is unchanged.
def most_booked_room(n, meetings, by="count"):
    # a meeting takes the lowest-numbered free room; if
    # none is
    # free it waits for the earliest room and keeps its
    # length
    free = list(range(n))
    heapq.heapify(free)
    # busy rooms: (time it becomes free, room)
    busy = []
    used = [0] * n
    for start, end in sorted(meetings):
        # rooms that finished by now are free again
        while busy and busy[0][0] <= start:
            _, room = heapq.heappop(busy)
            heapq.heappush(free, room)
        if free:
            room = heapq.heappop(free)
            heapq.heappush(busy, (end, room))
        else:
            # delayed: starts when the earliest room frees
            # up
            finish, room = heapq.heappop(busy)
            delayed_end = finish + (end - start)
            heapq.heappush(busy, (delayed_end, room))
        # FOLLOW-UP 3: by="time" ranks by total busy time
        # count meetings, or total length (not the delay)
```

```
        used[room] += 1 if by == "count" else end - start
    # ties go to the lowest room number
    return used.index(max(used))


if __name__ == "__main__":
    assert most_booked_room(2, [[0, 10], [1, 5], [2, 7],
                                [3, 4]]) == 0
    assert most_booked_room(3, [[1, 20], [2, 10], [3, 5],
                                [4, 9], [6, 8]]) == 1
    # a single room takes everything
    assert most_booked_room(1, [[0, 1], [0, 1]]) == 0
    # by busy time instead of the number of meetings
    meetings = [[0, 100], [1, 2], [2, 3], [3, 4]]
    assert most_booked_room(2, meetings, "count") == 1
    assert most_booked_room(2, meetings, "time") == 0
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are given n meeting rooms labeled 0..n-1 and a list of meetings meetings, where each meeting is [start, end] with start < end. Meetings are processed in increasing order of start time.

Assign each meeting to a room using these rules:

If one or more rooms are free at start , assign the meeting to the free room with the smallest index .
If no rooms are free at start , the meeting must wait until a room becomes free. It is then scheduled in the room that becomes available earliest (break ties by smallest room index). If it waits, its duration stays the same , so if the original duration is end - start and the room becomes free at time t , the meeting runs on [t, t + (end - start)] .

Each time a meeting is assigned to a room (even after waiting), that room’s meeting count increases by 1.

Task

Return the index of the room that hosted the most meetings. If there is a tie, return the smallest index.

FULL QUESTION TEXT FROM THE SITE

Input/Output
Input: n (int), meetings (list of [start,end])
Output: an integer room index
Constraints (reasonable interview assumptions)
1 <= n <= 10^5
1 <= len(meetings) <= 10^5
Times fit in 32-bit signed integer; waiting can increase end times beyond original but still fits in 64-bit.

ALSO REPORTED · 1 more

Assign meetings to rooms with delays | PracHub | Technical Screen

https://prachub.com/interview-questions/assign-meetings-to-rooms-with-delays

---

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 18 of 33

★ TOP 75: rank 58 in the Medium and Hard priority list

★ Compute servers needed for daily recurring jobs

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 41/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2025-11-05 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/compute-servers-needed-for-daily-recurring-jobs

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.
- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

Follow-up

QUESTION

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.
Constraints
0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.

EXAMPLES

```
Input: ([],)
Expected Output: 0
```

Explanation: There are no jobs, so no servers are needed.

```
Input: ([[5, 100, 200], [5, 150, 250], [5, 240,
    300]],)
Expected Output: 1
```

Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.

Hints

At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.
Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id may use the same dedicated server. For a given target_day, compute the total number of

APPROACH

Max concurrent jobs = servers needed; sweep line over start/end events, or min-heap of end times. Handle intervals wrapping midnight by splitting.

HOW TO THINK ABOUT IT

The number of servers is the largest number of jobs running at the same time. First remove the twist: overlapping intervals of the same job are really one longer interval, so merge the intervals of each job_id. Then it is the plain maximum overlap: sort start and end events and keep a running count.

Basis: LeetCode 253 (Meeting Rooms II) with a twist from the statement: intervals of the same job_id may overlap on one server and count once. Intervals are half-open [start, end) within one day.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def servers_needed(intervals):
    best = 0
    for t in range(DAY):
        jobs = set()
        for job_id, s, e in intervals:
            if s <= t < e:
                jobs.add(job_id)
        best = max(best, len(jobs))
    return best
```

TIME	SPACE
O(T * m)	O(m)

WHY IT IS SLOW

Depends on the length of the day, O(T * m). Merge each job's own intervals first, then sort start and end events and keep a running count, O(m log m).

OPTIMAL SOLUTION · Merge each job's intervals, then sweep start and end events

TIME	SPACE
O(m log m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Some rare jobs run for several days without restart and get a dedicated server.

Remove those jobs from the daily intervals and add one server per such job: servers = (number of dedicated jobs) + (maximum overlap of the remaining jobs).

FOLLOW-UP 2 Jobs that cross midnight.

Truncate at the end of the day for each day's schedule (as the statement says). For a recurring daily job that wraps, split it into two pieces (the end of the day and the start of the day); the sweep then handles both on the same circular day.

FOLLOW-UP 3 Return which server each job runs on.

Sort merged intervals by start and keep a min-heap of (end time, server id): reuse the server that frees up first if it is free, otherwise open a new one. The number of servers opened equals the maximum overlap.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute servers needed for daily
# recurring jobs
#
# You operate a cluster of identical servers that run
# recurring **daily** jobs. For a single day, you are
# given a list of job execution intervals. Each interval
# has: - job_id (string or integer), - start_time
# (timestamp within that day), - end_time (timestamp
# within that day), with start_time < end_time. Interpret
# each interval as a half-open time range [start_time,
# end_time) during which that job must be running on some
# server **on that day**. Rules and constraints: - Each
# physical server can execute at most **one job at a
# time**. - However, if multiple intervals in the input
# correspond to the **same job_id**, they may overlap in
# time **on the same server** (they are considered parts
# or retries of the same job and do not consume extra
# capacity).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Some rare jobs run for several days without restart
# and get a dedicated server.
# 2. Jobs that cross midnight.
# 3. Return which server each job runs on.
#
# INTERVIEW NOTES
# The number of servers is the largest number of jobs
# running at the same time. First remove the twist:
# overlapping intervals of the same job are really one
# longer interval, so merge the intervals of each job_id.
# Then it is the plain maximum overlap: sort start and end
# events and keep a running count.
#
# BRUTE FORCE (say it first): For every time unit count
# the distinct jobs running
# For each integer time t collect the set of job ids with
# an interval covering t and take the largest set size.
# Correct and the oracle; it depends on the length of the
# day, not only on m. Time O(T * m). Space O(m).


import heapq
from collections import defaultdict
```

```
# BRUTE FORCE (say it first): For every time unit count
# the distinct jobs running | time O(T * m)
def servers_by_time(intervals):
```

```
    if not intervals:
        return 0
    lo, hi = min(s for _, s, _ in intervals), max(
        e for _, _, e in intervals
    )
    return max(
        len({j for j, s, e in intervals if s <= t < e})
        for t in range(lo, hi)
    )


# HELPERS for the base question
def merge_by_job(intervals):
    # intervals of the SAME job may overlap on one server,
    # so
    # merge them first: (job, start, end) -> disjoint
    # blocks
    by_job = defaultdict(list)
    for job, start, end in intervals:
        by_job[job].append((start, end))
    blocks = []
    for job, spans in by_job.items():
        cur = None
        for s, e in sorted(spans):
            if cur and s <= cur[1]:
                cur[1] = max(cur[1], e)
            else:
                if cur:
                    blocks.append((cur[0], cur[1], job))
                cur = [s, e]
        blocks.append((cur[0], cur[1], job))
    return blocks


# BASE QUESTION (optimal): Merge each job's intervals,
# then sweep start and end events
# Idea: Group the intervals by job_id, sort and merge each
# group's overlapping or touching intervals, then collect
# all merged intervals.
# FOLLOW-UP 1: Some rare jobs run for several days without
# restart and get a dedicated server.
# Answer: Remove those jobs from the daily intervals and
```

```
# add one server per such job: servers = (number of
# dedicated jobs) + (maximum overlap of the remaining
# jobs).
def servers_needed(intervals, dedicated=()):
    # FOLLOW-UP 1: rare multi-day jobs get their own
    # server
    # dedicated jobs (they span several days) get their
    # own
    # server; the rest share servers: the answer is the
    # largest number of blocks running at the same moment
    dedicated = set(dedicated)
    shared = [i for i in intervals
              if i[0] not in dedicated]
    events = []
    for start, end, _ in merge_by_job(shared):
        events.append((start, 1))
        # [start, end): at equal times an end comes first
        events.append((end, -1))
    run = best = 0
    for _, delta in sorted(events):
        run += delta
        best = max(best, run)
    used = {i[0] for i in intervals if i[0] in dedicated}
    return best + len(used)


# FOLLOW-UP 2: Jobs that cross midnight.
# Answer: Truncate at the end of the day for each day's
# schedule (as the statement says).
def split_over_midnight(start, end, day=24 * 60):
    # a daily job that wraps past midnight: two pieces
    if start < end:
        return [(start, end)]
    return [(start, day), (0, end)]


# FOLLOW-UP 3: Return which server each job runs on.
# Answer: Sort merged intervals by start and keep a min-
# heap of (end time, server id): reuse the server that
# frees up first if it is free, otherwise open a new one.
# The number of servers opened equals the maximum overlap.
def assign_servers(intervals):
    # which server runs each block of each job?
    heap, plan, opened = [], [], 0
    blocks = sorted(merge_by_job(intervals))
```

```
    for start, end, job in blocks:
        if heap and heap[0][0] <= start:
            _, server = heapq.heappop(heap)
        else:
            server, opened = opened, opened + 1
        plan.append((job, server))
        heapq.heappush(heap, (end, server))
    return plan, opened


if __name__ == "__main__":
    jobs = [("a", 0, 10), ("a", 5, 12), ("b", 3, 8),
            ("c", 10, 15)]
    # job a is one block [0, 12); b overlaps it; c starts
    # at 10
    assert servers_needed(jobs) == 2
    # touching half-open intervals share a server
    assert servers_needed([("a", 0, 5), ("b", 5, 9)]) == 1
    assert servers_needed([]) == 0
    # job b runs for days: it gets a dedicated server
    assert servers_needed(jobs, dedicated=["b"]) == 3
    assert servers_needed(jobs, dedicated=["a", "b"]) == 3
    plan, count = assign_servers(jobs)
    assert count == 2 and len(plan) == 3
    assert split_over_midnight(1380, 60) == [(1380, 1440),
                                             (0, 60)]
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([])
```

Test cases

Input	Expected
`([],)`	`0`
`([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)`	`1`

ALSO REPORTED · 1 more

Compute minimum servers for cyclic tasks | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-servers-for-cyclic-tasks

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

MEDIUM | HackTheRounds | Phone Screen | 19 of 33

★ TOP 75: rank 59 in the Medium and Hard priority list

★ Meeting Rooms III Variant

STUDY: TYPE | **ALL TIME 58** | **30 DAYS 0** | **PRIORITY 39/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 (cyclic variant)

QUESTION

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

EXAMPLES

Example 1	
Input	`intervals = [[0, 30], [5, 10], [15, 20]]`
Output	`2`
Why	`Meeting [0,30] overlaps with [5,10] and [15,20], so we need 2 rooms.`

Example 2	
Input	`intervals = [[7, 10], [2, 4]]`
Output	`1`
Why	`No overlap, 1 room is sufficient.`

Example 3	
Input	`intervals = [[22, 2], [23, 1], [0, 3]]`
Output	`3`
Why	`All three meetings overlap around midnight.`

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 0 <= start < end <= 24 * 60 (minutes in a day)
- Meetings may wrap around (e.g., [23:00, 01:00])

APPROACH

Sweep line; split intervals that wrap past midnight

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the 24-hour day repeats cyclically, whether a meeting that wraps midnight must stay in one room the whole time, and whether a meeting ending exactly when another starts conflicts (usually not).

BRUTE FORCE TO SAY OUT LOUD

Try assigning meetings to rooms one by one with backtracking. Exponential.

OPTIMAL IDEA

Split a wrapping meeting [s, e] with e < s into [s, 24h] and [0, e] and run a sweep line over the events: sort start and end events, +1 at a start, -1 at an end (process ends first at equal times), and take the maximum running count. That gives the minimum number of rooms when pieces may use different rooms. If a wrapping meeting must stay in a single room the exact answer is circular-arc coloring, which can be larger than the maximum overlap (five arcs forming an odd cycle need three rooms with overlap two) and is hard in general, so state the assumption and mention a heuristic.

DRY-RUN THESE CASES

No wrap, one wrapping meeting, a wrap that overlaps the beginning of the day, back-to-back meetings, all meetings the same.

LIKELY FOLLOW-UPS

Return which room each meeting uses, or check feasibility with a given number of rooms.

PITFALLS

Counting back-to-back meetings as overlapping. Splitting a wrap but treating the two pieces as two meetings in the output.

HOW TO THINK ABOUT IT

Split a wrapped meeting into two pieces (from start to the end of the day, and from midnight to its end) and run the normal sweep line: the largest number of meetings active at once is the number of rooms needed. That is exact when pieces may sit in different rooms; if a meeting must keep one room, the arcs form a circular-arc graph and the true minimum can exceed the maximum overlap (five overlapping arcs around a circle need 3 rooms with overlap 2). Say which reading you assume.

Basis: LeetCode 253 (Meeting Rooms II) on a circular day. Meetings are half-open [start, end) in minutes; a meeting with end < start wraps past midnight. Two readings: the usual interview answer (split the wrapped meeting and take the maximum overlap) and the exact answer when a wrapped meeting must keep one room.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def rooms_needed(meetings):
    slots = [minutes_used(m) for m in meetings]
    for k in range(1, len(meetings) + 1):
        rooms = [set() for _ in range(k)]
        if assign(0, rooms, slots):
            return k
def assign(i, rooms, slots):
    if i == len(slots):
        return True
    for room in rooms:
        busy = slots[i]
        if not room & busy:  # no shared minute
            room |= busy
            if assign(i + 1, rooms, slots):
                return True
            room -= busy
    return False
```

TIME	SPACE
exponential	O(n)

WHY IT IS SLOW

Backtracking over room assignments is exponential. Splitting wrapped meetings and sweeping start and end events gives the maximum overlap in O(n log n); that equals the exact answer when meetings do not wrap.

OPTIMAL SOLUTION · Split wrapped meetings and sweep (maximum overlap)

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A wrapped meeting must stay in a single room.

Then it is circular-arc colouring, NP-hard in general. A common practical method: cut the circle at the minute with the fewest active meetings; the meetings crossing the cut use fixed rooms, and the rest are scheduled greedily with a heap. It can use more rooms than the maximum overlap, and the five-arc example shows why.

FOLLOW-UP 2 Return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room id); reuse the earliest-ending room if it is free, otherwise open a new one. For wrapped meetings schedule them first so their rooms are fixed.

FOLLOW-UP 3 Days can repeat (a wrapped meeting recurs every day).

A recurring meeting occupies its slots every day, so the circle model above is exactly right, and the no-wrap shortcut of ordering by start time is not.

FOLLOW-UP 4 Return which room each meeting uses.

Sort by start and keep a heap of (end time, room): reuse the earliest-ending room if it is free, otherwise open a new one. For meetings that do not wrap this uses exactly the minimum number of rooms (verified against the sweep); wrapped meetings need the cut-the-circle method.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Meeting Rooms III Variant
#
# Given an array of meeting time intervals where each
# interval is [start, end], determine the minimum number
# of conference rooms required to hold all meetings
# without overlap. Variant: times are cyclic within a
# 24-hour period (meetings can wrap around midnight).
#
# FOLLOW-UPS (answered below, in this order):
# 1. A wrapped meeting must stay in a single room.
# 2. Return which room each meeting uses.
# 3. Days can repeat (a wrapped meeting recurs every day).
# 4. Return which room each meeting uses.
#
# INTERVIEW NOTES
# Split a wrapped meeting into two pieces (from start to
# the end of the day, and from midnight to its end) and
# run the normal sweep line: the largest number of
# meetings active at once is the number of rooms needed.
# That is exact when pieces may sit in different rooms; if
# a meeting must keep one room, the arcs form a circular-
# arc graph and the true minimum can exceed the maximum
# overlap (five overlapping arcs around a circle need 3
# rooms with overlap 2). Say which reading you assume.
#
# BRUTE FORCE (say it first): Exact minimum with one room
# per meeting, by backtracking
# Treat each meeting as the set of minute slots it
# occupies (a wrapped meeting is two ranges of slots) and
# try to colour the meetings with k rooms for k = 1, 2,
# ...; two meetings conflict when their slot sets
# intersect. Exact for 'a meeting keeps its room', and the
# oracle for small cases; it is exponential. Time
# exponential. Space O(n).


import heapq


# HELPERS for the brute force
DAY = 24


def hours(meeting, day=DAY):
    # the hours a meeting occupies on the 24 hour circle
```

```
    s, e = meeting
    length = (e - s) % day or day
    return {(s + k) % day for k in range(length)}


def brute_min_rooms(meetings, day=DAY):
    return max((sum(1 for m in meetings
                    if h in hours(m, day))
                for h in range(day)), default=0)


# HELPERS for the base question
def pieces(meetings, day=DAY):
    # a meeting whose end is before its start wraps past
    # midnight: split it into the part before midnight and
    # the part after it
    for s, e in meetings:
        if e < s:
            yield s, day
            yield 0, e
        else:
            yield s, e


# BASE QUESTION (optimal): Split wrapped meetings and
# sweep (maximum overlap)
# Idea: Turn every meeting into one or two ordinary
# intervals, create +1 events at starts and -1 events at
# ends, sort them (ends before starts at the same minute,
# because intervals are half-open), and take the maximum
# running count.
def min_rooms_sweep(meetings, day=DAY):
    # sweep the time line: +1 at a start, -1 at an end. At
    # equal times an end comes first (a meeting ending at
    # 5
    # and one starting at 5 can share a room).
    events = []
    for s, e in pieces(meetings, day):
        events.append((s, 1))
        events.append((e, -1))
    events.sort()
    cur = best = 0
    for _, step in events:
        cur += step
        best = max(best, cur)
```

```
    return best


# FOLLOW-UP 1: A wrapped meeting must stay in a single
# room.
# Answer: Then it is circular-arc colouring, NP-hard in
# general.
def assign_single_rooms(meetings, day=DAY):
    # follow-up: a meeting that wraps must stay in ONE
    # room.
    # That is circular-arc colouring (NP-hard), so use the
    # practical method: cut the circle at the hour where
    # the
    # fewest meetings are running, give those meetings
    # their
    # own rooms, and place the rest greedily.
    busy = [sum(1 for m in meetings
                if cut in hours(m, day))
            for cut in range(day)]
    cut = busy.index(min(busy))
    # rotate so that the cut is time 0: (start, end) on a
    # line
    line = []
    for s, e in meetings:
        start = (s - cut) % day
        length = (e - s) % day or day
        line.append((start, start + length))
    free_at, deadline = [], []     # per room
```

```
    room_of = {}
    for i, (s, e) in enumerate(line):
        if e > day:
            # crosses the cut: busy until e - day, and
            # again
            # from s on, so the room is blocked after s
            room_of[i] = len(free_at)
            free_at.append(e - day)
            deadline.append(s)
    rest = [i for i in range(len(line))
            if i not in room_of]
    others = sorted(rest, key=lambda i: line[i])
    for i in others:
        s, e = line[i]
        # the room that has been free the longest and fits
        fits = [r for r in range(len(free_at))
                if free_at[r] <= s and e <= deadline[r]]
```

```
        if fits:
            room = max(fits, key=lambda r: free_at[r])
        else:
            room = len(free_at)
            free_at.append(0)
            deadline.append(day)
        room_of[i] = room
        free_at[room] = e
    return [room_of[i] for i in range(len(meetings))]


# FOLLOW-UP 2: Return which room each meeting uses.
# Answer: Sort by start and keep a min-heap of (end time,
# room id); reuse the earliest-ending room if it is free,
# otherwise open a new one. For wrapped meetings schedule
# them first so their rooms are fixed.
# FOLLOW-UP 4: Return which room each meeting uses.
# Answer: Sort by start and keep a heap of (end time,
# room): reuse the earliest-ending room if it is free,
# otherwise open a new one.
def assign_rooms(meetings):
    # follow-up: which room for each meeting (no
    # wrapping).
    # Sort by start; a heap of (end, room) gives the room
    # that
    # frees up first; reuse it if it is free, else open
    # one.
    order = sorted(range(len(meetings)),
                   key=lambda i: meetings[i][0])
    heap, room_of, rooms = [], {}, 0
    for i in order:
        s, e = meetings[i]
        if heap and heap[0][0] <= s:
            _, room = heapq.heappop(heap)
        else:
            room, rooms = rooms, rooms + 1
        room_of[i] = room
        heapq.heappush(heap, (e, room))
    return [room_of[i] for i in range(len(meetings))]


# HELPERS for the checks below
def valid_assignment(meetings, rooms, day=DAY):
    # check that no room holds two meetings at the same
    # hour
```

```
    used = set()
    for m, room in zip(meetings, rooms):
        for h in hours(m, day):
            if (room, h) in used:
                return False
            used.add((room, h))
    return True
```

```
if __name__ == "__main__":
    long = [[0, 30], [5, 10], [15, 20]]
    assert min_rooms_sweep(long) == 2
    assert min_rooms_sweep([[7, 10], [2, 4]]) == 1
    night = [[22, 2], [23, 1], [0, 3]]
    assert min_rooms_sweep(night) == 3
    assert min_rooms_sweep([]) == 0
    # one ends exactly when the next starts: same room
    assert min_rooms_sweep([[1, 5], [5, 9]]) == 1
    flat = [[0, 30], [5, 10], [15, 20]]
    rooms = assign_rooms(flat)
    assert rooms == [0, 1, 1]
    assert len(set(rooms)) == min_rooms_sweep(flat)
    two = [(22, 2), (1, 5)]
    assert valid_assignment(two, assign_single_rooms(two))
    import random
    for _ in range(400):
        ms = []
        for _ in range(random.randint(0, 7)):
            s, e = random.sample(range(DAY), 2)
            ms.append((s, e))
        assert min_rooms_sweep(ms) == brute_min_rooms(ms)
        rooms = assign_single_rooms(ms)
        assert valid_assignment(ms, rooms)
        assert len(set(rooms)) >= brute_min_rooms(ms)
        if all(s < e for s, e in ms):
            plain = assign_rooms(ms)
            assert valid_assignment(ms, plain)
            assert len(set(plain)) == brute_min_rooms(ms)
```

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

HARD | PracHub | Technical Screen | 20 of 33

Compute Turnstile Crossing Times

POSTED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 176/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: the interview site's own solution

LeetCode match: LeetCode 'Time Taken to Cross the Door' (exact number unverified) (inferred from overview)

Posted on PracHub: 2026-05-04 (updated 2026-05-10)

Original: https://prachub.com/coding-questions/compute-turnstile-crossing-times

QUESTION

Overview: This question evaluates proficiency in queue simulation, event-driven scheduling, and handling priority and tie-breaking rules while achieving linear time complexity.

APPROACH

Two queues (entering, exiting) ordered by arrival; simulate time with the direction-priority and tie-break rules, jumping the clock when idle. O(n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask: how many people can pass per second, which direction goes first when both are waiting, whether the rule depends on what happened in the previous second, and how ties on arrival time break (by id). Clarify what to return (time per person).

BRUTE FORCE TO SAY OUT LOUD

Simulate every second from the earliest to the latest time, checking both queues. Correct but its cost grows with the size of the timestamps, not the number of people.

OPTIMAL IDEA

Keep two FIFO queues, enter and exit, and a variable for what the turnstile did in the previous second. Each step apply the priority rule to pick a queue. When both queues are empty, jump the clock to the next arrival and reset the previous-state to idle if at least one second was skipped. That makes the simulation O(n) after sorting.

DRY-RUN THESE CASES

Two people arriving at the same second with opposite directions. A long idle gap followed by a simultaneous pair. All enters, then all exits. A person who arrives while the turnstile was just used by the other direction.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Ask what happens when the idle jump leaves exactly one empty second versus more. Count that the 'previous second' state can be none, not only enter or exit.

PITFALLS

Resetting state only when the gap is two or more seconds. Forgetting to push arrivals that happened during the jump. Comparing queues by id instead of arrival order.

HOW TO THINK ABOUT IT

A scheduling rule that looks at 'what happened last second' is a state machine: write the state down (previous direction: enter, exit, none) before coding. If time values can be huge, simulate events, not seconds: jump the clock to the next arrival when both queues are empty, and remember an idle gap resets the state.

Basis: LeetCode 2534, Time Taken to Cross the Door. Assumed rules (confirm them): one person per second; if an enter and an exit compete, the previous second decides (enter if it was an enter, else exit, and exit if the door was idle).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arrivals):
    t, prev = 0, None
    enter_q, exit_q = deque(), deque()
    while pending(arrivals, enter_q, exit_q):
        add_arrivals(t, arrivals, enter_q,
                     exit_q)
        who = pick_by_priority(prev, enter_q,
                               exit_q)
        if who is not None:
            passed[who] = t
            prev = direction_of(who)
        t += 1
    return passed
```

TIME	SPACE
O(T + n)	O(n)
where T is the largest timestamp	

WHY IT IS SLOW

A timestamp of 10^9 means a billion iterations even if nothing happens. Jumping to the next arrival when both queues are empty is O(n log n).

OPTIMAL SOLUTION · Event-driven simulation with two queues

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The priority when the turnstile was idle is different in this version of the question.

Keep the same loop and make the idle-state winner a parameter. Only the line that picks the queue changes, which is why the state machine should be written first.

FOLLOW-UP 2 Timestamps are up to 10^9.

That is exactly why the simulation jumps between events: the loop runs once per person, so the answer is O(n log n) no matter how large the times are.

FOLLOW-UP 3 Two turnstiles or a capacity of two people per second.

Run the same pick-a-queue step twice per tick (or per turnstile) and carry a previous-state variable per turnstile. Decide whether the two turnstiles share the queues.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute Turnstile Crossing Times
#
# This question evaluates proficiency in queue simulation,
# event-driven scheduling, and handling priority and tie-
# breaking rules while achieving linear time complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority when the turnstile was idle is different
# in this version of the question.
# 2. Timestamps are up to 10^9.
# 3. Two turnstiles or a capacity of two people per
# second.
#
# INTERVIEW NOTES
# A scheduling rule that looks at 'what happened last
# second' is a state machine: write the state down
# (previous direction: enter, exit, none) before coding.
# If time values can be huge, simulate events, not
# seconds: jump the clock to the next arrival when both
# queues are empty, and remember an idle gap resets the
# state.
#
# BRUTE FORCE (say it first): Simulate every second
# Walk t = 0, 1, 2, ... adding arrivals to two FIFO queues
# and letting one person through per second by the rule.
# It is easy to verify, but it spends time on every empty
# second, so a timestamp of 10^9 is hopeless. Time O(T +
# n) where T is the largest timestamp. Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Simulate every second | time
# O(T + n) where T is the largest timestamp
def turnstile_brute(events):
    n = len(events)
    order = sorted(
        range(n), key=lambda i: (events[i][0], i)
    )
    res, ent, ext, prev, t, k = (
        [None] * n,
        deque(),
        deque(),
        None,
        0,
        0,
    )
    while any(r is None for r in res):
        while k < n and events[order[k]][0] <= t:
            (
                ent
                if events[order[k]][1] == "enter"
                else ext
            ).append(order[k])
            k += 1
        if ent and ext:
            q = ent if prev == "enter" else ext
        else:
            q = ent or ext or None
        if q is None:
            prev = None
        else:
            p = q.popleft()
            res[p], prev = t, events[p][1]
        t += 1
    return res


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort arrivals once. When both queues are empty,
# jump straight to the next arrival time; if the jump
# leaves at least one idle second, reset the previous
# state to 'none'.
# FOLLOW-UP 1: The priority when the turnstile was idle is
# different in this version of the question.
# Answer: Keep the same loop and make the idle-state
# winner a parameter. Only the line that picks the queue
# changes, which is why the state machine should be
# written first.
def turnstile_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit");
    # the result is the second each person goes through
    n = len(events)
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            # nobody is waiting: jump to the next arrival
            t = max(t, events[order[i]][0])
            # an idle second forgets the last direction
            if t > last_used + 1:
                prev = None
        # everyone who has arrived by now joins a line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            lines[events[person][1]].append(person)
            i += 1
        if lines["enter"] and lines["exit"]:
            # FOLLOW-UP 1: idle tie-break set by
            # idle_first
            # both lines: keep the direction used last
            # second,
            # and use idle_first after an idle gap
            want = prev or idle_first
        else:
            want = "enter" if lines["enter"] else "exit"
```

```
        person = lines[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result


if __name__ == "__main__":
    both = [(0, "enter"), (0, "exit")]
    # idle tie: the exit goes first
    assert turnstile_times(both) == [1, 0]
    # a version where enter wins the idle tie
    assert turnstile_times(both, "enter") == [0, 1]
    # the direction in use keeps going
    keep = [(0, "enter"), (0, "enter"), (1, "exit")]
    assert turnstile_times(keep) == [0, 1, 2]
    # an idle second resets the state
    gap = [(0, "enter"), (10, "enter"), (10, "exit")]
    assert turnstile_times(gap) == [0, 11, 10]
    # huge timestamps are no problem: one loop per person
    far = [(10 ** 9, "enter"), (10 ** 9, "exit")]
    assert turnstile_times(far) == [10 ** 9 + 1, 10 ** 9]
    assert turnstile_times([]) == []
```

FUNCTION AND TEST CASES

Signature

```
def solution(time: list[int], direction: list[int]) -> lis
t[int]:
SAMPLE INPUT
[[0,0,1,5],[0,1,1,0]]
```

Test cases

Input	Expected
[[0, 0, 1, 5], [0, 1, 1, 0]]	[2, 0, 1, 5]
[[0, 0, 0], [0, 1, 0]]	[1, 0, 2]

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

HARD | PracHub | Technical Screen | 21 of 33

Minimize travel time with optional meeting point

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 59/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #2203 Minimum Weighted Subgraph With the Required Paths

LeetCode match: LC 743 Network Delay Time (multi-source Dijkstra); related LC 2203 (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-22)

Original: https://prachub.com/coding-questions/minimize-travel-time-with-optional-meeting-point

QUESTION

Overview: This question evaluates a candidate's understanding of graph shortest-path computation and meeting-point optimization, testing skills in handling weighted edges, aggregating path costs, and reasoning about simultaneous movements on a network.

APPROACH

Run Dijkstra from each traveler's start, then pick the node that minimizes the total (or max, per the statement) travel time; compare against the no-meeting option.

INTERVIEW-DAY PLAN

CLARIFY FIRST

From the scraped solution the setup is two travelers A and B and a destination. Confirm: do they have to travel together after meeting, do they wait for each other (the meeting time is the later arrival), is the graph directed or undirected, and what to return (the minimum time and the meeting node, with ties going to the smallest node, or -1 and -1 if impossible). My copy has only the overview, so verify these.

BRUTE FORCE TO SAY OUT LOUD

Try every node as a meeting point with a fresh shortest path computation each time. O(n * (E log V)).

OPTIMAL IDEA

Run Dijkstra from A, Dijkstra from B, and Dijkstra from the destination on the reversed graph (so you get every node's distance to the destination in one pass). For each node m the time is max(distA[m], distB[m]) + distToDest[m]. Take the minimum, with ties to the smallest node. Compare with the no-meeting option if each can also go directly. O((V + E) log V).

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

A and B start at the same node, a node that only one of them can reach, a directed graph where the way back is longer, two nodes with equal time.

LIKELY FOLLOW-UPS

More travelers (Dijkstra from each, take the max), minimize total distance instead of arrival, or an optional meeting point.

PITFALLS

Forgetting to reverse the graph for the destination distances in a directed graph. Using sum instead of max for waiting.

HOW TO THINK ABOUT IT

Break it into shortest paths from three places: from A, from B, and into the destination (a Dijkstra on the reversed graph if directed). For each node m the cost is max(distA[m], distB[m]) + dist(m to destination); take the minimum. Three single-source searches replace trying every meeting node with its own search.

Basis: Written from the scraped solution (confirm it): two travellers A and B, a destination; they may meet at a node (the meeting happens when the later one arrives) and then travel together to the destination; return [best time, best node] with ties to the smallest node, or [-1, -1].

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, A, B, dest):
    d = floyd_warshall(n, edges)  # all pairs
    best, best_m = INF, None
    for m in range(n):
        cost = max(d[A][m], d[B][m]) + d[m][dest]
        if cost < best:
            best, best_m = cost, m
    return best
```

TIME	SPACE
O(V^3)	O(V^2)

WHY IT IS SLOW

All-pairs distances cost O(V^3) when only three single-source distances are needed. Dijkstra from A, from B and into the destination is O((V + E) log V).

OPTIMAL SOLUTION · Three Dijkstra runs (from A, from B, and into the destination)

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 More than two travellers.

Run one Dijkstra per traveller and one from the destination, then for each node take the maximum arrival time (if they wait for each other) or the sum of distances (if the cost is total distance) plus the shared leg. K + 1 searches.

FOLLOW-UP 2 Meeting is optional.

Also compute the cost with no meeting (each goes straight to the destination: max(distA[dest], distB[dest])) and take the smaller. Meeting at the destination itself already covers it when the cost is the later arrival.

FOLLOW-UP 3 Edge weights change over time (traffic).

Dijkstra still works if the graph is FIFO (leaving later never arrives earlier); each edge's cost becomes a function of the departure time, evaluated when you relax it.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimize travel time with optional
# meeting point
#
# This question evaluates a candidate's understanding of
# graph shortest-path computation and meeting-point
# optimization, testing skills in handling weighted edges,
# aggregating path costs, and reasoning about simultaneous
# movements on a network.
#
# FOLLOW-UPS (answered below, in this order):
# 1. More than two travellers.
# 2. Meeting is optional.
# 3. Edge weights change over time (traffic).
#
# INTERVIEW NOTES
# Break it into shortest paths from three places: from A,
# from B, and into the destination (a Dijkstra on the
# reversed graph if directed). For each node m the cost is
# max(distA[m], distB[m]) + dist(m to destination); take
# the minimum. Three single-source searches replace trying
# every meeting node with its own search.
#
# BRUTE FORCE (say it first): Floyd-Warshall all-pairs,
# then try every meeting node
# Compute the shortest distance between every pair of
# nodes, then for each node m take max(d[A][m], d[B][m]) +
# d[m][dest]. Simple and the oracle; cubic in the number
# of nodes. Time O(V^3). Space O(V^2).


import heapq


# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Floyd-Warshall all-pairs,
# then try every meeting node | time O(V^3)
def meeting_point_floyd(n, edges, a, b, dest, directed):
    INF = float("inf")
    d = [
        [0 if i == j else INF for j in range(n)]
        for i in range(n)
    ]
    for u, v, w in edges:
        d[u][v] = min(d[u][v], w)
        if not directed:
            d[v][u] = min(d[v][u], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    best = (INF, -1)
    for m in range(n):
        t = max(d[a][m], d[b][m]) + d[m][dest]
        if t < best[0]:
            best = (t, m)
    return (
        [-1, -1]
        if best[0] == INF
        else [best[0], best[1]]
    )


# HELPERS for the base question
def dijkstra(graph, source):
    dist = [INF] * len(graph)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist


def build(n, edges, directed):
    # forward graph, and the reversed graph for "to dest"
    fwd = [[] for _ in range(n)]
    rev = [[] for _ in range(n)]
    for u, v, w in edges:
        fwd[u].append((v, w))
        rev[v].append((u, w))
        if not directed:
            fwd[v].append((u, w))
            rev[u].append((v, w))
    return fwd, rev


# BASE QUESTION (optimal): Three Dijkstra runs (from A,
# from B, and into the destination)
# Idea: distA and distB come from forward searches. The
# distance from every node to the destination is a search
# from the destination on the reversed graph (the same
# graph if undirected).
def meeting_point(n, edges, a, b, dest, directed=False):
    # both start at the same time, meet at node m, wait
    # for
    # each other, then travel together to dest. Meeting at
    # dest itself covers the "no meeting" plan.
    fwd, rev = build(n, edges, directed)
    da, db = dijkstra(fwd, a), dijkstra(fwd, b)
    # distance from every node TO dest = search the
    # reverse graph
    to_dest = dijkstra(rev, dest)
```

```
    best = (INF, -1)
    for m in range(n):
        total = max(da[m], db[m]) + to_dest[m]
        if total < best[0]:
            best = (total, m)
    return [-1, -1] if best[0] == INF else list(best)


# FOLLOW-UP 1: More than two travellers.
# Answer: Run one Dijkstra per traveller and one from the
# destination, then for each node take the maximum arrival
# time (if they wait for each other) or the sum of
# distances (if the cost is total distance) plus the
# shared leg.
def meeting_point_many(n, edges, starts, dest,
                       directed=False):
    # k travellers: one search each, plus one from dest
    fwd, rev = build(n, edges, directed)
    dists = [dijkstra(fwd, s) for s in starts]
    to_dest = dijkstra(rev, dest)
    best = (INF, -1)
    for m in range(n):
        wait = max(d[m] for d in dists)
        if wait + to_dest[m] < best[0]:
```

```
            best = (wait + to_dest[m], m)
    return [-1, -1] if best[0] == INF else list(best)


if __name__ == "__main__":
    # a star: both travellers reach the hub 2, then go to
    # 3
    edges = [(0, 2, 1), (1, 2, 2), (2, 3, 4), (0, 3, 9),
             (1, 3, 9)]
    # meet at 2: max(1, 2) + 4 = 6; going alone costs 9
    assert meeting_point(4, edges, 0, 1, 3) == [6, 2]
    # nobody can reach the destination
    short = [(0, 1, 1)]
    assert meeting_point(3, short, 0, 1, 2) == [-1, -1]
    # same start: they simply walk together
    assert meeting_point(4, edges, 0, 0, 3)[0] == 5
    # three travellers
    got = meeting_point_many(4, edges, [0, 1, 1], 3)
    assert got == [6, 2]
    # directed roads: reverse edges are not usable
    one_way = [(0, 1, 1), (1, 2, 1)]
    walk = meeting_point(3, one_way, 0, 1, 2, True)
    assert walk == [2, 1]
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, edges: list[list[int]], startA: int,
startB: int, dest: int, directed: bool = False) -> list[in
t]:
```

Sample call

```
solution(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)],
0, 2, 3, False)
```

Test cases

Input	Expected
(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)], 0, 2, 3, False)	[4, 1]
(4, [(0, 1, 2), (2, 1, 3), (1, 3, 2), (0, 3, 10), (2, 3, 5)], 0, 2, 3, True)	[5, 1]

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Intervals and scheduling

HARD | IN APP LC 218 | PracHub | Technical Screen | 22 of 33

Compute city skyline outline

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 125/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Intervals and scheduling

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #218 The Skyline Problem

LeetCode match: LC 218 The Skyline Problem

Posted on PracHub: 2025-12-08 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/compute-city-skyline-outline

QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers `[L, R, H]`:

- `L`: the x-coordinate of the left edge of the building,
- `R`: the x-coordinate of the right edge of the building (`L < R`),
- `H`: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height `H`. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points** `[x, height]` that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing `x`.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

Input

- An integer **n** — the number of buildings.
- A list of **n** buildings, where each building is given as a triple `[L, R, H]` of integers. You may assume:
- `1 <= n <= 10^4`
- `0 <= L < R <= 10^9`
- `1 <= H <= 10^9`

Output

Return a list of key points representing the skyline. Each key point is a pair `[x, height]` (or a 2-element array/tuple) of integers.

Example

Input:

- `n = 3` - buildings = `[[2, 9, 10], [3, 7, 15], [5, 12, 12]]` One valid output:

QUESTION

- `[[2, 10], [3, 15], [7, 12], [12, 0]]` Explanation (informal):
- From x = 2 to 3, the highest building has height 10.
- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

APPROACH

Sweep line over building edges with a max-heap of active heights (lazy deletion); emit a point whenever the max height changes. O(n log n). Divide and conquer also works.

HOW TO THINK ABOUT IT

The skyline height at x is the tallest building covering x, and it can only change at a building's left or right edge. So sweep over those x positions in order, keep the active buildings in a max-heap by height, and report a key point whenever the maximum changes. The details that matter are the event order at the same x and removing finished buildings lazily.

Basis: LeetCode 218, The Skyline Problem.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(buildings):
    xs = set()
    for L, R, H in buildings:
        xs |= {L, R}
    out, prev = [], 0
    for x in sorted(xs):
        h = 0
        for L, R, H in buildings:
            if L <= x < R:
                h = max(h, H)
        if h != prev:
            out.append([x, h])
            prev = h
    return out
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each edge scans all buildings, O(n^2). A sweep with a max-heap of active buildings and lazy deletion is O(n log n).

OPTIMAL SOLUTION · Sweep line with a max-heap and lazy deletion

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Divide and conquer instead of a heap.

Build the skyline of each half and merge the two by sweeping both lists with the current height of each; the merged height is the max of the two. T(n) = 2T(n/2) + O(n) = O(n log n), same as the heap.

FOLLOW-UP 2 Coordinates up to 10^9 or buildings of a very large count.

Only the edge coordinates matter, so nothing depends on the coordinate range. The sweep is O(n log n) in the number of buildings; use a balanced tree or a heap with lazy deletion for the active set.

FOLLOW-UP 3 Output the area under the skyline.

Sum height * (next x - x) over consecutive key points: one extra pass over the output.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute city skyline outline
#
# You are given a list of rectangular buildings in a 2D
# city skyline. Each building is represented by three
# integers [L, R, H]: - L: the x-coordinate of the left
# edge of the building, - R: the x-coordinate of the right
# edge of the building (L < R), - H: the height of the
# building. All buildings stand on a flat horizontal line
# (y = 0) and extend vertically up to height H. Buildings
# may overlap in the x-dimension. Your task is to compute
# the *skyline* formed by these buildings, as seen from a
# distance. The skyline should be represented as a list of
# **key points** [x, height] that describe the outer
# contour of the union of all buildings. A **key point**
# is a point where the height of the skyline changes. The
# output list must satisfy all of the following: 1. The
# list is sorted by increasing x. 2. No two consecutive
# points have the same height (i.e., the height must
# change at each key point). 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Divide and conquer instead of a heap.
# 2. Coordinates up to 10^9 or buildings of a very large
# count.
# 3. Output the area under the skyline.
#
# INTERVIEW NOTES
# The skyline height at x is the tallest building covering
# x, and it can only change at a building's left or right
# edge. So sweep over those x positions in order, keep the
# active buildings in a max-heap by height, and report a
# key point whenever the maximum changes. The details that
# matter are the event order at the same x and removing
# finished buildings lazily.
#
# BRUTE FORCE (say it first): Compute the height at every
# edge coordinate
# Collect all edge x-coordinates in order. At each one the
# height is the maximum height among buildings with L <= x
# < R. Output a point where the height differs from the
# previous one. Correct and the oracle; each coordinate
# scans all buildings. Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Compute the height at every
# edge coordinate | time O(n^2)
def skyline_brute(buildings):
    xs = sorted(
        {x for l, r, h in buildings for x in (l, r)}
    )
    out, prev = [], 0
    for x in xs:
        h = max(
            (
                hh
                for l, r, hh in buildings
                if l <= x < r
            ),
            default=0,
        )
        if h != prev:
            out.append([x, h])
            prev = h
    return out


# HELPERS for follow-up 1
def merge(a, b):
    # divide and conquer: merge two skylines with the
    # current
    # height of each; the result height is their maximum
    out, i, j, ha, hb = [], 0, 0, 0, 0
    while i < len(a) or j < len(b):
        take_a = i < len(a) and (j == len(b)
                                 or a[i][0] <= b[j][0])
        if take_a:
            x, ha = a[i]
            i += 1
        else:
            x, hb = b[j]
            j += 1
        # when both change at the same x, wait for the
        # second
        if i < len(a) and a[i][0] == x:
            continue
        if j < len(b) and b[j][0] == x:
            continue
        top = max(ha, hb)
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out


# BASE QUESTION (optimal): Sweep line with a max-heap and
# lazy deletion
# Idea: Make events (x, -height) for left edges and (x, 0,
# right) for right edges, sorted so that at the same x
```

```
# taller starts come first and ends come after starts.
# Keep a heap of (-height, right edge).
def skyline(buildings):
    # buildings are [left, right, height]
    # start events carry -height so taller starts sort
    # first;
    # end events carry 0
    events = sorted([(l, -h, r) for l, r, h in buildings]
                    + [(b[1], 0, 0) for b in buildings])
    out = []
    # the active buildings: (-height, right edge), ground
    # first
    heap = [(0, float("inf"))]
    for x, neg_h, right in events:
        # buildings that ended at or before x are gone
        while heap[0][1] <= x:
            heapq.heappop(heap)
        if neg_h:
            heapq.heappush(heap, (neg_h, right))
        top = -heap[0][0]
        # a key point only where the height changes
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out


# FOLLOW-UP 1: Divide and conquer instead of a heap.
# Answer: Build the skyline of each half and merge the two
# by sweeping both lists with the current height of each;
# the merged height is the max of the two. T(n) = 2T(n/2)
# + O(n) = O(n log n), same as the heap.
def skyline_dc(buildings):
    if len(buildings) == 1:
        l, r, h = buildings[0]
        return [[l, h], [r, 0]]
```

```
    if not buildings:
        return []
    mid = len(buildings) // 2
    return merge(skyline_dc(buildings[:mid]),
                 skyline_dc(buildings[mid:]))


# FOLLOW-UP 3: Output the area under the skyline.
# Answer: Sum height * (next x - x) over consecutive key
# points: one extra pass over the output.
def area_under(points):
    # sum of height * width between consecutive key points
    return sum(h * (points[i + 1][0] - x)
               for i, (x, h) in enumerate(points[:-1]))


if __name__ == "__main__":
    b = [[2, 9, 10], [3, 7, 15], [5, 12, 12],
         [15, 20, 10], [19, 24, 8]]
    want = [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10],
            [20, 8], [24, 0]]
    assert skyline(b) == want
    assert skyline_dc(b) == want
    flat = skyline([[0, 2, 3], [2, 5, 3]])
    assert flat == [[0, 3], [5, 0]]
    assert skyline([]) == []
    # 10*1 + 15*4 + 12*5 + 10*5 + 8*4 = 212
    assert area_under(want) == 212
    assert skyline([[1, 2, 1], [1, 2, 2], [1, 2, 3]]) == [
        [1, 3], [2, 0]]
```

FUNCTION AND TEST CASES

Sample call

```
getSkyline(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])
```

Test cases

Input	Expected
(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])	[[2, 10], [3, 15], [7, 12], [12, 0]]
(1, [[0, 2, 3]])	[[0, 3], [2, 0]]

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Shortest paths with a heap

MEDIUM | IN APP LC 1102 | PracHub | Technical Screen | 23 of 33

Find A Threshold-Limited Path With Minimum Required Safety

POSTED 2026-07-06 | 106 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 136/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Shortest paths with a heap

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1102 Path With Maximum Minimum Value

LeetCode match: LC 1102 Path With Maximum Minimum Value (inferred from overview)

Posted on PracHub: 2026-07-06 (updated 2026-07-08)

Original: https://prachub.com/coding-questions/find-a-threshold-limited-path-with-minimum-required-safety

QUESTION

Overview: Practice a graph path interview problem that asks for the minimum required safety value along a threshold-limited path. The prompt targets minimax path reasoning, filtered edges, Dijkstra-style relaxation, disconnected graphs, and edge-case handling.

APPROACH

Minimax Dijkstra: heap keyed by the best bottleneck (min safety along the path) while skipping edges beyond the threshold; or binary search the safety value + BFS.

HOW TO THINK ABOUT IT

When the quality of a path is decided by its weakest edge, the best path does not need to be short, only strong. Two equivalent tools: add edges from strongest to weakest with Union-Find until source and target connect (that edge's value is the answer), or run Dijkstra with a max-heap where extending a path takes the minimum so far. Disconnected graphs must return the 'no path' value.

Basis: LeetCode 1102 (Path With Maximum Minimum Value) and the bottleneck path on a graph. Assumed form (confirm it): edges (u, v, safety); the safety of a path is its smallest edge; find the best achievable path safety from source to target, or -1 if disconnected.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(graph, src, dst):
    best = -INF
    for path in all_paths(graph, src, dst):
        weakest = min(edge_safety(path))
        best = max(best, weakest)
    return best
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential. Adding edges from strongest to weakest in a Union-Find until the source and target connect is O(E log E).

OPTIMAL SOLUTION · Union-Find adding edges from strongest to weakest

TIME	SPACE
O(E log E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use a max-heap search instead of Union-Find.

Dijkstra where the value of a path is the minimum edge so far and you maximise it: pop the best bottleneck, extend with min(value, edge). O(E log V); Union-Find is simpler when you need only the value.

FOLLOW-UP 2 Answer many (source, target) queries.

Build the maximum spanning tree once (Kruskal) and answer each query as the minimum edge on the tree path, with binary lifting for O(log V) per query.

FOLLOW-UP 3 Edges below a threshold are forbidden (threshold-limited).

Filter them out before the search (or stop adding edges below the threshold in the Union-Find loop); if source and target are still disconnected the answer is -1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find A Threshold-Limited Path With
# Minimum Required Safety
#
# Practice a graph path interview problem that asks for
# the minimum required safety value along a threshold-
# limited path. The prompt targets minimax path reasoning,
# filtered edges, Dijkstra-style relaxation, disconnected
# graphs, and edge-case handling.
```

```
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use a max-heap search instead of Union-Find.
# 2. Answer many (source, target) queries.
# 3. Edges below a threshold are forbidden (threshold-
# limited).
#
# INTERVIEW NOTES
# When the quality of a path is decided by its weakest
# edge, the best path does not need to be short, only
# strong. Two equivalent tools: add edges from strongest
# to weakest with Union-Find until source and target
# connect (that edge's value is the answer), or run
# Dijkstra with a max-heap where extending a path takes
# the minimum so far. Disconnected graphs must return the
# 'no path' value.
#
# BRUTE FORCE (say it first): Try every simple path
# Depth-first search over simple paths from the source,
# tracking the minimum edge safety, and keep the best at
# the target. Exact and the oracle; exponential. Time
# exponential. Space O(path).


from collections import defaultdict
import heapq


# BRUTE FORCE (say it first): Try every simple path | time
# exponential
def bottleneck_brute(n, edges, s, t):
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        adj[v].append((u, w))
    best = [-1]
```

```
    def go(u, seen, worst):
        if u == t:
            best[0] = max(best[0], worst)
            return
        for v, w in adj[u]:
            if v not in seen:
                go(v, seen | {v}, min(worst, w))

    go(s, {s}, float("inf"))
    return (
        best[0]
        if best[0] != float("inf")
        else float("inf")
    )


# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# BASE QUESTION (optimal): Union-Find adding edges from
# strongest to weakest
# Idea: Sort the edges by safety, strongest first, and
# union their endpoints one by one.
def best_path_safety(n, edges, s, t, threshold=None):
```

```
    # edges are (u, v, safety). The safety of a path is
    # its
    # weakest edge; return the best safety, or -1 if s and
    # t
    # are not connected through edges >= threshold
    if s == t:
        return float("inf")
    uf = UnionFind(n)
    # add the safest edges first: the edge that joins s
```

```
    # and t
    # is the weakest edge of the best path
    for u, v, w in sorted(edges, key=lambda e: -e[2]):
        if threshold is not None and w < threshold:
            break
        uf.union(u, v)
        if uf.find(s) == uf.find(t):
            return w
    return -1


# FOLLOW-UP 1: Use a max-heap search instead of Union-
# Find.
# Answer: Dijkstra where the value of a path is the
# minimum edge so far and you maximise it: pop the best
# bottleneck, extend with min(value, edge). O(E log V);
# Union-Find is simpler when you need only the value.
def best_path_safety_heap(n, edges, s, t, threshold=None):
    # same answer with a max-heap (Dijkstra on the
    # bottleneck)
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        if threshold is None or w >= threshold:
            graph[u].append((v, w))
            graph[v].append((u, w))
    best = [-1] * n
    best[s] = float("inf")
    heap = [(-best[s], s)]
    while heap:
        neg, u = heapq.heappop(heap)
        if -neg < best[u]:
            continue
        for v, w in graph[u]:
            # extending a path can only lower its weakest
            # edge
            cand = min(-neg, w)
            if cand > best[v]:
                best[v] = cand
                heapq.heappush(heap, (-cand, v))
    return best[t]


if __name__ == "__main__":
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
    # 0-1-2-3 has weakest edge 3; 0-2-3 only 1
```

```
    assert best_path_safety(4, edges, 0, 3) == 3
    assert best_path_safety_heap(4, edges, 0, 3) == 3
    # edges below the threshold are forbidden
    assert best_path_safety(4, edges, 0, 3, 4) == -1
    assert best_path_safety_heap(4, edges, 0, 3, 4) == -1
    # disconnected, and source equals target
    assert best_path_safety(3, [(0, 1, 9)], 0, 2) == -1
    assert best_path_safety(3, [], 1, 1) == float("inf")
    one = [(0, 1, 9)]
    assert best_path_safety_heap(3, one, 0, 2) == -1
```

FUNCTION AND TEST CASES

Sample call

```
solution(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)
```

Test cases

Input	Expected
(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)	5
(3, [(0,1,7),(1,2,3)], 0, 2, 5)	-1

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Shortest paths with a heap

MEDIUM | PracHub | Technical Screen | 24 of 33

★ TOP 75: rank 65 in the Medium and Hard priority list

★ Compute shortest paths with blocked nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 60/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Shortest paths with a heap

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #743 Network Delay Time

LeetCode match: LC 743 Network Delay Time (Dijkstra with blocked nodes skipped)

Posted on PracHub: 2025-09-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/compute-shortest-paths-with-blocked-nodes

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.

Constraints

Nodes are labeled 0 through n-1Edges are unweighted

EXAMPLES

```
Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3],
    False)
Expected Output: [0, 1, 2, -1, -1]
Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)
Expected Output: [0, 1, 2, 3]
```

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

APPROACH

Run Dijkstra (BFS if unweighted) from s, never relaxing into blocked nodes; blocked/unreachable nodes stay at infinity. O((V+E) log V).

HOW TO THINK ABOUT IT

Blocked nodes are simply not in the graph, so skip them when you queue neighbours and never start from one. After that it is plain BFS from the source: first discovery gives the shortest distance. State the edge cases: a blocked source makes everything unreachable, and directed edges must only be added in their given direction.

Basis: Unweighted shortest distances (BFS) with some nodes removed from the graph. Output: a distance per node, -1 for unreachable or blocked nodes; directed=True uses edges one way only.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, blocked, s):
    keep = []
    for u, v, w in edges:
        if u not in blocked and v not in blocked:
            keep.append((u, v, w))
    d = floyd_warshall(n, keep)  # all pairs
    return d[s]
```

TIME	SPACE
O(V^3)	O(V^2)

WHY IT IS SLOW

All-pairs distances cost O(V^3) and O(V^2) memory when one source is needed. A BFS from the source that never enters a blocked node is O(V + E).

OPTIMAL SOLUTION · BFS that never enters a blocked node

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges have weights.

BFS is no longer correct: use Dijkstra with a heap and skip blocked nodes the same way. O((V + E) log V). Directed and undirected differ only in whether you add the reverse edge.

FOLLOW-UP 2 Directed versus undirected.

For an undirected graph add each edge in both directions; for a directed one only as given. A node may be reachable in one case and not in the other, so state which you assume.

FOLLOW-UP 3 Nodes get blocked over time and you must keep distances up to date.

Decremental shortest paths is hard in general; for few updates rerun BFS, and for many offline queries process the changes in reverse order (unblocking only improves distances).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute shortest paths with blocked nodes
#
# Given a graph with nodes and edges and a designated
# source node s, compute the shortest distance from s to
# every other node. Some nodes are inaccessible (blocked)
# and cannot be traversed or reached; treat them as
# removed from the graph. Return a distance for each node
# where unreachable nodes (including blocked ones) are -1.
# Describe your algorithm, data structures, and time/space
# complexity, and note any differences for directed vs.
# undirected graphs. Overview: This question evaluates
# proficiency with graph algorithms, shortest-path and
# reachability concepts when nodes are removed or blocked,
# and the ability to analyze time and space complexity
# across directed and undirected graphs.Given n nodes
# labeled 0..n-1, edges, a source, and blocked nodes,
# return shortest unweighted distances from the source.
# Blocked and unreachable nodes must be -1. Set
# directed=True to treat edges as directed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges have weights.
# 2. Directed versus undirected.
# 3. Nodes get blocked over time and you must keep
# distances up to date.
#
# INTERVIEW NOTES
# Blocked nodes are simply not in the graph, so skip them
# when you queue neighbours and never start from one.
# After that it is plain BFS from the source: first
# discovery gives the shortest distance. State the edge
# cases: a blocked source makes everything unreachable,
# and directed edges must only be added in their given
# direction.
#
# BRUTE FORCE (say it first): Floyd-Warshall on the graph
# without the blocked nodes
# Compute all-pairs distances ignoring blocked nodes and
# read the row of the source. Correct and the oracle;
# cubic in the number of nodes. Time O(V^3). Space O(V^2).


import heapq
from collections import defaultdict, deque
```

```
# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Floyd-Warshall on the graph
# without the blocked nodes | time O(V^3)
def blocked_distances_floyd(
    n, edges, source, blocked, directed
):
    INF = float("inf")
    bl = set(blocked)
    d = [
        [0 if i == j else INF for j in range(n)]
        for i in range(n)
    ]
    for u, v in edges:
        if u in bl or v in bl:
            continue
        d[u][v] = min(d[u][v], 1)
        if not directed:
            d[v][u] = min(d[v][u], 1)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    if source in bl:
        return [-1] * n
    return [
        -1 if x == INF or i in bl else x
        for i, x in enumerate(d[source])
    ]


# BASE QUESTION (optimal): BFS that never enters a blocked
# node
# Idea: Build adjacency lists (one direction only if
# directed). If the source is blocked return all -1.
def blocked_distances(n, edges, source, blocked,
                      directed=False):
    # BFS that treats blocked nodes as removed;
    # unreachable
    # and blocked nodes get -1
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        if not directed:
            adj[v].append(u)
    dist = [-1] * n
    dist[source] = 0
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in bad and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist


# FOLLOW-UP 1: The edges have weights.
# Answer: BFS is no longer correct: use Dijkstra with a
# heap and skip blocked nodes the same way. O((V + E) log
# V). Directed and undirected differ only in whether you
# add the reverse edge.
def blocked_distances_weighted(n, edges, source, blocked,
                               directed=False):
    # edges (u, v, w): BFS is wrong now, use Dijkstra
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        if not directed:
            adj[v].append((u, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
```

```
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if v not in bad and d + w < dist[v]:
                dist[v] = d + w
```

```
                heapq.heappush(heap, (d + w, v))
    return [-1 if d == INF else d for d in dist]


if __name__ == "__main__":
    edges = [(0, 1), (1, 2), (0, 3), (3, 2), (2, 4)]
    free = blocked_distances(5, edges, 0, [])
    assert free == [0, 1, 2, 1, 3]
    # blocking 1 forces the route through 3
    assert blocked_distances(5, edges, 0, [1])[2] == 2
    # blocking 2 cuts node 4 off; blocked nodes are -1
    cut = blocked_distances(5, edges, 0, [2])
    assert cut == [0, 1, -1, 1, -1]
    start = blocked_distances(3, edges[:2], 0, [0])
    assert start == [-1, -1, -1]
    # directed edges only go one way
    path = [(0, 1), (1, 2)]
    one_way = blocked_distances(3, path, 2, [], True)
    assert one_way == [-1, -1, 0]
    weighted = [(0, 1, 5), (1, 2, 1), (0, 2, 10)]
    wd = blocked_distances_weighted
    assert wd(3, weighted, 0, []) == [0, 5, 6]
    assert wd(3, weighted, 0, [1])[2] == 10
```

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Shortest paths with a heap

HARD | IN APP LC 1631, 778 | PracHub | Technical Screen | 25 of 33

Find minimum threshold enabling grid path

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 53/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Shortest paths with a heap

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1631 Path with Minimum Effort, LeetCode #778 Swim in Rising Water

LeetCode match: LC 1631 Path With Minimum Effort / LC 778 Swim in Rising Water (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-06-01)

Original: https://prachub.com/coding-questions/find-minimum-threshold-enabling-grid-path

QUESTION

Overview: This question evaluates a candidate's understanding of grid pathfinding, reachability under constraints, and algorithmic optimization, focusing on skills such as graph traversal and handling threshold-based feasibility.

APPROACH

Binary search the threshold and BFS/DFS for reachability, or run Dijkstra/union-find where path cost is the max cell value along the path.

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, and a larger limit never makes things worse, so there are two equivalent plans: binary search the limit with a flood fill as the check, or run Dijkstra where the cost of a path is its maximum so far (extending a path never lowers it). Say the cost function out loud first: it is the maximum, not the sum.

Basis: LeetCode 778, Swim in Rising Water: the smallest threshold such that a path from the top-left to the bottom-right cell exists using only cells at or below it. (The related LeetCode 1631 minimises the largest difference between adjacent cells, see the follow-up.)

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find minimum threshold enabling grid path
#
# This question evaluates a candidate's understanding of
# grid pathfinding, reachability under constraints, and
# algorithmic optimization, focusing on skills such as
# graph traversal and handling threshold-based
# feasibility.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
```

```
# 3. Return the path, not only the value.
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
# equivalent plans: binary search the limit with a flood
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.
#
# BRUTE FORCE (say it first): Try every simple path
# Depth-first search over all simple paths from the top-
# left to the bottom-right, tracking the largest cell,
# with pruning when it cannot beat the best. Exact and the
# oracle; exponential in the grid size. Time exponential.
# Space O(path).


import heapq


# BRUTE FORCE (say it first): Try every simple path | time
# exponential
def min_max_path_brute(H):
    m, n = len(H), len(H[0])
    best = [float("inf")]

    def go(r, c, seen, worst):
        worst = max(worst, H[r][c])
        if worst >= best[0]:
            return
```

```
        if (r, c) == (m - 1, n - 1):
            best[0] = worst
            return
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < m
                and 0 <= nc < n
                and (nr, nc) not in seen
            ):
                go(nr, nc, seen | {(nr, nc)}, worst)

    go(0, 0, {(0, 0)}, 0)
    return best[0]


# HELPERS for the base question
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


# ALSO: the Dijkstra version
def best_path(H, step_cost, start_cost):
    # Dijkstra where a path costs the MAX of its steps;
    # the
    # first time a cell leaves the heap its cost is final
    m, n = len(H), len(H[0])
    best = {(0, 0): start_cost}
    parent = {}
    heap = [(start_cost, 0, 0)]
```

```
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
```

```
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, step_cost(H, r, c, a, b))
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))
    return None, []


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
def min_max_path(H):
    # cost of a path = its largest cell value
    # (water rising over time is the same problem)
    def step(H, r, c, a, b):
        return H[a][b]
    return best_path(H, step, H[0][0])


# FOLLOW-UP 1: The cost is the largest difference between
# adjacent cells on the path (LeetCode 1631).
# Answer: Same Dijkstra with max, but the step cost is
# abs(height difference) instead of the cell value. The
# binary-search version also works: the check allows an
# edge only if the difference is at most the limit.
def min_effort_path(H):
    # LeetCode 1631: cost = the largest height difference
    # between two neighbouring cells on the path
    def step(H, r, c, a, b):
        return abs(H[a][b] - H[r][c])
    return best_path(H, step, 0)


if __name__ == "__main__":
    H = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    cost, path = min_max_path(H)
```

```
    # 0 -> 1 -> 3 -> 7 -> 6 or via 2, 5, 6: the max is 6
    assert cost == 6
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    assert max(H[r][c] for r, c in path) == cost
    assert min_max_path([[5]]) == (5, [(0, 0)])
    effort, path2 = min_effort_path([[1, 2, 2], [3, 8, 2],
                                     [5, 3, 5]])
    assert effort == 2
    assert min_effort_path([[7]]) == (0, [(0, 0)])
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 2], [1, 3]])
```

Test cases

Input	Expected
([[0, 2], [1, 3]],)	3
([[0, 100, 2], [1, 2, 2], [3, 4, 1]],)	2

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Shortest paths with a heap

HARD | PracHub | Technical Screen | 26 of 33

Compute minimax grid path and network delay

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 37.9 | 30 DAYS 0 | PRIORITY 99/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Shortest paths with a heap

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #1631 Path with Minimum Effort, LeetCode #743 Network Delay Time

LeetCode match: LC 1631 Path With Minimum Effort + LC 743 Network Delay Time (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/compute-minimax-grid-path-and-network-delay

QUESTION

Overview: This question evaluates algorithmic problem-solving in shortest-path and path-optimization contexts, specifically minimax pathfinding on a grid and single-source reachability/timing in a weighted directed graph, testing graph modeling, alternative path cost metrics, edge-case handling, and working with large numeric weights.

APPROACH

Part 1: Dijkstra (or binary search + BFS) where cost is the max edge/cell along the path. Part 2: Dijkstra from the source; answer is the max distance, or -1 if some node is unreachable.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Part 1: ask what the path cost is (the maximum cell value along the path, including the start), the allowed moves (4-directional) and the endpoints (top-left to bottom-right). Part 2: ask whether the graph is directed, what to return if some node is unreachable (-1), and the size of weights. The scraped solution covers only part 1.

BRUTE FORCE TO SAY OUT LOUD

Part 1: DFS over all paths, O(4^(mn)). Part 2: Bellman-Ford, which is fine for small graphs.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Part 1: Dijkstra where the cost of a path is the maximum seen so far, with a min-heap keyed on that maximum: new_cost = max(cost, cell). O(mn log mn). Equivalent: binary search the cap and BFS, or sort cells and union-find until start and end connect. Part 2: Dijkstra from the source; the time for a signal to reach everyone is the largest shortest distance, or -1 if any node is unreachable. O((V + E) log V).

DRY-RUN THESE CASES

Part 1: [[1,10,6],[1,3,2],[7,2,2]] gives 3 (as in the example), a 1 x 1 grid, a grid where the best path takes a detour. Part 2: source with no outgoing edges, a disconnected node, parallel edges.

LIKELY FOLLOW-UPS

Part 1 with the cost as the sum minus something, or paths that may revisit cells. Part 2 with large weights (use 64-bit integers).

PITFALLS

Using the sum instead of the maximum when relaxing. Not counting the start cell's own value.

HOW TO THINK ABOUT IT

Both are 'shortest path' with a different notion of cost. For the grid the cost of a path is its maximum, which still works in Dijkstra: the best way to reach a cell is the one with the smallest maximum, and extending a path can never lower it. For the network, Dijkstra from the source gives every arrival time, and the answer is the largest of them (-1 if some node is never reached).

Basis: Two parts: LeetCode 1631-style minimax path (path cost = the largest cell value on the path, start cell included) and LeetCode 743, Network Delay Time (time for a signal to reach every node, or -1).

BRUTE FORCE · Part 1: DFS over every simple path

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

The number of simple paths is exponential. Dijkstra where the cost of a path is its maximum cell (or a union-find over cells added in increasing order) is O(m * n log(m * n)).

BRUTE FORCE · Part 2: Bellman-Ford from the source

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, source):
    dist = [INF] * n
    dist[source] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    worst = max(dist)
    return -1 if worst == INF else worst
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

V - 1 rounds over all edges, O(V * E). Dijkstra with a heap is O((V + E) log V) for non-negative weights.

OPTIMAL SOLUTION · Part 1: Dijkstra where the cost of a path is its maximum cell

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Part 2: Dijkstra from the source, answer is the largest distance

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 1 with a union-find (sort cells, add them one by one).

Add cells in increasing order of height, union neighbours already added, and stop when the start and the goal are connected: that height is the answer. O(mn log mn) for the sort.

FOLLOW-UP 2 Part 2: weights are huge.

Use 64-bit integers (Python is fine); keep sums in a wide type, and with weights up to 10^9 and 10^5 edges a path can reach 10^14. **Cost:** unchanged O((V + E) log V); sums need 64-bit (or Python) integers.

FOLLOW-UP 3 Part 2: the signal can be blocked at some nodes, or each node adds a processing delay.

A node delay is added to the edge cost leaving the node (or modelled by splitting each node into in and out with an edge of the delay). The same Dijkstra then applies.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute minimax grid path and network
# delay
#
# This question evaluates algorithmic problem-solving in
# shortest-path and path-optimization contexts,
# specifically minimax pathfinding on a grid and single-
# source reachability/timing in a weighted directed graph,
# testing graph modeling, alternative path cost metrics,
# edge-case handling, and working with large numeric
# weights.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 1 with a union-find (sort cells, add them one by
# one).
# 2. Part 2: weights are huge.
# 3. Part 2: the signal can be blocked at some nodes, or
# each node adds a processing delay.
#
# INTERVIEW NOTES
# Both are 'shortest path' with a different notion of
# cost. For the grid the cost of a path is its maximum,
# which still works in Dijkstra: the best way to reach a
# cell is the one with the smallest maximum, and extending
# a path can never lower it. For the network, Dijkstra
# from the source gives every arrival time, and the answer
# is the largest of them (-1 if some node is never
# reached).
#
# BRUTE FORCE (say it first): Part 1: DFS over every
# simple path
# Try all simple paths from the top-left to the bottom-
# right and take the smallest maximum cell value. Exact
# and the oracle; exponential in the grid size. Time
# exponential. Space O(path).


import heapq


# BRUTE FORCE (say it first): Part 1: DFS over every
# simple path | time exponential
def minimax_path_dfs(H):
    m, n = len(H), len(H[0])
    best = [float("inf")]
```

```
    def go(r, c, seen, worst):
        worst = max(worst, H[r][c])
        if worst >= best[0]:
            return
        if (r, c) == (m - 1, n - 1):
            best[0] = worst
            return
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
```

```
        ):
            if (
                0 <= nr < m
                and 0 <= nc < n
                and (nr, nc) not in seen
            ):
                go(nr, nc, seen | {(nr, nc)}, worst)

    go(0, 0, {(0, 0)}, 0)
    return best[0]


# HELPERS for the base question
INF = float("inf")


def neighbors(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


# BASE QUESTION (optimal): Dijkstra where the cost of a
# path is its maximum cell
# Idea: Heap of (best maximum so far, cell). Pop the cell
# with the smallest bottleneck, and for each neighbour the
# new bottleneck is max(current, neighbour). Improve if
# smaller.
def minimax_path(H):
    # Part 1: the path whose highest cell is as low as
    # possible
    m, n = len(H), len(H[0])
```

```
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in neighbors(r, c, m, n):
            # the cost of a path is its maximum, never a
            # sum
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))
    return -1


# PART 2: network delay
def network_delay(n, edges, source, node_delay=None):
    # Part 2: when does the signal reach every node?
    # edges are (u, v, weight); -1 if some node is
    # unreachable
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        # a processing delay is paid when leaving a node
        pause = node_delay[u] if node_delay else 0
        for v, w in graph[u]:
            if d + pause + w < dist[v]:
                dist[v] = d + pause + w
                heapq.heappush(heap, (dist[v], v))
```

```
    worst = max(dist)
    return -1 if worst == INF else worst


# FOLLOW-UP 1: Part 1 with a union-find (sort cells, add
# them one by one).
```

```
# Answer: Add cells in increasing order of height, union
# neighbours already added, and stop when the start and
# the goal are connected: that height is the answer. O(mn
# log mn) for the sort.
def minimax_path_union_find(H):
    # the same answer: add cells from lowest to highest
    # and
    # stop when the start and the goal are connected
    m, n = len(H), len(H[0])
    parent = {}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    cells = sorted((H[r][c], r, c)
                   for r in range(m) for c in range(n))
    for h, r, c in cells:
        parent[(r, c)] = (r, c)
        for a, b in neighbors(r, c, m, n):
            if (a, b) in parent:
                parent[find((r, c))] = find((a, b))
        if (0, 0) in parent and (m - 1, n - 1) in parent:
            if find((0, 0)) == find((m - 1, n - 1)):
                return h
    return -1


if __name__ == "__main__":
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert minimax_path(H) == 2
    assert minimax_path(H) == minimax_path_union_find(H)
    assert minimax_path([[5]]) == 5
    edges = [(0, 1, 2), (1, 2, 3), (0, 2, 10)]
    assert network_delay(3, edges, 0) == 5
    # a node that cannot be reached
    assert network_delay(4, edges, 0) == -1
    # huge weights are fine with Python integers
    big = [(0, 1, 10 ** 9), (1, 2, 10 ** 9)]
    assert network_delay(3, big, 0) == 2 * 10 ** 9
    # each node adds a processing delay of 1
    slow = network_delay(3, big, 0, [1, 1, 1])
    assert slow == 2 * 10 ** 9 + 2
```

FUNCTION AND TEST CASES

Signature

```
def solution(H: list[list[int]]) -> int:
SAMPLE INPUT
[[1, 10, 6], [1, 3, 2], [7, 2, 2]]
```

Test cases

Input	Expected
`[[1, 10, 6], [1, 3, 2], [7, 2, 2]]`	`3`
`[[5, 1, 7], [4, 8, 2], [3, 6, 0]]`	`6`

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Shortest paths with a heap

HARD | IN APP LC 778, 1631 | PracHub | Technical Screen | 27 of 33

Minimize maximum height along a grid path

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 54/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Shortest paths with a heap

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #778 Swim in Rising Water, LeetCode #1631 Path with Minimum Effort

LeetCode match: LC 778 Swim in Rising Water / LC 1631

Posted on PracHub: 2026-01-04 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/minimize-maximum-height-along-a-grid-path

QUESTION

Overview: This question evaluates algorithmic pathfinding and graph-modeling skills, specifically reasoning about grid-based route selection where the objective is defined by an aggregate maximum over visited cell heights.

APPROACH

Min-heap Dijkstra keyed by the max height seen so far (or binary search the cap + BFS, or union-find by sorted cell heights). O(mn log mn).

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, and a larger limit never makes things worse, so there are two equivalent plans: binary search the limit with a flood fill as the check, or run Dijkstra where the cost of a path is its maximum so far (extending a path never lowers it). Say the cost function out loud first: it is the maximum, not the sum.

Basis: LeetCode 778, Swim in Rising Water: the smallest maximum height such that a path from the top-left to the bottom-right cell exists using only cells at or below it. (The related LeetCode 1631 minimises the largest difference between adjacent cells, see the follow-up.)

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimize maximum height along a grid path
#
# This question evaluates algorithmic pathfinding and
# graph-modeling skills, specifically reasoning about
# grid-based route selection where the objective is
# defined by an aggregate maximum over visited cell
# heights.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
# 3. Return the path, not only the value.
```

```
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
# equivalent plans: binary search the limit with a flood
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.
#
# BRUTE FORCE (say it first): Try every simple path
# Depth-first search over all simple paths from the top-
# left to the bottom-right, tracking the largest cell,
# with pruning when it cannot beat the best. Exact and the
# oracle; exponential in the grid size. Time exponential.
# Space O(path).


import heapq


# BRUTE FORCE (say it first): Try every simple path | time
# exponential
def min_max_path_brute(H):
    m, n = len(H), len(H[0])
    best = [float("inf")]

    def go(r, c, seen, worst):
        worst = max(worst, H[r][c])
        if worst >= best[0]:
            return
```

```
        if (r, c) == (m - 1, n - 1):
            best[0] = worst
            return
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < m
                and 0 <= nc < n
                and (nr, nc) not in seen
            ):
                go(nr, nc, seen | {(nr, nc)}, worst)

    go(0, 0, {(0, 0)}, 0)
    return best[0]


# HELPERS for the base question
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


def reachable(H, limit):
    # flood fill using only cells with height <= limit
    m, n = len(H), len(H[0])
    if H[0][0] > limit:
        return False
    seen, stack = {(0, 0)}, [(0, 0)]
    while stack:
        r, c = stack.pop()
        if (r, c) == (m - 1, n - 1):
            return True
```

```
        for a, b in around(r, c, m, n):
            if (a, b) not in seen and H[a][b] <= limit:
                seen.add((a, b))
                stack.append((a, b))
    return False


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
def min_max_path(H, with_path=False):
    # Dijkstra where a path costs the MAX of its cells
    m, n = len(H), len(H[0])
    best, parent = {(0, 0): H[0][0]}, {}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            if not with_path:
                return cost
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))


# ALTERNATIVE: binary search the answer
def min_max_path_bsearch(H):
    # binary search the answer over the sorted heights
    heights = sorted({h for row in H for h in row})
    lo, hi = 0, len(heights) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if reachable(H, heights[mid]):
            hi = mid
        else:
            lo = mid + 1
```

```
    return heights[lo]


# FOLLOW-UP 1: The cost is the largest difference between
# adjacent cells on the path (LeetCode 1631).
# Answer: Same Dijkstra with max, but the step cost is
# abs(height difference) instead of the cell value. The
# binary-search version also works: the check allows an
# edge only if the difference is at most the limit.
def min_effort_path(H):
    # LeetCode 1631: cost = the largest height DIFFERENCE
    m, n = len(H), len(H[0])
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in around(r, c, m, n):
            nxt = max(cost, abs(H[a][b] - H[r][c]))
```

```
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))


if __name__ == "__main__":
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert min_max_path(H) == min_max_path_bsearch(H)
    cost, path = min_max_path(H, True)
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    assert max(H[r][c] for r, c in path) == cost
    assert min_max_path([[7]]) == 7
    grid = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    assert min_max_path(grid) == 6
    assert min_max_path_bsearch(grid) == 6
    assert min_effort_path([[1, 2, 2], [3, 8, 2],
                            [5, 3, 5]]) == 2
```

FUNCTION AND TEST CASES

Sample call

```
solution([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Test cases

Input	Expected
`([[1, 2, 3], [4, 5, 6], [7, 8, 9]],)`	`9`
`([[8, 4, 7], [6, 5, 9], [3, 2, 1]],)`	`8`

ALSO REPORTED · 1 more

Path Minimizing the Maximum Cell (Swim in Rising Water) | 1Point3Acres | onsite-coding | asked 2026-07-08 | Low

https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter. Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

Preparation: Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation. Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Stacks

Site tags: array, monotonic-stack, stack, medium

MEDIUM | 1Point3Acres | tech-screen | 28 of 33

Visible People in a Line (Last Person, Then Everyone)

ASKED 2026-10-07 | 13 DAYS AGO | LAST 30 DAYS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 181/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Stacks

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1944 Number of Visible People in a Queue

LeetCode match: LC 1944 Number of Visible People in a Queue (related)

Frequency: Single report | Last asked: 2026-10-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/visible-people-in-line

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person,
    height 5, can see 3, 4, 2, 8, 9, 10).
```

APPROACH

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what blocks the view (someone strictly taller than both), how ties are treated, which directions count in Part 2, and whether the code runs (it does not: trace by hand).

BRUTE FORCE TO SAY OUT LOUD

Check every pair against the people between them. O(n^2) for the last person, O(n^3) for everyone.

OPTIMAL IDEA

Part 1: walk from the person before the last towards the front, keeping the maximum height between the last person and the current one; the current person is visible when that maximum is at most the larger of the two heights. O(n). Part 2: for each person, everyone up to the first strictly taller person on the right is visible, then only people at least as tall as everyone from there on; a monotonic stack gives the next at-least-as-tall and next strictly-taller indexes and the chain length in O(n). Run it on the reversed line for the left side and add.

DRY-RUN THESE CASES

A single person, all equal heights, strictly increasing and decreasing lines, the example from the statement (the last person sees 6), a tie case you invent.

LIKELY FOLLOW-UPS

Only looking in one direction, blocking on equal heights, or returning who they see instead of how many.

PITFALLS

Using a strict comparison where the rule says taller than both. Forgetting that the first taller person is itself visible. Rereading the example without tracing it by hand.

HOW TO THINK ABOUT IT

Person A sees person B when nobody strictly between them is taller than both, which means the tallest person between them is at most as tall as the taller of A and B. Walking away from the last person and keeping the running maximum of everyone passed lets you test each person in O(1). For everyone at once the visible people to the right form a chain: everything up to the first taller person, then only the people who are at least as tall as everyone before them. A monotonic stack builds those chains for all positions in O(n).

Basis: LeetCode 1944, Number of Visible People in a Queue, with the rule that a person is hidden only by someone taller than both.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sees(a, i, j):
    lo, hi = min(i, j), max(i, j)
    for k in range(lo + 1, hi):
        if a[k] > max(a[i], a[j]):
            return False
    return True
def count_for(a, i):
    return sum(1 for j in range(len(a))
               if j != i and sees(a, i, j))
```

TIME	SPACE
O(n^3) for everyone, O(n^2) for the last person	**O(1)**

BRUTE FORCE

WHY IT IS SLOW

Each pair rescans the people between it, O(n^3) for everyone and O(n^2) for the last person. One pass with a running maximum answers the last person in O(n), and a monotonic stack answers everyone in O(n).

OPTIMAL SOLUTION · One pass with a running maximum (last person)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 2: return the number of people each person can see.

Run the right-looking count for every position with a monotonic stack, run it again on the reversed line for the looking-left count, and add the two. The stack makes both passes O(n), instead of the O(n^2) you would get by repeating the last-person scan for every position.

FOLLOW-UP 2 How do ties and the direction of sight change the answer?

Under the stated rule only someone strictly taller than both blocks the view, so equal heights never block: in [5, 5, 5] the first person sees both others. If the interviewer wants 'taller or equal blocks', replace the non-strict comparisons with strict ones. If a person only looks towards the front, drop the reversed pass and use see_right on the reversed line alone.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Visible People in a Line (Last Person,
# Then Everyone)
#
# Given heights of people in a line, A can see B if nobody
# between them is taller than both. Part 1: count how many
# people the last person can see. Part 2: return that
# count for every person. Written in a plain editor with
# no compilation. Input: an array of heights representing
# a line of people. Visibility rule: person A can see
# person B if no one standing between them is taller than
# both A and B. Part 1: return how many people the last
# person in line can see. Part 2: for the same array,
# return, for every person in the line, how many people
# they can see. Examples: [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
# -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9,
# 10).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 2: return the number of people each person can
# see.
# 2. How do ties and the direction of sight change the
# answer?
#
# INTERVIEW NOTES
# Person A sees person B when nobody strictly between them
# is taller than both, which means the tallest person
# between them is at most as tall as the taller of A and
# B. Walking away from the last person and keeping the
# running maximum of everyone passed lets you test each
# person in O(1). For everyone at once the visible people
# to the right form a chain: everything up to the first
# taller person, then only the people who are at least as
# tall as everyone before them. A monotonic stack builds
# those chains for all positions in O(n).
#
# BRUTE FORCE (say it first): Test every pair
# For a pair (i, j) look at everyone between them and
# check that none is taller than both. Counting over all
# pairs is O(n^3). Fine as the oracle and for hand-tracing
# the example. Time O(n^3) for everyone, O(n^2) for the
# last person. Space O(1).


# BRUTE FORCE (say it first, it also checks the answers)
def sees(a, i, j):
    # the oracle: nobody strictly between is taller than
    # both
    lo, hi = min(i, j), max(i, j)
    return all(a[k] <= max(a[i], a[j])
               for k in range(lo + 1, hi))


# BRUTE FORCE (say it first, it also checks the answers)
def counts_brute(a):
    return [sum(1 for j in range(len(a))
                if j != i and sees(a, i, j))
            for i in range(len(a))]


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): One pass with a running maximum
# (last person)
# Idea: Walk from the person next to the last one towards
# the front, keeping the largest height seen strictly
# between the last person and the current one.
def last_sees(a):
    """Part 1: walk away from the last person keeping the
    tallest height seen strictly between us and them."""
    if not a:
        return 0
    last, between, count = a[-1], -INF, 0
    for k in range(len(a) - 2, -1, -1):
        # nobody between is taller than both of us
        if between <= max(last, a[k]):
            count += 1
        between = max(between, a[k])
    return count


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_right(a):
    """Part 2, looking right: everyone up to the first
    strictly taller person is visible, then only people at
```

```
    least as tall as everyone from that person on."""
    n = len(a)
    next_ge, next_gt = [None] * n, [None] * n
    stack = []
    for k in range(n - 1, -1, -1):
        # next person at least as tall as person k
        while stack and a[stack[-1]] < a[k]:
            stack.pop()
        next_ge[k] = stack[-1] if stack else None
        stack.append(k)
    stack = []
    for k in range(n - 1, -1, -1):
        # next person strictly taller than person k
        while stack and a[stack[-1]] <= a[k]:
            stack.pop()
        next_gt[k] = stack[-1] if stack else None
        stack.append(k)
    chain = [0] * n
    for k in range(n - 1, -1, -1):
        # how many "at least as tall" steps follow k
        if next_ge[k] is not None:
            chain[k] = 1 + chain[next_ge[k]]
    out = []
    for i in range(n):
        t = next_gt[i]
        if t is None:
            out.append(n - 1 - i)
        else:
            out.append(t - i + chain[t])
    return out


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_all(a):
    # looking left is looking right on the reversed line
    left = see_right(a[::-1])[::-1]
    return [x + y for x, y in zip(left, see_right(a))]


if __name__ == "__main__":
    line = [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
```

```
    assert last_sees(line) == 6
    assert see_all(line)[-1] == 6
    assert last_sees([]) == 0 and last_sees([7]) == 0
    # equal heights never block each other
    assert last_sees([5, 5, 5]) == 2
    assert see_all([5, 5, 5]) == [2, 2, 2]
    import random
    for _ in range(300):
        a = [random.randint(1, 6)
             for _ in range(random.randint(0, 9))]
        assert see_all(a) == counts_brute(a)
        if a:
            assert last_sees(a) == counts_brute(a)[-1]
```

NOTES FROM CANDIDATES

Pitfalls: The code is written in a plain text editor that does not compile or run; trace the example by hand before declaring the solution done.
Confirm the direction of "see" for Part 2 (toward the front only, or both directions) and how equal heights are treated before coding; the example does not exercise ties.

Follow-ups: Generalize from the last person to every person in the line (Part 2).

PREPARATION

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Stacks

MEDIUM | PracHub | Technical Screen | 29 of 33

★ TOP 75: rank 71 in the Medium and Hard priority list

★ Expand Recursive Key-Value Placeholders in a Template

POSTED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 57.7** | **30 DAYS 0** | **PRIORITY 46/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Stacks

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #394 Decode String

LeetCode match: No direct match; related LC 394 Decode String (inferred from overview)

Posted on PracHub: 2026-08-04 (updated 2026-08-16)

Original: https://prachub.com/coding-questions/expand-recursive-key-value-placeholders-in-a-template

QUESTION

Overview: Expand %KEY% placeholders in a template, recursively resolving placeholders inside mapped values. Use memoized dependency traversal to preserve literal text, avoid repeated work, and explain cycle, unknown-key, and escaping policies.

APPROACH

DFS expansion with memoization per key and a 'visiting' set to detect cycles; leave unknown keys per the spec; keep literal text unchanged.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the placeholder syntax (%KEY%), what to do with an unknown key (leave it as is or raise), how to write a literal percent sign (escaping such as %%), and how to handle cycles (raise an error).

BRUTE FORCE TO SAY OUT LOUD

Repeatedly run a regex replace on the whole string until nothing changes. It loops forever on a cycle.

OPTIMAL IDEA

Resolve each key with memoized DFS: resolve(key) expands the value of that key by scanning for placeholders and recursively resolving them, with a visiting set to detect cycles and a cache so each key is expanded once. Expand the template by scanning once and replacing each placeholder with resolve(name), leaving literal text untouched. Linear in the sizes of text and values.

DRY-RUN THESE CASES

A value that refers to another key, a cycle (A refers to B, B to A), an unknown key, a template with no placeholders, two placeholders adjacent, an escaped percent.

LIKELY FOLLOW-UPS

Escaping, recursion depth limit (iterative expansion), or limiting the output size to prevent blow-up.

PITFALLS

Re-expanding a key every time. Treating an unknown key and a cycle the same way.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

Expanding one key can require expanding others, so define expand(key) recursively and remember results (memoization) so a key used many times is expanded once. A 'currently expanding' set detects cycles: meeting a key that is already on the stack means a cycle. Decide the policy for unknown keys and for literal percent signs up front.

Basis: Written. Assumed rules (confirm them): placeholders look like %KEY%; the value of a key may itself contain placeholders; an unknown key is left as is; a cycle raises an error.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text):
    for _ in range(LIMIT):
        new = text
        for key, value in values.items():
            tag = "%" + key + "%"
            new = new.replace(tag, value)
        if new == text:
            return text
        text = new
    raise Error("cycle")
```

TIME	SPACE
O(depth * length)	O(length)

WHY IT IS SLOW

It re-expands the same key every time it appears and needs an arbitrary cap to stop on cycles. A memoised recursive expansion with a visiting set expands each key once and detects cycles exactly.

OPTIMAL SOLUTION · Memoised recursive expansion with a visiting set

TIME	SPACE
O(total size of all values + template)	O(number of keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A literal percent sign must be possible.

Choose an escape such as %% and handle it in the same scan: when the text has %%, output one % and skip both characters; otherwise try to match a placeholder. Test percent signs next to placeholders.

FOLLOW-UP 2 An unknown key should be an error, or replaced by an empty string.

Make the policy a parameter: raise KeyError, return the placeholder unchanged (as coded), or return ''. State your default.

FOLLOW-UP 3 Values are huge and expansion explodes.

The output can be exponentially larger than the input (a key referring to the previous key twice, repeatedly). Check the expanded length as you go and fail past a limit, or return a lazy expansion.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Expand Recursive Key-Value Placeholders
# in a Template
#
# Expand %KEY% placeholders in a template, recursively
# resolving placeholders inside mapped values. Use
# memoized dependency traversal to preserve literal text,
# avoid repeated work, and explain cycle, unknown-key, and
# escaping policies.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A literal percent sign must be possible.
# 2. An unknown key should be an error, or replaced by an
# empty string.
# 3. Values are huge and expansion explodes.
#
# INTERVIEW NOTES
# Expanding one key can require expanding others, so
# define expand(key) recursively and remember results
# (memoization) so a key used many times is expanded once.
# A 'currently expanding' set detects cycles: meeting a
# key that is already on the stack means a cycle. Decide
# the policy for unknown keys and for literal percent
# signs up front.
#
# BRUTE FORCE (say it first): Keep substituting until the
# text stops changing
# Replace every %KEY% found in the text by its value and
# repeat until nothing changes. Simple, but it never stops
# on a cycle (so it needs an iteration cap) and re-expands
# the same key every time it appears. Time O(depth *
# length). Space O(length).


import re


# BRUTE FORCE (say it first): Keep substituting until the
# text stops changing | time O(depth * length)
_PH = re.compile(r"%([A-Za-z_][A-Za-z0-9_]*)%")
```

```
# HELPERS for the base question
PATTERN = re.compile(r"%%|%(\w+)%")
```

```
def expand_by_rewriting(
    template, mapping, max_rounds=50
):
    text = template
    for _ in range(max_rounds):
        new = _PH.sub(
            lambda m: mapping.get(
                m.group(1), m.group(0)
            ),
            text,
        )
        if new == text:
            return text
        text = new
    raise ValueError("cycle")


# BASE QUESTION (optimal): Memoised recursive expansion
# with a visiting set
# Idea: expand(key) substitutes every placeholder in the
# key's value by expand(inner), caches the result, and
# uses a visiting set to raise on a cycle. Unknown keys
# stay as written.
# FOLLOW-UP 1: A literal percent sign must be possible.
# Answer: Choose an escape such as %% and handle it in the
# same scan: when the text has %%, output one % and skip
# both characters; otherwise try to match a placeholder.
# Test percent signs next to placeholders.
def expand(template, mapping, unknown="keep",
           limit=10 ** 6):
    # unknown: "keep" the text, "empty" it, or "error"
    cache, visiting = {}, set()

    def expand_text(text):
        def one(m):
            key = m.group(1)
            # FOLLOW-UP 1: %% is a literal percent sign
            if key is None:
                # %% is a literal percent sign
                return "%"
            if key not in mapping:
                # FOLLOW-UP 2: unknown key: keep, empty or
                # error
                if unknown == "error":
                    raise KeyError(key)
```

```
                if unknown == "empty":
                    return ""
                return m.group(0)
            return expand_key(key)
        out = PATTERN.sub(one, text)
        # FOLLOW-UP 3: huge values: cap the expansion
        # stop explosive expansions early
        if len(out) > limit:
            raise ValueError("expansion too large")
        return out

    def expand_key(key):
        # each key is expanded once and then cached
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError(f"cycle at {key}")
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]
```

```
    return expand_text(template)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    want = "I am admin My home is /admin/home"
    assert expand(text, m) == want
    # unknown keys: kept, emptied, or an error
    assert expand("a %X% b", m) == "a %X% b"
    assert expand("a %X% b", m, unknown="empty") == "a  b"
    try:
        expand("%X%", m, unknown="error")
        raise AssertionError("missing key not reported")
    except KeyError:
        pass
    # a literal percent, even next to a placeholder
    assert expand("100%% of %USER%", m) == "100% of admin"
    # a cycle is reported
    try:
        expand("%A%", {"A": "%B%", "B": "%A%"})
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
    # a key that doubles the previous one explodes
    boom = {"K0": "ab"}
    for i in range(1, 30):
        boom[f"K{i}"] = f"%K{i - 1}%%K{i - 1}%"
    try:
        expand("%K29%", boom, limit=1000)
        raise AssertionError("explosion not stopped")
    except ValueError:
        pass
```

FUNCTION AND TEST CASES

Signature

```
def expand_template(values: dict[str, str], template: str)
 -> str:
```

Sample call

```
expand_template({}, 'plain text')
```

Test cases

Input	Expected
`({}, 'plain text')`	`'plain text'`
`({'USER': 'admin', 'HOME': '/%USER%/home'}, 'I am %USER% My home is %HOME%')`	`'I am admin My home is /admin/home'`

ALSO REPORTED · 1 more

Template Variable Expansion with Follow-Ups | 1Point3Acres | onsite-coding | asked 2026-08-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

ALSO REPORTED · 1 more

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

Preparation: Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Stacks

Site tags: string, stack, parsing, medium

MEDIUM | IN APP LC 394 | 1Point3Acres | onsite-coding | 30 of 33

★ TOP 75: rank 60 in the Medium and Hard priority list

★ Decode String

ASKED 2026-07-29 | 83 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 0 | PRIORITY 45/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Stacks

Optimal answer from: your app's interview approach (LeetCode #394)

LeetCode match: LC 394 Decode String

Frequency: Low | Last asked: 2026-07-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/decode-string

QUESTION

EXAMPLES

```
"3[a]2[bc]" -> "aaabcbc".
"2[abc]3[cd]ef" -> "abcabccdcdcdef".
"3[a2[c]]" -> "accaccacc".
```

APPROACH

Write the iterative-stack version from memory in under 10 min. Practice the postfix-syntax variant (parse direction reversed). Drill the decode side together with the encode-shortest dual problem as a pair.

HOW TO THINK ABOUT IT

Nested repeats like 3[a2[c]] are a bracket structure, which means a stack: when you see '[' save what you have built and the repeat count; when you see ']' repeat the inner string and glue it to the saved prefix. Read counts as full numbers (more than one digit).

Basis: LeetCode 394, Decode String.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def parse():
    result = ""
    while pos < len(s) and s[pos] != "]":
        if s[pos].isdigit():
            n = read_number()
            pos += 1  # skip "["
            inner = parse()
            pos += 1  # skip "]"
            result += inner * n
        else:
            result += s[pos]
            pos += 1
    return result
```

BRUTE FORCE

TIME	SPACE
O(output length)	O(depth)

WHY IT IS SLOW

Time is proportional to the output, which is optimal; the cost is the recursion depth, equal to the nesting depth. A stack of (text so far, repeat count) avoids recursion.

OPTIMAL SOLUTION · Stack of (prefix, repeat) frames

TIME	SPACE
O(output length)	O(output + nesting depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

FOLLOW-UP 2 What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

FOLLOW-UP 3 The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as prev + inner * times. The length can be computed for strings that could never fit in memory.

FOLLOW-UP 4 Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

FOLLOW-UP 5 Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Decode String
#
# Examples: "3[a]2[bc]" -> "aaabcbc". "2[abc]3[cd]ef" ->
# "abcabccdcdcdef". "3[a2[c]]" -> "accaccacc".
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you do it recursively, and what is the
# risk?
# 2. What if the decoded string is enormous?
# 3. The decoded string would be enormous.
# 4. Return the character at position k only.
# 5. Invalid input (missing bracket, count with no
# bracket).
#
# INTERVIEW NOTES
# Nested repeats like 3[a2[c]] are a bracket structure,
# which means a stack: when you see '[' save what you have
# built and the repeat count; when you see ']' repeat the
# inner string and glue it to the saved prefix. Read
# counts as full numbers (more than one digit).
#
# BRUTE FORCE (say it first): Recursive descent over the
# string
# A function that reads letters until it meets a digit
# (then reads the number, the bracketed substring
# recursively, and repeats it) or a closing bracket. It
# mirrors the grammar and is the oracle; the recursion
# depth equals the nesting depth. Time O(output length).
# Space O(depth).


# BRUTE FORCE (say it first): Recursive descent over the
# string | time O(output length)
def decode_recursive(s):
    pos = 0

    def parse():
        nonlocal pos
        out = []
        while pos < len(s) and s[pos] != "]":
            if s[pos].isdigit():
                num = 0
                while s[pos].isdigit():
                    num = num * 10 + int(s[pos])
                    pos += 1
                pos += 1  # skip '['
                inner = parse()
                pos += 1  # skip ']'
                out.append(inner * num)
            else:
                out.append(s[pos])
                pos += 1
        return "".join(out)

    return parse()


# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames
# Idea: On '[' push the text built so far and the repeat
# count, then start fresh. On ']' pop and append the
# current text repeated count times to the saved prefix.
def decode(s):
    # stack of (text so far, repeat count) for each open [
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            # remember what we have, start the inner text
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)


# NESTED FORM (the recursive version)
def parse(s):
    # nested form: items are text or (times, items)
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            if num == 0:
                raise ValueError("[ needs a count")
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            if not stack:
                raise ValueError("unmatched ]")
            prev, times = stack.pop()
            prev.append((times, cur))
            cur = prev
        else:
            if num:
                raise ValueError("count without [")
            cur.append(ch)
    if stack:
        raise ValueError("missing ]")
    return cur


# FOLLOW-UP 3: The decoded string would be enormous.
# Answer: Compute its length (or a given character) from
# the stack without building it: keep (length so far,
# repeat count) frames and combine as prev + inner *
# times. The length can be computed for strings that could
# never fit in memory.
def length(items):
    # decoded length without building the string
    total = 0
    for it in items:
        if isinstance(it, str):
            total += 1
        else:
            total += it[0] * length(it[1])
    return total


# FOLLOW-UP 4: Return the character at position k only.
# Answer: Walk the structure with lengths: at each repeat
# reduce k modulo the inner length to find where in the
# inner string it falls (LeetCode 880 style).
def char_at(items, k):
    # the k-th decoded character (0-based), LeetCode 880
    # style
    for it in items:
        if isinstance(it, str):
            if k == 0:
```

```
                return it
            k -= 1
        else:
            times, inner = it
            inner_len = length(inner)
            if k < times * inner_len:
                # which copy does not matter: use k modulo
                return char_at(inner, k % inner_len)
            k -= times * inner_len
    return None


if __name__ == "__main__":
    assert decode("3[a]2[bc]") == "aaabcbc"
    assert decode("2[abc]3[cd]ef") == "abcabccdcdcdef"
    assert decode("3[a2[c]]") == "accaccacc"
    items = parse("3[a2[c]]")
    assert length(items) == 9
    assert [char_at(items, i) for i in range(9)] == list(
        decode("3[a2[c]]"))
    # a huge result is measured, never built
    big = parse("999[999[999[ab]]]")
    assert length(big) == 2 * 999 ** 3
    for bad in ("3[a", "a]", "[a]", "3a"):
        try:
            parse(bad)
            raise AssertionError(bad)
        except ValueError:
            pass
```

NOTES FROM CANDIDATES

Google variants include alternative syntax (group){k} (postfix repeat), nested grouping with curly braces, and escape characters.
A scale follow-up asks how the interface and memory strategy should change when the encoded string is extremely long.
Recursive solution is cleaner but the iterative stack is more robust to bizarre input.

PREPARATION

Write the iterative-stack version from memory in under 10 min.
Practice the postfix-syntax variant (parse direction reversed).
Drill the decode side together with the encode-shortest dual problem as a pair.

ALSO REPORTED · 1 more

String Decompression with Nested Counts | 1Point3Acres

https://www.1point3acres.com/interview/problems/company/google/string-decompression-nested-counts

Recursion with an index pointer is the cleanest implementation: each call returns the decoded substring of a single (...){k} block plus literals up to its terminator.
An iterative stack-based approach also works: push (StringBuilder, repeatCount) on (, pop and append on){k}.
The 2-digit k matters only for parsing — read until non-digit.
Watch out for literals between two groups: a(b){2}c(d){3} -> abbcddd.

ALSO REPORTED · 1 more

Preparation: Drill the canonical k[group] decode-string problem to muscle memory; the input grammar there is k[group] instead of (group){k}, but the algorithm is identical.
Practice both recursive and iterative versions — the iterative stack version generalizes more cleanly if the interviewer asks about iterative streaming.
Test with: deeply nested groups, groups containing only literals, two-digit counts ({12}).

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Stacks

Site tags: parsing, recursion, stack, medium

MEDIUM | 1Point3Acres | onsite-coding | 31 of 33

Nested Function Expression Evaluator

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | **ALL TIME 5** | **30 DAYS 0** | **PRIORITY 175/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Stacks

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 770 Basic Calculator IV (related)

Frequency: Low | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-function-expression-evaluator

QUESTION

EXAMPLES

```
add(1, sub(1, 0)) -> 2
add(2, mul(3, pow(4, 5))) -> 3074
```

APPROACH

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which functions exist, whether they take a fixed or variable number of arguments, whether numbers can be negative, how division behaves, and how errors are reported.

BRUTE FORCE TO SAY OUT LOUD

Repeatedly find the innermost call with a regex and replace it with its value. Quick but fragile.

OPTIMAL IDEA

Tokenize into identifiers, integers, commas and parentheses. Write parse(i) returning (value, next index): a number returns itself; a name must be followed by '(', then loop reading arguments until ')', then apply the function. A function table keeps new operators one line each. O(n), recursion depth equals nesting depth.

DRY-RUN THESE CASES

add(1, sub(1, 0)) is 2. add(2, mul(3, pow(4, 5))) is 3074. A bare number. Variable argument counts. Unknown function names.

LIKELY FOLLOW-UPS

Variables, iterative parsing with two stacks, or division semantics.

INTERVIEW-DAY PLAN

PITFALLS

Losing the next index after parsing an argument. Treating '-' in '-5' as an operator.

HOW TO THINK ABOUT IT

An expression with function calls inside function calls has a grammar: expression = number | name '(' expression, expression, ... ')'. Each call can be evaluated only after its arguments, so write a function that parses one expression and reports where it stopped (recursive descent). A tokenizer first, then parse(i) returning (value, next index), keeps the bookkeeping small.

Basis: Recursive descent parsing, in the spirit of LeetCode 224 / 726; closest on LeetCode is 770, Basic Calculator IV.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def evaluate(text):
    while has_call(text):
        call = find_innermost_call(text)
        value = apply(call)  # e.g. add(2, 3)
        text = text.replace(call, str(value))
    return int(text)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every replacement rescans and rebuilds the string, O(n) per call and O(n^2) in total, and it cannot report errors well. A tokenizer plus recursive descent reads the text once, O(n).

OPTIMAL SOLUTION · Tokenizer + recursive descent over name(arg, arg, ...)

TIME	SPACE
O(n)	O(depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support variadic functions and variables.

Collect arguments until ')' instead of assuming two (as the code does). For variables, pass an environment dict into parse and treat an identifier that is not followed by '(' as a lookup. Define the error behaviour for undefined names.

FOLLOW-UP 2 How do you make it iterative?

Use two stacks: one of pending function names and one of lists of evaluated arguments. On an identifier push the name and a new list, on a number append to the top list, on ')' pop both, apply the function, and append the result to the new top list. This removes the recursion-depth limit but is easier to get wrong off by one.

FOLLOW-UP 3 What about division?

State the contract: integer or float, truncate toward zero or floor (Python's // floors, which differs for negatives), and divide by zero handling. The implementation above truncates toward zero.

FOLLOW-UP 4 Add variables (an identifier with no parentheses is a lookup).

In parse, check whether the identifier is followed by '('; if not, return env[name]. Decide what an undefined name does (raise, or default to 0).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Nested Function Expression Evaluator
#
# Examples: add(1, sub(1, 0)) -> 2 add(2, mul(3, pow(4,
# 5))) -> 3074
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support variadic functions and variables.
# 2. How do you make it iterative?
# 3. What about division?
# 4. Add variables (an identifier with no parentheses is a
# lookup).
#
# INTERVIEW NOTES
# An expression with function calls inside function calls
# has a grammar: expression = number | name '('
# expression, expression, ... ')'. Each call can be
# evaluated only after its arguments, so write a function
# that parses one expression and reports where it stopped
# (recursive descent). A tokenizer first, then parse(i)
# returning (value, next index), keeps the bookkeeping
# small.
#
# BRUTE FORCE (say it first): Repeatedly replace the
# innermost call with its value
# Find a call whose arguments contain no parentheses (a
# regular expression can do this), compute it, replace the
# text with the result, and repeat until only a number is
# left. It needs no parser, and it is quadratic because
# every replacement rescans and rebuilds the string. Time
# O(n^2). Space O(n).


import math
import re
```

```
# BRUTE FORCE (say it first): Repeatedly replace the
# innermost call with its value | time O(n^2)
_OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a**b,
    "div": lambda a, b: int(a / b),
    "min": min,
```

```
    "max": max,
}


# HELPERS for the base question
TOKEN = re.compile(r"-?\d+|[A-Za-z_]\w*|[(),]")


# FOLLOW-UP 3: What about division?
# Answer: State the contract: integer or float, truncate
# toward zero or floor (Python's // floors, which differs
# for negatives), and divide by zero handling. The
# implementation above truncates toward zero.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def divide(a, b):
    # integer division that truncates toward zero
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return int(a / b)


OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a ** b,
    "div": divide,
    "min": min,
    "max": max,
}


def evaluate_by_replacement(expr):
    pattern = re.compile(r"([A-Za-z_]\w*)\(([^()]*)\)")
    while True:
        m = pattern.search(expr)
        if not m:
            return int(expr.strip())
        args = [int(x) for x in m.group(2).split(",")]
        expr = (
            expr[: m.start()]
            + str(_OPS[m.group(1)](*args))
            + expr[m.end() :]
        )
```

```
# BASE QUESTION (optimal): Tokenizer + recursive descent
# over name(arg, arg, ...)
# Idea: Tokenize into identifiers, integers, commas and
# parentheses.
def evaluate(expr, env=None):
    env = env or {}
    tokens = TOKEN.findall(expr)

    def parse(i):
        # returns (value, index after this expression)
        tok = tokens[i]
        if re.fullmatch(r"-?\d+", tok):
            return int(tok), i + 1
        # FOLLOW-UP 4: variables come from env
        # a name not followed by "(" is a variable
```

```
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if tok not in env:
                raise ValueError(f"undefined {tok!r}")
            return env[tok], i + 1
        if tok not in OPS:
            raise ValueError(f"unknown function {tok!r}")
        i += 2
        args = []
        # read arguments until the closing bracket
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return OPS[tok](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value


# FOLLOW-UP 2: How do you make it iterative?
# Answer: Use two stacks: one of pending function names
# and one of lists of evaluated arguments.
def evaluate_iterative(expr):
    # two stacks: function names and their argument lists
    names, args, tokens = [], [[]], TOKEN.findall(expr)
```

```
    for tok in tokens:
        if re.fullmatch(r"-?\d+", tok):
            args[-1].append(int(tok))
        elif tok == "(":
            # the name just pushed now owns a new list
            args.append([])
        elif tok == ")":
            result = OPS[names.pop()](*args.pop())
            args[-1].append(result)
        elif tok != ",":
            names.append(tok)
    return args[0][0]


if __name__ == "__main__":
    assert evaluate("add(1, sub(1, 0))") == 2
    assert evaluate("add(2, mul(3, pow(4, 5)))") == 3074
    # variadic functions
    assert evaluate("add(1, 2, 3, 4)") == 10
    assert evaluate("max(3, min(9, 4), 1)") == 4
    # variables come from an environment
    assert evaluate("add(x, mul(x, 2))", {"x": 5}) == 15
    # division truncates toward zero
    assert evaluate("div(-7, 2)") == -3
    for bad in ("foo(1)", "add(1", "add(1) 2", "y"):
        try:
            evaluate(bad)
            raise AssertionError(bad)
        except (ValueError, IndexError):
            pass
    # the iterative version agrees and has no depth limit
    big = "add(2, mul(3, pow(4, 5)))"
    assert evaluate_iterative(big) == 3074
    deep = "add(1, " * 3000 + "0" + ")" * 3000
    assert evaluate_iterative(deep) == 3000
```

NOTES FROM CANDIDATES

Tokenizer + recursive descent is the cleanest approach: parse operator, consume (, parse comma-separated child expressions, consume), then evaluate.
Iterative parsing works but is easy to make off-by-one errors under interview pressure; one candidate was specifically flagged for minor syntax and bounds issues.
Clarify division behavior for negative values and non-divisible integers.

PREPARATION

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Stacks

MEDIUM | PracHub | Technical Screen | 32 of 33

Find longest increasing contiguous subarray

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 146/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Stacks

Optimal answer from: the interview site's own solution

LeetCode match: LC 674 Longest Continuous Increasing Subsequence (variant: one modification) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-longest-increasing-contiguous-subarray

QUESTION

Overview: This question evaluates proficiency in array processing and algorithmic optimization, focusing on recognition of contiguous subarray properties and handling a single-value modification to achieve strict monotonicity.

APPROACH

Compute run lengths ending at i (left) and starting at i (right); for each i try changing nums[i] to bridge: left[i-1] + 1 + right[i+1] if nums[i+1] - nums[i-1] >= 2. O(n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether increasing is strict, whether the modified value can be any integer or any real, and whether you must change exactly one element or at most one.

BRUTE FORCE TO SAY OUT LOUD

Try changing each element to every useful value and recompute the run. O(n^2).

OPTIMAL IDEA

Compute left[i], the length of the strictly increasing run ending at i, and right[i], the run starting at i. Changing nums[i] can bridge the neighbors if nums[i + 1] - nums[i - 1] >= 2 (integers), giving left[i - 1] + 1 + right[i + 1]. Otherwise you can only extend one side: left[i - 1] + 1 or right[i + 1] + 1. Take the maximum over all i, and also the run with no change. O(n).

DRY-RUN THESE CASES

n = 1, a strictly decreasing array, a plateau like [1, 2, 2, 3], neighbors that differ by exactly 1 (cannot bridge with integers), the change at the first or last index.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Allow k changes (needs a sliding window with at most k bad positions), allow non-strict increase, or return the subarray bounds.

PITFALLS

Forgetting the integer gap condition. Index errors at the array ends.

HOW TO THINK ABOUT IT

A run that has a single 'bad' spot can be repaired by changing one element, so look at each position i as the element to change. The best run through i joins the increasing run that ends just before it and the one that starts just after it, and the join is legal only if the values leave enough room between the two neighbours (for integers, a[i + 1] - a[i - 1] >= 2).

Basis: LeetCode 674 (Longest Continuous Increasing Subsequence) with one allowed modification. Assumed rule (confirm it): strictly increasing, you may change at most one element to any integer.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    best = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if fixable_with_one_change(part):
                best = max(best, len(part))
    return best
def fixable_with_one_change(part):
    bad = [k for k in range(1, len(part))
           if part[k] <= part[k - 1]]
    if not bad:
        return True
    if len(bad) > 1:
        return False
    return has_room(part, bad[0])
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays, each tested in O(n): O(n^3). Run lengths ending and starting at each index, then trying each index as the changed element, is O(n).

OPTIMAL SOLUTION · Left and right run lengths, try every position as the changed element

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 No modification allowed (LeetCode 674).

One pass keeping the length of the current strictly increasing run; reset to 1 when a[i] <= a[i - 1]. O(n), O(1).

FOLLOW-UP 2 Up to k modifications.

Use a sliding window over the array that allows at most k positions where the increasing property fails and cannot be repaired; careful with the integer room condition between neighbours. Often handled with DP over (position, changes used).

FOLLOW-UP 3 Values are real numbers, not integers.

Then there is always room between two different neighbours, so the join condition becomes a[i + 1] > a[i - 1] instead of a gap of at least 2.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find longest increasing contiguous
# subarray
#
# This question evaluates proficiency in array processing
# and algorithmic optimization, focusing on recognition of
# contiguous subarray properties and handling a single-
# value modification to achieve strict monotonicity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. No modification allowed (LeetCode 674).
# 2. Up to k modifications.
# 3. Values are real numbers, not integers.
#
# INTERVIEW NOTES
# A run that has a single 'bad' spot can be repaired by
# changing one element, so look at each position i as the
# element to change. The best run through i joins the
# increasing run that ends just before it and the one that
# starts just after it, and the join is legal only if the
# values leave enough room between the two neighbours (for
# integers, a[i + 1] - a[i - 1] >= 2).
#
# BRUTE FORCE (say it first): Test every subarray for
# 'increasing after at most one change'
# For each subarray count the places where an element is
# not larger than its predecessor, and check whether one
# change can fix it (one such spot and the neighbours
# leave room, or none). Exact and slow; the oracle for the
# O(n) method. Time O(n^3). Space O(1).


# BRUTE FORCE (say it first): Test every subarray for
# 'increasing after at most one change' | time O(n^3)
def longest_one_change_brute(a):
    n = len(a)
    best = 0
    for i in range(n):
        for j in range(i, n):
            sub = a[i : j + 1]
            ok = any(
                all(
                    x[k] < x[k + 1]
                    for k in range(len(x) - 1)
                )
                for x in (
                    [sub]
                    + [
                        sub[:k] + [v] + sub[k + 1 :]
                        for k in range(len(sub))
                        for v in range(
                            min(a) - len(a) - 2,
                            max(a) + len(a) + 3,
                        )
                    ]
                )
            )
            if ok:
                best = max(best, len(sub))
    return best


# BASE QUESTION (optimal): Left and right run lengths, try
# every position as the changed element
# Idea: left[i] is the length of the strictly increasing
# run ending at i and right[i] the length of the one
# starting at i.
def longest_one_change(a, integers=True):
    # strictly increasing after changing AT MOST one value
    n = len(a)
    if n == 0:
        return 0
    # left[i]: run ending at i; right[i]: run starting at
    # i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if a[i] > a[i - 1]:
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if a[i] < a[i + 1]:
            right[i] = right[i + 1] + 1
    # the gap needed between two neighbours: 2 for whole
    # numbers (one number must fit between), > 0 for reals
    best = max(left)
    for i in range(n):
        # change a[i] to join the run on its left ...
        if i > 0:
            best = max(best, left[i - 1] + 1)
        # ... or the run on its right ...
        if i < n - 1:
            best = max(best, right[i + 1] + 1)
        # ... or both, when a value fits between the
        # neighbours
        if 0 < i < n - 1:
            room = a[i + 1] - a[i - 1]
            fits = room >= 2 if integers else room > 0
            if fits:
                joined = left[i - 1] + 1 + right[i + 1]
                best = max(best, joined)
    return min(best, n)


# FOLLOW-UP 1: No modification allowed (LeetCode 674).
# Answer: One pass keeping the length of the current
# strictly increasing run; reset to 1 when a[i] <= a[i -
# 1]. O(n), O(1).
def longest_increasing(a):
    # LeetCode 674: no change allowed, one pass
    best = run = 0
    for i, x in enumerate(a):
        run = run + 1 if i and a[i - 1] < x else 1
        best = max(best, run)
    return best
```

```
if __name__ == "__main__":
    assert longest_increasing([1, 3, 5, 4, 7]) == 3
    assert longest_increasing([]) == 0
    assert longest_one_change([1, 3, 5, 4, 7]) == 5
    # equal neighbours leave no room for a whole number
    assert longest_one_change([1, 2, 2, 3]) == 3
    nums = [1, 2, 2, 3]
    reals = longest_one_change(nums, integers=False)
    assert reals == 4
    assert longest_one_change([5, 4, 3, 2, 1]) == 2
    assert longest_one_change([7]) == 1
    assert longest_one_change([]) == 0
    # never more than the array length
    assert longest_one_change([1, 2, 3]) == 3
```

FUNCTION AND TEST CASES

Signature

```
def longestIncreasing(nums: list[int]) -> int:
```

Sample call

```
longestIncreasing([1, 2, 2, 3, 4])
```

Test cases

Input	Expected
`([1, 2, 2, 3, 4],)`	`3`
`([1, 2, 3, 4, 5],)`	`5`

STUDY HERE: topic 1, Heaps, Stacks and Queues | main topic: Queues

Site tags: iterator, queue, data-structure, object-design, medium

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 33 of 33

Implement Python's itertools.tee

ASKED 2026-08-20 | 61 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 225/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues: Queues

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-20 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/python-tee-independent-iterators

QUESTION

Implement the behavior of Python's **`tee`**: accept one iterator and an integer **n**, then return **n** independent iterators over the same source sequence. Define the required function and classes yourself and improve a naive design toward an efficient shared-state implementation.SWEiteratorqueuedata-structureobject-designmediumFrequencySingle reportLast asked2026-08-20Stageonsite-coding

APPROACH

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the source iterator can be consumed by one consumer first, what happens if one consumer is far ahead, and whether thread safety is needed.

BRUTE FORCE TO SAY OUT LOUD

Store the whole source in a list and give each consumer an index. Simple, but defeats the purpose for large or infinite iterators.

OPTIMAL IDEA

Keep a deque per consumer. When a consumer needs an item and its deque is empty, pull the next item from the source and append it to every consumer's deque. Each item is held only until all consumers have read it. Build with a shared state object and a small iterator class with __iter__ and __next__. The linked-list variant uses one chain of nodes with each consumer holding a pointer, which frees nodes as the slowest consumer moves.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Two consumers at different speeds, n = 1, n = 0, an empty source, an exhausted source.

LIKELY FOLLOW-UPS

Thread safety (a lock around the pull), and memory growth if one consumer stalls.

PITFALLS

Pulling from the source once per consumer instead of once per item. Not appending to the other deques.

HOW TO THINK ABOUT IT

The source can only be consumed once, so someone must remember items until every consumer has read them. The state to keep is exactly 'items the slowest consumer has not read yet', so either give each consumer a queue that the leader fills, or keep one shared linked list with a pointer per consumer so memory is freed automatically.

Basis: The itertools.tee documentation's recipe; the linked-list variant is how CPython shares buffers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tee(source, n):
    items = list(source)  # read it all now
    def reader():
        for x in items:
            yield x
    return [reader() for _ in range(n)]
```

TIME	SPACE
O(n) up front	**O(n)**

WHY IT IS SLOW

It reads the source completely before anything is consumed, never works on an infinite or lazy source, and keeps every item in memory. One buffer per consumer (or a shared linked list) holds only what the slowest consumer has not read.

OPTIMAL SOLUTION · One deque per consumer, filled lazily

TIME	SPACE
O(n) total, O(1) amortized per item per consumer	**O(lag between fastest and slowest consumer)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Memory grows when one consumer stalls. Is there a way to free items automatically?

Use one shared singly linked list with a pointer per consumer. The only references to old nodes are the consumers' pointers, so Python's garbage collector frees nodes behind the slowest consumer without any bookkeeping. This is the design CPython uses.

FOLLOW-UP 2 Make it safe for several threads.

Pull from the source under one lock and append to the queues inside it. Without the lock two consumers can pull two items and append them in different orders to different queues. **Cost:** one lock acquisition per item pulled, O(1) extra per item; contention grows with the number of consumers.

FOLLOW-UP 3 Why not just copy the iterator?

Most iterators (files, generators, sockets) cannot be copied. tee gives independent views by buffering, which is the only thing you can do when the source cannot rewind.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Python's itertools.tee
#
# Implement the behavior of Python's tee: accept one
# iterator and an integer n, then return n independent
# iterators over the same source sequence. Define the
# required function and classes yourself and improve a
# naive design toward an efficient shared-state
# implementation.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Memory grows when one consumer stalls. Is there a way
# to free items automatically?
# 2. Make it safe for several threads.
# 3. Why not just copy the iterator?
#
# INTERVIEW NOTES
# The source can only be consumed once, so someone must
# remember items until every consumer has read them. The
# state to keep is exactly 'items the slowest consumer has
# not read yet', so either give each consumer a queue that
# the leader fills, or keep one shared linked list with a
# pointer per consumer so memory is freed automatically.
#
# BRUTE FORCE (say it first): Materialize the whole source
# into a list
# Read everything into a list and give each consumer an
# index. Correct for finite data, but it consumes the
# source eagerly, never works on an infinite iterator and
# holds all items in memory. Time O(n) up front. Space
# O(n).


import threading
from collections import deque
```

```
# FIRST IDEA (simplest): copy into a list
def tee_naive(iterable, n=2):
    # simplest idea: copy everything into a list
    items = list(iterable)
    # but it never works for endless iterators
    return tuple(iter(items) for _ in range(n))


# BASE QUESTION (optimal): One deque per consumer, filled
```

```
# lazily
# Idea: When a consumer's deque is empty it pulls the next
# item from the source and appends it to every consumer's
# deque, then pops its own.
def tee_queues(iterable, n=2):
    source = iter(iterable)
    # one waiting line of values per consumer
    queues = [deque() for _ in range(n)]

    def consumer(mine):
        while True:
            if not mine:
                try:
                    value = next(source)
                except StopIteration:
                    return
                # give the new value to everybody
                for q in queues:
                    q.append(value)
            yield mine.popleft()

    return tuple(consumer(q) for q in queues)


# FOLLOW-UP 1: Memory grows when one consumer stalls. Is
# there a way to free items automatically?
# Answer: Use one shared singly linked list with a pointer
# per consumer. The only references to old nodes are the
# consumers' pointers, so Python's garbage collector frees
# nodes behind the slowest consumer without any
# bookkeeping.
class Cell:
    # one link of the shared list
    def __init__(self):
        self.filled = False
        self.value = None
        self.next = None


# FOLLOW-UP 2: Make it safe for several threads.
# Answer: Pull from the source under one lock and append
# to the queues inside it. Without the lock two consumers
# can pull two items and append them in different orders
# to different queues.
def tee(iterable, n=2):
```

```
    source = iter(iterable)
    # one lock keeps the pulls in a single order
    lock = threading.Lock()

    def consumer(cell):
        while True:
            with lock:
                if not cell.filled:
                    # the first reader pulls the value
                    try:
                        cell.value = next(source)
                    except StopIteration:
                        return
                    cell.filled = True
                    cell.next = Cell()
                value, cell = cell.value, cell.next
            yield value
```

```
    # every reader starts at the same first cell
    head = Cell()
    return tuple(consumer(head) for _ in range(n))


if __name__ == "__main__":
    a, b = tee(range(5))
    # readers are independent, at any speed
    assert next(a) == 0 and next(a) == 1
    assert list(b) == [0, 1, 2, 3, 4]
    assert list(a) == [2, 3, 4]
    x, y, z = tee_queues("abc", 3)
    assert list(x) == list(y) == list(z) == list("abc")
    # works on an endless source, which tee_naive cannot
    def count():
        i = 0
        while True:
            yield i
            i += 1
    e1, e2 = tee(count())
    assert [next(e1) for _ in range(3)] == [0, 1, 2]
    assert next(e2) == 0
    assert [list(i) for i in tee_naive([1, 2], 2)] == [
        [1, 2], [1, 2]]
```

NOTES FROM CANDIDATES

The round depends on Python iterator-protocol fluency as much as general data-structure reasoning. Clarify laziness, source exhaustion, invalid n, and whether concurrent consumption is in scope before implementing.
A correct baseline consumes the source lazily once and gives each returned iterator its own queue: when one queue is empty, pull one source value, append it to every queue, then return it from the requesting queue. If n iterators trail across a gap of g values, however, those queues can retain O(n·g) buffered copies. For the optimal shared-state design, keep one linked buffer (or indexed deque) of source values plus one cursor per iterator, and discard a buffered value only after every cursor has passed it. That preserves independent consumption with O(g+n) auxiliary state, where g is the distance between the fastest and slowest consumers.

PREPARATION

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 2 OF 10 | COMMON TOPIC

Arrays, Hashing and Two Pointers

30 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2	24	4
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Arrays, Hashing and Two Pointers | 30 questions | tap a title to jump to it

HASHING (13)

E	1. Robot Status Message Deduplication Window	1Point3Acres	2026-09-10
E	2. Parity Alternating Subarray Query	HackTheRounds	no date
M	3. Connected Crop Arrangement on a Grid	1Point3Acres	2026-10-04
M	4. Next-Word Predictor	1Point3Acres	2026-09-02
M	5. Filter Repeated Robot Status Messages	PracHub	2026-07-05
M	6. Website Activity Analytics: Unique Users + Session Time	1Point3Acres	2026-06-26
M	7. Find a Horizontal Cut That Bisects Rectangle Area	PracHub	2026-06-04
M	8. Gold Chain Split into Two Equal Halves	1Point3Acres	2026-04-21
M	9. Solve array-sum and city-community problems	PracHub	2026-02-20
M	10. Check pivot after removing one element	PracHub	2026-02-12
M	11. Reconstruct original array from doubled shuffle	PracHub	2026-01-04
H	12. Maximum-Area Rectangle from Planar Points	1Point3Acres	2026-09-29
H	13. Smallest Missing Positive Integer in Linear Time and Constant Extra Space	PracHub	2026-09-09

TWO POINTERS AND SLIDING WINDOW (10)

M	14. Range Duplicate Check with Constant-Time Queries	1Point3Acres	2026-09-29
M	15. Temperature Monitoring Data Structure	1Point3Acres	2026-09-10
M	16. Piano Hand Movement Grouping	1Point3Acres	2026-07-14
M	17. Sum of Good Subarrays (Adjacent Diff ±1)	1Point3Acres	2026-02-08
M	18. Run-Length Vector Storage + Dot Product	1Point3Acres	2026-01-21
M	19. Maximize 1D deliveries within distance and minimize total distance	PracHub	2025-12-27
M	20. Two-Day Log Intersection under Memory Limit	1Point3Acres	2025-11-27
M	21. Shortest Subarray with K Distinct Integers	HackTheRounds	no date
H	22. Necklace Cut into Two Halves with Equal D/R Counts	1Point3Acres	2026-05-03
H	23. Find longest duplicated substring	PracHub	2025-12-08

ARRAYS AND PREFIX SUMS (7)

M	24. Longest Same-Color Grid Sequence	1Point3Acres	2026-09-08
M	25. Solve several coding interview problems	PracHub	2026-03-04
M	26. Min deletions to avoid overlap in first k	PracHub	2026-02-12
M	27. Add two big integers from digit lists	PracHub	2025-11-24
M	28. Minimize calls to find all bad test pairs	PracHub	2025-11-05
M	29. Rotate Any m×n 2D Matrix	1Point3Acres	no date
M	30. Sum of Two Squares in Two Ways	HackTheRounds	no date

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

Site tags: hashmap, simulation, easy

EASY | 1Point3Acres | onsite-coding | 1 of 30

Robot Status Message Deduplication Window

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 33** | **PRIORITY 34/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (related)

Frequency: Low | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

QUESTION

Process timestamped robot messages and display a message only when it has not been displayed within the prior 10 seconds. Hidden arrivals do not refresh the last-displayed timestamp, and the exact 10-second boundary must be clarified.

Process a stream of (time, message) status events from a robot.
If the same message was displayed within the past 10 seconds, hide the new event.
Otherwise display it and update that message's last-displayed time.
Track the last time each message was displayed, not the last time it was received. A hidden duplicate must not refresh the window.
Clarify whether an event exactly 10 seconds after the prior display is allowed: the comparison may be > 10 or >= 10.

Logger API variant

Implement shouldPrintMessage(timestamp, message), returning true when the message has not been printed in the past 10 seconds and false otherwise.
Handle timestamps arriving out of order, including timestamp 12 arriving before timestamp 10.

EXAMPLES

```
time=10, message="solar panel activated" -> show
time=13, message="solar panel activated" -> hide
time=21, message="solar panel activated" -> show
```

The event at 21 compares against the display at 10, not the hidden arrival at 13.

APPROACH

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

HOW TO THINK ABOUT IT

The state that matters is the last time each message was displayed, not the last time it arrived. A message that is hidden must not move that time, otherwise a message repeated every few seconds would be hidden forever. A hash map from message to last displayed time answers each event in O(1).

Basis: LeetCode 359, Logger Rate Limiter (a message may be shown again only after the window has passed).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message) displayed
def on_event(t, msg):
    for t0, m0 in shown:
        if m0 == msg and t - t0 < 10:
            return False  # hidden
    shown.append((t, msg))
    return True  # displayed
```

TIME	SPACE
O(displayed) per event	O(displayed)

WHY IT IS SLOW

Each event scans the whole history. A map from message to its last displayed time answers each event in O(1).

OPTIMAL SOLUTION · Hash map of last displayed time

TIME	SPACE
O(1) per event	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Events can arrive out of order.

Decide the contract first: either a late event is judged against the stored time with abs(t - last), or you keep a sorted history per message and compare with the nearest displayed neighbours on both sides. The simple map is only correct for a chronological stream.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Robot Status Message Deduplication Window
#
# Process timestamped robot messages and display a message
# only when it has not been displayed within the prior 10
# seconds. Hidden arrivals do not refresh the last-
# displayed timestamp, and the exact 10-second boundary
# must be clarified. Process a stream of (time, message)
# status events from a robot. If the same message was
# displayed within the past 10 seconds, hide the new
# event. Otherwise display it and update that message's
# last-displayed time. Track the last time each message
# was displayed, not the last time it was received. A
# hidden duplicate must not refresh the window. Clarify
# whether an event exactly 10 seconds after the prior
# display is allowed: the comparison may be > 10 or >= 10.
# Logger API variant Implement
# shouldPrintMessage(timestamp, message), returning true
# when the message has not been printed in the past 10
# seconds and false otherwise.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Events can arrive out of order.
#
# INTERVIEW NOTES
# The state that matters is the last time each message was
# displayed, not the last time it arrived. A message that
# is hidden must not move that time, otherwise a message
# repeated every few seconds would be hidden forever. A
# hash map from message to last displayed time answers
# each event in O(1).
#
# BRUTE FORCE (say it first): Keep every displayed time
# and scan them
# Store the list of every (time, message) shown so far and
# scan it for the same message within the window. Correct,
# but the scan grows with the whole history. Time
# O(displayed) per event. Space O(displayed).


from bisect import bisect_left, bisect_right, insort


# BRUTE FORCE (say it first): Keep every displayed time
# and scan them | time O(displayed) per event
class DedupBrute:
    def __init__(self, window=10):
        self.window, self.shown = window, []

    def show(self, t, msg):
        if any(
            m == msg and t - s < self.window
            for s, m in self.shown
        ):
            return False
        self.shown.append((t, msg))
        return True


# BASE QUESTION (optimal): Hash map of last displayed time
# Idea: If the message was displayed less than `window`
# seconds ago hide it without changing the stored time;
# otherwise display it and store the new time.
class Dedup:
    """Chronological stream: remember the last display."""

    def __init__(self, window=10, boundary_allowed=True):
        self.window, self.last = window, {}
        # True: exactly `window` seconds later may show
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        # is this gap still inside the hiding window?
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        # only a displayed event starts a new window
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True


# FOLLOW-UP 1: Events can arrive out of order.
# Answer: Decide the contract first: either a late event
# is judged against the stored time with abs(t - last), or
# you keep a sorted history per message and compare with
# the nearest displayed neighbours on both sides.
class DedupOutOfOrder:
    """Events may arrive late: look at both neighbours."""

    def __init__(self, window=10):
        self.window = window
        # message -> sorted list of displayed times
        self.shown = {}

    def show(self, t, msg):
        times = self.shown.setdefault(msg, [])
        # any display within `window` on either side?
        lo = bisect_right(times, t - self.window)
        hi = bisect_left(times, t + self.window)
        if lo < hi:
            return False
        insort(times, t)
        return True


if __name__ == "__main__":
    d = Dedup()
    # 13 is hidden and does not move the window
    assert [d.show(t, "solar") for t in (10, 13, 21)] == [
        True, False, True]
    # exactly 10 seconds later shows by default
    d = Dedup()
    d.show(0, "a")
    assert d.show(10, "a")
    # ... but is hidden if the boundary is not allowed
    d = Dedup(boundary_allowed=False)
    d.show(0, "a")
    assert not d.show(10, "a")
    o = DedupOutOfOrder()
    assert o.show(20, "x")
    # a late event close to a displayed one is hidden
    assert not o.show(15, "x")
    # a late event far enough from every display shows
    assert o.show(5, "x")
```

NOTES FROM CANDIDATES

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.

For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

PREPARATION

Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand.
Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

EASY | HackTheRounds | Onsite | 2 of 30

Parity Alternating Subarray Query

STUDY: TALK | ALL TIME 34.9 | 30 DAYS 0 | PRIORITY 112/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #303 Range Sum Query - Immutable

LeetCode match: No direct match; prefix-sum range queries (idea of LC 303)

QUESTION

Given an array nums and multiple queries, each query asks whether a subarray nums[l:r] satisfies the condition that adjacent elements alternate in parity (odd-even-odd-even or even-odd-even-odd). Preprocess the array to answer each query in O(1) time.

EXAMPLES

Example 1	
Input	`nums = [1, 2, 3, 4, 5], queries = [[0, 4], [1, 3]]`
Output	`[true, true]`
Why	`All adjacent pairs alternate in parity.`

Example 2	
Input	`nums = [1, 3, 2, 4], queries = [[0, 1], [0, 3]]`
Output	`[false, false]`
Why	`nums[0]=1 and nums[1]=3 are both odd.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= queries.length <= 10^5
- 0 <= l <= r < nums.length

APPROACH

Prefix count of adjacent pairs with equal parity; a query [l, r] alternates iff the count over (l..r-1) is 0. O(1) per query.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the query range is inclusive or exclusive (the statement writes nums[l:r]), whether a single element counts as alternating, and the number of queries.

BRUTE FORCE TO SAY OUT LOUD

For each query scan the range checking adjacent parities. O(length) per query.

OPTIMAL IDEA

Precompute a prefix count B where B[i] is the number of adjacent pairs (j, j + 1) with j < i whose parities are equal. A subarray from l to r (inclusive) alternates exactly when the number of equal-parity pairs among positions l to r - 1 is zero, i.e. B[r]

- B[l] == 0. O(n) preprocessing, O(1) per query.

DRY-RUN THESE CASES

A single-element range, a range with two equal-parity neighbors, the whole array, l equal r, negative numbers (use x % 2 carefully).

LIKELY FOLLOW-UPS

Update an element (Fenwick tree), or return the longest alternating subarray.

PITFALLS

Off-by-one on the inclusive end. Parity of negative numbers in languages where % can be negative.

HOW TO THINK ABOUT IT

A subarray alternates in parity exactly when it contains no pair of neighbours with the same parity. 'Count of bad pairs in a range' is a prefix sum question, so precompute once and answer every query with one subtraction. Mark each position i by whether (nums[i], nums[i + 1]) is a bad pair.

Basis: Prefix counts for range queries (compare LeetCode 303). Assumed rule: a query (l, r) is inclusive on both ends; a single element is alternating.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def alternates(nums, l, r):
    for i in range(l, r):
        if nums[i] % 2 == nums[i + 1] % 2:
            return False
    return True
```

TIME	SPACE
O(length) per query	O(1)

WHY IT IS SLOW

Each query scans up to n elements. A prefix count of equal-parity neighbour pairs answers every query in O(1) after one O(n) pass.

OPTIMAL SOLUTION · Prefix count of equal-parity neighbour pairs

TIME	SPACE
O(n) to build, O(1) per query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array also changes (point updates).

Keep the bad-pair indicators in a Fenwick tree: an update changes at most two indicators, each an O(log n) update, and a query is two prefix sums.

FOLLOW-UP 2 Find the longest alternating subarray.

One pass with a running length that resets to 1 on a bad pair. O(n), O(1).

FOLLOW-UP 3 Offline queries, different property (for example all elements distinct).

The prefix-sum trick applies whenever the property is 'no violating neighbour pair'. Properties about distinctness need Mo's algorithm or an offline sweep with last-occurrence tracking. **Cost:** Mo's algorithm is O((n + q) sqrt(n)); the prefix trick for 'no violating neighbour pair' stays O(n + q).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Parity Alternating Subarray Query
#
# Given an array nums and multiple queries, each query
# asks whether a subarray nums[l:r] satisfies the
# condition that adjacent elements alternate in parity
# (odd-even-odd-even or even-odd-even-odd). Preprocess the
# array to answer each query in O(1) time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array also changes (point updates).
# 2. Find the longest alternating subarray.
# 3. Offline queries, different property (for example all
# elements distinct).
#
# INTERVIEW NOTES
# A subarray alternates in parity exactly when it contains
# no pair of neighbours with the same parity. 'Count of
# bad pairs in a range' is a prefix sum question, so
# precompute once and answer every query with one
# subtraction. Mark each position i by whether (nums[i],
# nums[i + 1]) is a bad pair.
#
# BRUTE FORCE (say it first): Scan the subarray for every
# query
# Check each adjacent pair in nums[l..r] for equal parity.
# Correct, and every query repeats work that other queries
# already did. Time O(length) per query. Space O(1).
def brute(nums, l, r):
    return all(nums[i] % 2 != nums[i + 1] % 2
               for i in range(l, r))


# BRUTE FORCE (say it first): Scan the subarray for every
# query | time O(length) per query
def alternating_brute(nums, l, r):
    return all(
        nums[i] % 2 != nums[i + 1] % 2
        for i in range(l, r)
    )


# HELPERS for follow-up 1
class Fenwick:
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i, delta):
        i += 1
        while i <= self.size:
            self.tree[i] += delta
            i += i & -i

    def prefix(self, i):
        # sum of the first i items
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total


# BASE QUESTION (optimal): Prefix count of equal-parity
# neighbour pairs
# Idea: bad[i] is the number of positions j < i with
# nums[j] and nums[j + 1] of the same parity. The pairs
# inside nums[l..r] are j = l .. r - 1, so the subarray
# alternates when bad[r] - bad[l] == 0.
class ParityQueries:
    """Static array. bad[i] = number of neighbouring pairs
    with equal parity among the first i pairs. A range
    alternates when it contains no such pair."""

    def __init__(self, nums):
        self.bad = [0]
        for a, b in zip(nums, nums[1:]):
            same = a % 2 == b % 2
            self.bad.append(self.bad[-1] + same)

    def alternates(self, l, r):
        # pairs l .. r - 1 are inside the range [l, r]
        return self.bad[r] - self.bad[l] == 0


# FOLLOW-UP 1: The array also changes (point updates).
# Answer: Keep the bad-pair indicators in a Fenwick tree:
# an update changes at most two indicators, each an O(log
# n) update, and a query is two prefix sums.
class ParityWithUpdates:
    """Follow-up: point updates. The equal-parity flags
    live in a Fenwick tree; changing one number changes
    at most two flags, and a query is two prefix sums."""

    def __init__(self, nums):
        self.nums = list(nums)
        self.flags = [0] * max(len(nums) - 1, 0)
```

```
        self.tree = Fenwick(len(self.flags))
        for i in range(len(self.flags)):
            self.refresh(i)

    def refresh(self, i):
        # recompute the flag of the pair (i, i + 1)
        a, b = self.nums[i], self.nums[i + 1]
        now = int(a % 2 == b % 2)
        self.tree.add(i, now - self.flags[i])
        self.flags[i] = now

    def update(self, i, value):
        self.nums[i] = value
        for pair in (i - 1, i):
            if 0 <= pair < len(self.flags):
                self.refresh(pair)

    def alternates(self, l, r):
        inside = self.tree.prefix(r) - self.tree.prefix(l)
        return inside == 0


# FOLLOW-UP 2: Find the longest alternating subarray.
# Answer: One pass with a running length that resets to 1
# on a bad pair. O(n), O(1).
def longest_alternating(nums):
    # follow-up: one pass with a running length that
    # restarts at 1 after a bad pair
    best = run = 1 if nums else 0
    for a, b in zip(nums, nums[1:]):
        run = run + 1 if a % 2 != b % 2 else 1
        best = max(best, run)
    return best


if __name__ == "__main__":
    q = ParityQueries([1, 2, 3, 4, 5])
```

```
    assert q.alternates(0, 4) and q.alternates(1, 3)
    q = ParityQueries([1, 3, 2, 4])
    assert not q.alternates(0, 1)
    assert not q.alternates(0, 3)
    assert q.alternates(1, 2)
    # a single element always alternates
    assert q.alternates(2, 2)
    assert longest_alternating([1, 3, 2, 4]) == 2
    assert longest_alternating([1, 2, 3, 4, 5]) == 5
    assert longest_alternating([]) == 0
    assert longest_alternating([7]) == 1
    import random
    for _ in range(200):
        nums = [random.randint(0, 9)
                for _ in range(random.randint(1, 10))]
        fixed = ParityQueries(nums)
        live = ParityWithUpdates(nums)
        for _ in range(20):
            if random.random() < 0.4:
                i = random.randrange(len(nums))
                nums[i] = random.randint(0, 9)
                live.update(i, nums[i])
            l = random.randrange(len(nums))
            r = random.randrange(l, len(nums))
            want = brute(nums, l, r)
            assert live.alternates(l, r) == want
        fixed = ParityQueries(nums)
        l = random.randrange(len(nums))
        r = random.randrange(l, len(nums))
        assert fixed.alternates(l, r) == brute(nums, l, r)
```

ALSO REPORTED · 2 more

Answer parity-alternation range queries | PracHub | Technical Screen

https://prachub.com/coding-questions/answer-parity-alternation-range-queries

Answer range queries for alternating parity subarrays | PracHub | Technical Screen

https://prachub.com/coding-questions/answer-range-queries-for-alternating-parity-subarrays

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

Site tags: grid, array, greedy, medium

MEDIUM | 1Point3Acres | phone-screen | 3 of 30

Connected Crop Arrangement on a Grid

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 207/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/connected-crop-arrangement-grid

QUESTION

Given an N x M grid and K crop counts summing to N*M, fill every cell so each crop's cells form one 4-connected region; return any valid arrangement. Follow-up: two adjacent square gardens (N x N and M x M) with counts summing to N^2 + M^2.

A garden is an N x M grid. There are K crop types labeled 1..K; crop i must occupy exactly count[i] cells.
Guaranteed: sum(count) = N * M, so every cell is used.
For every crop type, all of its cells must form one connected component under 4-directional adjacency (up, down, left, right; diagonals do not count).
Different crops have no relationship constraints with each other.
Return any valid arrangement.

Follow-up: two square gardens

Instead of one rectangle, there are two square gardens placed directly next to each other: garden A is N x N, garden B is M x M; together they form the planting area.
Now sum(count) = N^2 + M^2. The same single-connected-region rule applies across the combined shape.

EXAMPLES

Follow-up shapes for N = 2, M = 3 (digits mark garden A = 1, garden B = 2):

```
11222
11222
  222
```

or

```
11
11
222
222
222
```

APPROACH

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

HOW TO THINK ABOUT IT

Walk the grid in a snake (left to right on one row, right to left on the next) so that every cell is next to the one before it. If you give crop 1 the first count[1] cells of that walk, crop 2 the next count[2] cells and so on, each crop is a contiguous stretch of the walk, so its cells are connected. No search or backtracking is needed.

Basis: Written for this question: a constructive fill along a Hamiltonian path of the grid (a snake), no LeetCode problem matches it exactly.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def fill(crop):
    if crop == len(counts):
        return True
    size = counts[crop]
    for cells in combinations(free_cells, size):
        if is_connected(cells):
            place(cells, crop)
            if fill(crop + 1):
                return True
            remove(cells)
    return False
```

TIME	SPACE
exponential	O(n * m)

WHY IT IS SLOW

Exponential in the number of cells. Filling the crops along a snake path through the grid is O(n * m) and every crop stays connected by construction.

OPTIMAL SOLUTION · Fill crops along a snake path

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two adjacent square gardens (N x N next to M x M).

Use a different Hamiltonian path of the combined L-shaped area: snake down and up the columns of the first garden so that the last column ends in the top row, step into the second garden there and snake through it. Hand out the crop counts along that path exactly as before. The code checks that this path visits every cell once with each step adjacent, for every size from 1 to 6.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Connected Crop Arrangement on a Grid
#
# Given an N x M grid and K crop counts summing to N*M,
# fill every cell so each crop's cells form one
# 4-connected region; return any valid arrangement.
# Follow-up: two adjacent square gardens (N x N and M x M)
# with counts summing to N^2 + M^2. A garden is an N x M
# grid. There are K crop types labeled 1..K; crop i must
# occupy exactly count[i] cells. Guaranteed: sum(count) =
# N * M, so every cell is used. For every crop type, all
# of its cells must form one connected component under
# 4-directional adjacency (up, down, left, right;
# diagonals do not count). Different crops have no
# relationship constraints with each other. Return any
# valid arrangement. Follow-up: two square gardens Instead
# of one rectangle, there are two square gardens placed
# directly next to each other: garden A is N x N, garden B
# is M x M; together they form the planting area. Now
# sum(count) = N^2 + M^2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two adjacent square gardens (N x N next to M x M).
#
# INTERVIEW NOTES
# Walk the grid in a snake (left to right on one row,
# right to left on the next) so that every cell is next to
# the one before it. If you give crop 1 the first count[1]
# cells of that walk, crop 2 the next count[2] cells and
# so on, each crop is a contiguous stretch of the walk, so
# its cells are connected. No search or backtracking is
# needed.
#
# BRUTE FORCE (say it first): Backtrack over every
# assignment
# Try every way to place the counts and test connectivity.
# Correct and only possible on tiny grids, which makes it
# a good oracle for the constructive answer. Time
# exponential. Space O(n * m).


import itertools
from collections import defaultdict


# BRUTE FORCE (say it first): Backtrack over every
```

```
# assignment | time exponential
def crops_brute_possible(n, m, counts):
    cells = [(i, j) for i in range(n) for j in range(m)]

    def connected(group):
        group = set(group)
        if not group:
            return True
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack += [
                x
                for x in (
                    (r + 1, c),
                    (r - 1, c),
                    (r, c + 1),
                    (r, c - 1),
                )
                if x in group
            ]
        return seen == group

    def go(idx, remaining):
        if idx == len(counts):
            return True
        for pick in itertools.combinations(
            remaining, counts[idx]
        ):
            if connected(pick) and go(
                idx + 1,
                [c for c in remaining if c not in pick],
            ):
                return True
        return False

    return go(0, cells)


# HELPERS for the base question
def snake_cells(n, m):
    # row 0 left to right, row 1 right to left, ...
    return [(i, j if i % 2 == 0 else m - 1 - j)
            for i in range(n) for j in range(m)]


def fill_along(grid, path, counts):
    # hand out the path to crop 1, crop 2, ... in turn
    k = 0
    for crop, cnt in enumerate(counts, 1):
        for _ in range(cnt):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid


# HELPERS for follow-up 1
def two_gardens_path(n, m):
    # garden A is n x n; garden B is m x m on its right
    path = []
    # snake down and up the columns of garden A
    for c in range(n):
        # the last column must end in row 0
        down = (c % 2 == 0) == (n % 2 == 0)
        rows = range(n)
        if not down:
            rows = range(n - 1, -1, -1)
        path += [(r, c) for r in rows]
    # step into garden B at row 0 and snake through it
    for r in range(m):
        cols = range(n, n + m) if r % 2 == 0 \
            else range(n + m - 1, n - 1, -1)
        path += [(r, c) for c in cols]
    return path
```

```python
# BASE QUESTION (optimal): Fill crops along a snake path
# Idea: Generate the snake order of the cells and hand out
# count[0] cells to crop 1, the next count[1] to crop 2,
# and so on. Consecutive cells of the snake share an edge,
# so each crop is connected.
def arrange_crops(n, m, counts):
    # every cell is used exactly once
    assert sum(counts) == n * m
    grid = [[0] * m for _ in range(n)]
    # touching path cells keep each crop connected
```

```python
    return fill_along(grid, snake_cells(n, m), counts)


# FOLLOW-UP 1: Two adjacent square gardens (N x N next to
# M x M).
# Answer: Use a different Hamiltonian path of the combined
# L-shaped area: snake down and up the columns of the
# first garden so that the last column ends in the top
# row, step into the second garden there and snake through
# it.
def arrange_two_gardens(n, m, counts):
    assert sum(counts) == n * n + m * m
    # the grid is wide enough for both gardens
    grid = [[0] * (n + m) for _ in range(max(n, m))]
    path = two_gardens_path(n, m)
    return fill_along(grid, path, counts)


# HELPERS for the checks below
def crops_valid(grid, counts):
    cells = defaultdict(set)
    for i, row in enumerate(grid):
        for j, v in enumerate(row):
            if v:
                cells[v].add((i, j))
    for crop, cnt in enumerate(counts, 1):
        group = cells[crop]
        # the right number of cells
        if len(group) != cnt:
            return False
        # flood fill from one cell must reach them all
        stack, seen = [next(iter(group))], set()
        while stack:
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            for x in ((r + 1, c), (r - 1, c),
                      (r, c + 1), (r, c - 1)):
                if x in group:
                    stack.append(x)
        if seen != group:
            return False
    return True
```

```python
if __name__ == "__main__":
    # a 2 x 3 field with two crops
    g = arrange_crops(2, 3, [2, 4])
    assert crops_valid(g, [2, 4])
    # many tiny crops still connect
    ones = [1] * 9
    assert crops_valid(arrange_crops(3, 3, ones), ones)
    # two gardens: 2 x 2 next to 3 x 3 (13 cells)
    counts = [5, 4, 4]
    g2 = arrange_two_gardens(2, 3, counts)
    assert crops_valid(g2, counts)
    # the path visits every cell once, step by step
    p = two_gardens_path(2, 3)
    assert len(set(p)) == 13
```

```python
assert all(abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1
           for a, b in zip(p, p[1:]))
```

NOTES FROM CANDIDATES

Asked as the first-round online coding interview of an early-career L3 loop.
Because any valid arrangement is accepted, look for a constructive traversal order that keeps consecutive cells adjacent, rather than backtracking.
For the follow-up, the two gardens can be stacked side by side or one above the other; clarify how they are joined (which edges touch and the offset) before designing the traversal.

PREPARATION

Write a traversal over an N x M grid in which every consecutive pair of visited cells shares an edge, then fill crops in order along it; argue why every crop's cells stay connected.
Extend the same traversal to an L-shaped region made of two squares, handling the transition cell between gardens.
Test with K = 1, many crops of size 1, a single row or column, and both follow-up layouts above.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

Site tags: string, hashmap, data-structure

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 4 of 30

Next-Word Predictor

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 224/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

QUESTION

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.SWEstringhashmapdata-structureFrequencySingle reportLast asked2026-09-02Stageonsite-coding

APPROACH

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.

Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how sentences are split and cased, how ties are broken, what to return for an unseen word, and whether training is incremental.

BRUTE FORCE TO SAY OUT LOUD

Store all sentences and scan them for each prediction. O(corpus) per query.

OPTIMAL IDEA

Keep a dict from a word to a dict of next-word counts, and a separate best[word] updated whenever a count changes (if the new count beats the current best, or ties and wins the tie-break). Prediction is a dict lookup, O(1). Training is O(total words).

DRY-RUN THESE CASES

A repeated pair, a tie, a new leader taking over, a word with no successor, the last word of a sentence.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Top-k predictions (keep a heap per word), bigram versus trigram context, or decay for recent data.

PITFALLS

Updating best only on strict increase and ignoring the tie rule. Counting across sentence boundaries.

HOW TO THINK ABOUT IT

Prediction in O(1) means the answer must already be sitting in a table when the query arrives, so do the work while training. Keep counts per (word, next word) and, for each word, the current best successor. Each count only increases by one, so the best can be updated by comparing just the one count that changed.

Basis: Written; a bigram language model. Assumed rules (confirm them): words are split on spaces and compared case-sensitively; pairs never cross sentence boundaries; ties go to the alphabetically smaller word; an unseen word predicts None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
sentences = []
def train(sentence):
    sentences.append(sentence.split())
def predict(word):
    counts = Counter()
    for words in sentences:
        for a, b in zip(words, words[1:]):
            if a == word:
                counts[b] += 1
    if not counts:
        return None
    return max(counts, key=counts.get)
```

TIME	SPACE
O(corpus size) per prediction	**O(corpus size)**

WHY IT IS SLOW

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

OPTIMAL SOLUTION · Counts plus a maintained best successor

TIME	SPACE
O(1) per pair when training and per prediction	**O(number of distinct pairs)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

FOLLOW-UP 2 Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

FOLLOW-UP 3 The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Next-Word Predictor
#
# Process training data and predict the most frequent next
# word for a supplied word. Maintain prediction state
# during processing so prediction can run in constant
# time.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the top K predictions.
# 2. Use two words of context (trigrams).
# 3. The corpus keeps changing and old text should fade.
#
# INTERVIEW NOTES
# Prediction in O(1) means the answer must already be
# sitting in a table when the query arrives, so do the
# work while training. Keep counts per (word, next word)
# and, for each word, the current best successor. Each
# count only increases by one, so the best can be updated
# by comparing just the one count that changed.
#
# BRUTE FORCE (say it first): Scan the whole corpus on
# every prediction
# Keep the sentences and count successors of the query
# word from scratch on each call. Simple and correct, but
# every query is as slow as one full pass over the data.
# Time O(corpus size) per prediction. Space O(corpus
# size).


import heapq
from collections import Counter, defaultdict


# BRUTE FORCE (say it first): Scan the whole corpus on
# every prediction | time O(corpus size) per prediction
class PredictorBrute:
    def __init__(self):
        self.sentences = []

    def train(self, sentence):
        self.sentences.append(sentence.split())

    def predict(self, word):
        c = Counter(
            s[i + 1]
            for s in self.sentences
            for i in range(len(s) - 1)
            if s[i] == word
        )
        return (
            min(c, key=lambda w: (-c[w], w))
            if c
            else None
        )


# BASE QUESTION (optimal): Counts plus a maintained best
# successor
# Idea: counts[word][next] is incremented for every
# adjacent pair. After the increment compare that one
# count with the current best for the word (higher count
# wins, then the smaller word).
# FOLLOW-UP 1: Return the top K predictions.
# Answer: Keep the Counter per word and use
# heapq.nsmallest on (-count, word): O(d log K) per query,
# or keep a sorted structure per word if queries dominate.
# FOLLOW-UP 2: Use two words of context (trigrams).
# Answer: The key becomes the pair (previous two words)
# and everything else is identical, at the price of more
# memory. Back off to the bigram table when the pair was
# never seen.
class Predictor:
    # FOLLOW-UP 2: orders=(2, 1) means two words of
    # context
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
                       for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)

    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None

    # FOLLOW-UP 1: the top K predictions
    def top_k(self, word, k):
        # k most likely next words after one word
```

```
        by_ctx = self.counts.get(1, {})
        table = by_ctx.get((word,), Counter())
        return heapq.nsmallest(
            k, table, key=lambda w: (-table[w], w))


if __name__ == "__main__":
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    # "love" is followed by cats twice, dogs once
    assert p.predict("love") == "cats"
    # two words of context are tried first
    assert p.predict("i", "love") == "cats"
    # an unseen pair backs off to the last word
    assert p.predict("we", "love") == "cats"
    assert p.predict("zebra") is None
    assert p.top_k("love", 2) == ["cats", "dogs"]
    # a tie goes to the alphabetically smaller word
    q = Predictor()
    q.train("a z")
```

```
    q.train("a b")
    assert q.predict("a") == "b"
```

NOTES FROM CANDIDATES

Exact method signatures, tokenization rules, frequency ties, and behavior for an unseen word are unspecified.
The candidate used a dictionary and stored the current best successor during processing. Calling this state a cache caused a misunderstanding: the interviewer initially thought updates would happen during prediction. Explain the update timing explicitly.
After agreeing on tokenization and sequence boundaries, increment a counter for each adjacent pair (previous_word, next_word). For each previous word, retain a successor whose count is maximal. Compare the updated successor with the retained best immediately during processing; prediction only reads that stored best.
With insertion-only counts, a pair whose count did not change cannot newly overtake the best. Under a practice convention that retains the incumbent on ties, replace the best only when the updated count is strictly larger. If a different tie rule is required, apply it explicitly on equal counts.
Hash-table updates per observed pair and prediction are expected O(1), excluding tokenization/string hashing costs. Store O(P + V) state for P distinct pairs and V predecessor words. Clarify unseen-word results and whether separate training sequences create cross-boundary pairs.

PREPARATION

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

MEDIUM | PracHub | Technical Screen | 5 of 30

★ TOP 75: rank 51 in the Medium and Hard priority list

★ Filter Repeated Robot Status Messages

POSTED 2026-07-05 | 107 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 33** | **PRIORITY 32/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter

LeetCode match: LC 359 Logger Rate Limiter (inferred from overview)

Posted on PracHub: 2026-07-05 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/filter-repeated-robot-status-messages

QUESTION

Overview: Filter a timestamped robot status stream so a message reappears only when strictly more than ten seconds have passed since its last displayed occurrence. Preserve input order, track each exact case-sensitive message independently, and ensure hidden repeats do not reset the interval.

APPROACH

Hash map message -> last displayed timestamp; display only if now - last > 10 (strictly), then update. Preserve input order. O(1) per message.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact rule: show a message again only if more than 10 seconds have passed since it was last displayed (strictly more), whether messages are compared case-sensitively (yes), and whether hidden repeats extend the interval (no).

BRUTE FORCE TO SAY OUT LOUD

For each message scan the already displayed ones for the last occurrence. O(n^2).

OPTIMAL IDEA

Keep a dict from message to the time it was last displayed. For each message in order: if it is not in the dict or now - last > 10, display it and set last = now; otherwise skip it and do not update last. Output preserves input order. O(1) per message.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

A repeat at exactly 10 seconds (hidden), at 11 seconds (shown), a hidden repeat followed by another repeat 10 seconds after the first display (shown, since hidden repeats do not reset), different casing (different messages), the first message.

LIKELY FOLLOW-UPS

Bounded memory (evict messages older than the interval), or a per-robot map.

PITFALLS

Updating the last time on a hidden message. Using at least instead of strictly more than.

HOW TO THINK ABOUT IT

The decision for a message depends only on when that exact message was last displayed, so a dictionary from message to last display time is all the state you need. The detail that decides correctness: the stored time changes only when a message is shown (a hidden repeat must not reset the clock), and the comparison is strict (> 10).

Basis: Written (compare LeetCode 359, Logger Rate Limiter). Rule: show a message again only when strictly more than 10 seconds have passed since it was last shown; hidden repeats do not extend the interval; messages are case-sensitive; input order is kept.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
shown = []  # (time, message)
def on_message(t, msg):
    last = None
    for t0, m0 in shown:
        if m0 == msg:
            last = t0  # most recent display
    if last is None or t - last > 10:
        shown.append((t, msg))
        return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each message searches the whole history, O(n^2). A dictionary from message to its last displayed time makes each decision O(1).

OPTIMAL SOLUTION · Dictionary from message to the time it was last shown

TIME	SPACE
O(n)	O(distinct messages)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream never ends; memory must be bounded.

A message older than the gap can never be blocked again, so evict it. Keep a min-heap of (time shown, message) and pop entries older than the gap before each decision; delete the dictionary key only if it still points at that time.

FOLLOW-UP 2 Several robots send messages.

Use (robot id, message) as the key, or one filter per robot. The same logic applies per key.

FOLLOW-UP 3 The interval depends on the message severity.

Look up the gap from a severity table and use it in the comparison; the state (last shown time) is unchanged.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Filter Repeated Robot Status Messages
#
# Filter a timestamped robot status stream so a message
# reappears only when strictly more than ten seconds have
# passed since its last displayed occurrence. Preserve
# input order, track each exact case-sensitive message
# independently, and ensure hidden repeats do not reset
# the interval.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream never ends; memory must be bounded.
# 2. Several robots send messages.
# 3. The interval depends on the message severity.
#
# INTERVIEW NOTES
# The decision for a message depends only on when that
# exact message was last displayed, so a dictionary from
# message to last display time is all the state you need.
# The detail that decides correctness: the stored time
# changes only when a message is shown (a hidden repeat
# must not reset the clock), and the comparison is strict
# (> 10).
#
# BRUTE FORCE (say it first): Scan the already shown
# messages for each new one
# Keep the list of shown (time, message) pairs and, for
# each new message, look backward for its last shown time.
# Correct and the oracle; each message searches the whole
# history. Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Scan the already shown
# messages for each new one | time O(n^2)
def filter_status_brute(stream):
    shown, out = [], []
    for ts, msg in stream:
        last = next(
            (t for t, m in reversed(shown) if m == msg),
            None,
        )
        if last is None or ts - last > 10:
            shown.append((ts, msg))
            out.append((ts, msg))
    return out


# BASE QUESTION (optimal): Dictionary from message to the
# time it was last shown
# Idea: For each message in order: if it was never shown
# or now - last > 10, emit it and set last = now;
# otherwise drop it and leave last unchanged. One
# dictionary lookup per message.
def filter_status(stream, gap=10):
    # show a message only if MORE than `gap` seconds
    # passed
    # since it was last shown; hidden repeats change
    # nothing
    last, shown = {}, []
    for ts, msg in stream:
        if msg not in last or ts - last[msg] > gap:
            last[msg] = ts
            shown.append((ts, msg))
    return shown


# FOLLOW-UP 1: The stream never ends; memory must be
# bounded.
# Answer: A message older than the gap can never be
# blocked again, so evict it. Keep a min-heap of (time
# shown, message) and pop entries older than the gap
# before each decision; delete the dictionary key only if
# it still points at that time.
class BoundedFilter:
    """An endless stream: forget messages that can no
    longer be blocked, so memory stays small."""

    def __init__(self, gap=10):
        self.gap = gap
        self.last = {}
        # (time shown, message), oldest first
        self.heap = []

    def show(self, ts, msg):
        # drop entries older than the gap
        while self.heap:
            if ts - self.heap[0][0] <= self.gap:
                break
            old_ts, old_msg = heapq.heappop(self.heap)
            # delete only if it was not shown again since
            if self.last.get(old_msg) == old_ts:
                del self.last[old_msg]
        if msg in self.last:
            return False
        self.last[msg] = ts
        heapq.heappush(self.heap, (ts, msg))
        return True


# FOLLOW-UP 2: Several robots send messages.
# Answer: Use (robot id, message) as the key, or one
# filter per robot. The same logic applies per key.
class PerRobotFilter:
    """Several robots; the gap depends on severity."""

    def __init__(self, gaps, default=10):
        self.gaps, self.default = gaps, default
        self.last = {}

    def show(self, robot, ts, msg, severity=None):
        key = (robot, msg)
        gap = self.gaps.get(severity, self.default)
```

```
        last = self.last.get(key)
        if last is not None and ts - last <= gap:
            return False
        self.last[key] = ts
        return True


if __name__ == "__main__":
    stream = [(0, "a"), (5, "a"), (10, "a"), (11, "a"),
              (12, "b"), (22, "a")]
    # 10 is hidden (not MORE than 10 after 0); 11 shows
    assert filter_status(stream) == [(0, "a"), (11, "a"),
                                     (12, "b"), (22, "a")]
    # messages are case sensitive
    assert filter_status([(0, "A"), (1, "a")]) == [
        (0, "A"), (1, "a")]
    b = BoundedFilter()
    shows = [b.show(ts, msg) for ts, msg in stream]
    assert shows == [True, False, False, True, True, True]
    assert len(b.last) <= 2
    r = PerRobotFilter({"high": 2})
```

```
    assert r.show("r1", 0, "x")
    assert not r.show("r1", 9, "x")
    # another robot, and a stricter severity
    assert r.show("r2", 1, "x")
    assert r.show("r1", 1, "y", "high")
    assert r.show("r1", 4, "y", "high")
```

FUNCTION AND TEST CASES

Signature

```
def filter_messages(events: list[tuple[int, str]]) -> list
[tuple[int, str]]:
```

Sample call

```
filter_messages([])
```

Test cases

Input	Expected
`([],)`	`[]`
`([[10, "solar panel activated"], [13, "solar panel activated"], [20, "solar panel activated"], [21, "solar panel activated"]],)`	`[[10, "solar panel activated"], [21, "solar panel activated"]]`

ALSO REPORTED · 1 more

Robot Status Message Deduplication Window | 1Point3Acres | onsite-coding | asked 2026-09-10 | Low

https://www.1point3acres.com/interview/problems/company/google/robot-status-message-deduplication

ALSO REPORTED · 1 more

The state transition occurs only on a displayed event. Updating on every receipt changes the semantics and can suppress a frequently repeated message forever.
Resolve the exact-boundary rule before coding and make the comparison match that contract.
For out-of-order arrival, clarify how the 10-second interval applies to late events and whether an earlier print decision can ever be revisited. These semantics are not specified by the logger variant; do not assume the chronological-stream contract resolves them.

Preparation: Implement the stream processor with per-message state, then trace repeated and interleaved messages by hand. Write tests for the first occurrence, an event just inside the window, exactly at the boundary, just outside it, and a hidden event followed by an allowed display.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

Site tags: python, hashmap, data-analysis, sql, medium

MEDIUM | 1Point3Acres | phone-screen | 6 of 30

Website Activity Analytics: Unique Users + Session Time

ASKED 2026-06-26 | 116 DAYS AGO | LAST 6 MONTHS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 217/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-26 | Roles: DS

Original: https://www.1point3acres.com/interview/problems/company/google/website-activity-analytics-unique-users-session-time

QUESTION

Given website action records with **`user_id`**, timestamp, and activity, compute per-activity unique-user counts and average time spent on the site per user in seconds. The screen paired this Python task with a lighter SQL section and emphasized edge-case discussion over perfect final code.

Input is a collection of user-activity records. Each record contains:

user_id: numeric user identifier
timestamp: timestamp string such as 2024-07-26 10:00:00
activity: action name such as login, view, purchase, or logout

Implement two analytics tasks

Return a mapping from each activity type to the number of unique users who performed that activity.
Calculate the average time spent on the website per user in seconds.

Clarify session semantics before coding. The prompt uses login/logout-style events, but the exact treatment of users without an explicit logout, multiple sessions per user, and out-of-order records must be agreed with the interviewer.

EXAMPLES

```
user_activity = [
    {'user_id': 1, 'timestamp': '2024-07-26
        10:00:00', 'activity': 'login'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:05:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:10:00', 'activity': 'view'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:15:00', 'activity': 'purchase'},
    {'user_id': 2, 'timestamp': '2024-07-26
        10:20:00', 'activity': 'view'},
    {'user_id': 3, 'timestamp': '2024-07-26
        10:25:00', 'activity': 'login'},
    {'user_id': 1, 'timestamp': '2024-07-26
        10:30:00', 'activity': 'logout'},
]
```

APPROACH

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

HOW TO THINK ABOUT IT

Unique users per activity is a map from activity to a set of user ids. Time on site needs a definition: pair each login with the next logout of the same user, and say what you do for a login without a logout (here: the session ends at that user's last event). Parse timestamps once, sort each user's events by time, and agree on out-of-order rows and repeated logins before coding.

Basis: Written for this question (a grouping and aggregation task; the SQL half of the screen is the same idea with GROUP BY and COUNT(DISTINCT)).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for activity in activities:
    users = set()
    for rec in records:
        if rec.activity == activity:
            users.add(rec.user)
    unique[activity] = len(users)
for user in all_users:
    mine = [e for e in records if e.user == user]
    events = sorted(mine)
    spent = 0
    for login, logout in pairs(events):
        spent += logout.time - login.time
    total[user] = spent
average = sum(total.values()) / len(total)
```

TIME	SPACE
O(U * N + A * N)	O(U + A)

WHY IT IS SLOW

Rescans the whole log for every activity and every user. One pass that groups by activity and by user, then sorts each user's events once, is O(N log N).

OPTIMAL SOLUTION · One pass to group, then sessions per user

TIME	SPACE
O(N log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Users with several sessions, missing logouts or out-of-order rows.

Sort each user's events by time before pairing; a second login before a logout is ignored (or closes the old session, depending on the contract you agree). A login with no logout ends at the user's last event, or is dropped if you decide it is unreliable. Say the choice out loud: the interviewer accepted a strong discussion over perfect code.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Website Activity Analytics: Unique Users
# + Session Time
#
# Given website action records with user_id, timestamp,
# and activity, compute per-activity unique-user counts
# and average time spent on the site per user in seconds.
# The screen paired this Python task with a lighter SQL
# section and emphasized edge-case discussion over perfect
# final code. Input is a collection of user-activity
# records. Each record contains: user_id: numeric user
# identifier timestamp: timestamp string such as
# 2024-07-26 10:00:00 activity: action name such as login,
# view, purchase, or logout Implement two analytics tasks:
# Return a mapping from each activity type to the number
# of unique users who performed that activity. Calculate
# the average time spent on the website per user in
# seconds. Clarify session semantics before coding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Users with several sessions, missing logouts or out-
# of-order rows.
#
# INTERVIEW NOTES
# Unique users per activity is a map from activity to a
# set of user ids. Time on site needs a definition: pair
# each login with the next logout of the same user, and
# say what you do for a login without a logout (here: the
# session ends at that user's last event). Parse
# timestamps once, sort each user's events by time, and
# agree on out-of-order rows and repeated logins before
# coding.
#
# BRUTE FORCE (say it first): Rescan all records for every
# user and every activity
# For each activity loop over all records to collect
# users, and for each user loop over all records again to
# compute the session time. Correct, but it rescans the
# data many times. Time O(U * N + A * N). Space O(U + A).


from collections import defaultdict
from datetime import datetime


# BRUTE FORCE (say it first): Rescan all records for every
```

```
# user and every activity | time O(U * N + A * N)
def activity_brute(records):
    from datetime import datetime

    ts = lambda r: datetime.strptime(
        r["timestamp"], "%Y-%m-%d %H:%M:%S"
    )
    users = sorted({r["user_id"] for r in records})
    unique = {
        a: len(
            {
                r["user_id"]
                for r in records
                if r["activity"] == a
            }
        )
        for a in sorted(
            {r["activity"] for r in records}
        )
    }
    total = 0
    for u in users:
        evs = sorted(
            (ts(r), r["activity"])
            for r in records
            if r["user_id"] == u
        )
        start, secs = None, 0
        for t, act in evs:
            if act == "login" and start is None:
                start = t
            elif act == "logout" and start is not None:
                secs += (t - start).total_seconds()
                start = None
        if start is not None:
            secs += (evs[-1][0] - start).total_seconds()
        total += secs
    return unique, (
        total / len(users) if users else 0.0
    )


# HELPERS for the base question
FORMAT = "%Y-%m-%d %H:%M:%S"
```

```
def when(record):
    # timestamp text -> datetime
    return datetime.strptime(record["timestamp"], FORMAT)


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
def unique_users_by_activity(records):
    users = defaultdict(set)
    for r in records:
        # a set counts every user once per activity
        users[r["activity"]].add(r["user_id"])
    return {a: len(u) for a, u in sorted(users.items())}


# BASE QUESTION (optimal): One pass to group, then
# sessions per user
# Idea: Parse each record once. unique[activity] collects
# user ids in a set.
# FOLLOW-UP 1: Users with several sessions, missing
# logouts or out-of-order rows.
# Answer: Sort each user's events by time before pairing;
# a second login before a logout is ignored (or closes the
# old session, depending on the contract you agree).
def session_seconds(events):
    # events: one user's (time, activity), any order
    total, start = 0, None
    # FOLLOW-UP 1: sorted: rows may arrive out of order
    for t, act in sorted(events):
        if act == "login" and start is None:
            # a second login before a logout is ignored
            start = t
```

```
        elif act == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    # a login with no logout ends at the last event seen
    if start is not None:
        last = max(t for t, _ in events)
        total += (last - start).total_seconds()
    return total
```

```
# AVERAGE time on the site per user
def average_time_per_user(records):
    events = defaultdict(list)
    for r in records:
        item = (when(r), r["activity"])
        events[r["user_id"]].append(item)
    if not events:
        return 0.0
    # state the denominator: every user who appears
    per_user = events.values()
    total = sum(session_seconds(e) for e in per_user)
    return total / len(events)


if __name__ == "__main__":
    def rec(user, minute, activity):
        stamp = f"2024-07-26 10:{minute:02}:00"
        return {"user_id": user, "activity": activity,
                "timestamp": stamp}

    data = [rec(1, 0, "login"), rec(2, 5, "login"),
            rec(1, 10, "view"), rec(1, 15, "purchase"),
            rec(2, 20, "view"), rec(3, 25, "login"),
            rec(1, 30, "logout")]
    assert unique_users_by_activity(data) == {
        "login": 3, "logout": 1, "purchase": 1, "view": 2}
    # user 1: 1800 s, user 2: 900 s (no logout), user 3: 0
    assert average_time_per_user(data) == 900
    assert average_time_per_user([]) == 0.0
    # out-of-order rows are sorted first
    shuffled = [rec(1, 30, "logout"), rec(1, 0, "login")]
    assert average_time_per_user(shuffled) == 1800
    # two sessions add up
    two = [rec(1, 0, "login"), rec(1, 5, "logout"),
           rec(1, 10, "login"), rec(1, 20, "logout")]
    assert average_time_per_user(two) == 900
```

NOTES FROM CANDIDATES

SQL in the same screen was described as simple and covered joins / aggregation style work, but the exact SQL prompt did not leak.
The interviewer accepted a strong draft when all edge cases were discussed, even though the final code did not cover every case.
Spend time up front on timestamp parsing, ordering, repeated login/logout events, and users with missing terminal actions.

PREPARATION

Practice writing small Python analytics functions in a plain document without IDE help.
Drill grouping records by user and by activity with dictionaries / sets.
Rehearse explaining incomplete edge-case handling and dry-running a small event stream.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

MEDIUM | PracHub | Technical Screen | 7 of 30

Find a Horizontal Cut That Bisects Rectangle Area

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 8.1 | 30 DAYS 0 | PRIORITY 169/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #850 Rectangle Area II

LeetCode match: No direct match; related LC 850 Rectangle Area II (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-08-07)

Original: https://prachub.com/coding-questions/find-a-horizontal-cut-that-bisects-rectangle-area

QUESTION

Overview: Solve a computational-geometry problem that asks for the horizontal line dividing the combined area of many disjoint rectangles in half. The exercise emphasizes numerical precision, large coordinate ranges, performance at 200,000 rectangles, and careful treatment of partially intersected shapes.

APPROACH

Compute total area and the covered width per y-slab via a sweep; walk slabs accumulating area until half, then interpolate inside the slab. Use exact arithmetic for precision.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the rectangles are disjoint (the overview says so), the coordinate range, whether the answer must be exact (use fractions) and which y to return if a gap makes many cuts valid (the lowest).

BRUTE FORCE TO SAY OUT LOUD

Binary search on y, summing the rectangle area below y each time. O(n log precision), but floating point error is a risk.

OPTIMAL IDEA

Because the rectangles are disjoint, the area below y is a piecewise linear function whose slope is the sum of the widths of the rectangles spanning y. Create events at each rectangle's bottom (slope increases by its width) and top (slope decreases), sort by y, and walk the slabs accumulating area. When the next slab would pass half of the total, solve inside it: y = y0 + (half - accumulated) / slope, using integer or Fraction arithmetic. O(n log n) for 200,000 rectangles.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

One rectangle (the middle), two rectangles separated by a gap (slope zero there), the half exactly at a rectangle boundary, huge coordinates.

LIKELY FOLLOW-UPS

Overlapping rectangles (LeetCode 850, needs a segment tree over x), or a vertical cut.

PITFALLS

Floating point error when comparing areas. Not handling slabs with slope zero.

HOW TO THINK ABOUT IT

Area below a line y = c is a piecewise linear function of c, and between two consecutive rectangle edges its slope is just the total width of rectangles crossing that height (they are disjoint, so widths add). Walk the slabs between events, accumulate area, and when half the area falls inside a slab solve a linear equation. Use exact fractions so the answer is exact.

Basis: Written; a sweep over y events (compare LeetCode 850, Rectangle Area II, which is the harder overlapping version). Assumed rule: axis-aligned rectangles (x1, y1, x2, y2) that do not overlap; return the lowest y with half the total area below it.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(rects):
    total = sum(area(r) for r in rects)
    lo = lowest_edge(rects)
    hi = highest_edge(rects)
    for _ in range(100):
        mid = (lo + hi) / 2
        if area_below(rects, mid) >= total / 2:
            hi = mid
        else:
            lo = mid
    return hi
```

TIME	SPACE
O(R * iterations)	O(1)

WHY IT IS SLOW

Each step sums over all rectangles and the answer is only as precise as floating point allows. A sweep over sorted edges with exact fractions finds the slab and the cut directly in O(R log R).

OPTIMAL SOLUTION · Sweep the y events and interpolate inside one slab

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rectangles may overlap (LeetCode 850).

The slope is then the union width of the rectangles crossing a height, not the sum. Maintain it with a segment tree over compressed x coordinates (cover counts), O(R log R) overall.

FOLLOW-UP 2 Coordinates up to 10^9 and 200,000 rectangles.

Use integers or Fractions (areas reach 10^18, which fits in Python ints but not in 32-bit). The sweep is O(R log R); never use floats for the comparisons that decide the slab.

FOLLOW-UP 3 A vertical cut instead, or a cut at a given ratio.

Swap the axes for a vertical cut. For a ratio p, replace half by p * total in the slab test; the interpolation formula is unchanged.

FOLLOW-UP 4 A cut that leaves a given fraction of the area below it, not exactly half.

Replace 'half of the total' by p * total in the slab test; the interpolation formula inside the slab is the same. The code returns an exact Fraction and is checked by computing the area below the returned cut.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find a Horizontal Cut That Bisects
# Rectangle Area
#
# Solve a computational-geometry problem that asks for the
# horizontal line dividing the combined area of many
# disjoint rectangles in half. The exercise emphasizes
# numerical precision, large coordinate ranges,
# performance at 200,000 rectangles, and careful treatment
# of partially intersected shapes.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Rectangles may overlap (LeetCode 850).
# 2. Coordinates up to 10^9 and 200,000 rectangles.
# 3. A vertical cut instead, or a cut at a given ratio.
# 4. A cut that leaves a given fraction of the area below
# it, not exactly half.
#
# INTERVIEW NOTES
# Area below a line y = c is a piecewise linear function
# of c, and between two consecutive rectangle edges its
# slope is just the total width of rectangles crossing
# that height (they are disjoint, so widths add). Walk the
# slabs between events, accumulate area, and when half the
# area falls inside a slab solve a linear equation. Use
# exact fractions so the answer is exact.
#
# BRUTE FORCE (say it first): Binary search on the cut
# height
# The area below y is non-decreasing in y, so bisect y
# between the lowest and highest edge until the area below
# is half the total. It is simple and general, but it
# works in floating point (limited precision) and costs a
# full pass over all rectangles per step. Time O(R *
# iterations). Space O(1).


from collections import defaultdict
from fractions import Fraction


# BRUTE FORCE (say it first): Binary search on the cut
# height | time O(R * iterations)
def _area_below(rects, y):
    return sum(
        (x2 - x1) * max(0, min(y, y2) - y1)
        for x1, y1, x2, y2 in rects
    )


# HELPERS for the base question
def total_area(rects):
    # rects are disjoint (x1, y1, x2, y2)
    return sum((x2 - x1) * (y2 - y1)
               for x1, y1, x2, y2 in rects)


def bisect_cut_search(rects, iterations=100):
    total = sum(
        (x2 - x1) * (y2 - y1)
        for x1, y1, x2, y2 in rects
    )
    lo = min(r[1] for r in rects)
    hi = max(r[3] for r in rects)
    for _ in range(iterations):
        mid = (lo + hi) / 2
        if 2 * _area_below(rects, mid) >= total:
            hi = mid
        else:
            lo = mid
    return hi


# BASE QUESTION (optimal): Sweep the y events and
# interpolate inside one slab
# Idea: Make events (y, +width) at each rectangle's bottom
# and (y, -width) at its top and sort them. Between
# consecutive event heights the slope is the running sum
# of widths.
# FOLLOW-UP 4: A cut that leaves a given fraction of the
# area below it, not exactly half.
# Answer: Replace 'half of the total' by p * total in the
# slab test; the interpolation formula inside the slab is
# the same. The code returns an exact Fraction and is
# checked by computing the area below the returned cut.
def cut_height(rects, share=Fraction(1, 2)):
    # FOLLOW-UP 4: share= leaves any fraction below the
    # cut
    # the height with `share` of the total area below it
    # (1/2 = bisect). Exact Fractions, never floats.
    want = share * total_area(rects)
    # width gained when the sweep passes each height
    events = defaultdict(int)
    for x1, y1, x2, y2 in rects:
        events[y1] += x2 - x1
        events[y2] -= x2 - x1
    ys = sorted(events)
    width, so_far = 0, Fraction(0)
    for y0, y1 in zip(ys, ys[1:]):
        # inside this slab the covered width is constant
        width += events[y0]
        gain = width * (y1 - y0)
        if width and so_far + gain >= want:
```

```
            # the cut is inside this slab: interpolate
            return y0 + (want - so_far) / width
        so_far += gain
    return Fraction(ys[-1])


# HELPERS for the checks below
def area_below(rects, y):
    # exact area under the line at height y (used to
    # check)
    total = Fraction(0)
    for x1, y1, x2, y2 in rects:
        top = min(Fraction(y), y2)
        if top > y1:
            total += (x2 - x1) * (top - y1)
    return total


if __name__ == "__main__":
    stacked = [(0, 0, 2, 1), (0, 1, 2, 2)]
    assert cut_height(stacked) == 1
    # one tall rectangle: halfway up
    assert cut_height([(0, 0, 1, 4)]) == 2
    # a gap between rectangles: the cut sits at the first
    # top
    gap = [(0, 0, 1, 1), (0, 5, 1, 6)]
    assert cut_height(gap) == 1
    # the cut can fall inside a slab with a fractional
    # height
    odd = [(0, 0, 3, 1), (0, 1, 1, 2)]
    y = cut_height(odd)
    assert area_below(odd, y) * 2 == total_area(odd)
    # a different share: 1/4 of the area below
```

```
    q = cut_height(stacked, Fraction(1, 4))
    below = area_below(stacked, q)
    assert below * 4 == total_area(stacked)
    # large coordinates stay exact
    big = [(0, 0, 10 ** 9, 10 ** 9)]
    assert cut_height(big) == Fraction(10 ** 9, 2)
```

FUNCTION AND TEST CASES

Signature

```
def horizontal_bisector(rectangles: list[list[float]]) ->
float:
```

Sample call

```
horizontal_bisector([[0, 0, 4, 2]])
```

Test cases

Input	Expected
`([[0, 0, 4, 2]],)`	`1.0`
`([[0, 0, 2, 4], [10, 1, 2, 2]],)`	`2.0`

ALSO REPORTED · 1 more

Find where to cut cake into equal areas | PracHub | Technical Screen

https://prachub.com/coding-questions/find-where-to-cut-cake-into-equal-areas

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

Site tags: array, hashmap, medium

MEDIUM | 1Point3Acres | onsite-coding | 8 of 30

Gold Chain Split into Two Equal Halves

ASKED 2026-04-21 | 182 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 201/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-21 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gold-chain-split-equal

QUESTION

Onsite coding interview 1: given a gold chain represented by an array of segment weights, remove exactly one segment and check whether the remaining segments — when reconnected — can be cut once more to give two halves of equal total weight. Follow-up: enumerate all valid (remove, cut) pairs.SWEArrayhashmapmediumFrequencySingle reportLast asked2026-04-21Stageonsite-coding

APPROACH

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether removing a segment keeps the rest as a chain joined end to end, whether the cut after removal must leave two non-empty parts, and whether to return a boolean, one answer, or all valid pairs.

BRUTE FORCE TO SAY OUT LOUD

For each removed index, rebuild the array and try every cut with prefix sums. O(n^2).

OPTIMAL IDEA

Compute the total S and prefix sums P. Removing index r leaves total S - a[r], which must be even; the target is half = (S - a[r]) / 2. In the remaining chain a cut is valid when some non-empty prefix sums to half. A prefix that stops before r has sum P[j], and one that includes the segments after r has sum P[j] - a[r]. So for each r check whether half appears among the prefix sums before r, or whether half + a[r] appears among the prefix sums after r (keep a hash set of prefix sums seen so far, and one for those still ahead). Boolean answer in O(n); listing all valid (remove, cut) pairs is output-bound, up to O(n^2).

DRY-RUN THESE CASES

A chain of two segments, a removal at the first or last index, a total that becomes odd, equal weights.

LIKELY FOLLOW-UPS

Enumerate all valid (remove, cut) pairs, or allow removing k segments.

PITFALLS

Allowing an empty half. Forgetting the shift of indices after the removed segment.

INTERVIEW-DAY PLAN

Compute the total S and prefix sums P. Removing index r leaves total S - a[r], which must be even; the target is half = (S - a[r]) / 2. In the remaining chain a cut is valid when some non-empty prefix sums to half. A prefix that stops before r has sum P[j], and one that includes the segments after r has sum P[j] - a[r]. So for each r check whether half appears among the prefix sums before r, or whether half + a[r] appears among the prefix sums after r (keep a hash set of prefix sums seen so far, and one for those still ahead). Boolean answer in O(n); listing all valid (remove, cut) pairs is output-bound, up to O(n^2).

DRY-RUN THESE CASES

A chain of two segments, a removal at the first or last index, a total that becomes odd, equal weights.

LIKELY FOLLOW-UPS

Enumerate all valid (remove, cut) pairs, or allow removing k segments.

PITFALLS

Allowing an empty half. Forgetting the shift of indices after the removed segment.

HOW TO THINK ABOUT IT

After removing segment r the target half is (S - a[r]) / 2, and a cut means some prefix of the remaining chain has exactly that weight. A prefix of the remaining chain is either an ordinary prefix of the original array (stops before r) or an original prefix minus a[r] (passes over r). Each is one value to look up in a hash of prefix sums, so every r costs O(1).

Basis: Written; prefix sums with a hash lookup (like LeetCode 724 and 560). Assumed rule (confirm it): remove exactly one segment, reconnect the remaining chain end to end, then cut it once into two non-empty parts of equal total weight.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(w):
    for r in range(len(w)):
        chain = w[:r] + w[r + 1:]
        half, run = sum(chain) / 2, 0
        for x in chain:
            run += x
            if run == half:
                return True
    return False
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n removals, each rebuilt and scanned: O(n^2). Prefix sums with a lookup of the needed prefix value answer each removal in O(1).

OPTIMAL SOLUTION · Prefix sums with first and last occurrence maps

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 List every valid (removed segment, cut position) pair.

The answer itself can be quadratic, so a simple loop is acceptable. With positive weights prefix sums are strictly increasing, so each target has at most one cut position and the listing is O(n) after the hash lookups; with zeros or negatives a value can repeat, so keep a list of indices per prefix value.

FOLLOW-UP 2 The chain is a circle (cut anywhere).

Then a cut is two cut points on the circle and the removed segment just shortens it; use the circular window idea (duplicate the array and slide a window whose sum is half).

FOLLOW-UP 3 Remove k segments instead of one.

The remaining total depends on which k you remove, which becomes a subset-sum style search; ask for the limits before promising a fast solution.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Gold Chain Split into Two Equal Halves
#
# Onsite coding interview 1: given a gold chain
# represented by an array of segment weights, remove
# exactly one segment and check whether the remaining
# segments — when reconnected — can be cut once more to
# give two halves of equal total weight. Follow-up:
# enumerate all valid (remove, cut) pairs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. List every valid (removed segment, cut position)
# pair.
# 2. The chain is a circle (cut anywhere).
# 3. Remove k segments instead of one.
#
# INTERVIEW NOTES
# After removing segment r the target half is (S - a[r]) /
# 2, and a cut means some prefix of the remaining chain
# has exactly that weight. A prefix of the remaining chain
# is either an ordinary prefix of the original array
# (stops before r) or an original prefix minus a[r]
# (passes over r). Each is one value to look up in a hash
# of prefix sums, so every r costs O(1).
#
# BRUTE FORCE (say it first): For each removal rebuild the
# chain and try every cut
# Remove each segment, then test every cut position with a
# running sum. Correct, with a quadratic loop that is the
# baseline to improve. Time O(n^2). Space O(n).


# BRUTE FORCE (say it first): For each removal rebuild the
# chain and try every cut | time O(n^2)
def gold_chain_brute(a):
    n = len(a)
    for r in range(n):
        rest = a[:r] + a[r + 1 :]
        total = sum(rest)
        run = 0
        for c in range(len(rest) - 1):
            run += rest[c]
            if run * 2 == total:
                return True
    return False


# HELPERS for the base question
def prefix_sums(a):
    # P[j] = sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[-1] + x)
    return P


# BASE QUESTION (optimal): Prefix sums with first and last
# occurrence maps
# Idea: Let P[j] be the sum of the first j segments.
def gold_chain(a):
    # remove exactly one segment, then cut once into
    # two non-empty halves of equal weight
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[-1]
    # first and last position of every prefix sum (cuts
    # sit between segments, so positions 1..n-1)
    first, last = {}, {}
    for j in range(1, n):
        first.setdefault(P[j], j)
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2:
            continue
        half = left_over // 2
        # cut before the removed segment: P[j] == half
        if half in first and first[half] <= min(r, n - 2):
            return True
        # cut after it: P[j] - a[r] == half
        if half + a[r] in last and last[half + a[r]] > r:
            return True
    return False


# FOLLOW-UP 1: List every valid (removed segment, cut
# position) pair.
# Answer: The answer itself can be quadratic, so a simple
# loop is acceptable.
def valid_pairs(a):
    # follow-up: every (removed index, cut position) pair;
    # the answer can be quadratic, so a loop is fine
    pairs = []
    for r in range(len(a)):
        rest = a[:r] + a[r + 1:]
        total, run = sum(rest), 0
        # cut after `cut` segments of the reconnected
```

```
        # chain
        for cut in range(1, len(rest)):
            run += rest[cut - 1]
            if run * 2 == total:
                pairs.append((r, cut))
    return pairs


if __name__ == "__main__":
    # remove the middle 1, then cut 5 | 5
    assert gold_chain([5, 1, 5])
    assert gold_chain([2, 7, 2])
    assert not gold_chain([1, 2, 3])
    assert not gold_chain([1, 1])
    assert not gold_chain([1, 2, 4])
    assert (1, 1) in valid_pairs([1, 2, 1])
    # both methods agree on many small chains
    import itertools
    for chain in itertools.product(range(1, 5), repeat=5):
        assert gold_chain(list(chain)) == bool(
            valid_pairs(list(chain)))
```

NOTES FROM CANDIDATES

Precompute prefix sums; for each candidate removed index r, the remaining total is S - a[r]. If that is odd, skip. Otherwise check if there is a split point achieving (S - a[r]) / 2.
Use a hashmap from prefix-sum value to index for O(1) lookups -> overall O(N) for the boolean answer, O(N^2) worst case for enumerating all pairs (output-bound).
Watch out for cut positions adjacent to the removed link — must still respect contiguous-chain semantics.

PREPARATION

Drill The equal-subset-sum and partition-to-k-equal-sum-subsets problems are the standard drill partners.
Practice enumerating all valid cut positions, not just one — interviewers commonly add the follow-up after the boolean version works.
Watch the boundary: remove + cut should not yield an empty side.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

MEDIUM | PracHub | Technical Screen | 9 of 30

★ TOP 75: rank 13 in the Medium and Hard priority list

★ Solve array-sum and city-community problems

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 66 | 30 DAYS 46.9 | PRIORITY 7/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #560 Subarray Sum Equals K, LeetCode #200 Number of Islands

LeetCode match: LC 560 Subarray Sum Equals K + LC 200 Number of Islands (inferred from overview)

Posted on PracHub: 2026-02-20 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/solve-array-sum-and-city-community-problems

QUESTION

Overview: This question evaluates array-processing and grid/graph connectivity skills, covering counting subarrays with a target sum and identifying connected building communities in a grid; it belongs to the Coding & Algorithms domain and involves streaming/online algorithm reasoning as well as grid traversal concepts.

APPROACH

Prefix sum + hash map of counts for the subarray target; communities = connected components via DFS/union-find on the grid.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Array: count subarrays whose sum equals K; are values negative (decides the method) and can the input be a stream? Grid: building communities as connected components, is adjacency four-way, and what marks a building. The scraped solution covers only the array part.

BRUTE FORCE TO SAY OUT LOUD

Array: all subarrays with running sums, O(n^2). Grid: none needed, a flood fill is the baseline.

OPTIMAL IDEA

Array: prefix sums with a dict of how many times each prefix has occurred; for each element add the count of (prefix - K), then record the prefix. O(n) and it works online for a stream. A sliding window fails with negative values. Grid: DFS/BFS (iterative) or Union-Find to count connected components of building cells.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Array: [1,1,1] with K = 2 gives 2, [1,2,3] with K = 3 gives 2, negatives, K = 0 with zeros, an empty array. Grid: no buildings, one building, diagonal buildings (separate).

LIKELY FOLLOW-UPS

Array: longest subarray with sum K, or counting in a stream. Grid: the largest community, or communities as buildings are added.

PITFALLS

Not seeding the prefix dict with {0: 1}. Using a window technique with negative numbers.

HOW TO THINK ABOUT IT

Subarray sums with negatives allowed cannot use a sliding window; use prefix sums: a subarray ending here sums to K exactly when an earlier prefix equals (current prefix - K), so a dictionary of prefix counts answers it in one pass and works for streams. Communities are connected components, so flood fill or union-find over the building cells.

Basis: Two tasks: count subarrays with sum equal to K (LeetCode 560) and count connected building communities in a grid (LeetCode 200 idea). Assumed rule: buildings are '1' cells, communities connect horizontally and vertically.

BRUTE FORCE · Task 1: try every subarray

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, K):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total == K:
                count += 1
    return count
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

O(n^2). A dictionary of prefix sums (how many earlier prefixes equal current - K) is O(n) and also works on a stream and with negative numbers.

BRUTE FORCE · Task 2: relabel cells until the labels stop changing

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE · Task 2: relabel cells until the labels stop changing

```
label = {}
for i, cell in enumerate(buildings):
    label[cell] = i
changed = True
while changed:
    changed = False
    for cell in buildings:
        group = [cell] + neighbours(cell)
        best = min(label[x] for x in group)
        if best < label[cell]:
            label[cell] = best
            changed = True
answer = len(set(label.values()))
```

TIME	SPACE
O((m * n)^2) worst case	O(m * n)

WHY IT IS SLOW

Labels spread one step per round, so a long snake needs about m * n rounds of O(m * n): O((m * n)^2). Union-Find or one flood fill is near-linear.

OPTIMAL SOLUTION · Task 1: prefix sums in a dictionary

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task 2: Union-Find over building cells

TIME	SPACE
O(m * n * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The numbers are all positive.

A sliding window works in O(1) memory: grow the right end, shrink the left while the sum exceeds K. With negatives the window is not monotone, which is why prefix sums are needed.

FOLLOW-UP 2 Longest subarray with sum K.

Store the first index of each prefix sum instead of a count, and take the longest distance to prefix - K.

FOLLOW-UP 3 Buildings are added over time and you report the number of communities after each addition.

Union-Find with a counter: +1 for the new building, -1 for each union with a different neighbouring community (LeetCode 305).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve array-sum and city-community
# problems
#
# This question evaluates array-processing and grid/graph
# connectivity skills, covering counting subarrays with a
# target sum and identifying connected building
# communities in a grid; it belongs to the Coding &
# Algorithms domain and involves streaming/online
# algorithm reasoning as well as grid traversal concepts.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The numbers are all positive.
# 2. Longest subarray with sum K.
# 3. Buildings are added over time and you report the
# number of communities after each addition.
#
# INTERVIEW NOTES
# Subarray sums with negatives allowed cannot use a
# sliding window; use prefix sums: a subarray ending here
# sums to K exactly when an earlier prefix equals (current
# prefix - K), so a dictionary of prefix counts answers it
# in one pass and works for streams. Communities are
# connected components, so flood fill or union-find over
# the building cells.
#
# BRUTE FORCE (say it first): Task 1: try every subarray
# Extend each start with a running sum and count the ends
# where it equals K. Correct and the oracle; quadratic.
# Time O(n^2). Space O(1).


# BRUTE FORCE (say it first): Task 1: try every subarray |
# time O(n^2)
def subarray_sum_brute(nums, k):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            count += total == k
    return count


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))
```

```
# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def count_subarrays(nums, k):
    # prefix sums: a subarray ending here has sum k if
    # some
    # earlier prefix equals (prefix - k)
    seen, prefix, count = {0: 1}, 0, 0
    for x in nums:
        prefix += x
```

```
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count


# BASE QUESTION (optimal): prefix sums in a dictionary
# Idea: Walk the array keeping the prefix sum.
def communities(grid):
    # connected groups of buildings (1) in a grid
    seen, count = set(), 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != 1 or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            stack = [(r, c)]
            while stack:
                x, y = stack.pop()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < len(grid)):
                        continue
                    if not (0 <= b < len(grid[0])):
                        continue
                    land = grid[a][b] == 1
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        stack.append((a, b))
    return count


# FOLLOW-UP 1: The numbers are all positive.
# Answer: A sliding window works in O(1) memory: grow the
```

```
# right end, shrink the left while the sum exceeds K. With
# negatives the window is not monotone, which is why
# prefix sums are needed.
def count_positive(nums, k):
    # all numbers positive: a sliding window, O(1) memory
    count = left = total = 0
    for right, x in enumerate(nums):
        total += x
        while total > k and left <= right:
            total -= nums[left]
            left += 1
        if total == k:
            count += 1
    return count


# FOLLOW-UP 2: Longest subarray with sum K.
# Answer: Store the first index of each prefix sum instead
# of a count, and take the longest distance to prefix - K.
def longest_subarray(nums, k):
    # store the FIRST index of each prefix sum
    first, prefix, best = {0: -1}, 0, 0
    for i, x in enumerate(nums):
        prefix += x
        if prefix - k in first:
            best = max(best, i - first[prefix - k])
        first.setdefault(prefix, i)
    return best


# FOLLOW-UP 3: Buildings are added over time and you
# report the number of communities after each addition.
# Answer: Union-Find with a counter: +1 for the new
# building, -1 for each union with a different
# neighbouring community (LeetCode 305).
def communities_online(rows, cols, additions):
    # buildings added one by one: count after each
    # (LeetCode 305)
```

```
    parent, count, out = {}, 0, []

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
```

```
    for r, c in additions:
        if (r, c) not in parent:
            parent[(r, c)] = (r, c)
            count += 1
            for dr, dc in STEPS:
                nb = (r + dr, c + dc)
                if nb in parent:
                    a, b = find((r, c)), find(nb)
                    if a != b:
                        parent[a] = b
                        # two communities became one
                        count -= 1
        out.append(count)
    return out


if __name__ == "__main__":
    assert count_subarrays([1, 1, 1], 2) == 2
    assert count_subarrays([1, -1, 0], 0) == 3
    assert count_subarrays([], 0) == 0
    assert longest_subarray([1, -1, 5, -2, 3], 3) == 4
    assert longest_subarray([1, 2], 9) == 0
    assert count_positive([1, 2, 3, 2, 1], 3) == 3
    grid = [[1, 1, 0], [0, 0, 1], [1, 0, 1]]
    assert communities(grid) == 3
    adds = [(0, 0), (0, 1), (1, 2), (2, 2), (1, 1)]
    counts = communities_online(3, 3, adds)
    assert counts == [1, 1, 2, 2, 1]
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], k: int) -> int:
```

Sample call

```
solution([1, 1, 1], 2)
```

Test cases

Input	Expected
([1, 1, 1], 2)	2
([1, 2, 3], 3)	2

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

MEDIUM | PracHub | Technical Screen | 10 of 30

Check pivot after removing one element

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 116/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #724 Find Pivot Index

LeetCode match: LC 724 Find Pivot Index (variant: one removal) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/check-pivot-after-removing-one-element

QUESTION

Overview: This question evaluates array-manipulation skills, cumulative-sum/prefix-sum reasoning, and robustness in handling negative values and duplicates, testing algorithmic problem-solving competency in the Coding & Algorithms domain and focusing on practical application rather than purely conceptual understanding.

APPROACH

Prefix sums plus a hash map of left-side sums; for each removal candidate, check whether some index has equal left and right sums after the removal.

INTERVIEW-DAY PLAN

CLARIFY FIRST

The statement is not complete in my copy, so confirm: after removing exactly one element (or at most one), does there exist an index where the sum to the left equals the sum to the right (the pivot itself excluded)? Are negatives and duplicates allowed?

BRUTE FORCE TO SAY OUT LOUD

For each element removed, rebuild the array and test every pivot with prefix sums. O(n^2).

OPTIMAL IDEA

Let P[i] be the sum before index i and S the total. If the removed element r is to the right of the pivot i, the two sides are equal exactly when a[r] = S - 2 * P[i] - a[i]. If r is to the left of i, the condition is a[r] = 2 * P[i] + a[i]

INTERVIEW-DAY PLAN

- S. So for every pivot candidate the value that must be removed is determined; look it up in a dict of value to last index (needs an index greater than i) and a dict of value to first index (needs an index less than i). One pass, O(n) time and space. Say the brute force first, and mention you checked the formulas against it on random small arrays.

DRY-RUN THESE CASES

A one-element array, two elements, negatives, duplicate values, a pivot that exists without any removal, removing the pivot itself (not allowed by most readings).

LIKELY FOLLOW-UPS

Allow at most one removal or remove a pivot candidate, or count all pairs.

PITFALLS

Letting the removed element be the pivot. Forgetting that the prefix sums shift after a removal.

HOW TO THINK ABOUT IT

Use prefix sums. For a pivot at original index i and a removed element on one side, the balance equation fixes the value that has to be removed: it is a closed form in the prefix sum, the total and a[i]. So for every candidate pivot you only have to ask 'does that value occur on the correct side?', which a dictionary of first and last positions answers in O(1).

Basis: Variant of LeetCode 724 (Find Pivot Index). Assumed rule (confirm it): remove exactly one element; is there then an index where the sum on its left equals the sum on its right (the pivot excluded)?

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    for r in range(len(a)):
        b = a[:r] + a[r + 1:]
        for i in range(len(b)):
            if sum(b[:i]) == sum(b[i + 1:]):
                return True
    return False
```

TIME	SPACE
O(n^3)	**O(n)**
O(n^2) with prefix sums	

WHY IT IS SLOW

n removals times n pivots times a sum: O(n^3) (O(n^2) with prefix sums). For each pivot the value to remove is determined by prefix sums, so a dictionary lookup makes it O(n).

OPTIMAL SOLUTION · Closed-form value to remove, found in a dictionary of positions

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Without any removal (LeetCode 724).

One pass: the pivot is the index where left sum equals total minus left minus the element. O(n), O(1).

FOLLOW-UP 2 At most one removal (removal optional).

Answer true if pivot_index finds a pivot, otherwise run the removal test above.

FOLLOW-UP 3 Return which element to remove and the pivot.

Record i together with the index of the matching value (first or last occurrence) when the lookup succeeds, instead of returning True.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check pivot after removing one element
#
# This question evaluates array-manipulation skills,
# cumulative-sum/prefix-sum reasoning, and robustness in
# handling negative values and duplicates, testing
# algorithmic problem-solving competency in the Coding &
# Algorithms domain and focusing on practical application
# rather than purely conceptual understanding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Without any removal (LeetCode 724).
# 2. At most one removal (removal optional).
# 3. Return which element to remove and the pivot.
#
# INTERVIEW NOTES
# Use prefix sums. For a pivot at original index i and a
# removed element on one side, the balance equation fixes
# the value that has to be removed: it is a closed form in
# the prefix sum, the total and a[i]. So for every
# candidate pivot you only have to ask 'does that value
# occur on the correct side?', which a dictionary of first
# and last positions answers in O(1).
#
# BRUTE FORCE (say it first): Try every removal and every
# pivot
# For each removed index build the shorter array and test
# every pivot with sums. Obviously correct and the oracle.
# Time O(n^3), O(n^2) with prefix sums. Space O(n).


# BRUTE FORCE (checker)
def brute(a):
    # try every removal and every pivot: the demo's check
    for i in range(len(a)):
        rest = a[:i] + a[i + 1:]
        for j in range(len(rest)):
            if sum(rest[:j]) == sum(rest[j + 1:]):
                return True
    return False


# BASE QUESTION (optimal): Closed-form value to remove,
# found in a dictionary of positions
# Idea: Let P be the prefix sum before i and S the total.
def pivot_after_removal(a):
    # remove ONE element, then is there a pivot: an
    # element
    # whose left sum equals its right sum? Returns
    # (removed
    # index, pivot index) or None. O(n) with two scans.
    n, total = len(a), sum(a)
    prefix = [0]
    for x in a:
        prefix.append(prefix[-1] + x)
    # key[j]: left + left + a[j] when nothing is removed
    key = [2 * prefix[j] + a[j] for j in range(n)]
    # pivot j AFTER the removed index i:
    #   2 * prefix[j] - a[i] = total - a[j]  -> a[i] = key
    # - total
    where, ok = {}, None
    for j in range(n):
        i = where.get(key[j] - total)
        if i is not None:
            return i, j
        where[a[j]] = j
    # pivot j BEFORE the removed index i:
    #   2 * prefix[j] + a[j] = total - a[i]  -> a[i] =
    # total - key
    where = {}
    for j in range(n - 1, -1, -1):
        i = where.get(total - key[j])
        if i is not None:
            return i, j
        where[a[j]] = j
    return ok


# FOLLOW-UP 1: Without any removal (LeetCode 724).
# Answer: One pass: the pivot is the index where left sum
# equals total minus left minus the element. O(n), O(1).
def pivot_index(a):
    # LeetCode 724: no removal, O(1) extra space
    total, left = sum(a), 0
    for i, x in enumerate(a):
        if left == total - left - x:
            return i
        left += x
    return -1


# FOLLOW-UP 2: At most one removal (removal optional).
# Answer: Answer true if pivot_index finds a pivot,
# otherwise run the removal test above.
def has_pivot(a, allow_removal=True):
    # follow-up: removal is optional
    if pivot_index(a) != -1:
        return True
    if not allow_removal:
        return False
    return pivot_after_removal(a) is not None


if __name__ == "__main__":
    # [1,7,3,6,5,6] has a pivot at index 3 without
    # removal,
    # but removing one element never gives another
```

```
    assert pivot_after_removal([1, 7, 3, 6, 5, 6]) is None
    # [1, 1, 1, 1] has no pivot, but removing the element
    # at
    # index 1 leaves [1, 1, 1] whose middle element is one
    assert pivot_index([1, 1, 1, 1]) == -1
    assert pivot_after_removal([1, 1, 1, 1]) == (1, 2)
    assert pivot_index([1, 7, 3, 6, 5, 6]) == 3
    assert pivot_index([1, 2, 3]) == -1
    assert not has_pivot([1, 2, 3], allow_removal=False)
    # check against the brute force on many small arrays
    import itertools
    for n in range(1, 6):
        every = itertools.product([-2, 0, 1, 3], repeat=n)
        for a in every:
            a = list(a)
            found = pivot_after_removal(a) is not None
            assert found == brute(a)
    assert has_pivot([1, 1, 1, 1])
    ones = [1, 1, 1, 1]
    assert not has_pivot(ones, allow_removal=False)
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int]) -> bool:
```

Sample call

```
solution([1, 2])
```

Test cases

Input	Expected
`([1, 2],)`	`True`
`([2, 1, 1, 2],)`	`True`

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

MEDIUM | PracHub | Technical Screen | 11 of 30

Reconstruct original array from doubled shuffle

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29.8 | 30 DAYS 0 | PRIORITY 127/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: the interview site's own solution

LeetCode match: LC 2007 Find Original Array From Doubled Array (inferred from overview)

Posted on PracHub: 2026-01-04 (updated 2026-05-22)

Original: https://prachub.com/coding-questions/reconstruct-original-array-from-doubled-shuffle

QUESTION

Overview: This question evaluates array manipulation, multiset/frequency reasoning, pairing logic, and careful handling of edge cases such as zeros and negative numbers.

APPROACH

Sort (by absolute value) and use a counter: for each smallest x with count > 0, pair with 2x; handle zeros (need even count) and negatives. O(n log n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the output may be in any order, whether negatives and zeros are possible, and what to return if it is impossible (empty array).

BRUTE FORCE TO SAY OUT LOUD

Try every pairing recursively. Exponential.

OPTIMAL IDEA

If the length is odd return empty. Count values. Process numbers in order of absolute value; for each value x with remaining count, match it with 2x, decrement both counts, and fail if 2x has no count left. Zeros need an even count. Sorting by abs makes the smallest value always the original, never a double. O(n log n).

DRY-RUN THESE CASES

[1,3,4,2,6,8] gives [1,3,4]. An odd length. [0,0]. [0,0,0] fails. Negatives such as [-2,-4,1,2].

LIKELY FOLLOW-UPS

Linear time when the value range is small (counting array), or unsorted output.

PITFALLS

INTERVIEW-DAY PLAN

Sorting by value instead of absolute value for negatives. Mishandling zeros.

HOW TO THINK ABOUT IT

The element with the smallest absolute value cannot be the double of anything, so it is an original; its double must be present too. Pair them off, take the next smallest, and so on. Process values in order of absolute value so negatives and zeros follow the same rule, and handle zero separately (it is its own double, so it needs an even count).

Basis: LeetCode 2007, Find Original Array From Doubled Array.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(changed):
    items = sorted(changed, key=abs)
    originals = []
    while items:
        x = items.pop(0)
        if 2 * x not in items:
            return []
        items.remove(2 * x)
        originals.append(x)
    return originals
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

list.remove is a linear search, so each step is O(n) and the loop is O(n^2). A Counter processed in order of absolute value is O(n log n).

OPTIMAL SOLUTION · Counter processed in order of absolute value

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the originals in the order they appear in the input.

Compute the multiset of originals, then scan the input once and keep each element while its remaining original count is positive.

FOLLOW-UP 2 Values are small non-negative integers.

Use a counting array instead of sorting and a Counter: process values from small to large, O(n + range).

FOLLOW-UP 3 Why does processing by absolute value work with negatives?

For negatives the double has a larger absolute value, so the element with the smallest absolute value is always the original of its pair, exactly as with positives.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Reconstruct original array from doubled
# shuffle
#
# This question evaluates array manipulation,
# multiset/frequency reasoning, pairing logic, and careful
# handling of edge cases such as zeros and negative
# numbers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the originals in the order they appear in the
# input.
# 2. Values are small non-negative integers.
# 3. Why does processing by absolute value work with
# negatives?
#
# INTERVIEW NOTES
# The element with the smallest absolute value cannot be
# the double of anything, so it is an original; its double
# must be present too. Pair them off, take the next
# smallest, and so on. Process values in order of absolute
# value so negatives and zeros follow the same rule, and
# handle zero separately (it is its own double, so it
# needs an even count).
#
# BRUTE FORCE (say it first): Try to remove each smallest
# element together with its double from a list
# Sort by absolute value and, for each element still in
# the list, remove it and one copy of its double
# (list.remove is a linear search). Correct and short; the
# repeated linear searches make it quadratic. Time O(n^2).
# Space O(n).


from collections import Counter


# BRUTE FORCE (say it first): Try to remove each smallest
# element together with its double from a list | time
# O(n^2)
def original_brute(changed):
    if len(changed) % 2:
        return []
    rest = sorted(changed, key=lambda x: (abs(x), x))
    out = []
    while rest:
```

```
        x = rest.pop(0)
        if 2 * x not in rest:
            return []
        rest.remove(2 * x)
        out.append(x)
    return out


# BASE QUESTION (optimal): Counter processed in order of
# absolute value
# Idea: Count the values. For each value x in order of
# absolute value with a remaining count, it needs count[x]
# copies of 2x: if count[2x] is smaller, fail; otherwise
# consume them and append x that many times.
def find_original(changed):
    # `changed` holds every original x and its double 2x,
    # in
    # any order. Smallest absolute value first: it can
    # only be
    # an original, never someone's double.
    if len(changed) % 2:
        return []
    count, out = Counter(changed), []
    for x in sorted(count, key=lambda v: (abs(v), v)):
        if count[x] == 0:
            continue
        if x == 0:
            # zeros pair with each other
            if count[0] % 2:
                return []
            out += [0] * (count[0] // 2)
            count[0] = 0
            continue
        # every x needs its own 2x
        if count[2 * x] < count[x]:
            return []
        out += [x] * count[x]
        count[2 * x] -= count[x]
        count[x] = 0
    return out


# FOLLOW-UP 1: Return the originals in the order they
# appear in the input.
# Answer: Compute the multiset of originals, then scan the
```

```
# input once and keep each element while its remaining
# original count is positive.
def find_original_in_order(changed):
    # originals in the order they appear in the input
    originals = find_original(changed)
    if not originals and changed:
        return []
    left = Counter(originals)
    out = []
    for x in changed:
        if left[x] > 0:
            left[x] -= 1
            out.append(x)
    return out


# FOLLOW-UP 2: Values are small non-negative integers.
# Answer: Use a counting array instead of sorting and a
# Counter: process values from small to large, O(n +
# range).
def find_original_small(changed, limit):
    # values 0..limit: a counting array, O(n + limit)
    if len(changed) % 2:
        return []
    count = [0] * (2 * limit + 2)
    for x in changed:
        count[x] += 1
    out = []
    if count[0] % 2:
        return []
    out += [0] * (count[0] // 2)
    for x in range(1, limit + 1):
```

```
        if count[x]:
            if count[2 * x] < count[x]:
                return []
            out += [x] * count[x]
            count[2 * x] -= count[x]
    return out


if __name__ == "__main__":
    got = sorted(find_original([1, 3, 4, 2, 6, 8]))
    assert got == [1, 3, 4]
    assert find_original([6, 3, 0, 1]) == []
    assert find_original([1]) == []
```

```
    assert find_original([0, 0, 0, 0]) == [0, 0]
    assert find_original([0, 0, 0]) == []
    # negative numbers: -2 is the double of -1
    neg = sorted(find_original([-1, -2, 4, 2]))
    assert neg == [-1, 2]
    ordered = find_original_in_order([4, 1, 2, 8, 2, 4])
    assert ordered == [4, 1, 2]
    small = sorted(find_original_small([1, 2, 4, 8], 8))
    assert small == [1, 4]
    assert find_original_small([1, 3], 3) == []
```

FUNCTION AND TEST CASES

Signature

```
def solution(changed: list[int]) -> list[int]:
```

Sample call

```
solution([2, 1, 4, 8])
```

Test cases

Input	Expected
`([2, 1, 4, 8],)`	`[1, 4]`
`([-4, -2, -2, -1, 1, 2],)`	`[-2, -1, 1]`

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

Site tags: math, hashmap, hard

HARD | 1Point3Acres | onsite-coding | 12 of 30

Maximum-Area Rectangle from Planar Points

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 177/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #963 Minimum Area Rectangle II

LeetCode match: LC 963 Minimum Area Free Rectangle (related)

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/max-area-rectangle-from-points

QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

Input: a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

APPROACH

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

HOW TO THINK ABOUT IT

Four points form a rectangle exactly when two of its diagonals share the same midpoint and have the same length. So you never need to pick four points: bucket every pair of points by (sum of the two points, squared distance). Two different pairs in one bucket are the two diagonals of a rectangle, and the area is the cross product of two adjacent sides.

HOW TO THINK ABOUT IT

Basis: LeetCode 963, Minimum Area Free Rectangle, with the objective flipped to maximum area.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points):
    best = 0
    for p, q, r in permutations(points, 3):
        if dot(q - p, r - p) != 0:
            continue  # no right angle at p
        s = q + r - p  # the fourth corner
        if s in points:
            area = abs(cross(q - p, r - p))
            best = max(best, area)
    return best
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

Three nested loops over the points, O(n^3). Bucketing pairs by their diagonal midpoint and length finds rectangles from O(n^2) pairs.

OPTIMAL SOLUTION · Bucket pairs by diagonal midpoint and length

TIME	SPACE
O(n^2) pairs (worst case O(n^3) if every pair shares a bucket)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can you do better than O(n^2) pairs?

Not in general: the answer can depend on all pairs, and even deciding whether any rectangle exists is believed to need about n^2 work. The practical gain is in the buckets: most buckets hold one pair, so the work after bucketing is small. State the worst case honestly (points on a circle put many pairs in few buckets).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximum-Area Rectangle from Planar Points
#
# Given 2D points, find the maximum area of a rectangle
# whose four corners are input points; sides need not be
# parallel to the axes. Candidates compare it to LC 963
# (Minimum Area Rectangle II) but maximizing. Interviewers
# ask to improve beyond an O(N^3) approach. Input: a list
```

```
# of 2D coordinate points. Return the maximum area of any
# rectangle whose four vertices are all input points.
# Rectangle sides do not need to be parallel to the
# coordinate axes. Follow-up pressure: start from an
# O(N^3) idea, then optimize toward a better complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can you do better than O(n^2) pairs?
#
# INTERVIEW NOTES
# Four points form a rectangle exactly when two of its
# diagonals share the same midpoint and have the same
# length. So you never need to pick four points: bucket
# every pair of points by (sum of the two points, squared
# distance). Two different pairs in one bucket are the two
# diagonals of a rectangle, and the area is the cross
# product of two adjacent sides.
#
# BRUTE FORCE (say it first): Try every triple with a
# right angle
# Pick a corner p and two neighbours q and r with (q - p)
# . (r - p) = 0, then check that the fourth point q + r -
# p exists. Correct and the first thing to say; cubic is
# too slow for thousands of points. Time O(n^3). Space
# O(n).


from collections import defaultdict
from itertools import combinations


# HELPERS for the brute force
def cross(o, p, q):
    # twice the signed area of triangle o, p, q
    ax, ay = p[0] - o[0], p[1] - o[1]
    bx, by = q[0] - o[0], q[1] - o[1]
    return ax * by - ay * bx
```

```
# BRUTE FORCE (say it first)
def max_rect_area_cubic(points):
    # first idea to say aloud: try every corner triple
    pts = set(map(tuple, points))
    best = 0
    for p in pts:
        for q in pts:
            for r in pts:
                if len({p, q, r}) < 3:
                    continue
                # right angle at p between p->q and p->r
                dot = ((q[0] - p[0]) * (r[0] - p[0])
                       + (q[1] - p[1]) * (r[1] - p[1]))
                # the fourth corner must also exist
                s = (q[0] + r[0] - p[0],
                     q[1] + r[1] - p[1])
                if dot == 0 and s in pts:
                    area = abs(cross(p, q, r))
                    best = max(best, area)
    return best


# HELPERS for the base question
# FOLLOW-UP 1: Can you do better than O(n^2) pairs?
# Answer: Not in general: the answer can depend on all
# pairs, and even deciding whether any rectangle exists is
# believed to need about n^2 work.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def all_rectangle_areas(points):
    # duplicates cannot help, so drop them
    pts = list(set(map(tuple, points)))
    buckets = defaultdict(list)
    # every pair is a possible diagonal
    for p, q in combinations(pts, 2):
        # same centre (sum of corners) and same length
        key = (p[0] + q[0], p[1] + q[1],
               (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
        buckets[key].append((p, q))
    # two diagonals in one bucket form a rectangle
    for pairs in buckets.values():
        for (p, q), (r, s) in combinations(pairs, 2):
            # area = |cross| of two sides leaving p
            yield abs(cross(p, r, s))
```

```


# BASE QUESTION (optimal): Bucket pairs by diagonal
# midpoint and length
# Idea: For every pair (p, q) use the key (p.x + q.x, p.y
# + q.y, squared length). All pairs with the same key are
# diagonals of one circle around the same centre, so any
# two of them form a rectangle.
def max_rect_area(points):
    # best area, or 0 when no rectangle exists
    return max(all_rectangle_areas(points), default=0)


# RELATED VARIANT: smallest area (LeetCode 963)
def min_rect_area(points):
    # LeetCode 963 asks for the smallest instead
    return min(all_rectangle_areas(points), default=0)


if __name__ == "__main__":
    # a tilted square of area 2
    sq = [(0, 0), (1, 1), (0, 2), (-1, 1)]
    assert max_rect_area(sq) == 2
    assert max_rect_area_cubic(sq) == 2
    # axis-aligned 2 x 1 rectangle plus a stray point
    rect = [(0, 0), (2, 0), (0, 1), (2, 1), (5, 5)]
    assert max_rect_area(rect) == 2
    # collinear points make no rectangle
    assert max_rect_area([(0, 0), (1, 1), (2, 2)]) == 0
    # the minimum version picks the small one
    two = rect + [(0, 3), (2, 3)]
    assert min_rect_area(two) == 2
    assert max_rect_area(two) == 6
```

NOTES FROM CANDIDATES

Candidates describe it as LC 963 (Minimum Area Rectangle II) with the objective flipped to maximum area.
Asked as a standalone L4 onsite coding round; in another L4 onsite it was the second problem after a CPU scheduling question, and the interviewer explicitly expected a better-than-cubic solution.
Clarify up front: whether duplicate points can appear, what to return when no rectangle exists (commonly 0), and whether integer or floating-point coordinates are used (area precision matters for rotated rectangles).

PREPARATION

Practice the diagonal-grouping idea: two point pairs form a rectangle when they share the same midpoint and the same length, so bucket all pairs by (midpoint, squared length) and compare pairs within each bucket.
Write the area computation from two adjacent side vectors and verify it on both an axis-aligned and a 45-degree rotated rectangle.
Prepare the complexity argument: O(N^2) pairs to bucket, with the worst case driven by bucket sizes.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Hashing

HARD | IN APP LC 41 | PracHub | Technical Screen | 13 of 30

Smallest Missing Positive Integer in Linear Time and Constant Extra Space

POSTED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 48 30 DAYS 0 PRIORITY 73/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Hashing

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #41 First Missing Positive

LeetCode match: LC 41 First Missing Positive

Posted on PracHub: 2026-09-09 (updated 2026-09-28)

Original: https://prachub.com/coding-questions/smallest-missing-positive-integer-in-linear-time-and-constant-extra-space

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**

QUESTION

- Explanation: **1** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **nums = [1, 2, 2, 3]**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

APPROACH

Cyclic placement: swap each value v in 1..n into index v-1 until settled, then the first index i where nums[i] != i+1 gives the answer (else n+1). O(n) time, O(1) space.

HOW TO THINK ABOUT IT

The answer is between 1 and n + 1, because n numbers can cover at most n of the positive integers. So the array itself can be used as the 'seen' table: put each value v (1 <= v <= n) at index v - 1. After that, the first index i where nums[i] != i + 1 gives the answer i + 1.

Basis: LeetCode 41, First Missing Positive.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    seen = set(nums)
    x = 1
    while x in seen:
        x += 1
    return x
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but O(n) extra memory, which is what the follow-up asks to remove. Placing each value v at index v - 1 inside the array itself, then finding the first mismatch, uses O(1) space.

OPTIMAL SOLUTION · Cyclic placement: put each value v at index v - 1

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without swapping (sign marking).

Replace out-of-range values by n + 1, then for every value v in 1..n make the entry at index v - 1 negative as a 'seen' flag; the first positive entry marks the missing number. Same O(n) time and O(1) space.

FOLLOW-UP 2 The array must not be modified.

Then O(1) space with O(n) time is not possible in general; copy the array (O(n) space) or accept O(n log n) time with a sort of a copy. State the trade-off.

FOLLOW-UP 3 Smallest missing non-negative integer, or the k-th missing positive.

Shift the target range by one for non-negative. For the k-th missing positive in a sorted array use binary search on 'how many numbers are missing before index i' = a[i] - (i + 1).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Smallest Missing Positive Integer in
# Linear Time and Constant Extra Space
#
# You are given an unsorted list of integers nums, which
# may contain negative numbers, zeros, duplicates, and
# values far larger than the length of the list. Return
# the smallest positive integer, that is the smallest
# integer that is at least 1, that does not appear in
# nums. In the interview, a first solution that ran in
# linear time but used extra memory proportional to the
# input was accepted as a starting point, and the
# interviewer then asked for the same result using only
# constant extra space. Aim for that tighter target. ###
# Function Signature python def
# first_missing_positive(nums: list[int]) -> int: ###
# Rules - Target complexity: O(n) time and O(1) auxiliary
# space, where n = len(nums). The list may be rearranged
# or overwritten in place; its contents after the call do
# not matter. - Return only the missing value.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without swapping (sign marking).
# 2. The array must not be modified.
# 3. Smallest missing non-negative integer, or the k-th
# missing positive.
#
# INTERVIEW NOTES
# The answer is between 1 and n + 1, because n numbers can
# cover at most n of the positive integers. So the array
# itself can be used as the 'seen' table: put each value v
# (1 <= v <= n) at index v - 1. After that, the first
# index i where nums[i] != i + 1 gives the answer i + 1.
#
# BRUTE FORCE (say it first): Hash set of the values, then
# count up
# Put all numbers in a set and test 1, 2, 3, ... until one
# is missing. Linear time but linear extra memory, which
# is the version the interviewer wants to improve. Time
# O(n). Space O(n).
```

```
from bisect import bisect_left


# BASE QUESTION (optimal): Cyclic placement: put each
```

```
# value v at index v - 1
# Idea: For each position, while nums[i] is in 1..n and
# not already at its home index, swap it to index nums[i]
# - 1. Each swap puts one value in its final place, so the
# total number of swaps is at most n.
def first_missing_positive(nums):
    # put each value v in 1..n at index v - 1 by swapping
    a, n = list(nums), len(nums)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            j = a[i] - 1
            a[i], a[j] = a[j], a[i]
    # the first index that does not hold index + 1
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1


# EASY VERSION: O(n) memory (say it first)
def first_missing_set(nums):
    # the easy O(n) memory answer (fine as a first
    # version)
    seen, v = set(nums), 1
    while v in seen:
        v += 1
    return v


# VARIANT: k-th missing positive
def kth_missing_positive(sorted_nums, k):
    # follow-up: k-th missing positive in a sorted array.
    # missing before index i is a[i] - (i + 1): binary
    # search
    lo, hi = 0, len(sorted_nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if sorted_nums[mid] - (mid + 1) < k:
            lo = mid + 1
        else:
            hi = mid
    return lo + k


# FOLLOW-UP 1: Do it without swapping (sign marking).
```

```
# Answer: Replace out-of-range values by n + 1, then for
# every value v in 1..n make the entry at index v - 1
# negative as a 'seen' flag; the first positive entry
# marks the missing number. Same O(n) time and O(1) space.
def first_missing_marking(nums):
    # no swaps: use the sign of a[v - 1] as a "seen" flag
    a, n = list(nums), len(nums)
    # values outside 1..n cannot matter: replace them
    for i in range(n):
        if a[i] <= 0 or a[i] > n:
            a[i] = n + 1
    for i in range(n):
        v = abs(a[i])
        if v <= n:
            a[v - 1] = -abs(a[v - 1])
    # the first positive entry marks the missing number
    for i in range(n):
        if a[i] > 0:
            return i + 1
    return n + 1
```

```
if __name__ == "__main__":
    cases = [([3, 4, -1, 1], 2), ([1, 2, 0], 3),
             ([7, 8, 9, 11, 12], 1), ([], 1),
             ([1, 1, 1], 2), ([2, 1], 3)]
    for nums, want in cases:
        assert first_missing_positive(nums) == want
        assert first_missing_marking(nums) == want
        assert first_missing_set(nums) == want
    # the input list is left untouched
    original = [3, 4, -1, 1]
    first_missing_positive(original)
    assert original == [3, 4, -1, 1]
    # 2, 3, 4, 7, 11: the 5th missing positive is 9
    assert kth_missing_positive([2, 3, 4, 7, 11], 5) == 9
    assert kth_missing_positive([1, 2, 3, 4], 2) == 6
```

FUNCTION AND TEST CASES

Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Sample call

```
first_missing_positive([4, -2, 1, 2])
```

Test cases

Input	Expected
([4, -2, 1, 2],)	3
([5, 6, 100, -3],)	1

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

Site tags: array, sliding-window, hashmap, prefix-sum, medium

MEDIUM | 1Point3Acres | onsite-coding | 14 of 30

Range Duplicate Check with Constant-Time Queries

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 211/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/range-duplicate-check-constant-time-queries

QUESTION

Decide whether the elements in a range of an array are all unique. Progression: set-based duplicate check -> sliding window -> many range queries where reads dominate, requiring O(1) per query after linear-time preprocessing / updates with linear space.

Base: given an array and a range [l, r], determine whether the elements in that range are all unique.
The problem is staged:

Easy: duplicate detection with a set.
Sliding-window duplicate checking.
Many queries over different ranges: optimize for a read-heavy, write-light workload.

Target for the final stage: O(1) per query, preprocessing (and updates, in one version) in linear time, linear space.

APPROACH

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

HOW TO THINK ABOUT IT

A range [l, r] has a duplicate exactly when some index i in it has its next equal element at an index <= r. Let next[i] be the next index holding the same value and reach[l] the smallest next[i] over all i >= l. Then the range is all-unique iff reach[l] > r, a single comparison per query after one linear pass.

HOW TO THINK ABOUT IT

Basis: Written for this question (a precomputed next-occurrence array, similar in spirit to LeetCode 3, Longest Substring Without Repeating Characters).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def all_unique(a, l, r):
    seen = set()
    for x in a[l:r + 1]:
        if x in seen:
            return False
        seen.add(x)
    return True
```

TIME	SPACE
O(r - l) per query	O(r - l)

WHY IT IS SLOW

Each query costs the length of the range. A next-occurrence array with suffix minimums answers each query with one comparison.

OPTIMAL SOLUTION · Next-occurrence array with suffix minimum

TIME	SPACE
O(n) preprocessing, O(1) per query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array changes between queries.

Rebuild next and reach in O(n) after a batch of changes when reads dominate. If single-element updates are frequent, keep per-value sorted index lists and a segment tree of next-occurrence minima so an update and a query cost O(log n) instead.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Range Duplicate Check with Constant-Time
# Queries
#
# Decide whether the elements in a range of an array are
# all unique. Progression: set-based duplicate check ->
# sliding window -> many range queries where reads
# dominate, requiring O(1) per query after linear-time
# preprocessing / updates with linear space. Base: given
# an array and a range [l, r], determine whether the
# elements in that range are all unique. The problem is
```

```python
# staged: Easy: duplicate detection with a set. Sliding-
# window duplicate checking. Many queries over different
# ranges: optimize for a read-heavy, write-light workload.
# Target for the final stage: O(1) per query,
# preprocessing (and updates, in one version) in linear
# time, linear space.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array changes between queries.
#
# INTERVIEW NOTES
# A range [l, r] has a duplicate exactly when some index i
# in it has its next equal element at an index <= r. Let
# next[i] be the next index holding the same value and
# reach[l] the smallest next[i] over all i >= l. Then the
# range is all-unique iff reach[l] > r, a single
# comparison per query after one linear pass.
#
# BRUTE FORCE (say it first): Put the range in a set
# Insert the elements of the range into a set and compare
# sizes. Correct; every query costs the length of the
# range. Time O(r - l) per query. Space O(r - l).


from collections import Counter


# BRUTE FORCE (say it first): Put the range in a set |
# time O(r - l) per query
def range_unique_brute(a, l, r):
    return len(set(a[l : r + 1])) == r - l + 1


# HELPERS for the base question
INF = float("inf")
```

```python
# EASY VERSION (say it first)
def unique_with_set(a, l, r):
    # easy version: put the range in a set
    return len(set(a[l:r + 1])) == r - l + 1


# STAGE 2: sliding window
def longest_unique_window(a):
    # stage 2: sliding window with a counter
    seen, left, best = Counter(), 0, 0
    for right, x in enumerate(a):
        seen[x] += 1
        # shrink until x occurs only once
        while seen[x] > 1:
            seen[a[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best


# BASE QUESTION (optimal): Next-occurrence array with
# suffix minimum
# Idea: Scan right to left with a map from value to its
# latest index to fill next[i] (infinity if none). Then
# reach[i] = min(next[i], reach[i + 1]). Query(l, r) is
# reach[l] > r.
# FOLLOW-UP 1: The array changes between queries.
# Answer: Rebuild next and reach in O(n) after a batch of
# changes when reads dominate.
class RangeUnique:
    """O(n) build, then O(1) per query."""

    def __init__(self, a):
        self.a = list(a)
        self.rebuild()
```

```python
    def rebuild(self):
        a, n = self.a, len(self.a)
        # nxt[i] = next index holding the same value
        nxt, last = [INF] * n, {}
        for i in range(n - 1, -1, -1):
            nxt[i] = last.get(a[i], INF)
            last[a[i]] = i
```

```python
        # reach[i] = earliest repeat from index i onward
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            self.reach[i] = min(nxt[i], self.reach[i + 1])

    def unique(self, l, r):
        # no repeat starts in [l, r] and ends inside it
        return self.reach[l] > r

    # FOLLOW-UP 1: the array changes between queries
    def update(self, i, value):
        # reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()


if __name__ == "__main__":
    a = [1, 2, 3, 1, 4]
    q = RangeUnique(a)
    for l in range(len(a)):
        for r in range(l, len(a)):
            plain = unique_with_set(a, l, r)
            assert q.unique(l, r) == plain
    assert q.unique(0, 2) and not q.unique(0, 3)
    assert longest_unique_window(a) == 4
    # an update can create or remove a duplicate
    q.update(3, 9)
    assert q.unique(0, 4)
    q.update(0, 9)
    assert not q.unique(0, 3)
```

NOTES FROM CANDIDATES

In one L4 onsite, this was the second question of a round that opened with weighted job scheduling; the interviewer asked specifically for the precomputation that makes each query O(1). The interviewer states the complexity targets explicitly, so the round is judged on hitting them, not on a working brute force. Clarify whether the array can change between queries (one version mentions linear-time updates) and whether ranges are inclusive.

PREPARATION

Precompute, for each index i, the previous index holding the same value (prev[i]) with a single hashmap pass.
Turn that into an O(1) query: a range [l, r] is duplicate-free exactly when every prev[j] < l for j in the range — precompute a running "furthest start that keeps [start, j] unique" array so each query is one comparison.
Explain the cost of an update to one element (which prev entries change) and why it stays linear.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

Site tags: data-structure, streaming, sliding-window

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 15 of 30

Temperature Monitoring Data Structure

ASKED 2026-09-10 | 40 DAYS AGO | LAST 90 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 223/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/temperature-monitor-moving-window

QUESTION

Implement temperature insertion, a moving average, and a constant-time maximum query for a recent window. Clarify whether the window is based on reading count or elapsed days. SWEdata-structurestreamingsliding-windowFrequencySingle reportLast asked2026-09-10Stageonsite-coding

APPROACH

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans. Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the window is the last N readings or the last N days, what moving average should return before the window fills, and the number of readings.

BRUTE FORCE TO SAY OUT LOUD

Store everything in a list and compute the window average and max by scanning. O(window) per query.

OPTIMAL IDEA

For a count window keep a deque of the last N readings and a running sum for the average, and a monotonic decreasing deque for the maximum: pop smaller values from the back on insert and drop the front when it expires. All operations O(1) amortized. For time windows store (time, value) and evict by timestamp.

DRY-RUN THESE CASES

N = 1, duplicate maxima, negative readings, the maximum expiring, fewer readings than the window.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Minimum too, median over a window, or out-of-order timestamps.

PITFALLS

Removing the wrong element from the max deque when equal values repeat. Floating-point drift in the running sum.

HOW TO THINK ABOUT IT

Three operations on the last N readings. The average only needs a running sum, and the maximum needs a monotonic deque that drops smaller values from the back as larger ones arrive, so the front is always the max of the window. Decide first whether the window is a count of readings or a span of time.

Basis: Written; a sliding window with an average (LeetCode 346) and a window maximum (LeetCode 239), here as one data structure.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
readings = []
def insert(x):
    readings.append(x)
def average():
    window = readings[-N:]
    return sum(window) / len(window)
def maximum():
    return max(readings[-N:])
```

TIME	SPACE
O(N) per average or max	O(N)

WHY IT IS SLOW

Every query scans the window, O(N). A running sum gives the average in O(1), and a deque of decreasing values gives the maximum in O(1) amortised.

OPTIMAL SOLUTION · Running sum plus a monotonic deque

TIME	SPACE
O(1) amortized per operation	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The window is the last D days (elapsed time), not the last N readings.

Store (timestamp, value) pairs and evict from the front while the timestamp is at or before now minus the span, in both the window and the max deque. Everything else is unchanged.

FOLLOW-UP 2 Also support minimum, or the median.

Minimum uses a second monotonic deque with the opposite comparison. A median needs two heaps with lazy deletion or a sorted container, O(log N) per update, because it is not decomposable like sum and max.

FOLLOW-UP 3 Floating-point drift in the running sum.

Recompute the sum from the window every so often (say every N inserts), or use math.fsum or integer hundredths of a degree to avoid drift.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Temperature Monitoring Data Structure
#
# Implement temperature insertion, a moving average, and a
# constant-time maximum query for a recent window. Clarify
# whether the window is based on reading count or elapsed
# days.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The window is the last D days (elapsed time), not the
# last N readings.
# 2. Also support minimum, or the median.
# 3. Floating-point drift in the running sum.
#
# INTERVIEW NOTES
# Three operations on the last N readings. The average
# only needs a running sum, and the maximum needs a
# monotonic deque that drops smaller values from the back
# as larger ones arrive, so the front is always the max of
# the window. Decide first whether the window is a count
# of readings or a span of time.
#
# BRUTE FORCE (say it first): Keep a list and scan the
# window on every query
# Store every reading and compute sum and max over the
# last N each time. Correct and the right oracle; the cost
# is a full window scan for each query. Time O(N) per
# average or max. Space O(N).


from collections import deque


# BRUTE FORCE (say it first): Keep a list and scan the
# window on every query | time O(N) per average or max
class MonitorBrute:
    def __init__(self, n):
        self.n, self.data = n, []

    def insert(self, x):
        self.data.append(x)

    def average(self):
        w = self.data[-self.n :]
        return sum(w) / len(w)
```

```
    def maximum(self):
        return max(self.data[-self.n :])
```

```
# HELPERS for the base question
class MonotoneQueue:
    """Best window value is kept at the front."""

    def __init__(self, better):
        self.items = deque()
        # better(a, b): a is at least as good as b
        self.better = better

    def push(self, key, value):
        # values the new one beats can never be best
        while self.items and self.better(
                value, self.items[-1][1]):
            self.items.pop()
        self.items.append((key, value))

    def drop_while(self, too_old):
        # forget entries that left the window
        while self.items and too_old(self.items[0][0]):
            self.items.popleft()

    def best(self):
        return self.items[0][1]


# FOLLOW-UP 2: Also support minimum, or the median.
# Answer: Minimum uses a second monotonic deque with the
# opposite comparison. A median needs two heaps with lazy
# deletion or a sorted container, O(log N) per update,
# because it is not decomposable like sum and max.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class SlidingStats:
    def __init__(self):
        self.items = deque()
        self.total = 0
        self.maxq = MonotoneQueue(lambda a, b: a >= b)
        self.minq = MonotoneQueue(lambda a, b: a <= b)

    def add(self, key, value):
        self.items.append((key, value))
```

```
        self.total += value
        self.maxq.push(key, value)
        self.minq.push(key, value)

    def expire(self, too_old):
        # drop readings that left the window everywhere
        while self.items and too_old(self.items[0][0]):
            self.total -= self.items.popleft()[1]
        self.maxq.drop_while(too_old)
        self.minq.drop_while(too_old)

    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.maxq.best()

    def minimum(self):
        return self.minq.best()


# BASE QUESTION (optimal): Running sum plus a monotonic
# deque
# Idea: Keep the window in a deque and a running sum; on
# insert add the value and drop the oldest when the window
# is full.
class Monitor(SlidingStats):
    """Window = the last n readings."""

    def __init__(self, n):
        super().__init__()
```

```
        self.n, self.seen = n, 0

    def insert(self, x):
        # the reading number is its key
        self.add(self.seen, x)
        self.seen += 1
        # keep only the newest n readings
        limit = self.seen - self.n
        self.expire(lambda key: key < limit)


# FOLLOW-UP 1: The window is the last D days (elapsed
# time), not the last N readings.
# Answer: Store (timestamp, value) pairs and evict from
```

```
# the front while the timestamp is at or before now minus
# the span, in both the window and the max deque.
# Everything else is unchanged.
class TimedMonitor(SlidingStats):
    """Window = readings from the last `span` days."""

    def __init__(self, span):
        super().__init__()
        self.span = span

    def insert(self, day, x):
        self.add(day, x)
        # drop readings at or before day - span
        cutoff = day - self.span
        self.expire(lambda key: key <= cutoff)


if __name__ == "__main__":
    m = Monitor(3)
    for x in [5, 1, 4, 2]:
        m.insert(x)
    # window is [1, 4, 2]
    assert m.maximum() == 4 and m.minimum() == 1
    assert m.average() == 7 / 3
    m.insert(0)
    # window is [4, 2, 0]
    assert m.maximum() == 4 and m.minimum() == 0
    t = TimedMonitor(2)
    for day, x in [(1, 9), (2, 3), (3, 5)]:
        t.insert(day, x)
    # day 1 is two days old, so it left the window
    assert t.maximum() == 5 and t.average() == 4
```

NOTES FROM CANDIDATES

The introductory description refers to the most recent N days, while the API requirements refer to the most recent N readings. Clarify the window definition before coding; the day-based variant does not specify how timestamps enter the interface. Empty-window behavior and insertion complexity are unspecified.

For the reading-count window with positive N, retain the last N readings in a FIFO queue and maintain their sum. Add the new reading and subtract any expired reading. Agree whether a partially filled window returns its current mean or is unavailable. Maintain a second deque of reading indices with nonincreasing temperatures. Expire old indices at the front and remove smaller or equal temperatures from the back before adding the new index. The front holds the current maximum; indices distinguish duplicate readings.

Average and maximum queries are O(1). Each reading enters and leaves each deque at most once, so insertion is amortized O(1) but can take O(N) for one update; auxiliary space is O(N). This implements the reading-count contract. A day-based contract additionally needs timestamps, expiration timing, and an ordering rule.

PREPARATION

Implement all three reading-count operations using a running sum and deques; compare outputs with direct window scans. Test N=1, duplicate maxima, negative readings, expiration of the maximum, and partially filled windows under an explicitly chosen contract; explain amortized versus worst-case insertion cost.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

Site tags: array, simulation, sliding-window, medium

MEDIUM | 1Point3Acres | onsite-coding | 16 of 30

Piano Hand Movement Grouping

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 205/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/piano-hand-movement-grouping

QUESTION

EXAMPLES

```
With notes = [1, 3, 5, 7, 9, 11] and k = 4, the
    stated output is 2.
Earlier examples use [1,2,3,4,5] -> 0 and [5,9,1] ->
    1, without stating the reach parameter.
```

APPROACH

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

INTERVIEW-DAY PLAN

CLARIFY FIRST

The statement is ambiguous, so ask: does the hand cover a fixed interval of keys [p, p + k], is the interval fixed until the hand lifts, does the first placement count as a move, and do notes arrive in the given order. Notice the examples are not all consistent, so ask which one is the reference.

BRUTE FORCE TO SAY OUT LOUD

Try every starting position for each group and take the minimum. Exponential or quadratic.

OPTIMAL IDEA

Greedy one-pass scan: start a group at the first note with the hand covering [note, note + k]. For each next note, if it is inside the interval keep going; if not, start a new group at that note. The answer is the number of groups (or groups minus one if the first placement is free). Group boundaries come from the same scan. O(n).

DRY-RUN THESE CASES

A single note, the example [1, 3, 5, 7, 9, 11] with k = 4, notes going down and then up, k = 0.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

A hand that can slide, two hands, or return the group boundaries.

PITFALLS

Counting the initial placement inconsistently. Assuming notes are sorted.

HOW TO THINK ABOUT IT

Turn the physical rule into an invariant: the current group is valid while (largest note - smallest note) <= k, because you can place the hand at the smallest. The first note that breaks it must start a new group, and starting later never helps, so a single greedy pass is optimal. Ask whether the first placement counts as a move.

Basis: Written; interval grouping, like LeetCode 452 and 1326. Assumed rule (confirm it, the examples disagree): a hand placed on key p covers p to p + k inclusive and stays put until it is lifted; the notes come in order; the answer is the minimum number of placements, and the number of lifts is one fewer.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (n + 1)
best[0] = 0
for j in range(1, n + 1):
    for i in range(j):
        part = notes[i:j]
        if max(part) - min(part) <= k:
            best[j] = min(best[j], best[i] + 1)
answer = best[n]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every end tries every start and re-checks the range, O(n^2). A greedy pass that starts a new group only when the next note no longer fits is O(n).

OPTIMAL SOLUTION · Greedy scan, tracking the group's smallest and largest note

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where each group starts and where to place the hand.

Record the start index when a group opens and emit (start, end, smallest note) when it closes. The state transition is unchanged.

FOLLOW-UP 2 Count lifts, not placements.

Lifts are placements minus one if the first placement is free; the examples differ on this, so ask and state it. In code it is a single subtraction on the group count.

FOLLOW-UP 3 The hand may slide without a lift (costing something).

Then it becomes a DP over positions with a cost per slide distance, solved with the O(n * range) DP or by greedy if the cost is monotone. Ask for the cost model.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Piano Hand Movement Grouping
#
# Examples: With notes = [1, 3, 5, 7, 9, 11] and k = 4,
# the stated output is 2. Earlier examples use [1,2,3,4,5]
# -> 0 and [5,9,1] -> 1, without stating the reach
# parameter.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where each group starts and where to place the
# hand.
# 2. Count lifts, not placements.
# 3. The hand may slide without a lift (costing
# something).
#
# INTERVIEW NOTES
# Turn the physical rule into an invariant: the current
# group is valid while (largest note - smallest note) <=
# k, because you can place the hand at the smallest. The
# first note that breaks it must start a new group, and
# starting later never helps, so a single greedy pass is
# optimal. Ask whether the first placement counts as a
# move.
#
# BRUTE FORCE (say it first): DP over every way to cut the
# notes into consecutive groups
# best[j] is the fewest groups for the first j notes: try
# every start i of the last group and require max - min of
# notes[i:j] to be at most k. Quadratic but easy to trust.
# Time O(n^2). Space O(n).


# BRUTE FORCE (say it first): DP over every way to cut the
# notes into consecutive groups | time O(n^2)
def groups_brute(notes, k):
    n = len(notes)
    INF = float("inf")
    best = [0] + [INF] * n
    for j in range(1, n + 1):
        for i in range(j):
            if max(notes[i:j]) - min(notes[i:j]) <= k:
                best[j] = min(best[j], best[i] + 1)
    return best[n]


# HELPERS for the base question
```

```
def hand_positions(notes, k):
    # split the notes into groups that one hand position
    # (a span of at most k) can play, as early as possible
    groups = []
    for i, x in enumerate(notes):
        if groups:
            start, lo, hi = groups[-1]
            # does the note fit the current span?
            lo2, hi2 = min(lo, x), max(hi, x)
            if hi2 - lo2 <= k:
                groups[-1] = (start, lo2, hi2)
                continue
        # otherwise the hand must move: a new group starts
        groups.append((i, x, x))
    return groups


# BASE QUESTION (optimal): Greedy scan, tracking the
# group's smallest and largest note
# Idea: Keep lo and hi of the current group. Extend while
# hi - lo stays within k, otherwise close the group and
# start a new one at this note.
def groups(notes, k):
    # how many hand placements are needed
    return len(hand_positions(notes, k))


# FOLLOW-UP 1: Return where each group starts and where to
# place the hand.
# Answer: Record the start index when a group opens and
# emit (start, end, smallest note) when it closes. The
# state transition is unchanged.
def plan(notes, k):
    # where each group starts and ends, and where to put
    # the hand (on the lowest note of the group)
    out, spots = [], hand_positions(notes, k)
    for n, (start, lo, hi) in enumerate(spots):
        end = spots[n + 1][0] - 1 if n + 1 < len(spots) \
            else len(notes) - 1
        out.append((start, end, lo))
    return out


# FOLLOW-UP 2: Count lifts, not placements.
# Answer: Lifts are placements minus one if the first
```

```
# placement is free; the examples differ on this, so ask
# and state it. In code it is a single subtraction on the
# group count.
def lifts(notes, k):
    # the first placement is free; every move is a lift
    return max(groups(notes, k) - 1, 0)


if __name__ == "__main__":
    # the stated example: 2 placements for reach 4
    assert groups([1, 3, 5, 7, 9, 11], 4) == 2
    # the earlier examples, counted as lifts
    assert lifts([1, 2, 3, 4, 5], 4) == 0
    assert lifts([5, 9, 1], 4) == 1
    assert groups([], 4) == 0
    assert plan([1, 3, 5, 7, 9, 11], 4) == [(0, 2, 1),
                                            (3, 5, 7)]
    # a reach of 0 means a new placement for every change
    assert groups([2, 2, 3], 0) == 2
```

NOTES FROM CANDIDATES

Clarify whether a placement is fixed until the hand lifts, whether the covered interval may slide without counting as a lift, and whether the initial placement counts. These details materially change the answer.
The six-note example's explanation says a placement at 7 covers keys 7–11 but still counts another lift for 9 and 11. Treat that as an ambiguity to resolve before coding rather than silently assuming the output convention.
Maintain the feasible placement interval for the current group. The follow-up printout should reuse the same grouping state instead of duplicating the movement logic.

PREPARATION

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

Site tags: array, two-pointer, medium

MEDIUM | 1Point3Acres | onsite-coding | 17 of 30

Sum of Good Subarrays (Adjacent Diff ±1)

ASKED 2026-02-08 | 254 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 208/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-08 | Roles: MLE, SWE

Original: https://www.1point3acres.com/interview/problems/company/google/good-subarrays-diff-one

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
    -1).
```

APPROACH

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

HOW TO THINK ABOUT IT

A good subarray is a stretch where every step is +1 or every step is -1, so every good subarray ending at index i lies inside the current run. Instead of listing subarrays, keep for each direction the total of all suffix sums inside the run: extending a run of length L by a new element x adds (L + 1) * x to that total. Both directions can be updated in O(1), and the singleton is counted once.

Basis: Written for this question (a run-length DP in the style of LeetCode 2262, Total Appeal of A String).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if is_good(part):
                total += sum(part)
    return total
def is_good(part):
    pairs = zip(part, part[1:])
    steps = {y - x for x, y in pairs}
    return steps <= {1} or steps <= {-1}
```

TIME	SPACE
O(n^2) or O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays (O(n^3) with the sums). Run totals per direction update in O(1) per element, so one pass is O(n).

OPTIMAL SOLUTION · Two running run totals

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the singleton subtracted?

A one-element subarray [x] is both an increasing and a decreasing run of length 1, so it appears in both running totals. Subtracting x once at each index keeps it counted exactly once. A subarray of length 2 or more can only be in one total because its step is +1 or -1, not both.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Sum of Good Subarrays (Adjacent Diff ±1)
#
# MLE L5 coding round: sum over all contiguous subarrays
# whose elements form a strictly monotonic ±1 step
# sequence (every adjacent pair differs by exactly +1, OR
# every adjacent pair differs by exactly -1). Required
# time complexity is O(N). Input: integer array arr of
# length N. A subarray is good if its consecutive
# differences are either all +1 (monotone increasing by 1)
# or all -1 (monotone decreasing by 1). Singleton
# subarrays are always good. Output: the sum of element-
# sums across all good subarrays (each good subarray
# contributes the sum of its elements once). Required:
# O(N) time. Examples: arr = [3, 5, 6, 7, 6] Good
# subarrays: singletons: [3], [5], [6], [7], [6] monotone
# +1 runs: [5, 6], [6, 7], [5, 6, 7] monotone -1 runs: [7,
# 6] Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
# -1).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the singleton subtracted?
#
# INTERVIEW NOTES
# A good subarray is a stretch where every step is +1 or
# every step is -1, so every good subarray ending at index
# i lies inside the current run. Instead of listing
# subarrays, keep for each direction the total of all
# suffix sums inside the run: extending a run of length L
# by a new element x adds (L + 1) * x to that total. Both
# directions can be updated in O(1), and the singleton is
# counted once.
#
# BRUTE FORCE (say it first): Enumerate every subarray
# Check every subarray's differences and add up its
# elements. Correct and the oracle; quadratic or worse,
# but the problem asks for O(n). Time O(n^2) or O(n^3).
# Space O(1).


# BRUTE FORCE (checker)
def good_subarray_sum_brute(a):
    # check every subarray: the oracle for the demo
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
```

```python
            steps = {a[k + 1] - a[k] for k in range(i, j)}
            if steps <= {1} or steps <= {-1}:
                total += sum(a[i:j + 1])
    return total


# BASE QUESTION (optimal): Two running run totals
# Idea: For an increasing run of length L ending at i let
# S be the sum of all suffix sums of the run. When the
# next element x extends it, S becomes S + (L + 1) * x and
# L becomes L + 1; a break restarts at S = x, L = 1.
def good_subarray_sum(a):
    # a good subarray has all steps +1 or all steps -1.
    # For each index keep, per direction, the sum of the
    # sums of all good subarrays that END here.
    total = 0
    up_sum = up_len = down_sum = down_len = 0
    for i, x in enumerate(a):
        step = x - a[i - 1] if i else None
        if step == 1:
            # extend the +1 run: every old subarray grows
            # by
            # x, and [x] alone is new: (len + 1) * x in
            # all
            up_sum += (up_len + 1) * x
            up_len += 1
        else:
            up_sum, up_len = x, 1
```

```python
        if step == -1:
            down_sum += (down_len + 1) * x
            down_len += 1
        else:
            down_sum, down_len = x, 1
        # [x] is in both totals: count it once
        total += up_sum + down_sum - x
    return total


if __name__ == "__main__":
    arr = [3, 5, 6, 7, 6]
    # singletons 27 + [5,6] 11 + [6,7] 13 + [5,6,7] 18 +
    # [7,6] 13
    assert good_subarray_sum(arr) == 82
    assert good_subarray_sum([]) == 0
    assert good_subarray_sum([4]) == 4
```

```python
    # the O(n) answer matches the brute force everywhere
    import itertools
    for n in range(7):
        every = itertools.product([1, 2, 3, 4], repeat=n)
        for a in every:
            assert good_subarray_sum(list(a)) == (
                good_subarray_sum_brute(list(a)))
```

NOTES FROM CANDIDATES

Walk the array maintaining two run lengths: upRun (current trailing run of +1 steps) and downRun (current trailing run of -1 steps). At index i, only one of them is nonzero (the other resets to 1).
Each index i ends upRun + downRun - 1 good subarrays (subtracting the double-counted singleton). For each ending position, contribute the running prefix-sum tail of the active run.
Multiple candidates report getting stuck — verbalize the "contribution per index" framing out loud before coding.
Easy to mis-handle the reset boundary; double-check with a 2-element example before writing the loop.

PREPARATION

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

Site tags: data-structure, two-pointer, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 18 of 30

Run-Length Vector Storage + Dot Product

ASKED 2026-01-21 | 272 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 173/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1570 Dot Product of Two Sparse Vectors

Frequency: Single report | Last asked: 2026-01-21 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/compressed-vector-dot-product

QUESTION

EXAMPLES

```
Logical A = [3, 3, 3, 0, 0, 0, 4, 4]
Logical B = [1, 1, 1, 1, 2, 2, 2, 2]
Encoded A (RLE): [(3,3),(0,3),(4,2)]
Encoded B (RLE): [(1,4),(2,4)]
Dot product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 =
    9 + 0 + 16 = 25
```

APPROACH

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the encoding format (value, count pairs), whether runs of zeros are stored, the logical length of both vectors (must be equal), and whether updates are needed.

BRUTE FORCE TO SAY OUT LOUD

Expand both vectors and multiply element by element. O(n) memory.

OPTIMAL IDEA

Two pointers over the run lists. Take the overlap, which is min of the two remaining counts, add value_a * value_b * overlap, subtract the overlap from both, and advance whichever run is exhausted. O(r1 + r2), O(1) extra space.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

The example giving 25. One vector of a single run. Runs that end at the same time. All zeros.

LIKELY FOLLOW-UPS

A sparse vector against an RLE vector, setting an element, or dot product of k vectors.

PITFALLS

Advancing only one pointer when both runs finish together. Overflowing the product for large counts.

HOW TO THINK ABOUT IT

A run is a stretch where both numbers are constant, so the dot product of two runs is value times value times the length of their overlap. Walk the two run lists like the merge step of merge sort: consume the smaller remaining length from both, and advance whichever run you finished.

Basis: LeetCode 1570, Dot Product of Two Sparse Vectors, with run-length encoding as the compression.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def decode(runs):
    dense = []
    for value, count in runs:
        dense += [value] * count
    return dense
def brute(runs_a, runs_b):
    a, b = decode(runs_a), decode(runs_b)
    return sum(x * y for x, y in zip(a, b))
```

TIME	SPACE
O(logical length)	O(logical length)

WHY IT IS SLOW

It needs memory and time proportional to the logical length, which can be 10^9. Walking the two run lists together with two pointers costs only the number of runs.

OPTIMAL SOLUTION · Two-pointer over runs

TIME	SPACE
O(r1 + r2)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if one vector is mostly zeros and stored sparsely, and the other as runs?

Iterate the sparse side's nonzero indices and binary-search the run list for each index to read the other value: O(s log r). Choose the smaller representation to drive the loop.

FOLLOW-UP 2 How should the data be updated if you set element i?

Locate the run containing i (binary search over prefix run lengths), then split it into at most three runs and merge equal neighbours. That is O(log r) to find plus O(r) in the worst case to splice a list; use a balanced tree of runs for frequent updates.

FOLLOW-UP 3 One vector is a sparse map (index to value) and the other is run-length encoded.

Iterate the smaller structure (the sparse non-zeros) and binary search the run list for each index using prefix run lengths: O(s log r) instead of expanding anything.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Run-Length Vector Storage + Dot Product
#
# Examples: Logical A = [3, 3, 3, 0, 0, 0, 4, 4] Logical B
# = [1, 1, 1, 1, 2, 2, 2, 2] Encoded A (RLE):
# [(3,3),(0,3),(4,2)] Encoded B (RLE): [(1,4),(2,4)] Dot
# product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 = 9 + 0
# + 16 = 25
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if one vector is mostly zeros and stored
# sparsely, and the other as runs?
# 2. How should the data be updated if you set element i?
# 3. One vector is a sparse map (index to value) and the
# other is run-length encoded.
#
# INTERVIEW NOTES
# A run is a stretch where both numbers are constant, so
# the dot product of two runs is value times value times
# the length of their overlap. Walk the two run lists like
# the merge step of merge sort: consume the smaller
# remaining length from both, and advance whichever run
# you finished.
#
# BRUTE FORCE (say it first): Expand both vectors and
# multiply
# Decompress each run into repeated values and add up the
# element-wise products. Correct and the oracle; with
# logical lengths like 10^9 it cannot even be stored. Time
# O(logical length). Space O(logical length).


from bisect import bisect_right
from itertools import accumulate


# BRUTE FORCE (say it first): Expand both vectors and
# multiply | time O(logical length)
def rle_dot_expand(a, b):
    x = [v for v, c in a for _ in range(c)]
    y = [v for v, c in b for _ in range(c)]
    return sum(p * q for p, q in zip(x, y))


# BASE QUESTION (optimal): Two-pointer over runs
# Idea: Each run is (value, count). Compare the remaining
# lengths of the two current runs, multiply the values by
# the overlap, then advance whichever run was exhausted
# (or both).
def rle_dot(a, b):
    # two pointers over the runs: never expand anything
    i = j = 0
    left_a = a[0][1] if a else 0
    left_b = b[0][1] if b else 0
    total = 0
    while i < len(a) and j < len(b):
        # both runs overlap for min(left_a, left_b)
        # positions
        take = min(left_a, left_b)
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            left_a = a[i][1] if i < len(a) else 0
        if left_b == 0:
            j += 1
            left_b = b[j][1] if j < len(b) else 0
    return total


# FOLLOW-UP 1: What if one vector is mostly zeros and
# stored sparsely, and the other as runs?
# Answer: Iterate the sparse side's nonzero indices and
# binary-search the run list for each index to read the
# other value: O(s log r). Choose the smaller
# representation to drive the loop.
# FOLLOW-UP 3: One vector is a sparse map (index to value)
# and the other is run-length encoded.
# Answer: Iterate the smaller structure (the sparse non-
# zeros) and binary search the run list for each index
# using prefix run lengths: O(s log r) instead of
# expanding anything.
def sparse_dot_rle(sparse, runs):
    # sparse: {index: value}; runs: RLE of the other
    # vector
    ends = list(accumulate(count for _, count in runs))
    total = 0
    # iterate the small side; binary search the runs
    for index, value in sparse.items():
        k = bisect_right(ends, index)
        if k < len(runs):
            total += value * runs[k][0]
    return total


# FOLLOW-UP 2: How should the data be updated if you set
# element i?
# Answer: Locate the run containing i (binary search over
# prefix run lengths), then split it into at most three
# runs and merge equal neighbours.
def set_at(runs, index, value):
    # update one position: split a run, then merge
    # neighbours
    flat, pos = [], 0
    for v, count in runs:
        if pos <= index < pos + count:
            flat += [(v, index - pos), (value, 1),
                     (v, pos + count - index - 1)]
        else:
            flat.append((v, count))
        pos += count
    merged = []
    for v, count in flat:
        if count == 0:
            continue
        if merged and merged[-1][0] == v:
            merged[-1] = (v, merged[-1][1] + count)
        else:
```

```
            merged.append((v, count))
    return merged


# HELPERS for the checks below
def rle_encode(values):
    runs = []
    for v in values:
        if runs and runs[-1][0] == v:
            runs[-1][1] += 1
        else:
            runs.append([v, 1])
    return [tuple(r) for r in runs]


def rle_decode(runs):
    return [v for v, count in runs for _ in range(count)]
```

```
if __name__ == "__main__":
    A = [3, 3, 3, 0, 0, 0, 4, 4]
    B = [1, 1, 1, 1, 2, 2, 2, 2]
    ra, rb = rle_encode(A), rle_encode(B)
    assert ra == [(3, 3), (0, 3), (4, 2)]
    assert rle_decode(ra) == A
    # 3*1*3 + 0 + 4*2*2 = 25
    assert rle_dot(ra, rb) == 25
    assert rle_dot([], rb) == 0
    dense = sum(x * y for x, y in zip(A, B))
    assert dense == 25
    # sparse vector with non-zeros at 0 and 7
    sparse = {0: 2, 7: 5}
    assert sparse_dot_rle(sparse, rb) == 2 * 1 + 5 * 2
    # setting a value splits and merges runs
    assert set_at(ra, 4, 3) == [(3, 3), (0, 1), (3, 1),
                                (0, 1), (4, 2)]
    assert set_at([(1, 3)], 1, 1) == [(1, 3)]
```

NOTES FROM CANDIDATES

Two-pointer walk over the two RLE encodings: at each step compute the overlap length min(remainA, remainB), multiply values, advance the pointer whose run is exhausted.
Mention compression-friendly variations: sparse format wins when most entries are 0 (just enumerate non-zero indices and skip zeros entirely in the dot product).
Interviewer cared more about clean data-structure choice and the walk over both encodings than about absolute optimality.

PREPARATION

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

MEDIUM | PracHub | Technical Screen | 19 of 30

Maximize 1D deliveries within distance and minimize total distance

POSTED 2025-12-27 | 297 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36.9 | 30 DAYS 0 | PRIORITY 101/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #455 Assign Cookies, LeetCode #475 Heaters

LeetCode match: No direct match; related LC 455 Assign Cookies / LC 475 Heaters (inferred from overview)

Posted on PracHub: 2025-12-27 (updated 2026-04-23)

Original: https://prachub.com/coding-questions/maximize-1d-deliveries-within-distance-and-minimize-total-distance

QUESTION

Overview: This question evaluates combinatorial optimization and constrained matching skills, focusing on reasoning about distance-based pairings, lexicographic objective prioritization, and edge-case handling such as duplicates and empty inputs.

APPROACH

Sort both lists; two-pointer greedy matching each order to the nearest available driver within the limit, maximizing count first, then minimizing total distance as the tie-break.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what an order and a driver are (points on a line), the distance limit, whether each driver takes one order, and the objective order: first the maximum number of deliveries, then the smallest total distance among those.

BRUTE FORCE TO SAY OUT LOUD

Try every matching of orders to drivers. Exponential.

OPTIMAL IDEA

Sort both lists. On a line there is an optimal matching that does not cross, so a DP over (i, j), the first i orders and first j drivers, works: dp[i][j] is the best pair (maximum matches, minimum distance), taking the maximum of skipping an order, skipping a driver, or matching order i with driver j if their distance is within the limit. O(n * m), reducible to O(n * window) because only drivers within the limit matter. A two-pointer greedy that matches the nearest available driver gives the maximum count in O(n log n) but not necessarily the minimum total distance, so say that.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

No feasible pairs, duplicates, empty inputs, an order that could match two drivers equally, more drivers than orders.

LIKELY FOLLOW-UPS

Minimizing the maximum distance instead, or drivers with capacities.

PITFALLS

Assuming greedy nearest matching minimizes the total. Comparing the two objectives as a sum instead of lexicographically.

HOW TO THINK ABOUT IT

On a line there is always an optimal matching that does not cross itself, so sort both lists and match in order: a DP over (orders considered, drivers considered) with the pair (matches, total distance) compared lexicographically solves both objectives together. A greedy that matches each order to the nearest free driver gets the count right but not always the smallest total distance.

Basis: Written; matching on a line. Assumed rules (confirm them): orders and drivers are positions on a line; an order can be served by a driver within distance D; each driver takes at most one order; first maximise the number of deliveries, then minimise the total distance.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = (0, 0)  # (matches, -distance)
def go(i, used, matches, dist):
    global best
    if i == len(orders):
        best = max(best, (matches, -dist))
        return
    go(i + 1, used, matches, dist)  # skip
    for d, x in enumerate(drivers):
        gap = abs(orders[i] - x)
        if d not in used and gap <= D:
            go(i + 1, used | {d},
               matches + 1, dist + gap)
```

TIME	SPACE
O(drivers^orders)	O(n)

WHY IT IS SLOW

Each order branches over all drivers: exponential. After sorting, a DP over (orders considered, drivers considered) that compares (matches, minus distance) is O(n * m).

OPTIMAL SOLUTION · DP over sorted orders and drivers, maximising (matches, -distance)

TIME	SPACE
O(n * m)	O(n * m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the number of deliveries matters.

Two pointers on the sorted lists: match when within D, otherwise advance whichever is further left. O(n log n). It maximises the count (checked against exhaustive search) but not the total distance.

FOLLOW-UP 2 A driver can take several orders (capacity c).

Replicate each driver c times (as separate sorted points) and use the same DP, or treat it as min-cost flow with capacities.

FOLLOW-UP 3 Minimise the largest single distance instead of the total.

Binary search the largest allowed distance D and test whether all orders can be served with the two-pointer matching; the check is monotone in D. **Cost:** O(n log n + (n + m) log(range)): sorting plus a binary search whose check is a two-pointer pass.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximize 1D deliveries within distance
# and minimize total distance
#
# This question evaluates combinatorial optimization and
# constrained matching skills, focusing on reasoning about
# distance-based pairings, lexicographic objective
# prioritization, and edge-case handling such as
# duplicates and empty inputs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the number of deliveries matters.
# 2. A driver can take several orders (capacity c).
# 3. Minimise the largest single distance instead of the
# total.
#
# INTERVIEW NOTES
# On a line there is always an optimal matching that does
# not cross itself, so sort both lists and match in order:
# a DP over (orders considered, drivers considered) with
# the pair (matches, total distance) compared
# lexicographically solves both objectives together. A
# greedy that matches each order to the nearest free
# driver gets the count right but not always the smallest
# total distance.
#
# BRUTE FORCE (say it first): Try every set of matches
# recursively
# For each order either skip it or match it to any unused
# driver within D; keep the best (most matches, then
# smallest total distance). Exact and the oracle;
# exponential. Time O(drivers^orders). Space O(n).


# BRUTE FORCE (say it first): Try every set of matches
# recursively | time O(drivers^orders)
def deliveries_brute(orders, drivers, D):
    best = [(0, 0)]

    def go(i, used, count, dist):
        if i == len(orders):
            if (count, -dist) > (
                best[0][0],
                -best[0][1],
            ):
                best[0] = (count, dist)
            return
        go(i + 1, used, count, dist)
        for j, d in enumerate(drivers):
            if (
                j not in used
                and abs(orders[i] - d) <= D
            ):
                go(
                    i + 1,
                    used | {j},
                    count + 1,
                    dist + abs(orders[i] - d),
                )

    go(0, frozenset(), 0, 0)
    return best[0]


# BASE QUESTION (optimal): DP over sorted orders and
# drivers, maximising (matches, -distance)
# Idea: Sort both lists. dp[i][j] is the best (matches,
# -total distance) using the first i orders and the first
# j drivers: skip order i, skip driver j, or match them if
# their distance is at most D.
def deliveries(orders, drivers, D):
    # first maximise the number of deliveries, then
    # minimise
    # the total distance: a DP over the sorted positions
    o, d = sorted(orders), sorted(drivers)
    # dp[i][j] = (deliveries, -distance) using the first i
    # orders and j drivers; tuples compare
    # lexicographically
    dp = [[(0, 0)] * (len(d) + 1)
          for _ in range(len(o) + 1)]
    for i in range(1, len(o) + 1):
        for j in range(1, len(d) + 1):
            # skip this order or skip this driver ...
            best = max(dp[i - 1][j], dp[i][j - 1])
            gap = abs(o[i - 1] - d[j - 1])
            if gap <= D:
                # ... or pair them
                count, neg = dp[i - 1][j - 1]
                best = max(best, (count + 1, neg - gap))
            dp[i][j] = best
    count, neg = dp[-1][-1]
    return count, -neg


# FOLLOW-UP 1: Only the number of deliveries matters.
# Answer: Two pointers on the sorted lists: match when
# within D, otherwise advance whichever is further left.
# O(n log n). It maximises the count (checked against
# exhaustive search) but not the total distance.
def max_count(orders, drivers, D):
    # only the number matters: two pointers on sorted
    # lists
    o, d = sorted(orders), sorted(drivers)
    i = j = count = 0
    while i < len(o) and j < len(d):
        if abs(o[i] - d[j]) <= D:
            count, i, j = count + 1, i + 1, j + 1
        elif o[i] < d[j]:
            # this order is too far left for every driver
            i += 1
        else:
```

```
            j += 1
    return count


# FOLLOW-UP 2: A driver can take several orders (capacity
# c).
# Answer: Replicate each driver c times (as separate
# sorted points) and use the same DP, or treat it as min-
# cost flow with capacities.
def with_capacity(orders, drivers, D, cap):
    # each driver can take cap orders: repeat the driver
    return deliveries(orders, [x for x in drivers
                               for _ in range(cap)], D)


# FOLLOW-UP 3: Minimise the largest single distance
# instead of the total.
# Answer: Binary search the largest allowed distance D and
# test whether all orders can be served with the two-
# pointer matching; the check is monotone in D.
def smallest_limit(orders, drivers):
    # minimise the LARGEST single distance while serving
    # as
    # many orders as possible: binary search on the limit
    target = min(len(orders), len(drivers))
```

```
    lo, hi = 0, 10 ** 9
    while lo < hi:
        mid = (lo + hi) // 2
        if max_count(orders, drivers, mid) >= target:
            hi = mid
        else:
            lo = mid + 1
    return lo


if __name__ == "__main__":
    assert deliveries([1, 5], [2, 6], 1) == (2, 2)
    # pairing 1-2 and 3-4 beats 1-4 and 3-2? both count 2
    assert deliveries([1, 3], [2, 4], 1) == (2, 2)
    assert deliveries([], [1], 5) == (0, 0)
    # a distance limit of 0 needs identical positions
    assert deliveries([1, 2], [2, 3], 0) == (1, 0)
    assert max_count([1, 3], [2, 4], 1) == 2
    assert max_count([1, 10], [5], 2) == 0
    assert smallest_limit([1, 5], [2, 9]) == 4
    assert with_capacity([1, 2, 3], [2], 1, 3) == (3, 2)
```

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[int], shops: list[int], D: int)
-> tuple[int, int]:
```

Sample call

```
solution([1, 7, 10], [2, 6, 12], 2)
```

Test cases

Input	Expected
([1, 7, 10], [2, 6, 12], 2)	(3, 4)
([0, 4], [2, 5], 3)	(2, 3)

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

Site tags: hashmap, two-pointer, streaming, medium

MEDIUM | 1Point3Acres | onsite-coding | 20 of 30

Two-Day Log Intersection under Memory Limit

ASKED 2025-11-27 | 327 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 24/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 349 Intersection of Two Arrays (related)

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

Input: two log files; each line is (timestamp, obj_id, client_id). Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

Follow-up 1: complexity of set-intersection on the file contents.
Follow-up 2: memory cannot hold either file fully — design a streaming solution.

EXAMPLES

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients
    {c1, c2}.
Output: [X]
```

APPROACH

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

HOW TO THINK ABOUT IT

In memory it is a hash map from object to the set of clients, filtered to objects that appear in both files and have at least two clients. When the files do not fit in memory, sort both by object id (external sort) and walk them together: all rows of one object are adjacent, so you only ever hold the clients of one object at a time.

Basis: LeetCode 349, Intersection of Two Arrays, with a distinct-client condition and a limit on memory (external sort and a two-pointer merge).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(first, second):
    result = []
    for obj in {r.obj for r in first}:
        if obj not in {r.obj for r in second}:
            continue
        rows = [r for r in first + second
                if r.obj == obj]
        if len({r.client for r in rows}) >= 2:
            result.append(obj)
    return result
```

TIME	SPACE
O(n * m)	O(n + m)

WHY IT IS SLOW

O(n * m) time and both files must fit in memory. Sorting both files by object and merging them with two pointers is O(n log n) and holds one object's clients at a time.

OPTIMAL SOLUTION · Hash map of client sets (everything fits in memory)

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why prefer sort and two pointers to a streaming hash map?

A hash map needs memory proportional to the number of distinct objects (and clients), which is exactly what the limit forbids. An external sort needs only a chunk of memory and sequential disk passes, and the merge needs memory for one object's clients. If one object has an enormous client set, count distinct clients approximately (HyperLogLog) or stop as soon as two are seen.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Two-Day Log Intersection under Memory
# Limit
#
# PhD intern coding round 1: two log files with
# (timestamp, obj_id, client_id) entries. Find obj_ids
# that (a) appear in both files and (b) have at least two
# distinct client_ids. Follow-up imposes a memory limit,
# expecting external sort + two-pointer. Input: two log
# files; each line is (timestamp, obj_id, client_id). Find
# every obj_id such that: (a) appears in both files, and
# (b) is associated with at least 2 distinct client_ids
# (aggregated across both files). Follow-up 1: complexity
# of set-intersection on the file contents. Follow-up 2:
# memory cannot hold either file fully — design a
# streaming solution. Examples: File A: (t1, X, c1), (t2,
# X, c2), (t3, Y, c1) File B: (t4, X, c1), (t5, Z, c2) ->
# Only X is in both files; X has 2 distinct clients {c1,
# c2}. Output: [X]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why prefer sort and two pointers to a streaming hash
# map?
#
# INTERVIEW NOTES
# In memory it is a hash map from object to the set of
# clients, filtered to objects that appear in both files
# and have at least two clients. When the files do not fit
# in memory, sort both by object id (external sort) and
# walk them together: all rows of one object are adjacent,
# so you only ever hold the clients of one object at a
# time.
#
# BRUTE FORCE (say it first): For every object scan the
# other file
# For each object in the first file scan the second file
# and the first file again to count clients. Correct,
# quadratic, and it needs both files in memory. Time O(n *
# m). Space O(n + m).


from collections import defaultdict


# BRUTE FORCE (say it first): For every object scan the
# other file | time O(n * m)
```

```
def log_objects_brute(a, b):
    out = set()
    for obj in {o for _, o, _ in a}:
        if (
            any(o == obj for _, o, _ in b)
            and len(
                {c for _, o, c in a + b if o == obj}
            )
            >= 2
        ):
            out.add(obj)
    return sorted(out)


# HELPERS for follow-up 1
def groups(rows):
    # rows sorted by object: yield (object, its clients)
    current, clients = None, set()
    for _, obj, client in rows:
        if obj != current:
            if current is not None:
                yield current, clients
            current, clients = obj, set()
        clients.add(client)
    if current is not None:
        yield current, clients


# BASE QUESTION (optimal): Hash map of client sets
# (everything fits in memory)
# Idea: Build object -> set of clients for each file, keep
# the objects that appear in both, and count the union of
# their client sets.
def log_objects(day1, day2):
    # everything fits in memory: object -> set of clients
    c1, c2 = defaultdict(set), defaultdict(set)
    for _, obj, client in day1:
        c1[obj].add(client)
    for _, obj, client in day2:
        c2[obj].add(client)
    # in both files, with 2+ clients over both days
    both = c1.keys() & c2.keys()
    return sorted(o for o in both
                  if len(c1[o] | c2[o]) >= 2)
```

```
# FOLLOW-UP 1: Why prefer sort and two pointers to a
# streaming hash map?
# Answer: A hash map needs memory proportional to the
# number of distinct objects (and clients), which is
# exactly what the limit forbids.
def log_objects_sorted(rows1, rows2):
    # memory limit: both files arrive sorted by object (an
    # external sort), so only one object's clients are
    # held
    out = []
    g1, g2 = groups(rows1), groups(rows2)
    a, b = next(g1, None), next(g2, None)
    while a and b:
        if a[0] == b[0]:
            # same object in both files
            if len(a[1] | b[1]) >= 2:
                out.append(a[0])
            a, b = next(g1, None), next(g2, None)
        elif a[0] < b[0]:
            a = next(g1, None)
        else:
            b = next(g2, None)
    return out


if __name__ == "__main__":
    day1 = [(1, "X", "c1"), (2, "X", "c2"),
            (3, "Y", "c1")]
    day2 = [(4, "X", "c1"), (5, "Z", "c2")]
    assert log_objects(day1, day2) == ["X"]
    def by_object(rows):
        return sorted(rows, key=lambda r: r[1])

    assert log_objects_sorted(by_object(day1),
                              by_object(day2)) == ["X"]
    # one client in total is not enough
    one = [(1, "A", "c1")]
    assert log_objects(one, one) == []
    assert log_objects_sorted(one, one) == []
    # clients may be split across the two days
    d1, d2 = [(1, "A", "c1")], [(2, "A", "c2")]
    assert log_objects(d1, d2) == ["A"]
    assert log_objects_sorted(d1, d2) == ["A"]
    assert log_objects([], day2) == []
```

NOTES FROM CANDIDATES

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.
Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).
The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n log n) on disk vs O(n) RAM-bound for the hashmap).
For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

PREPARATION

Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

MEDIUM | HackTheRounds | Onsite | 21 of 30

★ TOP 75: rank 66 in the Medium and Hard priority list

★ Shortest Subarray with K Distinct Integers

STUDY: TYPE | **ALL TIME 42.5** | **30 DAYS 33** | **PRIORITY 38/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #992 Subarrays with K Different Integers, LeetCode #76 Minimum Window Substring

LeetCode match: LC 992 / LC 76 flavor

QUESTION

Given an array nums and an integer k, find the length of the shortest contiguous subarray that contains at least k distinct integers. If no such subarray exists, return -1.

EXAMPLES

Example 1	
Input	`nums = [1, 2, 1, 2, 3], k = 3`
Output	`3`
Why	`The shortest contiguous subarray with 3 distinct integers is [1, 2, 3].`

Example 2	
Input	`nums = [1, 1, 1, 1], k = 2`
Output	`-1`
Why	`Only 1 distinct integer exists.`

Example 3	
Input	`nums = [1, 2, 3, 4, 5], k = 3`
Output	`3`
Why	`Any 3 consecutive elements give 3 distinct integers.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^9
- 1 <= k <= nums.length

APPROACH

Sliding window with a frequency hash map, O(n); shrink while distinct >= k

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the output if none exists (-1), whether k can exceed the number of distinct values, and whether the subarray must be contiguous (yes).

BRUTE FORCE TO SAY OUT LOUD

Check every subarray and count distinct values. O(n^3), or O(n^2) with a running set.

OPTIMAL IDEA

Sliding window with a count dict. Move right to add elements; whenever the window has at least k distinct values, record its length and then shrink from the left while it still has at least k distinct, updating the answer each time. Each index enters and leaves once, so O(n).

DRY-RUN THESE CASES

k = 1 (answer 1), k larger than distinct values (-1), all equal values, the shortest window in the middle, k = 0.

LIKELY FOLLOW-UPS

Longest subarray with at most k distinct, count of subarrays with exactly k distinct (LeetCode 992 using at-most minus at-most).

PITFALLS

Shrinking the window before recording the answer. Not deleting keys when counts reach zero.

HOW TO THINK ABOUT IT

Once a window has at least k distinct values, shrinking it from the left can only keep or lose distinct values, so the shortest valid window ending at each right end is found by shrinking as far as possible. A frequency map tells you when a value leaves the window, so each index enters and leaves once.

Basis: Sliding window (compare LeetCode 992 and 340). Shortest contiguous subarray with at least k distinct values; -1 if none.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    best = -1
    for i in range(len(nums)):
        seen = set()
        for j in range(i, len(nums)):
            seen.add(nums[j])
            if len(seen) >= k:
                size = j - i + 1
                if best == -1 or size < best:
                    best = size
                break
    return best
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

BRUTE FORCE

O(n^2) in the worst case. A sliding window with a count map expands on the right and shrinks on the left, so each element is handled twice: O(n).

OPTIMAL SOLUTION · Sliding window with a frequency map

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Count the subarrays with exactly k distinct values (LeetCode 992).

exactly(k) = atMost(k) - atMost(k - 1), where atMost is a sliding window that shrinks while the distinct count exceeds the limit. Both are O(n).

FOLLOW-UP 2 Longest subarray with at most k distinct values.

The same window, but keep the longest window whose distinct count never exceeds k (shrink while it does). O(n).

FOLLOW-UP 3 The values are strings or large objects.

The frequency map works on any hashable value; for huge objects hash them first or map them to integer ids so the window comparisons stay cheap. **Cost:** O(n) plus hashing each element; mapping values to integer ids costs O(n) once.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shortest Subarray with K Distinct
# Integers
#
# Given an array nums and an integer k, find the length of
# the shortest contiguous subarray that contains at least
# k distinct integers. If no such subarray exists, return
# -1.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Count the subarrays with exactly k distinct values
# (LeetCode 992).
# 2. Longest subarray with at most k distinct values.
# 3. The values are strings or large objects.
#
# INTERVIEW NOTES
# Once a window has at least k distinct values, shrinking
# it from the left can only keep or lose distinct values,
# so the shortest valid window ending at each right end is
# found by shrinking as far as possible. A frequency map
# tells you when a value leaves the window, so each index
# enters and leaves once.
#
```

```
# BRUTE FORCE (say it first): Check every subarray
# For every start, extend the end while counting distinct
# values with a set, and record the shortest that reaches
# k. Correct, and quadratic. Time O(n^2). Space O(n).


from collections import Counter


def brute_shortest(nums, k):
    best = None
    for i in range(len(nums)):
        for j in range(i, len(nums)):
            if len(set(nums[i:j + 1])) >= k:
                size = j - i + 1
                if best is None or size < best:
                    best = size
    return -1 if best is None else best


# BRUTE FORCE (say it first): Check every subarray | time
# O(n^2)
def shortest_k_distinct_brute(nums, k):
```

```
    best = None
    for i in range(len(nums)):
        seen = set()
        for j in range(i, len(nums)):
            seen.add(nums[j])
            if len(seen) >= k:
                best = (
                    j - i + 1
                    if best is None
                    else min(best, j - i + 1)
                )
                break
    return -1 if best is None else best


# HELPERS for follow-up 1
def at_most(nums, k):
    # the number of subarrays with at most k distinct
    # values:
    # for every right end, all windows starting in
    # [left, right] qualify
    seen, left, total = Counter(), 0, 0
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) > k:
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
        total += right - left + 1
    return total


# BASE QUESTION (optimal): Sliding window with a frequency
# map
# Idea: Expand the right end, counting each value. While
# the window has at least k distinct values, record its
# length and shrink from the left (deleting a value from
# the map when its count reaches 0).
def shortest_k_distinct(nums, k):
    # a window grows on the right; whenever it holds at
    # least k distinct values, record it and shrink from
    # the
    # left as far as it stays valid
    seen, left, best = Counter(), 0, None
```

```
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) >= k:
```

```
            size = right - left + 1
            if best is None or size < best:
                best = size
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
    return -1 if best is None else best


# FOLLOW-UP 1: Count the subarrays with exactly k distinct
# values (LeetCode 992).
# Answer: exactly(k) = atMost(k) - atMost(k - 1), where
# atMost is a sliding window that shrinks while the
# distinct count exceeds the limit. Both are O(n).
def exactly(nums, k):
    # follow-up (LeetCode 992): exactly k distinct values
    return at_most(nums, k) - at_most(nums, k - 1)


# FOLLOW-UP 2: Longest subarray with at most k distinct
# values.
# Answer: The same window, but keep the longest window
# whose distinct count never exceeds k (shrink while it
# does). O(n).
def longest_at_most(nums, k):
    # follow-up: the longest window with at most k
    # distinct
    seen, left, best = Counter(), 0, 0
    for right, x in enumerate(nums):
        seen[x] += 1
        while len(seen) > k:
            seen[nums[left]] -= 1
            if seen[nums[left]] == 0:
                del seen[nums[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


# FOLLOW-UP 3: The values are strings or large objects.
# Answer: The frequency map works on any hashable value;
```

```
# for huge objects hash them first or map them to integer
# ids so the window comparisons stay cheap. Cost: O(n)
# plus hashing each element; mapping values to integer ids
# costs O(n) once.
def to_ids(items):
    # follow-up: strings or big objects. Map each distinct
    # value to a small integer once, then reuse the same
    # code.
    ids = {}
    return [ids.setdefault(x, len(ids)) for x in items]


if __name__ == "__main__":
    assert shortest_k_distinct([1, 2, 1, 3, 4], 3) == 3
    assert shortest_k_distinct([1, 1, 1], 2) == -1
    assert shortest_k_distinct([5], 1) == 1
    assert shortest_k_distinct([], 1) == -1
    assert exactly([1, 2, 1, 2, 3], 2) == 7
    assert exactly([1, 2, 1, 3, 4], 3) == 3
    assert longest_at_most([1, 2, 1, 2, 3], 2) == 4
    words = ["a", "b", "a", "c"]
    assert shortest_k_distinct(to_ids(words), 3) == 3
    assert to_ids(["x", "y", "x"]) == [0, 1, 0]
    import random
    for _ in range(400):
        nums = [random.randint(1, 4)
                for _ in range(random.randint(0, 10))]
        k = random.randint(1, 5)
        want = brute_shortest(nums, k)
        assert shortest_k_distinct(nums, k) == want
```

```
        count = sum(len(set(nums[i:j + 1])) == k
                    for i in range(len(nums))
                    for j in range(i, len(nums)))
        assert exactly(nums, k) == count
```

ALSO REPORTED · 2 more

Find shortest subarray with at least k distinct | PracHub | Technical Screen

https://prachub.com/coding-questions/find-shortest-subarray-with-at-least-k-distinct

Find shortest subarray with >=k distinct integers | PracHub | Technical Screen

https://prachub.com/coding-questions/find-shortest-subarray-with-k-distinct-integers

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

Site tags: string, two-pointer, math-reasoning, hard

HARD | 1Point3Acres | onsite-coding | 22 of 30

Necklace Cut into Two Halves with Equal D/R Counts

ASKED 2026-05-03 | 170 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 188/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-05-03 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/necklace-cut-equal-counts

QUESTION

EXAMPLES

```
s = "DRDR"    -> cut between index 1 and 2: "DR" +
    "DR".
s = "DDRR"    -> 1-cut not balanced ("DD" vs "RR");
    rotate, two cuts at offsets 1 and 3:
              pieces "DR" and "DR".
```

APPROACH

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the string is circular, whether the two pieces must be contiguous arcs, whether equal counts of each letter or equal strings are needed, and whether one cut or two are allowed.

BRUTE FORCE TO SAY OUT LOUD

Try every pair of cut positions and count letters. O(n^3) or O(n^2) with prefix counts.

OPTIMAL IDEA

Each arc must hold half of the D's and half of the R's, so the total counts of both must be even and each arc has length n / 2. Slide a window of length n / 2 around s + s tracking its D count; the window starting at i and the one starting at i + n / 2 are complementary, and counts change by at most one per step, so some start hits exactly half. O(n).

DRY-RUN THESE CASES

DRDR, DDRR (two cuts needed), an odd D count (impossible), a string of one letter.

LIKELY FOLLOW-UPS

Equal strings instead of equal counts, or three pieces.

INTERVIEW-DAY PLAN

PITFALLS

Assuming a linear cut is always enough. Ignoring parity.

HOW TO THINK ABOUT IT

Two arcs of a circle with the same letter counts must each have exactly half the D's and half the R's, so the total counts must be even and each arc has length n / 2. Slide a window of length n / 2 around the circle: the window and its complement have D counts adding to the total, and the count changes by at most one per step, so it must hit half somewhere.

Basis: Circular sliding window / prefix balance (cf. LeetCode 525, Contiguous Array). Assumption: cut the circular necklace at two points into two arcs, each with the same number of D and R.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for i, j in combinations(range(n), 2):
        arc1 = s[i:j]
        arc2 = s[j:] + s[:i]
        d1, d2 = arc1.count("D"), arc2.count("D")
        r1, r2 = arc1.count("R"), arc2.count("R")
        if d1 == d2 and r1 == r2:
            return True
    return False
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^2) pairs and each recount costs O(n), so O(n^3). Because the arcs must both have length n / 2, one circular window of that length slid around the string finds it in O(n).

OPTIMAL SOLUTION · Fixed-length circular window with the discrete intermediate value argument

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When is a single cut enough?

A single linear cut works when some prefix has exactly half the D's and half the R's, i.e. a prefix of length n/2 whose D count is d_total / 2. Otherwise rotate: the circular window above finds the start, which is equivalent to two cuts on the circle.

FOLLOW-UP 2 What if the two pieces must be equal strings, not just equal counts?

That is a different question: the necklace must be a square rotation, i.e. some rotation of s is t + t. Check whether s appears inside (s + s)[1:-1] at index n/2, or compare s[i:i + n/2] with the complementary arc using a rolling hash or KMP over s + s. It is a string matching problem, not a counting one.

FOLLOW-UP 3 The two pieces must be equal strings, not just equal counts.

Then the necklace must be a square in some rotation: some rotation of s equals t + t. Test with s in (s + s)[1:-1] restricted to an even-length split, or compare s[i:i + n/2] with s[i + n/2:i + n] using a rolling hash or KMP.

FOLLOW-UP 4 The two pieces must be equal strings, not just have equal counts.

A rotation of the necklace must be of the form t + t. Try every rotation and compare its two halves (O(n^2), or O(n) by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Necklace Cut into Two Halves with Equal
# D/R Counts
#
# Examples: s = "DRDR" -> cut between index 1 and 2: "DR"
+
# "DR". s = "DDRR" -> 1-cut not balanced ("DD" vs "RR");
# rotate, two cuts at offsets 1 and 3: pieces "DR" and
# "DR".
#
# FOLLOW-UPS (answered below, in this order):
# 1. When is a single cut enough?
# 2. What if the two pieces must be equal strings, not
# just equal counts?
# 3. The two pieces must be equal strings, not just equal
# counts.
# 4. The two pieces must be equal strings, not just have
# equal counts.
#
# INTERVIEW NOTES
# Two arcs of a circle with the same letter counts must
# each have exactly half the D's and half the R's, so the
# total counts must be even and each arc has length n / 2.
# Slide a window of length n / 2 around the circle: the
# window and its complement have D counts adding to the
# total, and the count changes by at most one per step, so
# it must hit half somewhere.
#
# BRUTE FORCE (say it first): Try every pair of cut points
# Choose two cut positions, form the two arcs, and compare
# their letter counts. Correct and the oracle; it checks
# every pair of cuts and recounts letters each time. Time
# O(n^3). Space O(n).
```

```
# BRUTE FORCE (say it first): Try every pair of cut points
# | time O(n^3)
def necklace_brute(s):
    n = len(s)
    for i in range(n):
        for j in range(i + 1, n):
            a, b = s[i:j], s[j:] + s[:i]
            if (
                a
                and b
                and a.count("D") == b.count("D")
                and a.count("R") == b.count("R")
            ):
                return True
    return False


# BASE QUESTION (optimal): Fixed-length circular window
# with the discrete intermediate value argument
# Idea: If each arc has half of the D's and half of the
# R's, each arc has length n/2, and the total D and R
# counts must both be even. Slide a window of length n/2
# around the circle (use s + s), tracking its D count.
def necklace_two_cuts(s):
    n = len(s)
    d_total, r_total = s.count("D"), s.count("R")
    # each piece needs half of both letters
    if n % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = n // 2, d_total // 2
    doubled = s + s
    # slide a window of length n/2 around the circle
    window = doubled[:half].count("D")
    for i in range(n):
        if window == target:
            # cuts at i and i + n/2 give two good halves
            first = doubled[i:i + half]
            second = doubled[i + half:i + n]
            return i, (i + half) % n, first, second
        # move the window one step: add the new, drop the
        # old
        window += (doubled[i + half] == "D")
        window -= (doubled[i] == "D")
    return None


# SINGLE CUT case
def single_cut(s):
    # one cut on the line: a prefix with half of
    # everything
    half = len(s) // 2
    if len(s) % 2:
        return None
    d_prefix = s[:half].count("D")
    return half if d_prefix * 2 == s.count("D") else None
```

```
# FOLLOW-UP 2: What if the two pieces must be equal
# strings, not just equal counts?
# Answer: That is a different question: the necklace must
# be a square rotation, i.e. some rotation of s is t + t.
# FOLLOW-UP 3: The two pieces must be equal strings, not
# just equal counts.
# Answer: Then the necklace must be a square in some
# rotation: some rotation of s equals t + t. Test with s
# in (s + s)[1:-1] restricted to an even-length split, or
# compare s[i:i + n/2] with s[i + n/2:i + n] using a
# rolling hash or KMP.
# FOLLOW-UP 4: The two pieces must be equal strings, not
# just have equal counts.
# Answer: A rotation of the necklace must be of the form t
# + t. Try every rotation and compare its two halves
```

```python
# (O(n^2), or O(n) by searching s in (s + s)[1:-1] and
# checking the match at an offset of n / 2).
def equal_string_halves(s):
    # follow-up: the two pieces must be the same string
    n = len(s)
    if n % 2:
        return None
    half, doubled = n // 2, s + s
    for i in range(n):
        if doubled[i:i + half] == doubled[i + half:i + n]:
            return i, (i + half) % n
    return None


if __name__ == "__main__":
    # DRDR: one cut between the halves works
    assert single_cut("DRDR") == 2
    cuts = necklace_two_cuts("DRDR")
    assert cuts is not None and cuts[2].count("D") == 1
    # DDRR needs two cuts on the circle: DR + DR
    assert single_cut("DDRR") is None
    i, j, a, b = necklace_two_cuts("DDRR")
    assert sorted(a) == sorted(b) == ["D", "R"]
    # odd totals have no answer
    assert necklace_two_cuts("DDR") is None
    assert necklace_two_cuts("DRDRD" + "R") is None
    # equal strings: DRDR is DR + DR, DDRR is not
    assert equal_string_halves("DRDR") is not None
    assert equal_string_halves("DDRR") is None
```

NOTES FROM CANDIDATES

Why 2 cuts always suffice: think of a running balance cnt(D) - cnt(R); over the full string it returns to 0. Any prefix splits the necklace into two arcs whose imbalances sum to 0; if the imbalance at some prefix is already 0, one cut suffices, otherwise a sliding window of length n/2 proves a 2-cut works.
Sliding-window approach: fix one cut at index i, look for j such that the window [i, j) has half the Ds and half the Rs. As i advances, j moves monotonically.
Don't get distracted by the necklace framing; once unwrapped onto a line of length 2n and constrained to half-length windows, the problem is standard.
Verbalize the 2-cut bound out loud before coding — interviewers ask why.

PREPARATION

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families).
Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Two pointers and sliding window

HARD | IN APP LC 1044 | PracHub | Technical Screen | 23 of 30

Find longest duplicated substring

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 122/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Two pointers and sliding window

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1044 Longest Duplicate Substring

LeetCode match: LC 1044 Longest Duplicate Substring

Posted on PracHub: 2025-12-08 (updated 2026-07-09)

Original: https://prachub.com/coding-questions/find-longest-duplicated-substring

QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., `s[i..j)` for `0 <= i < j <= len(s)`. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

Requirements

- Input: a string **s** of length **n** (e.g., `1 <= n <= 2 × 10^5`).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive `O(n^2)` substring comparison approach.

Examples

- `s = "banana"`
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include `"ana"`.
- `s = "abcd"`
- No duplicated substring, so output should be `""` (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

APPROACH

Binary search on the substring length; check each length with a Rabin-Karp rolling hash plus a set (verify on collision). Alternative: suffix array + LCP. O(n log n).

HOW TO THINK ABOUT IT

If a substring of length L repeats, then a substring of every shorter length repeats too, so 'is there a duplicate of length L' is monotone and the answer length can be binary searched. Each check slides a window of length L over the string with a rolling hash and looks for a repeated hash, verifying the match to guard against collisions.

Basis: LeetCode 1044, Longest Duplicate Substring (binary search on the length with a rolling hash).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for L in range(n - 1, 0, -1):
        seen = set()
        for i in range(n - L + 1):
            sub = s[i:i + L]
            if sub in seen:
                return sub
            seen.add(sub)
    return ""
```

TIME	SPACE
O(n^3)	O(n^2)

WHY IT IS SLOW

Each length builds up to n substrings of up to n characters: O(n^3) time and O(n^2) space. If a duplicate of length L exists then so does one of every shorter length, so binary search on L with a rolling hash is O(n log n).

OPTIMAL SOLUTION · Binary search on the length with a rolling hash (Rabin-Karp)

TIME	SPACE
O(n log n) expected	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Avoid hash collisions entirely.

Use a suffix array with the LCP array: the longest duplicate is the largest LCP between neighbours in suffix order. Naive construction (sorting suffixes) is O(n^2 log n); real suffix array algorithms are O(n log n) or O(n).

FOLLOW-UP 2 The substring must appear at least k times, or must not overlap.

At least k times: count equal hashes in the check (need k of them). Non-overlapping: when two windows match, require their start positions to differ by at least L.

FOLLOW-UP 3 The string is huge and does not fit in memory.

Hash windows on disk in passes, or use a suffix automaton built online; for approximate answers use minhash sketches. State that exact answers need memory proportional to the number of distinct windows. **Cost:** O(n log n) time and O(n) memory for the hashes; on disk the hashing passes multiply the I/O by about log n.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find longest duplicated substring
#
# You are given a string s consisting of lowercase English
# letters. A **substring** of s is any contiguous sequence
# of characters, i.e., s[i..j) for 0 <= i < j <= len(s).
# Design an algorithm to find **any one** of the longest
# substrings that appears **at least twice** in s. The two
# occurrences may overlap. If no substring appears at
# least twice, return the empty string. ### Requirements -
# Input: a string s of length n (e.g., 1 <= n <= 2 × 10^5)
.
# - Output: a string representing one of the longest
# duplicated substrings. - Aim for an algorithm
# substantially better than the naive O(n^2) substring
# comparison approach. ### Examples - s = "banana" -
# Possible duplicated substrings: "a", "an", "ana" -
# Longest length is 3 ("ana"), so acceptable outputs
# include "ana". - s = "abcd" - No duplicated substring,
# so output should be "" (empty string).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Avoid hash collisions entirely.
# 2. The substring must appear at least k times, or must
# not overlap.
# 3. The string is huge and does not fit in memory.
#
# INTERVIEW NOTES
# If a substring of length L repeats, then a substring of
# every shorter length repeats too, so 'is there a
# duplicate of length L' is monotone and the answer length
# can be binary searched. Each check slides a window of
# length L over the string with a rolling hash and looks
# for a repeated hash, verifying the match to guard
# against collisions.
#
# BRUTE FORCE (say it first): Check every length from
# longest down with a set of substrings
# For each length from n - 1 down, put all substrings of
# that length in a set until one repeats. Correct and the
# oracle; every check builds up to n substrings of length
# up to n. Time O(n^3). Space O(n^2).


# BRUTE FORCE (say it first): Check every length from
# longest down with a set of substrings | time O(n^3)
def longest_dup_brute(s):
    for L in range(len(s) - 1, 0, -1):
        seen = set()
        for i in range(len(s) - L + 1):
            sub = s[i : i + L]
            if sub in seen:
                return sub
            seen.add(sub)
    return ""


# HELPERS for the base question
MOD = (1 << 61) - 1


BASE = 131


def find_length(s, length):
    # a repeated window of this length? (rolling hash)
    codes = [ord(c) for c in s]
    power = pow(BASE, length, MOD)
    h = 0
    for i in range(length):
        h = (h * BASE + codes[i]) % MOD
    seen = {h: [0]}
    for i in range(1, len(s) - length + 1):
        # slide the window: drop the left, add the right
        h = (h * BASE - codes[i - 1] * power
             + codes[i + length - 1]) % MOD
        # a hash match may be a collision: compare the
        # text
        for j in seen.get(h, ()):
            if s[j:j + length] == s[i:i + length]:
                return s[i:i + length]
        seen.setdefault(h, []).append(i)
    return None


# BASE QUESTION (optimal): Binary search on the length
# with a rolling hash (Rabin-Karp)
# Idea: For a fixed length L compute the polynomial hash
# of every window in O(1) each by sliding it, store hashes
# with their start positions, and when a hash repeats
# compare the actual substrings (to rule out a collision).
def longest_dup(s):
    # binary search the length: if length L repeats, so do
    # all shorter lengths
    lo, hi, best = 1, len(s) - 1, ""
    while lo <= hi:
        mid = (lo + hi) // 2
        found = find_length(s, mid)
        if found:
            best, lo = found, mid + 1
        else:
            hi = mid - 1
    return best


# FOLLOW-UP 1: Avoid hash collisions entirely.
# Answer: Use a suffix array with the LCP array: the
# longest duplicate is the largest LCP between neighbours
# in suffix order. Naive construction (sorting suffixes)
# is O(n^2 log n); real suffix array algorithms are O(n
# log n) or O(n).
def longest_dup_suffix(s):
    # no hashing: sort the suffixes, compare neighbours
    # (the
    # simple O(n^2 log n) version of a suffix array)
    suffixes = sorted(s[i:] for i in range(len(s)))
    best = ""
    for a, b in zip(suffixes, suffixes[1:]):
        k = 0
        while k < min(len(a), len(b)) and a[k] == b[k]:
            k += 1
        if k > len(best):
            best = a[:k]
    return best
```

```
# FOLLOW-UP 2: The substring must appear at least k times,
# or must not overlap.
# Answer: At least k times: count equal hashes in the
# check (need k of them). Non-overlapping: when two
# windows match, require their start positions to differ
# by at least L.
def longest_repeated(s, times=2, non_overlapping=False):
    # at least `times` occurrences; optionally not
    # overlapping
    best = ""
    n = len(s)
```

```
    for length in range(1, n):
        spots = {}
        for i in range(n - length + 1):
            piece = s[i:i + length]
            spots.setdefault(piece, []).append(i)
        ok = None
        for text, starts in spots.items():
            if non_overlapping:
                # greedy: count starts at least `length`
                # apart
                taken, last = 0, -length
                for p in starts:
                    if p - last >= length:
                        taken, last = taken + 1, p
            else:
                taken = len(starts)
            if taken >= times:
                ok = text
                break
        if ok is None:
            break
        best = ok
    return best


if __name__ == "__main__":
    assert longest_dup("banana") in ("ana",)
    assert longest_dup("abcd") == ""
    assert len(longest_dup("aaaaa")) == 4
    assert len(longest_dup_suffix("banana")) == 3
    assert longest_dup_suffix("abcd") == ""
    for s in ("abracadabra", "mississippi", "abab"):
        fast = len(longest_dup(s))
        assert fast == len(longest_dup_suffix(s))
    # three occurrences needed
    assert longest_repeated("banana", 3) == "a"
    # overlapping copies are not allowed: "aa" cannot fit
    # in aaaa
    assert len(longest_repeated("aaaa", 2, True)) == 2
    assert len(longest_repeated("aaaa", 2)) == 3
```

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> str:
```

Sample call

```
solution('banana')
```

Test cases

Input	Expected
`('banana',)`	`'ana'`
`('abcd',)`	`''`

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Arrays and prefix sums

Site tags: array, grid

MEDIUM | 1Point3Acres | phone-screen | 24 of 30

Longest Same-Color Grid Sequence

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 19.6** | **30 DAYS 0** | **PRIORITY 148/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Arrays and prefix sums

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix (related)

Frequency: Single report | Last asked: 2026-09-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

QUESTION

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line.

Input is a two-dimensional grid of colors such as R, G, and B. Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally.

EXAMPLES

```
R R R G
R R B B
G R R R
B G B R

Answer: 4
```

APPROACH

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

HOW TO THINK ABOUT IT

A straight line can run in four directions (right, down, down-right, down-left). For each cell and each direction keep the length of the same-colour run that ends there: it is the predecessor's run plus one if the colours match, otherwise 1. Because every run only looks one cell back in a fixed direction, one pass in reading order is enough.

Basis: LeetCode 562, Longest Line of Consecutive One in Matrix, generalised to any colour.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = 0
    for r, c in all_cells(grid):
        for dr, dc in DIRECTIONS:
            n = 1
            while same(grid, r, c,
                       r + n * dr, c + n * dc):
                n += 1
            best = max(best, n)
    return best
```

TIME	SPACE
O(m * n * max(m, n))	O(1)

WHY IT IS SLOW

Re-walks the same cells in every line, O(m * n * max(m, n)). Keeping the run length ending at each cell in each direction is O(m * n).

OPTIMAL SOLUTION · Four run lengths per cell

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * n). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Same-Color Grid Sequence
#
# Given a grid of colors, return the longest consecutive
# same-color sequence along a horizontal, vertical, or
# diagonal line. Input is a two-dimensional grid of colors
# such as R, G, and B. Return the length of the longest
# consecutive sequence of a single color horizontally,
# vertically, or diagonally. Examples: R R R G R R B B G R
# R R B G B R Answer: 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. The line may also bend (a connected region).
#
# INTERVIEW NOTES
# A straight line can run in four directions (right, down,
# down-right, down-left). For each cell and each direction
# keep the length of the same-colour run that ends there:
# it is the predecessor's run plus one if the colours
# match, otherwise 1. Because every run only looks one
```

```
# cell back in a fixed direction, one pass in reading
# order is enough.
#
# BRUTE FORCE (say it first): Extend a line from every
# cell in every direction
# Start at each cell and walk in each of the four
# directions while the colour matches. Correct; it re-
# walks the same cells many times. Time O(m * n * max(m,
# n)). Space O(1).


# BRUTE FORCE (say it first): Extend a line from every
# cell in every direction | time O(m * n * max(m, n))
def longest_line_brute(g):
    best = 0
    for i in range(len(g)):
        for j in range(len(g[0])):
            for di, dj in (
                (0, 1),
                (1, 0),
                (1, 1),
                (1, -1),
            ):
                k, a, b = 0, i, j
                while (
                    0 <= a < len(g)
                    and 0 <= b < len(g[0])
                    and g[a][b] == g[i][j]
                ):
                    k += 1
                    a += di
                    b += dj
                best = max(best, k)
    return best


# BASE QUESTION (optimal): Four run lengths per cell
# Idea: Keep, for the previous row, the run length ending
# at each column in the vertical and both diagonal
# directions, and a running value for the horizontal run
# in the current row.
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best


# FOLLOW-UP 1: The line may also bend (a connected
# region).
# Answer: Then it is a different problem: the largest
# connected same-colour region, found with BFS/DFS or
# union-find in O(m * n). The run-length DP only handles
# straight lines because each run depends on exactly one
# predecessor.
def largest_region(g):
    # follow-up: the line may bend (connected region)
    seen, best = set(), 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack, size = [(r, c)], 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y),
                             (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0:
                        continue
                    if a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen \
                            and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            best = max(best, size)
    return best


if __name__ == "__main__":
    grid = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_same_color_line(grid) == 4
    assert longest_same_color_line([]) == 0
    # one column and alternating colours
    assert longest_same_color_line(["R", "R", "R"]) == 3
    assert longest_same_color_line(["RGR", "GRG"]) == 2
    # a bent region is longer than any straight line
    bent = ["RRR", "GGR"]
    assert longest_same_color_line(bent) == 3
    assert largest_region(["RR", "GR"]) == 3
```

NOTES FROM CANDIDATES

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

PREPARATION

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Arrays and prefix sums

MEDIUM | PracHub | Technical Screen | 25 of 30

Solve several coding interview problems

POSTED 2026-03-04 | 230 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 38.3 | 30 DAYS 0 | PRIORITY 98/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Arrays and prefix sums

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #160 Intersection of Two Linked Lists

LeetCode match: No direct match (multi-part); related LC 415 Add Strings, LC 160 Intersection of Two Linked Lists (inferred from overview)

Posted on PracHub: 2026-03-04 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/solve-several-coding-interview-problems

QUESTION

Overview: This set of problems evaluates proficiency in string processing and combinatorial parsing, arbitrary-precision arithmetic using string representations, pointer/ancestor-chain intersection detection, and computational geometry for overlap counting, emphasizing algorithmic design, correctness, and performance.

APPROACH

Multiple small tasks: string parsing, big-number string arithmetic, ancestor-chain intersection (hash set or depth alignment), etc. Solve each independently.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy lists four small tasks: string parsing with combinatorial counting, adding two big integers given as strings, finding where two ancestor chains intersect, and counting overlaps in a geometry setting. Confirm each task's exact input and output, because the overview is the only statement I have.

BRUTE FORCE TO SAY OUT LOUD

Do each directly: parse and count with recursion, add digit by digit with a carry, compare ancestor lists, and test every pair of shapes.

OPTIMAL IDEA

Big integers: walk both digit strings from the right with a carry and build the result (O(max(m, n))), as in LeetCode 415. Ancestor chains: put one chain in a set and walk the other, or align by depth and step both up until they meet, as in LeetCode 160; O(n) time, O(1) space for the aligned version. Overlap counting: a sweep line over start and end events, taking the running maximum or counting pairs depending on the question. String parsing and counting: define the state and use memoized recursion.

DRY-RUN THESE CASES

Big integers with different lengths, a final carry, leading zeros. Chains that never meet, meet at the head, are identical. Overlaps that touch at a boundary.

LIKELY FOLLOW-UPS

Subtraction or multiplication of big integers, lowest common ancestor with parent pointers, or counting overlapping rectangles.

PITFALLS

Starting to code before the four statements are confirmed. Forgetting the final carry.

HOW TO THINK ABOUT IT

A set of unrelated small tasks: split them, restate each contract, and solve the smallest one first. Big numbers: add digit by digit with a carry. Two chains that meet: walk both and switch to the other head at the end so the lengths cancel. Overlap counting: sort start and end events and keep a running count.

Basis: Three of the small tasks the overview lists, with the standard versions (my copy has only the overview): adding two big integers given as strings (LeetCode 415), intersection of two chains such as ancestors or linked lists (LeetCode 160), and the largest number of overlapping intervals.

BRUTE FORCE · Big numbers: convert to integers and add

BRUTE FORCE · PSEUDO-CODE

```
def add_strings(a, b):
    return str(int(a) + int(b))
```

TIME	SPACE
O(n^2)	O(n)
for the conversions	

WHY IT IS SLOW

Relies on arbitrary-precision integers (not in every language) and the string-to-integer conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

BRUTE FORCE · Chains that meet: store one chain in a set

BRUTE FORCE · PSEUDO-CODE

```
def intersection(a, b):
    seen = set()
    node = a
    while node:
        seen.add(node)
        node = node.parent
    node = b
    while node:
        if node in seen:
            return node
        node = node.parent
    return None
```

TIME	SPACE
O(m + n)	O(m)

WHY IT IS SLOW

O(m + n) time but O(m) extra memory. Two pointers that switch to the other chain's head at the end meet at the shared node with O(1) memory.

BRUTE FORCE · Overlap: test every time point

BRUTE FORCE · PSEUDO-CODE

```
def max_overlap(intervals):
    lo = min(s for s, e in intervals)
    hi = max(e for s, e in intervals)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in intervals:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, O(range * n). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Big numbers: digit by digit from the right with a carry

TIME	SPACE
O(max(m, n))	O(max(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Chains that meet: two pointers that switch heads at the end

TIME	SPACE
O(m + n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Overlap: sweep over sorted start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Ancestors in a tree with parent pointers, not a linked list.

The same two-pointer trick works on parent chains (LeetCode 1650); with depths known, lift the deeper node first and then move both up together.

FOLLOW-UP 2 Add two big numbers stored as linked lists.

If the lists store the least significant digit first, walk both together with the carry (LeetCode 2); if most significant first, reverse them or use stacks.

FOLLOW-UP 3 Count overlapping rectangles instead of intervals.

Sweep over x with a data structure over y (a segment tree with range add and max) to find the largest point depth in O(n log n); the pairwise count is O(n^2) unless you use a sweep with an interval tree.

FOLLOW-UP 4 Add two big numbers stored as linked lists (least significant digit first).

Walk both lists together with a carry and build the result list as you go; no reversal needed. O(max(m, n)) time. If the most significant digit comes first, reverse the lists or use stacks.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve several coding interview problems
#
# This set of problems evaluates proficiency in string
# processing and combinatorial parsing, arbitrary-
# precision arithmetic using string representations,
# pointer/ancestor-chain intersection detection, and
```

```
# computational geometry for overlap counting, emphasizing
# algorithmic design, correctness, and performance.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Ancestors in a tree with parent pointers, not a
# linked list.
# 2. Add two big numbers stored as linked lists.
# 3. Count overlapping rectangles instead of intervals.
# 4. Add two big numbers stored as linked lists (least
# significant digit first).
#
# INTERVIEW NOTES
# A set of unrelated small tasks: split them, restate each
# contract, and solve the smallest one first. Big numbers:
# add digit by digit with a carry. Two chains that meet:
# walk both and switch to the other head at the end so the
# lengths cancel. Overlap counting: sort start and end
# events and keep a running count.
#
# BRUTE FORCE (say it first): Big numbers: convert to
# integers and add
# int(a) + int(b), then str. Valid in Python, but it hides
# the algorithm and relies on arbitrary-precision
# integers. Time O(n^2) for the conversions. Space O(n).


# BRUTE FORCE (say it first): Big numbers: convert to
# integers and add | time O(n^2) for the conversions
def add_strings_int(a, b):
    return str(int(a) + int(b))


# HELPERS for follow-up 2
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def add_strings(a, b):
    # big numbers kept as digit strings, added from the
    # right
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        if i >= 0:
            total += int(a[i])
        if j >= 0:
            total += int(b[j])
        out.append(str(total % 10))
        carry = total // 10
        i, j = i - 1, j - 1
    return "".join(out[::-1])


# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def intersection(head_a, head_b):
    # two chains meeting: each pointer walks its chain and
    # then
    # the other one, so both walk the same total length
    a, b = head_a, head_b
    while a is not b:
        a = a.next if a else head_b
        b = b.next if b else head_a
    return a


# BASE QUESTION (optimal): Big numbers: digit by digit
# from the right with a carry
# Idea: Walk both strings from the right, add the two
# digits and the carry, append total % 10, carry total //
# 10; after the loop add a final carry digit if needed and
# reverse.
def overlap_depth(rects):
    # rects (x1, y1, x2, y2), half-open: the largest
    # number of
    # rectangles sharing one point. The best point can be
    # a
    # rectangle's lower-left corner, so test those.
    best = 0
    for x, y, _, _ in rects:
        inside = [r for r in rects if r[0] <= x < r[2]
                  and r[1] <= y < r[3]]
        best = max(best, len(inside))
    return best


# FOLLOW-UP 2: Add two big numbers stored as linked lists.
# Answer: If the lists store the least significant digit
# first, walk both together with the carry (LeetCode 2);
# if most significant first, reverse them or use stacks.
# FOLLOW-UP 4: Add two big numbers stored as linked lists
# (least significant digit first).
# Answer: Walk both lists together with a carry and build
# the result list as you go; no reversal needed. O(max(m,
# n)) time. If the most significant digit comes first,
# reverse the lists or use stacks.
def add_lists(a, b):
    # digits stored least significant first: add with a
    # carry
    dummy = tail = ListNode()
    carry = 0
    while a or b or carry:
        total = carry
        total += a.val if a else 0
        total += b.val if b else 0
        tail.next = ListNode(total % 10)
        tail = tail.next
        carry = total // 10
        a = a.next if a else None
        b = b.next if b else None
    return dummy.next


# FOLLOW-UP 3: Count overlapping rectangles instead of
# intervals.
# Answer: Sweep over x with a data structure over y (a
# segment tree with range add and max) to find the largest
# point depth in O(n log n); the pairwise count is O(n^2)
# unless you use a sweep with an interval tree.
def overlapping_pairs(rects):
    # how many pairs of rectangles overlap?
    count = 0
    for i in range(len(rects)):
        for j in range(i + 1, len(rects)):
            a, b = rects[i], rects[j]
            if a[0] < b[2] and b[0] < a[2] \
                    and a[1] < b[3] and b[1] < a[3]:
                count += 1
    return count


# HELPERS for the checks below
```

```
def to_list(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


if __name__ == "__main__":
    assert add_strings("999", "1") == "1000"
    assert add_strings("0", "0") == "0"
    big = add_strings("123456789123456789", "987654321")
    assert big == str(123456789123456789 + 987654321)
    shared = ListNode(8, ListNode(9))
    a = ListNode(1, ListNode(2, shared))
    b = ListNode(5, shared)
    assert intersection(a, b) is shared
    # no meeting: both end at None
    assert intersection(ListNode(1), ListNode(2)) is None
    # 342 + 465 = 807 with digits reversed
    x = ListNode(2, ListNode(4, ListNode(3)))
    y = ListNode(5, ListNode(6, ListNode(4)))
    assert to_list(add_lists(x, y)) == [7, 0, 8]
    rects = [(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]
    assert overlap_depth(rects) == 3
    assert overlapping_pairs(rects) == 3
    assert overlap_depth([]) == 0
```

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> list[str]:
```

Sample call

```
solution('.-')
```

Test cases

Input	Expected
`('.-',)`	`['A', 'ET']`
`('...',)`	`['EEE', 'EI', 'IE', 'S']`

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Arrays and prefix sums

MEDIUM | PracHub | Technical Screen | 26 of 30

Min deletions to avoid overlap in first k

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 195/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Arrays and prefix sums

Optimal answer from: the interview site's own solution

LeetCode match: No direct match (inferred) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-04-22)

Original: https://prachub.com/coding-questions/min-deletions-to-avoid-overlap-in-first-k

QUESTION

Overview: This question evaluates array and sequence manipulation skills, set-based reasoning about value overlap, and the ability to compute minimal edits while preserving relative order constraints.

APPROACH

Likely: compare the first k elements with the rest via sets and delete the minimum elements to remove value overlap while preserving order; confirm details on the PracHub page.

INTERVIEW-DAY PLAN

CLARIFY FIRST

This prompt is vague, so ask for the exact rule first. A common reading: the first k elements and the remaining elements must share no value, delete the fewest elements. Ask whether deleted elements must be removed from the first part or either part, and whether order matters.

BRUTE FORCE TO SAY OUT LOUD

Try every subset of deletions. Exponential.

OPTIMAL IDEA

Under that reading values are independent. For each value that appears in both parts, you must remove all its copies from one side, so you delete the smaller of its two counts. Sum over shared values. Count with two hash maps. O(n).

DRY-RUN THESE CASES

No shared values (0), every value shared, k = 0 or k = n, repeated values on one side only.

LIKELY FOLLOW-UPS

Delete only from the first part, weights on elements, or k given as a range.

PITFALLS

Starting to code before the rule is confirmed. Deleting elements one at a time greedily.

HOW TO THINK ABOUT IT

Values do not interact with each other, so each value can be decided on its own. A value that appears on both sides must disappear from one side entirely, and the cheapest side is the one with fewer copies. The answer is the sum, over values that appear on both sides, of the smaller of the two counts.

Basis: The statement is vague in my copy. Assumed rule (confirm it): given an array and k, the first k elements and the remaining elements must share no value; delete the fewest elements (from either part) to achieve that.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a, k):
    for size in range(len(a) + 1):
        ids = range(len(a))
        for gone in combinations(ids, size):
            keep = [i for i in ids
                    if i not in gone]
            head = {a[i] for i in keep if i < k}
            tail = {a[i] for i in keep if i >= k}
            if not head & tail:
                return size
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

2^n sets to try: exponential. Each shared value must disappear from one side, so the cost is the sum over shared values of the smaller of its two counts, in O(n).

OPTIMAL SOLUTION · Sum of min(count in the first k, count in the rest) over shared values

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only elements of the first part may be deleted.

Then every shared value must be removed from the first part entirely, so the cost is the number of first-part elements whose value occurs in the rest (a set lookup per element).

FOLLOW-UP 2 Elements have weights (deleting some is more costly).

Replace counts by the sum of weights per side for each value; the choice per value is still independent, so take the cheaper side. **Cost:** unchanged O(n); the per-value choice sums weights instead of counts.

FOLLOW-UP 3 The statement may differ from this reading.

Ask whether deletions can come from both parts, whether duplicates count separately, and whether order matters. Everything above relies on those three answers.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Min deletions to avoid overlap in first k
#
# This question evaluates array and sequence manipulation
# skills, set-based reasoning about value overlap, and the
# ability to compute minimal edits while preserving
# relative order constraints.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only elements of the first part may be deleted.
# 2. Elements have weights (deleting some is more costly).
# 3. The statement may differ from this reading.
#
# INTERVIEW NOTES
# Values do not interact with each other, so each value
# can be decided on its own. A value that appears on both
# sides must disappear from one side entirely, and the
# cheapest side is the one with fewer copies. The answer
# is the sum, over values that appear on both sides, of
# the smaller of the two counts.
#
# BRUTE FORCE (say it first): Try every set of deletions
# Enumerate subsets of elements to delete in increasing
# size, and return the first size that leaves the two
# parts disjoint. Exact and a good oracle, but
# exponential. Time O(2^n * n). Space O(n).


import itertools
from collections import Counter


# BRUTE FORCE (say it first): Try every set of deletions |
# time O(2^n * n)
def min_deletions_brute(a, k):
    n = len(a)
    for size in range(n + 1):
        for gone in itertools.combinations(
            range(n), size
        ):
            left = {
                a[i]
                for i in range(min(k, n))
                if i not in gone
            }
            right = {
                a[i]
                for i in range(min(k, n), n)
                if i not in gone
            }
            if not left & right:
                return size
    return n


# BASE QUESTION (optimal): Sum of min(count in the first
# k, count in the rest) over shared values
# Idea: Count each value in the first k elements and in
# the rest. For every value with a positive count on both
# sides, one side has to lose all of its copies, so add
# the smaller count.
def min_deletions(a, k):
    # the values in a[:k] and a[k:] must not overlap; for
    # each
    # shared value delete the copies on the cheaper side
    left, right = Counter(a[:k]), Counter(a[k:])
    return sum(min(left[v], right[v])
               for v in left.keys() & right.keys())


# PLAN: which indices to delete
def deletions_plan(a, k):
    # which indices to delete (both sides allowed)
    left, right = Counter(a[:k]), Counter(a[k:])
    out = []
    for v in left.keys() & right.keys():
        # delete the cheaper side's copies of the value
        side = range(k, len(a))
        if left[v] <= right[v]:
            side = range(k)
        out += [i for i in side if a[i] == v]
    return sorted(out)


# FOLLOW-UP 1: Only elements of the first part may be
# deleted.
# Answer: Then every shared value must be removed from the
# first part entirely, so the cost is the number of first-
# part elements whose value occurs in the rest (a set
# lookup per element).
def min_deletions_first_only(a, k):
    # only the first part may lose elements: every first-
    # part
    # element whose value appears later must go
    rest = set(a[k:])
    return sum(1 for x in a[:k] if x in rest)


# FOLLOW-UP 2: Elements have weights (deleting some is
# more costly).
# Answer: Replace counts by the sum of weights per side
# for each value; the choice per value is still
# independent, so take the cheaper side. Cost: unchanged
# O(n); the per-value choice sums weights instead of
# counts.
def min_deletions_weighted(a, weights, k):
    # deleting element i costs weights[i]: per value take
    # the
    # cheaper side as a whole
    left, right = Counter(), Counter()
    for i, x in enumerate(a):
        (left if i < k else right)[x] += weights[i]
    return sum(min(left[v], right[v])
               for v in left.keys() & right.keys())


if __name__ == "__main__":
    a = [1, 2, 2, 3, 2, 1]
    # first part [1, 2, 2], rest [3, 2, 1]: shared 1 and 2
    assert min_deletions(a, 3) == 1 + 1
    assert min_deletions_first_only(a, 3) == 3
    assert min_deletions([1, 2, 3], 1) == 0
    assert min_deletions([], 0) == 0
    # a heavy copy on one side makes the other side
    # cheaper
    w = [1, 5, 5, 1, 1, 1]
    assert min_deletions_weighted(a, w, 3) == 1 + 1
    plan = deletions_plan(a, 3)
    kept = [x for i, x in enumerate(a) if i not in plan]
    kept_left = set(kept[:3 - sum(i < 3 for i in plan)])
```

```
    kept_right = set(kept[3 - sum(i < 3 for i in plan):])
    assert not kept_left & kept_right
    assert len(plan) == min_deletions(a, 3)
```

FUNCTION AND TEST CASES

Signature

```
def solution(list1: list[int], list2: list[int], k: int) -
> int:
```

Sample call

```
solution([1, 2, 3], [2, 1, 4, 5], 2)
```

Test cases

Input	Expected
([1, 2, 3], [2, 1, 4, 5], 2)	2
([1, 2, 3, 4], [2, 5, 3, 6], 2)	1

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Arrays and prefix sums

MEDIUM | IN APP LC 2 | PracHub | Technical Screen | 27 of 30

★ TOP 75: rank 1 in the Medium and Hard priority list

★ Add two big integers from digit lists

POSTED 2025-11-24 | 330 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 78.7 | 30 DAYS 69.3 | PRIORITY 1/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Arrays and prefix sums

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #2 Add Two Numbers

LeetCode match: LC 2 Add Two Numbers

Posted on PracHub: 2025-11-24 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/add-two-big-integers-from-digit-lists

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)
- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

Another example

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

QUESTION

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

EXAMPLES

```
Input: ([2, 4, 3], [5, 6, 4])
Expected Output: [7, 0, 8]
Explanation: 342 + 465 = 807, stored reversed as [7,
    0, 8].
Input: ([9, 9, 9], [1])
Expected Output: [0, 0, 0, 1]
```

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

APPROACH

Walk both lists with a carry, build the result list. O(max(m, n)).

HOW TO THINK ABOUT IT

Addition by hand: start at the least significant end, add the two digits and the carry, write digit % 10, carry digit // 10. The only edge cases are different lengths and a final carry. Never convert to a machine integer in an interview about big numbers, since the point is that the numbers do not fit.

Basis: LeetCode 989 / 415 (Add to Array-Form of Integer, Add Strings). Assumed input: lists of decimal digits, most significant digit first.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_lists(a, b):
    x = digits_to_int(a)  # reverse-order digits
    y = digits_to_int(b)
    return int_to_digits(x + y)
```

TIME	SPACE
O(n^2) for the string-to-integer conversions	O(n)

WHY IT IS SLOW

It relies on arbitrary-size integers and the conversions are quadratic for very long inputs. Adding digit by digit from the right with a carry is O(max length).

OPTIMAL SOLUTION · Digit by digit with a carry

TIME	SPACE
O(max(m, n))	O(max(m, n))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The digits are stored least significant first in a linked list.

Then the head is already the low end, so walk both lists together with the same carry loop and build the result list as you go. No reversal needed (LeetCode 2).

FOLLOW-UP 2 Multiply instead of add.

Schoolbook multiplication: the product of digits a[i] and b[j] lands in position i + j + 1, with the carry spilling to i + j. O(m * n); Karatsuba or FFT for very long numbers.

FOLLOW-UP 3 Subtract, or numbers in another base.

Subtraction borrows instead of carrying and needs the larger number first (compare lengths, then digits). Another base replaces 10 with the base in % and //.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Add two big integers from digit lists
#
# You are given two non-empty singly linked lists (or
# arrays) representing two non-negative integers. The
# digits are stored in **reverse order**, and each node
# contains a single digit from 0 to 9. Your task is to add
# the two numbers and return the sum as a new linked list,
# also in reverse order. You may assume: - Each list has
# at least one node. - The integers do not contain leading
# zeros, except the number 0 itself. ### Example Input: -
# List 1: 2 -> 4 -> 3 (represents the integer 342) - List
2:
# 5 -> 6 -> 4 (represents the integer 465) Output: - 7 ->
0 ->
# 8 (represents the integer 807, since 342 + 465 = 807)
# ### Another example Input: - List 1: 9 -> 9 -> 9
# (represents 999) - List 2: 1 (represents 1) Output: - 0
# -> 0 -> 0 -> 1 (represents 1000) ### Constraints - Let n
 be
# the length of the first list and m be the length of the
# second list. - 1 <= n, m <= 10^5.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The digits are stored least significant first in a
# linked list.
# 2. Multiply instead of add.
# 3. Subtract, or numbers in another base.
#
# INTERVIEW NOTES
# Addition by hand: start at the least significant end,
# add the two digits and the carry, write digit % 10,
# carry digit // 10. The only edge cases are different
# lengths and a final carry. Never convert to a machine
# integer in an interview about big numbers, since the
# point is that the numbers do not fit.
#
# BRUTE FORCE (say it first): Convert both lists to
# integers and add
# Join the digits, int() them, add, and split the result
# into digits. It is a valid shortcut in Python (big
# integers), but it hides the algorithm, depends on the
# language having big integers, and the conversions are
# quadratic for very long inputs. Time O(n^2) for the
# string-to-integer conversions. Space O(n).


# BRUTE FORCE (say it first): Convert both lists to
# integers and add | time O(n^2) for the string-to-integer
```

```
# conversions
def add_digits_int(a, b):
    return [
        int(ch)
        for ch in str(
            int("".join(map(str, a)))
            + int("".join(map(str, b)))
        )
    ]


# HELPERS for follow-up 1
class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next
```

```
# BASE QUESTION (optimal): Digit by digit with a carry
# Idea: Walk both lists from the right with indices i and
# j; at each step add a[i] (or 0 once exhausted), b[j] (or
# 0) and the carry, append sum % 10, and carry = sum //
# 10.
def add_digits(a, b):
    # arrays with the most significant digit FIRST
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry
        total += a[i] if i >= 0 else 0
        total += b[j] if j >= 0 else 0
        out.append(total % 10)
        carry = total // 10
        i, j = i - 1, j - 1
    return out[::-1] or [0]


# FOLLOW-UP 1: The digits are stored least significant
# first in a linked list.
# Answer: Then the head is already the low end, so walk
# both lists together with the same carry loop and build
# the result list as you go. No reversal needed (LeetCode
# 2).
def add_lists(a, b):
    # digits least significant first: add with a carry,
    # one pass
    dummy = tail = ListNode()
```

```
    carry = 0
    while a or b or carry:
        total = carry
        total += a.val if a else 0
        total += b.val if b else 0
        tail.next = ListNode(total % 10)
        tail, carry = tail.next, total // 10
        a = a.next if a else None
        b = b.next if b else None
    return dummy.next


# FOLLOW-UP 2: Multiply instead of add.
# Answer: Schoolbook multiplication: the product of digits
# a[i] and b[j] lands in position i + j + 1, with the
# carry spilling to i + j. O(m * n); Karatsuba or FFT for
# very long numbers.
def multiply_digits(a, b):
    # schoolbook: digit a[i] * b[j] lands at position i +
    # j + 1
    res = [0] * (len(a) + len(b))
    for i in range(len(a) - 1, -1, -1):
        for j in range(len(b) - 1, -1, -1):
            total = a[i] * b[j] + res[i + j + 1]
            res[i + j + 1] = total % 10
            res[i + j] += total // 10
    while len(res) > 1 and res[0] == 0:
        res.pop(0)
    return res


# FOLLOW-UP 3: Subtract, or numbers in another base.
# Answer: Subtraction borrows instead of carrying and
# needs the larger number first (compare lengths, then
# digits). Another base replaces 10 with the base in % and
# //.
def subtract_digits(a, b):
    # a - b where a >= b (most significant first): borrow
    i, j, borrow, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0:
        d = a[i] - borrow - (b[j] if j >= 0 else 0)
        borrow = 1 if d < 0 else 0
        out.append(d + 10 * borrow)
        i, j = i - 1, j - 1
```

```
    while len(out) > 1 and out[-1] == 0:
```

```
        out.pop()
    return out[::-1]


# HELPERS for the checks below
def make(digits):
    # digits least significant first -> linked list
    head = tail = ListNode(digits[0])
    for d in digits[1:]:
        tail.next = ListNode(d)
        tail = tail.next
    return head


def to_digits(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


if __name__ == "__main__":
    # 342 + 465 = 807, digits reversed
    got = add_lists(make([2, 4, 3]), make([5, 6, 4]))
    assert to_digits(got) == [7, 0, 8]
    # a final carry adds a digit: 99 + 1 = 100
    carry = add_lists(make([9, 9]), make([1]))
    assert to_digits(carry) == [0, 0, 1]
    assert add_digits([9, 9, 9], [1]) == [1, 0, 0, 0]
    assert add_digits([0], [0]) == [0]
    product = multiply_digits([1, 2, 3], [4, 5])
    assert product == [5, 5, 3, 5]
    assert multiply_digits([0], [9, 9]) == [0]
    assert subtract_digits([1, 0, 0], [1]) == [9, 9]
    assert subtract_digits([5, 0], [5, 0]) == [0]
```

FUNCTION AND TEST CASES

Sample call

```
add_two_numbers([2, 4, 3], [5, 6, 4])
```

Test cases

Input	Expected
([2, 4, 3], [5, 6, 4])	[7, 0, 8]
([9, 9, 9], [1])	[0, 0, 0, 1]

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Arrays and prefix sums

MEDIUM | PracHub | Technical Screen | 28 of 30

Minimize calls to find all bad test pairs

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TALK NO LEETCODE FREQUENCY PRIORITY 194/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Arrays and prefix sums

Optimal answer from: the interview site's own solution

LeetCode match: No direct match (adaptive group testing)

Posted on PracHub: 2025-11-05 (updated 2026-06-18)

Original: https://prachub.com/coding-questions/minimize-calls-to-find-all-bad-test-pairs

QUESTION

You have (n) atomic tests (t_1, t_2, \dots, t_n). You are given access to a black-box function:

```
bool runTest(set<Test> S)
```

which behaves as follows:

- **`runTest(S) == True`** if the subset (S) contains **no** incompatible (bad) pair of tests.
- **`runTest(S) == False`** if the subset (S) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** ((t_i, t_j)), where (i\ne j). You do **not** know in advance which pairs are bad or how many bad pairs there are. Your goal is to discover **all** bad pairs while making as few calls to **`runTest`** as possible. You may adaptively choose each subset (S) based on the results of previous calls.

Tasks

1. Design an algorithm that, using only calls to **`runTest`**, finds **all** bad pairs ((t_i, t_j)).
2. Analyze the number of calls to **`runTest`** your algorithm requires, as a function of:

- (n): the total number of tests, and - (k): the total number of bad pairs.

3. Argue why your algorithm is correct (i.e., why it will eventually find every bad pair and not report any good pair as bad). You do not need to provide full production code, but describe the algorithm clearly enough that it could be implemented.

You have n atomic tests numbered 0 to n - 1. Some unknown unordered pairs of tests are incompatible, called bad pairs. A black-box oracle runTest(S) returns True if the subset S contains no bad pair, and False if S contains at least one bad pair. In this non-interactive version, you are given bad_pairs only so your code can simulate the oracle. Implement the following deterministic oracle-discovery strategy and return both the bad pairs it discovers and the number of oracle calls it makes. Strategy:

QUESTION

1. Process vertices from right to left: v = n - 1, n - 2, ..., 0.
2. Maintain all bad pairs already discovered among vertices greater than the current v.
3. For the suffix vertices v + 1 through n - 1, greedily partition them into color classes such that no discovered bad pair appears inside one color class. Since all suffix-suffix bad pairs have already been discovered, every color class is truly compatible internally.
4. For each color class C, find all bad partners of v inside C using recursive group testing:

- Call runTest({v} union C).
- If the oracle returns True, v has no bad pair with any test in C.
- If C has size 1 and the oracle returns False, that single pair is bad.
- Otherwise split C into two halves and recurse on both halves. Return all discovered bad pairs sorted lexicographically, where each pair is represented as [i, j] with i < j, along with the exact number of runTest calls made by this strategy.

Constraints
0 <= n <= 5000 <= len(bad_pairs) <= n * (n - 1) / 2Each bad pair contains two distinct test indices in the range [0, n - 1]bad_pairs may be given in any endpoint order

EXAMPLES

```
Input: (0, [])
Expected Output: ([], 0)
```

Explanation: There are no tests and therefore no oracle calls are needed.

```
Input: (4, [])
Expected Output: ([], 3)
```

Explanation: For v = 2, 1, and 0, the algorithm queries the compatible suffix once and finds no bad pairs. v = 3 has an empty suffix.

Hints

If a candidate set C is internally compatible, then runTest({v} union C) tells you whether v conflicts with at least one element of C.Process tests from right to left so that when handling v, all bad pairs inside the suffix v + 1 ... n - 1 are already known and can be used to build compatible groups.

APPROACH

Binary search to find a minimal failing prefix (its last element is in a bad pair), then binary search among earlier tests for its partner; remove and repeat. About O(p log n) calls for p pairs.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what runTest returns (true if the set contains no incompatible pair), whether a test can be in more than one bad pair, and whether you may call it on any subset.

BRUTE FORCE TO SAY OUT LOUD

Call runTest on every pair of tests. O(n^2) calls.

OPTIMAL IDEA

Find a minimal failing prefix by binary search: add tests one at a time conceptually and binary search for the smallest prefix that fails; the last element of that prefix is part of a bad pair. Then binary search over the earlier tests to find its partner. Record the pair, remove one of the two as needed, and repeat. About O(p log n) calls for p bad pairs.

DRY-RUN THESE CASES

No bad pairs (one call). One bad pair. A test that conflicts with many others. Two disjoint bad pairs.

LIKELY FOLLOW-UPS

Batch calls, noisy tests, or a limit on subset size.

PITFALLS

Removing both members of a pair and missing other pairs that involve them. Binary searching without the monotone property of prefixes.

HOW TO THINK ABOUT IT

Each call is a yes/no question about a whole set, so binary search is possible: a failing set can be narrowed by halving. Keep the tests you have processed in groups that each pass runTest (independent sets). A new test only needs to be checked against each group, and if a group plus the new test fails, binary search finds its partners in that group in about log(size) calls each.

Basis: Written; group testing / finding all edges of a hidden graph with independent-set queries. runTest(S) is True when S contains no incompatible (bad) pair.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
bad = []
for a, b in combinations(tests, 2):
    if not runTest({a, b}):
        bad.append((a, b))
```

TIME	SPACE
n(n - 1) / 2 calls	O(1)

WHY IT IS SLOW

n(n - 1) / 2 calls even when nothing is bad. Keeping groups of mutually compatible tests and binary searching only when a group fails needs about n + p log n calls for p bad pairs.

OPTIMAL SOLUTION · Independent groups plus binary search for partners

TIME	SPACE
O(n * g + p log n) calls (g = number of groups, p = number of bad pairs)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many calls does it need, and can you prove a lower bound?

With p bad pairs among n tests about p log n calls are needed in the worst case (each pair carries log n bits of information about where it is), and the group method uses O(n * g + p log n). With no bad pairs it uses n - 1 calls, one per test after the first.

FOLLOW-UP 2 Only one bad pair exists.

Binary search for the smallest failing prefix (its last element is in the pair), then binary search among the earlier elements for the partner: about 2 log n calls.

FOLLOW-UP 3 runTest is expensive but can run in parallel.

Test all groups for the new element in the same round, or process several elements at once with their own groups, trading a few extra calls for fewer sequential rounds.

FOLLOW-UP 4 Only one bad pair exists.

Binary search the shortest failing prefix (its last element is in the pair), then binary search the earlier tests for the partner: about 2 log2(n) calls instead of n(n - 1) / 2 (verified by counting calls).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimize calls to find all bad test pairs
#
# You have \(n\) atomic tests \(t_1, t_2, \dots, t_n\).
# You are given access to a black-box function: python
# bool runTest(set<Test> S) which behaves as follows: -
# runTest(S) == True if the subset \(S\) contains **no**
# incompatible (bad) pair of tests. - runTest(S) == False
# if the subset \(S\) contains **at least one**
# incompatible (bad) pair of tests. There is some unknown
# set of unordered **bad pairs** \((t_i, t_j)\), where
# \(i\ne j\). You do **not** know in advance which pairs
# are bad or how many bad pairs there are. Your goal is to
# discover **all** bad pairs while making as few calls to
# runTest as possible. You may adaptively choose each
# subset \(S\) based on the results of previous calls. ###
```

```
# Tasks 1. Design an algorithm that, using only calls to
# runTest, finds **all** bad pairs \((t_i, t_j)\). 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many calls does it need, and can you prove a
# lower bound?
# 2. Only one bad pair exists.
# 3. runTest is expensive but can run in parallel.
# 4. Only one bad pair exists.
#
# INTERVIEW NOTES
# Each call is a yes/no question about a whole set, so
# binary search is possible: a failing set can be narrowed
# by halving. Keep the tests you have processed in groups
# that each pass runTest (independent sets). A new test
# only needs to be checked against each group, and if a
# group plus the new test fails, binary search finds its
# partners in that group in about log(size) calls each.
#
# BRUTE FORCE (say it first): Call runTest on every pair
# Try each pair {a, b} on its own. Always correct, but the
# number of calls grows quadratically even if there are no
# bad pairs at all. Time n(n - 1) / 2 calls. Space O(1).


import itertools


# BRUTE FORCE: ask about every pair
def bad_pairs_brute(tests, run_test):
```

```
    # every pair costs a call: n * (n - 1) / 2 calls
    pairs = itertools.combinations(tests, 2)
    return {(a, b) for a, b in pairs
            if not run_test({a, b})}


# HELPERS for the base question
def shortest_failing_prefix(items, extra, run_test):
    # binary search: the smallest prefix of `items` that
    # fails
    # together with `extra`; its last item is a partner
    lo, hi = 1, len(items)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(items[:mid]) | extra):
            lo = mid + 1
        else:
            hi = mid
    return items[lo - 1]


def partners(group, t, run_test):
    # every member of the group that is bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = shortest_failing_prefix(rest, {t}, run_test)
        found.append(x)
        rest.remove(x)
    return found


# BASE QUESTION (optimal): Independent groups plus binary
# search for partners
# Idea: Process tests one by one. Keep groups of tests
# that pass runTest.
def find_bad_pairs(tests, run_test):
    # keep groups of tests that pass together; a new test
    # is
    # checked against each group and joins one with no
    # partner
    groups, pairs = [], set()
    for t in tests:
```

```
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
```

```
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            if not found and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs


# FOLLOW-UP 2: Only one bad pair exists.
# Answer: Binary search for the smallest failing prefix
# (its last element is in the pair), then binary search
# among the earlier elements for the partner: about 2 log
# n calls.
# FOLLOW-UP 4: Only one bad pair exists.
# Answer: Binary search the shortest failing prefix (its
# last element is in the pair), then binary search the
# earlier tests for the partner: about 2 log2(n) calls
# instead of n(n - 1) / 2 (verified by counting calls).
def one_bad_pair(tests, run_test):
    # exactly one bad pair: about 2 * log2(n) calls
    if run_test(set(tests)):
        return None
    # the shortest failing prefix ends with one of the
    # pair
    b = shortest_failing_prefix(tests, set(), run_test)
    earlier = tests[:tests.index(b)]
    # then search the earlier tests for the partner
    a = shortest_failing_prefix(earlier, {b}, run_test)
    return (a, b)


if __name__ == "__main__":
    def make_oracle(bad):
        calls = [0]

        def run_test(subset):
            calls[0] += 1
            ordered = sorted(subset)
            pairs = itertools.combinations(ordered, 2)
            return not any(p in bad for p in pairs)
        return run_test, calls

    bad = {(3, 17), (5, 29)}
```

```
    run, calls = make_oracle(bad)
    assert find_bad_pairs(list(range(30)), run) == bad
    assert calls[0] < 435 // 2
    run, calls = make_oracle(set())
    assert find_bad_pairs(list(range(30)), run) == set()
    assert calls[0] <= 30
    run, calls = make_oracle({(10, 40)})
    assert one_bad_pair(list(range(64)), run) == (10, 40)
    # about 2 * log2(64) = 12 calls, far below 2016 pairs
    assert calls[0] <= 16
    run, _ = make_oracle({(1, 2)})
    assert bad_pairs_brute([1, 2, 3, 4], run) == {(1, 2)}
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, bad_pairs: list[list[int]]) -> tuple[
list[list[int]], int]:
```

Sample call

```
solution(0, [])
```

Test cases

Input	Expected
`(0, [])`	`([], 0)`
`(4, [])`	`([], 3)`

ALSO REPORTED · 1 more

Find All Bad Pairs Using runTest Oracle | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/find-bad-pairs-with-oracle

Standard "group testing" / Dorfman testing problem. Walk through the analogy with COVID pooled tests for intuition.
Algorithm sketch:
Test the full set. If true, no bad pairs.
Otherwise, binary-search to find one bad pair by partitioning the set and testing halves.
Remove the items that participate in the located bad pair; repeat.
Tighter bound: with up to K bad pairs, total queries are O(K log N) using adaptive group testing.
A naive $O(N^2)$ pairwise scheme works but the interviewer expects you to beat it with adaptive splitting.
Verbalize correctness: each binary-search step rules out half the candidates; you can always reduce the search space until exactly one bad pair is isolated.

Preparation: Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Arrays and prefix sums

Site tags: array, grid, medium

MEDIUM | 1Point3Acres | onsite-coding | 29 of 30

Rotate Any m×n 2D Matrix

STUDY: TYPE | **ALL TIME 56.3** | **30 DAYS 0** | **PRIORITY 57/228**

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Arrays and prefix sums

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #48 Rotate Image

LeetCode match: LC 48 Rotate Image (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/rotate-matrix-rectangle

QUESTION

EXAMPLES

```
[[1,2,3],[4,5,6]] (2×3) rotated 90° CW ->
    [[4,1],[5,2],[6,3]] (3×2).
```

APPROACH

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which direction and angle, whether in place is required, and the dimensions. A non-square matrix cannot be rotated in place in the same buffer, since its shape changes.

BRUTE FORCE TO SAY OUT LOUD

Create a new matrix and copy each cell to its rotated position with an index formula. O(mn) space.

OPTIMAL IDEA

Clockwise 90 degrees: reverse the rows and transpose, [list(r) for r in zip(*matrix[::-1])]. Counter-clockwise: transpose then reverse the rows. 180 degrees: reverse the rows and each row. For a square matrix rotate in place by cycling four cells per layer.

DRY-RUN THESE CASES

The 2 x 3 example giving [[4,1],[5,2],[6,3]], 1 x 1, 1 x n, an empty matrix, four rotations returning the original.

LIKELY FOLLOW-UPS

In place square rotation, rotation by k times 90 degrees, or non-square in place with a flat array and cycles.

PITFALLS

Mixing up the index formula direction. Trying to rotate a rectangle in place in a 2D list.

HOW TO THINK ABOUT IT

A 90 degree clockwise rotation sends the cell at row i, column j to row j, column (m - 1 - i) of the new n x m matrix. That formula is the whole problem; 'reverse the rows then transpose' is just a short way to write it. A rectangle cannot be rotated in place because its shape changes.

Basis: Generalizes LeetCode 48, Rotate Image, from an n x n matrix to any m x n matrix.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def rotate(old):
    m, n = len(old), len(old[0])
    new = [[0] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = old[i][j]
    return new
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

This is already optimal in time, O(m * n); its cost is the second matrix, O(m * n) extra space. A rectangle cannot be rotated in place (its shape changes), so it is the expected answer; the zip(reversed rows) one-liner is the same idea.

OPTIMAL SOLUTION · Rotate 90 degrees clockwise by reverse + transpose

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can a non-square matrix be rotated in place?

Not in a rectangular buffer, because the dimensions swap. You can do it in place only if the matrix is stored as a flat array and you permute the elements along cycles of the index map (i, j) to (j, m - 1 - i), which is O(mn) time but needs cycle bookkeeping. In an interview, allocate the new matrix and say why.

FOLLOW-UP 2 How would you rotate by k * 90 degrees?

Reduce k modulo 4 first. Then apply the single-turn function that many times, or write the four direct index maps so you only build the output once.

FOLLOW-UP 3 Rotate by k quarter turns.

Reduce k modulo 4 first; four turns are the identity. Apply the single-turn function that many times (at most three), or write the three index formulas directly to build the answer once.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Rotate Any m×n 2D Matrix
#
# Examples: [[1,2,3],[4,5,6]] (2×3) rotated 90° CW ->
# [[4,1],[5,2],[6,3]] (3×2).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can a non-square matrix be rotated in place?
# 2. How would you rotate by k * 90 degrees?
# 3. Rotate by k quarter turns.
#
# INTERVIEW NOTES
# A 90 degree clockwise rotation sends the cell at row i,
# column j to row j, column (m - 1 - i) of the new n x m
# matrix. That formula is the whole problem; 'reverse the
# rows then transpose' is just a short way to write it. A
# rectangle cannot be rotated in place because its shape
# changes.
#
# BRUTE FORCE (say it first): Allocate the new matrix and
# apply the index formula
# Create an n x m matrix and copy each cell old[i][j] to
# new[j][m - 1 - i]. Correct and the clearest statement of
# the rotation; the zip trick below is the same thing in
# one line. Time O(m * n). Space O(m * n).


# BRUTE FORCE (say it first): Allocate the new matrix and
# apply the index formula | time O(m * n)
def rotate_by_formula(matrix):
    m = len(matrix)
    if m == 0:
        return []
    n = len(matrix[0])
    new = [[None] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = matrix[i][j]
    return new


# BASE QUESTION (optimal): Rotate 90 degrees clockwise by
# reverse + transpose
# Idea: Rotating clockwise puts the last row first, so
# reverse the rows and transpose (zip with *). A non-
# square matrix changes shape from m x n to n x m, which
```

```
# is why it cannot be done in place the way LeetCode 48
# is.
def rotate_cw(matrix):
    # reverse the rows, then swap rows and columns
    return [list(row) for row in zip(*matrix[::-1])]


# COUNTER-CLOCKWISE
def rotate_ccw(matrix):
    # swap rows and columns, then reverse the rows
    return [list(row) for row in zip(*matrix)][::-1]


# INDEX RULE version
def rotate_by_index(matrix):
    # the same result from the index rule, built once:
    # new[j][m - 1 - i] = old[i][j]
    m, n = len(matrix), len(matrix[0]) if matrix else 0
    new = [[None] * m for _ in range(n)]
    for i in range(m):
        for j in range(n):
            new[j][m - 1 - i] = matrix[i][j]
    return new


# FOLLOW-UP 2: How would you rotate by k * 90 degrees?
# Answer: Reduce k modulo 4 first. Then apply the single-
# turn function that many times, or write the four direct
# index maps so you only build the output once.
# FOLLOW-UP 3: Rotate by k quarter turns.
# Answer: Reduce k modulo 4 first; four turns are the
# identity. Apply the single-turn function that many times
# (at most three), or write the three index formulas
# directly to build the answer once.
def rotate_k(matrix, k):
    # four quarter turns are the identity: use k mod 4
    for _ in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix


if __name__ == "__main__":
    a = [[1, 2, 3], [4, 5, 6]]
    # the 2 x 3 example becomes 3 x 2
    assert rotate_cw(a) == [[4, 1], [5, 2], [6, 3]]
```

```
    assert rotate_by_index(a) == rotate_cw(a)
    assert rotate_ccw(a) == [[3, 6], [2, 5], [1, 4]]
    # a clockwise then an anticlockwise turn cancel out
    assert rotate_ccw(rotate_cw(a)) == a
    assert rotate_k(a, 4) == a
    assert rotate_k(a, 2) == [[6, 5, 4], [3, 2, 1]]
    assert rotate_k(a, 5) == rotate_cw(a)
    assert rotate_cw([[7]]) == [[7]]
    assert rotate_cw([]) == []
```

NOTES FROM CANDIDATES

The in-place rectangle rotation is essentially cycle-decomposition on the index permutation — explain it but don't burn time implementing.
Reported follow-up: rotate inside a window of a larger matrix while keeping the rest untouched.

PREPARATION

Write square in-place rotation from memory in under 8 min. Have a 60-second explanation for why rectangle in-place is hard and what trade-offs you'd take.

STUDY HERE: topic 2, Arrays, Hashing and Two Pointers | main topic: Arrays and prefix sums

MEDIUM | HackTheRounds | Onsite | 30 of 30

Sum of Two Squares in Two Ways

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 206/228

ABOUT THIS QUESTION

Topic: Arrays, Hashing and Two Pointers: Arrays and prefix sums

Optimal answer from: the interview site's own solution

QUESTION

Given an integer n, find all integers s < n such that s can be expressed as the sum of two squares in exactly two distinct unordered ways. For example: 50 = 1^2 + 7^2 = 5^2 + 5^2.

EXAMPLES

Example 1	
Input	`n = 100`
Output	`[50, 65, 85]`
Why	`50 = 1^2 + 7^2 = 5^2 + 5^2, 65 = 1^2 + 8^2 = 4^2 + 7^2, 85 = 2^2 + 9^2 = 6^2 + 7^2`

Example 2	
Input	`n = 50`
Output	`[]`
Why	`No number less than 50 has exactly two representations.`

CONSTRAINTS

- 1 <= n <= 2^31 - 1

APPROACH

Taxicab-number style: count a^2 + b^2 over a <= b with a hash map, keep sums with exactly two representations

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether pairs are unordered with a <= b, whether zero is allowed, and whether the bound is strict (s < n).

BRUTE FORCE TO SAY OUT LOUD

For each s below n try every a and b. O(n * sqrt(n)).

OPTIMAL IDEA

Loop over a <= b with a^2 + b^2 < n and count each sum in a hash map. Return the sums whose count is exactly two. There are about pi * n / 8 such pairs, so O(n) time and space. For the example, 50 = 1^2 + 7^2 = 5^2 + 5^2, so 50 qualifies when n > 50.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

n = 50 (50 itself is excluded because the bound is strict), a small n with no answers, a sum with three representations (excluded), a perfect square such as 25 = 0 + 25 = 9 + 16 if zero is allowed.

LIKELY FOLLOW-UPS

Exactly k ways, or counting without a dictionary using a sieve.

PITFALLS

Counting (a, b) and (b, a) as different. Including s equal to n.

HOW TO THINK ABOUT IT

Do not test each s separately. Generate every pair (a, b) once, add its sum to a counter, and read the answer from the counter. The loop is bounded by a^2 + b^2 < n, so a and b are at most sqrt(n); a <= b removes order duplicates.

Basis: Taxicab-number style counting. The examples fix the rule: a and b are positive integers with a <= b, so 25 = 3^2 + 4^2 has only one representation (0^2 + 5^2 is not allowed).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    result = []
    for s in range(1, n):
        ways = 0
        for a in range(1, isqrt(s) + 1):
            rest = s - a * a
            b = isqrt(rest)
            if b >= a and b * b == rest:
                ways += 1
        if ways == 2:
            result.append(s)
    return result
```

TIME	SPACE
O(n * sqrt(n))	O(1)

WHY IT IS SLOW

Every s searches all pairs again, O(n * sqrt(n)). Generating each pair once and counting sums in a hash map is O(n).

OPTIMAL SOLUTION · Count every pair once in a hash map

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Exactly k ways instead of two.

Change the comparison from == 2 to == k; the counting loop is the same. For k = 3 the smallest is 325.

FOLLOW-UP 2 Allow zero (0^2 + b^2).

Start a at 0; then perfect squares like 25 gain a representation, so some answers appear or disappear. Check against the examples to see which convention the question uses (here, positive only).

FOLLOW-UP 3 n is huge (10^12) and you only need to test one number.

Factorise the number: the count of representations as a sum of two squares depends only on the primes congruent to 1 mod 4 (Jacobi's formula), so a factorisation answers it in sublinear time.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Sum of Two Squares in Two Ways
#
# Given an integer n, find all integers s < n such that s
# can be expressed as the sum of two squares in exactly
# two distinct unordered ways. For example: 50 = 1^2 + 7^2
# = 5^2 + 5^2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Exactly k ways instead of two.
# 2. Allow zero (0^2 + b^2).
# 3. n is huge (10^12) and you only need to test one
# number.
#
# INTERVIEW NOTES
# Do not test each s separately. Generate every pair (a,
# b) once, add its sum to a counter, and read the answer
# from the counter. The loop is bounded by a^2 + b^2 < n,
# so a and b are at most sqrt(n); a <= b removes order
# duplicates.
#
# BRUTE FORCE (say it first): For each s below n, try
# every a and b
# For every s count the pairs (a, b) with a <= b and a^2 +
# b^2 = s. Correct, but it re-searches pairs for every s.
# Time O(n * sqrt(n)). Space O(1).


from collections import Counter
from math import isqrt


def brute_count(m, zero=False):
    a = 0 if zero else 1
    count = 0
    while 2 * a * a <= m:
        rest = m - a * a
        b = isqrt(rest)
        if b * b == rest and rest > 0 and b >= a:
            count += 1
        a += 1
    return count


# BRUTE FORCE (say it first): For each s below n, try
# every a and b | time O(n * sqrt(n))
def two_squares_brute(n):
    out = []
    for s in range(1, n):
        ways = sum(
            1
            for a in range(1, int(s**0.5) + 1)
            for b in range(a, int(s**0.5) + 1)
            if a * a + b * b == s
        )
        if ways == 2:
            out.append(s)
    return out


# HELPERS for follow-up 3
def factorise(m):
    # trial division: fine up to about 10 ** 12
    out, p = {}, 2
    while p * p <= m:
        while m % p == 0:
            out[p] = out.get(p, 0) + 1
            m //= p
        p += 1
    if m > 1:
        out[m] = out.get(m, 0) + 1
    return out


# BASE QUESTION (optimal): Count every pair once in a hash
# map
# Idea: Loop a from 1 while 2a^2 < n and b from a while
# a^2 + b^2 < n, adding each sum to a Counter. There are
# about pi * n / 8 pairs. The answer is the sorted list of
# sums whose count is exactly 2.
# FOLLOW-UP 1: Exactly k ways instead of two.
# Answer: Change the comparison from == 2 to == k; the
# counting loop is the same. For k = 3 the smallest is
# 325.
# FOLLOW-UP 2: Allow zero (0^2 + b^2).
# Answer: Start a at 0; then perfect squares like 25 gain
# a representation, so some answers appear or disappear.
# Check against the examples to see which convention the
# question uses (here, positive only).
def sums_in_k_ways(n, k=2, zero=False):
    # every s < n is counted once per pair a <= b with
    # a*a + b*b = s. Keep the sums seen exactly k times.
    ways = Counter()
    # FOLLOW-UP 2: zero=True allows 0^2 + b^2
    a = 0 if zero else 1
    while 2 * a * a < n:
        b = a
        while a * a + b * b < n:
            ways[a * a + b * b] += 1
            b += 1
        a += 1
    # FOLLOW-UP 1: k ways instead of exactly two
    return sorted(s for s, c in ways.items()
                  if c == k and s > 0)


# FOLLOW-UP 3: n is huge (10^12) and you only need to test
# one number.
# Answer: Factorise the number: the count of
# representations as a sum of two squares depends only on
# the primes congruent to 1 mod 4 (Jacobi's formula), so a
# factorisation answers it in sublinear time.
def representations(m, zero=False):
    # follow-up: ONE huge number. The number of ways
    # depends
    # only on the primes that are 1 mod 4 (Jacobi): with
    # B = product of (exponent + 1) there are 4B ordered
    # signed pairs. Remove the special shapes a == 0 and
    # a == b, then every other unordered pair is counted 8
```

```
    # times.
    if m <= 0:
        return 0
    boost = 1
    for p, e in factorise(m).items():
        if p % 4 == 3 and e % 2:
            return 0
        if p % 4 == 1:
            boost *= e + 1
    is_square = isqrt(m) ** 2 == m
    half = m % 2 == 0 and isqrt(m // 2) ** 2 == m // 2
    plain = (4 * boost - 4 * is_square - 4 * half) // 8
    return plain + half + (is_square if zero else 0)


if __name__ == "__main__":
    assert sums_in_k_ways(100) == [50, 65, 85]
```

```
    assert sums_in_k_ways(10) == []
    assert sums_in_k_ways(1000, 3)[0] == 325
    # with zero allowed, perfect squares gain a way
    assert 25 not in sums_in_k_ways(100)
    assert 25 in sums_in_k_ways(100, 2, zero=True)
    assert representations(50) == 2
    assert representations(325) == 3
    assert representations(3) == 0
    assert representations(25) == 1
    assert representations(25, zero=True) == 2
    # a 12 digit number in a moment (13 * 5 * 5 * ...)
    assert representations(5 ** 5 * 13 * 17 * 29) > 10
    for m in range(1, 600):
        assert representations(m) == brute_count(m), m
        both = representations(m, True)
        assert both == brute_count(m, True)
        if m < 100:
            two = representations(m) == 2
            assert (m in sums_in_k_ways(100)) == two
```

ALSO REPORTED · 1 more

Find numbers with exactly two sum-of-squares forms | PracHub | Technical Screen

https://prachub.com/coding-questions/find-numbers-with-exactly-two-sum-of-squares-forms

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 3 OF 10 | CORE TOPIC

Trees and Tries

22 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	17	4
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Trees and Tries | 22 questions | tap a title to jump to it

TREES (15)

E	1. Find right-side view of binary tree	PracHub	2025-11-01
M	2. Expression Tree Single-Leaf Mutation	1Point3Acres	2026-09-27
M	3. Compute File System Total Size	1Point3Acres	2026-09-02
M	4. Nested List Weight Sum (LC 339) with Follow-Ups	1Point3Acres	2026-08-16
M	5. Longest Consecutive Path in Binary Tree	1Point3Acres	2026-07-02
M	6. Employee Hierarchy Add / Delete	1Point3Acres	2026-06-10
M	7. Streaming Insert + Find K-th Largest	1Point3Acres	2026-02-04
M	8. Find mode in a trinary search tree	PracHub	2025-11-16
M	9. Solve tree, graph, sliding-window problems	PracHub	2025-08-04
M	10. Implement zigzag level-order traversal	PracHub	2025-08-01
M	11. Lowest Common Ancestor of a Binary Tree III	HackTheRounds	no date
M	12. Lowest Common Ancestor	HackTheRounds	no date
H	13. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree	PracHub	2026-09-16
H	14. Huffman Tree Construction	1Point3Acres	2026-06-01
H	15. Compute distance-sum from every tree node	PracHub	2026-01-22

TRIES (7)

M	16. URL Request Handler Router	1Point3Acres	2026-10-16
M	17. Trie Prefix Search / Autocomplete	1Point3Acres	2026-07-02
M	18. Greedy Longest-Match Dictionary Tokenizer	1Point3Acres	2026-06-15
M	19. Check if all substrings are dictionary words	PracHub	2026-03-11
M	20. Return dictionary words matching a prefix	PracHub	2026-03-09
M	21. Solve chat, grid paths, and car rentals	PracHub	2025-12-02
H	22. Implement Longest-Match Text Replacement	PracHub	2026-01-20

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

EASY | IN APP LC 199 | PracHub | Technical Screen | 1 of 22

Find right-side view of binary tree

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 34.9 | 30 DAYS 0 | PRIORITY 111/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #199 Binary Tree Right Side View

LeetCode match: LC 199 Binary Tree Right Side View

Posted on PracHub: 2025-11-01 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-right-side-view-of-binary-tree

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where **`TreeNode`** is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode righ
t; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

APPROACH

BFS taking the last node of each level, or DFS visiting right before left and recording the first node seen at each depth. O(n).

HOW TO THINK ABOUT IT

The right side view is the last node of every level, or equivalently the first node you reach at each depth if you visit right children before left. Either way you need the depth of each node, so carry it in the traversal.

Basis: LeetCode 199, Binary Tree Right Side View.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def right_side_view(root):
    levels = level_order(root)  # list of lists
    return [level[-1] for level in levels]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Stores every node's value when only one per level is needed, O(n) space. A depth-first walk visiting the right child first and recording the first value at each depth needs O(h).

OPTIMAL SOLUTION · DFS visiting right before left, first node at each depth

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Left side view, or both.

Swap the order of the two recursive calls. For both views record the first and last node per depth in one pass.

FOLLOW-UP 2 Avoid recursion.

Use the BFS version and keep only the last node per level (no per-level list), or an explicit stack of (node, depth) pushing left before right so that right is popped first.

FOLLOW-UP 3 Top view or bottom view.

Track a horizontal distance per node (left minus one, right plus one) and keep the first (top) or last (bottom) node seen per distance in BFS order.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find right-side view of binary tree
#
# You are given the root of a binary tree. From the
# **right side** of the tree, at each depth you can see
# exactly one node: the rightmost node at that depth.
# Write a function that returns a list of the values of
# the nodes visible when looking at the tree from the
# right side, ordered from **top to bottom**. --- ###
# Function Signature (one possible form) text
# rightSideView(root: TreeNode) -> List[int] Where
# TreeNode is defined as: text class TreeNode { int val;
# TreeNode left; TreeNode right; } --- ### Example
# Consider this binary tree: text 1 / \ 2 3 \ \ 5 4 The
# right-side view is: text [1, 3, 4] Explain the algorithm
# you would use, including its time and space complexity.
# You may use either breadth-first search (BFS) or depth-
# first search (DFS).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Left side view, or both.
# 2. Avoid recursion.
# 3. Top view or bottom view.
#
# INTERVIEW NOTES
# The right side view is the last node of every level, or
# equivalently the first node you reach at each depth if
# you visit right children before left. Either way you
# need the depth of each node, so carry it in the
# traversal.
#
# BRUTE FORCE (say it first): Collect every level, then
# take the last of each
# Build the full list of nodes per level with a queue and
# return the last value of each list. It stores every node
# of the tree when only one per level is needed. Time
# O(n). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Collect every level, then
# take the last of each | time O(n)
def right_side_levels(root):
    if not root:
```

```
        return []
    levels, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        levels.append(level)
    return [lv[-1] for lv in levels]


# BASE QUESTION (optimal): DFS visiting right before left,
# first node at each depth
# Idea: Visit the right child first. The first node you
# meet at a depth that is not yet recorded is the
# rightmost one at that depth. Only O(height) memory is
# needed.
def right_side_view(root):
    # DFS, right child first: the first node seen at each
    # depth is the rightmost one
    out = []

    def dfs(node, depth):
        if node is None:
```

```
            return
        if depth == len(out):
            out.append(node.val)
        dfs(node.right, depth + 1)
        dfs(node.left, depth + 1)

    dfs(root, 0)
    return out


# FOLLOW-UP 1: Left side view, or both.
# Answer: Swap the order of the two recursive calls. For
# both views record the first and last node per depth in
# one pass.
def left_and_right_views(root):
    # both views in one pass: first and last node per
    # level
```

```
    left, right = [], []
    queue = deque([root] if root else [])
    while queue:
        left.append(queue[0].val)
        right.append(queue[-1].val)
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return left, right


# FOLLOW-UP 2: Avoid recursion.
# Answer: Use the BFS version and keep only the last node
# per level (no per-level list), or an explicit stack of
# (node, depth) pushing left before right so that right is
# popped first.
def right_side_view_bfs(root):
    # no recursion: keep only the LAST node of every level
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append(queue[-1].val)
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
    return out


# FOLLOW-UP 3: Top view or bottom view.
# Answer: Track a horizontal distance per node (left minus
# one, right plus one) and keep the first (top) or last
# (bottom) node seen per distance in BFS order.
def top_and_bottom_view(root):
    # horizontal distance: left child -1, right child +1
    top, bottom = {}, {}
    queue = deque([(root, 0)] if root else [])
    while queue:
        node, hd = queue.popleft()
        # the first node seen at a distance is the top
        # one;
        # the last one seen is the bottom one
        top.setdefault(hd, node.val)
```

```
        bottom[hd] = node.val
        if node.left:
            queue.append((node.left, hd - 1))
        if node.right:
            queue.append((node.right, hd + 1))
    order = sorted(top)
    return ([top[h] for h in order],
            [bottom[h] for h in order])
```

```
# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


if __name__ == "__main__":
    #     1
    #    / \
    #   2   3
    #    \    \
    #     5    4
    tree = TreeNode(1, TreeNode(2, None, TreeNode(5)),
                    TreeNode(3, None, TreeNode(4)))
    assert right_side_view(tree) == [1, 3, 4]
    assert right_side_view_bfs(tree) == [1, 3, 4]
    views = left_and_right_views(tree)
    assert views == ([1, 2, 5], [1, 3, 4])
    assert right_side_view(None) == []
    # a left-only tree is still seen from the right
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))
    assert right_side_view(chain) == [1, 2, 3]
    top, bottom = top_and_bottom_view(tree)
    assert top == [2, 1, 3, 4] and bottom == [2, 5, 3, 4]
```

FUNCTION AND TEST CASES

Signature

```
def rightSideView(level_order: list[int | None]) -> list[i
nt]:
```

Sample call

```
rightSideView([1, 2, 3, None, 5, None, 4])
```

Test cases

Input	Expected
`([1, 2, 3, None, 5, None, 4],)`	`[1, 3, 4]`
`([1, 2, 3, 4],)`	`[1, 3, 4]`

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

Site tags: tree, dfs, simulation, medium, state-machine

MEDIUM | 1Point3Acres | onsite-coding | 2 of 22

Expression Tree Single-Leaf Mutation

ASKED 2026-09-27 | 23 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 203/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/expression-tree-single-leaf-mutation

QUESTION

Evaluate a boolean expression tree, then answer the root value after each single-leaf flip. Current variants include XOR/OR/AND/NOT operators, no parent pointer on the node interface, and an expectation that the candidate cache parent links or paths during the initial traversal.SWEtreedfssimulationmediumstate-machineFrequencyLowLast asked2026-09-27Stageonsite-coding

APPROACH

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which operators exist (AND, OR, XOR, NOT), whether XOR and others are binary only, whether nodes have parent pointers, and whether leaves are flipped one at a time and then restored or cumulatively.

BRUTE FORCE TO SAY OUT LOUD

For each leaf flip it, evaluate the whole tree, and flip it back. O(n^2).

OPTIMAL IDEA

Evaluate once and cache each node's value. During that traversal record each leaf's path to the root (or parent links). To flip a leaf, recompute only the nodes on its path upward, stopping early when a parent's value does not change because the root value is then the same as before. That is O(depth) per flip.

DRY-RUN THESE CASES

A single leaf tree, a NOT at the root, XOR chains where every flip changes the root, OR with a true sibling (early stop).

LIKELY FOLLOW-UPS

Several flips applied together, or an undo.

INTERVIEW-DAY PLAN

PITFALLS

Mutating the cached values without restoring them between independent flips.

HOW TO THINK ABOUT IT

Flipping one leaf can only change the nodes on its path to the root, so cache every node's value and recompute only that path. The path is found through parent links you build during the first traversal. Stop early when a recomputed node keeps its old value, because nothing above it can change.

Basis: Written; incremental evaluation on a tree. Assumed rules (confirm them): leaves hold booleans; internal nodes are AND, OR, XOR (binary) or NOT (unary); report the root value after flipping each leaf on its own (flip, read the root, restore).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, leaves):
    answers = []
    for leaf in leaves:
        leaf.value = not leaf.value
        answers.append(evaluate(root))
        leaf.value = not leaf.value  # flip back
    return answers
```

TIME	SPACE
O(n^2)	O(h)

WHY IT IS SLOW

L leaves times an O(n) evaluation: O(n^2). Caching every node's value and recomputing only the path to the root (stopping when a value does not change) costs O(depth) per leaf.

OPTIMAL SOLUTION · Cache values and parents, recompute only the path with early stop

TIME	SPACE
O(n + L * depth)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The node interface has no parent pointer.

Build the path while collecting leaves: carry the list of ancestors with each stack entry, and use that list for the walk up. It costs O(depth) memory per leaf; the parent dict does the same job with O(n) total.

FOLLOW-UP 2 Apply several flips at once, or flips that stay.

The same cached values work: toggle, walk up with early stop, and keep the changes. For many flips between queries, recompute each affected path in turn.

FOLLOW-UP 3 Which leaves change the answer at all?

Compute top-down 'sensitivity': a leaf matters if every ancestor's other input lets it through (AND with true, OR with false, any XOR or NOT). One pass gives all of them in O(n).

FOLLOW-UP 4 Which leaves can change the answer at all?

Propagate a 'matters' flag downward: through AND the other side must be true, through OR the other side must be false, through XOR and NOT it always passes. A leaf matters exactly when the flag reaches it (checked against flipping every leaf). Subtree values are cached in practice, giving O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Expression Tree Single-Leaf Mutation
#
# Evaluate a boolean expression tree, then answer the root
# value after each single-leaf flip. Current variants
# include XOR/OR/AND/NOT operators, no parent pointer on
# the node interface, and an expectation that the
# candidate cache parent links or paths during the initial
# traversal.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The node interface has no parent pointer.
# 2. Apply several flips at once, or flips that stay.
# 3. Which leaves change the answer at all?
# 4. Which leaves can change the answer at all?
#
# INTERVIEW NOTES
# Flipping one leaf can only change the nodes on its path
# to the root, so cache every node's value and recompute
# only that path. The path is found through parent links
# you build during the first traversal. Stop early when a
# recomputed node keeps its old value, because nothing
# above it can change.
#
# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back
# For each of the L leaves, toggle it, evaluate the entire
# tree, and toggle it back. It is the correct baseline and
# the oracle for the cached version. Time O(n^2). Space
# O(h).


# HELPERS for the brute force
def _apply(op, a, b=None):
    return {
        "AND": lambda: a and b,
        "OR": lambda: a or b,
        "XOR": lambda: a != b,
        "NOT": lambda: not a,
    }[op]()


def _eval_full(node):
    if node.op is None:
        return node.val
    if node.op == "NOT":
        return not _eval_full(node.left)
    return _apply(
        node.op,
        _eval_full(node.left),
        _eval_full(node.right),
    )


def _leaves(root):
    out, stack = [], [root]
    while stack:
        n = stack.pop()
        if n.op is None:
            out.append(n)
        else:
            if n.right:
                stack.append(n.right)
            stack.append(n.left)
    return out


# BRUTE FORCE (say it first): Flip, re-evaluate the whole
# tree, flip back | time O(n^2)
class XNode:
    def __init__(
        self, op=None, left=None, right=None, val=None
    ):
        self.op, self.left, self.right, self.val = (
            op,
            left,
            right,
            val,
        )


def flips_brute(root):
    out = []
    for leaf in _leaves(root):
        leaf.val = not leaf.val
        out.append(_eval_full(root))
        leaf.val = not leaf.val
    return out


# HELPERS for the base question
def combine(op, a, b=None):
    # value of one gate from its input values
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
    if op == "XOR":
        return a != b
    # NOT has one input
    return not a


def right_value(value, node):
    # NOT has no right child
    return value[id(node.right)] if node.right else None


def build_cache(root):
    # one traversal caches parents, values and leaves
    parent, value, leaves = {id(root): None}, {}, []
```

```
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[id(child)] = node
                stack.append(child)
    # reversed order visits children before parents
    for node in reversed(order):
        if node.op is None:
            value[id(node)] = node.val
            leaves.append(node)
        else:
            a = value[id(node.left)]
            b = right_value(value, node)
            value[id(node)] = combine(node.op, a, b)
    return parent, value, leaves


# BASE QUESTION (optimal): Cache values and parents,
# recompute only the path with early stop
# Idea: One traversal stores every node's value and its
# parent.
def flip_answers(root):
```

```
    parent, value, leaves = build_cache(root)
    answers = []
    for leaf in leaves:
        undo, node = [], leaf
        new = not value[id(leaf)]
        # walk up only while a value really changes
        while node is not None and value[id(node)] != new:
            undo.append((node, value[id(node)]))
            value[id(node)] = new
            node = parent[id(node)]
            if node is None:
                break
            a = value[id(node.left)]
            b = right_value(value, node)
            new = combine(node.op, a, b)
        answers.append(value[id(root)])
        # put every changed value back for the next leaf
        for node, old in reversed(undo):
            value[id(node)] = old
    return leaves, answers


# FOLLOW-UP 3: Which leaves change the answer at all?
# Answer: Compute top-down 'sensitivity': a leaf matters
# if every ancestor's other input lets it through (AND
# with true, OR with false, any XOR or NOT). One pass
# gives all of them in O(n).
# FOLLOW-UP 4: Which leaves can change the answer at all?
# Answer: Propagate a 'matters' flag downward: through AND
# the other side must be true, through OR the other side
# must be false, through XOR and NOT it always passes.
def sensitive_leaves(root):
    # leaves whose flip can change the root value
    _, value, _ = build_cache(root)
    found, stack = [], [(root, True)]
    while stack:
        node, matters = stack.pop()
        if node.op is None:
            if matters:
                found.append(node)
            continue
        l, r = node.left, node.right
        # the flag passes if the other side lets it
        left_ok = right_ok = matters
        if node.op == "AND":
```

```
            left_ok = matters and value[id(r)]
            right_ok = matters and value[id(l)]
        elif node.op == "OR":
            left_ok = matters and not value[id(r)]
            right_ok = matters and not value[id(l)]
        stack.append((l, left_ok))
        if r is not None:
            stack.append((r, right_ok))
    return found


# HELPERS for the checks below
class Node:
    def __init__(self, op=None, left=None,
                 right=None, val=None):
        # op is AND, OR, XOR, NOT, or None for a leaf
        self.op, self.val = op, val
        self.left, self.right = left, right


def evaluate(node):
    # plain recursive evaluation (also the checker)
    if node.op is None:
        return node.val
    a = evaluate(node.left)
    b = evaluate(node.right) if node.right else None
    return combine(node.op, a, b)


if __name__ == "__main__":
    t, f = Node(val=True), Node(val=False)
    tree = Node("XOR", Node("AND", t, Node(val=True)),
                Node("NOT", f))
    leaves, answers = flip_answers(tree)
    # every cached answer matches a full re-evaluation
    for leaf, ans in zip(leaves, answers):
        leaf.val = not leaf.val
        assert evaluate(tree) == ans
        leaf.val = not leaf.val
    # the tree is unchanged afterwards
    assert evaluate(tree) is False
    # sensitivity agrees with really flipping each leaf
    base = evaluate(tree)
    for leaf in leaves:
        leaf.val = not leaf.val
```

```
        changes = evaluate(tree) != base
        leaf.val = not leaf.val
        assert changes == (leaf in sensitive_leaves(tree))
```

NOTES FROM CANDIDATES

Brute force is: collect leaves by inorder traversal, flip each leaf, and re-evaluate the whole tree.
The natural optimization is to cache subtree values and only recompute the path from the flipped leaf to the root.
Clarify whether XOR is binary-only and whether the tree is guaranteed valid.
Early-stop optimization is expected: after a leaf flip, recompute upward only until a parent value stays unchanged, then reuse the previous root value.
Settle how intermediate update state will be represented before implementation. In an L4 onsite, uncertainty about that representation caused a promising approach to fall back to brute-force code and contributed to negative coding feedback.

PREPARATION

Practice tree evaluation for boolean expression trees.
Add parent pointers or store root paths during leaf collection to support path-only recomputation.
Drill AND / OR / XOR truth tables so updates are mechanical.

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

Site tags: tree, dfs, medium

MEDIUM | 1Point3Acres | onsite-coding, phone-screen | 3 of 22

Compute File System Total Size

ASKED 2026-09-02 | 48 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 204/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/file-system-total-size

QUESTION

EXAMPLES

```
root = Dir(
  Dir(File(10), File(20)),
  File(5),
  Dir(File(30)),
)
Total: 65
```

APPROACH

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the structure can contain cycles or shared directories (links), whether files have sizes only, and whether to handle very deep trees.

BRUTE FORCE TO SAY OUT LOUD

Recursive DFS adding file sizes. Simple and usually the expected first answer.

OPTIMAL IDEA

DFS with a visited set if cycles or shared nodes are possible, so each node is counted once. Use an explicit stack for deep trees. If queried repeatedly, memoize directory totals and invalidate ancestors when a file changes. O(n) per computation.

DRY-RUN THESE CASES

The example giving 65, an empty directory, a directory containing only directories, a symlink loop.

LIKELY FOLLOW-UPS

Cache totals, handle updates, or find the largest directory.

PITFALLS

INTERVIEW-DAY PLAN

Recursion depth on a long chain. Double counting shared nodes.

HOW TO THINK ABOUT IT

A total over a tree is a post-order sum. Two things change the answer in real file systems: depth (recursion limit) and sharing or cycles (the same node reachable twice must be counted once). A visited set solves both the cycle risk and the double counting, and an explicit stack solves the depth.

Basis: Written; a tree sum (like LeetCode 690 / 1448 style traversals). Assumed model: File(size) and Dir(children...); directories may be shared (links) in the harder version.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def size(node):
    if is_file(node):
        return node.size
    total = 0
    for child in node.children:
        total += size(child)
    return total
```

TIME	SPACE
O(n)	O(depth) call stack

WHY IT IS SLOW

Linear time, which is optimal; the weaknesses are the recursion depth on very deep trees and an endless loop if a link creates a cycle. An explicit stack with a visited set fixes both.

OPTIMAL SOLUTION · Iterative traversal with a visited set

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

FOLLOW-UP 2 Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

FOLLOW-UP 3 Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. **Cost:** one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute File System Total Size
#
# Examples: root = Dir( Dir(File(10), File(20)), File(5),
# Dir(File(30)), ) Total: 65
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is queried often and files change.
# 2. Hard links: the same file appears under two
# directories.
# 3. Find the largest directory.
#
# INTERVIEW NOTES
# A total over a tree is a post-order sum. Two things
# change the answer in real file systems: depth (recursion
# limit) and sharing or cycles (the same node reachable
# twice must be counted once). A visited set solves both
# the cycle risk and the double counting, and an explicit
# stack solves the depth.
#
# BRUTE FORCE (say it first): Plain recursion
# Sum a file's size, or the sum over a directory's
# children. Correct for ordinary trees and the answer most
# interviews want first. It overflows on very deep trees
# and loops forever on a cycle. Time O(n). Space O(depth)
# call stack.


import heapq


# HELPERS for the brute force
class File:
    def __init__(self, size):
        self.size, self.parent = size, None


# BRUTE FORCE (say it first): Plain recursion | time O(n)
def total_size_recursive(node):
    if isinstance(node, File):
        return node.size
    return sum(
        total_size_recursive(c) for c in node.children
    )
```

```
# HELPERS for follow-up 3
class Dir:
    def __init__(self, *children):
        self.children, self.parent = list(children), None
        # cached total; None means "needs recomputing"
        self.cached = None
        for child in children:
            child.parent = self


def size_of(node):
    # a file's size, or a directory's cached total
    if isinstance(node, File):
        return node.size
    return node.cached


def cached_total(root):
    # post-order: a directory after all its children
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        if isinstance(node, Dir) and node.cached is None:
            stack.extend(node.children)
    for node in reversed(order):
        if isinstance(node, Dir) and node.cached is None:
            kids = node.children
            node.cached = sum(size_of(c) for c in kids)
    return root.cached


# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# Idea: An explicit stack removes the recursion limit, and
# a set of visited node ids makes each node count once, so
# shared directories are not double counted and cycles
# terminate.
# FOLLOW-UP 2: Hard links: the same file appears under two
# directories.
# Answer: Count a file once per identity (a visited set of
# ids, as coded) if the question asks for disk usage, but
# once per path if it asks for the sum of the sizes shown.
# Ask which.
def total_size(root, count_once=True):
```

```
    # iterative walk; count_once handles hard links
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        # FOLLOW-UP 2: hard links: count a shared file
        # once
        if count_once:
            if id(node) in seen:
                continue
            seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total


# FOLLOW-UP 1: The tree is queried often and files change.
# Answer: Cache each directory's total and, when a file
# changes, clear the cache only for its ancestors (walk
# parent links). A query recomputes only cleared nodes.
# Updates cost O(depth).
def resize(file, size):
    # a file changed: only its ancestors go stale
    file.size = size
    node = file.parent
    while node is not None:
        node.cached = None
        node = node.parent


# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def directory_totals(root):
    # the size of every directory, in one pass
```

```
    cached_total(root)
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        if isinstance(node, Dir):
            out.append(node.cached)
```

```
            stack.extend(node.children)
    return out


# FOLLOW-UP 3: Find the largest directory.
# Answer: Compute totals in post-order and keep the
# maximum; it is the same pass. For top-K directories keep
# a heap of size K. Cost: one post-order pass, O(n);
# keeping the top K directories with a heap of size K
# makes it O(n log K).
def top_k_directories(root, k):
    # keep a heap of size k: O(n log k)
    return heapq.nlargest(k, directory_totals(root))


if __name__ == "__main__":
    inner = Dir(File(10), File(20))
    root = Dir(inner, File(5), Dir(File(30)))
    assert total_size(root) == 65
    assert cached_total(root) == 65
    # change one file: only its ancestors are recomputed
    first = inner.children[0]
    resize(first, 15)
    assert root.cached is None and inner.cached is None
    assert cached_total(root) == 70
    assert top_k_directories(root, 2) == [70, 35]
    # a hard link: the same file under two directories
    shared = File(7)
    both = Dir(Dir(shared), Dir(shared))
    assert total_size(both) == 7
    assert total_size(both, count_once=False) == 14
```

NOTES FROM CANDIDATES

If the interviewer says "strict tree", drop the visited set and stick with plain DFS — extra cycle handling reads as over-engineering.
For the cache follow-up, store the cached total on each directory node; when a file changes, walk parent pointers invalidating along the path.
The interviewer also pushes for an iterative DFS (explicit stack) after the recursive solution lands — have both ready.

PREPARATION

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

Site tags: tree, recursion, parsing, medium

MEDIUM | 1Point3Acres | onsite-coding | 4 of 22

Nested List Weight Sum (LC 339) with Follow-Ups

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 16.5** | **30 DAYS 0** | **PRIORITY 156/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 339 Nested List Weight Sum

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-list-weighted-sum-parsing

QUESTION

EXAMPLES

```
"[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×2 + 14×2
    + 2×2 + 91×3 = 320.
Use [2, [91]] as a manual trace for bracket depth and
    the final-number flush.
```

APPROACH

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the input is already parsed or a string, whether negatives and multi-digit numbers appear, and whether weight is depth from the top (1 at the outside) or the inverse weight variant.

BRUTE FORCE TO SAY OUT LOUD

Parse into a tree of lists and recurse with depth. Simple but recursion can overflow.

OPTIMAL IDEA

For a string, scan once with a depth counter: '[' adds one, ']' subtracts one, and an integer is multiplied by the current depth when its last digit ends (flush the number on any non-digit and at the end of the string). For parsed lists use BFS level by level. O(n), O(1) extra for the scanner.

DRY-RUN THESE CASES

[3, 8, [2, 14], [2, [91]]] gives 320. A number glued to the closing bracket. A negative number. A deeply nested list.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

The inverse weight version (LeetCode 364), depth limit, or iterative versus recursive.

PITFALLS

Not flushing a trailing number. Treating '-' as a separate token.

HOW TO THINK ABOUT IT

The weight of a number is just the nesting depth where it appears, and the nesting depth is the number of '[' still open. That means you never need to build the nested structure: scan once with a counter, and flush a number whenever a non-digit arrives (and at the end).

Basis: LeetCode 339, Nested List Weight Sum, with the nested list given as a string.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(text):
    nested = json.loads(text)  # parse the lists
    return depth_sum(nested, 1)
def depth_sum(items, depth):
    total = 0
    for item in items:
        if isinstance(item, int):
            total += item * depth
        else:
            total += depth_sum(item, depth + 1)
    return total
```

TIME	SPACE
O(n)	O(depth)

WHY IT IS SLOW

It builds the whole nested structure and recurses as deep as the nesting, so memory and the call stack grow with depth. A single left-to-right scan with a bracket counter needs O(1) extra space.

OPTIMAL SOLUTION · Single pass over the string with a depth counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

FOLLOW-UP 2 What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

FOLLOW-UP 3 The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Nested List Weight Sum (LC 339) with
# Follow-Ups
#
# Examples: "[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×
2
# + 14×2 + 2×2 + 91×3 = 320. Use [2, [91]] as a manual
# trace for bracket depth and the final-number flush.
#
# FOLLOW-UPS (answered below, in this order):
# 1. When does the recursive (DFS) version become unsafe?
# 2. What is the inverse version (deeper integers weigh
# less)?
# 3. The inverse version (shallow integers weigh less,
# LeetCode 364).
#
# INTERVIEW NOTES
# The weight of a number is just the nesting depth where
# it appears, and the nesting depth is the number of '['
# still open. That means you never need to build the
# nested structure: scan once with a counter, and flush a
# number whenever a non-digit arrives (and at the end).
#
# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth
# json.loads turns the text into nested lists; a recursive
# function adds each integer times its depth. Clear, but
# it builds the whole structure and recurses as deep as
# the nesting. Time O(n). Space O(depth).
```

```
import json


# BRUTE FORCE (say it first): Parse the string into lists,
# then recurse with the depth | time O(n)
def depth_sum_parsed(s):
    def go(items, depth):
        return sum(
            (
                x * depth
                if isinstance(x, int)
                else go(x, depth + 1)
            )
            for x in items
        )
```

```
    return go(json.loads(s), 1)


# HELPERS for the base question
def with_depths(nested):
    # every integer with its depth (top level = 1),
    # using a stack so deep input cannot overflow
    stack = [(nested, 1)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            yield item, depth - 1


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum(nested):
    # deeper integers weigh more (LeetCode 339)
    return sum(v * d for v, d in with_depths(nested))


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
# Idea: Every integer is weighted by the number of '['
# currently open, so no tree needs to be built. Accumulate
# digits in a buffer and flush the buffer when a non-digit
# arrives or the string ends.
def weight_sum_string(s):
    # one pass over the text, O(1) extra space
    depth = total = 0
    num, sign, have = 0, 1, False
    # a closing sentinel flushes a trailing number
    for ch in s + "]":
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        # any other character ends the number being read
        if have:
```

```
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total
```

```
# FOLLOW-UP 2: What is the inverse version (deeper
# integers weigh less)?
# Answer: That is LeetCode 364. Do a level-by-level pass
# that keeps a running sum of everything seen so far and
# add it to the total once per level: total += running_sum
# on each level.
# FOLLOW-UP 3: The inverse version (shallow integers weigh
# less, LeetCode 364).
# Answer: Process level by level and keep a running sum of
# every integer seen so far; add the running sum to the
# total once per level that holds an integer or lies above
# one (trim trailing levels that contain only empty
# lists).
def inverse_weight_sum(nested):
    # deeper integers weigh less (LeetCode 364)
    pairs = list(with_depths(nested))
    if not pairs:
        return 0
    deepest = max(d for _, d in pairs)
    # weight = deepest + 1 - depth
    return sum(v * (deepest + 1 - d) for v, d in pairs)


if __name__ == "__main__":
    data = [3, 8, [2, 14], [2, [91]]]
    # 3 + 8 + 2*2 + 14*2 + 2*2 + 91*3 = 320
    assert weight_sum(data) == 320
    text = "[3, 8, [2, 14], [2, [91]]]"
    assert weight_sum_string(text) == 320
    assert weight_sum_string("[2, [91]]") == 2 + 91 * 2
    # inverse weights: [[1,1],2,[1,1]] -> 8
    assert inverse_weight_sum([[1, 1], 2, [1, 1]]) == 8
    assert inverse_weight_sum([1, [4, [6]]]) == 17
    # empty lists add nothing
```

```
    assert weight_sum([[], [[]]]) == 0
    # 20000 levels deep is still fine
    deep = []
    for _ in range(20000):
        deep = [deep, 1]
    assert weight_sum(deep) > 0
```

NOTES FROM CANDIDATES

The easy-to-miss bug is leaving the last buffered number uncommitted when the string ends. Manually tracing the small nested case exposed this during the interview, and fixing it proactively was part of the signal.
DFS visits each nested value once in O(n) time and uses O(d) call-stack space at maximum depth d; BFS is also O(n) time with a frontier proportional to the widest level. In the string parser, track the sign and whether a number is currently buffered, flush only when that flag is set, and flush before decrementing depth on].

PREPARATION

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

Site tags: tree, dfs, recursion, medium

MEDIUM | 1Point3Acres | phone-screen | 5 of 22

Longest Consecutive Path in Binary Tree

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 16.5** **30 DAYS 0** **PRIORITY 157/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #549 Binary Tree Longest Consecutive Sequence II

LeetCode match: LC 549 Binary Tree Longest Consecutive Sequence II (related)

Frequency: Single report | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-longest-consecutive-path

QUESTION

Given a binary tree, return the length of the longest parent-to-child path whose values increase consecutively by 1. Follow-up: allow the path to extend in any direction through the tree, not only from parent to child.SWEtreedfsrecursionmediumFrequencySingle reportLast asked2026-07-02Stagephone-screen

APPROACH

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether length counts nodes or edges, whether the path goes only parent to child (base) or can bend through a node (follow-up), and whether the sequence must increase by exactly one.

BRUTE FORCE TO SAY OUT LOUD

From every node walk down both children recording the longest run. O(n^2) on a skewed tree.

OPTIMAL IDEA

Base: DFS passing the current run length; it grows by one when the child is exactly the parent plus one, otherwise it resets to 1; keep the global best. O(n). Follow-up: each node returns the longest increasing and longest decreasing chain going down from it; the best path through a node is inc + dec - 1.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

A single node, an empty tree, duplicate values, a split branch, a descending chain.

LIKELY FOLLOW-UPS

The any-direction version (above), and an iterative version for very deep trees.

PITFALLS

Counting edges when the interviewer expects nodes. Resetting the run to 0 instead of 1.

HOW TO THINK ABOUT IT

Each node needs to know the longest run that ends at its parent, so pass the current length down and reset it to 1 when the value is not parent + 1. For the bending version each node must report two lengths upward (increasing and decreasing chains going down), and the best path through a node combines one of each.

Basis: LeetCode 298 (parent-to-child) and LeetCode 549 (any direction), Binary Tree Longest Consecutive Sequence I and II.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def walk(node):
    best = 1
    for child in (node.left, node.right):
        if child and child.val == node.val + 1:
            best = max(best, 1 + walk(child))
    return best
answer = 0
for node in all_nodes(root):
    answer = max(answer, walk(node))
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

A node deep in a chain is walked again from every ancestor, O(n * h). Passing the current run length down in one traversal makes it O(n).

OPTIMAL SOLUTION · Top-down DFS carrying the current run

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does the any-direction version return for a single node, or for duplicate values?

A single node is a path of length 1 (0 edges), and an empty tree is 0. Equal values never extend a chain because the step must be exactly +1 or -1, so duplicates reset both counters to 1 at that node.

FOLLOW-UP 2 Can you avoid recursion?

Yes: do an iterative post-order or a BFS that stores (node, run) pairs for the top-down version. The bending version needs child results before the parent, so use an explicit stack with a post-order visit or process nodes in reverse BFS order using a dict from node to (inc, dec). It matters because a chain-shaped tree makes the recursive depth n.

FOLLOW-UP 3 Return the path itself, not only its length.

Carry the current run as a list (or its start node and length) in the traversal and copy it into the best answer when it becomes longer.

FOLLOW-UP 4 The path may go in any direction (LeetCode 549).

Return the pair (longest increasing chain down, longest decreasing chain down) from each node, and update the global best with inc + dec - 1 at the node. Verified against exhaustive path enumeration for small trees.

FOLLOW-UP 5 The values must increase by 1 or decrease by 1 along the path but not mix.

That is exactly the any-direction version: the increasing and decreasing parts meet at the bend; a path cannot zig-zag between increasing and decreasing steps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Consecutive Path in Binary Tree
#
# Given a binary tree, return the length of the longest
# parent-to-child path whose values increase consecutively
# by 1. Follow-up: allow the path to extend in any
# direction through the tree, not only from parent to
# child.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does the any-direction version return for a
# single node, or for duplicate values?
# 2. Can you avoid recursion?
# 3. Return the path itself, not only its length.
# 4. The path may go in any direction (LeetCode 549).
# 5. The values must increase by 1 or decrease by 1 along
# the path but not mix.
#
# INTERVIEW NOTES
# Each node needs to know the longest run that ends at its
# parent, so pass the current length down and reset it to
# 1 when the value is not parent + 1. For the bending
# version each node must report two lengths upward
# (increasing and decreasing chains going down), and the
# best path through a node combines one of each.
#
# BRUTE FORCE (say it first): Start a downward walk from
# every node
# For each node, walk down both children while the next
# value is exactly one more, tracking the longest run.
# Correct and the oracle for the base question; nodes deep
# in a chain are re-walked from every ancestor. Time O(n *
# h). Space O(h).


# BRUTE FORCE (say it first): Start a downward walk from
# every node | time O(n * h)
def longest_consecutive_brute(root):
    best = 0

    def down(node):
        a = b = 0
        if node.left and node.left.val == node.val + 1:
            a = down(node.left)
        if (
            node.right
            and node.right.val == node.val + 1
        ):
            b = down(node.right)
        return 1 + max(a, b)

    def visit(node):
        nonlocal best
        if node:
            best = max(best, down(node))
            visit(node.left)
            visit(node.right)

    visit(root)
    return best


# BASE QUESTION (optimal): Top-down DFS carrying the
# current run
# Idea: Pass the length of the run ending at the parent.
# If the child is exactly parent + 1 the run grows,
# otherwise it restarts at 1. Track the global best.
# Length is counted in nodes; state that choice out loud.
def longest_consecutive(root, want_path=False):
    # parent -> child path whose values go up by exactly 1
    best, best_path = 0, []
    stack = [(root, None, [])] if root else []
    while stack:
        node, parent, run = stack.pop()
        # extend the run or start a new one at this node
        if parent is not None and node.val == parent + 1:
            run = run + [node.val]
        else:
            run = [node.val]
        if len(run) > best:
            best, best_path = len(run), run
        for child in (node.left, node.right):
            if child is not None:
                stack.append((child, node.val, run))
    # FOLLOW-UP 3: want_path=True returns the path itself
    return best_path if want_path else best


# FOLLOW-UP 4: The path may go in any direction (LeetCode
```

```
# 549).
# Answer: Return the pair (longest increasing chain down,
```

```
# longest decreasing chain down) from each node, and
# update the global best with inc + dec - 1 at the node.
# Verified against exhaustive path enumeration for small
# trees.
def longest_any_direction(root):
    # LeetCode 549: the path may bend at one node
    if root is None:
        return 0
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                stack.append(child)
    # inc[x] / dec[x]: longest chain going down from x
    inc, dec, best = {}, {}, 1
    # children are processed before their parents
    for node in reversed(order):
        up = down = 1
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                up = max(up, inc[child] + 1)
            if child.val == node.val - 1:
                down = max(down, dec[child] + 1)
        inc[node], dec[node] = up, down
        # the bend joins both chains (counted node once)
        best = max(best, up + down - 1)
    return best


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


if __name__ == "__main__":
    #     1
    #    / \
    #   2   0
    #  /
    # 3
```

```
    left = TreeNode(2, TreeNode(3))
    tree = TreeNode(1, left, TreeNode(0))
    assert longest_consecutive(tree) == 3
    assert longest_consecutive(tree, True) == [1, 2, 3]
    # bending: 2 - 1 - 0 or 3 - 2 - 1 - 0 is length 4
    assert longest_any_direction(tree) == 4
    assert longest_consecutive(None) == 0
    assert longest_any_direction(None) == 0
    # duplicates never extend a chain
    dup = TreeNode(5, TreeNode(5), TreeNode(5))
    assert longest_consecutive(dup) == 1
    assert longest_any_direction(dup) == 1
    # a long chain needs no recursion
    chain = TreeNode(0)
    cur = chain
    for i in range(1, 5000):
        cur.left = TreeNode(i)
        cur = cur.left
    assert longest_consecutive(chain) == 5000
```

NOTES FROM CANDIDATES

In the base version, carry the current consecutive length while traversing downward and reset to 1 whenever the child is not exactly parent + 1.
For the any-direction follow-up, track increasing and decreasing chain lengths at each node, then combine through the node when the left/right child values support a consecutive bridge.
Clarify whether the answer is measured in nodes or edges before coding; Google candidates usually state node count.

PREPARATION

Write the parent-to-child traversal in under 10 minutes with a global best.
Practice the bidirectional tree follow-up where each node returns two lengths upward.
Dry-run single-node, duplicate-value, and split-branch cases before discussing complexity.

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

Site tags: tree, hashmap, object-design, medium

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 6 of 22

Employee Hierarchy Add / Delete

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 227/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/employee-hierarchy-add-delete

QUESTION

Design an employee hierarchy class that supports adding employees under managers and deleting employees by id. Follow-ups cover deleting managers, deleting the CEO, cycle handling, and choosing a replacement manager by seniority.SWEtreehashmapobject-designmediumFrequencySingle reportLast asked2026-06-10Stageonsite-coding

APPROACH

Implement tree mutation operations with parent pointers. Practice deleting an internal node while reconnecting children. Add cycle checks with DFS or parent-chain walking before accepting a new edge.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what happens when a manager is deleted (promote a report, reassign to the grandparent, or reject), what happens when the CEO is deleted, whether adding can create a cycle, and how seniority is defined.

BRUTE FORCE TO SAY OUT LOUD

Store parent pointers only and search the whole tree to find children on every delete. O(n).

OPTIMAL IDEA

Keep id to node, and for each node a parent pointer and a children set. Add: validate the manager exists and the new id is free. Delete: take the children of the removed node and attach them to a replacement (the deleted node's parent, or a chosen successor), update pointers, and remove it from its parent's children. CEO needs a stated policy. Compare candidates by (level descending, insertion order ascending) for the seniority rule.

DRY-RUN THESE CASES

Delete a leaf, a middle manager, the CEO, an unknown id, adding with a missing manager.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Cycle detection by walking up parent pointers, bulk deletes, or a history of changes.

PITFALLS

Leaving dangling children references. Not choosing a policy for the root.

HOW TO THINK ABOUT IT

A hierarchy is a tree you update, so store links in both directions (parent per node, children per node). Then every operation touches only the neighbours of one node instead of searching the tree. For each delete case (leaf, middle, root) say the policy first, because the data structure is easy once the rule is fixed.

Basis: Written. Assumed policies (confirm them): deleting a manager hands their reports to that manager's own manager; deleting the CEO is allowed only if exactly one report exists (that report becomes CEO), otherwise it is rejected; ids are unique.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
manager = {}  # employee -> manager
def delete(e):
    reports = [x for x in manager
               if manager[x] == e]
    for x in reports:
        manager[x] = manager[e]
    del manager[e]
```

TIME	SPACE
O(n) per delete	**O(n)**

WHY IT IS SLOW

Every delete scans all employees, O(n). Keeping a children set per employee makes it O(number of reports).

OPTIMAL SOLUTION · Parent map plus children sets

TIME	SPACE
O(r) per delete where r is the number of reports	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support moving an employee under another manager and reject cycles.

Walk up from the proposed new manager through manager pointers; if you meet the employee being moved, the move would put a node under its own descendant, so reject it. O(depth).

FOLLOW-UP 2 When a manager leaves, promote the most senior report instead of the grandparent.

Store a hire order (insertion counter) per employee and pick the report with the smallest one: the new manager takes over the other reports. Say how you break ties.

FOLLOW-UP 3 The CEO leaves.

Pick a policy and say it: reject, promote the most senior report (the rest report to them), or leave the company rootless and hold a set of roots. The structure above needs only a promoted root.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Employee Hierarchy Add / Delete
#
# Design an employee hierarchy class that supports adding
# employees under managers and deleting employees by id.
# Follow-ups cover deleting managers, deleting the CEO,
# cycle handling, and choosing a replacement manager by
# seniority.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support moving an employee under another manager and
# reject cycles.
# 2. When a manager leaves, promote the most senior report
# instead of the grandparent.
# 3. The CEO leaves.
#
# INTERVIEW NOTES
# A hierarchy is a tree you update, so store links in both
# directions (parent per node, children per node). Then
# every operation touches only the neighbours of one node
# instead of searching the tree. For each delete case
# (leaf, middle, root) say the policy first, because the
# data structure is easy once the rule is fixed.
#
# BRUTE FORCE (say it first): Store only parent pointers
# and search for children
# Keep a dict from employee to manager. To delete, scan
# all entries to find the reports and reassign them.
# Correct, but every delete is a full scan. Time O(n) per
# delete. Space O(n).


# BRUTE FORCE (say it first): Store only parent pointers
# and search for children | time O(n) per delete
class OrgBrute:
    def __init__(self, ceo):
        self.manager = {ceo: None}

    def add(self, emp, mgr):
        if (
            emp in self.manager
            or mgr not in self.manager
        ):
            return False
        self.manager[emp] = mgr
        return True
```

```
    def delete(self, emp):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        reports = [
            e
            for e, m in self.manager.items()
            if m == emp
        ]
        if mgr is None and len(reports) != 1:
            return False
        for r in reports:
            self.manager[r] = mgr
        del self.manager[emp]
        return True


# BASE QUESTION (optimal): Parent map plus children sets
# Idea: Keep manager[emp] and reports[emp] (a set). Add
# inserts into both. Delete reattaches each report to the
# deleted node's manager and removes the node from its
# manager's set.
# FOLLOW-UP 1: Support moving an employee under another
# manager and reject cycles.
# Answer: Walk up from the proposed new manager through
# manager pointers; if you meet the employee being moved,
# the move would put a node under its own descendant, so
# reject it. O(depth).
class Org:
    def __init__(self, ceo):
        self.root = ceo
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        # hire order, used for "most senior" choices
        self.hired = {ceo: 0}
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
```

```
        self.clock += 1
        return True

    def senior(self, people):
        # the person hired earliest
        return min(people, key=lambda p: self.hired[p])

    def delete(self, emp, policy="grandparent"):
        if emp not in self.manager:
            return False
        mgr = self.manager[emp]
        kids = set(self.reports[emp])
        # FOLLOW-UP 3: the CEO leaves
        if mgr is None and not kids:
            # the CEO alone: nothing could take over
            return False
        # FOLLOW-UP 2: promote the most senior report
        if policy == "senior" or mgr is None:
            # the most senior report takes the place of
            # emp
            heir = self.senior(kids) if kids else None
        else:
            heir = mgr
        for r in kids:
            if r != heir:
                self.reassign(r, heir)
        if mgr is None:
            # the promoted report becomes the new CEO
            self.root = heir
```

```
            self.manager[heir] = None
        elif policy == "senior" and heir != mgr:
            self.reassign(heir, mgr)
        if mgr is not None:
            self.reports[mgr].discard(emp)
        del self.manager[emp], self.reports[emp]
        return True

    def reassign(self, emp, new_mgr):
        old = self.manager[emp]
        if old is not None and old in self.reports:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)

    # FOLLOW-UP 1: move an employee, reject cycles
```

```
    def move(self, emp, new_mgr):
        # walk up from the new manager: meeting emp is a
        # cycle
        node = new_mgr
        while node is not None:
            if node == emp:
                return False
            node = self.manager[node]
        self.reassign(emp, new_mgr)
        return True


if __name__ == "__main__":
    o = Org("ceo")
    for e, m in [("a", "ceo"), ("b", "a"), ("c", "a")]:
        assert o.add(e, m)
    assert not o.add("a", "ceo")
    assert not o.add("x", "nobody")
    # deleting a manager: reports go to the grandparent
    assert o.delete("a")
    assert o.manager["b"] == "ceo"
    assert o.manager["c"] == "ceo"
    # a move that would create a cycle is rejected
    o.add("d", "b")
    assert not o.move("b", "d") and o.move("d", "c")
    # promote the most senior report instead
    p = Org("ceo")
    for e, m in [("m", "ceo"), ("x", "m"), ("y", "m")]:
        p.add(e, m)
    assert p.delete("m", "senior")
    assert p.manager["x"] == "ceo"
    assert p.manager["y"] == "x"
    # the CEO leaves: the most senior report takes over
    q = Org("ceo")
    q.add("a", "ceo")
    q.add("b", "ceo")
    assert q.delete("ceo")
    assert q.root == "a" and q.manager["b"] == "a"
    assert not Org("solo").delete("solo")
```

NOTES FROM CANDIDATES

Use maps from id to node and id to children; keep parent pointers so deletes do not require a full traversal.
CEO deletion needs an explicit policy: reject, promote a child, or create a new root. State the policy and keep it consistent.
For the seniority follow-up, maintain insertion order on nodes and compare (level desc, insertion_order asc).

PREPARATION

Implement tree mutation operations with parent pointers. Practice deleting an internal node while reconnecting children. Add cycle checks with DFS or parent-chain walking before accepting a new edge.

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

Site tags: data-structure, tree, medium

MEDIUM | IN APP LC 703 | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 7 of 22

★ TOP 75: rank 68 in the Medium and Hard priority list

★ Streaming Insert + Find K-th Largest

ASKED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 118/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: your app's interview approach (LeetCode #703)

LeetCode match: LC 703 Kth Largest Element in a Stream

Frequency: Single report | Last asked: 2026-02-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/insert-find-kth-largest

QUESTION

EXAMPLES

```
insert(3)
insert(3)
insert(2)
findLargest(0) -> 3
findLargest(1) -> 3
findLargest(2) -> 2
```

APPROACH

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

HOW TO THINK ABOUT IT

If k can change from query to query you need every value, in a structure that supports rank queries; if k is fixed you only need the k largest, and a min-heap of size k gives the answer at its root. Ask which of the two it is: it decides everything.

Basis: LeetCode 703, Kth Largest Element in a Stream, generalized so k is passed to each query (0-indexed in the examples).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []
def insert(x):
    items.append(x)
def find_largest(k):
    ordered = sorted(items, reverse=True)
    return ordered[k]
```

BRUTE FORCE

TIME	SPACE
O(n log n) per query	O(n)

WHY IT IS SLOW

Every query re-sorts the whole history, O(n log n). A sorted list with binary search makes each query O(1) after an O(log n) search plus shift; for a fixed k a size-k heap is O(log k).

OPTIMAL SOLUTION · Sorted list with bisect (k chosen per query)

TIME	SPACE
O(log n) search + O(n) shift per insert, O(1) query	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if values can also be deleted?

The heap can no longer answer because removing an arbitrary element is O(k). Keep a sorted structure that supports delete: a balanced BST or sorted container with order statistics, or a Fenwick tree over compressed values (add -1 to delete). Both give O(log n) insert, delete and k-th.

FOLLOW-UP 2 What is the trade-off between the two designs?

The heap is the smallest and fastest when k is fixed and small (O(log k), O(k) memory) but cannot change k or delete. The sorted list answers any k and is simplest, at O(n) memory and a linear shift per insert. Choose from the requirements, and say which assumption you made.

FOLLOW-UP 3 Values can also be deleted.

A heap cannot delete an arbitrary value efficiently. Keep a sorted structure: a sorted list with bisect (O(n) per delete because of shifting) or a balanced tree / Fenwick tree over compressed values (O(log n) for insert, delete and k-th).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Insert + Find K-th Largest
#
# Examples: insert(3) insert(3) insert(2) findLargest(0) -
>
# 3 findLargest(1) -> 3 findLargest(2) -> 2
#
# FOLLOW-UPS (answered below, in this order):
```

```
# 1. What if values can also be deleted?
# 2. What is the trade-off between the two designs?
# 3. Values can also be deleted.
#
# INTERVIEW NOTES
# If k can change from query to query you need every
# value, in a structure that supports rank queries; if k
# is fixed you only need the k largest, and a min-heap of
# size k gives the answer at its root. Ask which of the
# two it is: it decides everything.
#
# BRUTE FORCE (say it first): Store everything and sort
# for every query
# Append values to a list; on a query sort it in
# descending order and read index k. Correct and the
# oracle; each query re-sorts everything. Time O(n log n)
# per query. Space O(n).


import heapq
from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Store everything and sort
# for every query | time O(n log n) per query
class StreamKthBrute:
    def __init__(self):
        self.a = []

    def insert(self, x):
        self.a.append(x)

    def findLargest(self, k):
        s = sorted(self.a, reverse=True)
        return s[k] if 0 <= k < len(s) else None


# BASE QUESTION (optimal): Sorted list with bisect (k
# chosen per query)
```

```
# Idea: Keep all values in an ascending list with insort.
# The k-th largest (0-indexed as in the examples) is a[-1
# - k].
# FOLLOW-UP 1: What if values can also be deleted?
# Answer: The heap can no longer answer because removing
# an arbitrary element is O(k).
# FOLLOW-UP 3: Values can also be deleted.
# Answer: A heap cannot delete an arbitrary value
# efficiently.
class StreamKth:
    """Any k, with deletes: a sorted list."""

    def __init__(self):
        self.a = []

    def insert(self, x):
        # keep the list sorted: O(log n) search, O(n)
        # shift
        insort(self.a, x)

    # FOLLOW-UP 1, 3: values can also be deleted
    def delete(self, x):
        # remove one copy of x if present
        i = bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False

    def find_largest(self, k):
        # k = 0 is the largest value
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
```

```
        return None


# FOLLOW-UP 2: What is the trade-off between the two
# designs?
# Answer: The heap is the smallest and fastest when k is
# fixed and small (O(log k), O(k) memory) but cannot
# change k or delete. The sorted list answers any k and is
# simplest, at O(n) memory and a linear shift per insert.
class FixedKth:
    """Fixed k, inserts only: a min-heap of the best k."""

    def __init__(self, k):
        self.k, self.heap = k, []

    def insert(self, x):
        heapq.heappush(self.heap, x)
        # keep only the k largest values
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)

    def kth_largest(self):
        # the smallest of the k largest
        if len(self.heap) < self.k:
            return None
        return self.heap[0]


if __name__ == "__main__":
    s = StreamKth()
    for x in (3, 3, 2):
        s.insert(x)
    top3 = [s.find_largest(k) for k in range(3)]
    assert top3 == [3, 3, 2]
    assert s.find_largest(3) is None
    # deletes work too
    assert s.delete(3) and not s.delete(9)
    assert [s.find_largest(k) for k in range(2)] == [3, 2]
    f = FixedKth(2)
    assert f.kth_largest() is None
    for x in (5, 1, 9, 7):
        f.insert(x)
    # the two largest are 9 and 7, so the 2nd largest is 7
    assert f.kth_largest() == 7
```

NOTES FROM CANDIDATES

An order-statistics structure (size-augmented BST / balanced tree / skiplist) gives O(log n) for both ops.
An order-statistics Fenwick tree over a coordinate-compressed value range gives O(log V).
A naive sorted array gives O(log n) find but O(n) insert; mention as a baseline.
Two heaps (the canonical streaming-median pattern) only work for a fixed k; the interviewer wants arbitrary k per call.

PREPARATION

Sketch the size-augmented BST node (left subtree count + duplicate count) on paper before the interview.
Practice the Fenwick-tree order-statistics trick (find k-th by binary search on prefix sums).
Be ready to defend the chosen data structure: balanced BST is clean but verbose; Fenwick is short but assumes bounded values.

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

MEDIUM | IN APP LC 501 | PracHub | Technical Screen | 8 of 22

Find mode in a trinary search tree

POSTED 2025-11-16 | 338 DAYS AGO | OLDER

STUDY: TALK ALL TIME 25.5 30 DAYS 0 PRIORITY 137/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #501 Find Mode in Binary Search Tree

LeetCode match: LC 501 Find Mode in Binary Search Tree

Posted on PracHub: 2025-11-16 (updated 2026-06-20)

Original: https://prachub.com/coding-questions/find-mode-in-a-trinary-search-tree

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val N
ode middle; // all values = val Node right; // all val
ues > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

QUESTION

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer

EXAMPLES

```
Input: ([], [], [], [], -1)
Expected Output: []
```

Explanation: The tree is empty, so there are no modes.

```
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
```

Explanation: A single-node tree has that node's value as the only mode.

Hints

In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.

Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer
Examples
Input: ([], [], [], [], -1)
Expected Output: []

QUESTION

Explanation: The tree is empty, so there are no modes.
Input: ([-2147483648], [-1], [-1], [-1], 0)
Expected Output: [-2147483648]
Explanation: A single-node tree has that node's value as the only mode.
Hints
Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

APPROACH

In-order traversal (left, middle, right) so equal values are consecutive; track current run length and the max; O(h) extra space.

HOW TO THINK ABOUT IT

In a valid search tree an in-order walk (left, middle, right) visits values in sorted order, so equal values are adjacent and you can count runs with a single running counter and O(height) memory instead of a dictionary. If the tree may be broken, the order guarantee is gone and you need a real frequency table.

Basis: LeetCode 501, Find Mode in Binary Search Tree, with a middle child for equal values. Valid case: left < val, middle = val, right > val.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def modes(root):
    counts = Counter()
    stack = [root]
    while stack:
        node = stack.pop()
        if node:
            counts[node.val] += 1
            stack += [node.left, node.middle,
                      node.right]
    top = max(counts.values())
    return [v for v in counts
            if counts[v] == top]
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but memory for every distinct value. In a valid tree an in-order walk (left, middle, right) gives equal values consecutively, so a running counter needs only O(height) space.

OPTIMAL SOLUTION · In-order traversal counting runs (O(height) extra space)

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree may be broken (it does not respect the ordering).

In-order no longer gives sorted values, so equal values need not be adjacent. Fall back to a frequency table over a full traversal (the Counter version): O(n) time, O(distinct values) space.

FOLLOW-UP 2 Only O(1) extra space (no stack, no dictionary).

Use Morris traversal for the in-order walk (temporary threads) with the same running-count logic; it mutates the tree briefly, so restore the pointers. Adapting it to the middle child needs care.

FOLLOW-UP 3 The tree is huge and values arrive in a stream.

For a stream use a count dictionary (or a Misra-Gries summary if only frequent values matter); the sorted-order trick needs the whole tree in order. **Cost:** O(1) per value with a counter and O(d) memory for the distinct values (or O(1 / epsilon) with Misra-Gries).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find mode in a trinary search tree
#
# You are given the root of a **trinary search tree**
# where each node has up to three children: text class
# Node { int val; Node left; // all values < val Node
# middle; // all values = val Node right; // all values >
# val } Assume the tree is intended to satisfy the
# ordering rules above. ## Task Return the **mode(s)** of
# the tree: the value(s) that occur most frequently among
# all nodes. - If there is one mode, return a single-
# element list. - If multiple values tie for highest
# frequency, return **all** such values (in any order). ##
# Constraints (typical) - Number of nodes n can be large
# (e.g., up to 10^5). - Values can be any 32-bit signed
# integers.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree may be broken (it does not respect the
# ordering).
# 2. Only O(1) extra space (no stack, no dictionary).
# 3. The tree is huge and values arrive in a stream.
#
# INTERVIEW NOTES
# In a valid search tree an in-order walk (left, middle,
# right) visits values in sorted order, so equal values
```

```
# are adjacent and you can count runs with a single
# running counter and O(height) memory instead of a
# dictionary. If the tree may be broken, the order
# guarantee is gone and you need a real frequency table.
#
# BRUTE FORCE (say it first): Collect every value in a
# Counter
# Traverse the whole tree, count each value and return the
# ones with the highest count. Correct for any tree (even
# a broken one) and the oracle; it uses memory
# proportional to the number of distinct values. Time
# O(n). Space O(n).


from collections import Counter


# BRUTE FORCE (say it first): Collect every value in a
# Counter | time O(n)
def modes_counter(root):
```

```
    c, stack = Counter(), [root] if root else []
    while stack:
        x = stack.pop()
        c[x.val] += 1
        stack.extend(
            ch
            for ch in (x.left, x.middle, x.right)
            if ch
        )
    if not c:
        return []
    top = max(c.values())
    return sorted(v for v, k in c.items() if k == top)


# BASE QUESTION (optimal): In-order traversal counting
# runs (O(height) extra space)
# Idea: Walk left, middle, right with an explicit stack.
def find_modes(root):
    # in-order: equal values are next to each other, so a
    # running count needs no dictionary
    best, modes, cur, run = 0, [], None, 0

    def visit(v):
        nonlocal best, modes, cur, run
        run = run + 1 if v == cur else 1
        cur = v
        if run > best:
            best, modes = run, [v]
        elif run == best and v not in modes:
            modes.append(v)

    stack, node = [], root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        visit(node.val)
        # the middle chain repeats node.val once per child
        mid = node.middle
        while mid:
            visit(mid.val)
            mid = mid.middle
        node = node.right
```

```
    return sorted(modes)


# FOLLOW-UP 1: The tree may be broken (it does not respect
# the ordering).
# Answer: In-order no longer gives sorted values, so equal
# values need not be adjacent. Fall back to a frequency
# table over a full traversal (the Counter version): O(n)
# time, O(distinct values) space.
def find_modes_counter(root):
    # a broken tree (ordering not respected): count
    # everything
    counts, stack = Counter(), [root] if root else []
    while stack:
        node = stack.pop()
        counts[node.val] += 1
        for child in (node.left, node.middle, node.right):
            if child:
                stack.append(child)
    if not counts:
        return []
    top = max(counts.values())
    return sorted(v for v in counts if counts[v] == top)


# HELPERS for the checks below
class TNode:
    def __init__(self, val, left=None, middle=None,
                 right=None):
        # left < val, middle == val, right > val
        self.val, self.left = val, left
        self.middle, self.right = middle, right


if __name__ == "__main__":
    #        5
    #     /  |  \
    #    3   5   8
    #   /|       |
    #  3 3       8
    threes = TNode(3, None, TNode(3, None, TNode(3)))
    tree = TNode(5, threes, TNode(5),
                 TNode(8, None, TNode(8)))
    assert find_modes(tree) == [3]
    assert find_modes_counter(tree) == [3]
```

```
    # a tie between 1 and 2
    tie = TNode(2, TNode(1), TNode(2), None)
    assert find_modes(tie) == [2]
    assert find_modes_counter(tie) == [2]
    assert find_modes(None) == []
    assert find_modes_counter(None) == []
    tie2 = TNode(2, TNode(1), None, TNode(3))
    assert find_modes(tie2) == [1, 2, 3]
```

FUNCTION AND TEST CASES

Signature

```
def solution(values: list[int], left: list[int], middle: l
ist[int], right: list[int], root: int) -> list[int]:
```

Sample call

```
solution([], [], [], [], -1)
```

Test cases

Input	Expected
([], [], [], [], -1)	[]
([42], [-1], [-1], [-1], 0)	[42]

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

MEDIUM | PracHub | Technical Screen | 9 of 22

★ TOP 75: rank 46 in the Medium and Hard priority list

★ Solve tree, graph, sliding-window problems

POSTED 2025-08-04 | 442 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 29/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #103 Binary Tree Zigzag Level Order Traversal, LeetCode #207 Course Schedule

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal + LC 207 Course Schedule

Posted on PracHub: 2025-08-04 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/solve-tree-graph-sliding-window-problems

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters. https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/ https://leetcode.com/problems/course-schedule/description/ https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form

EXAMPLES

```
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
```

Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.

QUESTION

```
Input: ([1],)
Expected Output: [[1]]
```

Explanation: A single-node tree has only one level.

Hints

A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.

Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.

Constraints

1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses

Examples

Input: (2, [[1, 0]])

Expected Output: True

Explanation: Course 0 can be taken first, then course 1.

Input: (2, [[1, 0], [0, 1]])

Expected Output: False

Explanation: The two courses depend on each other, forming a cycle.

Hints

Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.

Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.

Constraints

0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters

Examples

Input: ("eceba", 2)

Expected Output: 3

Explanation: The longest valid substring is 'ece', which has length 3.

Input: ("aa", 1)

Expected Output: 2

Explanation: The whole string contains only one distinct character.

Hints

A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

APPROACH

BFS by level with reversed alternate levels; topological sort / DFS cycle detection; plus a sliding-window task.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Three LeetCode problems: 103 zigzag level order, 207 course schedule, 340 longest substring with at most K distinct characters. For each ask the usual edge questions: empty tree, duplicate prerequisites, K = 0.

BRUTE FORCE TO SAY OUT LOUD

Zigzag: do a normal level order and reverse alternate levels afterwards. Course schedule: DFS from every course to find cycles. Substring: check every substring.

OPTIMAL IDEA

Zigzag: BFS level by level, reverse (or use a deque appended on the left or right) on every other level. O(n). Course schedule: Kahn's algorithm, with in-degree counts and a queue; if processed courses equal the course count there is no cycle. O(V + E). Substring (340): sliding window with a dict of counts; expand right, and while distinct characters exceed K shrink from the left, deleting a character at count 0. Track the maximum window. O(n).

DRY-RUN THESE CASES

Zigzag: empty tree, a single node, a skewed tree. Course schedule: no prerequisites, a self loop, a two-course cycle. Substring: K = 0 (answer 0), K larger than distinct characters, a repeated character.

LIKELY FOLLOW-UPS

Course Schedule II (return an order), longest substring with exactly K distinct, at most K distinct in a stream.

PITFALLS

Reversing the wrong levels. Not removing a key from the window dict when its count hits zero.

HOW TO THINK ABOUT IT

Three standard patterns. Level order needs a queue and a direction flag per level. 'Can everything be finished' is cycle detection in a directed graph: Kahn's algorithm removes nodes with no remaining prerequisites, and a cycle leaves some behind. 'Longest window with at most K of something' is a sliding window with a count map that shrinks from the left when the limit is exceeded.

Basis: Three LeetCode problems: 103 Binary Tree Zigzag Level Order, 207 Course Schedule and 340 Longest Substring with At Most K Distinct Characters.

BRUTE FORCE · Course schedule: DFS from every course looking for a cycle

BRUTE FORCE · PSEUDO-CODE

```
def can_finish(n, prereqs):
    graph = build_graph(n, prereqs)
    for course in range(n):
        if reaches(graph, course, course):
            return False  # a cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Each search can walk the whole graph again, O(V * (V + E)). Kahn's algorithm (remove courses with no unmet prerequisites) is O(V + E).

BRUTE FORCE · Longest window: check every substring

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, K):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            seen.add(s[j])
            if len(seen) > K:
                break
            best = max(best, j - i + 1)
    return best
```

TIME	SPACE
O(n^2)	O(k)

WHY IT IS SLOW

O(n^2) in the worst case. A sliding window with a count map that shrinks from the left when it exceeds K distinct characters is O(n).

BRUTE FORCE · Zigzag: level order, then reverse the odd levels

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Same O(n) time with one extra copy of every second level; filling each level from alternating ends of a deque avoids it.

OPTIMAL SOLUTION · Course schedule: Kahn's topological sort

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Longest window: sliding window with a count map

TIME	SPACE
O(n)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Zigzag: fill each level from the correct end

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Course Schedule II: return an order in which to take the courses.

The same Kahn loop records the order in which courses are removed; if fewer than n courses are removed, return an empty list.

FOLLOW-UP 2 Longest substring with exactly K distinct characters.

Take atMost(K) minus atMost(K - 1) for counting substrings, or keep the window and only record lengths when the distinct count is exactly K (careful with the shrink).

FOLLOW-UP 3 Zigzag on an n-ary tree, or the right-to-left variant.

Replace the two children by the child list, and swap which levels are appended on the left.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve tree, graph, sliding-window
# problems
#
# ##### Question LeetCode 103. Binary Tree Zigzag Level
# Order Traversal – given a binary tree root, return its
# zigzag level-order traversal. LeetCode 207. Course
# Schedule – given course prerequisites, determine if all
# courses can be finished by detecting cycles with
# topological sort. LeetCode 340. Longest Substring with
# At Most K Distinct Characters – given string s and
# integer k, return the length of the longest substring
# containing at most k distinct characters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Course Schedule II: return an order in which to take
# the courses.
# 2. Longest substring with exactly K distinct characters.
# 3. Zigzag on an n-ary tree, or the right-to-left
# variant.
#
# INTERVIEW NOTES
# Three standard patterns. Level order needs a queue and a
# direction flag per level. 'Can everything be finished'
# is cycle detection in a directed graph: Kahn's algorithm
# removes nodes with no remaining prerequisites, and a
# cycle leaves some behind. 'Longest window with at most K
# of something' is a sliding window with a count map that
# shrinks from the left when the limit is exceeded.
#
# BRUTE FORCE (say it first): Course schedule: DFS from
# every course looking for a cycle
# From each course follow prerequisites; if the walk
# returns to the start there is a cycle. Correct and the
# oracle; the same parts of the graph are walked again and
# again. Time O(V * (V + E)). Space O(V).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Course schedule: DFS from
# every course looking for a cycle | time O(V * (V + E))
def can_finish_dfs(n, prereqs):
    adj = defaultdict(list)
    for a, b in prereqs:
        adj[b].append(a)

    def reaches(start):
        seen, stack = set(), list(adj[start])
        while stack:
            x = stack.pop()
            if x == start:
                return True
            if x not in seen:
                seen.add(x)
                stack.extend(adj[x])
        return False

    return not any(reaches(c) for c in range(n))


# HELPERS for the base question
# FOLLOW-UP 1: Course Schedule II: return an order in
# which to take the courses.
# Answer: The same Kahn loop records the order in which
# courses are removed; if fewer than n courses are
# removed, return an empty list.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def course_order(n, prereqs):
    # (a, b): to take a you must take b first. Kahn's
    # algorithm;
    # if some course never frees up there is a cycle
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
```

```
    queue = deque(c for c in range(n) if indeg[c] == 0)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == n else []


# HELPERS for follow-up 2
```

```
def count_at_most_k(s, k):
    # number of substrings with at most k distinct letters
    if k < 1:
        return 0
    count, left, total = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        total += right - left + 1
    return total


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def zigzag(root):
    # level order, alternating direction
    out, left_to_right = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            # append for left-to-right, appendleft
            # otherwise
            if left_to_right:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        left_to_right = not left_to_right
    return out


# BASE QUESTION (optimal): Course schedule: Kahn's
```

```
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
# of the courses that depend on it, and queue those that
# reach zero.
def can_finish(n, prereqs):
    return len(course_order(n, prereqs)) == n


# BASE QUESTION (optimal): Course schedule: Kahn's
# topological sort
# Idea: Count prerequisites per course and start from
# courses with none. Remove a course, decrease the count
```

```
# of the courses that depend on it, and queue those that
# reach zero.
def longest_at_most_k(s, k):
    # sliding window with a count of each letter inside it
    count, left, best = defaultdict(int), 0, 0
    for right, ch in enumerate(s):
        count[ch] += 1
        # too many distinct letters: shrink from the left
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        best = max(best, right - left + 1)
    return best


# FOLLOW-UP 2: Longest substring with exactly K distinct
# characters.
# Answer: Take atMost(K) minus atMost(K - 1) for counting
# substrings, or keep the window and only record lengths
# when the distinct count is exactly K (careful with the
# shrink).
def count_exactly_k(s, k):
    # exactly k = at most k minus at most k - 1
    most = count_at_most_k(s, k)
    return most - count_at_most_k(s, k - 1)


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
```

```
        self.val, self.left, self.right = val, left, right


def build(values):
    # level-order list, None for a missing child
    if not values:
        return None
    root = TreeNode(values[0])
    queue, i = deque([root]), 1
    while queue and i < len(values):
        node = queue.popleft()
        for side in ("left", "right"):
            if i < len(values) and values[i] is not None:
                child = TreeNode(values[i])
                setattr(node, side, child)
                queue.append(child)
            i += 1
    return root


if __name__ == "__main__":
    tree = build([3, 9, 20, None, None, 15, 7])
    assert zigzag(tree) == [[3], [20, 9], [15, 7]]
    assert zigzag(None) == []
    assert can_finish(2, [(1, 0)])
    assert not can_finish(2, [(1, 0), (0, 1)])
    order = course_order(4, [(1, 0), (2, 1), (3, 2)])
    assert order == [0, 1, 2, 3]
    assert course_order(2, [(0, 1), (1, 0)]) == []
    assert longest_at_most_k("eceba", 2) == 3
    assert longest_at_most_k("aa", 1) == 2
    assert longest_at_most_k("abc", 0) == 0
    assert count_exactly_k("abab", 2) == 6
    assert count_at_most_k("abab", 1) == 4
```

FUNCTION AND TEST CASES

Signature

```
def solution(root: list[str]) -> list[list[int]]:
```

Sample call

```
solution([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
([3, 9, 20, None, None, 15, 7],)	[[3], [20, 9], [15, 7]]
([1],)	[[1]]

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

MEDIUM | IN APP LC 103 | PracHub | Technical Screen | 10 of 22

Implement zigzag level-order traversal

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TALK ALL TIME 38.7 30 DAYS 0 PRIORITY 96/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #103 Binary Tree Zigzag Level Order Traversal

LeetCode match: LC 103 Binary Tree Zigzag Level Order Traversal

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/implement-zigzag-level-order-traversal

QUESTION

Given the root of a binary tree, return the node values in zigzag level order: visit nodes level by level, reading the first level from left to right, the next from right to left, and so on alternating each level. Output a list of lists of integers where each inner list contains the values for one level in the required direction. Aim for O(n) time and O(w) extra space, where n is the number of nodes and w is the maximum width of the tree.

APPROACH

BFS by level; reverse (or use a deque appending left/right) on alternate levels. O(n).

HOW TO THINK ABOUT IT

It is a plain level-order traversal where every second level is read in the other direction. Either collect each level and reverse the odd ones, or fill each level from the correct end of a deque so no reversal is needed.

Basis: LeetCode 103, Binary Tree Zigzag Level Order Traversal.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def zigzag(root):
    levels = level_order(root)
    for i in range(1, len(levels), 2):
        levels[i].reverse()
    return levels
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Same O(n) time, plus an extra pass copying the reversed levels. Filling each level from the left or right end of a deque as it is built needs no reversal.

OPTIMAL SOLUTION · BFS filling each level from the left or the right

TIME	SPACE
O(n)	O(width)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Bottom-up level order (leaf level first).

Run the normal traversal and reverse the list of levels at the end.

FOLLOW-UP 2 Do it with DFS.

Pass the depth; append the value to levels[depth], at the end when the depth is even and at the front (insert at 0 or a deque) when odd. Recursion depth is the tree height.

FOLLOW-UP 3 N-ary tree.

Replace the two children by the list of children; the direction flag works the same way.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement zigzag level-order traversal
#
# Given the root of a binary tree, return the node values
# in zigzag level order: visit nodes level by level,
# reading the first level from left to right, the next
# from right to left, and so on alternating each level.
# Output a list of lists of integers where each inner list
# contains the values for one level in the required
# direction. Aim for O(n) time and O(w) extra space, where
# n is the number of nodes and w is the maximum width of
# the tree.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Bottom-up level order (leaf level first).
# 2. Do it with DFS.
# 3. N-ary tree.
#
# INTERVIEW NOTES
# It is a plain level-order traversal where every second
# level is read in the other direction. Either collect
# each level and reverse the odd ones, or fill each level
# from the correct end of a deque so no reversal is
# needed.
#
# BRUTE FORCE (say it first): Level order, then reverse
# every second level
```

```
# Do a normal breadth-first traversal collecting each
# level, and reverse the levels with odd index at the end.
# Easy to get right, with one extra pass over those
# levels. Time O(n). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Level order, then reverse
# every second level | time O(n)
def zigzag_reverse(root):
    levels, queue = [], deque([root] if root else [])
    while queue:
        levels.append([n.val for n in queue])
        for _ in range(len(queue)):
            n = queue.popleft()
            queue.extend(
                c for c in (n.left, n.right) if c
```

```
            )
    return [
        lv[::-1] if i % 2 else lv
        for i, lv in enumerate(levels)
    ]


# BASE QUESTION (optimal): BFS filling each level from the
# left or the right
# Idea: Keep a left-to-right flag. For each level build a
# deque; append on the right when going left to right and
# on the left when going right to left. Flip the flag
# after each level.
def zigzag_level_order(root):
    # BFS by levels; flip the direction on every level
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        level = deque()
        for _ in range(len(queue)):
            node = queue.popleft()
            if ltr:
                level.append(node.val)
            else:
                level.appendleft(node.val)
            for child in (node.left, node.right):
                if child:
                    queue.append(child)
        out.append(list(level))
        ltr = not ltr
    return out


# FOLLOW-UP 1: Bottom-up level order (leaf level first).
# Answer: Run the normal traversal and reverse the list of
# levels at the end.
def level_order_bottom_up(root):
    # leaf level first: the normal traversal, reversed at
    # the end
    out, queue = [], deque([root] if root else [])
    while queue:
        out.append([n.val for n in queue])
        for _ in range(len(queue)):
            node = queue.popleft()
            for child in (node.left, node.right):
```

```
                if child:
                    queue.append(child)
    return out[::-1]


# FOLLOW-UP 2: Do it with DFS.
# Answer: Pass the depth; append the value to
# levels[depth], at the end when the depth is even and at
# the front (insert at 0 or a deque) when odd. Recursion
# depth is the tree height.
def zigzag_dfs(root):
    # DFS: pass the depth; odd levels are filled at the
    # front
    levels = []

    def go(node, depth):
        if node is None:
            return
        if depth == len(levels):
            levels.append(deque())
        if depth % 2 == 0:
            levels[depth].append(node.val)
        else:
            levels[depth].appendleft(node.val)
        go(node.left, depth + 1)
        go(node.right, depth + 1)

    go(root, 0)
    return [list(x) for x in levels]


# FOLLOW-UP 3: N-ary tree.
# Answer: Replace the two children by the list of
# children; the direction flag works the same way.
def zigzag_nary(root):
    # an n-ary tree: a list of children instead of two
    out, ltr = [], True
    queue = deque([root] if root else [])
    while queue:
        vals = [n.val for n in queue]
        out.append(vals if ltr else vals[::-1])
        for _ in range(len(queue)):
            queue.extend(queue.popleft().children)
        ltr = not ltr
    return out
```

```


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


class NNode:
    def __init__(self, val, children=()):
        self.val, self.children = val, list(children)


if __name__ == "__main__":
    right = TreeNode(20, TreeNode(15), TreeNode(7))
    tree = TreeNode(3, TreeNode(9), right)
    want = [[3], [20, 9], [15, 7]]
    assert zigzag_level_order(tree) == want
    assert zigzag_dfs(tree) == want
    up = level_order_bottom_up(tree)
    assert up == [[15, 7], [9, 20], [3]]
    assert zigzag_level_order(None) == []
    assert zigzag_dfs(None) == []
    kids = [NNode(2, [NNode(5)]), NNode(3), NNode(4)]
    root = NNode(1, kids)
    assert zigzag_nary(root) == [[1], [4, 3, 2], [5]]
```

FUNCTION AND TEST CASES

Signature

```
def zigzagLevelOrder(values: list[int | None]) -> list[lis
t[int]]:
```

Sample call

```
zigzagLevelOrder([3, 9, 20, None, None, 15, 7])
```

Test cases

Input	Expected
([3, 9, 20, None, None, 15, 7],)	[[3], [20, 9], [15, 7]]
([1],)	[[1]]

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

MEDIUM | IN APP LC 1650 | HackTheRounds | Phone Screen | 11 of 22

Lowest Common Ancestor of a Binary Tree III

STUDY: TALK | ALL TIME 14.7 | 30 DAYS 0 | PRIORITY 163/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1650 Lowest Common Ancestor of a Binary Tree III

LeetCode match: LC 1650

QUESTION

Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA). Each node has a parent pointer.

EXAMPLES

Example 1	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1`
Output	`3`
Why	`The LCA of nodes 5 and 1 is 3.`

Example 2	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4`
Output	`5`
Why	`The LCA of nodes 5 and 4 is 5.`

Example 3	
Input	`root = [1,2], p = 1, q = 2`
Output	`1`
Why	`Node 1 is ancestor of node 2.`

CONSTRAINTS

- Number of nodes: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

APPROACH

Parent pointers: two-pointer swap trick, or a set of ancestors

HOW TO THINK ABOUT IT

With parent pointers each node is the head of a linked list that ends at the root, so the question is where two linked lists meet. If the lists have different lengths, walk each pointer to the end and then restart it on the other node: both pointers then travel the same total distance and arrive at the meeting node together.

Basis: LeetCode 1650, Lowest Common Ancestor of a Binary Tree III (every node has a parent pointer).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lca(p, q):
    ancestors = set()
    node = p
    while node:
        ancestors.add(node)
        node = node.parent
    node = q
    while node:
        if node in ancestors:
            return node
        node = node.parent
    return None
```

TIME	SPACE
O(h)	O(h)

WHY IT IS SLOW

Uses O(h) extra space for the set. Two pointers that switch to the other node's start when they fall off the root meet at the answer with O(1) space.

OPTIMAL SOLUTION · Two pointers that switch to the other node at the end

TIME	SPACE
O(h)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without the swap trick (it feels like magic).

Compute the depth of each node by walking up, lift the deeper one by the difference, then step both up together until they meet. Same O(h) time and O(1) space and easier to justify.

FOLLOW-UP 2 Many LCA queries on the same tree.

Preprocess with binary lifting (up[k][v] is the 2^k-th ancestor): O(n log n) to build, O(log n) per query. Or an Euler tour with a range-minimum structure for O(1) queries.

FOLLOW-UP 3 The nodes might not be in the same tree.

Then the two pointers both end as None after two passes, and None is the correct answer. The loop terminates because both pointers reach None at the same step.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Lowest Common Ancestor of a Binary Tree
# III
#
# Given two nodes of a binary tree p and q, return their
# lowest common ancestor (LCA). Each node has a parent
# pointer.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it without the swap trick (it feels like magic).
# 2. Many LCA queries on the same tree.
# 3. The nodes might not be in the same tree.
#
# INTERVIEW NOTES
# With parent pointers each node is the head of a linked
# list that ends at the root, so the question is where two
# linked lists meet. If the lists have different lengths,
# walk each pointer to the end and then restart it on the
# other node: both pointers then travel the same total
# distance and arrive at the meeting node together.
#
# BRUTE FORCE (say it first): Collect the ancestors of one
# node in a set
# Walk from p to the root storing every ancestor
# (including p), then walk from q upward until you hit a
# stored node. Simple and correct; it uses memory for the
# ancestor set. Time O(h). Space O(h).


def brute(p, q):
    # oracle: the first ancestor of q that is also one of
    # p
    above = set()
    while p:
        above.add(p)
        p = p.parent
    while q and q not in above:
        q = q.parent
    return q


# BRUTE FORCE (say it first): Collect the ancestors of one
# node in a set | time O(h)
class PNode:
    def __init__(self, val):
        self.val, self.left, self.right, self.parent = (
            val,
            None,
            None,
            None,
        )


# HELPERS for the base question
def lca_set(p, q):
    seen = set()
    while p:
        seen.add(id(p))
        p = p.parent
    while q:
        if id(q) in seen:
            return q
        q = q.parent
    return None


# HELPERS for follow-up 1
def depth(node):
    d = 0
    while node.parent:
        node = node.parent
        d += 1
    return d


# BASE QUESTION (optimal): Two pointers that switch to the
# other node at the end
# Idea: Move a from p and b from q one step up per round.
# When a pointer falls off the root, restart it at the
# other node.
def lca_swap(p, q):
    # two pointers climb to the root; at the top each one
    # jumps to the other node's start. Both walk the same
    # total distance, so they meet at the LCA (or both
    # reach
    # None together when the nodes are in different
    # trees).
    a, b = p, q
    while a is not b:
        a = a.parent if a else q
        b = b.parent if b else p
    return a


# FOLLOW-UP 1: Do it without the swap trick (it feels like
# magic).
# Answer: Compute the depth of each node by walking up,
# lift the deeper one by the difference, then step both up
# together until they meet. Same O(h) time and O(1) space
# and easier to justify.
def lca_by_depth(p, q):
    # follow-up: no trick. Lift the deeper node by the
    # difference in depth, then climb both until they
    # meet.
    dp, dq = depth(p), depth(q)
    while dp > dq:
        p, dp = p.parent, dp - 1
    while dq > dp:
        q, dq = q.parent, dq - 1
    while p is not q:
        p, q = p.parent, q.parent
        if p is None or q is None:
            return None
    return p


# FOLLOW-UP 2: Many LCA queries on the same tree.
# Answer: Preprocess with binary lifting (up[k][v] is the
# 2^k-th ancestor): O(n log n) to build, O(log n) per
# query. Or an Euler tour with a range-minimum structure
# for O(1) queries.
class LiftingLCA:
    """Follow-up: many queries on one tree.
    up[k][i] is the 2**k-th ancestor of node i: O(n log n)
    to build and O(log n) per query."""

    def __init__(self, root):
        self.ids = {root: 0}
        self.nodes = [root]
        parent, self.depth = [0], [0]
```

```
        # the list grows while we walk it: a BFS order
        for node in self.nodes:
            at = self.ids[node]
            for child in (node.left, node.right):
                if child:
                    self.ids[child] = len(self.nodes)
```

```
                    self.nodes.append(child)
                    parent.append(at)
                    self.depth.append(self.depth[at] + 1)
        self.up = [parent]
        count = len(self.nodes)
        while (1 << len(self.up)) < count:
            prev = self.up[-1]
            row = [prev[prev[i]] for i in range(count)]
            self.up.append(row)

    def query(self, p, q):
        a, b = self.ids[p], self.ids[q]
        if self.depth[a] < self.depth[b]:
            a, b = b, a
        # lift the deeper node to the same depth
        diff = self.depth[a] - self.depth[b]
        for k, row in enumerate(self.up):
            if diff >> k & 1:
                a = row[a]
        if a == b:
            return self.nodes[a]
        # lift both as high as possible while they differ
        for row in reversed(self.up):
            if row[a] != row[b]:
                a, b = row[a], row[b]
        return self.nodes[self.up[0][a]]


# HELPERS for the checks below
class Node:
    def __init__(self, val, parent=None):
        self.val, self.parent = val, parent
        self.left = self.right = None


def attach(parent, val, side):
    # create a child and link it both ways
    child = Node(val, parent)
    setattr(parent, side, child)
    return child


def random_tree(size):
    import random
    root = Node(0)
```

```
    nodes = [root]
    while len(nodes) < size:
        parent = random.choice(nodes)
        side = random.choice(["left", "right"])
        if getattr(parent, side) is None:
            nodes.append(attach(parent, len(nodes), side))
    return nodes


if __name__ == "__main__":
    root = Node(3)
    five = attach(root, 5, "left")
    one = attach(root, 1, "right")
    six = attach(five, 6, "left")
    two = attach(five, 2, "right")
    four = attach(two, 4, "left")
    assert lca_swap(six, four) is five
    assert lca_swap(five, four) is five
    assert lca_swap(six, one) is root
    assert lca_swap(root, root) is root
```

```
    other = Node(99)
    assert lca_swap(six, other) is None
    assert lca_by_depth(six, other) is None
    assert lca_by_depth(four, one) is root
    index = LiftingLCA(root)
    assert index.query(six, four) is five
    assert index.query(root, four) is root
    import random
    for _ in range(100):
        nodes = random_tree(random.randint(1, 12))
        index = LiftingLCA(nodes[0])
        for _ in range(20):
            p = random.choice(nodes)
            q = random.choice(nodes)
            want = brute(p, q)
            assert lca_swap(p, q) is want
            assert lca_by_depth(p, q) is want
            assert index.query(p, q) is want
```

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

MEDIUM | IN APP LC 236 | HackTheRounds | Onsite | 12 of 22

Lowest Common Ancestor

STUDY: TALK | ALL TIME 41.5 | 30 DAYS 0 | PRIORITY 88/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #236 Lowest Common Ancestor of a Binary Tree

LeetCode match: LC 236

QUESTION

Given a binary tree and two nodes p and q, find their lowest common ancestor (LCA): the lowest node that has both p and q as descendants (a node can be a descendant of itself). Follow-up: what if it is an n-ary tree?

EXAMPLES

Example 1	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1`
Output	`3`
Why	`LCA of 5 and 1 is 3.`

Example 2	
Input	`root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4`
Output	`5`
Why	`A node can be its own ancestor.`

CONSTRAINTS

- Number of nodes in tree: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

APPROACH

Recursive DFS; follow-up: n-ary tree

HOW TO THINK ABOUT IT

Ask each subtree 'do you contain p or q?'. The first node where both subtrees report a hit (or the node is p or q itself and a subtree has the other) is the answer. One pass returns the answer upward: a node returns itself if p or q, the child result if only one side found something, and itself if both sides did.

Basis: LeetCode 236, Lowest Common Ancestor of a Binary Tree (and its n-ary follow-up).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def path_to(root, target):
    if root is None:
        return None
    if root is target:
        return [root]
    for child in root.children:
        sub = path_to(child, target)
        if sub:
            return [root] + sub
    return None
def lca(root, p, q):
    path_p = path_to(root, p)
    path_q = path_to(root, q)
    answer = None
    for a, b in zip(path_p, path_q):
        if a is not b:
            break
        answer = a
    return answer
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Two full paths stored, O(n) space, and two traversals. One recursive pass that reports whether each subtree contains p or q finds the answer in O(n) time and O(h) space.

OPTIMAL SOLUTION · One recursive pass

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is n-ary.

The same recursion: collect the non-empty results from all children; if two or more children reported a hit this node is the answer, otherwise pass the single result (or None) up.

FOLLOW-UP 2 A very deep tree, or no recursion allowed.

Do one traversal that records each node's parent (a dict), then use the ancestor-set method from the parent-pointer version. O(n) time and space, no recursion.

FOLLOW-UP 3 p or q might not be in the tree.

The recursive answer is wrong in that case (it returns the one that exists). Count how many of p and q were seen during the DFS, or first check both exist, and return None if not.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Lowest Common Ancestor
#
# Given a binary tree and two nodes p and q, find their
# lowest common ancestor (LCA): the lowest node that has
# both p and q as descendants (a node can be a descendant
# of itself). Follow-up: what if it is an n-ary tree?
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is n-ary.
# 2. A very deep tree, or no recursion allowed.
# 3. p or q might not be in the tree.
#
# INTERVIEW NOTES
# Ask each subtree 'do you contain p or q?'. The first
# node where both subtrees report a hit (or the node is p
# or q itself and a subtree has the other) is the answer.
# One pass returns the answer upward: a node returns
# itself if p or q, the child result if only one side
# found something, and itself if both sides did.
#
# BRUTE FORCE (say it first): Find the root-to-node path
# for each node and compare
# Compute the path from the root to p and to q, then the
# last common node on the two paths is the LCA. Correct
# and easy to explain; it stores two full paths. Time
# O(n). Space O(n).


def brute(root, p, q):
    # oracle: the deepest node whose subtree holds both
    def holds(node, target):
        return node is target or any(
            holds(c, target) for c in node.children)

    best, stack = None, [root]
    while stack:
        node = stack.pop()
        if holds(node, p) and holds(node, q):
            best = node
            stack.extend(node.children)
    return best


# BRUTE FORCE (say it first): Find the root-to-node path
# for each node and compare | time O(n)
```

```
def _path(root, target):
    stack = [(root, [root])]
    while stack:
        n, path = stack.pop()
        if n is target:
            return path
        for c in (n.left, n.right):
            if c:
                stack.append((c, path + [c]))
    return []


# HELPERS for the base question
class BNode:
    def __init__(self, val, left=None, right=None):
        self.val, self.left, self.right = (
            val,
            left,
            right,
        )
```

```
def lca_paths(root, p, q):
    a, b = _path(root, p), _path(root, q)
    lca = None
    for x, y in zip(a, b):
        if x is y:
            lca = x
        else:
            break
    return lca


# BASE QUESTION (optimal): One recursive pass
# Idea: dfs(node) returns node if node is p or q,
# otherwise combines its children: if both children return
# something, node is the LCA; if only one does, pass that
# up.
# FOLLOW-UP 1: The tree is n-ary.
# Answer: The same recursion: collect the non-empty
# results from all children; if two or more children
# reported a hit this node is the answer, otherwise pass
# the single result (or None) up.
def lca(root, p, q):
    # each call reports where p or q was found below. A
```

```
    # node
    # with hits from two different children (or that is p
    # or
    # q itself) is the answer; a single hit is passed up.
    def find(node):
        if node is None:
            return None
        if node is p or node is q:
            return node
        # FOLLOW-UP 1: n-ary: collect hits from every
        # child
        hits = [r for r in map(find, node.children) if r]
        if len(hits) >= 2:
            return node
        return hits[0] if hits else None

    return find(root)


# FOLLOW-UP 2: A very deep tree, or no recursion allowed.
# Answer: Do one traversal that records each node's parent
# (a dict), then use the ancestor-set method from the
# parent-pointer version. O(n) time and space, no
# recursion.
def parents_of(root):
    # follow-up: one traversal with a stack (no recursion)
    # that records every node's parent
    parent, stack = {root: None}, [root]
    while stack:
        node = stack.pop()
        for child in node.children:
            parent[child] = node
            stack.append(child)
    return parent


# FOLLOW-UP 2: A very deep tree, or no recursion allowed.
# Answer: Do one traversal that records each node's parent
# (a dict), then use the ancestor-set method from the
# parent-pointer version. O(n) time and space, no
# recursion.
def lca_iterative(root, p, q):
    # climb from p collecting its ancestors, then climb
    # from
    # q until we meet one of them: O(n) time and space
```

```
    parent = parents_of(root)
    above = set()
    node = p
    while node:
        above.add(node)
        node = parent[node]
    node = q
    while node not in above:
        node = parent[node]
    return node


# FOLLOW-UP 3: p or q might not be in the tree.
# Answer: The recursive answer is wrong in that case (it
# returns the one that exists). Count how many of p and q
# were seen during the DFS, or first check both exist, and
# return None if not.
def lca_checked(root, p, q):
    # follow-up: p or q may be missing from the tree. The
    # plain recursion would return the one that exists, so
    # first check that both are really in the tree.
    parent = parents_of(root)
    if p not in parent or q not in parent:
        return None
    return lca_iterative(root, p, q)


# HELPERS for the checks below
class Node:
    def __init__(self, val, children=None):
        self.val = val
        # two children for a binary tree, any number for
        # n-ary
        self.children = children or []


if __name__ == "__main__":
    six, four = Node(6), Node(4)
    two = Node(2, [Node(7), four])
    five = Node(5, [six, two])
    one = Node(1, [Node(0), Node(8)])
    root = Node(3, [five, one])
    assert lca(root, six, four) is five
    assert lca(root, five, four) is five
    assert lca(root, six, one) is root
```

```
    # an n-ary tree: three children under one node
    kids = [Node(i) for i in range(3)]
    top = Node("top", [Node("mid", kids), Node("side")])
    assert lca(top, kids[0], kids[2]).val == "mid"
    found = lca_iterative(top, kids[0], kids[2])
    assert found.val == "mid"
    stranger = Node(404)
    assert lca_checked(root, six, stranger) is None
    assert lca_checked(root, six, four) is five
    # a very deep tree: the iterative version has no limit
    chain = Node(0)
    cur = chain
    for i in range(1, 5000):
        child = Node(i)
        cur.children.append(child)
        cur = child
    assert lca_iterative(chain, cur, chain) is chain
    import random
    for _ in range(100):
        # a random tree: every new node gets a random
        # parent
        nodes = [Node(0)]
        for i in range(1, random.randint(1, 10)):
            child = Node(i)
            random.choice(nodes).children.append(child)
            nodes.append(child)
        p, q = random.choice(nodes), random.choice(nodes)
        want = brute(nodes[0], p, q)
```

```
        assert lca(nodes[0], p, q) is want
        assert lca_iterative(nodes[0], p, q) is want
```

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

HARD | IN APP LC 1325 | PracHub | Technical Screen | 13 of 22

Repeatedly Delete Leaves Matching a Target Value From a Binary Tree

POSTED 2026-09-16 | 34 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 10.6 30 DAYS 0 PRIORITY 164/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1325 Delete Leaves With a Given Value

LeetCode match: LC 1325 Delete Leaves With a Given Value

Posted on PracHub: 2026-09-16 (updated 2026-09-20)

Original: https://prachub.com/coding-questions/repeatedly-delete-leaves-matching-a-target-value-from-a-binary-tree

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

```
remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]
```

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.
- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

QUESTION

Examples

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

APPROACH

Post-order DFS: process children first, then delete the node if it became a leaf with value == target. O(n).

HOW TO THINK ABOUT IT

'Repeatedly' hides an order: a node becomes deletable only after its children are processed. That is a post-order dependency, so a single bottom-up pass does what many passes would do. Whenever a rule says 'repeat until nothing changes' on a tree, ask whether children-first order collapses it to one pass.

Basis: LeetCode 1325, Delete Leaves With a Given Value.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def delete_pass(node):
    if node is None:
        return None, 0
    node.left, a = delete_pass(node.left)
    node.right, b = delete_pass(node.right)
    if is_leaf(node) and node.val == target:
        return None, a + b + 1
    return node, a + b
while True:
    root, removed = delete_pass(root)
    if removed == 0:
        break
```

TIME	SPACE
O(n * h)	O(h)

WHY IT IS SLOW

A chain of matching nodes needs one pass per node, n passes of O(n). Post-order (children first) deletes the whole chain in a single O(n) pass.

OPTIMAL SOLUTION · One post-order pass

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is a 100,000-node chain. How do you avoid a stack overflow?

Collect nodes in pre-order with an explicit stack and remember each node's parent, then walk that list in reverse (children before parents) and unlink matching leaves. This keeps the post-order guarantee without recursion.

FOLLOW-UP 2 Delete any node whose whole subtree sums to zero, or whose value matches a predicate.

Same shape of solution: the decision for a node depends on information returned from its children. Return (new subtree, subtree sum) from the helper and drop the node when the rule holds. The key is to define what a child reports upward. **Cost:** one post-order pass, O(n) time and O(h) space.

FOLLOW-UP 3 Return how many nodes were deleted.

Count inside the same post-order helper (a nonlocal counter, or return it with the subtree). Never do a second traversal to count. **Cost:** O(1) extra, counted inside the same O(n) pass.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Repeatedly Delete Leaves Matching a
# Target Value From a Binary Tree
#
# Given a binary tree and an integer target, delete every
# leaf whose value equals target. Deleting a leaf can turn
# its parent into a leaf; if that parent's value also
# equals target, it must be deleted as well, and so on,
# until no leaf with value target remains. Return the
# resulting tree, which is empty if the root itself ends
# up deleted. ### Function Signature
# remove_target_leaves(tree: list[int | None], target:
# int) -> list[int | None] ### Tree Encoding The input and
# the output use the same level-order encoding: - An empty
# tree is []. - Otherwise the first element is the root's
# value. Present nodes are then visited in breadth-first
# order, left to right, and for each one its left child
# and then its right child are appended, using None for an
# absent child. Children of absent nodes are not listed. -
# Trailing None values are omitted.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The tree is a 100,000-node chain. How do you avoid a
# stack overflow?
# 2. Delete any node whose whole subtree sums to zero, or
# whose value matches a predicate.
# 3. Return how many nodes were deleted.
#
# INTERVIEW NOTES
# 'Repeatedly' hides an order: a node becomes deletable
# only after its children are processed. That is a post-
# order dependency, so a single bottom-up pass does what
# many passes would do. Whenever a rule says 'repeat until
# nothing changes' on a tree, ask whether children-first
# order collapses it to one pass.
#
# BRUTE FORCE (say it first): Remove matching leaves,
# repeat until a pass changes nothing
# Each pass deletes only the current leaves with the
# target value; a parent that becomes a matching leaf
# waits for the next pass. On a chain of n matching nodes
# that is n passes of O(n) each. Time O(n * h). Space
# O(h).


from collections import deque


# BRUTE FORCE (say it first): Remove matching leaves,
# repeat until a pass changes nothing | time O(n * h)
def remove_leaves_brute(root, target):
    def one_pass(node):
        if node is None:
            return None, False
        if node.left is None and node.right is None:
            return (
                (None, True)
                if node.val == target
                else (node, False)
            )
        node.left, a = one_pass(node.left)
        node.right, b = one_pass(node.right)
        return node, a or b

    changed = True
    while changed:
        root, changed = one_pass(root)
    return root


# BASE QUESTION (optimal): One post-order pass
# Idea: Process both children first, then look at the
# node: if it is now a leaf and its value is the target,
# return None to its parent.
def remove_leaves(root, target):
    # recursive post-order: children first, then the node
    deleted = 0

    def dfs(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = dfs(node.left)
        node.right = dfs(node.right)
        # a leaf with the target value disappears
        if not node.left and not node.right \
                and node.val == target:
            deleted += 1
            return None
        return node

    return dfs(root), deleted


# FOLLOW-UP 1: The tree is a 100,000-node chain. How do
# you avoid a stack overflow?
# Answer: Collect nodes in pre-order with an explicit
# stack and remember each node's parent, then walk that
# list in reverse (children before parents) and unlink
```

```
# matching leaves. This keeps the post-order guarantee
# without recursion.
def remove_leaves_iterative(root, target):
    # no recursion: safe for a 100,000-node chain
    if root is None:
        return None, 0
    parent, order, stack = {root: None}, [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[child] = node
                stack.append(child)
    deleted = 0
    # reversed order: children are handled before parents
    for node in reversed(order):
        if node.left or node.right or node.val != target:
            continue
        deleted += 1
        up = parent[node]
        if up is None:
            return None, deleted
        if up.left is node:
            up.left = None
        else:
            up.right = None
    return root, deleted


# HELPERS for the checks below
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right


def build(values):
```

```
    # level-order list with None for missing children
    if not values:
        return None
    root = TreeNode(values[0])
    queue, i = deque([root]), 1
    while queue and i < len(values):
        node = queue.popleft()
        # next two values are this node's children
        for side in ("left", "right"):
            if i < len(values) and values[i] is not None:
                child = TreeNode(values[i])
                setattr(node, side, child)
                queue.append(child)
            i += 1
    return root


def to_list(root):
    # the same encoding, with trailing Nones removed
    out, queue = [], deque([root])
    while queue:
        node = queue.popleft()
        out.append(node.val if node else None)
        if node:
            queue.append(node.left)
            queue.append(node.right)
    while out and out[-1] is None:
        out.pop()
    return out


def remove_target_leaves(values, target):
    root, _ = remove_leaves(build(values), target)
    return to_list(root)
```

```
if __name__ == "__main__":
    tree = [1, 2, 3, 2, None, 2, 4]
    result = remove_target_leaves(tree, 2)
    assert result == [1, None, 3, None, 4]
    # the whole tree can disappear
    assert remove_target_leaves([1, 1, 1], 1) == []
    assert remove_target_leaves([], 5) == []
    root, count = remove_leaves(build(tree), 2)
    assert count == 3
```

```
    again = remove_leaves_iterative(build(tree), 2)
    assert again[1] == 3
    assert to_list(again[0]) == to_list(root)
    # a 100,000 node chain works without recursion
    chain = TreeNode(7)
    cur = chain
    for _ in range(99999):
        cur.left = TreeNode(7)
        cur = cur.left
    gone = remove_leaves_iterative(chain, 7)
    assert gone == (None, 100000)
```

FUNCTION AND TEST CASES

Signature

```
def remove_target_leaves(tree: list[int | None], target: i
nt) -> list[int | None]:
```

Sample call

```
remove_target_leaves([5, 7, 7, 7, 3, None, 7], 7)
```

Test cases

Input	Expected
`([5, 7, 7, 7, 3, None, 7], 7)`	`[5, 7, None, None, 3]`
`([3, 3, 1], 3)`	`[3, None, 1]`

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

Site tags: tree, heap, greedy, hard

HARD | 1Point3Acres | onsite-coding | 14 of 22

Huffman Tree Construction

ASKED 2026-06-01 | 141 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 189/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/huffman-tree-construction

QUESTION

EXAMPLES

```
{"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45}
    -> standard Huffman tree with f near the root.
```

APPROACH

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether to return the tree, a code table, or the encoded message, how ties are broken, and what to do with one symbol.

BRUTE FORCE TO SAY OUT LOUD

Repeatedly scan for the two smallest weights. O(n^2).

OPTIMAL IDEA

Min-heap of nodes keyed by (weight, counter). Pop two, merge under a new node with the summed weight, push back, until one node remains. Walk the tree assigning '0' to left and '1' to right. O(n log n). Decode by walking the tree bit by bit. The counter prevents comparing nodes on ties.

DRY-RUN THESE CASES

The CLRS example (total cost 224, 'f' gets a one-bit code). One symbol (code '0'). Equal weights.

LIKELY FOLLOW-UPS

Canonical codes, decoding, or the two-queue O(n) method on sorted input.

PITFALLS

Comparing tree nodes on equal weights. Forgetting the single symbol case.

HOW TO THINK ABOUT IT

The two rarest symbols belong at the deepest level as siblings, so merge them first, treat the merged node as a new symbol with the combined weight, and repeat. 'Take the two smallest' is the job of a min-heap; tie-breaking with a counter keeps the heap from comparing tree nodes.

Basis: Classic greedy algorithm (CLRS 16.3); related to LeetCode 1167, Minimum Cost to Connect Sticks.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]  # two smallest
        items = items[2:] + [a + b]
        cost += a + b
    return cost
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Finding the two smallest by scanning costs O(n) per merge and n - 1 merges, O(n^2). A min-heap does each merge in O(log n).

OPTIMAL SOLUTION · Min-heap merge of the two lightest nodes

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the greedy choice optimal?

Exchange argument: in some optimal tree the two least frequent symbols are siblings at the deepest level (swapping a shallower, more frequent symbol with a deeper, less frequent one never increases cost). Merging them into one symbol reduces the problem to n - 1 symbols with the same optimal-substructure property, so induction closes it.

FOLLOW-UP 2 How do you make the code canonical, and why would you?

Keep only each symbol's code length from the tree, sort symbols by (length, symbol), and assign consecutive binary codes. A decoder then needs just the lengths rather than the whole tree, which is how DEFLATE stores its tables.

FOLLOW-UP 3 Time complexity if symbols arrive already sorted?

Two-queue method: one queue holds the sorted leaves, the other the merged nodes (which are produced in nondecreasing order). Always pop the smaller front of the two queues. That is O(n) after sorting.

FOLLOW-UP 4 Make the codes canonical so only the code lengths need to be stored.

Sort symbols by (code length, symbol) and assign consecutive binary numbers, shifting left when the length grows. A decoder rebuilds the codes from the lengths alone (this is how DEFLATE stores its tables).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Huffman Tree Construction
#
# Examples: {"a": 5, "b": 9, "c": 12, "d": 13, "e": 16,
# "f": 45} -> standard Huffman tree with f near the root.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the greedy choice optimal?
# 2. How do you make the code canonical, and why would
# you?
# 3. Time complexity if symbols arrive already sorted?
# 4. Make the codes canonical so only the code lengths
# need to be stored.
#
# INTERVIEW NOTES
# The two rarest symbols belong at the deepest level as
# siblings, so merge them first, treat the merged node as
# a new symbol with the combined weight, and repeat. 'Take
# the two smallest' is the job of a min-heap; tie-breaking
# with a counter keeps the heap from comparing tree nodes.
#
# BRUTE FORCE (say it first): Scan for the two smallest
# weights on every merge
# Keep a list of (weight, tree) items; each round scan for
# the two smallest, merge, and put the result back. The
# same greedy as the heap version, with a linear scan
# instead of a heap operation. Returns the total cost (sum
# of weight * code length). Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Scan for the two smallest
# weights on every merge | time O(n^2)
def huffman_cost_scan(weights):
    items = list(weights)
    cost = 0
    while len(items) > 1:
        items.sort()
        a, b = items[0], items[1]
        cost += a + b
        items = items[2:] + [a + b]
    return cost


# HELPERS for the base question
def huffman_lengths(freq):
    # code length of every symbol (depth in the tree)
    if not freq:
        return {}
    if len(freq) == 1:
        # a lone symbol still needs one bit
        return {next(iter(freq)): 1}
    # heap entries: (weight, tie-break, symbols below)
    heap = [(w, i, [s]) for i, (s, w) in
            enumerate(sorted(freq.items()))]
    heapq.heapify(heap)
    depth = {s: 0 for s in freq}
    tie = len(heap)
    while len(heap) > 1:
        # merge the two lightest; everything below goes
        # deeper
        w1, _, a = heapq.heappop(heap)
        w2, _, b = heapq.heappop(heap)
        for s in a + b:
            depth[s] += 1
        heapq.heappush(heap, (w1 + w2, tie, a + b))
        tie += 1
    return depth


# FOLLOW-UP 2: How do you make the code canonical, and why
# would you?
# Answer: Keep only each symbol's code length from the
# tree, sort symbols by (length, symbol), and assign
# consecutive binary codes. A decoder then needs just the
# lengths rather than the whole tree, which is how DEFLATE
# stores its tables.
# FOLLOW-UP 4: Make the codes canonical so only the code
# lengths need to be stored.
# Answer: Sort symbols by (code length, symbol) and assign
# consecutive binary numbers, shifting left when the
# length grows. A decoder rebuilds the codes from the
# lengths alone (this is how DEFLATE stores its tables).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def canonical_codes(lengths):
    # assign consecutive binary numbers by (length,
    # symbol)
    code, prev, out = 0, 0, {}
    for sym, length in sorted(lengths.items(),
                              key=lambda x: (x[1], x[0])):
        # moving to a longer length shifts the code left
        code <<= length - prev
        out[sym] = format(code, f"0{length}b")
        code += 1
        prev = length
    return out


# BASE QUESTION (optimal): Min-heap merge of the two
# lightest nodes
# Idea: Put every symbol in a min-heap keyed by frequency,
# repeatedly pop the two lightest nodes, merge them under
# a new internal node with the summed weight and push it
# back. The last node is the root.
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))


# FOLLOW-UP 3: Time complexity if symbols arrive already
# sorted?
# Answer: Two-queue method: one queue holds the sorted
# leaves, the other the merged nodes (which are produced
# in nondecreasing order). Always pop the smaller front of
# the two queues. That is O(n) after sorting.
def lengths_two_queues(sorted_items):
    # input already sorted by weight: O(n) with two queues
```

```
    from collections import deque
    leaves = deque((w, [s]) for s, w in sorted_items)
    merged = deque()
    depth = {s: 0 for s, _ in sorted_items}

    def pop_smaller():
        # the smaller front of the two queues
        if not merged or (leaves and
                          leaves[0][0] <= merged[0][0]):
            return leaves.popleft()
        return merged.popleft()

    while len(leaves) + len(merged) > 1:
        w1, a = pop_smaller()
        w2, b = pop_smaller()
        for s in a + b:
            depth[s] += 1
```

```
        merged.append((w1 + w2, a + b))
    return depth


# HELPERS for the checks below
def decode(bits, codes):
    # read bits until they match a code (prefix-free)
    back = {v: k for k, v in codes.items()}
    out, cur = [], ""
    for b in bits:
        cur += b
        if cur in back:
            out.append(back[cur])
            cur = ""
    return "".join(out)


if __name__ == "__main__":
    freq = {"a": 5, "b": 9, "c": 12,
            "d": 13, "e": 16, "f": 45}
    lengths = huffman_lengths(freq)
    # the most frequent symbol is next to the root
    assert lengths["f"] == 1
    # total cost of the optimal tree
    assert sum(freq[s] * lengths[s] for s in freq) == 224
    codes = huffman_codes(freq)
    # prefix-free: no code starts another code
    for x in codes.values():
        for y in codes.values():
            assert x == y or not y.startswith(x)
    text = "abfadc"
    bits = "".join(codes[ch] for ch in text)
    assert decode(bits, codes) == text
    items = sorted(freq.items(), key=lambda x: x[1])
    two = lengths_two_queues(items)
    assert sum(freq[s] * two[s] for s in freq) == 224
    assert huffman_codes({"x": 3}) == {"x": "0"}
    assert huffman_codes({}) == {}
```

NOTES FROM CANDIDATES

L6 onsite — interviewer expects clean code with a documented tree node, plus a discussion of encoding / decoding correctness.
Common follow-up: how to produce the code table (DFS from root, accumulating bits).
Trade-off discussion: Huffman is optimal for known per-symbol probability but not for adaptive / streaming inputs.

PREPARATION

Write the heap-based Huffman build from memory in under 15 min.
Drill the encode + decode helpers (DFS to build the code map; tree walk to decode).
Be ready to compare against arithmetic coding (better compression, costlier).

STUDY HERE: topic 3, Trees and Tries | main topic: Trees

HARD | IN APP LC 834 | PracHub | Technical Screen | 15 of 22

Compute distance-sum from every tree node

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 123/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Trees

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #834 Sum of Distances in Tree

LeetCode match: LC 834 Sum of Distances in Tree

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/compute-distance-sum-from-every-tree-node

QUESTION

Overview: This question evaluates understanding of tree and graph algorithms, including computation of pairwise distances and techniques for aggregating path lengths across nodes.

APPROACH

Rerooting DP: one DFS computes subtree sizes and the root's distance sum, a second DFS derives each child as ans[child] = ans[parent] - size[child] + (n - size[child]). O(n).

HOW TO THINK ABOUT IT

Computing the sum of distances from each node separately costs O(n) per node. Instead compute it once for the root, then derive the neighbour's answer in O(1): moving the root from a parent to its child makes every node in the child's subtree one step closer and every other node one step farther. That needs only the subtree sizes.

Basis: LeetCode 834, Sum of Distances in Tree (re-rooting dynamic programming).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, adj):
    ans = []
    for node in range(n):
        dist = bfs(adj, node)
        ans.append(sum(dist))
    return ans
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

BRUTE FORCE

n searches of O(n) each, O(n^2). Compute the answer for the root, then move the root edge by edge using subtree sizes (re-rooting), O(n).

OPTIMAL SOLUTION · Re-rooting with subtree sizes

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Find the node with the smallest distance sum (the tree's centroid).

The answer array above gives it directly: argmin of ans. Equivalent: the centroid, found by walking toward the heavier subtree.

FOLLOW-UP 2 The tree has weighted edges.

Re-rooting still works, but the change when moving the root across an edge of weight w is w * (n - 2 * size[c]) instead of n - 2 * size[c]. Sizes and the first pass are the same.

FOLLOW-UP 3 Maximum distance (eccentricity) from each node instead of the sum.

The farthest node from any node is one end of a diameter; compute distances from both diameter ends and take the maximum per node. O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute distance-sum from every tree node
#
# This question evaluates understanding of tree and graph
# algorithms, including computation of pairwise distances
# and techniques for aggregating path lengths across
# nodes.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Find the node with the smallest distance sum (the
# tree's centroid).
# 2. The tree has weighted edges.
# 3. Maximum distance (eccentricity) from each node
# instead of the sum.
#
# INTERVIEW NOTES
# Computing the sum of distances from each node separately
# costs O(n) per node. Instead compute it once for the
# root, then derive the neighbour's answer in O(1): moving
# the root from a parent to its child makes every node in
# the child's subtree one step closer and every other node
# one step farther. That needs only the subtree sizes.
#
```

```
# BRUTE FORCE (say it first): BFS from every node
# For each node run a BFS and add up the distances.
# Correct and the oracle; n searches of O(n) each. Time
# O(n^2). Space O(n).


from collections import defaultdict
from collections import deque


# BRUTE FORCE (say it first): BFS from every node | time
# O(n^2)
def distance_sums_brute(n, edges):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    out = []
    for s in range(n):
        dist, queue = {s: 0}, deque([s])
        while queue:
            u = queue.popleft()
            for v in adj[u]:
```

```
                if v not in dist:
                    dist[v] = dist[u] + 1
                    queue.append(v)
        out.append(sum(dist.values()))
    return out


# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for e in edges:
        u, v = e[0], e[1]
        w = e[2] if len(e) > 2 else 1
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


def distances_from(adj, s):
    # distance from s to every node (a tree: one pass)
    dist, stack = {s: 0}, [s]
    while stack:
        u = stack.pop()
        for v, w in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + w
                stack.append(v)
    return dist


# BASE QUESTION (optimal): Re-rooting with subtree sizes
# Idea: Root the tree at 0 and compute size[v] (subtree
# sizes) and ans[0] = the sum of depths.
def distance_sums(n, edges):
    # sum of distances from every node; edges may be
    # weighted
    adj = build(n, edges)
    parent, order, up = [-1] * n, [0], [0] * n
    for u in order:
        for v, w in adj[u]:
            if v != parent[u]:
                parent[v], up[v] = u, w
                order.append(v)
    size = [1] * n
    for u in reversed(order):
```

```
        if parent[u] >= 0:
            size[parent[u]] += size[u]
    # the root's sum is a plain sum of distances
    ans = [0] * n
    ans[0] = sum(distances_from(adj, 0).values())
    # root moves across an edge of weight w: size[u] nodes
    # get
    # w closer, the other n - size[u] get w farther
    for u in order[1:]:
        shift = up[u] * (n - 2 * size[u])
        ans[u] = ans[parent[u]] + shift
    return ans


# FOLLOW-UP 1: Find the node with the smallest distance
# sum (the tree's centroid).
# Answer: The answer array above gives it directly: argmin
# of ans. Equivalent: the centroid, found by walking
# toward the heavier subtree.
def best_center(n, edges):
    # the node with the smallest distance sum
    sums = distance_sums(n, edges)
    return sums.index(min(sums))


# FOLLOW-UP 3: Maximum distance (eccentricity) from each
# node instead of the sum.
# Answer: The farthest node from any node is one end of a
# diameter; compute distances from both diameter ends and
# take the maximum per node. O(n).
def eccentricities(n, edges):
    # the farthest node from anywhere is an end of a
    # diameter
    adj = build(n, edges)
    d0 = distances_from(adj, 0)
    a = max(d0, key=d0.get)
    da = distances_from(adj, a)
    b = max(da, key=da.get)
    db = distances_from(adj, b)
    return [max(da[v], db[v]) for v in range(n)]


if __name__ == "__main__":
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    want = [8, 12, 6, 10, 10, 10]
```

```
    assert distance_sums(6, edges) == want
    assert best_center(6, edges) == 2
    assert distance_sums(1, []) == [0]
    # weighted: the path 0 -(2)- 1 -(3)- 2
    w = [[0, 1, 2], [1, 2, 3]]
    assert distance_sums(3, w) == [7, 5, 8]
    # eccentricity: the farthest node from each node
    assert eccentricities(6, edges) == [2, 3, 2, 3, 3, 3]
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, edges: list[list[int]]) -> list[int]:
```

Sample call

```
solution(1, [])
```

Test cases

Input	Expected
(1, [])	[0]
(4, [[0, 1], [1, 2], [1, 3]])	[5, 3, 5, 5]

ALSO REPORTED · 1 more

Tree Distance Sum (Re-root DP) | 1Point3Acres | onsite-coding | asked 2026-01-19 | Single report

https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

The interviewer explicitly liked the "baseline first, optimize later" structure — name the $O(n^2)$ and move on within the first ~10 minutes.

Two-pass technique to memorize:

Post-order DFS rooted at any node: compute subSize[u] and subSum[u] (sum of distances inside u's subtree). Recurrence: subSum[u] += subSum[v] + subSize[v].

Pre-order DFS doing re-root: when moving root from parent p to child v, ans[v] = ans[p] + (n - subSize[v]) - subSize[v].

Verbalize the geometric intuition for the re-root formula: moving root toward v shortens distance to the subSize[v] nodes inside v's subtree by 1 and lengthens the other n - subSize[v] by 1.

Preparation: Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.

Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.

Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

STUDY HERE: topic 3, Trees and Tries | main topic: Tries

Site tags: string, parsing, data-structure, trie, dfs

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen, onsite-coding | 16 of 22

URL Request Handler Router

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 222/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/url-request-handler-router

QUESTION

EXAMPLES

```
Register /users/{userId}/photo/{photoId}. A lookup
    for /users/123/photo/456 matches that registered
    handler.
```

APPROACH

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how to choose between several matching patterns (literal over placeholder?), what to return on no match or duplicate registration, and how to treat trailing slashes, query strings and case.

BRUTE FORCE TO SAY OUT LOUD

Store the patterns in a list and compare segment by segment. O(H * L) per lookup. It is a correct baseline and gives a reference to test the trie against.

OPTIMAL IDEA

Segment trie: each node has literal children and one wildcard child, with the original pattern stored at the end node. Lookup explores the literal child first and the wildcard child on failure, because a literal branch can die later. Accept only when all segments are consumed and a handler is stored.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

An exact route, a placeholder route, both literal and placeholder matching the same URL, segment-count mismatch, no match.

LIKELY FOLLOW-UPS

Methods (GET and POST), wildcard at the end, priority rules, and many routes.

PITFALLS

Picking the literal branch greedily without falling back. Overwriting patterns that differ only in placeholder names.

HOW TO THINK ABOUT IT

Matching a path against many patterns is a prefix problem, so share the prefixes in a trie of segments. The catch is the wildcard: a literal branch can fail later, so you cannot pick it greedily. Try the literal child first and fall back to the wildcard child. That also encodes the precedence rule for free.

Basis: Written. Assumed rules (confirm them): patterns look like /users/{id}/photo; {name} matches exactly one segment; a literal beats a placeholder at the first segment where they differ; no match returns None; trailing slashes are ignored.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(url):
    parts = url.split("/")
    matches = []
    for pattern in patterns:
        p = pattern.split("/")
        if len(p) != len(parts):
            continue
        if all(is_param(x) or x == y
               for x, y in zip(p, parts)):
            matches.append(pattern)
    return best_by_precedence(matches)
```

TIME	SPACE
O(H * L) per lookup	**O(1)** extra

WHY IT IS SLOW

Each lookup compares against every registered pattern, O(patterns * segments). A trie of segments with a literal child and a wildcard child follows only the compatible branches.

OPTIMAL SOLUTION · Segment trie with literal-first fallback

TIME	SPACE
O(L * branching) per lookup, O(total pattern length) to build	**O(total pattern length)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Different HTTP methods per route.

Make the method the first segment of the path (a tiny trick) or keep a dict method to trie. Lookup uses the method as the first step.

FOLLOW-UP 2 A wildcard at the end that matches the rest of the path (/static/*).

Give each trie node a 'rest' handler. During lookup, remember the deepest node that has one and use it if the exact walk fails. Precedence: exact, then placeholder, then rest-wildcard.

FOLLOW-UP 3 Return the extracted parameters (id = 7).

Carry a list of (name, segment) while walking; append when you take a wildcard edge and drop it when you backtrack. Store the placeholder names with the pattern, not in the trie.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: URL Request Handler Router
#
# Examples: Register /users/{userId}/photo/{photoId}. A
# lookup for /users/123/photo/456 matches that registered
# handler.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Different HTTP methods per route.
# 2. A wildcard at the end that matches the rest of the
# path (/static/*).
# 3. Return the extracted parameters (id = 7).
#
# INTERVIEW NOTES
# Matching a path against many patterns is a prefix
# problem, so share the prefixes in a trie of segments.
# The catch is the wildcard: a literal branch can fail
# later, so you cannot pick it greedily. Try the literal
# child first and fall back to the wildcard child. That
# also encodes the precedence rule for free.
#
# BRUTE FORCE (say it first): Scan every registered
# pattern
# Compare the URL's segments with each pattern's segments
# (same count, each one equal or a placeholder), collect
# the matches and keep the best by specificity. It is the
# correctness baseline, and a good oracle for the trie.
# Time O(H * L) per lookup. Space O(1) extra.


# HELPERS for the brute force
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    return [s for s in path.split("/") if s]


# BRUTE FORCE (say it first): try every route in a list
def fits(pat, segs):
    # the placeholder values if the route fits, else None
    params = {}
    for i, p in enumerate(pat):
        if p == "*":
            return params if len(segs) > i else None
        if i >= len(segs):
            return None
        if p.startswith("{") and p.endswith("}"):
            params[p[1:-1]] = segs[i]
        elif p != segs[i]:
            return None
    return params if len(pat) == len(segs) else None


class RouterBrute:
    def __init__(self):
        self.routes = []

    def add(self, pattern, handler=None, method="GET"):
        pat = [method] + segments(pattern)
        self.routes.append((pat, handler or pattern))

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        best = None
        for pat, handler in self.routes:
            params = fits(pat, segs)
            if params is None:
                continue
            # literal beats placeholder beats trailing *
            score = [2 if p == s else 1 if p[0] == "{"
                     else 0 for p, s in zip(pat, segs)]
            if best is None or score >= best[0]:
                best = (score, handler, params)
        return (best[1], best[2]) if best else (None, {})


# HELPERS for the base question
class Node:
    def __init__(self):
        # fixed segment -> child node
        self.literal = {}
        # child for a {placeholder} segment
        self.wild = None
        self.wild_name = None
        # handler when the path ends here
        self.handler = None
        # handler for a trailing "*" (rest of the path)
        self.rest = None


# BASE QUESTION (optimal): Segment trie with literal-first
# fallback
# Idea: Each trie node has literal children and one
# wildcard child; the pattern is stored at its last node.
# Lookup tries the literal child first, and if that whole
# branch fails it backs up and tries the wildcard child.
# FOLLOW-UP 1: Different HTTP methods per route.
# Answer: Make the method the first segment of the path (a
# tiny trick) or keep a dict method to trie. Lookup uses
# the method as the first step.
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern, handler=None, method="GET"):
        node = self.root
        # FOLLOW-UP 1: the HTTP method is the first
        # segment
        # the method is simply the first segment
        for seg in [method] + segments(pattern):
            # FOLLOW-UP 2: a trailing * matches the rest
            # of the path
            if seg == "*":
                # all deeper paths use this handler
                node.rest = handler or pattern
```

```
                return
            if seg.startswith("{") and seg.endswith("}"):
                node.wild = node.wild or Node()
                node.wild_name = seg[1:-1]
                node = node.wild
            else:
                lit = node.literal
                node = lit.setdefault(seg, Node())
        node.handler = handler or pattern

    def match(self, url, method="GET"):
        segs = [method] + segments(url)
        params = {}

        def walk(node, i):
            if i == len(segs):
                return node.handler
            # 1) exact segment first
            child = node.literal.get(segs[i])
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
```

```
                    return found
            # 2) then a placeholder, remembering its value
            if node.wild is not None:
                # FOLLOW-UP 3: remember the value (id = 7)
                params[node.wild_name] = segs[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                # undo the value when backtracking
                params.pop(node.wild_name, None)
            # 3) finally a trailing wildcard
            return node.rest

        handler = walk(self.root, 0)
        return handler, dict(params)


if __name__ == "__main__":
    r = Router()
    r.add("/users/{userId}/photo/{photoId}", "photo")
    r.add("/users/me", "me")
    r.add("/static/*", "static")
    r.add("/orders/{id}", "create", method="POST")
    hit = r.match("/users/123/photo/456")
    ids = {"userId": "123", "photoId": "456"}
    assert hit == ("photo", ids)
    # a literal beats a placeholder
    assert r.match("/users/me")[0] == "me"
    assert r.match("/users/me/photo/9")[0] == "photo"
    # trailing wildcard takes the rest
    assert r.match("/static/css/app.css")[0] == "static"
    # different method, different route
    post = r.match("/orders/5", "POST")
    assert post == ("create", {"id": "5"})
    assert r.match("/orders/5")[0] is None
    # the brute force agrees with the trie
    import random
    words = ["a", "b", "{}", "*"]
    for _ in range(300):
        fast, slow = Router(), RouterBrute()
        for k in range(random.randint(1, 5)):
            n = random.randint(1, 3)
            parts = [random.choice(words)
                     for _ in range(n)]
            if "*" in parts:
```

```
                parts = parts[:parts.index("*") + 1]
            # a placeholder is named after its position
            parts = ["{p%d}" % i if p == "{}" else p
                     for i, p in enumerate(parts)]
```

```
            pattern = "/" + "/".join(parts)
            fast.add(pattern, f"h{k}")
            slow.add(pattern, f"h{k}")
        for _ in range(8):
            n = random.randint(1, 3)
            url = "/" + "/".join(random.choice("abc")
                                 for _ in range(n))
            assert fast.match(url) == slow.match(url), url
```

NOTES FROM CANDIDATES

Clarify how to choose between multiple matching handlers, including a literal segment competing with a placeholder. The precedence rule is unspecified.
The no-match result, duplicate-registration behavior, and URL normalization rules are also unspecified; do not assume a canonical behavior before discussing them.
Matching approach
Start with a scan of registered patterns: compare segment counts, then require each position to contain either the same literal or a placeholder. This is a correctness baseline that can collect all matches before applying the agreed precedence rule.
A segment trie shares prefixes. Keep literal children and a separate wildcard child; at each URL segment explore both compatible branches. A literal branch can fail later, so choosing it greedily can miss a valid wildcard match.
Store the original registered patterns at terminal nodes. Patterns with different placeholder names can share the same trie path; retain each original rather than overwriting it. Accept terminals only after consuming the entire URL.
For tokenized paths of L segments and H handlers, the scan costs O(HL) segment comparisons in the worst case. A wildcard trie can branch, so lookup is not necessarily O(L): account for visited states, matching candidates, and string-processing costs separately. Keep normalization

and winner selection tied to the clarified contract.

PREPARATION

In 15 minutes, implement the scan baseline and check the existing example, segment-count mismatches, and no-match cases under an explicitly chosen return convention.
In 25 minutes, build the segment trie and compare its complete match set with the scan on small generated inputs. Include a literal branch that fails later and patterns with different placeholder names but the same structure.
In 10 minutes, write a contract checklist for precedence, duplicate registration, trailing slashes, query strings, and URL decoding; separate agreed rules from assumptions in your tests.

STUDY HERE: topic 3, Trees and Tries | main topic: Tries

Site tags: trie, tree, dfs, medium

MEDIUM | IN APP LC 208 | 1Point3Acres | tech-screen, onsite-coding | 17 of 22

Trie Prefix Search / Autocomplete

ASKED 2026-07-02 | 110 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 36** **30 DAYS 0** **PRIORITY 108/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: your app's interview approach (LeetCode #208)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Frequency: Medium | Last asked: 2026-07-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

QUESTION

LeetCode 208 + 1268. Build a trie from a dictionary and return all words sharing a query prefix. Standard Google onsite coding.

Build a trie from a dictionary / array of words.
Support searchPrefix(prefix) -> list[str]: return all words that start with the query prefix, lexicographically sorted unless ranking is specified.
Some interviewers phrase the query as "a few letters" or "substring"; clarify whether they truly mean arbitrary substring search or the standard prefix/autocomplete contract before coding.
Common follow-ups: deletion, fuzzy match (one-character edit), autocompletion ranking by frequency, and returning only the best K completions.

EXAMPLES

```
Dict ["apple","app","apricot","banana"], prefix "ap"
    -> ["app","apple","apricot"].
```

APPROACH

Write the trie class from memory in under 12 min.
Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

HOW TO THINK ABOUT IT

A trie shares prefixes, so walking down the query prefix costs O(length of the prefix) no matter how many words exist. Everything under that node is a completion. For autocomplete you either collect the words below the prefix node with a DFS (visiting children in sorted order gives sorted output) or store the best few words at every node while inserting.

Basis: LeetCode 208, Implement Trie, plus LeetCode 1268, Search Suggestions System (top completions per prefix).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def search_prefix(words, prefix):
    found = []
    for w in words:
        if w.startswith(prefix):
            found.append(w)
    return sorted(found)
```

TIME	SPACE
O(n * L + n log n) per query	**O(output)**

WHY IT IS SLOW

Every query reads the whole dictionary. A trie reaches the prefix in O(length of the prefix) and reads only the completions.

OPTIMAL SOLUTION · Trie with a DFS over sorted children

TIME	SPACE
O(L + output) per query after O(total characters) to build	**O(total characters)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the best K completions ranked by frequency.

Store at every node the K best (frequency, word) pairs and update them while inserting; a query then reads one node in O(L + K) and never walks the subtree. The price is more memory and slower inserts, which is the right trade when reads dominate (autocomplete).

FOLLOW-UP 2 Allow one wrong character (fuzzy match).

Walk the trie with a budget of one edit: at each level follow the matching child with the full budget and every other child with the budget reduced to 0. The work grows with the alphabet size times the prefix length, not with the dictionary size.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Trie Prefix Search / Autocomplete
#
# LeetCode 208 + 1268. Build a trie from a dictionary and
# return all words sharing a query prefix. Standard Google
# onsite coding. Build a trie from a dictionary / array of
# words. Support searchPrefix(prefix) -> list[str]: return
# all words that start with the query prefix,
```

```
# lexicographically sorted unless ranking is specified.
# Some interviewers phrase the query as "a few letters" or
# "substring"; clarify whether they truly mean arbitrary
# substring search or the standard prefix/autocomplete
# contract before coding. Common follow-ups: deletion,
# fuzzy match (one-character edit), autocompletion ranking
# by frequency, and returning only the best K completions.
# Examples: Dict ["apple","app","apricot","banana"],
# prefix "ap" -> ["app","apple","apricot"].
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the best K completions ranked by
# frequency.
# 2. Allow one wrong character (fuzzy match).
#
# INTERVIEW NOTES
# A trie shares prefixes, so walking down the query prefix
# costs O(length of the prefix) no matter how many words
# exist. Everything under that node is a completion. For
# autocomplete you either collect the words below the
# prefix node with a DFS (visiting children in sorted
# order gives sorted output) or store the best few words
# at every node while inserting.
#
# BRUTE FORCE (say it first): Filter the whole word list
# Scan every word, keep those starting with the prefix and
# sort them. Correct and the oracle; every query pays for
# the whole dictionary. Time O(n * L + n log n) per query.
# Space O(output).


# BRUTE FORCE (say it first): Filter the whole word list |
# time O(n * L + n log n) per query
def prefix_words_brute(words, prefix):
    return sorted(
        {w for w in words if w.startswith(prefix)}
    )
```

```
# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # how often the word ending here was inserted
        self.freq = 0
        # best (negative freq, word) pairs under this node
        self.top = []


# BASE QUESTION (optimal): Trie with a DFS over sorted
# children
# Idea: Insert each word, marking the last node. For a
# query walk the prefix; if a step is missing return an
# empty list.
class Trie:
    def __init__(self, k=3):
        self.root, self.k = Node(), k

    def insert(self, word, freq=1):
        node = self.root
        path = [node]
        for ch in word:
            node = node.children.setdefault(ch, Node())
            path.append(node)
        node.freq += freq
        # refresh the top-k list on every node of the path
        for n in path:
            n.top = [x for x in n.top if x[1] != word]
            n.top.append((-node.freq, word))
            n.top.sort()
            del n.top[self.k:]

    def find(self, prefix):
        # walk down the prefix; None if it is missing
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search_prefix(self, prefix):
        # every word with this prefix, in alphabetical
        # order
        node = self.find(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack:
            n, text = stack.pop()
            if n.freq:
                out.append(text)
            for ch in n.children:
                stack.append((n.children[ch], text + ch))
        return sorted(out)

    def suggest(self, prefix):
        # best k completions, read from one node: O(L + k)
        node = self.find(prefix)
        return [w for _, w in node.top] if node else []

    def fuzzy(self, prefix):
        # words whose first letters differ in at most one
        # place
        out, stack = [], [(self.root, "", 0)]
        while stack:
            n, text, wrong = stack.pop()
            if len(text) == len(prefix):
                out += self.search_below(n, text)
                continue
            for ch, child in n.children.items():
                miss = wrong + (ch != prefix[len(text)])
                if miss <= 1:
                    stack.append((child, text + ch, miss))
        return sorted(out)

    def search_below(self, node, text):
        out, stack = [], [(node, text)]
        while stack:
            n, t = stack.pop()
            if n.freq:
                out.append(t)
            for ch, child in n.children.items():
                stack.append((child, t + ch))
        return out


if __name__ == "__main__":
    t = Trie(k=2)
    for w in ["apple", "app", "apricot", "banana"]:
        t.insert(w)
    ap = t.search_prefix("ap")
    assert ap == ["app", "apple", "apricot"]
    assert t.search_prefix("x") == []
    assert t.search_prefix("") == [
        "app", "apple", "apricot", "banana"]
    # frequency ranking: app was typed three more times
    t.insert("app", 3)
    assert t.suggest("ap") == ["app", "apple"]
    assert t.suggest("zzz") == []
    # one wrong letter in the prefix is allowed
    assert "banana" in t.fuzzy("vana")
    assert t.fuzzy("zzzz") == []
```

NOTES FROM CANDIDATES

Standard implementation: each trie node has a children map and an is_end flag.
Two collection patterns at the prefix: (a) DFS from the prefix node and collect all word-end nodes, (b) store the top-K candidates at every node during insert (Google-preferred for autocomplete).
The standard sorted-suggestions follow-up wants 3 suggestions per prefix-length — handle with sorted children or per-node sorted lists.

PREPARATION

Write the trie class from memory in under 12 min.
Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY HERE: topic 3, Trees and Tries | main topic: Tries

Site tags: string, trie, hashmap, greedy, medium

MEDIUM | 1Point3Acres | phone-screen | 18 of 22

Greedy Longest-Match Dictionary Tokenizer

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 215/228**

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.
Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.
Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

APPROACH

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

HOW TO THINK ABOUT IT

At each position you want the longest dictionary key that is a prefix of the remaining text. Putting the keys in a trie lets you walk the text character by character from that position and remember the deepest end-of-key node you passed, instead of trying every length. If the tie-break rule changes (priority instead of length), store the priority at the terminal node and keep the best terminal seen.

Basis: Written for this question (longest-prefix matching with a trie, as in LeetCode 208 and the maximal-munch rule of lexers).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tokenize(text):
    out, i = [], 0
    while i < len(text):
        for size in range(max_key_len, 0, -1):
            key = text[i:i + size]
            if key in dictionary:
                out.append(dictionary[key])
                i += size
                break
        else:
            out.append(text[i])  # literal
            i += 1
    return out
```

TIME	SPACE
O(n * L) lookups, each O(L)	**O(dictionary)**

WHY IT IS SLOW

Every lookup hashes a substring, O(L^2) per position. Walking a trie of the keys one character at a time costs O(L) per position.

OPTIMAL SOLUTION · Trie walk keeping the deepest (or best-priority) terminal

TIME	SPACE
O(n * L)	**O(total key length)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The dictionary is huge.

A flat map of every key wastes memory because keys share prefixes. A compressed (radix) trie stores each shared stretch once, or shard the keys by first characters across machines. Lookup stays one pass over the text; only the node layout changes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Greedy Longest-Match Dictionary Tokenizer
#
# Phone-screen coding round (R1). Tokenize a text against
# a dictionary of key:id entries using greedy longest-
# match; emit the matched id when a key matches, otherwise
# output the unmatched character as a literal. One follow-
# up on changing the tie-breaking / priority rule for
# which token wins. A second follow-up appears when the
# dictionary is very large: discuss how to keep the
# matcher memory- and lookup-efficient at scale. Input: a
# text string and a dictionary mapping key (string) -> id.
# Sweep the text left-to-right. At each position, find the
# longest dictionary key that matches a prefix of the
# remaining text; emit its id. If no key matches at the
# current position, emit the single character as a literal
# and advance by one. Output the concatenation (or list)
# of ids and literal characters in order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The dictionary is huge.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary key
# that is a prefix of the remaining text. Putting the keys
# in a trie lets you walk the text character by character
# from that position and remember the deepest end-of-key
# node you passed, instead of trying every length. If the
# tie-break rule changes (priority instead of length),
# store the priority at the terminal node and keep the
# best terminal seen.
#
# BRUTE FORCE (say it first): Hash map lookup for every
# prefix length
# At every position try the prefix lengths from longest to
# shortest in a dictionary lookup. Correct and easy; each
# lookup hashes a substring, so it costs O(L^2) per
# position. Time O(n * L) lookups, each O(L). Space
# O(dictionary).


# BRUTE FORCE (say it first): Hash map lookup for every
# prefix length | time O(n * L) lookups, each O(L)
def tokenize_brute(text, table):
    out, i, longest = (
        [],
```

```
        0,
        max(map(len, table), default=0),
    )
    while i < len(text):
        for k in range(
            min(longest, len(text) - i), 0, -1
        ):
            if text[i : i + k] in table:
                out.append(table[text[i : i + k]])
                i += k
                break
        else:
            out.append(text[i])
            i += 1
    return out


# BASE QUESTION (optimal): Trie walk keeping the deepest
# (or best-priority) terminal
# Idea: Insert every key into a trie with its id at the
# last node. From each text position walk down while the
# next character exists, remembering the last terminal
# passed.
# FOLLOW-UP 1: The dictionary is huge.
# Answer: A flat map of every key wastes memory because
# keys share prefixes. A compressed (radix) trie stores
# each shared stretch once, or shard the keys by first
# characters across machines.
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id. priority(key): lower wins;
        # with
        # None the longest match wins
        self.priority = priority
        self.root = {}
        for key, ident in table.items():
            node = self.root
            for ch in key:
                node = node.setdefault(ch, {})
            # the end of a key stores the key and its id
            node["$"] = (key, ident)

    def better(self, new, old):
        # should the match `new` replace the match `old`?
        if old is None:
```

```
            return True
        if self.priority is None:
            # deeper (longer) matches come later: keep
            # them
            return True
        rank = self.priority
        return rank(new[0]) < rank(old[0])

    def longest_at(self, text, i):
        # walk the trie from position i
        node, best, j = self.root, None, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            end = node.get("$")
            if end and self.better(end, best):
                best = end
        return best

    def tokenize(self, text):
        out, i = [], 0
        while i < len(text):
            match = self.longest_at(text, i)
            if match is None:
                # no key starts here: keep the character
                out.append(text[i])
                i += 1
            else:
                key, ident = match
                out.append(ident)
                i += len(key)
        return out


# SIMPLER VERSION: one dictionary lookup per length
def tokenize_map(text, table):
    # simpler version: a dictionary lookup per length
    out, i = [], 0
    longest = max(map(len, table), default=0)
    while i < len(text):
        top = min(longest, len(text) - i)
        for size in range(top, 0, -1):
            if text[i:i + size] in table:
                out.append(table[text[i:i + size]])
                i += size
```

```
                break
        else:
            out.append(text[i])
            i += 1
    return out


if __name__ == "__main__":
    table = {"ab": 1, "abc": 2, "c": 3}
    t = Tokenizer(table)
    # abc is longer than ab, so it wins at position 0
    assert t.tokenize("abcab") == [2, 1]
    assert t.tokenize("xyz") == ["x", "y", "z"]
    assert t.tokenize("") == []
    assert tokenize_map("abcab", table) == [2, 1]
    # a single-letter key still applies the longest rule
    assert t.tokenize("cab") == [3, 1]
    # follow-up: priority by dictionary order instead
    order = list(table)
    pri = Tokenizer(table, priority=order.index)
    assert pri.tokenize("abc") == [1, 3]
```

NOTES FROM CANDIDATES

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.
Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.
Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).
If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

PREPARATION

Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.
For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.
If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

STUDY HERE: topic 3, Trees and Tries | main topic: Tries

MEDIUM | PracHub | Technical Screen | 19 of 22

Check if all substrings are dictionary words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 43.5 | 30 DAYS 0 | PRIORITY 82/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: No direct match; related LC 139 Word Break

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-dictionary-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

APPROACH

Enumerate all substrings of length >= 3 and look each up in the set; exit on the first failure. Prune: if any length-3 window fails return false immediately. O(n^2) lookups.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the dictionary access (a set or a trie), the maximum length of s, and whether case matters. A string of length n has about n^2 / 2 substrings of length at least 3, and each is a separate requirement.

BRUTE FORCE TO SAY OUT LOUD

Enumerate every substring of length at least 3 and look it up. O(n^2) lookups, each costing O(length) to hash, so O(n^3) characters.

OPTIMAL IDEA

Exit on the first failure; checking all length-3 windows first prunes most inputs. To avoid the length factor, walk a trie: for each start i descend the trie along s[i:], and at every depth of at least 3 require the end-of-word flag; if the path breaks, return false. That is O(n^2) node steps in total.

DRY-RUN THESE CASES

A string shorter than 3 (true vacuously), cats (needs cat, ats, cats), a failure on the first window, a failure only on the longest substring.

LIKELY FOLLOW-UPS

Anagram version (next question), or return the first failing substring.

PITFALLS

Checking only the whole string. Treating a string shorter than 3 as false.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

Every substring is a separate requirement, so no clever skipping exists: you must test about n^2 / 2 substrings. What you can improve is the cost per test and the early exit. A trie lets you extend a start position one character at a time without rehashing the whole substring, and the first failure ends everything.

Basis: Written; every contiguous substring of length >= 3 must be a dictionary word. The dictionary is a set (optionally preprocessed into a trie).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, dictionary):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            if s[i:j] not in dictionary:
                return False
    return True
```

TIME	SPACE
O(n^2)	**O(1)**
lookups, O(n^3) characters hashed	extra

WHY IT IS SLOW

About n^2 / 2 lookups and each hashes a whole substring, O(n^3) characters. Walking a trie from each start, one character at a time, is O(n^2) with no hashing of long strings.

OPTIMAL SOLUTION · Walk a trie from each start position

TIME	SPACE
O(n^2)	**O(dictionary size)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the first failing substring.

Same loops, but return the substring instead of False, which is what a user wants to see. Check the shorter ones first so the report is the smallest counterexample.

FOLLOW-UP 2 The string is very long.

The check is inherently quadratic in the number of substrings; any substring that fails ends the search, so cheap necessary checks first help: every length-3 window must be a word (O(n) lookups), then length 4, and so on.

FOLLOW-UP 3 The dictionary is huge and sharded.

Batch the lookups by length (a set per length) or use a Bloom filter in front of the real dictionary to reject most misses cheaply. **Cost:** still O(n^2) lookups, spread over shards (each lookup O(L)); a Bloom filter in front removes most remote calls.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check if all substrings are dictionary
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** is a valid dictionary word (i.e., for all i, j
# with 0 <= i <= j < n and j - i + 1 >= 3, s[i..j] in
# dict). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - You may assume
# dictionary lookups are available (e.g.,
# dict.contains(word)), or you can preprocess the
# dictionary. Example: - If s = "cats", you must check
# "cat", "ats", and "cats" (and any other substrings of
# length >= 3).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the first failing substring.
# 2. The string is very long.
# 3. The dictionary is huge and sharded.
#
# INTERVIEW NOTES
# Every substring is a separate requirement, so no clever
# skipping exists: you must test about n^2 / 2 substrings.
# What you can improve is the cost per test and the early
# exit. A trie lets you extend a start position one
# character at a time without rehashing the whole
# substring, and the first failure ends everything.
#
# BRUTE FORCE (say it first): Enumerate each substring and
# look it up in a set
# Slice s[i:j] for every i, j with j - i >= 3 and test
# membership. Correct and simple; every lookup re-hashes
# the whole substring. Time O(n^2) lookups, O(n^3)
# characters hashed. Space O(1) extra.


# BRUTE FORCE (say it first): Enumerate each substring and
# look it up in a set | time O(n^2) lookups, O(n^3)
# characters hashed
def all_substrings_words_brute(s, words):
    return all(
        s[i:j] in words
        for i in range(len(s))
        for j in range(i + 3, len(s) + 1)
    )


# HELPERS for the base question
# FOLLOW-UP 1: Return the first failing substring.
# Answer: Same loops, but return the substring instead of
# False, which is what a user wants to see. Check the
# shorter ones first so the report is the smallest
# counterexample.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def first_failing(s, words, min_len=3):
    # the SHORTEST substring (length >= min_len) that is
    # not a
    # word, or None. Short lengths first gives the
    # smallest
    # counterexample, and cheap checks run before
    # expensive ones.
    words = set(words)
    longest = max(map(len, words), default=0)
    for length in range(min_len, len(s) + 1):
        # longer than every word: the first window already
        # fails
        if length > longest:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if piece not in words:
                return piece
    return None


# BASE QUESTION (optimal): Walk a trie from each start
# position
# Idea: For each start i descend the trie along s[i:]. At
# depth 1 and 2 only walk; from depth 3 on the current
# node must be marked as a word end.
def all_substrings_words(s, words, min_len=3):
    return first_failing(s, words, min_len) is None


# SAME CHECK with a trie
class Trie:
    """The same check with a trie."""

    def __init__(self, words):
        self.root = {}
        for w in words:
            node = self.root
            for ch in w:
                node = node.setdefault(ch, {})
            node["$"] = True

    def all_substrings(self, s, min_len=3):
        for i in range(len(s)):
            node = self.root
            for depth, ch in enumerate(s[i:], 1):
                node = node.get(ch)
                if node is None:
                    # no word has this prefix: fails if
                    # the
                    # substring is long enough to count
                    return len(s) - i < min_len
                if depth >= min_len and "$" not in node:
                    return False
        return True
```

```
if __name__ == "__main__":
    words = {"aaa", "aaaa", "aaaaa"}
    assert all_substrings_words("aaaaa", words)
    assert Trie(words).all_substrings("aaaaa")
    # "aab" is a window that is not a word
    assert first_failing("aaab", words) == "aab"
    assert first_failing("aab", words) == "aab"
    assert not all_substrings_words("aab", words)
    assert not Trie(words).all_substrings("aab")
    # shorter than 3 letters: nothing to check
    assert all_substrings_words("ab", set())
    assert Trie([]).all_substrings("ab")
    # longer than every word: fails immediately
    assert first_failing("aaaaaa", words) == "aaaaaa"
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, dictionary: list[str]) -> bool:
```

Sample call

```
solution('cat', ['cat'])
```

Test cases

Input	Expected
('cat', ['cat'])	True
('cats', ['cat', 'cas', 'cts', 'ats', 'cats'])	True

STUDY HERE: topic 3, Trees and Tries | main topic: Tries

MEDIUM | IN APP LC 208 | PracHub | Technical Screen | 20 of 22

Return dictionary words matching a prefix

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 106/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/return-dictionary-words-matching-a-prefix

QUESTION

You are given a list of strings **`words`** (a dictionary) and a string **`prefix`**. Return all words in **`words`** that start with **`prefix`**. Clarifications/constraints:

- Matching is case-sensitive.
- If no words match, return an empty list.
- If **`words`** contains duplicates, include duplicates in the output.
- You may return the result in any order unless the interviewer asks for a specific ordering (e.g., lexicographic).

APPROACH

Sort once and binary search the prefix range, or build a trie for repeated queries. O(log n + m) per query after preprocessing.

HOW TO THINK ABOUT IT

If the dictionary is sorted, all words with a prefix form one contiguous block, and its edges are found with binary search: the first word >= prefix, and the first word that is >= prefix with its last character incremented. For many queries on a static list this beats a trie in memory.

Basis: Search Suggestions System (LeetCode 1268) and prefix queries on a sorted list.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words, prefix):
    found = [w for w in words
             if w.startswith(prefix)]
    return found
```

BRUTE FORCE

TIME	SPACE
O(W * L) per query	O(m)

WHY IT IS SLOW

Each query scans the whole dictionary, O(W * L). With the words sorted, binary search finds the block of matches directly.

OPTIMAL SOLUTION · Sort once, binary search the range

TIME	SPACE
O(log W + m) per query after O(W log W) sorting	O(W)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the first K matches (autocomplete).

Stop the forward walk after K words, as the limit parameter does; with the sorted list the first K in alphabetical order are already the first K in the block.

FOLLOW-UP 2 Words are added while queries run.

Keep the list sorted with bisect.insort (O(n) per insert, fast in practice) or use a balanced tree; a trie handles inserts in O(L).

FOLLOW-UP 3 Case-insensitive matching.

Store a casefolded sort key next to each word (a tuple (key, word)) and search on the key; do not lowercase the displayed word.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Return dictionary words matching a prefix
#
# You are given a list of strings words (a dictionary) and
# a string prefix. Return all words in words that start
# with prefix. Clarifications/constraints: - Matching is
# case-sensitive. - If no words match, return an empty
# list. - If words contains duplicates, include duplicates
# in the output. - You may return the result in any order
# unless the interviewer asks for a specific ordering
# (e.g., lexicographic).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the first K matches (autocomplete).
# 2. Words are added while queries run.
# 3. Case-insensitive matching.
#
# INTERVIEW NOTES
# If the dictionary is sorted, all words with a prefix
```

```
# form one contiguous block, and its edges are found with
# binary search: the first word >= prefix, and the first
# word that is >= prefix with its last character
# incremented. For many queries on a static list this
# beats a trie in memory.
#
# BRUTE FORCE (say it first): Filter the whole list per
# query
# Test startswith on every word. Correct and fine for one
# query; each query pays for the whole list. Time O(W * L)
# per query. Space O(m).


from bisect import bisect_left, bisect_right, insort


# BRUTE FORCE (say it first): Filter the whole list per
# query | time O(W * L) per query
def prefix_matches_brute(words, prefix):
    return sorted(
        w for w in words if w.startswith(prefix)
    )


# BASE QUESTION (optimal): Sort once, binary search the
# range
# Idea: With the words sorted, the matches are the words
```

```
# from bisect_left(prefix) until the first word that no
# longer starts with the prefix.
# FOLLOW-UP 1: Only the first K matches (autocomplete).
# Answer: Stop the forward walk after K words, as the
# limit parameter does; with the sorted list the first K
# in alphabetical order are already the first K in the
# block.
# FOLLOW-UP 2: Words are added while queries run.
# Answer: Keep the list sorted with bisect.insort (O(n)
# per insert, fast in practice) or use a balanced tree; a
# trie handles inserts in O(L).
class PrefixIndex:
    def __init__(self, words):
        # a sorted copy: all words with a prefix sit
        # together
        self.words = sorted(words)

    # FOLLOW-UP 1: limit= returns only the first K
    def matches(self, prefix, limit=None):
        # first possible position, then walk while it
        # matches
        lo = bisect_left(self.words, prefix)
        # one past the last word that starts with the
        # prefix
        hi = bisect_right(self.words, prefix + "")
        found = self.words[lo:hi]
        return found if limit is None else found[:limit]

    # FOLLOW-UP 2: words added while queries run
    def add(self, word):
        # words can arrive while queries run: keep it
        # sorted
        insort(self.words, word)


# SIMPLE SCAN (say it first)
def matches_scan(words, prefix):
    # the simple O(n * L) answer, in the original order
    return [w for w in words if w.startswith(prefix)]


# FOLLOW-UP 3: Case-insensitive matching.
# Answer: Store a casefolded sort key next to each word (a
# tuple (key, word)) and search on the key; do not
# lowercase the displayed word.
```

```
class CaseInsensitiveIndex:
    """Ignore case, but show the original words."""

    def __init__(self, words):
        keyed = ((w.casefold(), w) for w in words)
        self.items = sorted(keyed)

    def matches(self, prefix):
        key = prefix.casefold()
        lo = bisect_left(self.items, (key,))
        out = []
        while lo < len(self.items) and \
                self.items[lo][0].startswith(key):
            out.append(self.items[lo][1])
            lo += 1
        return out


if __name__ == "__main__":
    words = ["apple", "app", "apricot", "Apple", "banana",
             "app"]
    idx = PrefixIndex(words)
    # duplicates are kept; matching is case sensitive
    assert idx.matches("ap") == ["app", "app", "apple",
                                 "apricot"]
    assert idx.matches("x") == []
    assert idx.matches("") == sorted(words)
    assert idx.matches("ap", limit=2) == ["app", "app"]
    scan = sorted(matches_scan(words, "ap"))
    assert scan == idx.matches("ap")
    idx.add("apex")
    assert "apex" in idx.matches("ap")
    ci = CaseInsensitiveIndex(words)
    got = ci.matches("APP")
    assert got == ["app", "app", "Apple", "apple"]
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], prefix: str) -> list[str]:
```

Sample call

```
solution(['apple', 'app', 'banana', 'apply'], 'app')
```

Test cases

Input	Expected
`(['apple', 'app', 'banana', 'apply'], 'app')`	`['apple', 'app', 'apply']`
`(['Cat', 'car', 'Cart', 'dog'], 'Ca')`	`['Cat', 'Cart']`

STUDY HERE: topic 3, Trees and Tries | main topic: Tries

MEDIUM | PracHub | Technical Screen | 21 of 22

★ TOP 75: rank 70 in the Medium and Hard priority list

★ Solve chat, grid paths, and car rentals

POSTED 2025-12-02 | 322 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 43/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words, LeetCode #62 Unique Paths, LeetCode #253 Meeting Rooms II

LeetCode match: LC 692 Top K Frequent Words + LC 62/63 Unique Paths + LC 253

Posted on PracHub: 2025-12-02 (updated 2026-05-02)

Original: https://prachub.com/interview-questions/solve-chat-grid-paths-and-car-rentals

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.

Input:

- An array **`messages`** of length **`n`**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **`k`** (1 <= k <= number of distinct users).

Output:

- Return a list of **`k`** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **`k`** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **`k`** is much smaller than the number of distinct users. Design and implement a function, for example:
- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:

QUESTION

- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.
- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

QUESTION

Tasks: 1.
Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.
Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.
Output:

- An integer `C` = minimal number of cars needed.
- A mapping from each `rental_id` to a `car_id` in the range `1..C`, such that no two rentals assigned to the same `car_id` overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals `[s1, e1)` and `[s2, e2)` overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if `e1 == s2` they do **not** over

APPROACH

Top-K messages via counts + heap; grid paths via DP (with obstacles if present); car rentals = minimum fleet = max overlapping reservations (sweep line).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Three independent tasks. Chat: activity means the number of messages, ties resolved how? Grid paths: moves right or down only, with or without obstacles, modulo? Car rentals: minimum cars equals the maximum overlap of reservations; does a car returned at time t serve a rental starting at t?

BRUTE FORCE TO SAY OUT LOUD

Chat: sort the counts. Grid: recursion without memory. Cars: assign reservations to cars one by one.

OPTIMAL IDEA

Chat: Counter and heapq.nsmallest with the key (count descending, user id ascending). Grid: dp over cells with a rolling row, dp[j] += dp[j - 1], and 0 on obstacles; O(mn) time, O(n) space. Cars: sort the reservations by start and keep a min-heap of end times; reuse the earliest-ending car if it is free, else add one. The heap size at the end is the answer.

DRY-RUN THESE CASES

Chat: ties, fewer users than K. Grid: 1 x n, an obstacle on the start, no path. Cars: back-to-back rentals, one rental, all identical.

LIKELY FOLLOW-UPS

Chat in a stream, grid paths with diagonal moves, cars with a fixed fleet size and rejecting requests.

PITFALLS

Treating back-to-back as overlapping. Forgetting the modulo in the grid count if the statement requires it.

HOW TO THINK ABOUT IT

Three unrelated tasks, so say so and give each a one-line plan. Chat: count, then take the top K with a heap. Grid: each column depends only on the previous one, so keep one column. Cars: the minimum number of cars is the largest number of rentals that overlap at once, which a heap of return times tracks while also giving an assignment.

Basis: Three independent tasks: top K frequent items (LeetCode 347 / 692), grid path counting (a Unique Paths variant) and minimum meeting rooms with assignment (LeetCode 253 plus the assignment step).

BRUTE FORCE · Q1: sort all users by (count, user id)

BRUTE FORCE · PSEUDO-CODE

```
def top_k_users(log, k):
    counts = Counter(msg.user for msg in log)
    def rank(u):
        return (-counts[u], u)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorting all d distinct users costs O(d log d) even though only K are needed; a heap of size K costs O(d log k).

BRUTE FORCE · Q2: plain recursion over the moves

BRUTE FORCE · PSEUDO-CODE

```
def ways(r, c):
    if c == n - 1:
        return 1 if r == n - 1 else 0
    total = 0
    for dr in (-1, 0, 1):
        if 0 <= r + dr < n:
            total += ways(r + dr, c + 1)
    return total
```

TIME	SPACE
O(3^n)	O(n)

WHY IT IS SLOW

Three calls per cell and the same cell is reached by many paths, so the call tree has about 3^n nodes. One column at a time with a rolling array is O(m * n).

BRUTE FORCE · Q3: count the overlap at every time and assign greedily without a heap

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE · Q3: count the overlap at every time and assign greedily without a heap

```
def cars_needed(rentals):
    free_at = []  # per car
    for start, end in sorted(rentals):
        for i, f in enumerate(free_at):
            if f <= start:
                free_at[i] = end  # reuse
                break
        else:
            free_at.append(end)  # new car
    return len(free_at)
```

TIME	SPACE
O(R^2)	O(R)

WHY IT IS SLOW

Each rental scans every car, so the cost is O(R * cars), up to O(R^2). A heap of free times finds the earliest-free car in O(log R).

OPTIMAL SOLUTION · Q1: top-K active users with a size-K heap

TIME	SPACE
O(n + d log k) for d distinct users	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: count constrained paths with a rolling column

TIME	SPACE
O(m * n)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q3: minimum cars and one valid assignment

TIME	SPACE
O(R log R)	O(R)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1: what if the log is a stream and memory is limited?

Use a heavy-hitters sketch: Misra-Gries keeps k counters and is exact for any user with more than n / (k + 1) messages; a count-min sketch plus a small heap estimates the top K in O(1) memory per update. For exact answers across machines, count per shard, send each shard's local top-K plus a threshold, and merge.

FOLLOW-UP 2 Q2: what changes if there are obstacles or the path may also go straight up and down?

Obstacles just force ways[r] = 0 for blocked cells in the column being built. Allowing vertical moves inside a column makes each column a chain, so the transition becomes a linear recurrence solved by a prefix pass, still O(m * n). For huge n, use matrix exponentiation of the m x m transition matrix: O(m^3 log n).

FOLLOW-UP 3 Q3: how do you answer 'is one more rental possible' online?

Keep the busy heap and the free-car count between calls. A new rental fits without a new car if any car's return time is <= its pickup, which is a heap peek after popping everything that returned before the pickup. That is O(log R) per query.

FOLLOW-UP 4 Q2 with obstacles.

A blocked cell contributes zero ways when its column is built; everything else is unchanged. O(m * n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve chat, grid paths, and car rentals
#
# You are given three independent programming tasks. For
# each, describe and implement an efficient algorithm, and
# be prepared to analyze its time and space complexity.
# --- ### Q1. Find top-K most active chat users You are
# given a log of chat messages from a messaging
# application. **Input**: - An array messages of length n,
# where each element messages[i] is the user_id (string or
# integer) of the sender of the i-th message. - An integer
# k (1 <= k <= number of distinct users). **Output**: -
# Return a list of k user IDs corresponding to the users
# who have sent the most messages. **Details &
# requirements**: - The "activity" of a user is defined as
# the number of messages they have sent (i.e., their
# frequency in messages). - If there are fewer than k
# distinct users, return all distinct users.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1: what if the log is a stream and memory is
# limited?
# 2. Q2: what changes if there are obstacles or the path
# may also go straight up and down?
# 3. Q3: how do you answer 'is one more rental possible'
# online?
# 4. Q2 with obstacles.
#
# INTERVIEW NOTES
# Three unrelated tasks, so say so and give each a one-
# line plan. Chat: count, then take the top K with a heap.
# Grid: each column depends only on the previous one, so
# keep one column. Cars: the minimum number of cars is the
# largest number of rentals that overlap at once, which a
# heap of return times tracks while also giving an
# assignment.
#
```

```
# BRUTE FORCE (say it first): Q1: sort all users by
# (count, user id)
# Count the messages and sort all distinct users, then
# take the first K. Correct and the oracle; sorting all d
# users does more work than a heap of size K. Time O(d log
# d). Space O(d).


import heapq
```

```
from collections import Counter


# BRUTE FORCE (say it first): Q1: sort all users by
# (count, user id) | time O(d log d)
def top_users_sort(messages, k):
    c = Counter(messages)
    return sorted(c, key=lambda u: (-c[u], u))[:k]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def top_k_users(messages, k):
    # Q1, heap: O(n + d log k); sorted order breaks ties
    counts = Counter(messages)
    best = heapq.nsmallest(
        k, counts.items(), key=lambda kv: (-kv[1], kv[0]))
    return [user for user, _ in best]


# Q1 second approach: sort every user
def top_k_users_sorted(messages, k):
    # Q1, second approach: sort every user, O(d log d)
    counts = Counter(messages)
    ranked = sorted(counts, key=lambda u: (-counts[u], u))
    return ranked[:k]


# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def count_paths(m, n, blocked=(), mod=None):
    # Q2: start (m-1, 0), end (m-1, n-1), moves right-up,
    # right, right-down. One column of ways is enough:
    # O(m).
    wall = set(blocked)
    ways = [0] * m
    ways[m - 1] = 1
```

```
    for c in range(1, n):
        nxt = [0] * m
        for r in range(m):
            # FOLLOW-UP 4: blocked cells (obstacles) give
            # no paths
            if (r, c) in wall:
                # a blocked cell contributes no paths
                continue
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
            if r + 1 < m:
                total += ways[r + 1]
            nxt[r] = total % mod if mod else total
        ways = nxt
    return ways[m - 1]
```

```
# BASE QUESTION (optimal): Q1: top-K active users with a
# size-K heap
# Idea: Count messages in a hash map, then keep a min-heap
# of the best k (count, user). That beats sorting all d
# users when k is small. Ties break by smaller user id so
# the answer is deterministic.
def min_cars(rentals):
    # Q3: rentals are (pickup, return); how many cars are
    # needed?
    # A heap holds the return times of the cars on the
    # road.
    cars = []
    for pickup, back in sorted(rentals):
        if cars and cars[0] <= pickup:
            # a car came back in time: reuse it
            heapq.heapreplace(cars, back)
        else:
            heapq.heappush(cars, back)
    return len(cars)


# FOLLOW-UP 3: Q3: how do you answer 'is one more rental
# possible' online?
# Answer: Keep the busy heap and the free-car count
# between calls. A new rental fits without a new car if
# any car's return time is <= its pickup, which is a heap
# peek after popping everything that returned before the
```

```
# pickup.
class RentalDesk:
    """Online: can one more rental be served right now?"""

    def __init__(self, fleet):
        self.fleet, self.out = fleet, []

    def rent(self, pickup, back):
        # cars returned by `pickup` are free again
        while self.out and self.out[0] <= pickup:
            heapq.heappop(self.out)
        if len(self.out) >= self.fleet:
            return False
        heapq.heappush(self.out, back)
        return True


if __name__ == "__main__":
    log = ["a", "b", "a", "c", "a", "b"]
    assert top_k_users(log, 2) == ["a", "b"]
    assert top_k_users_sorted(log, 5) == ["a", "b", "c"]
    # grid paths: a 3 x 3 grid has 2 paths along the
    # bottom
    assert count_paths(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths(4, 5) == 9
    # an obstacle in the middle of the bottom row
    assert count_paths(3, 3, blocked=[(2, 1)]) == 1
    modulus = 10 ** 9 + 7
    assert count_paths(5, 30, mod=modulus) < modulus
    cars = [(1, 4), (2, 5), (4, 6), (5, 8)]
    assert min_cars(cars) == 2
    assert min_cars([]) == 0
    desk = RentalDesk(1)
    assert desk.rent(1, 4) and not desk.rent(2, 3)
    assert desk.rent(4, 6)
```

FULL QUESTION TEXT FROM THE SITE

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity.

Q1. Find top-K most active chat users

You are given a log of chat messages from a messaging application.

Input:

An array messages of length n , where each element messages[i] is the user_id (string or integer) of the sender of the i-th message.
An integer k (1 <= k <= number of distinct users).

Output:

Return a list of k user IDs corresponding to the users who have sent the most messages.

Details & requirements:

The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in messages).
If there are fewer than k distinct users, return all distinct users.
If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by user_id in ascending order.
Aim for an algorithm that is more efficient than sorting all users by count when k is much smaller than the number of distinct users.

Design and implement a function, for example:

List<UserId> topKActiveUsers(List<UserId> messages, int k)

Also explain:

At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
The time and space complexity of each approach.
Q2. Count paths in a 2D grid with constrained moves

You have a 2D grid with m rows and n columns. Index rows from top to bottom as 0..m-1 and columns from left to right as 0..n-1.

The start cell is the bottom-left corner: (m-1, 0) .
The end cell is the bottom-right corner: (m-1, n-1) .

From any cell (r, c), you may move only to the next column to the right, in one of up to three ways (if within bounds):

Right-up: (r-1, c+1)
Right: (r, c+1)
Right-down: (r+1, c+1)

You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

Given integers m and n , compute the total number of distinct paths from the start cell (m-1, 0) to the end cell (m-1, n-1) using only the allowed moves.

Requirements:

FULL QUESTION TEXT FROM THE SITE

Implement a function such as long countPaths(int m, int n) .
The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct.
Q3. Minimum cars needed for rentals and assignment

You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

An array of R rental records. Each record has:
rental_id : a unique identifier (e.g., integer or string).
pickup_time : an integer start time.
return_time : an integer end time, with pickup_time < return_time .

Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time t and another starts at time t, the same car can serve both.

Tasks:

Capacity : Compute the minimum number of cars required so that all rentals can be fulfilled without time conflicts on any single car.
Assignment : Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

An integer C = minimal number of cars needed.
A mapping from each rental_id to a car_id in the range 1..C , such that no two rentals assigned to the same car_id overlap in time.

Requirements & clarifications:

Overlap definition: two rentals with intervals [s1, e1) and [s2, e2) overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if e1 == s2 they do not overlap.
Aim for an algorithm that is efficient for up to, say, 10^5 rental records.
Discuss a naive approach and then an optimized approach (for example, using a min-heap/priority queue keyed by the earliest return time to track currently in-use cars).
Analyze the time and space complexity of your final solution.

STUDY HERE: topic 3, Trees and Tries | main topic: Tries

HARD | IN APP LC 648 | PracHub | Technical Screen | 22 of 22

Implement Longest-Match Text Replacement

POSTED 2026-01-20 | 273 DAYS AGO | OLDER

STUDY: TALK ALL TIME 31.9 30 DAYS 0 PRIORITY 120/228

ABOUT THIS QUESTION

Topic: Trees and Tries: Tries

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #648 Replace Words

LeetCode match: LC 648 Replace Words / LC 833 Find And Replace in String

Posted on PracHub: 2026-01-20 (updated 2026-06-13)

Original: https://prachub.com/coding-questions/implement-longest-match-text-replacement

QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed.

Example:

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2",
 "xy": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

Explanation: At index **`0`**, both **`"ab"`** and **`"abc"`** match, so choose the longest token **`"abc"`** and output **`"2"`**. Then **`"d"`** has no match, so output it literally. Then **`"xy"`** matches and outputs **`"3"`**. Finally, **`"z"`** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?

Part 1: Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

1. Find every token that matches the substring starting at that position.

QUESTION

2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.
3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.

EXAMPLES

```
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy":
    "3"})
Expected Output: ["2", "d", "3", "z"]
```

Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.

```
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
```

Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.

Hints

A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.

Part 2: Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

APPROACH

Build a trie of tokens; scan the text and at each position walk the trie, remembering the longest terminal match; emit its id and jump ahead, else copy the char. Aho-Corasick if the text is large.

HOW TO THINK ABOUT IT

At each position you want the longest dictionary token starting there. Checking every token costs the size of the dictionary per character, so share prefixes in a trie: walk the text from the current position down the trie, remember the last node that ends a token, and stop when the trie has no further edge. The remembered node is the longest match.

Basis: LeetCode 648 (Replace Words) and the longest-prefix matching used in tokenizers and routing tables. Scan left to right, at each position take the longest dictionary token that matches, else copy the character.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def replace(text, entries):
    out, i = [], 0
    while i < len(text):
        best = ""
        for token in entries:
            if text.startswith(token, i):
                if len(token) > len(best):
                    best = token
        if best:
            out.append(entries[best])
            i += len(best)
        else:
            out.append(text[i])
            i += 1
    return out
```

TIME	SPACE
O(len(text) * tokens * token length)	O(1) extra

WHY IT IS SLOW

Each position tests every token. A trie of the tokens is walked from i, remembering the last token end, so a position costs at most the longest token.

OPTIMAL SOLUTION · Trie walk that remembers the last token end

TIME	SPACE
O(len(text) * longest token)	O(total token length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The priority is not longest match: insertion order, a score, or a custom comparator.

Collect every token that matches at the position (walk the trie and gather all token ends along the path, not just the last) and choose with the comparator. The trie gives the candidates; the policy is separate.

FOLLOW-UP 2 The text is huge or the dictionary changes often.

Use Aho-Corasick if you need all matches in one pass over the text, or a trie that supports inserts and deletes; a longest-match scan is then linear in the text length times the longest token.

FOLLOW-UP 3 Overlapping matches or word boundaries.

Require a boundary before and after a token by checking the neighbouring characters when you record a candidate end, or tokenise on word boundaries first and look up whole words. **Cost:** still O(len(text) * longest token) with the trie walk; a boundary check is O(1) per candidate.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Longest-Match Text Replacement
#
# You are given a text string and a dictionary of
# replacement entries. Each dictionary entry has the form
# token:id, where token is a string pattern and id is the
# value that should be emitted when that token is matched.
# Implement a function that scans the text from left to
# right and produces an output sequence using greedy
# longest-prefix matching: 1. At the current text
# position, find all dictionary tokens that match the
# substring starting at that position. 2. If at least one
# token matches, choose the longest matching token, append
# its corresponding id to the output, and advance by the
# length of that token. 3. If no token matches, append the
# original character at the current position to the output
# and advance by one character. 4. Continue until the
# entire text has been processed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority is not longest match: insertion order, a
# score, or a custom comparator.
# 2. The text is huge or the dictionary changes often.
# 3. Overlapping matches or word boundaries.
#
# INTERVIEW NOTES
# At each position you want the longest dictionary token
# starting there. Checking every token costs the size of
# the dictionary per character, so share prefixes in a
# trie: walk the text from the current position down the
# trie, remember the last node that ends a token, and stop
# when the trie has no further edge. The remembered node
# is the longest match.
#
# BRUTE FORCE (say it first): At every position try every
# dictionary token
```

```
# For each position test startswith for every token and
# keep the longest. Correct and the oracle; each position
# checks the entire dictionary. Time O(len(text) * tokens
# * token length). Space O(1) extra.


# BRUTE FORCE (say it first): At every position try every
# dictionary token | time O(len(text) * tokens * token
# length)
def replace_longest_brute(text, mapping):
    out, i = [], 0
    while i < len(text):
        best = None
        for tok in mapping:
            if text.startswith(tok, i) and (
                best is None or len(tok) > len(best)
            ):
                best = tok
        if best:
            out.append(mapping[best])
            i += len(best)
        else:
            out.append(text[i])
            i += 1
    return out


# BASE QUESTION (optimal): Trie walk that remembers the
# last token end
# Idea: Build a trie of the tokens (each token's last node
# stores its id). At position i walk down the trie along
# the text; whenever the current node stores an id, record
# it and its length.
# FOLLOW-UP 1: The priority is not longest match:
# insertion order, a score, or a custom comparator.
# Answer: Collect every token that matches at the position
# (walk the trie and gather all token ends along the path,
# not just the last) and choose with the comparator. The
# trie gives the candidates; the policy is separate.
class Replacer:
    def __init__(self, mapping):
        # mapping: token -> id. A trie shares the
        # prefixes.
        self.root = {}
        for token, ident in mapping.items():
            node = self.root
            for ch in token:
                node = node.setdefault(ch, {})
            node["$"] = (token, ident)

    def candidates(self, text, i):
        # every token that matches at position i, shortest
        # first (all ends along the trie path, not just
        # the last)
        out, node, j = [], self.root, i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node:
                out.append(node["$"])
        return out

    # FOLLOW-UP 1: choose= picks by priority, not length
    def replace(self, text, choose=None,
                word_boundary=False):
        # choose(candidates) picks the winner; the default
        # is the
        # longest token
        choose = choose or (lambda cs: cs[-1])
        out, i = [], 0
        while i < len(text):
            found = self.candidates(text, i)
            # FOLLOW-UP 3: only whole-word matches
            if word_boundary:
                found = [c for c in found if
                         self.edge(text, i, len(c[0]))]
            if found:
                token, ident = choose(found)
                out.append(ident)
                i += len(token)
            else:
                # nothing matches here: keep the character
                out.append(text[i])
                i += 1
        return out

    @staticmethod
    def edge(text, i, size):
        # letters must not touch the match on either side
        before = i == 0 or not text[i - 1].isalnum()
        end = i + size
        after = end == len(text)
        after = after or not text[end].isalnum()
        return before and after


if __name__ == "__main__":
    table = {"ab": 1, "abc": 2, "c": 3}
    r = Replacer(table)
    # abc beats ab at position 0
    assert r.replace("abcab") == [2, 1]
    assert r.replace("xyz") == ["x", "y", "z"]
    assert r.replace("") == []
    # priority by insertion order instead of length
    order = list(table)
    first = r.replace("abc", lambda cs: min(
        cs, key=lambda c: order.index(c[0])))
    assert first == [1, 3]
    # whole words only
    words = Replacer({"cat": "C"})
    got = words.replace("cat concat cat", None, True)
    assert got[0] == "C" and got[-1] == "C"
    assert "C" not in got[1:-1]
```

FUNCTION AND TEST CASES

Signature

```
def solution(text: str, dictionary: dict[str, str]) -> lis
t[str]:
```

Sample call

```
solution("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
```

Test cases

Input	Expected
("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})	["2", "d", "3", "z"]
("aaaaa", {"a": "X", "aa": "Y"})	["Y", "Y", "X"]

ALSO REPORTED · 1 more

Greedy Longest-Match Dictionary Tokenizer | 1Point3Acres | phone-screen | asked 2026-06-15 | Low

https://www.1point3acres.com/interview/problems/company/google/tokenize-longest-dict-match-greedy

The key:id dictionary format and the literal-passthrough for unmatched characters are confirmed across multiple candidates.

Two natural shapes:

Hashmap of keys + per-position scan up to maxKeyLen. Easy to write, O(n · maxKeyLen) time; O(n · L) if the longest key length L dominates.

Trie of keys + walk the trie one character at a time from each position; remember the deepest reached is_end node and emit it. O(n · L) worst case but does only one pass per starting position and adapts cleanly to the priority-rule follow-up (store priority at each terminal node and keep the best matched terminal along the walk, not necessarily the deepest).

Edge cases to discuss: overlapping keys with different lengths ("ab" vs "abc"), key whose match would consume the rest of the string, empty dictionary, key that is a single character (still apply longest-match, not literal-fallback).

If the follow-up generalizes to "non-overlapping tokenization that maximizes total score", that's the word-break with weights DP variant — flag it as a different problem class rather than patching the greedy.

Preparation: Implement both versions (hashmap-scan and trie-walk) from scratch in under 15 min each. Run them on a small dict like {"the": 1, "theater": 2, "a": 3} against text "theater" to confirm the trie returns id 2 and the hashmap version returns the same.

For the priority-tie follow-up: practice attaching a priority field to terminal nodes and picking the highest-priority terminal seen along the trie walk, not just the deepest. Time complexity should not change.

If asked to stream the input, switch to the Aho-Corasick mental model — the trie augmented with failure links emits all dictionary matches in a single linear pass over the text.

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 4 OF 10 | CORE TOPIC

Graphs and Search

21 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

19	2
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Graphs and Search | 21 questions | tap a title to jump to it

BFS AND DFS (11)

M	1. Count Islands in a Grid and Track the Count as Land Cells Are Added	PracHub	2026-10-02
M	2. Path Minimizing the Maximum Cell (Swim in Rising Water)	1Point3Acres	2026-09-29
M	3. Rotten Oranges / Multi-Source BFS (taxis)	1Point3Acres	2026-09-08
M	4. Count Clusters of 2D Points Within a Radius	PracHub	2026-06-18
M	5. Find shortest relationship path using BFS	PracHub	2026-02-12
M	6. Validate parent array forms a tree	PracHub	2026-02-11
M	7. Solve matrix groups and recipe inventory	PracHub	2026-01-22
M	8. Compute distance to nearest taxi in grid	PracHub	2026-01-10
M	9. Validate course catalog dependencies	PracHub	2025-08-01
M	10. Solve Grid and Counting Problems	PracHub	2025-01-25
H	11. Infection Spread on a Grid (Cellular Automaton)	PracHub	2026-06-14

TOPOLOGICAL SORT AND CYCLES (4)

M	12. Template Variable Expansion with Follow-Ups	1Point3Acres	2026-10-17
M	13. Course Scheduling and Dependency Variants	1Point3Acres	2026-10-04
M	14. Determine whether two nodes are related	PracHub	2026-01-22
M	15. Shortest Directed Cycle Through a Node	1Point3Acres	no date

UNION-FIND (3)

M	16. Number of Islands	1Point3Acres	2026-09-30
M	17. 2D Points Connected Components by Radius	1Point3Acres	2026-08-04
H	18. Friend Graph with Timeline (Friend / Unfriend Events)	1Point3Acres	2026-09-29

SHORTEST PATHS (3)

M	19. Train / Flight Schedule Reachability	1Point3Acres	2026-10-08
M	20. Shortest Path Nodes and Escaping a Pursuer	1Point3Acres	2026-09-29
M	21. Minimum Direction Violations in a Directed Graph	1Point3Acres	2026-07-19

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 305 | PracHub | Technical Screen | 1 of 21

Count Islands in a Grid and Track the Count as Land Cells Are Added

POSTED 2026-10-02 | 18 DAYS AGO | LAST 30 DAYS

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 104/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #305 Number of Islands II

LeetCode match: LC 305 Number of Islands II

Posted on PracHub: 2026-10-02 (updated 2026-10-03)

Original: https://prachub.com/coding-questions/count-islands-in-a-grid-and-track-the-count-as-land-cells-are-added

QUESTION

A map is a grid with **m** rows and **n** columns in which every cell is land (**1**) or water (**0**). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- **grid[r][c]** is **1** for land and **0** for water, where **r** is the row index and **c** is the column index.
- An addition **(r, c)** turns cell **(r, c)** into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length **len(additions) + 1**. Element **0** is the number of islands in the initial grid. For **i >= 1**, element **i** is the number of islands after the first **i** additions have been applied.

Constraints

- **1 <= m <= 1000**, **1 <= n <= 1000** and **m * n <= 200000**
- Every row of **grid** has exactly **n** entries, each **0** or **1**.
- **0 <= len(additions) <= 200000**
- Every addition **(r, c)** satisfies **0 <= r < m** and **0 <= c < n**. The same cell may appear more than once.
- Every count is at most **m * n**, so all values fit in a 32-bit signed integer.

Examples

QUESTION

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

APPROACH

Union-find over cells: start with initial islands, then each addition creates a component and unions with the four neighbors; track the component count. Ignore repeated cells. O((mn + k) alpha).

HOW TO THINK ABOUT IT

Re-running a flood fill after every addition costs O(mn) each time. Land only ever appears, so islands only merge: Union-Find fits exactly. A new land cell starts as a new island (count + 1) and every union with a neighbouring land cell that belongs to a different island removes one island (count - 1).

Basis: LeetCode 305, Number of Islands II. Report the count before any addition and after each; adding a cell that is already land changes nothing.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def add_land(r, c):
    grid[r][c] = 1
    return count_islands(grid)  # full flood fill
answers = [add_land(r, c) for r, c in additions]
```

TIME	SPACE
O(A * m * n)	O(m * n)

WHY IT IS SLOW

BRUTE FORCE

Each addition costs a pass over the whole grid, O(A * m * n). Union-Find merges the new cell with its land neighbours and updates the count in near-constant time.

OPTIMAL SOLUTION · Union-Find updated cell by cell

TIME	SPACE
O(m * n + A * alpha(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Land can also be removed (or turned back to water).

Union-Find cannot split sets. Either recompute the affected island with a local flood fill, or process all operations offline with a segment tree over time and a rollback Union-Find.

FOLLOW-UP 2 The grid is 10^9 x 10^9 with a few additions.

Keep the Union-Find on a dictionary keyed by coordinates, as the code does; memory depends on the number of land cells only, never on the grid size. **Cost:** O(a * alpha) for a additions and O(a) memory, independent of the grid size.

FOLLOW-UP 3 Report the size of the largest island after each addition.

Store a size per root in the Union-Find (union by size) and keep a running maximum, which only grows when land is added; each step stays near-constant time. Checked against recomputing the largest island from scratch. **Cost:** O(alpha) per addition with size tracking; the maximum is one running variable.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Islands in a Grid and Track the
# Count as Land Cells Are Added
#
# A map is a grid with m rows and n columns in which every
# cell is land (1) or water (0). An island is a maximal
# group of land cells connected horizontally or
# vertically. Cells that touch only at a corner are not
# connected. You are given the initial map and a sequence
# of additions, each of which turns one cell into land.
# Report the number of islands before any addition, and
# again after each addition. ### Function Signature python
# def island_counts(grid: list[list[int]], additions:
# list[tuple[int, int]]) -> list[int]: ### Rules -
# grid[r][c] is 1 for land and 0 for water, where r is the
# row index and c is the column index. - An addition (r,
# c) turns cell (r, c) into land. If that cell is already
# land, either initially or because of an earlier
# addition, the map does not change. - Additions are
# applied in the given order. Land never turns back into
# water.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Land can also be removed (or turned back to water).
# 2. The grid is 10^9 x 10^9 with a few additions.
# 3. Report the size of the largest island after each
# addition.
#
# INTERVIEW NOTES
# Re-running a flood fill after every addition costs O(mn)
# each time. Land only ever appears, so islands only
# merge: Union-Find fits exactly. A new land cell starts
# as a new island (count + 1) and every union with a
# neighbouring land cell that belongs to a different
# island removes one island (count - 1).
#
# BRUTE FORCE (say it first): Recount all islands after
# every addition
# Apply the addition and run a full flood-fill count
# again. Correct and the oracle; every addition costs a
# pass over the entire grid. Time O(A * m * n). Space O(m
# * n).


# BRUTE FORCE (say it first): Recount all islands after
# every addition | time O(A * m * n)
```

```
def _count_islands(grid):
    m, n = len(grid), len(grid[0]) if grid else 0
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in seen:
                count += 1
                stack = [(r, c)]
                seen.add((r, c))
                while stack:
                    x, y = stack.pop()
                    for nx, ny in (
                        (x + 1, y),
                        (x - 1, y),
                        (x, y + 1),
                        (x, y - 1),
                    ):
                        if (
                            0 <= nx < m
                            and 0 <= ny < n
                            and grid[nx][ny] == 1
                            and (nx, ny) not in seen
                        ):
                            seen.add((nx, ny))
                            stack.append((nx, ny))
    return count


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def island_counts_recount(grid, additions):
    g = [row[:] for row in grid]
    out = [_count_islands(g)]
    for r, c in additions:
        g[r][c] = 1
        out.append(_count_islands(g))
    return out


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
```

```
# already land; otherwise make it a new set (count + 1)
```

```
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
class Islands:
    def __init__(self):
        # land cell -> parent cell, and sizes of the roots
        self.parent, self.size = {}, {}
        self.count, self.largest = 0, 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add(self, r, c):
        if (r, c) in self.parent:
            return
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1
        self.largest = max(self.largest, 1)
        # join with every land neighbour
        for dr, dc in STEPS:
            nb = (r + dr, c + dc)
            if nb in self.parent:
                self.union((r, c), nb)

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
        # union by size: the small island joins the big
        # one
        if self.size[a] < self.size[b]:
            a, b = b, a
        self.parent[b] = a
        self.size[a] += self.size[b]
        self.count -= 1
        self.largest = max(self.largest, self.size[a])


# BASE QUESTION (optimal): Union-Find updated cell by cell
# Idea: Build the initial islands with unions (count
# them), then for each addition: ignore it if the cell is
```

```
# already land; otherwise make it a new set (count + 1)
# and union it with each land neighbour, subtracting one
# from the count every time two different sets merge.
def island_counts(grid, additions):
    # counts before any addition, then after each one
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.count]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.count)
    return out


# FOLLOW-UP 3: Report the size of the largest island after
# each addition.
# Answer: Store a size per root in the Union-Find (union
# by size) and keep a running maximum, which only grows
# when land is added; each step stays near-constant time.
# Checked against recomputing the largest island from
# scratch.
def largest_after_each(grid, additions):
```

```
    # the follow-up: size of the biggest island each time
    isl = Islands()
    for r, row in enumerate(grid):
        for c, v in enumerate(row):
            if v == 1:
                isl.add(r, c)
    out = [isl.largest]
    for r, c in additions:
        isl.add(r, c)
        out.append(isl.largest)
    return out


if __name__ == "__main__":
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    # 2 islands; the adds join them into one; a repeat is
    # ignored
    assert island_counts(grid, adds) == [2, 2, 2, 1, 1]
    sizes = largest_after_each(grid, adds)
```

```
    assert sizes == [1, 2, 3, 5, 5]
    # corners do not connect cells
    assert island_counts([[1, 0], [0, 1]], []) == [2]
    # the grid may be huge: only land cells are stored
    big = Islands()
    big.add(10 ** 9, 10 ** 9)
    big.add(10 ** 9, 10 ** 9 + 1)
    assert big.count == 1 and big.largest == 2
```

FUNCTION AND TEST CASES

Signature

```
def island_counts(grid: list[list[int]], additions: list[l
ist[int]]) -> list[int]:
```

Sample call

```
island_counts([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]],
[])
```

Test cases

Input	Expected
`([[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], [])`	`[3]`
`([[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0], [0, 1], [1, 2], [2, 1], [1, 1], [1, 1]])`	`[0, 1, 1, 2, 3, 1, 1]`

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

Site tags: graph, heap, algorithm, medium

MEDIUM | IN APP LC 778 | 1Point3Acres | onsite-coding | 2 of 21

Path Minimizing the Maximum Cell (Swim in Rising Water)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 55/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: your app's interview approach (LeetCode #778)

LeetCode match: LC 778 Swim in Rising Water

Frequency: Low | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

QUESTION

A bottleneck-path family: minimize the maximum value encountered on a grid path, then generalize the same reasoning to a weighted graph whose edges carry safety values and an admissibility threshold.

Grid variant

Input is a grid of heights and start/end cells; movement is through four-neighbors.
Cost of a path is the maximum cell value along it.
Return the minimum possible path cost.

Weighted-graph safety variant

Input is a graph whose edges carry safety values, a start node, an end node, and a safety threshold.
Find the lowest safety value required by any valid start-to-end path while every traversed edge remains under the supplied threshold.
Return -1 when no admissible path exists.
Clarify whether the threshold comparison is strict or inclusive before coding.

Mouse-and-cat safest-path variant (L4 phone screen)

Warm-up: an N x N grid with a start S, a target T, and impassable water cells; with 4-directional moves, decide whether any path from S to T exists.
Main question: the grid also contains a cat at C. The mouse moves only on land, 4-directionally, from S to the cheese at T. Find a path that maximizes the minimum Manhattan distance to the cat over all cells on the path.
This flips the objective (maximize a minimum instead of minimize a maximum), but it is the same bottleneck-path structure once each cell's distance to the cat is known.

Follow-up: Network Delay Time variant

In a directed graph with edge weights, return the maximum shortest-path distance from a source to all nodes, or -1 if any node is unreachable.

QUESTION

EXAMPLES

Grid

```
[ 0  1  2  3  4 ]
[24 23 22 21  5 ]
[12 13 14 15 16 ]
[11 17 18 19 20 ]
[10  9  8  7  6 ]
```

Start=(0,0), End=(4,0) in (row, col) — the bottom-left cell with value 10. Answer: 16 (the path's max cell is 16).

APPROACH

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, not the sum. That cost can only grow as the path grows, so Dijkstra still works if you replace 'distance + weight' by max(distance, weight): the first time a cell leaves the heap its cost is final. The cat version flips the objective (maximise a minimum), which is the same shape with a max-heap and each cell scored by its distance to the cat.

Basis: LeetCode 778, Swim in Rising Water (bottleneck path), plus the 'safest path from a cat' flip: maximise the minimum Manhattan distance.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, start, end):
    for t in sorted(set(all_values(grid))):
        if bfs_reaches(grid, start, end, t):
            return t  # smallest limit
```

TIME	SPACE
O(V^2)	O(V)

WHY IT IS SLOW

One full BFS per distinct value, O(V^2). Dijkstra on the running maximum finds the answer in one pass, O(V log V).

OPTIMAL SOLUTION · Dijkstra on the running maximum

TIME	SPACE
O(V log V)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Maximise the minimum distance to a cat instead.

Same bottleneck shape with the objective flipped: score each cell by its Manhattan distance to the cat, run Dijkstra with a max-heap on the running minimum, and skip water. The same monotonicity argument holds (extending a path can only lower its minimum).

FOLLOW-UP 2 Why is Dijkstra correct for a max-along-path cost?

The cost max(path) never decreases when a path is extended, so a vertex's cheapest cost cannot be improved by going through a vertex that is popped later. That is the only property Dijkstra needs from its cost function (monotone, non-negative extension).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Path Minimizing the Maximum Cell (Swim in
# Rising Water)
#
# A bottleneck-path family: minimize the maximum value
# encountered on a grid path, then generalize the same
# reasoning to a weighted graph whose edges carry safety
# values and an admissibility threshold. Grid variant
# Input is a grid of heights and start/end cells; movement
# is through four-neighbors. Cost of a path is the maximum
# cell value along it. Return the minimum possible path
# cost. Weighted-graph safety variant Input is a graph
# whose edges carry safety values, a start node, an end
# node, and a safety threshold. Find the lowest safety
# value required by any valid start-to-end path while
# every traversed edge remains under the supplied
# threshold. Return -1 when no admissible path exists.
# Clarify whether the threshold comparison is strict or
# inclusive before coding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Maximise the minimum distance to a cat instead.
# 2. Why is Dijkstra correct for a max-along-path cost?
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, not the sum.
# That cost can only grow as the path grows, so Dijkstra
# still works if you replace 'distance + weight' by
# max(distance, weight): the first time a cell leaves the
# heap its cost is final. The cat version flips the
# objective (maximise a minimum), which is the same shape
# with a max-heap and each cell scored by its distance to
# the cat.
#
# BRUTE FORCE (say it first): Try every threshold and test
# reachability
# For each candidate value t in increasing order run a BFS
# using only cells <= t and stop at the first t that
# connects start to end. Correct, but it repeats a full
# search per value. Time O(V^2). Space O(V).


from collections import deque
import heapq
```

```
# BRUTE FORCE (say it first): Try every threshold and test
# reachability | time O(V^2)
def bottleneck_brute(g, start, end):
    n, m = len(g), len(g[0])
    for t in sorted({x for row in g for x in row}):
        if (
            g[start[0]][start[1]] > t
            or g[end[0]][end[1]] > t
        ):
            continue
        seen, q = {start}, deque([start])
        while q:
            r, c = q.popleft()
            if (r, c) == end:
                return t
            for a, b in (
                (r + 1, c),
                (r - 1, c),
                (r, c + 1),
                (r, c - 1),
            ):
                if (
                    0 <= a < n
                    and 0 <= b < m
                    and (a, b) not in seen
                    and g[a][b] <= t
                ):
                    seen.add((a, b))
                    q.append((a, b))
    return None


# HELPERS for the base question
INF = float("inf")


def around(r, c, n, m):
    # four-neighbour cells inside an n x m grid
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < n and 0 <= b < m:
            yield a, b


# BASE QUESTION (optimal): Dijkstra on the running maximum
```

```
# Idea: Heap entries are (cost so far, cell). Popping a
# cell fixes its cost; each neighbour gets max(cost,
# neighbour value). Stop when the end cell is popped. Skip
# stale heap entries.
def min_max_path(g, start, end):
    n, m = len(g), len(g[0])
    # best[cell] = smallest possible "highest cell so far"
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, (r, c) = heapq.heappop(heap)
        # skip outdated heap entries
        if cost > best[(r, c)]:
            continue
        if (r, c) == end:
            return cost
        for a, b in around(r, c, n, m):
            # path cost = highest cell so far or this cell
            nxt = max(cost, g[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, (a, b)))
    return None
```

```python
# VARIANT: weighted graph with a safety threshold
def min_bottleneck_edge(graph, s, t, limit):
    # graph[u] = [(v, safety)]; every edge must be < limit
    best = {s: 0}
    heap = [(0, s)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:
            continue
        if u == t:
            return cost
        for v, w in graph.get(u, []):
            # edges at or above the limit are not allowed
            if w >= limit:
                continue
            nxt = max(cost, w)
            if nxt < best.get(v, INF):
                best[v] = nxt
                heapq.heappush(heap, (nxt, v))
    # no admissible path
    return -1


# RELATED: network delay (plain Dijkstra)
def network_delay(n, edges, src):
    # plain Dijkstra: largest shortest distance, or -1
    graph = {}
    for u, v, w in edges:
        graph.setdefault(u, []).append((v, w))
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    # unreachable nodes make the answer -1
    return max(dist.values()) if len(dist) == n else -1


# FOLLOW-UP 1: Maximise the minimum distance to a cat
# instead.
# Answer: Same bottleneck shape with the objective
# flipped: score each cell by its Manhattan distance to
# the cat, run Dijkstra with a max-heap on the running
# minimum, and skip water.
def safest_path_score(grid, start, target, cat):
    # maximise the smallest cat distance ('#' is water)
    n, m = len(grid), len(grid[0])

    def dist(r, c):
        return abs(r - cat[0]) + abs(c - cat[1])

    best = {start: dist(*start)}
    heap = [(-best[start], start)]
    while heap:
        neg, (r, c) = heapq.heappop(heap)
        if -neg < best[(r, c)]:
            continue
        if (r, c) == target:
            return -neg
        for a, b in around(r, c, n, m):
            if grid[a][b] == "#":
                continue
            # the path is only as safe as its closest cell
            score = min(-neg, dist(a, b))
            if score > best.get((a, b), -1):
                best[(a, b)] = score
                heapq.heappush(heap, (-score, (a, b)))
    return None


if __name__ == "__main__":
    g = [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5],
         [12, 13, 14, 15, 16], [11, 17, 18, 19, 20],
         [10, 9, 8, 7, 6]]
    assert min_max_path(g, (0, 0), (4, 0)) == 16
    # start equals end: just that cell
    assert min_max_path(g, (2, 2), (2, 2)) == 14
    gr = {0: [(1, 5), (2, 9)], 1: [(2, 3)]}
    assert min_bottleneck_edge(gr, 0, 2, 10) == 5
    assert min_bottleneck_edge(gr, 0, 2, 5) == -1
    cat_grid = ["...", ".#.", "..."]
    cat, a, b = (0, 2), (0, 0), (2, 2)
    score = safest_path_score(cat_grid, a, b, cat)
    assert score == 2
    chain = [(0, 1, 2), (1, 2, 3)]
    assert network_delay(3, chain, 0) == 5
    assert network_delay(3, [(0, 1, 2)], 0) == -1
```

NOTES FROM CANDIDATES

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter.

Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

In the mouse-and-cat variant, each cell's score is a direct Manhattan distance to a single cat, so it can be computed per cell; clarify whether water cells block only the mouse or also affect distance, and whether S or T themselves count toward the minimum.

PREPARATION

Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation.
Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

Site tags: grid, bfs, medium

MEDIUM | IN APP LC 994 | 1Point3Acres | onsite-coding, phone-screen | 3 of 21

Rotten Oranges / Multi-Source BFS (taxis)

ASKED 2026-09-08 | 42 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 0** | **PRIORITY 67/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Also in your app: LeetCode #994 Rotting Oranges

LeetCode match: LC 994 Rotting Oranges

Frequency: Low | Last asked: 2026-09-08 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/rotten-oranges-multi-source-bfs-taxis

QUESTION

EXAMPLES

```
Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges
    variant): return number of minutes until all
    reachable 1s become 2 (4).
```

Taxi variant: for each empty cell, distance to nearest taxi.

APPROACH

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

HOW TO THINK ABOUT IT

'Spread outward from several starting points at the same time and count the rounds' is multi-source BFS: put every source in the queue before you start, and process the queue one round at a time. Rounds are minutes. Always count the fresh ones first so that unreachable oranges are detected at the end.

Basis: LeetCode 994, Rotting Oranges, and the same multi-source BFS skeleton as LeetCode 542 and 286.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    minutes = 0
    while True:
        fresh = fresh_next_to_rotten(grid)
        if not fresh:
            break
        for r, c in fresh:
            grid[r][c] = 2
        minutes += 1
    return -1 if any_fresh(grid) else minutes
```

TIME	SPACE
O(minutes * m * n)	O(m * n)

WHY IT IS SLOW

Each minute scans the entire grid, O(minutes * m * n). A queue of the newly rotten oranges touches each cell only once, O(m * n).

OPTIMAL SOLUTION · Multi-source BFS by minute

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Taxi variant: distance from every empty cell to the nearest taxi

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

FOLLOW-UP 2 Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

FOLLOW-UP 3 Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

FOLLOW-UP 4 A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Rotten Oranges / Multi-Source BFS (taxis)
#
# Examples: Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-
# oranges variant): return number of minutes until all
# reachable 1s become 2 (4). Taxi variant: for each empty
# cell, distance to nearest taxi.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Sources move or new taxis appear. How do you keep the
# distances fresh?
# 2. Why is multi-source BFS the same cost as single-
# source BFS?
# 3. Signal variant: a signal spreads from sources, how
# long until every target gets it?
# 4. A signal spreads from sources to targets: how long
# until every target is reached?
#
# INTERVIEW NOTES
# 'Spread outward from several starting points at the same
# time and count the rounds' is multi-source BFS: put
# every source in the queue before you start, and process
# the queue one round at a time. Rounds are minutes.
# Always count the fresh ones first so that unreachable
# oranges are detected at the end.
#
# BRUTE FORCE (say it first): Re-scan the whole grid every
# minute
# Each minute, find every fresh orange next to a rotten
# one and rot them all at once, until nothing changes.
# Easy to reason about and the oracle; each minute touches
# the entire grid. Time O(minutes * m * n). Space O(m *
# n).


from collections import deque


# BRUTE FORCE (say it first): Re-scan the whole grid every
# minute | time O(minutes * m * n)
def oranges_by_scanning(grid):
    g = [row[:] for row in grid]
    minutes = 0
    while True:
        rot = [
            (r, c)
            for r in range(len(g))
            for c in range(len(g[0]))
            if g[r][c] == 1
            and any(
                0 <= r + dr < len(g)
                and 0 <= c + dc < len(g[0])
                and g[r + dr][c + dc] == 2
                for dr, dc in (
                    (1, 0),
                    (-1, 0),
                    (0, 1),
                    (0, -1),
                )
            )
        ]
        if not rot:
            break
        for r, c in rot:
            g[r][c] = 2
        minutes += 1
    return -1 if any(1 in row for row in g) else minutes


# HELPERS for the base question
def multi_source_bfs(rows, cols, sources, open_cell):
    # distance from the nearest source to every cell
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c),
                       (r, c + 1), (r, c - 1)):
            inside = 0 <= nr < rows and 0 <= nc < cols
            if inside and (nr, nc) not in dist:
                if open_cell(nr, nc):
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
    return dist


# BASE QUESTION (optimal): Multi-source BFS by minute
# Idea: Push every rotten orange into the queue at minute
# 0 and expand level by level; each level is one minute.
# Count the fresh oranges and decrement as they rot, so
# the answer is -1 if any remain at the end.
def rotting_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    cells = [(r, c) for r in range(rows)
             for c in range(cols)]
    rotten = [x for x in cells if grid[x[0]][x[1]] == 2]
    fresh = [x for x in cells if grid[x[0]][x[1]] == 1]
    # rot spreads through any non-empty cell
    dist = multi_source_bfs(
        rows, cols, rotten, lambda r, c: grid[r][c] != 0)
    # a fresh orange that is never reached means -1
    if any(x not in dist for x in fresh):
        return -1
    return max((dist[x] for x in fresh), default=0)


# TAXI VARIANT: distance to the nearest taxi
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked
    rows, cols = len(grid), len(grid[0])
    taxis = [(r, c) for r in range(rows)
             for c in range(cols) if grid[r][c] == "T"]
    dist = multi_source_bfs(
        rows, cols, taxis, lambda r, c: grid[r][c] != "#")
    # None where no taxi can reach the cell
    return [[dist.get((r, c)) for c in range(cols)]
```

```
            for r in range(rows)]


# FOLLOW-UP 4: A signal spreads from sources to targets:
# how long until every target is reached?
# Answer: One multi-source BFS from the sources, then the
# answer is the largest distance over the targets (-1 if
# any target is unreachable, 0 if there are no targets).
# Per-source BFS would cost O(mn * sources).
def signal_time(rows, cols, sources, targets, blocked=()):
    # how long until every target hears the signal?
    dist = multi_source_bfs(
        rows, cols, list(sources),
        lambda r, c: (r, c) not in blocked)
    if any(t not in dist for t in targets):
        return -1
    # no targets means no waiting
    return max((dist[t] for t in targets), default=0)
```

```
if __name__ == "__main__":
    assert rotting_minutes([[2, 1, 1], [1, 1, 0],
                            [0, 1, 1]]) == 4
    # a fresh orange walled off by an empty cell
    assert rotting_minutes([[2, 0, 1]]) == -1
    assert rotting_minutes([[0, 2]]) == 0
    assert nearest_taxi(["T.#", "..."]) == [
        [0, 1, None], [1, 2, 3]]
    # signal from two sources to two targets
    assert signal_time(1, 5, [(0, 0), (0, 4)],
                       [(0, 1), (0, 2)]) == 2
    assert signal_time(1, 3, [(0, 0)], [(0, 2)],
                       blocked={(0, 1)}) == -1
    assert signal_time(1, 3, [(0, 0)], []) == 0
```

NOTES FROM CANDIDATES

BFS from all sources simultaneously is O(mn); per-source BFS is O(mn × #sources).
The moving-source follow-up trades exactness for amortized cost — common answer is periodic rebuild with bounded staleness.
For the signal variant, use the maximum source-distance over targets as the completion time; an unreachable target makes that candidate infeasible. Agree that no targets requires zero time. A correct baseline for destroying at most one blocker is to run multi-source BFS with no deletion, then once with each blocker made traversable, restoring it before the next trial. Minimize the completion time across feasible trials; return -1 if none is feasible. With B blockers on an R by C grid, this takes O((B + 1) * R * C) time and O(R * C) reusable auxiliary space. A BFS state that allows each path to break its own blocker does not enforce one shared destroyed blocker for all targets.

PREPARATION

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | PracHub | Technical Screen | 4 of 21

Count Clusters of 2D Points Within a Radius

POSTED 2026-06-18 | 124 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 40.7 | 30 DAYS 0 | PRIORITY 90/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (inferred from overview)

Posted on PracHub: 2026-06-18 (updated 2026-07-01)

Original: https://prachub.com/coding-questions/count-clusters-of-2d-points-within-a-radius

QUESTION

Overview: This question tests graph traversal skills, specifically identifying connected components among a set of 2D points using distance-based adjacency. It assesses the ability to model a geometric problem as a graph and apply DFS, BFS, or Union-Find, a common evaluation of algorithmic thinking in coding interviews. It also probes awareness of efficiency trade-offs when scaling to large point sets.

APPROACH

Union-find over points with distance <= r; count roots. Use squared distance; spatial grid hashing for speed.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the distance is Euclidean, whether the radius is inclusive (a distance of at most r connects), whether points are integers, and the number of points.

BRUTE FORCE TO SAY OUT LOUD

Union-Find over all pairs of points, comparing every pair. O(n^2) distance checks.

OPTIMAL IDEA

Compare squared distances to r squared (no square roots, exact for integers). For many points hash them into grid cells of side r so each point only checks its own and the eight neighboring cells; use Union-Find to merge and count the components with distinct roots. About O(n) expected for evenly spread points.

DRY-RUN THESE CASES

No points, one point, two points exactly r apart (connected if inclusive), a chain of points each close to the next (one cluster), duplicate points.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Dynamic points, a very large radius, or reporting the largest cluster.

PITFALLS

Using the cell size smaller than r so close points fall in non-adjacent cells. Comparing floating-point distances for equality.

HOW TO THINK ABOUT IT

Model it as a graph where points within r are neighbours and count connected components with Union-Find. The all-pairs check is O(n^2); to speed it up, hash points into square cells of side r so that only points in the same or an adjacent cell can be within r. Compare squared distances to r * r to avoid square roots.

Basis: Connected components of a distance graph (LeetCode 547 idea). Two points are connected when their Euclidean distance is at most r; a cluster is a connected component.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
parent = list(range(n))
def find(x):
    while parent[x] != x:
        x = parent[x]
    return x
for a, b in combinations(range(n), 2):
    if dist2(points[a], points[b]) <= r * r:
        parent[find(a)] = find(b)
clusters = len({find(i) for i in range(n)})
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n^2 / 2 distance checks. Hashing points into grid cells of side r means each point is compared only with points in its own and neighbouring cells.

OPTIMAL SOLUTION · Grid hashing with cells of side r, then Union-Find

TIME	SPACE
O(n) expected for evenly spread points	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Report the size of the largest cluster.

Count points per root after the unions (a Counter of find(i)). The unions are the same.

FOLLOW-UP 2 Points arrive one at a time.

Insert the point into its grid cell, union it with every point within r in the 9 surrounding cells, and keep the component count: components += 1 - (number of distinct roots merged).

FOLLOW-UP 3 Millions of points in 3D or higher.

The grid generalises (3^d neighbouring cells), but for high dimensions use a k-d tree or locality-sensitive hashing; or DBSCAN, which is this problem with a minimum neighbour count. **Cost:** grid hashing visits 3^d neighbouring cells per point, O(n * 3^d); a k-d tree builds in O(n log n) with O(n^(1 - 1/d) + k) range queries.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Clusters of 2D Points Within a
# Radius
#
# This question tests graph traversal skills, specifically
# identifying connected components among a set of 2D
# points using distance-based adjacency. It assesses the
# ability to model a geometric problem as a graph and
# apply DFS, BFS, or Union-Find, a common evaluation of
# algorithmic thinking in coding interviews. It also
# probes awareness of efficiency trade-offs when scaling
# to large point sets.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Report the size of the largest cluster.
# 2. Points arrive one at a time.
# 3. Millions of points in 3D or higher.
#
# INTERVIEW NOTES
# Model it as a graph where points within r are neighbours
# and count connected components with Union-Find. The all-
# pairs check is O(n^2); to speed it up, hash points into
# square cells of side r so that only points in the same
# or an adjacent cell can be within r. Compare squared
# distances to r * r to avoid square roots.
#
# BRUTE FORCE (say it first): Union-Find over all pairs of
# points
# Compare every pair, union the ones within r, and count
# the roots. Correct and simple; the quadratic pair loop
# is the cost. Time O(n^2). Space O(n).


from collections import Counter, defaultdict


# BRUTE FORCE (say it first): Union-Find over all pairs of
# points | time O(n^2)
def _find_factory(n):
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    return parent, find


# HELPERS for the base question
# FOLLOW-UP 2: Points arrive one at a time.
# Answer: Insert the point into its grid cell, union it
# with every point within r in the 9 surrounding cells,
# and keep the component count: components += 1 - (number
# of distinct roots merged).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class Clusters:
    """Points arrive one by one; count the clusters."""

    def __init__(self, r):
        self.r = r
        # grid cell (side r) -> indices of points inside
        self.cells = defaultdict(list)
        self.points, self.parent = [], []
        self.components = 0

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def near(self, p, q):
        dx, dy = p[0] - q[0], p[1] - q[1]
        return dx * dx + dy * dy <= self.r * self.r

    def add(self, x, y):
        i = len(self.points)
        self.points.append((x, y))
        self.parent.append(i)
        self.components += 1
        size = self.r or 1
        cx, cy = x // size, y // size
        self.cells[(cx, cy)].append(i)
        # look only at the 9 cells around the new point
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                key = (cx + dx, cy + dy)
                cell = self.cells.get(key, ())
                for j in cell:
                    q = self.points[j]
                    if j != i and self.near((x, y), q):
                        a, b = self.find(i), self.find(j)
                        if a != b:
                            self.parent[a] = b
                            self.components -= 1
        return self.components

    def largest(self):
        sizes = Counter(self.find(i)
                        for i in range(len(self.points)))
        return max(sizes.values(), default=0)


def clusters_all_pairs(points, r):
    parent, find = _find_factory(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if (points[i][0] - points[j][0]) ** 2 + (
                points[i][1] - points[j][1]
```

```
            ) ** 2 <= r * r:
                parent[find(i)] = find(j)
    return len({find(i) for i in range(len(points))})


# BASE QUESTION (optimal): Grid hashing with cells of side
# r, then Union-Find
# Idea: Put each point in the cell (x // r, y // r). A
# point within distance r of another can only be in the
# same cell or one of the 8 neighbouring cells, so compare
# only with those.
def count_clusters(points, r):
    c = Clusters(r)
    for x, y in points:
        c.add(x, y)
    return c.components


# FOLLOW-UP 1: Report the size of the largest cluster.
# Answer: Count points per root after the unions (a
# Counter of find(i)). The unions are the same.
def largest_cluster(points, r):
    c = Clusters(r)
    for x, y in points:
```

```
        c.add(x, y)
    return c.largest()


if __name__ == "__main__":
    line = [(0, 0), (3, 0), (6, 0), (20, 20)]
    assert count_clusters(line, 3) == 2
    assert largest_cluster(line, 3) == 3
    assert count_clusters([], 3) == 0
    # online: the count changes as points arrive
    c = Clusters(2)
    assert [c.add(*p) for p in [(0, 0), (10, 10), (1, 1),
                                (5, 5)]] == [1, 2, 2, 3]
    # (3, 3) is alone; (4, 4) then bridges it to (5, 5)
    assert c.add(3, 3) == 4
    assert c.add(4, 4) == 3
    assert count_clusters([(1, 1), (1, 1)], 0) == 1
```

FUNCTION AND TEST CASES

Signature

```
def countClusters(points: list[list[int]], r: int) -> int:
```

Sample call

```
countClusters([[0, 0], [1, 0], [10, 10]], 2)
```

Test cases

Input	Expected
([[0, 0], [1, 0], [10, 10]], 2)	2
([[0, 0], [3, 0], [6, 0]], 3)	1

ALSO REPORTED · 2 more

Count Connected Clusters of Two-Dimensional Points | PracHub | Technical Screen

https://prachub.com/coding-questions/count-connected-clusters-of-two-dimensional-points

2D Points Connected Components by Radius | 1Point3Acres | phone-screen | asked 2026-08-04 | Low

ALSO REPORTED · 2 more

https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

DFS, BFS, or Union-Find all fit the core connected-components model.
For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.
Clarify squared-distance comparison to avoid unnecessary square roots.

Preparation: Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 127 | PracHub | Technical Screen | 5 of 21

Find shortest relationship path using BFS

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 41.8 | 30 DAYS 0 | PRIORITY 87/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #127 Word Ladder

LeetCode match: LC 127 Word Ladder (BFS shortest path) (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/find-shortest-relationship-path-using-bfs

QUESTION

Overview: This question evaluates proficiency in graph representations and traversal algorithms—specifically BFS and adjacency-structure construction—for computing unweighted shortest paths and reasoning about reachability.

APPROACH

Build an adjacency list, BFS from the source with a parent map, then reconstruct the path; O(V+E). Consider bidirectional BFS.

HOW TO THINK ABOUT IT

Unweighted means every hop costs the same, so breadth-first search finds the fewest hops: it explores all people one hop away, then two hops, and the first time it reaches the target is the shortest path. Keep a parent map to rebuild the path and a visited set so nobody is queued twice.

Basis: LeetCode 127 (Word Ladder) idea: shortest path in an unweighted graph of people. Return the list of people from source to target, or None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = None
def dfs(node, path):
    global best
    if node == target:
        if not best or len(path) < len(best):
            best = list(path)
        return
    for nxt in graph[node]:
        if nxt not in path:
            dfs(nxt, path + [nxt])
dfs(source, [source])
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

The number of simple paths is exponential. Breadth-first search reaches the target along a shortest path first, O(V + E), and bidirectional BFS explores far fewer nodes on big graphs.

OPTIMAL SOLUTION · Breadth-first search with a parent map

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is huge (a social network).

Bidirectional BFS searches from both ends and stops when the frontiers meet; with branching factor b and distance d it explores about 2 * b^(d/2) nodes instead of b^d.

FOLLOW-UP 2 Return every shortest path, or count them.

Keep, for each node, its distance and a list of all parents at that distance; counting is a DP over the BFS layers, and listing the paths is a DFS over the parent lists.

FOLLOW-UP 3 Relationships have different strengths (weights).

BFS is no longer right: use Dijkstra with a heap for non-negative weights. Keep the same parent map to rebuild the path.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find shortest relationship path using BFS
#
# This question evaluates proficiency in graph
# representations and traversal algorithms—specifically
# BFS and adjacency-structure construction—for computing
# unweighted shortest paths and reasoning about
# reachability.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is huge (a social network).
# 2. Return every shortest path, or count them.
# 3. Relationships have different strengths (weights).
#
# INTERVIEW NOTES
# Unweighted means every hop costs the same, so breadth-
# first search finds the fewest hops: it explores all
# people one hop away, then two hops, and the first time
```

```
# it reaches the target is the shortest path. Keep a
# parent map to rebuild the path and a visited set so
# nobody is queued twice.
#
# BRUTE FORCE (say it first): Depth-first search over all
# simple paths
# Try every simple path from the source and keep the
# shortest that reaches the target. Correct and the
# oracle; exponential in the size of the graph. Time
# exponential. Space O(path).


import heapq
from collections import defaultdict, deque


# BRUTE FORCE (say it first): Depth-first search over all
# simple paths | time exponential
def shortest_relationship_dfs(adj, s, t):
    best = [None]

    def go(u, path):
        if best[0] is not None and len(path) >= len(
            best[0]
        ):
            return
        if u == t:
            best[0] = list(path)
            return
        for v in adj.get(u, ()):
            if v not in path:
                path.append(v)
                go(v, path)
                path.pop()

    go(s, [s])
    return best[0]


# BASE QUESTION (optimal): Breadth-first search with a
# parent map
# Idea: Queue the source and mark it visited. Pop a
# person, queue each unvisited neighbour with parent =
# this person, and stop when the target is queued. Follow
# parents back to the source and reverse to get the path.
def shortest_path(adj, s, t):
    # BFS: the first time we reach a node is by a shortest
    # path
    if s == t:
        return [s]
    parent, queue = {s: None}, deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in parent:
                parent[v] = u
                if v == t:
                    # walk the parent links back to s
                    path = [v]
                    while parent[path[-1]] is not None:
                        path.append(parent[path[-1]])
                    return path[::-1]
                queue.append(v)
    return None


# FOLLOW-UP 1: The graph is huge (a social network).
# Answer: Bidirectional BFS searches from both ends and
# stops when the frontiers meet; with branching factor b
# and distance d it explores about 2 * b^(d/2) nodes
# instead of b^d.
def bidirectional(adj, s, t):
    # a huge graph: search from both ends, expand the
    # smaller
    # side, stop when the searches meet
    if s == t:
        return 0
    dist_s, dist_t = {s: 0}, {t: 0}
    front_s, front_t = [s], [t]
    while front_s and front_t:
        if len(front_s) > len(front_t):
            front_s, front_t = front_t, front_s
            dist_s, dist_t = dist_t, dist_s
        nxt = []
        for u in front_s:
            for v in adj.get(u, ()):
                if v in dist_t:
                    return dist_s[u] + 1 + dist_t[v]
                if v not in dist_s:
                    dist_s[v] = dist_s[u] + 1
                    nxt.append(v)
        front_s = nxt
    return None


# FOLLOW-UP 2: Return every shortest path, or count them.
# Answer: Keep, for each node, its distance and a list of
# all parents at that distance; counting is a DP over the
# BFS layers, and listing the paths is a DFS over the
# parent lists.
def all_shortest_paths(adj, s, t):
    # every shortest path: remember ALL parents at the
    # best level
    dist, queue = {s: 0}, deque([s])
    parents = defaultdict(list)
    while queue:
        u = queue.popleft()
        for v in adj.get(u, ()):
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
            if dist[v] == dist[u] + 1:
                parents[v].append(u)
    if t not in dist:
        return []
    paths = []

    def build(node, tail):
        if node == s:
            paths.append([s] + tail)
            return
        for p in parents[node]:
            build(p, [node] + tail)

    build(t, [])
    return paths


# FOLLOW-UP 3: Relationships have different strengths
# (weights).
# Answer: BFS is no longer right: use Dijkstra with a heap
# for non-negative weights. Keep the same parent map to
# rebuild the path.
def weighted_path(graph, s, t):
    # strengths differ: Dijkstra, same parent map
    dist, parent = {s: 0}, {s: None}
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, ()):
            if d + w < dist.get(v, float("inf")):
```

```
                dist[v], parent[v] = d + w, u
                heapq.heappush(heap, (d + w, v))
    if t not in dist:
        return None
    path = [t]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    return dist[t], path[::-1]


if __name__ == "__main__":
    adj = {"a": ["b", "c"], "b": ["d"], "c": ["d"],
           "d": ["e"], "e": []}
    assert shortest_path(adj, "a", "e") in (
        ["a", "b", "d", "e"], ["a", "c", "d", "e"])
    assert shortest_path(adj, "a", "a") == ["a"]
    assert shortest_path(adj, "e", "a") is None
    both_ways = {"a": ["b", "c"], "b": ["a", "d"],
                 "c": ["a", "d"], "d": ["b", "c", "e"],
                 "e": ["d"]}
```

```
    assert bidirectional(both_ways, "a", "e") == 3
    assert bidirectional(both_ways, "a", "a") == 0
    paths = all_shortest_paths(adj, "a", "e")
    assert sorted(paths) == [["a", "b", "d", "e"],
                             ["a", "c", "d", "e"]]
    graph = {"a": [("b", 1), ("c", 5)], "b": [("c", 1)]}
    cheapest = weighted_path(graph, "a", "c")
    assert cheapest == (2, ["a", "b", "c"])
    assert weighted_path(graph, "c", "a") is None
```

FUNCTION AND TEST CASES

Signature

```
def solution(relationships: list[list[int]], start: int, t
arget: int) -> int:
```

Sample call

```
solution([['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D')
```

Test cases

Input	Expected
`[['A', 'B'], ['B', 'C'], ['C', 'D']], 'A', 'D'`	`3`
`[[1, 2], [2, 5], [1, 3], [3, 4], [4, 5]], 1, 5`	`2`

ALSO REPORTED · 1 more

Degrees of Friendship | Exponent

https://www.aced.io/questions/3628/degrees-of-friendship

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 261 | PracHub | Technical Screen | 6 of 21

Validate parent array forms a tree

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 29 | 30 DAYS 0 | PRIORITY 128/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #261 Graph Valid Tree

LeetCode match: LC 261 Graph Valid Tree

Posted on PracHub: 2026-02-11 (updated 2026-04-23)

Original:
https://prachub.com/coding-questions/validate-parent-array-forms-a-tree

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2.
No cycles (following parent pointers from any node must eventually reach the root). 3.
Connectivity (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

QUESTION

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.
3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints
1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

EXAMPLES

```
Input: [-1, 0, 0, 1, 1]
Expected Output: True
```

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

```
Input: [2, -1, 1, 2]
Expected Output: True
Explanation: Node 1 is the root. The paths are
    0->2->1, 2->1, and 3->2->1, so every node reaches
    the same root.
```

Hints

First count how many roots there are. A valid tree must have exactly one **`-1`** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

APPROACH

Exactly one root (parent = -1 or self), every other node has one parent; detect cycles by walking up with a visited set or union-find; all nodes must reach the root. O(n).

HOW TO THINK ABOUT IT

Exactly one -1 gives the root; then the rest is whether every node reaches it. Following parents from each node can loop (a cycle) or run out of range. Do it with memoised walks so every node is processed once: mark nodes on the current walk, and if the walk meets one of them again it is a cycle; if it meets a node already known to reach the root, all nodes on the walk reach it too.

Basis: LeetCode 261 (Graph Valid Tree) in parent-pointer form: exactly one root (-1), no cycles, every node reaches the root.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_tree(parent):
    n = len(parent)
    roots = [i for i in range(n)
             if parent[i] == -1]
    if len(roots) != 1:
        return False
    if any(p >= n for p in parent):
        return False
    for node in range(n):
        cur, steps = node, 0
        while cur != -1 and steps <= n:
            cur = parent[cur]
            steps += 1
        if cur != -1:
            return False  # a cycle
    return True
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Every node walks its whole chain, O(n^2). Marking nodes as 'on this walk' or 'verified' lets every node be processed once, O(n).

OPTIMAL SOLUTION · One root, then colour each node once while walking up

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The graph is given as undirected edges (LeetCode 261).

A tree has exactly n - 1 edges and no cycle: check the edge count, then union endpoints and fail on an edge that joins two nodes already connected. O(n alpha(n)).

FOLLOW-UP 2 Report the reason it is invalid.

Return which rule failed: number of roots, an out-of-range parent, or the node where the cycle was detected (the node that was met twice on one walk).

FOLLOW-UP 3 A binary tree given as child arrays (LeetCode 1361).

Check that exactly one node has in-degree 0, every other node has in-degree 1, and that a traversal from the root visits all n nodes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Validate parent array forms a tree
#
# ## Problem You are given an integer array parent of
# length n describing a directed parent pointer for each
# node i (nodes are labeled 0..n-1). - parent[i] is the
# parent of node i. - Exactly one node should be the root,
# indicated by parent[root] = -1. Determine whether these
# n nodes form a **valid rooted tree**. A valid rooted
# tree must satisfy all of the following: 1. **Exactly one
# root** (exactly one index i with parent[i] = -1). 2.
# **No cycles** (following parent pointers from any node
# must eventually reach the root). 3. **Connectivity**
# (every node is reachable from the root; equivalently,
# every node has exactly one simple path to the root). ##
# Input - parent: integer array of length n where each
# value is either -1 or in [0, n-1]. ## Output - Return
# true if parent represents a valid rooted tree; otherwise
# return false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The graph is given as undirected edges (LeetCode
# 261).
# 2. Report the reason it is invalid.
# 3. A binary tree given as child arrays (LeetCode 1361).
#
# INTERVIEW NOTES
# Exactly one -1 gives the root; then the rest is whether
# every node reaches it. Following parents from each node
# can loop (a cycle) or run out of range. Do it with
# memoised walks so every node is processed once: mark
# nodes on the current walk, and if the walk meets one of
# them again it is a cycle; if it meets a node already
# known to reach the root, all nodes on the walk reach it
# too.
#
# BRUTE FORCE (say it first): Follow the parent chain from
# every node, up to n steps
# From each node follow parents for at most n steps; if
# you do not reach the root in n steps there is a cycle.
# Correct and the oracle; each node walks its whole chain
# separately. Time O(n^2). Space O(1).


from collections import defaultdict
```

```
# BRUTE FORCE (say it first): Follow the parent chain from
# every node, up to n steps | time O(n^2)
def valid_tree_walk(parent):
    n = len(parent)
    roots = [i for i, p in enumerate(parent) if p == -1]
    if len(roots) != 1 or any(
        p != -1 and not (0 <= p < n) for p in parent
    ):
        return False
    for i in range(n):
        x, steps = i, 0
        while parent[x] != -1 and steps <= n:
            x = parent[x]
            steps += 1
        if parent[x] != -1:
            return False
    return True


# HELPERS for the base question
```

```
# FOLLOW-UP 2: Report the reason it is invalid.
# Answer: Return which rule failed: number of roots, an
# out-of-range parent, or the node where the cycle was
# detected (the node that was met twice on one walk).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def reason(parent):
    # None if valid, else the first rule that fails
    n = len(parent)
    roots = parent.count(-1)
    if roots != 1:
        return f"{roots} roots (need exactly 1)"
    for i, p in enumerate(parent):
        if p != -1 and not 0 <= p < n:
            return f"node {i}: out-of-range parent {p}"
        if p == i:
            return f"node {i} is its own parent"
    # 0 = unseen, 1 = on the current walk, 2 = checked
    state = [0] * n
    for start in range(n):
        path, x = [], start
        while x != -1 and state[x] == 0:
            state[x] = 1
            path.append(x)
            x = parent[x]
        # meeting a node twice on one walk is a cycle
        if x != -1 and state[x] == 1:
            return f"cycle through node {x}"
        for v in path:
            state[v] = 2
    return None


# BASE QUESTION (optimal): One root, then colour each node
# once while walking up
# Idea: Check there is exactly one -1 and all other values
# are in range. Then walk up from each unvisited node
# marking nodes as 'on this walk'.
def validate_parent_array(parent):
    return reason(parent) is None


# FOLLOW-UP 1: The graph is given as undirected edges
# (LeetCode 261).
# Answer: A tree has exactly n - 1 edges and no cycle:
# check the edge count, then union endpoints and fail on
# an edge that joins two nodes already connected. O(n
# alpha(n)).
def valid_tree_edges(n, edges):
    # LeetCode 261: undirected edges
    if len(edges) != n - 1:
        return False
    root = list(range(n))

    def find(x):
        while root[x] != x:
            root[x] = root[root[x]]
            x = root[x]
        return x

    for u, v in edges:
        a, b = find(u), find(v)
        # an edge inside a group closes a cycle
        if a == b:
            return False
        root[a] = b
    return True


# FOLLOW-UP 3: A binary tree given as child arrays
# (LeetCode 1361).
# Answer: Check that exactly one node has in-degree 0,
# every other node has in-degree 1, and that a traversal
# from the root visits all n nodes.
def valid_binary_tree(n, left, right):
    # LeetCode 1361: child arrays, -1 means no child
    indeg = [0] * n
    for kids in (left, right):
        for c in kids:
            if c != -1:
                indeg[c] += 1
    roots = [i for i in range(n) if indeg[i] == 0]
    if len(roots) != 1 or max(indeg) > 1:
        return False
    seen, stack = {roots[0]}, [roots[0]]
    while stack:
        u = stack.pop()
        for c in (left[u], right[u]):
            if c != -1:
                if c in seen:
                    return False
                seen.add(c)
                stack.append(c)
    # every node must be reachable from the root
    return len(seen) == n


if __name__ == "__main__":
    assert validate_parent_array([-1, 0, 0, 1])
    assert validate_parent_array([-1])
    # two roots, none, and a cycle 1 -> 2 -> 1
    assert not validate_parent_array([-1, -1])
    assert not validate_parent_array([1, 0])
    assert "cycle" in reason([-1, 2, 1])
    assert not validate_parent_array([-1, 5])
    assert "out-of-range" in reason([-1, 5])
    assert "own parent" in reason([-1, 1])
    star = [(0, 1), (0, 2), (0, 3), (1, 4)]
    assert valid_tree_edges(5, star)
    loop = [(0, 1), (1, 2), (2, 0)]
    assert not valid_tree_edges(4, loop)
    left, right = [1, -1, 3, -1], [2, -1, -1, -1]
    assert valid_binary_tree(4, left, right)
    assert not valid_binary_tree(2, [1, 0], [-1, -1])
```

FUNCTION AND TEST CASES

Signature

```
def solution(parent: list[int]) -> bool:
SAMPLE INPUT
[-1, 0, 0, 1, 1]
```

Test cases

Input	Expected
[-1, 0, 0, 1, 1]	True
[2, -1, 1, 2]	True

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | PracHub | Technical Screen | 7 of 21

Solve matrix groups and recipe inventory

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 33.8 | 30 DAYS 0 | PRIORITY 113/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #2115 Find All Possible Recipes from Given Supplies

LeetCode match: No direct match; related LC 2115 Find All Possible Recipes from Given Supplies (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-12)

Original: https://prachub.com/coding-questions/solve-matrix-groups-and-recipe-inventory

QUESTION

Overview: This pair of problems evaluates block-wise grid processing and aggregation for minimum-value selection alongside dependency-resolution and resource-allocation reasoning for recipe inventory planning, testing competencies in array manipulation, tie-breaking and constraint handling, graph-based dependency management and cycle detection, and algorithmic complexity analysis; they are commonly asked to gauge an applicant's ability to implement correct, efficient solutions under input-size and edge-case constraints. They belong to the Coding & Algorithms domain with subdomains in data structures (arrays/grids), algorithm design and analysis, and graph theory, and primarily require practical application of algorithmic techniques with attention to time/space complexity rather than purely theoretical proofs.

APPROACH

Part 1: iterate block by block and take the min per group. Part 2: topological sort/DFS over recipe dependencies with an inventory map.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Matrix groups: how the grid is split into blocks (group size) and what is taken per block (the minimum). Recipes: can a recipe be made from supplies, do recipes need other recipes, are amounts counted or only availability (LeetCode 2115 is availability), how are cycles treated.

BRUTE FORCE TO SAY OUT LOUD

Matrix: scan each block directly. Recipes: recursive search from each recipe without memory.

OPTIMAL IDEA

Matrix: iterate block by block with nested loops and take the minimum of each block, O(mn). Recipes: build a graph from ingredients to recipes with in-degrees and run Kahn's algorithm starting from the supplies; a recipe becomes available when all its ingredients are; recipes on a cycle never become available. If quantities matter, DFS with memoization over recipes tracking inventory, and decide how to share a scarce ingredient between recipes.

DRY-RUN THESE CASES

A grid not divisible by the block size, a recipe whose ingredient is another recipe, a cycle between two recipes, a missing ingredient, an empty supply list.

LIKELY FOLLOW-UPS

Amounts per ingredient (resource allocation), the maximum number of each recipe, or the cheapest set of recipes.

PITFALLS

Treating availability as quantities. Not handling a cycle.

HOW TO THINK ABOUT IT

Blocks: visit each cell once and update the minimum of its block, indexed by (row // k, col // k). Recipes: this is a dependency graph; a recipe is makeable when all its ingredients are supplies or makeable recipes. Count unmet ingredients per recipe and process recipes as they become available (Kahn's algorithm); recipes on a cycle never become available.

Basis: Two parts (my copy has only the overview, so the rules are assumed): take the minimum of every k x k block of a matrix, and decide which recipes can be made from supplies where recipes may need other recipes (LeetCode 2115, Find All Possible Recipes).

BRUTE FORCE · Blocks: collect each block's values and call min

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid, h, w):
    answers = []
    for block in blocks(grid, h, w):
        cells = []
        for r, c in block:
            cells.append(grid[r][c])
        answers.append(min(cells))
    return answers
```

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

Copies every block into a list. One pass that updates the minimum of cell (r // k, c // k) needs no copies.

BRUTE FORCE · Recipes: keep re-scanning until no recipe changes

BRUTE FORCE · PSEUDO-CODE

```
available = set(supplies)
while True:
    added = False
    for recipe in recipes:
        if recipe.name in available:
            continue
        needs = recipe.needs
        if all(i in available for i in needs):
            available.add(recipe.name)
            added = True
    if not added:
        break
```

TIME	SPACE
O(R^2 * I)	O(R)

WHY IT IS SLOW

Many passes over all recipes, O(R^2 * I). Counting each recipe's missing ingredients and processing items as they become available (Kahn's algorithm) is O(R + I).

OPTIMAL SOLUTION · Blocks: one pass updating a minimum per block

TIME	SPACE
O(m * n)	O(blocks)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Recipes: Kahn's algorithm on the ingredient graph

TIME	SPACE
O(R + I)	O(R + I)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Ingredients have quantities and a shared ingredient can only be used once.

Then availability is not enough: you also have to decide the order in which to spend scarce items. A priority order with a simulation (as coded) is a heuristic; the exact maximum is a resource-allocation problem (flow or integer programming).

FOLLOW-UP 2 Blocks that do not tile the matrix exactly.

The last block row or column is smaller; the single-pass update handles that automatically because the block index is r // k and c // k.

FOLLOW-UP 3 Detect which recipes are impossible because of a cycle.

After Kahn's algorithm, recipes whose counts never reached zero are either missing an ingredient or on a cycle; a DFS with colours over just those recipes tells the two cases apart.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve matrix groups and recipe inventory
#
# This pair of problems evaluates block-wise grid
# processing and aggregation for minimum-value selection
# alongside dependency-resolution and resource-allocation
# reasoning for recipe inventory planning, testing
# competencies in array manipulation, tie-breaking and
# constraint handling, graph-based dependency management
# and cycle detection, and algorithmic complexity
# analysis; they are commonly asked to gauge an
# applicant's ability to implement correct, efficient
# solutions under input-size and edge-case constraints.
# They belong to the Coding & Algorithms domain with
# subdomains in data structures (arrays/grids), algorithm
# design and analysis, and graph theory, and primarily
# require practical application of algorithmic techniques
# with attention to time/space complexity rather than
# purely theoretical proofs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Ingredients have quantities and a shared ingredient
# can only be used once.
# 2. Blocks that do not tile the matrix exactly.
# 3. Detect which recipes are impossible because of a
# cycle.
#
# INTERVIEW NOTES
# Blocks: visit each cell once and update the minimum of
# its block, indexed by (row // k, col // k). Recipes:
# this is a dependency graph; a recipe is makeable when
# all its ingredients are supplies or makeable recipes.
# Count unmet ingredients per recipe and process recipes
# as they become available (Kahn's algorithm); recipes on
# a cycle never become available.
#
# BRUTE FORCE (say it first): Blocks: collect each block's
# values and call min
# For every block gather its cells into a list and take
# the minimum. Correct and the oracle; it copies every
# block. Time O(m * n). Space O(m * n).


from collections import defaultdict, deque
```

```
# BRUTE FORCE (say it first): Blocks: collect each block's
# values and call min | time O(m * n)
def block_mins_collect(matrix, k):
    m, n = len(matrix), len(matrix[0])
    return [
        [
            min(
                matrix[r][c]
                for r in range(i, min(i + k, m))
                for c in range(j, min(j + k, n))
            )
            for j in range(0, n, k)
```

```
        ]
        for i in range(0, m, k)
    ]


# BASE QUESTION (optimal): Blocks: one pass updating a
# minimum per block
# Idea: Each cell updates the minimum of block (r // k, c
# // k). No block list is built; the last blocks may be
# smaller when k does not divide the size.
def block_mins(matrix, k):
    # the minimum of every k x k block; edge blocks may be
    # smaller, which r // k and c // k handle by
    # themselves
    m, n = len(matrix), len(matrix[0])
    rows, cols = (m + k - 1) // k, (n + k - 1) // k
    out = [[float("inf")] * cols for _ in range(rows)]
    for r in range(m):
        for c in range(n):
            cell = out[r // k][c // k]
            out[r // k][c // k] = min(cell, matrix[r][c])
    return out


# PART 2: which recipes can be made
def possible_recipes(recipes, supplies):
    # recipes: name -> list of ingredients (which may be
    # other
    # recipes). Kahn's algorithm over missing-ingredient
    # counts.
    have = set(supplies)
    missing, users = {}, defaultdict(list)
    for name, items in recipes.items():
```

```
        missing[name] = 0
        for item in set(items):
            if item not in have:
                missing[name] += 1
                users[item].append(name)
    queue = deque(r for r, m in missing.items() if m == 0)
    made = []
    while queue:
        name = queue.popleft()
        made.append(name)
        # a finished recipe is a new ingredient for its
        # users
        for other in users[name]:
            missing[other] -= 1
            if missing[other] == 0:
                queue.append(other)
    return made


# FOLLOW-UP 1: Ingredients have quantities and a shared
# ingredient can only be used once.
# Answer: Then availability is not enough: you also have
# to decide the order in which to spend scarce items.
def recipes_with_quantities(recipes, needs, stock):
    # ingredients have amounts and stock is used up. Make
    # each recipe at most once, in priority order, while
    # the
    # stock allows (a made recipe is new stock). This is a
    # heuristic: the exact best choice is a flow problem.
    stock, made = dict(stock), []
    progress = True
    while progress:
        progress = False
        for name, need in zip(recipes, needs):
            if name in made:
                continue
            enough = all(stock.get(x, 0) >= q
                         for x, q in need.items())
            if enough:
```

```
                for x, q in need.items():
                    stock[x] -= q
                stock[name] = stock.get(name, 0) + 1
                made.append(name)
                progress = True
    return made
```

```


# FOLLOW-UP 3: Detect which recipes are impossible because
# of a cycle.
# Answer: After Kahn's algorithm, recipes whose counts
# never reached zero are either missing an ingredient or
# on a cycle; a DFS with colours over just those recipes
# tells the two cases apart.
def why_impossible(recipes, supplies):
    # for each recipe that cannot be made: a missing raw
    # ingredient, or a cycle
    made = set(possible_recipes(recipes, supplies))
    known = set(supplies) | set(recipes)
    reasons = {}
    for name, items in recipes.items():
        if name in made:
            continue
        gone = [i for i in items if i not in known]
        reasons[name] = "cycle"
        if gone:
            reasons[name] = "missing " + gone[0]
    return reasons


if __name__ == "__main__":
    matrix = [[5, 3, 8, 1], [2, 9, 4, 7], [6, 0, 3, 2]]
    # 2 x 2 blocks, the last row of blocks is only 1 row
    # tall
    assert block_mins(matrix, 2) == [[2, 1], [0, 2]]
    assert block_mins([[4]], 3) == [[4]]
    recipes = {"bread": ["flour", "yeast"],
               "sandwich": ["bread", "meat"],
               "burger": ["bread", "meat", "bun"]}
    stock = ["flour", "yeast", "meat"]
    got = possible_recipes(recipes, stock)
    assert got == ["bread", "sandwich"]
    # two recipes that need each other, and a missing item
    cyc = {"a": ["b"], "b": ["a"], "c": ["x"]}
    assert possible_recipes(cyc, []) == []
    assert why_impossible(cyc, []) == {"a": "cycle",
                                       "b": "cycle",
                                       "c": "missing x"}
    # amounts: one bread uses the only 2 flour, so the
    # cake
    # (which needs 2 flour too) cannot be made
```

```
    names = ["bread", "cake"]
    needs = [{"flour": 2}, {"flour": 2}]
    two = {"flour": 2}
    made = recipes_with_quantities(names, needs, two)
    assert made == ["bread"]
    four = {"flour": 4}
    both = recipes_with_quantities(names, needs, four)
    assert both == ["bread", "cake"]
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[int]:
```

Sample call

```
solution([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9,8,2,5],[10,11
,12,0,3,4],[9,8,7,6,5,4],[3,2,9,7,8,9]])
```

Test cases

Input	Expected
([[5,4,3,9,8,7],[6,2,8,5,4,6],[7,3,9, 8,2,5],[10,11,12,0,3,4],[9,8,7,6,5,4] ,[3,2,9,7,8,9]],)	(0, 3, 3)
([[-1,5,6,-1,7,8],[9,10,11,12,13,14], [15,16,17,18,19,20],[21,22,23,24,25,2 6],[27,28,29,30,31,32],[33,34,35,36,3 7,38]],)	(-1, 0, 0)

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 542 | PracHub | Technical Screen | 8 of 21

Compute distance to nearest taxi in grid

POSTED 2026-01-10 | 283 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 102/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #542 01 Matrix

LeetCode match: LC 542 01 Matrix (inferred from overview)

Posted on PracHub: 2026-01-10 (updated 2026-06-20)

Original: https://prachub.com/coding-questions/compute-distance-to-nearest-taxi-in-grid

QUESTION

Overview: This question evaluates understanding of graph traversal and shortest-path reasoning on grids, algorithmic problem-solving for computing distances from many targets, and the ability to manage time and space complexity on large inputs.

APPROACH

Multi-source BFS from all taxis simultaneously; the BFS level is the distance for each cell. O(mn).

HOW TO THINK ABOUT IT

Asking for the distance from every cell to its nearest source is one BFS started from all sources at once, not one BFS per cell. Put all taxis in the queue at distance 0; the first time the search reaches a cell is its shortest distance, because BFS visits cells in order of distance.

Basis: LeetCode 542, 01 Matrix, with 'T' for a taxi, '#' for a blocked cell and '.' for open ground. Distance counts moves through open cells (4-directional).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def nearest_taxi(grid, start):
    queue, seen = deque([(start, 0)]), {start}
    while queue:
        cell, d = queue.popleft()
        if is_taxi(grid, cell):
            return d
        for nxt in neighbours(grid, cell):
            if nxt not in seen:
                seen.add(nxt)
                queue.append((nxt, d + 1))
    return -1
answer = {}
for c in cells:
    answer[c] = nearest_taxi(grid, c)
```

BRUTE FORCE

TIME	SPACE
O((m * n)^2)	O(m * n)

WHY IT IS SLOW

A separate BFS per cell, O((m * n)^2). One BFS that starts from all taxis at once reaches every cell at its nearest-taxi distance, O(m * n).

OPTIMAL SOLUTION · One multi-source BFS

TIME	SPACE
O(m * n)	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A new taxi appears.

Start a BFS from the new taxi but only continue into cells where you improve the stored distance. The distances can only decrease, so the work is bounded by the cells whose answer actually changes.

FOLLOW-UP 2 Taxis move or disappear.

Distances can increase, which the decrease-only update cannot handle. Rebuild with one multi-source BFS (O(mn)), perhaps lazily every few updates and accepting bounded staleness between rebuilds.

FOLLOW-UP 3 Return which taxi is nearest, not only the distance.

Store a source id with each cell in the BFS: a cell copies the id of the cell that first reached it. Ties go to whichever reached it first, so say how you want ties broken.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute distance to nearest taxi in grid
#
# This question evaluates understanding of graph traversal
# and shortest-path reasoning on grids, algorithmic
# problem-solving for computing distances from many
# targets, and the ability to manage time and space
# complexity on large inputs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A new taxi appears.
# 2. Taxis move or disappear.
# 3. Return which taxi is nearest, not only the distance.
#
# INTERVIEW NOTES
```

```
# Asking for the distance from every cell to its nearest
# source is one BFS started from all sources at once, not
# one BFS per cell. Put all taxis in the queue at distance
# 0; the first time the search reaches a cell is its
# shortest distance, because BFS visits cells in order of
# distance.
#
# BRUTE FORCE (say it first): BFS from every cell to find
# its nearest taxi
# For each open cell run a BFS until the first taxi is
# met. Correct, but it repeats almost the same search from
# every cell. Time O((m * n)^2). Space O(m * n).


from collections import deque


# BRUTE FORCE (say it first): BFS from every cell to find
# its nearest taxi | time O((m * n)^2)
def taxi_distances_brute(grid):
    R, C = len(grid), len(grid[0])
    out = [[-1] * C for _ in range(R)]
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "#":
                continue
            seen, queue = {(r, c)}, deque([(r, c, 0)])
            while queue:
                x, y, d = queue.popleft()
                if grid[x][y] == "T":
                    out[r][c] = d
```

```
                    break
                for nx, ny in (
                    (x + 1, y),
                    (x - 1, y),
                    (x, y + 1),
                    (x, y - 1),
                ):
                    if (
                        0 <= nx < R
                        and 0 <= ny < C
                        and grid[nx][ny] != "#"
                        and (nx, ny) not in seen
                    ):
                        seen.add((nx, ny))
                        queue.append((nx, ny, d + 1))
    return out


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def spread(grid, dist, queue, better=False):
    # grow outward; with better=True a cell may be updated
    # when a
    # new, closer source reaches it (distances only go
    # down)
    R, C = len(grid), len(grid[0])
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if not (0 <= a < R and 0 <= b < C):
                continue
            if grid[a][b] == "#":
                continue
            new = dist[r][c] + 1
            unseen = dist[a][b] == -1
            if unseen or (better and new < dist[a][b]):
                dist[a][b] = new
                queue.append((a, b))
```

```
# BASE QUESTION (optimal): One multi-source BFS
# Idea: Initialise the queue with every taxi at distance 0
```

```
# and expand in rounds. A cell receives its distance the
# first time it is reached; blocked cells are never
# entered; cells that no taxi can reach stay at -1.
def taxi_distances(grid):
    # 'T' taxi, '.' road, '#' wall. One BFS that starts
    # from
    # every taxi at once gives each cell its nearest taxi.
    R, C = len(grid), len(grid[0])
    dist = [[-1] * C for _ in range(R)]
    queue = deque()
    for r in range(R):
        for c in range(C):
            if grid[r][c] == "T":
                dist[r][c] = 0
                queue.append((r, c))
    spread(grid, dist, queue)
    return dist


# FOLLOW-UP 1: A new taxi appears.
# Answer: Start a BFS from the new taxi but only continue
# into cells where you improve the stored distance. The
# distances can only decrease, so the work is bounded by
# the cells whose answer actually changes.
def add_taxi(grid, dist, r, c):
    # a new taxi: only cells it beats are touched
    dist[r][c] = 0
    spread(grid, dist, deque([(r, c)]), better=True)


# FOLLOW-UP 3: Return which taxi is nearest, not only the
# distance.
# Answer: Store a source id with each cell in the BFS: a
# cell copies the id of the cell that first reached it.
# Ties go to whichever reached it first, so say how you
# want ties broken.
def nearest_taxi_ids(grid):
    # which taxi is nearest? each cell copies the id of
    # the
    # cell that reached it first (ties: the first to
    # arrive)
    R, C = len(grid), len(grid[0])
    owner = [[None] * C for _ in range(R)]
    queue, taxi_id = deque(), 0
    for r in range(R):
```

```
        for c in range(C):
            if grid[r][c] == "T":
                owner[r][c] = taxi_id
                taxi_id += 1
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            inside = 0 <= a < R and 0 <= b < C
            if not inside or grid[a][b] == "#":
                continue
            if owner[a][b] is None:
                owner[a][b] = owner[r][c]
                queue.append((a, b))
    return owner


if __name__ == "__main__":
    grid = ["T...", ".#..", "...T"]
    d = taxi_distances(grid)
    # the second taxi at the bottom right is closer to (0,
```

```
    # 3)
    assert d[0] == [0, 1, 2, 2]
    assert d[2] == [2, 2, 1, 0]
    assert d[1][1] == -1
    # a new taxi in the middle of the top row
    add_taxi(grid, d, 0, 2)
    assert d[0] == [0, 1, 0, 1]
    assert d[2][0] == 2
    # unreachable cells stay -1
    walled = ["T#.", "##."]
    assert taxi_distances(walled)[0][2] == -1
    ids = nearest_taxi_ids(grid)
    assert ids[0][0] == 0 and ids[2][3] == 1
    assert ids[1][1] is None
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0,0,0],[0,1,0],[0,0,0]])
```

Test cases

Input	Expected
`([[0,0,0],[0,1,0],[0,0,0]],)`	`[[2,1,2],[1,0,1],[2,1,2]]`
`([[1,0,0,0],[0,0,0,1],[0,0,0,0]],)`	`[[0,1,2,1],[1,2,1,0],[2,3,2,1]]`

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | IN APP LC 207 | PracHub | Technical Screen | 9 of 21

★ TOP 75: rank 33 in the Medium and Hard priority list

★ Validate course catalog dependencies

POSTED 2025-08-01 | 445 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.9 | 30 DAYS 33 | PRIORITY 30/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #207 Course Schedule

LeetCode match: LC 207 Course Schedule

Posted on PracHub: 2025-08-01 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/validate-course-catalog-dependencies

QUESTION

Design a function to validate an e-learning course catalog. You are given: (1) a set of course IDs, and (2) a list of prerequisite pairs (u, v) meaning course v requires course u. Determine if the catalog is valid: no prerequisite references a non-existent course, and there are no cycles in the dependency graph. If valid, return true and a possible ordering of courses; otherwise return false and report the set of missing course references and at least one cycle you detect. Specify your algorithm, time and space complexity.

APPROACH

Check every prerequisite refers to a known course ID, then Kahn's topological sort or DFS coloring to detect cycles.

HOW TO THINK ABOUT IT

Two separate validity rules: every course mentioned must exist, and the dependency graph must have no cycle. The first is a set lookup per pair. For the second, Kahn's algorithm removes courses with no unmet prerequisites; anything left over is on a cycle or depends on one, and a DFS restricted to the leftovers reveals an actual cycle to report.

Basis: LeetCode 207 / 210 (Course Schedule) plus a check for references to unknown courses. A pair (u, v) means course v requires u.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def validate(courses, prereqs):
    for u, v in prereqs:
        if u not in courses or v not in courses:
            return False  # unknown id
    graph = build_graph(prereqs)
    for c in courses:
        if reaches(graph, c, c):
            return False  # cycle
    return True
```

TIME	SPACE
O(V * (V + E))	O(V)

WHY IT IS SLOW

Every course can trigger a walk of the whole graph, O(V * (V + E)). Kahn's algorithm is O(V + E) and its leftovers also reveal a cycle to report.

OPTIMAL SOLUTION · Missing references, then Kahn's algorithm, then a cycle from the leftovers

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return all missing course references and one cycle.

The function above returns the sorted unknown ids when any exist and otherwise a cycle (a list of courses starting and ending at the same course), found by walking backwards along prerequisites inside the set of courses Kahn's algorithm could not remove (every such course has a leftover prerequisite, so the walk cannot dead-end).

FOLLOW-UP 2 Group courses into semesters (the earliest round each can be taken).

Run Kahn's algorithm level by level: all courses with no remaining prerequisites form round 1, remove them, the next set forms round 2, and so on. The number of rounds is the longest dependency chain.

FOLLOW-UP 3 Find all cycles, not just one.

Compute strongly connected components (Tarjan or Kosaraju); every component with more than one course (or a self-loop) is a cycle group to report.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Validate course catalog dependencies
#
# Design a function to validate an e-learning course
# catalog. You are given: ( 1) a set of course IDs, and (
# 2) a list of prerequisite pairs (u, v) meaning course v
# requires course u. Determine if the catalog is valid: no
# prerequisite references a non-existent course, and there
# are no cycles in the dependency graph. If valid, return
# true and a possible ordering of courses; otherwise
# return false and report the set of missing course
# references and at least one cycle you detect. Specify
# your algorithm, time and space complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return all missing course references and one cycle.
# 2. Group courses into semesters (the earliest round each
# can be taken).
# 3. Find all cycles, not just one.
#
# INTERVIEW NOTES
# Two separate validity rules: every course mentioned must
# exist, and the dependency graph must have no cycle. The
# first is a set lookup per pair. For the second, Kahn's
# algorithm removes courses with no unmet prerequisites;
# anything left over is on a cycle or depends on one, and
# a DFS restricted to the leftovers reveals an actual
# cycle to report.
#
# BRUTE FORCE (say it first): DFS from every course
# looking for a cycle
# First collect references to unknown courses; then, for
# each course, search for a path back to itself. Correct
# and the oracle; the same parts of the graph are searched
# repeatedly. Time O(V * (V + E)). Space O(V).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): DFS from every course
# looking for a cycle | time O(V * (V + E))
def catalog_valid_brute(courses, prereqs):
    cs = set(courses)
    missing = sorted(
        {
            x
            for u, v in prereqs
            for x in (u, v)
            if x not in cs
        }
    )
    if missing:
        return False
    adj = defaultdict(list)
    for u, v in prereqs:
        adj[u].append(v)

    def reaches_self(start):
        seen, stack = set(), list(adj[start])
        while stack:
            x = stack.pop()
            if x == start:
                return True
            if x not in seen:
                seen.add(x)
                stack.extend(adj[x])
        return False

    return not any(reaches_self(c) for c in cs)


# HELPERS for the base question
# FOLLOW-UP 3: Find all cycles, not just one.
# Answer: Compute strongly connected components (Tarjan or
# Kosaraju); every component with more than one course (or
# a self-loop) is a cycle group to report.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def find_cycle(known, prereqs, indeg):
    # every leftover course still has a leftover
    # prerequisite,
    # so walking backwards must eventually repeat a course
    left = {c for c in known if indeg[c] > 0}
    before = defaultdict(list)
    for u, v in prereqs:
        if u in left and v in left:
            before[v].append(u)
    x = min(left, key=str)
    walk, pos = [], {}
    while x not in pos:
        pos[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    cycle = walk[pos[x]:] + [x]
    # walked against the edges: reverse to list
    # prerequisites first
    return cycle[::-1]


# BASE QUESTION (optimal): Missing references, then Kahn's
# algorithm, then a cycle from the leftovers
# Idea: Report unknown courses (any referenced id not in
# the set). If none, run Kahn's algorithm: repeatedly
# remove courses with no remaining prerequisites, building
# an order.
# FOLLOW-UP 1: Return all missing course references and
# one cycle.
# Answer: The function above returns the sorted unknown
# ids when any exist and otherwise a cycle (a list of
# courses starting and ending at the same course), found
# by walking backwards along prerequisites inside the set
# of courses Kahn's algorithm could not remove (every such
# course has a leftover prerequisite, so the walk cannot
# dead-end).
def validate_catalog(courses, prereqs):
    # prereqs (u, v): course v requires course u.
    # Returns (ok, missing ids, order or one cycle)
    known = set(courses)
    missing = sorted({x for pair in prereqs for x in pair
                      if x not in known}, key=str)
    if missing:
        return False, missing, None
    adj, indeg = defaultdict(list), {c: 0 for c in known}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    ready = [c for c in known if indeg[c] == 0]
    queue = deque(sorted(ready, key=str))
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in adj[c]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0:
```

```
                queue.append(nxt)
    if len(order) == len(known):
        return True, [], order
    return False, [], find_cycle(known, prereqs, indeg)


# FOLLOW-UP 2: Group courses into semesters (the earliest
# round each can be taken).
# Answer: Run Kahn's algorithm level by level: all courses
# with no remaining prerequisites form round 1, remove
# them, the next set forms round 2, and so on. The number
# of rounds is the longest dependency chain.
def semesters(courses, prereqs):
    # earliest round each course can be taken: Kahn by
    # levels
    adj = defaultdict(list)
    indeg = {c: 0 for c in courses}
    for u, v in prereqs:
        adj[u].append(v)
        indeg[v] += 1
    level = sorted(c for c in courses if indeg[c] == 0)
    rounds = []
    while level:
        rounds.append(level)
        nxt = []
        for c in level:
            for v in adj[c]:
                indeg[v] -= 1
                if indeg[v] == 0:
                    nxt.append(v)
        level = sorted(nxt)
    return rounds


if __name__ == "__main__":
    courses = ["a", "b", "c", "d"]
    pre = [("a", "b"), ("b", "c"), ("a", "d")]
    ok, missing, order = validate_catalog(courses, pre)
    assert ok and order[0] == "a"
    assert order.index("b") < order.index("c")
    # an unknown course id
    bad = validate_catalog(courses, pre + [("zzz", "a")])
    assert bad == (False, ["zzz"], None)
    # a cycle b -> c -> b
    loop = pre + [("c", "b")]
```

```
    ok, _, cycle = validate_catalog(courses, loop)
    assert not ok and cycle[0] == cycle[-1]
    assert set(cycle) == {"b", "c"}
    rounds = semesters(courses, pre)
    assert rounds == [["a"], ["b", "d"], ["c"]]
    assert validate_catalog([], []) == (True, [], [])
```

FUNCTION AND TEST CASES

Signature

```
def solution(course_ids: list[str], prereqs: list[tuple[st
r, str]]) -> tuple[bool, list[str], list[str], list[str]]:
```

Sample call

```
solution(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])
```

Test cases

Input	Expected
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C')])`	`(True, ['A', 'B', 'C'], [], [])`
`(['A', 'B', 'C'], [('A', 'B'), ('B', 'C'), ('C', 'A')])`	`(False, [], [], ['A', 'B', 'C', 'A'])`

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

MEDIUM | PracHub | Technical Screen | 10 of 21

★ TOP 75: rank 27 in the Medium and Hard priority list

★ Solve Grid and Counting Problems

POSTED 2025-01-25 | 633 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 64.8 | 30 DAYS 33 | PRIORITY 18/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #200 Number of Islands

LeetCode match: LC 200 Number of Islands (+ a counting task)

Posted on PracHub: 2025-01-25 (updated 2026-04-24)

Original: https://prachub.com/coding-questions/solve-grid-and-counting-problems

QUESTION

A software engineering intern phone screen included two coding tasks: 1.

Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.

Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character. Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.

Constraints

0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length

EXAMPLES

```
Input: []
```

Expected Output:

Explanation: Empty grid contains no land, so there are no islands.

```
Input: [['1']]
```

QUESTION

Expected Output:

Explanation: A single land cell forms one island.

Hints

Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.

Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.

Constraints

0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type

Examples

Input: []

Expected Output:

Explanation: No letters means it is impossible to form GOOGLE.

Input: ['G','O','O','G','L','E']

Expected Output:

Explanation: There is exactly one way to choose all six letters.

Hints

First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1 L and 1 E.

APPROACH

Islands: DFS/BFS or union-find over '1' cells. Second counting task: use prefix-sum + hash map or math counting per the statement.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two tasks. Islands: four-directional adjacency, grid up to 300 x 300, empty grid allowed. Counting: choose letters from an array so the chosen ones can be rearranged to spell GOOGLE, each index used at most once, different indices are different choices.

BRUTE FORCE TO SAY OUT LOUD

Islands: none needed, DFS is the standard. Counting: enumerate all 6-element subsets of the array and test each one. O(n^6).

OPTIMAL IDEA

Islands: scan cells, start a DFS or BFS (iterative, to avoid recursion limits) from each unvisited land cell, count the starts. O(mn). Counting: GOOGLE needs G twice, O twice, L once, E once. The number of ways is C(g, 2) * C(o, 2) * l * e, where g, o, l, e are the counts of those letters in the array, since the chosen multiset is fixed and only the indices vary. O(n) to count.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Islands: empty grid, a single land cell, diagonal-only neighbors (separate islands). Counting: missing a letter (0), exactly one G (0), lots of copies, an empty array.

LIKELY FOLLOW-UPS

Islands: add land one cell at a time with Union-Find. Counting: a different target word, or count ways when letters have to appear in order (subsequence DP).

PITFALLS

Treating diagonal cells as connected. Counting arrangements instead of choices of indices.

HOW TO THINK ABOUT IT

Islands: every unvisited land cell starts a new island; mark everything connected to it. Counting: the target multiset is fixed (G twice, O twice, L once, E once), so the number of ways to choose the indices is a product of binomial coefficients, one factor per distinct letter. Do not enumerate arrangements.

Basis: Two tasks: LeetCode 200 (Number of Islands) and counting the ways to choose letters from an array that can be rearranged into GOOGLE (each index used at most once).

BRUTE FORCE · Islands: recursive flood fill

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == "0" or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
```

TIME	SPACE
O(m * n)	O(m * n) recursion

WHY IT IS SLOW

Time is O(m * n), but the recursion can be m * n deep on a snake-shaped island and overflow the stack. A queue or an explicit stack avoids that.

BRUTE FORCE · Counting: enumerate every 6-element selection

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    count = 0
    for idx in combinations(range(len(a)), 6):
        letters = sorted(a[i] for i in idx)
        if letters == sorted("GGOOLE"):
            count += 1
    return count
```

BRUTE FORCE · Counting: enumerate every 6-element selection

TIME	SPACE
O(C(n, 6) * 6)	O(1)

WHY IT IS SLOW

C(n, 6) selections. The letters needed are fixed, so the answer is C(#G, 2) * C(#O, 2) * #L * #E from one pass of counts.

OPTIMAL SOLUTION · Islands: iterative BFS

TIME	SPACE
O(m * n)	O(min(m, n)) queue

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Counting: product of binomial coefficients

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Any target word, not only GOOGLE.

Count the needed letters of the word; the answer is the product over distinct letters of C(available, needed).

FOLLOW-UP 2 The letters must appear in order (a subsequence of the array spells the word).

Then it is the 'distinct subsequences' DP: ways[j] counts the ways to spell the first j letters; scan the array and update j from high to low. O(n * len(word)).

FOLLOW-UP 3 Islands arrive one cell at a time.

Use Union-Find: each new land cell makes the count go up by one and each union with a neighbouring different island makes it go down by one (LeetCode 305).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve Grid and Counting Problems
#
# A software engineering intern phone screen included two
# coding tasks: 1. **Count islands in a grid** Given an m
# x n grid of characters where '1' represents land and '0'
# represents water, return the number of islands. An
```

```
# island is formed by connecting adjacent land cells
# horizontally or vertically. You may assume all four
# edges of the grid are surrounded by water. 2. **Count
# ways to form a target word** Given an array of uppercase
# letters, count how many different ways you can choose
# letters from the array so that the chosen letters can be
# rearranged to spell GOOGLE. Each array element can be
# used at most once, and letters at different indices are
# considered different choices even if they contain the
# same character. Design efficient algorithms for both
# tasks, and discuss the time and space complexity of your
# approach.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Any target word, not only GOOGLE.
# 2. The letters must appear in order (a subsequence of
# the array spells the word).
# 3. Islands arrive one cell at a time.
#
# INTERVIEW NOTES
# Islands: every unvisited land cell starts a new island;
# mark everything connected to it. Counting: the target
# multiset is fixed (G twice, O twice, L once, E once), so
# the number of ways to choose the indices is a product of
# binomial coefficients, one factor per distinct letter.
# Do not enumerate arrangements.
#
# BRUTE FORCE (say it first): Islands: recursive flood
# fill
# Scan the grid; at each unvisited land cell, count one
# island and recurse into the four neighbours. Correct and
# the textbook answer; deep recursion can overflow on a
# large snake-like island. Time O(m * n). Space O(m * n)
# recursion.


from collections import Counter, deque
from math import comb
```

```
# BRUTE FORCE (say it first): Islands: recursive flood
# fill | time O(m * n)
def islands_recursive(grid):
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen = set()

    def fill(r, c):
        if (
            not (0 <= r < m and 0 <= c < n)
            or grid[r][c] != "1"
            or (r, c) in seen
        ):
            return
        seen.add((r, c))
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            fill(r + dr, c + dc)

    count = 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == "1" and (r, c) not in seen:
                count += 1
                fill(r, c)
    return count
```

```
# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): Islands: iterative BFS
# Idea: Same scan, but spread with a queue instead of
# recursion, so the depth of an island cannot overflow the
# stack. Marking visited cells in place (or with a set)
# keeps each cell processed once.
def num_islands(grid):
    # '1' is land: a BFS marks each island once
```

```
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen, count = set(), 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            queue = deque([(r, c)])
            while queue:
                x, y = queue.popleft()
                for dx, dy in STEPS:
                    a, b = x + dx, y + dy
                    if not (0 <= a < m and 0 <= b < n):
                        continue
                    land = grid[a][b] == "1"
                    if land and (a, b) not in seen:
                        seen.add((a, b))
                        queue.append((a, b))
    return count


# PART 2: count GOOGLE in the array
# FOLLOW-UP 1: Any target word, not only GOOGLE.
# Answer: Count the needed letters of the word; the answer
# is the product over distinct letters of C(available,
# needed).
def ways_to_spell(letters, word="GOOGLE"):
    # choose which array positions to use: for every
    # needed
    # letter choose that many of the available copies
    have, need = Counter(letters), Counter(word)
    total = 1
    for ch, k in need.items():
        total *= comb(have[ch], k)
    return total


# FOLLOW-UP 2: The letters must appear in order (a
# subsequence of the array spells the word).
# Answer: Then it is the 'distinct subsequences' DP:
# ways[j] counts the ways to spell the first j letters;
# scan the array and update j from high to low. O(n *
```

```
# len(word)).
def ways_in_order(letters, word="GOOGLE"):
    # the letters must appear in order (a subsequence):
    # the
    # "distinct subsequences" DP, updated from high to low
    ways = [1] + [0] * len(word)
    for ch in letters:
        for j in range(len(word), 0, -1):
            if word[j - 1] == ch:
                ways[j] += ways[j - 1]
    return ways[len(word)]


if __name__ == "__main__":
    assert num_islands([]) == 0
```

```
    assert num_islands([["1"]]) == 1
    grid = ["11000", "11000", "00100", "00011"]
    assert num_islands(grid) == 3
    assert ways_to_spell([]) == 0
    assert ways_to_spell(list("GOOGLE")) == 1
    # three O's: choose 2 of them -> 3 ways
    assert ways_to_spell(list("GOOOGLE")) == 3
    # an extra G and an extra E: 3 * 2 = 6 ways
    assert ways_to_spell(list("GGGOOLEE")) == 3 * 2
    assert ways_to_spell(list("GOGLE")) == 0
    # any target word
    assert ways_to_spell(list("AABB"), "AB") == 4
    assert ways_in_order(list("GOOGLE")) == 1
    assert ways_in_order(list("GGOOGLE")) == 2
    assert ways_in_order(list("GOGOLE")) == 0
    assert ways_in_order(list("EGOOGL")) == 0
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[str]]) -> int:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`0`
`[['1']]`	`1`

STUDY HERE: topic 4, Graphs and Search | main topic: BFS and DFS

HARD | PracHub | Technical Screen | 11 of 21

Infection Spread on a Grid (Cellular Automaton)

POSTED 2026-06-14 | 128 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 0 | PRIORITY 65/228

ABOUT THIS QUESTION

Topic: Graphs and Search: BFS and DFS

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #994 Rotting Oranges, LeetCode #289 Game of Life

LeetCode match: LC 994 Rotting Oranges / LC 289 Game of Life (inferred from overview)

Posted on PracHub: 2026-06-14 (updated 2026-06-24)

Original: https://prachub.com/coding-questions/infection-spread-on-a-grid-cellular-automaton

QUESTION

Overview: This question tests a candidate's ability to implement and optimize a synchronous cellular automaton simulation on a 2D grid, assessing proficiency in BFS-based propagation, state machine design, and incremental computation. It falls under algorithms and grid simulation, evaluating practical coding skill and the ability to handle progressively complex constraints including recovery delays, barriers, and early-termination fixed-point detection.

APPROACH

Keep a frontier of newly changed cells and update only their neighbors each step (multi-source BFS style) instead of rescanning the grid; use a state machine for cell transitions.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy has only the overview, so ask for the rules: the states (healthy, infected, recovered or immune), how infection spreads (four neighbors per step, synchronously), the recovery delay, whether barriers block spread, and what to return (the number of steps until nothing changes, or the final grid).

BRUTE FORCE TO SAY OUT LOUD

Rescan the entire grid on every step and compute the next grid from the old one. O(steps * mn).

OPTIMAL IDEA

Keep a frontier of cells that changed in the last step. Each step only their neighbors can change, so process the frontier like a multi-source BFS and update a separate next frontier to keep the update synchronous. Store the infection time per cell to apply the recovery delay (recover when step minus infected time reaches the delay). Stop when the frontier is empty and no recoveries are pending, since the grid has reached a fixed point. O(mn) overall.

DRY-RUN THESE CASES

No infected cells (0 steps), an isolated healthy cell behind barriers, a delay of 1, all cells infected at the start, two sources reaching a cell at the same time.

LIKELY FOLLOW-UPS

Different neighborhoods, probabilistic spread, or very large grids (sparse frontier).

PITFALLS

Updating cells in place during a step, which makes the spread asynchronous. Missing pending recoveries when deciding to stop.

INTERVIEW-DAY PLAN

Keep a frontier of cells that changed in the last step. Each step only their neighbors can change, so process the frontier like a multi-source BFS and update a separate next frontier to keep the update synchronous. Store the infection time per cell to apply the recovery delay (recover when step minus infected time reaches the delay). Stop when the frontier is empty and no recoveries are pending, since the grid has reached a fixed point. O(mn) overall.

DRY-RUN THESE CASES

No infected cells (0 steps), an isolated healthy cell behind barriers, a delay of 1, all cells infected at the start, two sources reaching a cell at the same time.

LIKELY FOLLOW-UPS

Different neighborhoods, probabilistic spread, or very large grids (sparse frontier).

PITFALLS

Updating cells in place during a step, which makes the spread asynchronous. Missing pending recoveries when deciding to stop.

HOW TO THINK ABOUT IT

A synchronous cellular automaton is a BFS in disguise: the step number is the BFS level. Only cells next to cells that just changed can change next, so keep a frontier instead of rescanning the grid. Always compute the next state from the old state (not in place) so all cells update together.

Basis: Written. Assumed rules (confirm them): '.' healthy, 'I' infected, 'X' barrier; each step every healthy cell with an infected 4-neighbour becomes infected, all at once; the answer is the number of steps until nothing changes.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    steps = 0
    while True:
        nxt = copy(grid)
        for r, c in all_cells(grid):
            if grid[r][c] == HEALTHY:
                if sick_neighbour(grid, r, c):
                    nxt[r][c] = INFECTED
        if nxt == grid:
            return steps
        grid = nxt
        steps += 1
```

TIME	SPACE
O(steps * m * n)	O(m * n)

WHY IT IS SLOW

Every step scans the whole grid even when only a thin edge is changing, O(steps * m * n). Keeping a frontier of newly infected cells (BFS) touches each cell once, O(m * n).

OPTIMAL SOLUTION · Frontier BFS (only recently infected cells spread)

TIME	SPACE
O(m * n)	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Add a recovery delay: an infected cell recovers after d steps and cannot spread afterwards.

A cell infects every healthy neighbor on the very next step, so recovery does not change how long spreading takes; it only changes what the grid looks like at time t. Store the step at which each cell was infected (the BFS level) and derive the state at any time: healthy, infected, or recovered when t minus its infection step is at least d.

FOLLOW-UP 2 Why can you stop early?

When the next frontier is empty no cell can change in any later step (the update only depends on the frontier), so the grid is at a fixed point. In a variant with pending recoveries keep going until those are done too.

FOLLOW-UP 3 Make the spread probabilistic.

A frontier cell may fail to infect a neighbor this step, so it stays in the frontier. Keep frontier cells until all their healthy neighbors are infected, and draw a random number per (cell, neighbor, step). The expected number of steps now needs simulation.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Infection Spread on a Grid (Cellular
# Automaton)
#
# This question tests a candidate's ability to implement
# and optimize a synchronous cellular automaton simulation
# on a 2D grid, assessing proficiency in BFS-based
# propagation, state machine design, and incremental
# computation. It falls under algorithms and grid
# simulation, evaluating practical coding skill and the
# ability to handle progressively complex constraints
# including recovery delays, barriers, and early-
# termination fixed-point detection.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Add a recovery delay: an infected cell recovers after
# d steps and cannot spread afterwards.
# 2. Why can you stop early?
# 3. Make the spread probabilistic.
#
# INTERVIEW NOTES
# A synchronous cellular automaton is a BFS in disguise:
# the step number is the BFS level. Only cells next to
# cells that just changed can change next, so keep a
# frontier instead of rescanning the grid. Always compute
# the next state from the old state (not in place) so all
# cells update together.
#
# BRUTE FORCE (say it first): Rescan the whole grid every
# step
# Build the next grid from the current one by checking
# every cell's four neighbors, and repeat until a step
# changes nothing. Easy to get right, so it doubles as a
# test oracle. The cost is that every step touches the
# whole grid even when only a thin edge is changing. Time
# O(steps * m * n). Space O(m * n).


# BRUTE FORCE (say it first): Rescan the whole grid every
# step | time O(steps * m * n)
def spread_steps_brute(grid):
    g = [list(r) for r in grid]
    steps = 0
    while True:
        nxt = [row[:] for row in g]
        changed = False
        for r in range(len(g)):
            for c in range(len(g[0])):
                if g[r][c] == "." and any(
                    0 <= r + dr < len(g)
                    and 0 <= c + dc < len(g[0])
                    and g[r + dr][c + dc] == "I"
                    for dr, dc in (
                        (1, 0),
                        (-1, 0),
                        (0, 1),
                        (0, -1),
                    )
                ):
                    nxt[r][c] = "I"
                    changed = True
        if not changed:
            return steps
        g, steps = nxt, steps + 1


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# TIME each cell gets infected
def infection_times(grid):
    # step at which each cell gets infected (a BFS level);
    # 'I' = infected at time 0, '.' = healthy, '#' =
    # blocked
    rows, cols = len(grid), len(grid[0])
    when = {}
    frontier = [(r, c) for r in range(rows)
                for c in range(cols) if grid[r][c] == "I"]
    for cell in frontier:
        when[cell] = 0
    step = 0
    while frontier:
        step += 1
        nxt = []
        for r, c in frontier:
            for dr, dc in STEPS:
                a, b = r + dr, c + dc
                inside = 0 <= a < rows and 0 <= b < cols
                # every healthy neighbour is infected next
                # step
                if inside and grid[a][b] == "." \
                        and (a, b) not in when:
                    when[(a, b)] = step
```

```
                    nxt.append((a, b))
        frontier = nxt
    return when


# BASE QUESTION (optimal): Frontier BFS (only recently
# infected cells spread)
# Idea: Put every initially infected cell in the frontier.
# Each step, look only at the neighbors of frontier cells,
# infect the healthy ones, and make them the next
# frontier.
def spread_steps(grid):
    # steps until nothing changes any more
    when = infection_times(grid)
    return max(when.values(), default=0)


# FOLLOW-UP 1: Add a recovery delay: an infected cell
# recovers after d steps and cannot spread afterwards.
# Answer: A cell infects every healthy neighbor on the
# very next step, so recovery does not change how long
# spreading takes; it only changes what the grid looks
# like at time t.
def grid_at(grid, t, recovery=None):
    # the grid after t steps; with a recovery delay d an
    # infected cell turns into 'R' (it spread already, so
    # the
    # timing above does not change)
    when = infection_times(grid)
    out = []
    for r, row in enumerate(grid):
        line = ""
        for c, ch in enumerate(row):
            start = when.get((r, c))
            if start is None or start > t:
                line += ch
            elif recovery and t >= start + recovery:
                line += "R"
            else:
                line += "I"
        out.append(line)
    return out
```

```
if __name__ == "__main__":
    g = ["I..", "...", "..."]
    assert spread_steps(g) == 4
    assert grid_at(g, 1) == ["II.", "I..", "..."]
    # with recovery after 1 step the source recovers first
    rec = grid_at(g, 1, recovery=1)
    assert rec == ["RI.", "I..", "..."]
    # a wall splits the grid: the far cell is never
    # infected
    wall = ["I#.", "I#."]
    assert spread_steps(wall) == 0
    assert grid_at(wall, 9)[0] == "I#."
    assert spread_steps(["..."]) == 0
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[str]) -> list[str]:
```

Sample call

```
solution(['HHH', 'HIH', 'HHH'])
```

Test cases

Input	Expected
`(['HHH', 'HIH', 'HHH'],)`	`['HIH', 'III', 'HIH']`
`(['HHH'],)`	`['HHH']`

STUDY HERE: topic 4, Graphs and Search | main topic: Topological sort and cycles

Site tags: string, parsing, graph, topological-sort, dfs, cycle-detection, medium

MEDIUM | 1Point3Acres | onsite-coding | 12 of 21

Template Variable Expansion with Follow-Ups

ASKED 2026-10-17 | 3 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 214/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Topological sort and cycles

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-17 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string.
Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values.
Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

APPROACH

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

HOW TO THINK ABOUT IT

Agree the grammar first: %NAME% is a placeholder, %% is a literal percent sign, an unmatched % is an error and an unknown name is an error. Values may contain placeholders themselves, so expand a value by expanding its own placeholders first. A cycle (A uses B, B uses A) would recurse forever, so detect it with a visiting set or, equivalently, with indegrees and a queue.

Basis: Written for this question; resolving values that contain other placeholders is a dependency graph problem (as in LeetCode 210, Course Schedule II).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text, depth=0):
    if depth > LIMIT:
        raise Error("too deep: cycle?")
    out = ""
    for token in tokens(text):
        if token == "%%":
            out += "%"
        elif is_key(token):
            value = values[name_of(token)]
            out += expand(value, depth + 1)
        else:
            out += token
    return out
```

TIME	SPACE
exponential in the worst ... exponential in the worst case	**O(depth)**

WHY IT IS SLOW

Re-expands shared values every time and finds cycles only by running out of depth. Memoising each value and keeping a visiting set expands each key once and reports a cycle at once.

OPTIMAL SOLUTION · Depth-first expansion with memo and a visiting set

TIME	SPACE
O(total length of the expanded text + keys)	**O(keys)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Detect cycles before expanding.

Model each key as a vertex with an edge to every key its value mentions, count indegrees and run Kahn's algorithm: if fewer vertices than keys come out, some keys sit on a cycle and cannot be resolved. This reports the problem up front instead of failing halfway through a long document.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Template Variable Expansion with Follow-
# Ups
#
# Replace %KEY% placeholders using a key-value map whose
# values may contain other placeholders. Clarify malformed
# delimiters and literal-percent escaping, flatten nested
# values, and detect cyclic dependencies before expansion.
# Input: a key-value map and a string containing
# placeholders delimited by %, such as %USER%. Replace
# every placeholder with the corresponding value. Map
# values can themselves contain placeholders, so resolve
# or flatten dependencies before expanding the final
# string. Define behavior for malformed delimiters such as
# unmatched percent signs, for a literal % in output, and
# for nested-looking forms before coding; the prompt
# intentionally leaves these contracts open. Follow-up:
# detect cyclic dependencies among map values. Model keys
# as a dependency graph and use indegrees plus a queue to
# determine whether a complete topological resolution is
# possible.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Detect cycles before expanding.
#
# INTERVIEW NOTES
# Agree the grammar first: %NAME% is a placeholder, %% is
# a literal percent sign, an unmatched % is an error and
# an unknown name is an error. Values may contain
# placeholders themselves, so expand a value by expanding
# its own placeholders first. A cycle (A uses B, B uses A)
# would recurse forever, so detect it with a visiting set
# or, equivalently, with indegrees and a queue.
#
# BRUTE FORCE (say it first): Recursive expansion without
# memo or cycle check, capped by depth
# Recursively substitute placeholders and stop with an
# error after a fixed depth. Correct for acyclic maps;
# repeats shared work and detects cycles only by running
# out of depth. Time exponential in the worst case. Space
# O(depth).


from collections import defaultdict, deque
```

```
# BRUTE FORCE (say it first): Recursive expansion without
# memo or cycle check, capped by depth | time exponential
# in the worst case
def expand_brute(text, mapping, depth=0):
    if depth > 30:
        raise ValueError("cycle")
    out, i = [], 0
    while i < len(text):
        if text[i] != "%":
            out.append(text[i])
            i += 1
        elif text[i : i + 2] == "%%":
            out.append("%")
            i += 2
        else:
            j = text.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            out.append(
                expand_brute(
                    mapping[text[i + 1 : j]],
                    mapping,
                    depth + 1,
                )
            )
            i = j + 1
    return "".join(out)


# HELPERS for the base question
def tokens(s):
    # split text into ("text", char) and ("var", name)
    i = 0
    while i < len(s):
        if s[i] != "%":
            yield "text", s[i]
            i += 1
        elif s[i:i + 2] == "%%":
            # %% is a literal percent sign
            yield "text", "%"
            i += 2
        else:
            j = s.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            yield "var", s[i + 1:j]
            i = j + 1


# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
# Idea: expand(text) scans the text once: %% becomes %,
# %NAME% becomes the resolved value of NAME.
def expand_template(text, mapping):
    done, visiting = {}, set()

    def resolve(name):
        # values may hold placeholders: expand them first
        if name in done:
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        return "".join(resolve(x) if kind == "var" else x
                       for kind, x in tokens(s))

    return scan(text)


# FOLLOW-UP 1: Detect cycles before expanding.
# Answer: Model each key as a vertex with an edge to every
# key its value mentions, count indegrees and run Kahn's
# algorithm: if fewer vertices than keys come out, some
# keys sit on a cycle and cannot be resolved.
def has_cycle(mapping):
    # Kahn's algorithm over key -> keys it mentions
    deps = {}
    for k, v in mapping.items():
        deps[k] = {x for kind, x in tokens(v)
                   if kind == "var"}
    users, indeg = defaultdict(list), {}
    for k, d in deps.items():
        known = d & mapping.keys()
        indeg[k] = len(known)
        for x in known:
```

```
            users[x].append(k)
    queue = deque(k for k, n in indeg.items() if n == 0)
    resolved = 0
    while queue:
        k = queue.popleft()
        resolved += 1
        for u in users[k]:
            indeg[u] -= 1
            if indeg[u] == 0:
                queue.append(u)
    # keys left over sit on a cycle
    return resolved != len(mapping)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    assert expand_template(text, m) == (
        "I am admin My home is /admin/home")
    sure = expand_template("100%% sure", {})
    assert sure == "100% sure"
    assert not has_cycle(m)
    loop = {"A": "%B%", "B": "%A%"}
    assert has_cycle(loop)
    for bad, error in (("a % b", ValueError),
                       ("%NOPE%", KeyError)):
        try:
            expand_template(bad, {})
            raise AssertionError("should have failed")
        except error:
            pass
    try:
        expand_template("%A%", loop)
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
```

NOTES FROM CANDIDATES

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

PREPARATION

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY HERE: topic 4, Graphs and Search | main topic: Topological sort and cycles

Site tags: graph, topological-sort, cycle-detection, medium

MEDIUM | IN APP LC 210 | 1Point3Acres | onsite-coding | 13 of 21

Course Scheduling and Dependency Variants

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 179/228

ABOUT THIS QUESTION

Topic: Graphs and Search: Topological sort and cycles

Optimal answer from: your app's interview approach (LeetCode #210)

LeetCode match: LC 210 Course Schedule II

Frequency: Medium | Last asked: 2026-10-04

Original: https://www.1point3acres.com/interview/problems/company/google/course-schedule-dependency-variants

QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

Course-ordering variant

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

Damaged-node and path variant

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

Merge-two-ordered-sequences variant

Input: two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

QUESTION

EXAMPLES

Merge-two-ordered-sequences

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1:
    contradiction)
```

APPROACH

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

HOW TO THINK ABOUT IT

Every 'A must come before B' rule is an edge. A valid ordering exists exactly when the graph has no cycle, and Kahn's algorithm finds one: repeatedly take a vertex with no unmet prerequisites. The two-array merge is the same problem: each adjacent pair inside an array is an edge and equal values are the same vertex; the merge is possible only if the combined graph has no cycle.

Basis: LeetCode 210, Course Schedule II, plus a merge-two-ordered-sequences variant (the same graph in disguise).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, rules):
    for order in permutations(range(n)):
        pos = {v: i for i, v in enumerate(order)}
        ok = all(pos[a] < pos[b]
                 for a, b in rules)
        if ok:
            return True
    return False
```

TIME	SPACE
O(V! * E)	O(V)

WHY IT IS SLOW

V! orderings. Kahn's algorithm repeatedly takes a vertex with no unmet prerequisites and finishes in O(V + E).

OPTIMAL SOLUTION · Kahn's algorithm (course order, or can two sequences merge)

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you check that an order is valid?

Record each vertex's position in the order, check that every vertex appears exactly once, then check that for every edge (u, v) the position of u is smaller than the position of v. Accept any valid order, not one fixed answer, because independent vertices can swap.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Course Scheduling and Dependency Variants
#
# Topological-ordering family. Variants: course ordering
# with validity checks and a nested-list extension;
# merging two arrays whose internal order defines
# precedence (return false on a conflict cycle); and a
# damaged-node dependency-path variant that minimizes
# broken nodes on a route. Course-ordering variant Work
# through the course-ordering problem, including validity
# checks, complexity analysis, and test cases. Extend the
# result to a nested-list format. The grouping rule is
# unspecified; clarify what each inner list represents
# before implementing the extension. Damaged-node and path
# variant Remove damaged nodes and their downstream
# effects, then find a valid path. A follow-up minimizes
# broken nodes on a required route. Clarify the graph
# direction, path endpoints, and meaning of broken nodes
# before choosing an algorithm. Merge-two-ordered-
# sequences variant Input: two integer arrays a and b.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you check that an order is valid?
#
# INTERVIEW NOTES
# Every 'A must come before B' rule is an edge. A valid
# ordering exists exactly when the graph has no cycle, and
# Kahn's algorithm finds one: repeatedly take a vertex
# with no unmet prerequisites. The two-array merge is the
# same problem: each adjacent pair inside an array is an
# edge and equal values are the same vertex; the merge is
# possible only if the combined graph has no cycle.
#
# BRUTE FORCE (say it first): Try every permutation of the
# vertices
# Generate permutations and check every constraint.
# Correct and a good oracle for tiny inputs; factorial
# time is unusable beyond about ten vertices. Time O(V! *
# E). Space O(V).


import itertools
from collections import defaultdict, deque


# BRUTE FORCE (say it first): Try every permutation of the
# vertices | time O(V! * E)
def merge_orders_brute(a, b):
    vs = sorted(set(a) | set(b))
    edges = [
        (x[i], x[i + 1])
        for x in (a, b)
        for i in range(len(x) - 1)
    ]
    for perm in itertools.permutations(vs):
        pos = {v: i for i, v in enumerate(perm)}
        if all(pos[u] < pos[v] for u, v in edges):
            return True
    return False


# HELPERS for the base question
def edges_from_sequences(*sequences):
    # each neighbouring pair in a sequence is a rule
    return [(s[i], s[i + 1]) for s in sequences
            for i in range(len(s) - 1)]


def merge_two_orders(a, b):
    # shared values are the same vertex
    edges = edges_from_sequences(a, b)
    # a cycle (conflict) gives None
    return topological_order(set(a) | set(b), edges)


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def topological_order(vertices, edges):
    # edge (u, v) means u must come before v
    adj = defaultdict(list)
    indeg = {v: 0 for v in vertices}
    for u, v in edges:
        adj[u].append(v)
        # count unmet prerequisites of v
        indeg[v] = indeg.get(v, 0) + 1
        indeg.setdefault(u, 0)
    # start with everything that has no prerequisite
    queue = deque(v for v, d in indeg.items() if d == 0)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        # finishing u frees its dependents
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                queue.append(v)
    # fewer than all vertices means a cycle: no order
    return order if len(order) == len(indeg) else None


# BASE QUESTION (optimal): Kahn's algorithm (course order,
# or can two sequences merge)
# Idea: Build the graph (prerequisite -> dependent,
# including isolated vertices), count indegrees and keep a
# queue of vertices with indegree 0. Pop one, append it to
# the order and lower the indegree of its dependents.
def can_merge(a, b):
    # True when some order keeps both sequences in order
    return merge_two_orders(a, b) is not None


# FOLLOW-UP 1: How do you check that an order is valid?
# Answer: Record each vertex's position in the order,
# check that every vertex appears exactly once, then check
```

```
# that for every edge (u, v) the position of u is smaller
# than the position of v.
def is_valid_order(order, vertices, edges):
    # every vertex must appear exactly once
    if len(order) != len(set(vertices)):
        return False
    if set(order) != set(vertices):
        return False
    pos = {v: i for i, v in enumerate(order)}
    # every prerequisite must come before its dependent
    return all(pos[u] < pos[v] for u, v in edges)


if __name__ == "__main__":
    # course order with an isolated course 3
    courses, rules = range(4), [(0, 1), (1, 2)]
    order = topological_order(courses, rules)
```

```
    assert is_valid_order(order, courses, rules)
    # a swapped order breaks a rule
    swapped = [1, 0, 2, 3]
    assert not is_valid_order(swapped, courses, rules)
    # a cycle has no order
    loop = [(0, 1), (1, 0)]
    assert topological_order(range(2), loop) is None
    # merge examples from the question
    assert can_merge([1, 2, 3], [2, 4, 5])
    merged = merge_two_orders([1, 2, 3], [2, 4, 5])
    assert merged is not None
    assert not can_merge([1, 6, 4], [4, 1])
    # a repeated value in one array is a self conflict
    assert not can_merge([1, 2, 1], [])
```

NOTES FROM CANDIDATES

For standard prerequisite-ordering practice, represent each course as a vertex and each prerequisite as an edge from prerequisite to dependent. Include isolated courses. The result orders every course after its prerequisites; a cycle prevents a complete ordering.

Maintain indegrees and a queue of zero-indegree vertices. Emit a vertex, decrement its outgoing neighbors, and enqueue newly ready vertices. If fewer than V vertices are emitted, report failure under the chosen return convention. This takes O(V + E) time and space; check each edge against positions in the output to validate an ordering.

This ordering algorithm does not determine the nested-list grouping rule or solve the damaged-node path objective. Keep those contracts separate rather than assuming they are ordinary topological sorting.

The merge-two-sequences variant is the same ordering engine in disguise: shared values are the same node, and each adjacent pair inside an array is an edge. A cycle means the merge is impossible. It was asked as the final onsite round of an early-career L3 loop, and the candidate nearly ran out of time — recognizing it as a graph problem early is the main hurdle.

PREPARATION

In 20 minutes, implement prerequisite ordering with cycle detection; check isolated courses, independent choices, a chain, and a directed cycle.
In 10 minutes, write an independent result checker for vertex coverage and every prerequisite constraint. Accept different valid orders.
In 10 minutes, list the missing grouping and path-cost rules, and explain which algorithm decisions depend on each answer.

STUDY HERE: topic 4, Graphs and Search | main topic: Topological sort and cycles

MEDIUM | PracHub | Technical Screen | 14 of 21

Determine whether two nodes are related

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 24.5 | 30 DAYS 0 | PRIORITY 142/228

ABOUT THIS QUESTION

Topic: Graphs and Search: Topological sort and cycles

Optimal answer from: the interview site's own solution

LeetCode match: LC 1971 Find if Path Exists in Graph (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/determine-whether-two-nodes-are-related

QUESTION

Overview: This question evaluates understanding of graph representations, reachability and connectivity concepts for ancestry relationships, including handling disconnected components and potential cycles.

APPROACH

Build the ancestry graph and BFS/DFS (or union-find for undirected relatedness); handle disconnected components.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what related means: one is an ancestor of the other, they share a common ancestor, or they are connected in any direction. Ask whether the graph is a tree, a DAG, or may have cycles, and whether it is disconnected.

BRUTE FORCE TO SAY OUT LOUD

DFS from the first node looking for the second, and again from the second. O(V + E) per query.

OPTIMAL IDEA

Build the adjacency list once. For ancestry in a DAG compute the ancestor set of each node (DFS or BFS) and test for the target or a non-empty intersection. For undirected relatedness use union-find, which answers queries in near constant time after one pass. Track visited to survive cycles.

DRY-RUN THESE CASES

A node related to itself, two nodes in separate components, a long chain, a cycle.

LIKELY FOLLOW-UPS

Many queries (precompute components or ancestors), the lowest common ancestor, or updates to the graph.

PITFALLS

Missing a visited set. Treating direction as undirected, or the reverse.

HOW TO THINK ABOUT IT

Ask what 'related' means: connected in any direction (a connectivity question: union-find or one traversal), or ancestor/descendant, or share an ancestor. With many queries, preprocess once: components give O(1) answers. Always make the visited set part of the design, since relationship data can have cycles.

Basis: LeetCode 1971, Find if Path Exists in Graph, for connectivity, and ancestor queries on a parent map for the 'ancestry' reading. Assumed meaning (confirm it): related = connected by any path of relationships.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def related(a, b):
    graph = build_adjacency(edges)
    seen = set()
    dfs(graph, a, seen)
    return b in seen
```

TIME	SPACE
O(V + E) per query	O(V)

WHY IT IS SLOW

Every query repeats the same traversal, O(V + E) each. A Union-Find built once answers each query in near-constant time.

OPTIMAL SOLUTION · Union-Find built once, constant-time queries

TIME	SPACE
O(E alpha(V)) to build, O(alpha(V)) per query	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 'Related' means ancestor or descendant, or sharing an ancestor.

Store the data as a parent map and walk up from one node while recording ancestors, then walk up from the other until you hit one. Keep a seen set so a corrupt cycle cannot loop forever.

FOLLOW-UP 2 Relationships arrive over time and you must answer queries between updates.

Union-Find handles incremental unions in near-constant time. If relationships can also be removed, you need offline processing or a dynamic-connectivity structure.

FOLLOW-UP 3 The graph is directed and 'related' means reachability from a to b.

Union-Find no longer works (it ignores direction). Use DFS/BFS per query, or compute strongly connected components and reachability on the condensed DAG for many queries.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine whether two nodes are related
#
# This question evaluates understanding of graph
# representations, reachability and connectivity concepts
# for ancestry relationships, including handling
# disconnected components and potential cycles.
#
# FOLLOW-UPS (answered below, in this order):
# 1. 'Related' means ancestor or descendant, or sharing an
# ancestor.
# 2. Relationships arrive over time and you must answer
# queries between updates.
# 3. The graph is directed and 'related' means
# reachability from a to b.
#
# INTERVIEW NOTES
# Ask what 'related' means: connected in any direction (a
# connectivity question: union-find or one traversal), or
# ancestor/descendant, or share an ancestor. With many
# queries, preprocess once: components give O(1) answers.
# Always make the visited set part of the design, since
# relationship data can have cycles.
#
# BRUTE FORCE (say it first): Search from the first node
# for every query
# Build adjacency lists and run a DFS from a looking for b
# each time. Correct, and each query repeats the same
# traversal work. Time O(V + E) per query. Space O(V).


from collections import defaultdict
from collections import deque


# BRUTE FORCE (say it first): Search from the first node
# for every query | time O(V + E) per query
def related_dfs(n, edges, a, b):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    seen, stack = {a}, [a]
    while stack:
        x = stack.pop()
        if x == b:
            return True
        for y in adj[x]:
            if y not in seen:
                seen.add(y)
                stack.append(y)
    return False


# BASE QUESTION (optimal): Union-Find built once,
# constant-time queries
# Idea: Union the endpoints of every relationship; two
# nodes are related exactly when they have the same root.
class Relations:
    """Related = joined by a chain of relations."""

    def __init__(self, n, edges=()):
        self.parent = list(range(n))
        self.size = [1] * n
        for u, v in edges:
            self.union(u, v)

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        # relations can arrive over time: near-constant
        # time
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # the smaller group joins the bigger one
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

    def related(self, a, b):
        return self.find(a) == self.find(b)


# HELPER: ancestors of a node
def ancestors(parent, x):
    # x and everything above it; a seen set stops cycles
    seen = []
    while x is not None and x not in seen:
        seen.append(x)
        x = parent.get(x)
    return seen


# FOLLOW-UP 1: 'Related' means ancestor or descendant, or
# sharing an ancestor.
# Answer: Store the data as a parent map and walk up from
# one node while recording ancestors, then walk up from
# the other until you hit one. Keep a seen set so a
# corrupt cycle cannot loop forever.
def kin(parent, a, b):
    # related if one is an ancestor of the other, or they
    # share an ancestor
    up_a = ancestors(parent, a)
    up_b = ancestors(parent, b)
    return bool(set(up_a) & set(up_b))


# FOLLOW-UP 3: The graph is directed and 'related' means
# reachability from a to b.
# Answer: Union-Find no longer works (it ignores
# direction). Use DFS/BFS per query, or compute strongly
# connected components and reachability on the condensed
# DAG for many queries.
```

```
def reachable(graph, a, b):
    # directed: can we get from a to b? (Union-Find cannot
    # tell, it ignores direction)
    seen, queue = {a}, deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in graph.get(u, ()):
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False


if __name__ == "__main__":
    r = Relations(5, [(0, 1), (1, 2)])
```

```
    assert r.related(0, 2) and not r.related(0, 3)
    r.union(3, 4)
    assert r.related(3, 4) and not r.related(2, 4)
    r.union(2, 3)
    assert r.related(0, 4)
    assert r.related(1, 1)
    # a family tree: parent[child] = parent
    parent = {"kid": "mom", "mom": "gran",
              "uncle": "gran", "stranger": None}
    assert kin(parent, "kid", "gran")
    assert kin(parent, "kid", "uncle")
    assert not kin(parent, "kid", "stranger")
    # a corrupt cycle cannot loop forever
    assert kin({"a": "b", "b": "a"}, "a", "b")
    # direction matters for reachability
    g = {"a": ["b"], "b": ["c"]}
    assert reachable(g, "a", "c")
    assert not reachable(g, "c", "a")
```

FUNCTION AND TEST CASES

Signature

```
def solution(relations: list[list[int]], a: int, b: int) -
> bool:
```

Sample call

```
solution([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)
```

Test cases

Input	Expected
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 4)`	`True`
`([(1, 2), (1, 3), (3, 4), (10, 11)], 2, 11)`	`False`

STUDY HERE: topic 4, Graphs and Search | main topic: Topological sort and cycles

Site tags: graph, bfs, medium

MEDIUM | 1Point3Acres | onsite-coding | 15 of 21

Shortest Directed Cycle Through a Node

STUDY: TALK | **ALL TIME 5** | **30 DAYS 0** | **PRIORITY 174/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Topological sort and cycles

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2608 Shortest Cycle in a Graph (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-cycle-through-node

QUESTION

EXAMPLES

```
Edges [(0,1),(1,2),(2,0)], v = 0 -> 3.
Edges [(0,1),(1,2)], v = 0 -> -1.
```

APPROACH

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether edges are directed (yes), whether self loops count (cycle of length 1), what to return if there is none (-1), and whether edges have weights.

BRUTE FORCE TO SAY OUT LOUD

Enumerate all simple paths from v back to v with DFS. Exponential.

OPTIMAL IDEA

BFS from v. The first time you see an edge from a visited node u back to v, the cycle length is dist[u] + 1, and BFS order makes it the shortest. Do not mark v as discovered before checking for re-entry. O(V + E).

DRY-RUN THESE CASES

A triangle (3). A path with no return (-1). A self loop (1). Two cycles through v of different lengths.

LIKELY FOLLOW-UPS

Shortest cycle in the whole graph (BFS from every node), weighted edges (Dijkstra), or undirected graphs.

PITFALLS

Marking v visited at the start, which hides the cycle. Counting nodes instead of edges.

HOW TO THINK ABOUT IT

A cycle through v is a path from v back to v, so find shortest distances from v and watch for an edge that points back at v. Breadth-first search discovers nodes in order of distance, so the first edge back to v closes the shortest cycle. Do not mark v as 'seen' in a way that hides the edge that returns to it.

Basis: Related to LeetCode 2608, Shortest Cycle in a Graph, but directed and anchored at a given node.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = INF
def dfs(u, length, on_path):
    global best
    for w in graph[u]:
        if w == v:
            best = min(best, length + 1)
        elif w not in on_path:
            dfs(w, length + 1, on_path | {w})
dfs(v, 0, {v})
answer = -1 if best == INF else best
```

TIME	SPACE
exponential	O(V)

WHY IT IS SLOW

The number of simple paths can be exponential in the graph size. A breadth-first search from v reaches the shortest cycle in O(V + E) because it visits nodes in order of distance.

OPTIMAL SOLUTION · BFS from v, close the cycle on the first edge back to v

TIME	SPACE
O(V + E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you find the shortest cycle in the whole graph?

Run the same BFS from every vertex and take the minimum: O(V * (V + E)). For undirected graphs, BFS from each node and treat any non-tree edge as closing a cycle of length dist[u] + dist[w] + 1 (the LeetCode 2608 version).

FOLLOW-UP 2 What if edges have positive weights?

Replace BFS with Dijkstra from v and close the cycle at the cheapest dist[u] + weight(u, v). That is O((V + E) log V) with a heap. With negative weights use Bellman-Ford and handle negative cycles separately.

FOLLOW-UP 3 The shortest cycle anywhere in the graph (the girth).

Run the same BFS from every node and take the smallest result: O(V * (V + E)). That is the standard way and fine for graphs up to a few thousand nodes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shortest Directed Cycle Through a Node
#
# Examples: Edges [(0,1),(1,2),(2,0)], v = 0 -> 3. Edges
# [(0,1),(1,2)], v = 0 -> -1.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you find the shortest cycle in the whole
# graph?
# 2. What if edges have positive weights?
# 3. The shortest cycle anywhere in the graph (the girth).
#
# INTERVIEW NOTES
# A cycle through v is a path from v back to v, so find
# shortest distances from v and watch for an edge that
# points back at v. Breadth-first search discovers nodes
# in order of distance, so the first edge back to v closes
# the shortest cycle. Do not mark v as 'seen' in a way
# that hides the edge that returns to it.
#
# BRUTE FORCE (say it first): Enumerate every simple cycle
# through v with DFS
# Explore all simple paths from v and record the length
# whenever an edge returns to v. Exact and the oracle, but
# the number of simple paths can be exponential. Time
# exponential. Space O(V).


import heapq
from collections import deque


# BRUTE FORCE (say it first): Enumerate every simple cycle
# through v with DFS | time exponential
def shortest_cycle_dfs(n, edges, v):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
    best = [float("inf")]

    def go(u, seen, length):
        for w in adj[u]:
            if w == v:
                best[0] = min(best[0], length + 1)
            elif w not in seen:
                go(w, seen | {w}, length + 1)
```

```
    go(v, {v}, 0)
    return -1 if best[0] == float("inf") else best[0]


# HELPERS for the base question
def make_graph(n, edges):
    adj = [[] for _ in range(n)]
    for a, b, *w in edges:
        # edges may carry a weight (default 1)
        adj[a].append((b, w[0] if w else 1))
    return adj


# HELPERS for follow-up 2
INF = float("inf")


# BASE QUESTION (optimal): BFS from v, close the cycle on
# the first edge back to v
# Idea: A directed cycle through v is a path v to u
# followed by an edge u to v. BFS visits nodes in
# nondecreasing distance, so the first time we see an edge
# back to v its length is dist[u] + 1 and is the shortest.
def shortest_cycle_through(n, edges, v):
    # BFS from v: the first edge that comes back to v
    # closes
    # the shortest cycle through v
    adj = make_graph(n, edges)
    dist, queue = {v: 0}, deque([v])
    while queue:
        u = queue.popleft()
        for w, _ in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1


# FOLLOW-UP 1: How do you find the shortest cycle in the
# whole graph?
# Answer: Run the same BFS from every vertex and take the
# minimum: O(V * (V + E)). For undirected graphs, BFS from
```

```
# each node and treat any non-tree edge as closing a cycle
# of length dist[u] + dist[w] + 1 (the LeetCode 2608
# version).
# FOLLOW-UP 3: The shortest cycle anywhere in the graph
# (the girth).
# Answer: Run the same BFS from every node and take the
# smallest result: O(V * (V + E)). That is the standard
# way and fine for graphs up to a few thousand nodes.
def girth(n, edges):
    # the shortest cycle anywhere: BFS from every node
    sizes = [shortest_cycle_through(n, edges, v)
             for v in range(n)]
    found = [s for s in sizes if s > 0]
    return min(found) if found else -1


# FOLLOW-UP 2: What if edges have positive weights?
# Answer: Replace BFS with Dijkstra from v and close the
# cycle at the cheapest dist[u] + weight(u, v). That is
# O((V + E) log V) with a heap. With negative weights use
# Bellman-Ford and handle negative cycles separately.
def weighted_cycle_through(n, edges, v):
    # positive weights: Dijkstra, then close the cycle at
    # v
    adj = make_graph(n, edges)
    dist, heap, best = {v: 0}, [(0, v)], INF
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, INF):
            continue
        for w, cost in adj[u]:
            if w == v:
                best = min(best, d + cost)
            elif d + cost < dist.get(w, INF):
                dist[w] = d + cost
                heapq.heappush(heap, (d + cost, w))
    return -1 if best == INF else best
```

```
if __name__ == "__main__":
    ring = [(0, 1), (1, 2), (2, 0)]
    assert shortest_cycle_through(3, ring, 0) == 3
    path = [(0, 1), (1, 2)]
    assert shortest_cycle_through(3, path, 0) == -1
    # a self loop is a cycle of length 1
```

```
    assert shortest_cycle_through(2, [(0, 0)], 0) == 1
    # a shorter cycle exists elsewhere: 1 <-> 2
    both = ring + [(1, 2), (2, 1)]
    assert girth(3, both) == 2
    assert girth(3, [(0, 1), (1, 2)]) == -1
    # weighted: the cheap way back is 0 -> 1 -> 0 (cost 3)
    weighted = [(0, 1, 1), (1, 0, 2),
                (0, 2, 1), (2, 0, 9)]
    assert weighted_cycle_through(3, weighted, 0) == 3
```

NOTES FROM CANDIDATES

Strictly directed; do not relax to undirected.
BFS layer 0 is v itself; the first re-discovery during expansion is the cycle length.
Common bug: marking v as visited immediately blocks the re-entry.

PREPARATION

Write the directed BFS-from-source version from memory in under 12 min.
Practice the "re-entry detection" pattern (don't mark v visited until after pop).

STUDY HERE: topic 4, Graphs and Search | main topic: Union-find

Site tags: grid, dfs, bfs, union-find, medium

MEDIUM | IN APP LC 200 | 1Point3Acres | onsite-coding, phone-screen | 16 of 21

★ TOP 75: rank 17 in the Medium and Hard priority list

★ Number of Islands

ASKED 2026-09-30 | 20 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 64.8** | **30 DAYS 33** | **PRIORITY 19/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Union-find

Optimal answer from: your app's interview approach (LeetCode #200)

LeetCode match: LC 200 Number of Islands

Frequency: Medium | Last asked: 2026-09-30 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/number-of-islands

QUESTION

EXAMPLES

```
[[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2.
Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1,
    1] (last addition merges the two land cells).
```

APPROACH

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

HOW TO THINK ABOUT IT

Each island is a connected group of land cells, so count how many times you start a fresh search at an unvisited land cell. The search itself (DFS or BFS) just marks everything reachable. If land is added over time, do not rerun the search: Union-Find keeps the count up to date.

Basis: LeetCode 200, Number of Islands, and LeetCode 305, Number of Islands II (streaming additions with Union-Find).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(grid):
    seen, count = set(), 0
    for r, c in all_cells(grid):
        if grid[r][c] == 0 or (r, c) in seen:
            continue
        count += 1
        dfs(r, c)  # marks the whole island
    return count
def dfs(r, c):
    if out_of_grid(r, c) or grid[r][c] == 0:
        return
    if (r, c) in seen:
        return
    seen.add((r, c))
    for dr, dc in DIRECTIONS:
        dfs(r + dr, c + dc)
```

TIME	SPACE
O(m * n)	O(m * n) recursion

WHY IT IS SLOW

Time is already O(m * n); the weakness is the recursion depth, which can reach m * n on a snake-shaped island and overflow the stack. An explicit queue or stack avoids that.

OPTIMAL SOLUTION · Iterative DFS flood fill

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if land can also be removed?

Union-Find cannot split sets. Either rebuild the affected component with a local BFS from each former neighbour, or process the whole timeline offline with a segment tree over time and union-find with rollback (the offline dynamic connectivity technique).

FOLLOW-UP 2 How do you handle a grid too large for memory?

Process it in horizontal strips: label islands within the strip, keep only the boundary row's labels, and merge labels across strips with a union-find over label ids. Memory is O(width + number of labels on the boundary).

FOLLOW-UP 3 Find the area of the largest island.

Count the cells during each search and keep the maximum. Same traversal, one counter.

FOLLOW-UP 4 The grid is huge and does not fit in memory.

Process it in strips of rows, label islands in each strip, and merge labels across strip boundaries with a Union-Find over the labels on the boundary row. **Cost:** O(m * n) time overall and O(width + labels on the boundary row) memory per strip.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Number of Islands
#
# Examples: [[1,1,0,0],[1,1,0,0],[0,0,1,1],[0,0,1,1]] -> 2
.
# Streaming: [[0,0],[0,1],[1,1]] adds -> counts [1, 1, 1]
# (last addition merges the two land cells).
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if land can also be removed?
# 2. How do you handle a grid too large for memory?
# 3. Find the area of the largest island.
# 4. The grid is huge and does not fit in memory.
#
# INTERVIEW NOTES
# Each island is a connected group of land cells, so count
# how many times you start a fresh search at an unvisited
# land cell. The search itself (DFS or BFS) just marks
# everything reachable. If land is added over time, do not
# rerun the search: Union-Find keeps the count up to date.
#
# BRUTE FORCE (say it first): Recursive flood fill
# Scan; at each unvisited land cell count one island and
# recurse in four directions to mark it. The textbook
# answer and the oracle; deep recursion can overflow on a
# large snake-shaped island. Time O(m * n). Space O(m * n)
# recursion.


# BRUTE FORCE (say it first): Recursive flood fill | time
# O(m * n)
def islands_flood(grid):
    if not grid:
        return 0
    m, n = len(grid), len(grid[0])
    seen = set()

    def fill(r, c):
        if (
            not (0 <= r < m and 0 <= c < n)
            or grid[r][c] != 1
            or (r, c) in seen
        ):
            return
        seen.add((r, c))
        for dr, dc in (
            (1, 0),
            (-1, 0),
            (0, 1),
            (0, -1),
        ):
            fill(r + dr, c + dc)

    count = 0
    for r in range(m):
        for c in range(n):
            if grid[r][c] == 1 and (r, c) not in seen:
                count += 1
                fill(r, c)
    return count


# HELPERS for the base question
DIRS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def neighbors(r, c, rows, cols):
    # the cells beside (r, c) that are inside the grid
    for dr, dc in DIRS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            yield nr, nc


def flood(grid, start, seen):
    # visit all land joined to start; return the count
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack, size = [start], 0
    # a stack instead of recursion: no recursion limit
    while stack:
        r, c = stack.pop()
        size += 1
        for nb in neighbors(r, c, rows, cols):
            if grid[nb[0]][nb[1]] == 1 and nb not in seen:
                seen.add(nb)
                stack.append(nb)
    return size


def island_sizes(grid):
    if not grid:
        return []
    seen, sizes = set(), []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            # an island starts at land not visited yet
            if grid[r][c] == 1 and (r, c) not in seen:
                sizes.append(flood(grid, (r, c), seen))
    return sizes


# BASE QUESTION (optimal): Iterative DFS flood fill
# Idea: Scan every cell; each unvisited land cell starts a
# new island, and an explicit stack marks all connected
# land as visited. Iterative DFS avoids Python's recursion
# limit on large grids.
def num_islands(grid):
    # one entry per island
    return len(island_sizes(grid))


# VARIANT: land cells arrive one at a time
class IslandCounter:
    """Follow-up: land cells arrive one at a time."""

    def __init__(self, rows, cols):
        self.rows, self.cols = rows, cols
        # land cell -> parent cell (union-find)
        self.parent = {}
        self.count = 0

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x
```

```
    def add_land(self, r, c):
        # adding the same cell twice changes nothing
        if (r, c) in self.parent:
            return self.count
        self.parent[(r, c)] = (r, c)
        self.count += 1
        # join with every land neighbour
        for nb in neighbors(r, c, self.rows, self.cols):
```

```
            if nb in self.parent:
                a, b = self.find((r, c)), self.find(nb)
                if a != b:
                    self.parent[a] = b
                    # two islands merged into one
                    self.count -= 1
        return self.count


# FOLLOW-UP 3: Find the area of the largest island.
# Answer: Count the cells during each search and keep the
# maximum. Same traversal, one counter.
def largest_island(grid):
    # follow-up: the area of the biggest island
    return max(island_sizes(grid), default=0)


if __name__ == "__main__":
    g = [[1, 1, 0, 0], [1, 1, 0, 0],
         [0, 0, 1, 1], [0, 0, 1, 1]]
    assert num_islands(g) == 2
    assert largest_island(g) == 4
    assert num_islands([]) == 0
    # streaming adds: counts stay 1, 1, 1
    ic = IslandCounter(2, 2)
    adds = [(0, 0), (0, 1), (1, 1)]
    counts = [ic.add_land(r, c) for r, c in adds]
    assert counts == [1, 1, 1]
    # two separate cells make two islands
    ic = IslandCounter(3, 3)
    assert ic.add_land(0, 0) == 1
    assert ic.add_land(2, 2) == 2
    assert ic.add_land(1, 1) == 3
```

NOTES FROM CANDIDATES

For streaming, Union-Find with path compression is the natural choice — each addLand is amortized alpha(n).
In one onsite, the follow-up arrived after a working BFS solution and explicitly required a Union-Find rewrite; retrofitting the original traversal consumed the remaining time.
Common edge cases: diagonal connectivity (some prompts use 8-connectivity), grid with no land.

PREPARATION

Write both the static count and the streaming addLand version from memory in under 15 min total.
Be ready to switch between DFS / BFS / Union-Find on demand.

STUDY HERE: topic 4, Graphs and Search | main topic: Union-find

Site tags: graph, dfs, bfs, union-find, clustering, medium

MEDIUM | 1Point3Acres | phone-screen | 17 of 21

2D Points Connected Components by Radius

ASKED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 40.7** | **30 DAYS 0** | **PRIORITY 91/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Union-find

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (related)

Frequency: Low | Last asked: 2026-08-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

QUESTION

Given 2D points and radius **r**, connect two points when their distance is at most **r**; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

APPROACH

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

HOW TO THINK ABOUT IT

Two points are neighbours when their squared distance is at most r squared (no square roots needed), and connectivity is transitive, so this is counting connected components: union-find or a DFS over the neighbour relation. Checking every pair is O(n^2); bucketing points into cells of side r means a point only needs to be compared with the 3 x 3 cells around it.

Basis: LeetCode 547, Number of Provinces, where two points are linked when their distance is at most r.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points, r):
    graph = defaultdict(list)
    ids = range(len(points))
    for i, j in combinations(ids, 2):
        if dist2(points[i], points[j]) <= r * r:
            graph[i].append(j)
            graph[j].append(i)
    seen, clusters = set(), 0
    for i in range(len(points)):
        if i not in seen:
            clusters += 1
            dfs(graph, i, seen)
    return clusters
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every pair is compared, O(n^2). Putting points in cells of side r means each point is compared only with the 3 x 3 cells around it.

OPTIMAL SOLUTION · Union-find with a spatial grid

TIME	SPACE
O(n) about when points are spread out (O(n^2) if they all share a cell)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why compare squared distances?

sqrt is slow and inexact; dist <= r is the same as dist^2 <= r^2 for non-negative numbers, and with integer coordinates the comparison is exact. It also avoids floating-point surprises right at the boundary.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: 2D Points Connected Components by Radius
#
# Given 2D points and radius r, connect two points when
# their distance is at most r; connectivity is transitive,
# and the output is the final number of clusters. The
# interviewer accepted or preferred a brute-force neighbor
# scan after discussing optimization ideas. Input: a list
# of 2D points and a radius r. Two points are connected if
# their Euclidean distance is <= r. Connectivity is
# transitive: if A connects to B and B connects to C, all
# three belong to the same cluster. Return the number of
# final clusters.
#
# FOLLOW-UPS (answered below, in this order):
```

```python
# 1. Why compare squared distances?
#
# INTERVIEW NOTES
# Two points are neighbours when their squared distance is
# at most r squared (no square roots needed), and
# connectivity is transitive, so this is counting
# connected components: union-find or a DFS over the
# neighbour relation. Checking every pair is O(n^2);
# bucketing points into cells of side r means a point only
# needs to be compared with the 3 x 3 cells around it.
#
# BRUTE FORCE (say it first): Compare every pair and run a
# DFS
# Compare every pair of points, build the neighbour lists
# and count components with a DFS. This is the baseline
# the interviewer accepted, and it is fine for a few
# thousand points. Time O(n^2). Space O(n).


from collections import defaultdict


# BRUTE FORCE (say it first): Compare every pair and run a
# DFS | time O(n^2)
def components_brute(points, r):
    n, seen, count = len(points), set(), 0
    near = (
        lambda a, b: (a[0] - b[0]) ** 2
        + (a[1] - b[1]) ** 2
        <= r * r
    )
    for s in range(n):
        if s in seen:
            continue
        count += 1
        stack = [s]
        seen.add(s)
        while stack:
            u = stack.pop()
            for v in range(n):
                if v not in seen and near(
                    points[u], points[v]
                ):
                    seen.add(v)
                    stack.append(v)
    return count


# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# FOLLOW-UP 1: Why compare squared distances?
# Answer: sqrt is slow and inexact; dist <= r is the same
# as dist^2 <= r^2 for non-negative numbers, and with
# integer coordinates the comparison is exact. It also
# avoids floating-point surprises right at the boundary.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def near(p, q, r):
    # squared distance: no square root, exact for ints
    dx, dy = p[0] - q[0], p[1] - q[1]
    return dx * dx + dy * dy <= r * r


# BASELINE: compare every pair
def count_clusters_pairs(points, r):
    # the accepted baseline: compare every pair, O(n^2)
    uf = UnionFind(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if near(points[i], points[j], r):
                uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


# BASE QUESTION (optimal): Union-find with a spatial grid
# Idea: Put each point in the cell (x // r, y // r). Two
# points within distance r are in the same or adjacent
# cells, so only those 9 cells are compared.
def count_clusters(points, r):
    # grid of cells r wide: only 9 cells need a look
    size = r if r > 0 else 1
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // size, y // size)].append(i)
    uf = UnionFind(len(points))
    for i, (x, y) in enumerate(points):
        cx, cy = x // size, y // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                cell = cells.get((cx + dx, cy + dy), ())
                for j in cell:
                    q = points[j]
                    if j > i and near((x, y), q, r):
                        uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


if __name__ == "__main__":
    # three points joined only through the middle one
    line = [(0, 0), (3, 0), (6, 0)]
    assert count_clusters(line, 3) == 1
    assert count_clusters_pairs(line, 3) == 1
    # the far point stays alone
    far = [(0, 0), (3, 0), (7, 0)]
    assert count_clusters(far, 3) == 2
    # radius 0 only joins identical points
    same = [(1, 1), (1, 1), (2, 2)]
    assert count_clusters(same, 0) == 2
    assert count_clusters([], 5) == 0
    # both methods agree on a scatter of points
    pts = [(i * 7 % 13, i * 5 % 11) for i in range(30)]
    for r in (0, 1, 2, 3, 5):
        assert count_clusters(pts, r) == (
            count_clusters_pairs(pts, r))
```

NOTES FROM CANDIDATES

DFS, BFS, or Union-Find all fit the core connected-components model.

For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.

Clarify squared-distance comparison to avoid unnecessary square roots.

PREPARATION

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

STUDY HERE: topic 4, Graphs and Search | main topic: Union-find

Site tags: graph, union-find, hard

HARD | 1Point3Acres | onsite-coding, phone-screen | 18 of 21

Friend Graph with Timeline (Friend / Unfriend Events)

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 186/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Union-find

Optimal answer from: written and tested here for this exact question

Frequency: Medium | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/friend-graph-with-timeline

QUESTION

EXAMPLES

```
Base: N=3, events [(1,A,B,friend), (5,B,C,friend)] ->
    answer 5.
Follow-up 1: after the above events, query (0, A, C)
    -> false, (5, A, C) -> true.
```

APPROACH

Have weighted/path-compressed union-find ready and warm. Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.

Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the answer is for the base question (earliest time all N people are connected), what to return if it never happens, whether unfriend events exist, and whether queries ask about a past time.

BRUTE FORCE TO SAY OUT LOUD

For each query rebuild the graph from the events up to that time and run BFS. O(E + N) per query.

OPTIMAL IDEA

Base: sort events and union-find, return the time the component count hits 1. Query at a time t: union by size without path compression and record each union's time; two people are connected at t if the largest time on the path between them is at most t. Unfriend: offline, place each edge's lifetime into a segment tree over time and run DFS with a rollback union-find, checking connectivity at the leaves.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

The example giving 5. A graph that never connects. Two events at the same time. Unfriend then refriend.

LIKELY FOLLOW-UPS

Online unfriend (Holm-de Lichtenberg-Thorup or link-cut trees), counting components over time.

PITFALLS

Using path compression where rollback is needed. Treating unfriend like union-find undo.

HOW TO THINK ABOUT IT

Everyone becomes connected at the first moment the graph has one component. If friendships only appear, sort by time and use union-find (components only merge). Unfriending breaks that: removing an edge can split a component and union-find cannot undo. With all events known up front, handle each edge's lifetime in a segment tree over time and use a rollback union-find.

Basis: LeetCode 1101, The Earliest Moment When Everyone Become Friends, plus connectivity queries over time and unfriend events (offline dynamic connectivity). Assumption: events are (time, a, b, 'friend'|'unfriend').

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, events):
    for t in sorted({e.time for e in events}):
        graph = build_graph(events, upto=t)
        if dfs_count(graph, start=0) == n:
            return t  # everyone reached
    return -1
```

TIME	SPACE
O(T * (N + E))	O(N + E)
for T distinct times	

WHY IT IS SLOW

Each time point replays all events and searches the whole graph, O(T * (N + E)). A union-find over sorted events (friend-only) is near-linear; unfriend events need offline dynamic connectivity.

OPTIMAL SOLUTION · Base: earliest time everyone is connected (Union-Find on sorted events)

TIME	SPACE
O(E log E)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does unfriending break plain Union-Find?

A union merges two sets and forgets which edge caused it, so removing that edge cannot tell which members should separate. Online, fully dynamic connectivity needs heavy structures (Holm-de Lichtenberg-Thorup or link-cut trees, polylogarithmic time). When all events are known in advance, the offline segment-tree-over-time plus rollback DSU is the practical answer and the one to code in an interview.

FOLLOW-UP 2 What is the simple answer if the interviewer wants something quick?

Rebuild the graph from the events up to time t and run BFS or DFS for each query: O(E + N) per query. State the cost and then explain how the offline method amortizes it. A correct simple answer is accepted; naming the optimal one shows depth.

FOLLOW-UP 3 Report how many friend groups exist after each event (friends only).

Union-find with a component counter: decrement when a union merges two different roots; record the counter after each event. O(E alpha(N)).

FOLLOW-UP 4 Queries ask 'were A and B connected at time t?' with no unfriending.

Union by size without path compression, recording the time of each union on the edge it creates; two people are connected at t if the largest time on the path between them is at most t.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Friend Graph with Timeline (Friend /
# Unfriend Events)
#
# Examples: Base: N=3, events [(1,A,B,friend),
# (5,B,C,friend)] -> answer 5. Follow-up 1: after the abov
e
# events, query (0, A, C) -> false, (5, A, C) -> true.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does unfriending break plain Union-Find?
# 2. What is the simple answer if the interviewer wants
# something quick?
# 3. Report how many friend groups exist after each event
# (friends only).
# 4. Queries ask 'were A and B connected at time t?' with
# no unfriending.
#
# INTERVIEW NOTES
# Everyone becomes connected at the first moment the graph
# has one component. If friendships only appear, sort by
# time and use union-find (components only merge).
# Unfriending breaks that: removing an edge can split a
# component and union-find cannot undo. With all events
# known up front, handle each edge's lifetime in a segment
# tree over time and use a rollback union-find.
#
# BRUTE FORCE (say it first): Rebuild the graph at every
# event time and search it
# For each distinct time, replay all events up to it to
# get the current friend set and run a DFS from person 0
# to see whether everyone is reachable. Simple, correct
# for friend and unfriend events, and the oracle; it
# rebuilds the graph for every time. Time O(T * (N + E))
# for T distinct times. Space O(N + E).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Rebuild the graph at every
# event time and search it | time O(T * (N + E)) for T
# distinct times
def earliest_connected_rebuild(n, events):
    if n == 1:
        return 0
    for t in sorted({e[0] for e in events}):
        edges = set()
        for et, a, b, kind in sorted(
            events, key=lambda e: e[0]
        ):
            if et > t:
                break
            key = (min(a, b), max(a, b))
            (
                edges.add(key)
                if kind == "friend"
                else edges.discard(key)
            )
        seen, stack = {0}, [0]
        while stack:
            u = stack.pop()
            for a, b in edges:
                for x, y in ((a, b), (b, a)):
                    if x == u and y not in seen:
                        seen.add(y)
                        stack.append(y)
        if len(seen) == n:
            return t
    return -1


# HELPERS for the base question
def by_time(events):
    # events are (time, a, b, "friend" or "unfriend")
    return sorted(events, key=lambda e: e[0])


# FOLLOW-UP 4: Queries ask 'were A and B connected at time
# t?' with no unfriending.
# Answer: Union by size without path compression,
# recording the time of each union on the edge it creates;
# two people are connected at t if the largest time on the
# path between them is at most t.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class TimedUnionFind:
    """Union by size, no path compression, so every
    link keeps the time it was made."""

    def __init__(self, n):
        # every person starts alone in a group of one
        self.parent = list(range(n))
        # sizes decide which group hangs under which
        self.size = [1] * n
```

```
        # time a node was linked under its parent
        self.link_time = [0] * n
        # how many separate groups exist right now
        self.groups = n

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, t):
        # find the leaders of both groups
        ra, rb = self.find(a), self.find(b)
        # already in one group: nothing changes
        if ra == rb:
            return False
        # make ra the bigger group
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        # hang the smaller group under the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        # remember when this link appeared
        self.link_time[rb] = t
        # two groups became one
        self.groups -= 1
        return True

    def connected_at(self, a, b, t):
        # a person is always connected to themselves
        if a == b:
            return True
        # climb from a, noting the newest link
        seen, newest, x = {a: 0}, 0, a
        while self.parent[x] != x:
            newest = max(newest, self.link_time[x])
            x = self.parent[x]
            seen[x] = newest
        # climb from b until we reach an ancestor of a
        newest, y = 0, b
        while y not in seen:
```

```
            # b's leader reached without meeting a
            if self.parent[y] == y:
                return False
            newest = max(newest, self.link_time[y])
            y = self.parent[y]
        # connected at t if all path links are <= t
        return max(newest, seen[y]) <= t


# BASE QUESTION (optimal): earliest time everyone is
# connected (Union-Find on sorted events)
# Idea: Sort by time and union the two people for each
# friend event. The first time the component count reaches
# 1 is the answer; -1 if it never does.
def earliest_all_connected(n, events):
    # zero or one person is connected from the start
    if n <= 1:
        return 0
    uf = TimedUnionFind(n)
    # handle friendships in time order
    for t, a, b, kind in by_time(events):
        # the base question has friend events only
        if kind == "friend":
            uf.union(a, b, t)
            # one group left: everyone is connected now
            if uf.groups == 1:
                return t
    # the groups never merged into one
    return -1
```

```
# FOLLOW-UP 1: Why does unfriending break plain Union-
# Find?
# Answer: A union merges two sets and forgets which edge
# caused it, so removing that edge cannot tell which
# members should separate.
# FOLLOW-UP 2: What is the simple answer if the
# interviewer wants something quick?
# Answer: Rebuild the graph from the events up to time t
# and run BFS or DFS for each query: O(E + N) per query.
# State the cost and then explain how the offline method
# amortizes it.
def connected_with_unfriends(n, events, a, b, t):
    # replay every event up to time t on a set of edges
    edges = set()
```

```
    for time, x, y, kind in by_time(events):
        if time > t:
            break
        pair = (min(x, y), max(x, y))
        # friend adds the edge, unfriend removes it
        if kind == "friend":
            edges.add(pair)
        else:
            edges.discard(pair)
    # build neighbour lists from the surviving edges
    adj = defaultdict(list)
    for x, y in edges:
        adj[x].append(y)
        adj[y].append(x)
    # breadth-first search from a looking for b
    seen, queue = {a}, deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in adj[u]:
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False


# FOLLOW-UP 3: Report how many friend groups exist after
# each event (friends only).
# Answer: Union-find with a component counter: decrement
# when a union merges two different roots; record the
# counter after each event. O(E alpha(N)).
def groups_after_each_event(n, events):
    uf = TimedUnionFind(n)
    counts = []
    # replay friend events in time order
    for t, a, b, kind in by_time(events):
        if kind == "friend":
            uf.union(a, b, t)
            # record the group count after this event
            counts.append(uf.groups)
    return counts


if __name__ == "__main__":
```

```
    # people A, B, C are 0, 1, 2
    ev = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]
    # everyone is connected at time 5
    assert earliest_all_connected(3, ev) == 5
    # a missing friendship means never connected
    assert earliest_all_connected(3, ev[:1]) == -1
    assert earliest_all_connected(1, []) == 0
    # groups: 3 -> 2 -> 1
    assert groups_after_each_event(3, ev) == [2, 1]
    # A and C are apart at time 0, together at time 5
    uf = TimedUnionFind(3)
```

```
    for t, a, b, k in ev:
        uf.union(a, b, t)
    assert not uf.connected_at(0, 2, 4)
    assert uf.connected_at(0, 2, 5)
    # an unfriend breaks the link again
    ev2 = ev + [(7, 1, 2, "unfriend")]
    assert connected_with_unfriends(3, ev2, 0, 2, 6)
    assert not connected_with_unfriends(3, ev2, 0, 2, 8)
```

NOTES FROM CANDIDATES

The base problem looks innocent; the follow-up about unfriending is the actual signal. In practice, interviewers accept "rebuild graph from scratch per query + BFS" as a working answer if you also articulate why the optimal algorithm is fully dynamic connectivity.

The same family shows up as "friends form a chain over time" or "chess pieces R / L that can move plus walls" — the connectivity backbone is identical.

The follow-up the interviewer is fishing for is consistently a segment tree built over the event timeline; have the offline algorithm ready.

Two algorithmic families converge here: (1) offline — sort all events by time, run union-find, snapshot connectivity at each query T; (2) online with unfriend support — vanilla union-find doesn't handle deletion, so either rebuild per query, use link-cut trees, or apply offline reverse-time processing (treat unfriends as adds in reverse). Clarify with the interviewer which they want.

In an L4 R1 variant, the interviewer requested a graph-and-DFS implementation for the unfriend extension, then continued asking for an optimization that reused the earlier union-find work. Completing the rebuild approach did not end that follow-up.

PREPARATION

Have weighted/path-compressed union-find ready and warm.

Practice the offline trick: segment tree over time intervals containing each edge, DFS the tree applying / rolling back unions.

Memorize the punchline for fully-dynamic: "online deletion breaks union-find; offline is OK; for online you need Holm–Lichtenberg or link-cut tree (O(polylog n))."

STUDY HERE: topic 4, Graphs and Search | main topic: Shortest paths

Site tags: graph, bfs, shortest-path, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 19 of 21

Train / Flight Schedule Reachability

ASKED 2026-10-08 | 12 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 182/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Shortest paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2092 Find All People With Secret (related)

Frequency: Low | Last asked: 2026-10-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/train-schedule-reachability

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late
    for the Y->B connection, but the second one makes
    it.
```

APPROACH

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a connection with departure equal to arrival is allowed, whether a leg can arrive at the same time it leaves, whether times are integers, and whether the answer is yes or no or the route.

INTERVIEW-DAY PLAN

BRUTE FORCE TO SAY OUT LOUD

Search over (station, arrival time) states following any leg that departs later. O(legs^2).

OPTIMAL IDEA

Sort the legs by departure time and keep earliest[station]. The source starts at minus infinity. For each leg, if earliest[origin] is before (or equal to, if allowed) the departure, set earliest[destination] = min(earliest[destination], arrival). The destination is reachable when it has an entry. O(L log L); the table also gives the earliest arrival time, and a parent leg per station gives the route.

DRY-RUN THESE CASES

The example with the late first A to Y leg, source equals destination, no legs, a connection with arrival exactly equal to departure, parallel legs, a cycle in the stations.

LIKELY FOLLOW-UPS

Return the route or the earliest arrival, many queries on one schedule (group by origin and use a time-ordered heap), or minimum number of legs.

PITFALLS

Ignoring time and treating it as station reachability (the stated trap). Mixing up strict and non-strict connection times. Processing legs in input order instead of departure order.

HOW TO THINK ABOUT IT

Plain station-to-station reachability is wrong because a leg is only usable if you are already at its departure station before it leaves. Time is part of the state, so ask what you need to know about each station: the earliest time you can be there. Process the legs in order of departure time; a leg is usable if the earliest arrival at its origin is not later than its departure, and then it can improve the earliest arrival at its destination. One sort and one pass answer reachability, and the same table gives the earliest arrival time.

Basis: LeetCode 2092, Find All People With Secret, and LeetCode 787, Cheapest Flights Within K Stops: reachability where time is part of the state.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def reachable(legs, src, dst):
    stack, seen = [(src, -INF)], set()
    while stack:
        station, now = stack.pop()
        if station == dst:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for a, b, dep, arr in legs:
            if a == station and dep > now:
                stack.append((b, arr))
    return False
```

BRUTE FORCE

TIME	SPACE
O(legs^2)	O(legs)

WHY IT IS SLOW

Every state rescans all legs and the same station is reached at many times, O(legs^2). Sorting the legs by departure and keeping only the earliest arrival per station answers in O(L log L).

OPTIMAL SOLUTION · Sort by departure and keep earliest arrivals

TIME	SPACE
O(L log L)	O(S)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the earliest arrival time at the destination.

The table already holds it: earliest_arrivals(legs, source)[dest], or infinity when the station is unreachable. That is the same one pass, so it costs nothing extra.

FOLLOW-UP 2 Print the route, not just yes or no.

Remember the leg that last improved each station's earliest arrival, then walk back from the destination to the source and reverse. Because only a strictly better arrival replaces the stored leg, the stored legs form a valid chain in time.

FOLLOW-UP 3 Many queries on the same schedule: what changes in the graph representation?

Group the legs by origin station and sort each list by departure time. For a query the earliest arrival is relaxed with a heap ordered by time (Dijkstra on time), and the first leg you may board at a station is found with bisect on its departure list, so each query costs O(L log S) instead of re-sorting every time. If the source is fixed and the queries only differ by destination, keep the earliest-arrival table and answer each in O(1).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Train / Flight Schedule Reachability
#
# Given trains (or flights) as (from, to, departure,
# arrival), decide whether you can travel from a source to
# a destination, boarding a later leg only if it departs
# after you arrive. Graph reachability with time state,
# solved with BFS / DFS. Input: a schedule of legs, each
# with a departure station, an arrival station, a
# departure time, and an arrival time; plus a source and a
# destination. Output: whether the destination is
# reachable from the source, where a leg can only be taken
# if it departs after you arrive at its departure station.
# The same prompt appears as a flight schedule between
# airports in new-grad R1. Examples: trains = [ ("A", "Y",
# dep=1, arr=20), ("A", "Y", dep=2, arr=5), ("Y", "B",
# dep=6, arr=9) ] source = "A", destination = "B" Output:
# true — the first A->Y train arrives too late for the Y->
B
# connection, but the second one makes it.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the earliest arrival time at the destination.
# 2. Print the route, not just yes or no.
# 3. Many queries on the same schedule: what changes in
# the graph representation?
#
# INTERVIEW NOTES
# Plain station-to-station reachability is wrong because a
# leg is only usable if you are already at its departure
# station before it leaves. Time is part of the state, so
# ask what you need to know about each station: the
# earliest time you can be there. Process the legs in
# order of departure time; a leg is usable if the earliest
# arrival at its origin is not later than its departure,
# and then it can improve the earliest arrival at its
# destination. One sort and one pass answer reachability,
# and the same table gives the earliest arrival time.
#
# BRUTE FORCE (say it first): Explore every (station,
# arrival time) state
# Search from the source over states (station, time you
# arrived), following every leg that departs after the
# current arrival. Correct and the obvious first answer,
# but each state rescans all legs. Time O(legs^2). Space
# O(legs).
```

```
# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first, it also checks the answers)
def reachable_brute(legs, source, dest, strict=True):
    # the oracle: search over (station, arrival time)
    # states
    seen, stack = set(), [(source, -INF)]
    while stack:
        station, now = stack.pop()
        if station == dest:
            return True
        if (station, now) in seen:
            continue
        seen.add((station, now))
        for frm, to, dep, arr in legs:
            ok = dep > now if strict else dep >= now
            if frm == station and ok:
                stack.append((to, arr))
    return False


# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def earliest_arrivals(legs, source, strict=True):
    """Time is part of the state: keep the earliest time
    we can be at each station. Legs go in departure
    order, so every leg that could feed this one is
    already done."""
```

```
    earliest = {source: -INF}
    by_departure = sorted(legs, key=lambda x: x[2])
    for frm, to, dep, arr in by_departure:
        have = earliest.get(frm, INF)
        # we must already be there before it leaves
        if have < dep or (not strict and have == dep):
            if arr < earliest.get(to, INF):
                earliest[to] = arr
    return earliest
```

```
# BASE QUESTION (optimal): Sort by departure and keep
# earliest arrivals
# Idea: Sort the legs by departure time. Keep
# earliest[station] = earliest known arrival, with the
# source at minus infinity.
def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)


# FOLLOW-UP 2: Print the route, not just yes or no.
# Answer: Remember the leg that last improved each
# station's earliest arrival, then walk back from the
# destination to the source and reverse.
def itinerary(legs, source, dest, strict=True):
    """Follow-up: the route. Remember the leg that last
    improved each station, then walk back from the end."""
    earliest, came = {source: -INF}, {}
    for leg in sorted(legs, key=lambda x: x[2]):
        frm, to, dep, arr = leg
        have = earliest.get(frm, INF)
        usable = have < dep or (not strict
                                and have == dep)
        if usable and arr < earliest.get(to, INF):
            earliest[to], came[to] = arr, leg
    if dest not in earliest:
        return None
    route = []
    while dest != source:
        leg = came[dest]
        route.append(leg)
        dest = leg[0]
    return route[::-1]


if __name__ == "__main__":
    legs = [("A", "Y", 1, 20), ("A", "Y", 2, 5),
            ("Y", "B", 6, 9)]
    assert reachable(legs, "A", "B")
    # without the early A to Y train the connection is
    # lost
    assert not reachable(legs[:1] + legs[2:], "A", "B")
    assert earliest_arrivals(legs, "A")["B"] == 9
    assert itinerary(legs, "A", "B") == [legs[1], legs[2]]
    assert itinerary(legs, "B", "A") is None
    assert reachable(legs, "A", "A")
```

```
    touch = [("A", "B", 1, 5), ("B", "C", 5, 8)]
    assert not reachable(touch, "A", "C")
    assert reachable(touch, "A", "C", strict=False)
    import random
    for _ in range(300):
        many = []
        for _ in range(random.randint(0, 8)):
            dep = random.randint(0, 9)
            many.append((random.choice("ABCD"),
                         random.choice("ABCD"), dep,
                         dep + random.randint(1, 4)))
        src = random.choice("ABCD")
        dst = random.choice("ABCD")
        for strict in (True, False):
            want = reachable_brute(many, src, dst, strict)
            got = reachable(many, src, dst, strict)
```

```
            assert got == want
```

NOTES FROM CANDIDATES

Pitfalls: Plain station-level reachability is wrong: in the example, A->Y->B exists as a path, but only one of the two A->Y legs makes the connection.
Clarify whether a connection with departure exactly equal to arrival is allowed.

Follow-ups: Follow-ups push on graph representation

PREPARATION

Implement BFS / DFS over (station, arrival time) states that only follows legs departing after the current arrival.
Hand-trace the example, including the late A->Y leg that must be rejected.

STUDY HERE: topic 4, Graphs and Search | main topic: Shortest paths

Site tags: graph, bfs, shortest-path, medium

MEDIUM | 1Point3Acres | onsite-coding | 20 of 21

Shortest Path Nodes and Escaping a Pursuer

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 212/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Shortest paths

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/shortest-path-nodes-and-pursuer-escape

QUESTION

Unweighted undirected graph: shortest path from Alice's node to a target (BFS). Follow-up 1: all nodes that lie on at least one shortest path. Follow-up 2: Bob starts at another node at the same time; decide whether Alice can follow some shortest path to the target without being caught.

Input: an undirected, unweighted graph; Alice starts at one node and must reach a target node.
Return the shortest path (length) from Alice to the target.
Follow-up 1: find all nodes that can appear on some shortest path from Alice to the target.
Follow-up 2: Bob is at another node. Alice picks one shortest path and starts moving; Bob starts moving at the same time. Decide whether Alice can reach the target safely without being caught by Bob.

APPROACH

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target]. For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

HOW TO THINK ABOUT IT

BFS from Alice gives dA and BFS from the target gives dT. A node is on some shortest path exactly when dA[v] + dT[v] equals the shortest length. For the pursuer, BFS from Bob gives dB. State the rule you assume: Alice is caught at a node v if Bob can be there no later than she is (dB[v] <= dA[v]). Alice is safe when a shortest path exists whose every node has dB[v] > dA[v].

Basis: Written for this question (BFS layers, as in LeetCode 1976, Number of Ways to Arrive at Destination).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(graph, alice, bob, target):
    dA = bfs_dist(graph, alice)
    dB = bfs_dist(graph, bob)
    paths = all_paths(graph, alice, target)
    for path in paths:
        if len(path) - 1 != dA[target]:
            continue  # not a shortest path
        if all(dB[v] > dA[v] for v in path):
            return True  # Bob is always late
    return False
```

TIME	SPACE
exponential in the worst ... exponential in the worst case	**O(paths)**

WHY IT IS SLOW

The number of shortest paths can be exponential. Three BFS runs (from Alice, the target and Bob) decide every node in O(V + E).

OPTIMAL SOLUTION · Three BFS runs and a safe shortest-path DAG

TIME	SPACE
O(V + E)	**O(V)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if Bob can also meet Alice in the middle of an edge?

Then arriving at the same node at the same time is not the only way to be caught: Alice moving u -> v while Bob moves v -> u swaps places. Add that case by also rejecting an edge (u, v) when dB[v] == dA[u] and dB[u] == dA[v]. Always clarify what 'caught' means first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shortest Path Nodes and Escaping a
# Pursuer
#
# Unweighted undirected graph: shortest path from Alice's
# node to a target (BFS). Follow-up 1: all nodes that lie
# on at least one shortest path. Follow-up 2: Bob starts
# at another node at the same time; decide whether Alice
# can follow some shortest path to the target without
# being caught. Input: an undirected, unweighted graph;
# Alice starts at one node and must reach a target node.
# Return the shortest path (length) from Alice to the
# target. Follow-up 1: find all nodes that can appear on
# some shortest path from Alice to the target. Follow-up
# 2: Bob is at another node. Alice picks one shortest path
# and starts moving; Bob starts moving at the same time.
# Decide whether Alice can reach the target safely without
# being caught by Bob.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if Bob can also meet Alice in the middle of an
# edge?
#
# INTERVIEW NOTES
# BFS from Alice gives dA and BFS from the target gives
# dT. A node is on some shortest path exactly when dA[v] +
# dT[v] equals the shortest length. For the pursuer, BFS
# from Bob gives dB. State the rule you assume: Alice is
# caught at a node v if Bob can be there no later than she
# is (dB[v] <= dA[v]). Alice is safe when a shortest path
# exists whose every node has dB[v] > dA[v].
#
# BRUTE FORCE (say it first): Enumerate every shortest
# path
# Depth-first search all simple paths of the shortest
# length from Alice to the target and check each node
# against Bob's arrival time. Correct and the oracle; the
# number of shortest paths can be huge. Time exponential
# in the worst case. Space O(paths).


from collections import deque


# HELPERS for the brute force
def bfs(adj, s):
    # distance from s to every reachable node
    d = {s: 0}
    q = deque([s])
    while q:
        u = q.popleft()
        for v in adj[u]:
            if v not in d:
                d[v] = d[u] + 1
                q.append(v)
    return d


# BRUTE FORCE (say it first): Enumerate every shortest
# path | time exponential in the worst case
def escape_brute(adj, a, t, b):
    def bfs(s):
        d = {s: 0}
        q = deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if v not in d:
                    d[v] = d[u] + 1
                    q.append(v)
        return d

    da, db = bfs(a), bfs(b)
    if t not in da:
        return None, set(), False
    nodes, safe = set(), False

    def go(u, path):
        nonlocal safe
        if len(path) - 1 > da[t]:
            return
        if u == t:
            if len(path) - 1 == da[t]:
                nodes.update(path)
                if all(
                    db.get(v, 10**9) > i
                    for i, v in enumerate(path)
                ):
                    safe = True
            return
        for v in adj[u]:
            if v not in path:
                go(v, path + [v])

    go(a, [a])
    return da[t], nodes, safe


# HELPERS for the base question
FAR = 10 ** 9


# BASE QUESTION (optimal): Three BFS runs and a safe
# shortest-path DAG
# Idea: dA, dT and dB come from BFS. The nodes on some
# shortest path are those with dA[v] + dT[v] == dA[t].
def shortest_length(adj, a, t):
    # the base question: edges on the shortest path
    return bfs(adj, a).get(t)


# QUESTION FOLLOW-UP 1: all nodes on a shortest path
def nodes_on_shortest_paths(adj, a, t):
    da, dt = bfs(adj, a), bfs(adj, t)
    if t not in da:
        return set()
    # a node is on some shortest path if the two
    # distances add up to the full length
    return {v for v in da if v in dt
            and da[v] + dt[v] == da[t]}


# QUESTION FOLLOW-UP 2: can Alice escape Bob?
def can_escape(adj, a, t, b, meet_on_edge=False):
    da, db = bfs(adj, a), bfs(adj, b)
    if t not in da:
        return False
    on_path = nodes_on_shortest_paths(adj, a, t)

    def safe(u, v):
        # Bob must reach v strictly after Alice does
        if db.get(v, FAR) <= da[v]:
            return False
        # FOLLOW-UP 1: meet_on_edge=True: they meet inside
        # an edge
        # optional: they swap places on the edge u-v
```

```
        swap = db.get(v, FAR) == da[u]
        swap = swap and db.get(u, FAR) == da[v]
        return not (meet_on_edge and swap)

    # Bob standing on Alice's start catches her at once
    if db.get(a, FAR) <= 0:
        return False
    # walk the shortest-path layers using safe nodes only
    frontier, seen = [a], {a}
    while frontier:
        nxt = []
        for u in frontier:
            for v in adj[u]:
                if v in on_path and da[v] == da[u] + 1 \
                        and v not in seen and safe(u, v):
                    seen.add(v)
                    nxt.append(v)
        frontier = nxt
    return t in seen


if __name__ == "__main__":
    # a square 0-1-3 and 0-2-3 plus a tail 3-4
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3],
           3: [1, 2, 4], 4: [3]}
    assert shortest_length(adj, 0, 4) == 3
    on_path = nodes_on_shortest_paths(adj, 0, 4)
    assert on_path == {0, 1, 2, 3, 4}
    # Bob waiting at the target catches Alice there
    assert not can_escape(adj, 0, 4, 4)
    chain = {0: [1], 1: [0, 2], 2: [1, 3], 3: [2]}
    # Bob starts at the target and reaches it first
    assert not can_escape(chain, 0, 3, 3)
    # Bob starts behind Alice: she stays ahead
    assert can_escape(chain, 1, 3, 0)
```

NOTES FROM CANDIDATES

Asked as an L4 onsite coding round.
Follow-up 2 has an open definition of "caught": clarify whether it means arriving at the same node at the same time, Bob arriving first, or also meeting on an edge; also clarify whether Alice may choose her shortest path knowing Bob's position.

PREPARATION

Run BFS from both the start and the target; a node v is on some shortest path exactly when distA[v] + distT[v] == distA[target].
For the pursuer follow-up, add a BFS from Bob and compare arrival times node by node along candidate shortest paths; practice turning "exists a safe shortest path" into a search over only those on-path nodes where Alice arrives strictly before Bob.
Test with disconnected targets, Bob starting on the target, and multiple equally short routes where only some are safe.

STUDY HERE: topic 4, Graphs and Search | main topic: Shortest paths

Site tags: graph, bfs, dijkstra, medium

MEDIUM | 1Point3Acres | onsite-coding | 21 of 21

Minimum Direction Violations in a Directed Graph

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | **ALL TIME 10.6** | **30 DAYS 0** | **PRIORITY 165/228**

ABOUT THIS QUESTION

Topic: Graphs and Search: Shortest paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1466 Reorder Routes to Make All Paths Lead to the City Zero (related)

Frequency: Single report | Last asked: 2026-07-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/minimum-direction-violations-directed-graph

QUESTION

EXAMPLES

```
n = 5
edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
start = 0
end = 4

path: 0 -> 1 -> 2 -> 3 -> 4
cost: 0 + 1 + 0 + 1 = 2
output: 2
```

APPROACH

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what a violation is (traversing an edge against its direction), whether edge costs are all 1, what to return when the end cannot be reached, and whether the graph has parallel edges or self loops.

BRUTE FORCE TO SAY OUT LOUD

Plain BFS ignoring direction minimizes edges, not violations, so say why it is wrong. Dijkstra with costs 0 and 1 works at O((n + m) log n).

OPTIMAL IDEA

Add every edge twice: forward with weight 0, backward with weight 1. Run 0-1 BFS with a deque: relax a weight-0 edge by pushing to the front, a weight-1 edge to the back. The deque always holds nodes whose distances differ by at most 1, so O(n + m).

DRY-RUN THESE CASES

Start equals end (0). A graph with no path. A path that needs every edge reversed. A shortcut that costs more edges but fewer violations.

LIKELY FOLLOW-UPS

General edge costs (Dijkstra), shortest path itself (parent pointers), or negative costs (Bellman-Ford).

PITFALLS

Using visited flags instead of a distance check. Pushing weight-0 edges to the back.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

Going against an edge costs 1 and going with it costs 0, so it is a shortest-path problem with costs of only 0 and 1. Plain BFS counts edges, not violations, so it is the wrong tool. A deque lets 0-cost moves jump the queue (push front) and 1-cost moves wait their turn (push back), which is Dijkstra without the heap.

Basis: Related to LeetCode 1466, Reorder Routes: a shortest path where traveling an edge backwards costs 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, start, end):
    arcs = []
    for u, v in edges:
        arcs.append((u, v, 0))  # along the edge
        arcs.append((v, u, 1))  # against it
    dist = [INF] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[end]
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Every round scans all edges, V - 1 rounds: O(V * E). Costs of only 0 and 1 allow a deque search (0-1 BFS) in O(V + E).

OPTIMAL SOLUTION · 0-1 BFS on the doubled graph

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is 0-1 BFS correct, and how does it compare with Dijkstra?

The deque always holds nodes whose distances differ by at most one: front elements are at distance d, back elements at d + 1. Relaxing a zero-weight edge keeps the distance at d (front), a one-weight edge makes d + 1 (back). That is the same order a heap would produce, so it is Dijkstra without the log factor: O(n + m) versus O((n + m) log n).

FOLLOW-UP 2 What if edge costs can be arbitrary non-negative numbers?

Use Dijkstra with a heap on the same doubled graph (forward cost = 0 or the edge cost, backward cost = the violation cost). With negative costs use Bellman-Ford and watch for negative cycles.

FOLLOW-UP 3 Edge costs are arbitrary non-negative numbers (different for each direction).

Use Dijkstra with a heap on the same doubled graph, giving each arc its own cost. 0-1 BFS only works because the costs are 0 or 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimum Direction Violations in a
# Directed Graph
#
# Examples: n = 5 edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
# start = 0 end = 4 path: 0 -> 1 -> 2 -> 3 -> 4 cost: 0 +
# 1 + 0 + 1 = 2 output: 2
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is 0-1 BFS correct, and how does it compare with
# Dijkstra?
# 2. What if edge costs can be arbitrary non-negative
# numbers?
# 3. Edge costs are arbitrary non-negative numbers
# (different for each direction).
#
# INTERVIEW NOTES
# Going against an edge costs 1 and going with it costs 0,
# so it is a shortest-path problem with costs of only 0
# and 1. Plain BFS counts edges, not violations, so it is
# the wrong tool. A deque lets 0-cost moves jump the queue
# (push front) and 1-cost moves wait their turn (push
# back), which is Dijkstra without the heap.
#
# BRUTE FORCE (say it first): Bellman-Ford on the doubled
# graph
# Give every edge a forward copy with cost 0 and a
# backward copy with cost 1, and relax all edges V - 1
# times. Correct and simple, and the oracle for the 0-1
# BFS; it is slower because it repeats full passes over
# the edges. Time O(V * E). Space O(V).


import heapq
```

```
from collections import deque


# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Bellman-Ford on the doubled
# graph | time O(V * E)
def violations_bellman_ford(n, edges, start, end):
    INF = float("inf")
    dist = [INF] * n
    dist[start] = 0
    arcs = [(u, v, 0) for u, v in edges] + [
        (v, u, 1) for u, v in edges
    ]
    for _ in range(max(0, n - 1)):
        changed = False
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    return -1 if dist[end] == INF else dist[end]


# BASE QUESTION (optimal): 0-1 BFS on the doubled graph
# Idea: Add every edge twice: forward with weight 0,
# backward with weight 1. Shortest path with weights in
# {0, 1} is a deque BFS: push a zero-weight neighbour to
# the front and a one-weight neighbour to the back.
def min_violations(n, edges, start, end):
    # 0-1 BFS: following an edge is free, reversing costs
    # 1
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # cost 0 goes to the front, cost 1 to the
                # back
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return -1 if dist[end] == INF else dist[end]


# FOLLOW-UP 3: Edge costs are arbitrary non-negative
# numbers (different for each direction).
# Answer: Use Dijkstra with a heap on the same doubled
# graph, giving each arc its own cost. 0-1 BFS only works
# because the costs are 0 or 1.
def cheapest_route(n, edges, start, end):
    # edges are (u, v, forward_cost, backward_cost)
    graph = [[] for _ in range(n)]
    for u, v, fwd, back in edges:
        graph[u].append((v, fwd))
        graph[v].append((u, back))
    dist = [INF] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
```

```
        d, u = heapq.heappop(heap)
        # skip outdated heap entries
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return -1 if dist[end] == INF else dist[end]


if __name__ == "__main__":
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    # path 0 -> 1 -> 2 -> 3 -> 4 reverses two edges
    assert min_violations(5, edges, 0, 4) == 2
    assert min_violations(5, edges, 0, 0) == 0
    assert min_violations(3, [(0, 1)], 0, 2) == -1
    # the same graph with every reversal costing 1
    costed = [(u, v, 0, 1) for u, v in edges]
    assert cheapest_route(5, costed, 0, 4) == 2
    # different cost per direction
    assert cheapest_route(2, [(0, 1, 5, 2)], 0, 1) == 5
    assert cheapest_route(2, [(0, 1, 5, 2)], 1, 0) == 2
```

NOTES FROM CANDIDATES

Represent each original edge twice: the forward direction with weight 0 and the reverse direction with weight 1.
A regular BFS minimizes the number of edges, not the number of direction violations.
The linear-time target is O(n + m) time and space.

PREPARATION

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 5 OF 10 | LESS COMMON TOPIC

Design, Streaming and Scale

16 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	15
EASY	MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Design, Streaming and Scale | 16 questions | tap a title to jump to it

DATA STRUCTURE DESIGN (14)

E	1. Implement Batched Undo/Redo Layer	PracHub	2026-02-04
M	2. Implement a Reusable Locker Allocation System	PracHub	2026-08-14
M	3. Build Prefix Lookup with a Trie	PracHub	2026-07-03
M	4. Design a restaurant waitlist system	PracHub	2026-03-01
M	5. Determine Reporting Relationships	PracHub	2026-02-22
M	6. Implement Trie Insert and Exact-Word Search	PracHub	2026-02-20
M	7. Design Tic-Tac-Toe With K Players	PracHub	2026-02-12
M	8. Restaurant Waitlist: join / delete / find_first_match	1Point3Acres	2026-02-01
M	9. Design a logger that returns top-K messages	PracHub	2026-01-06
M	10. Implement iterator merging lists with filtering	PracHub	2026-01-06
M	11. Design set with O(1) random access	PracHub	2026-01-01
M	12. Match people to questions using tags and priorities	PracHub	2025-12-27
M	13. Design server allocation for multi-type nodes	PracHub	2025-09-06
M	14. Hit Counter	HackTheRounds	no date

STREAMING AND SCALE (2)

M	15. Find top/bottom-k words in list or stream	PracHub	2026-01-11
M	16. Design an elevator control system	PracHub	2025-09-06

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

EASY | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 1 of 16

Implement Batched Undo/Redo Layer

POSTED 2026-02-04 | 258 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 158/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #1472 Design Browser History

LeetCode match: LC 1472 Design Browser History (related: undo/redo stacks)

Posted on PracHub: 2026-02-04 (updated 2026-05-06)

Original: https://prachub.com/coding-questions/implement-batched-undo-redo-layer

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch.

Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value afte
r the batch string old_value; // value before the batc
h bool has_old_value; // whether the key existed befor
e the batch }
```

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

QUESTION

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op.
Constraints
0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

EXAMPLES

```
Input: [('begin',), ('set', 'color', 'red'),
    ('end',), ('snapshot',), ('undo',),
    ('snapshot',), ('redo',), ('snapshot',)]
Expected Output: [{'color': 'red'}, {}, {'color':
    'red'}]
```

Explanation: A single batch is committed, undone, and redone.

```
Input: [('begin',), ('set', 'a', '1'), ('end',),
    ('begin',), ('set', 'a', '2'), ('set', 'a', '3'),
    ('set', 'b', '9'), ('end',), ('snapshot',),
    ('undo',), ('snapshot',), ('redo',),
    ('snapshot',)]
Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'},
    {'a': '3', 'b': '9'}]
```

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.

Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

APPROACH

Hold the open batch as a map of key -> (oldValue, newValue); endBatch pushes it onto the undo stack and clears the redo stack. undo applies the old values and moves the batch to the redo stack, redo reverses that. Handle an empty batch and repeated sets of one key in a batch (keep the first old value).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what undo does on an empty stack, whether an empty batch counts as an action, whether new edits after an undo clear the redo history (yes, normally), and how nested batches behave.

BRUTE FORCE TO SAY OUT LOUD

Snapshot the whole map after every batch and keep a list of snapshots. Simple but memory heavy.

OPTIMAL IDEA

Hold the open batch as a dict key to (old value, new value), recording the old value only the first time a key is touched in the batch. endBatch pushes it on the undo stack and clears the redo stack. undo pops, restores the old values (deleting keys that did not exist), and pushes onto redo. redo reapplies the new values. O(size of batch) each.

DRY-RUN THESE CASES

Set the same key twice in one batch, set a new key then undo (it must disappear), undo and redo with nothing available, a new batch after an undo (redo cleared), an empty batch.

LIKELY FOLLOW-UPS

Nested batches, bounded history, or undo of individual operations.

PITFALLS

Recording the old value on every set instead of only the first. Forgetting to clear redo on a new batch.

HOW TO THINK ABOUT IT

Undo and redo are two stacks of actions. The question is what an action is: a batch is one action, so record per batch only what is needed to reverse it (the old value of each key touched, and the new one), not a whole copy of the document. A new edit after an undo kills the redo history.

Basis: LeetCode 1472 (Design Browser History) for the stacks idea, plus transactional batching.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
undo, redo = [], []
def commit_batch():
    undo.append(copy.deepcopy(props))
    redo.clear()
def do_undo():
    redo.append(copy.deepcopy(props))
    props.clear()
    props.update(undo.pop())
def do_redo():
    undo.append(copy.deepcopy(props))
    props.clear()
    props.update(redo.pop())
```

TIME	SPACE
O(size of document) per batch	**O(size * batches)**

WHY IT IS SLOW

A one-key change copies the whole document, O(size) per batch and O(size * batches) memory. Storing only the old and new value of each changed key makes each batch O(keys changed).

OPTIMAL SOLUTION · Store only the changed keys per batch

TIME	SPACE
O(keys changed in the batch) per operation	**O(total changed keys)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Limit the history to the last N batches.

Keep the undo stack in a deque with maxlen N so the oldest batch is dropped automatically. Redo is unaffected.

FOLLOW-UP 2 Nested batches.

Keep a stack of open batches and merge a child into its parent on endBatch (keep the parent's old value if the key was already touched). Only the outermost endBatch commits an undoable action.

FOLLOW-UP 3 Undo of individual setProperty calls, not whole batches.

Treat every setProperty outside a batch as a one-key batch (the code does), so undo granularity equals commit granularity.

FOLLOW-UP 4 Limit the history to the last N batches.

Cap the undo stack: when it grows beyond N drop the oldest batch (use a deque with maxlen, or pop(0) as here). The redo stack is unaffected. Memory is then O(N batches).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Batched Undo/Redo Layer
#
# You are implementing a simplified document layer for a
# design tool. The layer stores properties as a
# map<string, string>. Implement a class that supports the
# following operations: - beginBatch(): start a batch of
# edits. - setProperty(key, value): set or overwrite a
# property inside the current batch. - endBatch(): commit
# the batch as one undoable action. - undo(): revert the
# most recently committed batch. - redo(): reapply the
# most recently undone batch. Requirements: 1. A batch may
# update the same key multiple times. 2. undo() must
# restore the exact document state from before that batch.
# 3. redo() must restore the exact document state after
# that batch. 4. If a property did not exist before the
# batch, then undo() should remove that property. 5. If
# undo() has been called and then a new batch is
# committed, the redo history must be cleared. 6. Follow-
# up: make the history memory-efficient.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Limit the history to the last N batches.
# 2. Nested batches.
# 3. Undo of individual setProperty calls, not whole
# batches.
# 4. Limit the history to the last N batches.
#
# INTERVIEW NOTES
# Undo and redo are two stacks of actions. The question is
# what an action is: a batch is one action, so record per
# batch only what is needed to reverse it (the old value
# of each key touched, and the new one), not a whole copy
# of the document. A new edit after an undo kills the redo
# history.
#
# BRUTE FORCE (say it first): Snapshot the whole document
# after each batch
# Push a copy of the map on every commit; undo and redo
# move between snapshots. Obviously correct and a good
# oracle; it copies the whole document for a one-key
# change. Time O(size of document) per batch. Space O(size
# * batches).


from collections import deque
```

```


# HELPERS for the base question
class Change:
    """One key in one batch: value before and after."""

    def __init__(self, key, old, has_old, new):
        self.key, self.old, self.has_old, self.new = (
            key, old, has_old, new)


# BASE QUESTION (optimal): Store only the changed keys per
# batch
# Idea: While a batch is open keep {key: (old, new)} with
# the old value recorded only the first time a key is
# touched. endBatch pushes it on the undo stack and clears
# the redo stack.
# FOLLOW-UP 1: Limit the history to the last N batches.
# Answer: Keep the undo stack in a deque with maxlen N so
# the oldest batch is dropped automatically. Redo is
# unaffected.
# FOLLOW-UP 2: Nested batches.
# Answer: Keep a stack of open batches and merge a child
# into its parent on endBatch (keep the parent's old value
# if the key was already touched). Only the outermost
# endBatch commits an undoable action.
class Layer:
    def __init__(self, history=None):
        self.props = {}
        # FOLLOW-UP 1, 4: history=N keeps the last N
        # batches
        # committed batches; a deque with a maximum length
        # forgets the oldest ones when the history is
        # capped
        self.undo_stack = deque(maxlen=history)
        self.redo_stack = []
        # open batches (a stack, so batches can nest):
        # each is
        # a dict key -> Change, so a key appears only once
        self.open = []

    def begin_batch(self):
        self.open.append({})

    def set_property(self, key, value):
```

```
        if not self.open:
            # outside a batch: a batch of one change
            self.begin_batch()
            self.set_property(key, value)
            self.end_batch()
            return
        batch = self.open[-1]
        if key in batch:
            # touched again: keep the ORIGINAL old value
            # and
            # only update the final value
            batch[key].new = value
        else:
            batch[key] = Change(key, self.props.get(key),
                                key in self.props, value)
        self.props[key] = value

    def end_batch(self):
        batch = self.open.pop()
        if self.open:
            # FOLLOW-UP 2: nested batches fold into the
            # parent
            # a nested batch folds into its parent
            self.merge(self.open[-1], batch)
        elif batch:
            # only the outermost batch is one undoable
            # action;
            # a new action clears the redo history
            self.undo_stack.append(list(batch.values()))
            self.redo_stack.clear()

    @staticmethod
    def merge(parent, child):
        for key, change in child.items():
            if key in parent:
                parent[key].new = change.new
            else:
                parent[key] = change
```

```
    def undo(self):
        if not self.undo_stack:
            return False
        changes = self.undo_stack.pop()
        for c in changes:
            if c.has_old:
```

```
                self.props[c.key] = c.old
            else:
                # the key did not exist before: remove it
                self.props.pop(c.key, None)
        self.redo_stack.append(changes)
        return True

    def redo(self):
        if not self.redo_stack:
            return False
        changes = self.redo_stack.pop()
        for c in changes:
            self.props[c.key] = c.new
        self.undo_stack.append(changes)
        return True


if __name__ == "__main__":
    layer = Layer()
    layer.begin_batch()
    layer.set_property("color", "red")
    layer.set_property("color", "blue")
    layer.set_property("size", "4")
    layer.end_batch()
    assert layer.props == {"color": "blue", "size": "4"}
    # each key is stored once, whatever the number of
    # edits
    assert len(layer.undo_stack[0]) == 2
    layer.undo()
    assert layer.props == {}
    layer.redo()
    assert layer.props == {"color": "blue", "size": "4"}
    layer.begin_batch()
    layer.set_property("color", "green")
    layer.end_batch()
    layer.undo()
    # the old value comes back, not the first value of the
    # batch
    assert layer.props["color"] == "blue"
    # a new batch clears the redo history
    layer.set_property("x", "1")
    assert not layer.redo()
    # nested batches commit as one action
    layer.begin_batch()
    layer.begin_batch()
```

```
    layer.set_property("y", "2")
    layer.end_batch()
    layer.set_property("z", "3")
    layer.end_batch()
    layer.undo()
    assert "y" not in layer.props
    assert "z" not in layer.props
    # history limited to the last 2 batches
    small = Layer(history=2)
    for i in range(4):
        small.set_property("k", str(i))
    assert len(small.undo_stack) == 2
    small.undo()
    small.undo()
    assert not small.undo() and small.props["k"] == "1"
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[list[str]]) -> list[dict[str
, str]]:
SAMPLE INPUT
[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot
',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
```

Test cases

Input	Expected
`[('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'color': 'red'}, {}, {'color': 'red'}]`
`[('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'), ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]`	`[{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]`

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 2 of 16

Implement a Reusable Locker Allocation System

POSTED 2026-08-14 | 67 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 8.1 | 30 DAYS 0 | PRIORITY 167/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

LeetCode match: LC 1845 Seat Reservation Manager

Posted on PracHub: 2026-08-14 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

QUESTION

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.

Constraints

1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'],
    ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

APPROACH

Min-heap of released locker numbers plus a counter for the next never-used number; store pops the smallest, pickup pushes back. Track the last successful store separately. O(log n) per op.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what store and pickup return, what happens when a customer stores twice, when no locker is free, and whether numbers are reused smallest first.

BRUTE FORCE TO SAY OUT LOUD

Scan lockers 1..n for the first free one on every store: O(n) per operation.

OPTIMAL IDEA

Keep a min-heap of released locker numbers and a counter next_new for lockers never used. store: if the customer already has one return -1; take the heap minimum if the heap is not empty, otherwise next_new if it is at most n, else return -1; remember customer to locker in a dict. pickup: free the customer's locker by pushing it on the heap. O(log n) per operation, and memory depends on used lockers, not n.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

Pickup then store reuses the smallest freed number. Double store by one customer. Store when full. Pickup with no locker.

LIKELY FOLLOW-UPS

n up to a billion (still fine because of the counter), thread safety with a lock, or lockers with sizes.

PITFALLS

Initializing the heap with all n numbers. Not removing the customer record on pickup.

HOW TO THINK ABOUT IT

'Smallest free number' with numbers coming back means a min-heap of returned numbers. Numbers that were never handed out do not need to be stored: keep a counter for the next unused number and use the heap first. That keeps memory proportional to the number of lockers in use, not to n.

Basis: Written; the smallest-available-number pattern (LeetCode 1845 Seat Reservation Manager, 2336 Smallest Number in Infinite Set).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
owner = {}  # customer -> locker
def store(c):
    if c in owner:
        return -1
    taken = set(owner.values())
    for locker in range(1, n + 1):
        if locker not in taken:
            owner[c] = locker
            return locker
    return -1
def pickup(c):
    owner.pop(c, None)
```

TIME	SPACE
O(n) per operation	O(n)

WHY IT IS SLOW

Each store scans up to n lockers, O(n). A min-heap of freed numbers plus a counter for the next unused one makes each operation O(log n).

OPTIMAL SOLUTION · Min-heap of released numbers plus a next-unused counter

TIME	SPACE
O(log n) per operation	O(lockers released and not reused)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 There are 10^9 lockers.

The counter and the heap already make the structure independent of n: nothing is allocated up front, and memory depends only on how many lockers have been released.
Cost: O(log R) per operation and O(R) memory, where R is the number of lockers released and not yet reused; independent of n.

FOLLOW-UP 2 Many threads call store and pickup.

Protect the heap, the counter and the customer map with one lock so a locker number is never handed out twice (or fine-grained: a lock around the (heap, counter) pair).

FOLLOW-UP 3 Lockers of different sizes.

Keep one heap and counter per size class, and let a customer choose the smallest class that fits. The structure per class is the same.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement a Reusable Locker Allocation
# System
#
# ## Problem A locker bank has lockers numbered from 1
# through n. Process operations in order. store(customer)
# assigns the smallest available locker and returns its
# number, or -1 if that customer already has a locker or
# none is available. pickup(customer) releases and returns
# that customer's locker, or -1 if no locker is assigned.
# last() returns the locker number from the most recent
# successful store, or -1 if there has not been one.
# Return one result per operation. ### Constraints &
# Assumptions - 1 <= n <= 200,000. - There are at most
# 200,000 operations. - Customer IDs are non-empty
# strings. - A released locker becomes immediately
# available for reuse. ### Clarifications - Failed store
# and pickup operations do not change last(). - The
# allocation rule is always the smallest currently free
# locker number. - A customer can store again after a
# successful pickup.
#
# FOLLOW-UPS (answered below, in this order):
# 1. There are 10^9 lockers.
# 2. Many threads call store and pickup.
# 3. Lockers of different sizes.
#
# INTERVIEW NOTES
# 'Smallest free number' with numbers coming back means a
# min-heap of returned numbers. Numbers that were never
# handed out do not need to be stored: keep a counter for
# the next unused number and use the heap first. That
# keeps memory proportional to the number of lockers in
# use, not to n.
#
# BRUTE FORCE (say it first): Scan the lockers for the
# first free one
# Keep an array of owners and scan from 1 for the first
# empty slot on store. Obviously correct, and each store
# can touch every locker. Time O(n) per operation. Space
```

```
# O(n).


import heapq
```

```
# BRUTE FORCE (say it first): Scan the lockers for the
# first free one | time O(n) per operation
class LockersBrute:
    def __init__(self, n):
        self.owner, self.of = [None] * (n + 1), {}

    def store(self, customer):
        if customer in self.of:
            return -1
        for i in range(1, len(self.owner)):
            if self.owner[i] is None:
                self.owner[i], self.of[customer] = (
                    customer,
                    i,
                )
                return i
        return -1

    def pickup(self, customer):
        i = self.of.pop(customer, None)
        if i is None:
            return -1
        self.owner[i] = None
        return i


# BASE QUESTION (optimal): Min-heap of released numbers
# plus a next-unused counter
# Idea: store: if the customer already has a locker return
# -1; take the smallest released number from the heap if
# any, else the next never-used number if it is at most n,
# else -1.
class Lockers:
    def __init__(self, n):
        self.n = n
        # the next never-used locker number
        self.next_new = 1
        # released lockers, smallest on top
        self.free = []
        # customer -> locker number
        self.of = {}
        # locker of the latest successful store
        self.last_store = -1

    def store(self, customer):
```

```
        if customer in self.of:
            return -1
        if self.free:
            # a released locker is always smaller than any
            # new one
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        locker = self.of.pop(customer, None)
        if locker is None:
            return -1
        # it becomes available again at once
        heapq.heappush(self.free, locker)
```

```
        return locker

    def last(self):
        # failed operations never change this
        return self.last_store


# FOLLOW-UP 2: Many threads call store and pickup.
# Answer: Protect the heap, the counter and the customer
# map with one lock so a locker number is never handed out
# twice (or fine-grained: a lock around the (heap,
# counter) pair).
class SafeLockers(Lockers):
    """Many threads: one lock around every operation."""

    def __init__(self, n):
        import threading
        super().__init__(n)
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return super().store(customer)
```

```
    def pickup(self, customer):
        with self.lock:
            return super().pickup(customer)


# HELPERS for the checks below
def process(n, operations):
    box, out = Lockers(n), []
    for op in operations:
        if op[0] == "store":
            out.append(box.store(op[1]))
        elif op[0] == "pickup":
            out.append(box.pickup(op[1]))
        else:
            out.append(box.last())
    return out


if __name__ == "__main__":
    ops = [["store", "a"], ["store", "b"],
           ["pickup", "a"], ["store", "c"], ["last"]]
    assert process(3, ops) == [1, 2, 1, 1, 1]
    # nothing stored yet, and a missing pickup
    empty = [["last"], ["pickup", "x"]]
    assert process(2, empty) == [-1, -1]
    # full, and a customer who already has a locker
    full = [["store", "a"], ["store", "a"],
            ["store", "b"], ["store", "c"]]
    assert process(2, full) == [1, -1, 2, -1]
    # failed operations leave last() alone
    more = full + [["last"]]
    assert process(2, more)[-1] == 2
    # a huge locker bank costs nothing up front
    one = [["store", "a"], ["last"]]
    assert process(10 ** 9, one) == [1, 1]
    s = SafeLockers(2)
    assert s.store("a") == 1 and s.pickup("a") == 1
```

FUNCTION AND TEST CASES

Signature

```
def process_locker_operations(n: int, operations: list[lis
t[str]]) -> list[int]:
```

Sample call

```
process_locker_operations(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
```

Test cases

Input	Expected
`(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])`	`[1, 2, 1, 1, 1]`
`(5, [])`	`[]`

ALSO REPORTED · 1 more

Implement a Reusable Locker Allocation System | PracHub | Technical Screen

https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system-2

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 3 of 16

Build Prefix Lookup with a Trie

POSTED 2026-07-03 | 109 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 36 | 30 DAYS 0 | PRIORITY 105/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree) (inferred from overview)

Posted on PracHub: 2026-07-03 (updated 2026-07-04)

Original: https://prachub.com/coding-questions/build-prefix-lookup-with-a-trie

QUESTION

Overview: Practice a Google coding interview problem focused on build prefix lookup with a trie. The prompt emphasizes edge cases, clean implementation, and verifiable test behavior without revealing the solution.

APPROACH

Trie of dict words; walk the prefix, then DFS the subtree to collect completions (optionally store counts at nodes).

HOW TO THINK ABOUT IT

Prefix questions repeat work when each query scans the whole word list. A trie stores each prefix once: walk down by the query, then everything below that node is a completion. Keep counters or a word flag in the nodes to answer follow-ups such as 'how many words start with this' or 'top K completions' without another walk.

Basis: LeetCode 208 (Implement Trie) and 1268 (Search Suggestions System).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def lookup(words, prefix):
    found = []
    for word in words:
        if word.startswith(prefix):
            found.append(word)
    return found
```

TIME	SPACE
O(W * L) per query	O(1) extra

BRUTE FORCE

WHY IT IS SLOW

Every query touches every word, O(W * L). A trie walks only the prefix and then the subtree below it.

OPTIMAL SOLUTION · Trie with a DFS below the prefix node

TIME	SPACE
O(L) to walk the prefix + O(size of the answer)	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many words start with this prefix?

Store a counter in each node that is incremented when a word passes through it; the answer is the counter at the end of the prefix walk, O(L) with no DFS.

FOLLOW-UP 2 Return only the top 3 completions (by frequency or alphabetically).

Keep at each node a short list of its best K words (updated on insert), or bound the DFS: visit children in alphabetical order and stop after K words. That is the LeetCode 1268 style answer.

FOLLOW-UP 3 Memory is large (many words with long common prefixes).

A compressed (radix) trie stores whole edge labels instead of single characters; a sorted array with binary search over prefixes uses the least memory for a static dictionary. **Cost:** a radix trie has O(number of words) nodes instead of O(total characters); a sorted array uses O(total characters) with O(L log W) lookups.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Build Prefix Lookup with a Trie
#
# Practice a Google coding interview problem focused on
# build prefix lookup with a trie. The prompt emphasizes
# edge cases, clean implementation, and verifiable test
# behavior without revealing the solution.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many words start with this prefix?
# 2. Return only the top 3 completions (by frequency or
# alphabetically).
# 3. Memory is large (many words with long common
# prefixes).
#
# INTERVIEW NOTES
```

```
# Prefix questions repeat work when each query scans the
# whole word list. A trie stores each prefix once: walk
# down by the query, then everything below that node is a
# completion. Keep counters or a word flag in the nodes to
# answer follow-ups such as 'how many words start with
# this' or 'top K completions' without another walk.
#
# BRUTE FORCE (say it first): Scan every word for the
# prefix
# For each query test word.startswith(prefix) on every
# dictionary word. Correct, but each query pays for the
# whole dictionary. Time O(W * L) per query. Space O(1)
# extra.


# BRUTE FORCE (say it first): Scan every word for the
# prefix | time O(W * L) per query
def completions_brute(words, prefix):
    return sorted(
        w for w in set(words) if w.startswith(prefix)
    )


# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        # does a word end here?
        self.is_word = False
        # how many words pass through this node
```

```
        self.count = 0


# BASE QUESTION (optimal): Trie with a DFS below the
# prefix node
# Idea: Insert every word into nested dicts, marking word
# ends. To answer a prefix, walk down one character at a
# time (no match means no completions), then DFS the
# subtree collecting words.
# FOLLOW-UP 1: How many words start with this prefix?
# Answer: Store a counter in each node that is incremented
# when a word passes through it; the answer is the counter
# at the end of the prefix walk, O(L) with no DFS.
class PrefixTrie:
    def __init__(self, words=()):
        self.root = Node()
        for w in words:
            self.insert(w)

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
            node.count += 1
        # a repeated word is only counted as one word
        if not node.is_word:
            node.is_word = True
        else:
            self.undo_count(word)

    def undo_count(self, word):
        node = self.root
        for ch in word:
            node = node.children[ch]
            node.count -= 1

    def walk(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
```

```
        return node

    # FOLLOW-UP 1: how many words start with the prefix
```

```
    def count_prefix(self, prefix):
        # how many words start with the prefix: O(len)
        node = self.walk(prefix)
        if prefix == "":
            return self.root_count()
        return node.count if node else 0

    def root_count(self):
        kids = self.root.children.values()
        return sum(c.count for c in kids)

    # FOLLOW-UP 2: only the top k completions
    def suggest(self, prefix, k=3):
        # first k completions in alphabetical order; the
        # DFS
        # visits children in order and stops after k words
        node = self.walk(prefix)
        out = []
        if node is None:
            return out
        stack = [(node, prefix)]
        while stack and len(out) < k:
            n, text = stack.pop()
            if n.is_word:
                out.append(text)
            # reversed so the smallest letter is popped
            # first
            for ch in sorted(n.children, reverse=True):
                stack.append((n.children[ch], text + ch))
        return out

    def completions(self, prefix):
        # every word with the prefix
        return self.suggest(prefix, k=10 ** 9)


if __name__ == "__main__":
    t = PrefixTrie(["apple", "app", "apricot", "banana",
                    "apple"])
    ap = t.completions("ap")
    assert ap == ["app", "apple", "apricot"]
    everything = t.completions("")
    assert everything[-1] == "banana"
    assert t.completions("x") == []
    assert t.count_prefix("ap") == 3
```

```
    assert t.count_prefix("b") == 1
    assert t.count_prefix("zz") == 0
    # the top 2 in alphabetical order
    assert t.suggest("ap", 2) == ["app", "apple"]
    assert t.count_prefix("") == 4
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], queries: list[str]) -> list
[list[str]]:
SAMPLE INPUT
{"words":["car","cat","dog"],"queries":["ca","d"]}
```

Test cases

Input	Expected
{"words":["car","cat","dog"],"queries":["ca","d"]}	[["car","cat"],["dog"]]

Input	Expected
`{"words":[],"queries":["a"]}`	`[[]]`

ALSO REPORTED · 1 more

Trie Prefix Search / Autocomplete | 1Point3Acres | tech-screen, onsite-coding | asked 2026-07-02 | Medium

https://www.1point3acres.com/interview/problems/company/google/trie-prefix-search-autocomplete

Standard implementation: each trie node has a children map and an is_end flag.
Two collection patterns at the prefix: (a) DFS from the prefix node and collect all word-end nodes, (b) store the top-K candidates at every node during insert (Google-preferred for autocomplete).
The standard sorted-suggestions follow-up wants 3 suggestions per prefix-length — handle with sorted children or per-node sorted lists.

Preparation: Write the trie class from memory in under 12 min. Drill the DFS-collect and the sorted-per-node variants; know which is faster in which scenario.
For frequency-ranking, practice the design-autocomplete variant (per-node top-K updated on insert) as a stretch goal.

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 4 of 16

★ TOP 75: rank 73 in the Medium and Hard priority list

★ Design a restaurant waitlist system

POSTED 2026-03-01 | 233 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.3 | 30 DAYS 0 | PRIORITY 49/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #146 LRU Cache

LeetCode match: No direct match; related LC 146 LRU Cache (ordered map)

Posted on PracHub: 2026-03-01 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/design-a-restaurant-waitlist-system

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **c** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**
   - Adds a new party to the end of the waitlist and returns its id.
2. **`cancelParty(partyId) -> bool`**
   - Removes the party from the waitlist if present.
   - Returns **`true`** if removed, otherwise **`false`**.
3. **`seatTable(capacity) -> partyId or null`**
   - Finds and removes the earliest party with **`size <= capacity`**.
   - Returns that party's id, or **`null`** if nobody fits.
4. **`getPosition(partyId) -> int`**
   - Returns how many parties are ahead of this party in the current waitlist order (0-based).
   - If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.
- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

APPROACH

Hash map partyId -> party plus an ordered queue (linked list/OrderedDict) for arrival order; on a free table of size s, seat the earliest waiting party that fits; cancel is O(1) with lazy deletion.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact API (add party, cancel, seat when a table of capacity c frees up), what happens when no party fits, and whether a larger table may seat a smaller party (yes, if size <= c).

BRUTE FORCE TO SAY OUT LOUD

Keep a list in arrival order; on a table of capacity c scan from the front for the first party that fits. O(n) per seating, O(n) per cancel.

OPTIMAL IDEA

Keep id to party in a dict. For efficiency use a segment tree over arrival order storing the minimum party size in each range; the earliest party that fits is the leftmost position with size <= c, found by descending the tree in O(log n). Cancel and seat set that position's size to infinity. The simpler version uses an ordered dict plus lazy deletion and scans, O(n) in the worst case, which is fine to start with and to improve.

DRY-RUN THESE CASES

A table that fits nobody, a party that cancels before being seated, ties on arrival, a table that fits the second party but not the first, duplicate party id.

LIKELY FOLLOW-UPS

Large parties waiting too long (starvation), time-based expiry, or estimating wait time.

PITFALLS

Skipping a party that cancelled but is still in the queue. Choosing the best fit instead of the earliest fit.

HOW TO THINK ABOUT IT

'Earliest arrival whose size fits' is a query on an ordered sequence: find the leftmost position whose value is at most c. A scan does it in O(n); a segment tree that stores the minimum size over each range answers it in O(log n) by walking down to the leftmost child whose range minimum fits. Cancelling or seating simply sets that position to infinity.

HOW TO THINK ABOUT IT

Basis: Written design (compare LeetCode 146 for the ordered-map idea). API assumed (confirm it): add(id, name, size), cancel(id), seat(capacity) seats and returns the earliest-arrived waiting party whose size is at most the capacity, or None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
parties = []  # in arrival order
def join(party):
    parties.append(party)
def seat(capacity):
    for party in parties:
        if party.size <= capacity:
            parties.remove(party)
            return party
    return None
def cancel(party_id):
    for party in parties:
        if party.id == party_id:
            parties.remove(party)
            return
```

TIME	SPACE
O(n) per seat and per cancel	O(n)

WHY IT IS SLOW

Both operations walk the list, O(n). A segment tree of the smallest size per range finds the leftmost party that fits in O(log n).

OPTIMAL SOLUTION · Segment tree of minimum size over arrival order

TIME	SPACE
O(log n) per operation	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Parties that wait too long should be seated first (starvation).

Add an aging rule: if the earliest party has waited longer than a threshold and a table of at least its size appears, seat it before anyone else. The segment tree can query 'leftmost size <= c' with c = the table, so order is preserved.

FOLLOW-UP 2 Seat the best-fitting party, not the earliest.

That is a different policy: keep sizes in a sorted structure of (size, arrival) and take the largest size at most the capacity (ties by arrival). A sorted container with bisect gives O(log n) lookups.

FOLLOW-UP 3 Estimate the waiting time of a party.

Track the arrival rate and typical table turnover per capacity; the estimate is the expected time until enough tables of at least that size free up, ahead of the party in the queue.

FOLLOW-UP 4 Seat the best-fitting party (the largest that fits), not the earliest.

Group waiting parties by size in queues (arrival order inside a size) and keep the sizes in a sorted list: the best fit for a table of capacity c is the largest size at most c, found with bisect, then the oldest party of that size. O(log distinct sizes) per seating. Verified against a linear scan.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design a restaurant waitlist system
#
# You are implementing the waitlist system for a
# restaurant. Parties arrive over time, can cancel, and
# get seated when a table becomes available. ## Rules -
# Each party has a unique partyId, a name, and a size
# (number of people). - Parties are seated in **arrival
# order**, but only if they **fit** the available table. -
# When a table of capacity c becomes available, you must
# seat the **earliest-arrived** party whose size <= c. -
# If no party fits, the table remains unused for that
# event. ## Operations to support Design a data structure
# / class that supports the following operations
# efficiently: 1. addParty(name, size) -> partyId - Adds a
# new party to the end of the waitlist and returns its id.
# 2. cancelParty(partyId) -> bool - Removes the party from
# the waitlist if present. - Returns true if removed,
# otherwise false. 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Parties that wait too long should be seated first
# (starvation).
# 2. Seat the best-fitting party, not the earliest.
# 3. Estimate the waiting time of a party.
# 4. Seat the best-fitting party (the largest that fits),
# not the earliest.
#
# INTERVIEW NOTES
# 'Earliest arrival whose size fits' is a query on an
# ordered sequence: find the leftmost position whose value
# is at most c. A scan does it in O(n); a segment tree
# that stores the minimum size over each range answers it
# in O(log n) by walking down to the leftmost child whose
# range minimum fits. Cancelling or seating simply sets
# that position to infinity.
#
# BRUTE FORCE (say it first): Scan the waiting list in
# arrival order
# Keep parties in arrival order in a list. seat scans from
# the front for the first party whose size fits; cancel
# removes by id. Correct and the oracle; every operation
# walks the list. Time O(n) per seat and per cancel. Space
# O(n).
```

```python
from bisect import bisect_right, insort
from collections import defaultdict, deque


# BRUTE FORCE (say it first): Scan the waiting list in
# arrival order | time O(n) per seat and per cancel
class WaitlistBrute:
    def __init__(self):
        self.parties = []

    def add(self, pid, name, size):
        self.parties.append((pid, name, size))

    def cancel(self, pid):
        before = len(self.parties)
        self.parties = [
            p for p in self.parties if p[0] != pid
        ]
        return len(self.parties) < before

    def seat(self, capacity):
        for i, p in enumerate(self.parties):
            if p[2] <= capacity:
                del self.parties[i]
                return p[0]
        return None


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Segment tree of minimum size
# over arrival order
# Idea: Each arrival gets the next position. The tree
# stores, for each range, the smallest waiting party size
# (infinity for empty or seated slots).
class Waitlist:
    """Earliest party that fits: a segment tree of the
    smallest size over arrival slots."""

    def __init__(self, capacity=1024):
        self.cap = capacity
        self.tree = [INF] * (2 * capacity)
        self.count = 0
```

```python
        self.slot, self.who = {}, {}

    def update(self, i, value):
        i += self.cap
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = min(self.tree[2 * i],
                               self.tree[2 * i + 1])

    def add(self, party_id, name, size):
        self.slot[party_id] = self.count
        self.who[self.count] = party_id
        self.update(self.count, size)
        self.count += 1

    def cancel(self, party_id):
        i = self.slot.get(party_id)
        if i is None or self.tree[i + self.cap] == INF:
            return False
        self.update(i, INF)
        return True

    def seat(self, capacity):
        # no party small enough: nobody is seated
        if self.tree[1] > capacity:
            return None
        i = 1
```

```python
        # go left whenever the left half holds a party
        # that fits
        while i < self.cap:
            left = self.tree[2 * i] <= capacity
            i = 2 * i if left else 2 * i + 1
        self.update(i - self.cap, INF)
        return self.who[i - self.cap]


# FOLLOW-UP 2: Seat the best-fitting party, not the
# earliest.
# Answer: That is a different policy: keep sizes in a
# sorted structure of (size, arrival) and take the largest
# size at most the capacity (ties by arrival). A sorted
# container with bisect gives O(log n) lookups.
# FOLLOW-UP 4: Seat the best-fitting party (the largest
# that fits), not the earliest.
```

```python
# Answer: Group waiting parties by size in queues (arrival
# order inside a size) and keep the sizes in a sorted
# list: the best fit for a table of capacity c is the
# largest size at most c, found with bisect, then the
# oldest party of that size.
class BestFit:
    """Policy B: the largest party that fits the table."""

    def __init__(self):
        # size -> parties of that size, oldest first
        self.queues = defaultdict(deque)
        self.sizes = []

    def add(self, party_id, size):
        if not self.queues[size]:
            insort(self.sizes, size)
        self.queues[size].append(party_id)

    def seat(self, capacity):
        k = bisect_right(self.sizes, capacity)
        if k == 0:
            return None
        size = self.sizes[k - 1]
        party = self.queues[size].popleft()
        if not self.queues[size]:
            self.sizes.pop(k - 1)
        return party


if __name__ == "__main__":
    w = Waitlist(8)
    w.add("p1", "Ann", 4)
    w.add("p2", "Bob", 2)
    w.add("p3", "Cy", 6)
    # a table for 3: the earliest party that fits is Bob
    assert w.seat(3) == "p2"
    # cancel Ann; a table for 6 now seats Cy
    assert w.cancel("p1") and not w.cancel("p1")
    assert w.seat(6) == "p3"
    assert w.seat(10) is None
    b = BestFit()
    parties = (("a", 2), ("b", 4), ("c", 4), ("d", 6))
    for pid, size in parties:
        b.add(pid, size)
    # a table for 5: the biggest fit is size 4, oldest
```

```python
    # first
    assert b.seat(5) == "b"
    assert b.seat(5) == "c"
    assert b.seat(5) == "a"
    assert b.seat(1) is None
```

FUNCTION AND TEST CASES

Test cases

Input	Expected
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('seatTable', 2), ('seatTable', 4)]`	`[1, 2, 2, 1]`
`[('addParty', 'A', 4), ('addParty', 'B', 2), ('addParty', 'C', 3), ('getPosition', 3), ('seatTable', 2), ('getPosition', 3), ('cancelParty', 1), ('getPosition', 3), ('cancelParty', 1), ('seatTable', 2), ('seatTable', 3), ('getPosition', 3)]`	`[1, 2, 3, 2, 2, 1, True, 0, False, None, 3, -1]`

ALSO REPORTED · 1 more

Restaurant Waitlist: join / delete / find_first_match | 1Point3Acres | onsite-coding | asked 2026-02-01 | Single report

https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

Preparation: Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 5 of 16

Determine Reporting Relationships

POSTED 2026-02-22 | 240 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.4 | 30 DAYS 33 | PRIORITY 63/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #721 Accounts Merge

LeetCode match: No direct match; related LC 721 Accounts Merge (union-find)

Posted on PracHub: 2026-02-22 (updated 2026-04-21)

Original: https://prachub.com/coding-questions/determine-reporting-relationships

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.
- Repeating the same valid relationship is allowed.

QUESTION

- An employee is not considered their own manager.

Constraints

0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles

EXAMPLES

```
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3),
    ("is_manager", 1, 3)])
Expected Output: [True]
```

Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.

```
Input: ([("add_manager", 2, 4), ("is_manager", 1, 4),
    ("add_manager", 1, 2), ("is_manager", 1, 4),
    ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
```

Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4. Hints**`add_peer(a, b)`** means **a** and **b** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **b** through parent pointers.

APPROACH

Manager edges as a directed graph/parent map; peers via union-find or shared-manager sets; queries answer reachability up the manager chain. Keep both structures in sync.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether is_manager means direct manager or any manager up the chain (the example implies the chain), whether peers can be added before their manager is known, and whether input is always consistent (the statement says yes).

BRUTE FORCE TO SAY OUT LOUD

Store all relations and recompute reachability with a graph search on each query. O(V + E) per query.

OPTIMAL IDEA

Keep a Union-Find over employees for peer groups and attach one manager to each group. add_manager(a, b) sets the manager of b's group to a. add_peer(a, b) unions the two groups and, if both had managers, keeps the (necessarily identical) manager. is_manager(a, b): look up the manager of b's group, return true if it is a, otherwise continue from that manager's group up the chain. Each query walks at most the depth of the hierarchy.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

add_manager(m, x) then add_peer(x, y) gives is_manager(m, y) true; add_peer before add_manager; a manager's manager (indirect); an employee with no manager; a query on unknown ids.

LIKELY FOLLOW-UPS

Direct manager only, the closest common manager, or removing relationships.

PITFALLS

Forgetting to merge managers when two groups unite. Treating peers as managers of each other.

HOW TO THINK ABOUT IT

Peers form groups (a peer relation is transitive: same manager as a person who has the same manager as another). A group has at most one direct manager, so store the manager on the group. To answer is_manager(a, b) go from b's group to its manager, then that manager's group to its manager, and so on, looking for a. Union-find makes the groups cheap to merge.

Basis: Written (compare LeetCode 721, Accounts Merge, for the union-find idea). Operations: add_manager(a, b) (a is b's direct manager), add_peer(a, b) (same direct manager), is_manager(a, b) (a is anywhere above b). Inferences must follow from earlier calls.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
managers, peers = [], []  # the stored facts
def is_manager(a, b):
    groups = merge_peers(peers)
    manager_of = {}
    for x, y in managers:  # x manages y
        boss = groups.find(x)
        manager_of[groups.find(y)] = boss
    node = groups.find(b)
    while node in manager_of:
        node = manager_of[node]
        if node == groups.find(a):
            return True
    return False
```

TIME	SPACE
O(F * employees)	O(F)
per query	

WHY IT IS SLOW

Each query redoes the whole inference over all facts. Union-Find for peer groups with a manager stored on each group keeps every update near-constant and a query O(depth).

OPTIMAL SOLUTION · Union-Find for peer groups with a manager stored on each group

TIME	SPACE
O(alpha)	O(employees)
per update, O(depth) per query	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the closest common manager of two employees.

Collect the chain of managers above one employee into a set, then climb from the other until you meet it. O(depth).

FOLLOW-UP 2 Facts can be wrong or contradictory.

When uniting groups that already have different managers, the input is inconsistent: raise, or record a conflict instead of silently overwriting. Also guard the upward walk with a visited set so a cycle cannot loop forever.

FOLLOW-UP 3 Remove a relationship.

Union-Find cannot split groups; keep the facts and rebuild lazily, or store the peers as an explicit graph and recompute connected components on demand.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine Reporting Relationships
#
# Build an in-memory organization model that supports
# three operations on employee IDs: 1. add_manager(a, b):
# employee a is the direct manager of employee b. 2.
# add_peer(a, b): employees a and b share the same direct
# manager. 3. is_manager(a, b): return whether employee a
# is a manager of employee b. The answer should account
# for relationships that can be inferred from prior
# operations. For example, if add_manager(m, x) and
# add_peer(x, y) were previously recorded, then
# is_manager(m, y) should return true. You may assume the
# input is logically consistent and does not create
# invalid cycles in the org chart. Explain how you would
# design the data structures and implement these
# operations efficiently.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the closest common manager of two employees.
# 2. Facts can be wrong or contradictory.
# 3. Remove a relationship.
#
# INTERVIEW NOTES
# Peers form groups (a peer relation is transitive: same
# manager as a person who has the same manager as
# another). A group has at most one direct manager, so
# store the manager on the group. To answer is_manager(a,
# b) go from b's group to its manager, then that manager's
# group to its manager, and so on, looking for a. Union-
```

```
# find makes the groups cheap to merge.
#
# BRUTE FORCE (say it first): Recompute all inferences
# from the stored facts on every query
# Keep the list of facts. For a query, repeatedly merge
# peers into groups and attach managers until nothing
# changes, then walk up from b. Correct and the oracle;
# every query redoes the whole inference. Time O(F *
# employees) per query. Space O(F).


# BRUTE FORCE (say it first): Recompute all inferences
# from the stored facts on every query | time O(F *
# employees) per query
class OrgBrute:
    def __init__(self):
```

```
        self.mgr, self.peers = [], []

    def add_manager(self, a, b):
        self.mgr.append((a, b))

    def add_peer(self, a, b):
        self.peers.append((a, b))

    def is_manager(self, a, b):
        group = {}
        people = {
            x for p in self.mgr + self.peers for x in p
        }
        for x in people:
            group[x] = {x}
        for x, y in self.peers:
            if group[x] is not group[y]:
                merged = group[x] | group[y]
                for z in merged:
                    group[z] = merged
        boss = {}
        for m, x in self.mgr:
            for z in group[x]:
                boss[id(group[x])] = m
        cur, steps = b, 0
        while (
            cur in group
            and id(group[cur]) in boss
            and steps <= len(people)
        ):
            cur = boss[id(group[cur])]
            if cur == a:
                return True
            steps += 1
        return False


# BASE QUESTION (optimal): Union-Find for peer groups with
# a manager stored on each group
# Idea: Each employee starts in its own group. add_peer
# unions the two groups (and keeps the manager, if either
# has one). add_manager sets the manager of b's group to
# a.
# FOLLOW-UP 1: Return the closest common manager of two
# employees.
```

```
# Answer: Collect the chain of managers above one employee
# into a set, then climb from the other until you meet it.
# O(depth).
class Org:
    def __init__(self):
        # union-find over peers (people with the same
        # manager)
        self.parent = {}
        # peer-group root -> the manager of that group
        self.boss = {}
```

```
    def find(self, x):
        self.parent.setdefault(x, x)
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        # two groups with different managers contradict
        if ra in self.boss and rb in self.boss \
                and self.boss[ra] != self.boss[rb]:
            # FOLLOW-UP 2: contradictory facts are
            # rejected
            raise ValueError("conflicting managers")
        self.parent[rb] = ra
        # the merged group keeps whichever manager is
        # known
        if rb in self.boss and ra not in self.boss:
            self.boss[ra] = self.boss[rb]

    def add_manager(self, a, b):
        # a is the direct manager of b (and b's peers)
        self.find(a)
        root = self.find(b)
        if root in self.boss and self.boss[root] != a:
            raise ValueError("conflicting managers")
        self.boss[root] = a

    def manager_of(self, x):
        return self.boss.get(self.find(x))
```

```
    def is_manager(self, a, b):
        # climb b's chain of managers; a visited set stops
        # cycles
        seen, cur = set(), b
        while cur is not None and cur not in seen:
            seen.add(cur)
            cur = self.manager_of(cur)
            if cur == a:
                return True
        return False

    # FOLLOW-UP 1: the closest common manager
    def common_manager(self, a, b):
        # the closest manager above both people
        chain, cur = set(), self.manager_of(a)
        while cur is not None and cur not in chain:
            chain.add(cur)
            cur = self.manager_of(cur)
        cur, seen = self.manager_of(b), set()
        while cur is not None and cur not in seen:
            if cur in chain:
                return cur
            seen.add(cur)
            cur = self.manager_of(cur)
        return None


if __name__ == "__main__":
    o = Org()
    o.add_manager("ceo", "vp")
    o.add_manager("vp", "dev")
    o.add_peer("dev", "qa")
    # qa shares dev's manager, so vp manages qa too
    assert o.is_manager("vp", "qa")
    assert o.is_manager("ceo", "qa")
    assert not o.is_manager("qa", "vp")
    assert not o.is_manager("dev", "qa")
    assert o.common_manager("dev", "qa") == "vp"
```

```
    assert o.common_manager("vp", "qa") == "ceo"
    # peers added before the manager is known
    p = Org()
    p.add_peer("a", "b")
    p.add_manager("boss", "a")
    assert p.is_manager("boss", "b")
    # contradictory facts are reported
```

```
    try:
        o.add_manager("ceo", "dev")
        raise AssertionError("should conflict")
    except ValueError:
        pass
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[tuple[str, int, int]]) -> li
st[bool]:
SAMPLE INPUT
([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager"
, 1, 3)])
```

Test cases

Input	Expected
`([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])`	`[True]`
`([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4), ("is_manager", 2, 4), ("is_manager", 4, 2)])`	`[False, True, True, False]`

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | IN APP LC 208 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 6 of 16

Implement Trie Insert and Exact-Word Search

POSTED 2026-02-20 | 242 DAYS AGO | OLDER

STUDY: TALK ALL TIME 36 30 DAYS 0 PRIORITY 107/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #208 Implement Trie (Prefix Tree)

LeetCode match: LC 208 Implement Trie (Prefix Tree)

Posted on PracHub: 2026-02-20 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-trie-insert-and-exact-word-search

QUESTION

Problem

Implement a trie that supports **`insert(word)`** and **`search(word)`**. Insert adds a lowercase word. Search returns true only when the complete word was previously inserted; a stored word merely sharing the requested prefix is not a match. Process a sequence of operations and return the results of all searches.

Constraints & Assumptions

- There are at most 200,000 operations.
- Every word has length between 1 and 1,000.
- The total number of characters across all operations is at most 1,000,000.
- Words contain lowercase English letters.

Clarifications

- Inserting the same word repeatedly is allowed and has no additional effect.
- A node must distinguish an inserted word ending there from a prefix only.
- Return booleans in the same order as search operations appear.

Examples

```
operations = [insert("apple"), search("apple"), search("app"), insert("app"), search("app")] output = [true, false, true]
```

Hints

```
Represent prefixes Each edge consumes one character and each node represents the prefix along its path.
```

QUESTION

```
Mark complete words Reaching the final character is not enough unless the terminal node records an insertion.
```

APPROACH

Trie nodes with children and an end-of-word flag; search returns true only if the final node is marked as a word.

HOW TO THINK ABOUT IT

A trie needs a flag that marks 'a word ends here'; without it search('app') would be true whenever 'apple' was inserted. Walking a prefix and checking the flag are the two operations: search checks the flag, startsWith does not.

Basis: LeetCode 208, Implement Trie (Prefix Tree).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
words = []
def insert(w):
    words.append(w)
def search(w):
    return w in words
def starts_with(p):
    for word in words:
        if word.startswith(p):
            return True
    return False
```

TIME	SPACE
O(W * L) per operation	O(total characters)

WHY IT IS SLOW

Each call scans every stored word, O(W * L). A trie with an end-of-word flag makes every call O(L).

OPTIMAL SOLUTION · Trie of nodes with an end-of-word flag

TIME	SPACE
O(L) per operation	O(total characters)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support '.' as a wildcard in search (LeetCode 211).

On '.', try every child at that level and recurse; other characters follow the single matching child. Worst case O(26^L) per query on adversarial patterns, so mention the bound.

FOLLOW-UP 2 Delete a word.

Unmark the end flag and, walking back up, remove nodes that have no children and are not word ends. Keep a path stack while walking down so you can prune on the way back. **Cost:** O(L) time and O(L) space for the path stack.

FOLLOW-UP 3 Unicode or large alphabets.

Store children in a dict keyed by character instead of a 26-slot array, as the code does; it handles any alphabet and wastes no space on missing letters. **Cost:** O(L) per operation with dictionary children; memory O(total characters).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Trie Insert and Exact-Word
# Search
#
# ## Problem Implement a trie that supports insert(word)
# and search(word). Insert adds a lowercase word. Search
# returns true only when the complete word was previously
# inserted; a stored word merely sharing the requested
# prefix is not a match. Process a sequence of operations
# and return the results of all searches. ### Constraints
# & Assumptions - There are at most 200,000 operations. -
# Every word has length between 1 and 1,000. - The total
# number of characters across all operations is at most
# 1,000,000. - Words contain lowercase English letters.
# ### Clarifications - Inserting the same word repeatedly
# is allowed and has no additional effect. - A node must
# distinguish an inserted word ending there from a prefix
# only. - Return booleans in the same order as search
# operations appear.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support '.' as a wildcard in search (LeetCode 211).
# 2. Delete a word.
# 3. Unicode or large alphabets.
#
# INTERVIEW NOTES
# A trie needs a flag that marks 'a word ends here';
# without it search('app') would be true whenever 'apple'
# was inserted. Walking a prefix and checking the flag are
# the two operations: search checks the flag, startsWith
# does not.
#
# BRUTE FORCE (say it first): Keep a list of words and
# scan
# Store the words in a list; search compares equality with
# each, startsWith tests the prefix on each. Correct, with
# every call linear in the dictionary size. Time O(W * L)
# per operation. Space O(total characters).


# BRUTE FORCE (say it first): Keep a list of words and
# scan | time O(W * L) per operation
class TrieBrute:
    def __init__(self):
        self.words = []
```

```
    def insert(self, word):
        self.words.append(word)
```

```
    def search(self, word):
        return word in self.words

    def startsWith(self, prefix):
        return any(
            w.startswith(prefix) for w in self.words
        )


# HELPERS for the base question
class Node:
    def __init__(self):
        self.children = {}
        self.end = False


# BASE QUESTION (optimal): Trie of nodes with an end-of-
# word flag
# Idea: Each node has children by character and a flag.
# insert creates missing nodes and marks the last one.
# search walks the word and requires the flag at the end;
# startsWith only requires that the walk succeeds.
# FOLLOW-UP 1: Support '.' as a wildcard in search
# (LeetCode 211).
# Answer: On '.', try every child at that level and
# recurse; other characters follow the single matching
# child. Worst case O(26^L) per query on adversarial
# patterns, so mention the bound.
class Trie:
    def __init__(self):
        self.root = Node()

    def walk(self, text):
        # follow the letters; None if the path breaks
        node = self.root
        for ch in text:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def insert(self, word):
```

```
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, Node())
        node.end = True

    def search(self, word):
        # the whole word must have been inserted
        node = self.walk(word)
        return node is not None and node.end

    def starts_with(self, prefix):
        return self.walk(prefix) is not None

    # FOLLOW-UP 1: '.' matches any one letter
    def search_wild(self, pattern):
        # '.' matches any one letter (LeetCode 211)
        def go(node, i):
            if i == len(pattern):
                return node.end
            ch = pattern[i]
            if ch == ".":
                # try every child at this level
                kids = node.children.values()
                return any(go(c, i + 1) for c in kids)
            child = node.children.get(ch)
            return child is not None and go(child, i + 1)

        return go(self.root, 0)
```

```
    # FOLLOW-UP 2: delete a word
    def delete(self, word):
        # unmark the word, then prune nodes nothing needs
        path = [self.root]
        for ch in word:
            nxt = path[-1].children.get(ch)
            if nxt is None:
                return False
            path.append(nxt)
        if not path[-1].end:
            return False
        path[-1].end = False
        # walk back up removing empty, non-word nodes
        for i in range(len(word), 0, -1):
            node = path[i]
            if node.end or node.children:
```

```
                break
            del path[i - 1].children[word[i - 1]]
        return True


# HELPERS for the checks below
def run(operations):
    # ["insert", w] or ["search", w] -> results of the
    # searches
    trie, out = Trie(), []
    for op, word in operations:
        if op == "insert":
            trie.insert(word)
        else:
            out.append(trie.search(word))
    return out


if __name__ == "__main__":
    ops = [("insert", "apple"), ("search", "apple"),
           ("search", "app"), ("insert", "app"),
           ("search", "app")]
    assert run(ops) == [True, False, True]
    t = Trie()
    t.insert("apple")
    # a shared prefix is not a match for search
    assert not t.search("app") and t.starts_with("app")
    assert not t.starts_with("b")
    t.insert("bad")
    t.insert("dad")
    assert t.search_wild("b.d") and t.search_wild(".ad")
    assert not t.search_wild("b..d")
    assert t.search_wild("...")
    # delete keeps shorter words that share the path
    t.insert("app")
    assert t.delete("apple")
    assert t.search("app") and not t.search("apple")
    assert not t.delete("apple")
    assert t.delete("app") and not t.starts_with("a")
```

FUNCTION AND TEST CASES

Signature

```
def process_trie_operations(operations: list[list[str]]) -
> list[bool]:
```

Sample call

```
process_trie_operations([['insert', 'apple'], ['search', '
apple'], ['search', 'app'], ['insert', 'app'], ['search',
'app']])
```

Test cases

Input	Expected
`([['insert', 'apple'], ['search', 'apple'], ['search', 'app'], ['insert', 'app'], ['search', 'app']],)`	`[True, False, True]`
`([],)`	`[]`

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 7 of 16

Design Tic-Tac-Toe With K Players

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 153/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

LeetCode match: LC 348 Design Tic-Tac-Toe

Posted on PracHub: 2026-02-12 (updated 2026-05-21)

Original: https://prachub.com/coding-questions/design-tic-tac-toe-with-k-players

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **`n x n`** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.
- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.

QUESTION

- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

Constraints

1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

EXAMPLES

```
Input: (3, 2, [])
Expected Output: []
```

Explanation: No moves are made, so there are no statuses to report.

```
Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2,
    1, 1], [1, 0, 2], [2, 2, 2]])
Expected Output: ['Continue', 'Continue', 'Continue',
    'Continue', '1 won', 'Game Over']
```

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

APPROACH

Per-player row, column, diagonal and anti-diagonal counters; a player wins when any counter reaches n. O(1) per move.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a win needs a full row, column or diagonal (n in a row) or only m in a row, how invalid moves are reported, what happens on a full board, and the API (move(row, col, player) returns winner or none).

BRUTE FORCE TO SAY OUT LOUD

After each move scan the row, column and both diagonals of the board. O(n) per move.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Keep counters per player for each row, each column, the main diagonal and the anti-diagonal. A move increments up to four counters; a player wins when any of their counters reaches n. O(1) per move, O(k * n) memory using a dict of counters. Reject moves on occupied cells.

DRY-RUN THESE CASES

A diagonal win, an anti-diagonal win, a row win by the last player, a draw, a move on an occupied cell, a move after the game has ended.

LIKELY FOLLOW-UPS

Win length m smaller than n (scan four directions from the last move for up to m cells), undo a move, or an unbounded sparse board.

PITFALLS

Counting an occupied cell twice. Not ending the game after a win.

HOW TO THINK ABOUT IT

Do not look at the board after each move. A win needs n marks of the same player on one line, so keep a counter per player and per line (each row, each column, the two diagonals) and bump the at-most-four lines that the move touches. The move wins if any of those counters reaches n.

Basis: LeetCode 348, Design Tic-Tac-Toe, generalized to k players. Assumed rule (confirm it): a player wins by filling a whole row, a whole column or a whole diagonal of the n x n board.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def move(player, r, c):
    board[r][c] = player
    lines = [row(r), column(c)]
    if r == c:
        lines.append(main_diagonal())
    if r + c == n - 1:
        lines.append(anti_diagonal())
    for line in lines:
        if line.count(player) == n:
            return "WIN"
    return "CONTINUE"
```

TIME	SPACE
O(n) per move	O(n^2)

WHY IT IS SLOW

Each move rescans up to four lines, O(n). Keeping a counter per player per row, column and diagonal makes each move O(1).

OPTIMAL SOLUTION · Counters per player and per line

TIME	SPACE
O(1) per move	O(k * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Win with m in a row, not the full line.

Counters per line no longer work, because a line can hold a run anywhere. After each move look along the four directions from that cell, counting same-player neighbours up to m in both directions. O(m) per move.

FOLLOW-UP 2 Undo the last move.

Decrement the same counters you incremented and clear the cell; remember the move history in a stack. The counters make undo as cheap as a move.

FOLLOW-UP 3 The board is huge or unbounded.

Store only the occupied cells in a dict (as TicTacToeM does) and keep the counter-per-line version only if the win condition is a complete line of a bounded board. **Cost:** O(1) per move with line counters on a bounded board, or O(m) per move scanning four directions for an m-in-a-row win; memory O(moves).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design Tic-Tac-Toe With K Players
#
# Design and implement a Tic-Tac-Toe game engine. You are
# given: - k players, each with a unique player ID or
# mark. - An n x n board, initially empty. - Players take
# turns placing their own mark on an empty cell. Unlike
# classic Tic-Tac-Toe, a player wins as soon as they form
# **3 consecutive marks** in a straight line, regardless
# of how large n is. A straight line may be horizontal,
# vertical, diagonal down-right, or diagonal down-left.
# Implement a class or set of functions that supports
# making moves and returns the game status after each
# move, such as: - "Game Over" if a move is attempted
# after the game has ended. - "<player> won" when the
# latest move causes that player to have 3 consecutive
# marks. - A non-terminal status such as "Continue" if the
# game should keep going. Your implementation should
# handle invalid moves, such as placing a mark outside the
# board or on an occupied cell.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Win with m in a row, not the full line.
# 2. Undo the last move.
# 3. The board is huge or unbounded.
#
# INTERVIEW NOTES
```

```python
# Do not look at the board after each move. A win needs n
# marks of the same player on one line, so keep a counter
# per player and per line (each row, each column, the two
# diagonals) and bump the at-most-four lines that the move
# touches. The move wins if any of those counters reaches
# n.
#
# BRUTE FORCE (say it first): Scan the lines through the
# last move
# Store the board and, after each move, check the move's
# row, column and (if on one) the diagonals for n equal
# marks. Simple and fast enough for small n, but every
# move rescans up to four lines. Time O(n) per move. Space
# O(n^2).


# BRUTE FORCE (say it first): Scan the lines through the
# last move | time O(n) per move
class TicTacToeBrute:
```

```python
    def __init__(self, n):
        self.n, self.board = n, [
            [0] * n for _ in range(n)
        ]

    def move(self, r, c, player):
        if self.board[r][c]:
            return -1
        self.board[r][c] = player
        n, b = self.n, self.board
        if all(
            b[r][j] == player for j in range(n)
        ) or all(b[i][c] == player for i in range(n)):
            return player
        if r == c and all(
            b[i][i] == player for i in range(n)
        ):
            return player
        if r + c == n - 1 and all(
            b[i][n - 1 - i] == player for i in range(n)
        ):
            return player
        return 0


# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


# BASE QUESTION (optimal): Counters per player and per
# line
# Idea: rows[p][r], cols[p][c], diag[p] and anti[p] count
# each player's marks on those lines. A move increments
# the counters for its row, column and, if applicable, the
# diagonals; the player wins when one reaches n.
# FOLLOW-UP 1: Win with m in a row, not the full line.
# Answer: Counters per line no longer work, because a line
# can hold a run anywhere. After each move look along the
# four directions from that cell, counting same-player
# neighbours up to m in both directions. O(m) per move.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        # FOLLOW-UP 3: a huge or unbounded board
        # only occupied cells are stored (works for huge
```

```python
        # boards)
        self.cells = {}
        self.history = []
        self.winner = None

    def turn(self):
        # players go 1, 2, ..., k, then 1 again
        return len(self.history) % self.k + 1

    def run_length(self, r, c, dr, dc, player):
        # marks of `player` in a line through (r, c)
        run = 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
            while self.cells.get((a, b)) == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        if self.winner is not None:
            return "Game Over"
        # an invalid move changes nothing
        if not 1 <= player <= self.k:
            return "Invalid move"
        if player != self.turn():
            return "Invalid move"
        if not (0 <= r < self.n and 0 <= c < self.n):
            return "Invalid move"
        if (r, c) in self.cells:
            return "Invalid move"
        self.cells[(r, c)] = player
        self.history.append((r, c))
        # only the four lines through the new mark can win
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc, player)
            # FOLLOW-UP 1: win with `target` in a row
            if run >= self.target:
                self.winner = player
                return f"{player} won"
        return "Continue"

    # FOLLOW-UP 2: undo the last move
    def undo(self):
        # take the last move back (also reopens a finished
```

```python
        # game)
        if not self.history:
            return False
        del self.cells[self.history.pop()]
        self.winner = None
        return True


# HELPERS for the checks below
def process(n, k, moves):
    game = TicTacToe(n, k)
    return [game.move(*m) for m in moves]


if __name__ == "__main__":
    moves = [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1],
             [1, 0, 2], [2, 2, 2]]
    assert process(3, 2, moves) == [
        "Continue", "Continue", "Continue", "Continue",
        "1 won", "Game Over"]
    # invalid moves do not use up the turn
    got = process(3, 2, [[2, 0, 0], [1, 5, 5], [1, 0, 0],
                         [2, 0, 0], [3, 1, 1], [2, 1, 1]])
    bad = "Invalid move"
    ok = "Continue"
    assert got == [bad, bad, ok, bad, bad, ok]
    g = TicTacToe(3, 2)
    for m in moves[:5]:
        g.move(*m)
    assert g.winner == 1 and g.undo() and g.winner is None
    assert g.move(1, 0, 2) == "1 won"
    # a huge board costs nothing: only taken cells are
    # stored
    big = TicTacToe(10 ** 9, 2)
```

```
    assert big.move(1, 10 ** 9 - 1, 5) == "Continue"
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, k: int, moves: list[list[int]]) -> li
st[str]:
```

Sample call

```
solution(3, 2, [])
```

Test cases

Input	Expected
`(3, 2, [])`	`[]`
`(5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])`	`['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']`

ALSO REPORTED · 1 more

Tic-Tac-Toe System with K Players and 3-in-a-Row Win | 1Point3Acres | phone-screen, tech-screen | asked 2026-05-18 | Single report

https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

Check four directions through the new mark: horizontal, vertical, main diagonal, anti-diagonal. Count contiguous marks on both sides; win if total reaches 3.
Avoid scanning the whole board after every move unless n is tiny. The local directional check is O(1) for the fixed 3-in-a-row target, or O(target) if generalized.
Clarify invalid moves, turn order, draw state, and whether 3-in-a-row must be exactly 3 or at least 3.

Preparation: Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

Site tags: data-structure, oop-design, medium

MEDIUM | 1Point3Acres | onsite-coding | 8 of 16

Restaurant Waitlist: join / delete / find_first_match

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 216/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/waitlist-data-structure-with-table-size-match

QUESTION

Onsite coding round: design and implement a restaurant waitlist data structure supporting join, delete, and find_first_match(table_size). The find call returns the earliest-joined user whose party size fits the given table.

Implement a class with these methods

join(user) — append user (with their party_size) to the waitlist.
delete(user) — remove user from the waitlist (could be anywhere in the queue).
find_first_match(table_size: int) — return the earliest user still on the waitlist whose party_size <= table_size. Do not remove on find.

Implicit constraints

Order of joins matters (FIFO within those that match).
Users may be deleted out of order, so a plain array/linked list with O(1) join cannot give O(1) delete and find together.

EXAMPLES

```
join(Alice, size=2)
join(Bob, size=4)
join(Carol, size=6)
find_first_match(3) -> Alice   // only Alice fits
find_first_match(5) -> Alice   // Alice still earliest
    matching
delete(Alice)
find_first_match(5) -> Bob
find_first_match(2) -> null     // no one fits
```

APPROACH

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

HOW TO THINK ABOUT IT

Join needs the order of arrival, delete can hit anyone, and find must return the earliest person whose party fits the table. Keep one queue per party size and a sequence number per person: find looks at the front of every queue for sizes up to the table size and takes the smallest sequence number. Deletions are marked and skipped lazily when a queue front is read.

Basis: Written for this question (a doubly linked list plus hash map as in LeetCode 146, LRU Cache, with queues bucketed by party size).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
queue = []  # in arrival order
def join(user):
    queue.append(user)
def delete(user):
    queue.remove(user)
def find_first_match(t):
    for user in queue:
        if user.party_size <= t:
            return user
    return None
```

TIME	SPACE
O(n) per find and per delete	O(n)

WHY IT IS SLOW

Delete and find are linear in the list. One queue per party size with lazy deletion makes join and delete O(1) and find touch only the queue fronts.

OPTIMAL SOLUTION · Queue per party size with lazy deletion

TIME	SPACE
O(1) join and delete, O(S) find (S distinct sizes up to the table size)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What would you cache to speed up repeated finds?

Keep a small structure that answers 'earliest sequence among sizes <= t' directly: a segment tree or Fenwick tree over party size storing the minimum front sequence number, updated when a queue front changes. Find becomes O(log S) instead of O(S).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Restaurant Waitlist: join / delete /
# find_first_match
#
# Onsite coding round: design and implement a restaurant
# waitlist data structure supporting join, delete, and
# find_first_match(table_size). The find call returns the
# earliest-joined user whose party size fits the given
# table. Implement a class with these methods: join(user)
# — append user (with their party_size) to the waitlist.
# delete(user) — remove user from the waitlist (could be
# anywhere in the queue). find_first_match(table_size:
# int) — return the earliest user still on the waitlist
# whose party_size <= table_size. Do not remove on find.
# Implicit constraints Order of joins matters (FIFO within
# those that match). Users may be deleted out of order, so
# a plain array/linked list with O(1) join cannot give
# O(1) delete and find together.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What would you cache to speed up repeated finds?
#
# INTERVIEW NOTES
# Join needs the order of arrival, delete can hit anyone,
# and find must return the earliest person whose party
# fits the table. Keep one queue per party size and a
# sequence number per person: find looks at the front of
# every queue for sizes up to the table size and takes the
# smallest sequence number. Deletions are marked and
# skipped lazily when a queue front is read.
#
# BRUTE FORCE (say it first): Keep one list in join order
# and scan it
# Append on join, remove by value on delete and scan from
# the front on find. Correct and the oracle; delete and
# find are linear. Time O(n) per find and per delete.
# Space O(n).


from collections import defaultdict, deque


# BRUTE FORCE (say it first): Keep one list in join order
# and scan it | time O(n) per find and per delete
class WaitlistBrute:
    def __init__(self):
        self.people = []

    def join(self, name, size):
        self.people.append((name, size))

    def delete(self, name):
        self.people = [
            p for p in self.people if p[0] != name
        ]

    def find_first_match(self, table):
        return next(
            (n for n, s in self.people if s <= table),
            None,
        )


# BASE QUESTION (optimal): Queue per party size with lazy
# deletion
# Idea: join stores (sequence number, name) in the queue
# for its size and in a map from name to its record.
# delete just removes the record from the map.
class Waitlist:
    def __init__(self):
        # arrival counter: lower means joined earlier
        self.seq = 0
        # person -> their arrival number (people still
        # waiting)
        self.live = {}
        # party size -> line of (arrival number, person)
        self.queues = defaultdict(deque)

    def join(self, name, size):
        self.seq += 1
        self.live[name] = self.seq
        self.queues[size].append((self.seq, name))

    def delete(self, name):
        # lazy: the entry in its queue is skipped later
        self.live.pop(name, None)

    def front(self, queue):
        # drop deleted people from the front of a queue
        while queue:
            seq, name = queue[0]
            if self.live.get(name) == seq:
                break
            queue.popleft()
        return queue[0] if queue else None

    def find_first_match(self, table):
        best = None
        for size, queue in self.queues.items():
            if size > table:
                continue
            head = self.front(queue)
            # the earliest head among all sizes that fit
            if head and (not best or head[0] < best[0]):
                best = head
        return best[1] if best else None


if __name__ == "__main__":
    w = Waitlist()
    for name, size in [("Alice", 2), ("Bob", 4)]:
        w.join(name, size)
    w.join("Carol", 6)
    assert w.find_first_match(3) == "Alice"
    # Alice is still the earliest who fits 5
    assert w.find_first_match(5) == "Alice"
    # find does not remove anyone
    assert w.find_first_match(5) == "Alice"
    w.delete("Alice")
    assert w.find_first_match(5) == "Bob"
    assert w.find_first_match(1) is None
    # the same size: arrival order decides
    w.join("Dan", 4)
    w.delete("Bob")
    assert w.find_first_match(4) == "Dan"
    # deleting someone who is not there is harmless
    w.delete("Nobody")
```

NOTES FROM CANDIDATES

A doubly-linked list + hashmap (LRU-style) gives O(1) join and delete; find_first_match then walks the list.
For faster find, bucket users by party size (e.g. one queue per size), and on find_first_match(t) scan sizes 1..t and pick the queue with the smallest head timestamp.
The interviewer cares more about clean class design + tradeoff discussion than a single optimal solution; mention what you'd cache to speed up repeated finds.

PREPARATION

Drill LRU-cache style implementations (hashmap + DLL) to muscle memory.
Sketch a second variant with party-size buckets and verbalize the trade-off (find_first_match becomes O(table_size) instead of O(n) but uses more memory).
Practice articulating which call should be fast in a real restaurant — typically find_first_match runs once per available table.

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 9 of 16

★ TOP 75: rank 36 in the Medium and Hard priority list

★ Design a logger that returns top-K messages

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 33 | PRIORITY 33/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #359 Logger Rate Limiter, LeetCode #692 Top K Frequent Words

LeetCode match: LC 359 Logger Rate Limiter + LC 692 Top K Frequent Words (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-23)

Original: https://prachub.com/coding-questions/design-a-logger-that-returns-top-k-messages

QUESTION

Overview: This question evaluates skills in designing in-memory data structures and algorithms for frequency counting and interval querying, including extracting top-K elements with a heap, within the Coding & Algorithms domain.

APPROACH

Hash map message -> count (plus a deque of timestamps for interval queries); top-K with a size-k heap, ties broken lexicographically.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy has only the overview, so ask what the API is: log(timestamp, message), top-K over all time or over an interval, whether duplicates within a time gap are suppressed (the LeetCode 359 rate limiter), and how ties break (lexicographically, for LeetCode 692).

BRUTE FORCE TO SAY OUT LOUD

Store every entry in a list and sort or scan on each query. O(n log n) per query.

OPTIMAL IDEA

A dict message to count (and, for interval queries, a dict message to a list of timestamps kept in order so a time window is two binary searches). Top-K uses a heap of size K with the key (count descending, message ascending): heapq.nsmallest(k, items, key=(-count, message)). The rate limiter part is a dict message to last printed time, allowing a message when the gap is at least 10 seconds. O(d log K) per top-K query.

DRY-RUN THESE CASES

Fewer than K distinct messages, ties on count, the same message repeated inside the rate-limit window, an empty logger.

LIKELY FOLLOW-UPS

Streaming top-K with bounded memory (Space-Saving), or evicting old entries.

PITFALLS

Not specifying the tie rule. Using the full sort when only K items are needed.

HOW TO THINK ABOUT IT

Store enough to answer both questions without scanning everything. A dict from message to its list of timestamps (appended in time order) answers 'how many in [start, end]' with two binary searches, and a heap of size K over (count, message) gives the top K. Say how the logger handles repeated messages and what 'top' means with ties.

Basis: Written (compare LeetCode 359 Logger Rate Limiter and 692 Top K Frequent Words). Assumed API (confirm it): log(timestamp, message); top_k(k, start, end) over an optional time window; ties by message alphabetically.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
logs = []  # (timestamp, message)
def top_k(k, start, end):
    counts = Counter()
    for ts, msg in logs:
        if start <= ts <= end:
            counts[msg] += 1
    def rank(m):
        return (-counts[m], m)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d) per query	O(n)

WHY IT IS SLOW

Every query reads the whole log, O(n + d log d). Per-message sorted timestamps with binary search answer a window count in O(log n), and a heap picks the top k.

OPTIMAL SOLUTION · Per-message sorted timestamps plus a heap for the top K

TIME	SPACE
O(1) per log, O(d log n + d log k) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print a message only if it was not printed in the last 10 seconds (LeetCode 359).

Keep a dict message to the time it was last printed and compare: print if the gap is at least 10 (and update the time only when you print). O(1) per call.

FOLLOW-UP 2 Memory must stay bounded.

Expire timestamps older than the largest window you support (drop from the front of each list, delete messages with no timestamps left), or approximate counts with a count-min sketch plus a heap for the heavy hitters. **Cost:** O(1) amortized to drop expired timestamps; memory O(messages inside the window).

FOLLOW-UP 3 Very high top_k frequency of calls.

Maintain counts per message incrementally and a top-K structure (a heap with lazy updates or a sorted structure) so top_k costs O(k) instead of O(d log k); windowed queries still need per-message timestamps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design a logger that returns top-K
# messages
#
# This question evaluates skills in designing in-memory
# data structures and algorithms for frequency counting
# and interval querying, including extracting top-K
# elements with a heap, within the Coding & Algorithms
# domain.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print a message only if it was not printed in the
# last 10 seconds (LeetCode 359).
# 2. Memory must stay bounded.
# 3. Very high top_k frequency of calls.
#
# INTERVIEW NOTES
# Store enough to answer both questions without scanning
# everything. A dict from message to its list of
# timestamps (appended in time order) answers 'how many in
# [start, end]' with two binary searches, and a heap of
# size K over (count, message) gives the top K. Say how
# the logger handles repeated messages and what 'top'
# means with ties.
#
# BRUTE FORCE (say it first): Keep every log entry and
# scan it for each query
# Store all (timestamp, message) pairs; for a query count
# the messages inside the window and sort. Correct and the
# oracle; every query reads the whole log. Time O(n + d
# log d) per query. Space O(n).


from collections import Counter
import heapq
from bisect import bisect_left, bisect_right
from collections import defaultdict


# BRUTE FORCE (say it first): Keep every log entry and
# scan it for each query | time O(n + d log d) per query
class LoggerBrute:
    def __init__(self):
        self.entries = []

    def log(self, ts, msg):
        self.entries.append((ts, msg))

    def top_k(self, k, start=None, end=None):
        c = Counter(
            m
            for t, m in self.entries
            if (start is None or t >= start)
            and (end is None or t <= end)
        )
        return sorted(c, key=lambda m: (-c[m], m))[:k]


# BASE QUESTION (optimal): Per-message sorted timestamps
# plus a heap for the top K
# Idea: log appends the timestamp to the message's list
# (timestamps arrive in order, so lists stay sorted). A
# window count for one message is bisect_right(end) -
# bisect_left(start).
class Logger:
    def __init__(self):
        # message -> its timestamps, in order of arrival
        self.times = defaultdict(list)

    def log(self, ts, msg):
        self.times[msg].append(ts)

    def top_k(self, k, start=None, end=None):
        # the k most frequent messages in [start, end]
        counts = {}
        for msg, ts in self.times.items():
            lo, hi = 0, len(ts)
            if start is not None:
                lo = bisect_left(ts, start)
            if end is not None:
                hi = bisect_right(ts, end)
            if hi > lo:
                counts[msg] = hi - lo
        # ties are broken alphabetically
        def rank(m):
            return (-counts[m], m)
        return heapq.nsmallest(k, counts, key=rank)

    def expire(self, before):
        # bounded memory: forget timestamps older than
        # `before`
        for msg in list(self.times):
            ts = self.times[msg]
            self.times[msg] = ts[bisect_left(ts, before):]
```

```
            if not self.times[msg]:
                del self.times[msg]


# FOLLOW-UP 1: Print a message only if it was not printed
# in the last 10 seconds (LeetCode 359).
# Answer: Keep a dict message to the time it was last
# printed and compare: print if the gap is at least 10
# (and update the time only when you print). O(1) per
# call.
class RateLimiter:
    """LeetCode 359: one print per message per gap."""

    def __init__(self, gap=10):
        self.gap, self.last = gap, {}

    def should_print(self, ts, msg):
        last = self.last.get(msg)
        if last is not None and ts - last < self.gap:
            return False
        # only a printed message starts a new window
        self.last[msg] = ts
        return True


if __name__ == "__main__":
    log = Logger()
    events = [(1, "a"), (2, "b"), (3, "a"),
              (4, "c"), (5, "a"), (6, "b")]
    for ts, msg in events:
        log.log(ts, msg)
    assert log.top_k(2) == ["a", "b"]
    # only the window [4, 6]: a once, b once, c once
    assert log.top_k(3, 4, 6) == ["a", "b", "c"]
    assert log.top_k(2, 100, 200) == []
    log.expire(4)
    assert log.top_k(5) == ["a", "b", "c"]
    r = RateLimiter()
    times = (1, 2, 11, 12, 21)
    got = [r.should_print(t, "x") for t in times]
    assert got == [True, False, True, False, True]
```

FUNCTION AND TEST CASES

Signature

```
def solution(logs: list[tuple[int, str]], queries: list[li
st[int]]) -> list[list[str]]:
```

Sample call

```
solution([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3
, 2), (1, 10, 3)])
```

Test cases

Input	Expected
`([(1, 'A'), (2, 'B'), (3, 'A'), (10, 'C')], [(1, 3, 2), (1, 10, 3)])`	`[['A', 'B'], ['A', 'B', 'C']]`
`([(1, 'dog'), (2, 'cat'), (2, 'dog'), (4, 'cat'), (5, 'ant')], [(1, 4, 2)])`	`[['cat', 'dog']]`

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 10 of 16

Implement iterator merging lists with filtering

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 155/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #281 Zigzag Iterator

LeetCode match: LC 281 Zigzag Iterator (+ filter/dedupe) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/implement-iterator-merging-lists-with-filtering

QUESTION

Overview: This question evaluates iterator/generator design, sequence merging, stateful filtering and deduplication, checking a candidate's ability to maintain input ordering while excluding blocked IDs and preventing duplicate yields.

APPROACH

Keep one pointer per list and a lookahead 'next valid' value for hasNext/next; skip filtered and duplicate values; or merge with a heap if sorted.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how the lists are merged: round robin (like LeetCode 281 Zigzag Iterator), one after the other, or by sorted value; what is filtered (a blocked set of ids); whether duplicates must be removed across lists; and what next() does when the iterator is exhausted.

BRUTE FORCE TO SAY OUT LOUD

Build one merged list up front with the filter applied. Simple but not lazy and not memory friendly.

OPTIMAL IDEA

A lazy iterator with one pointer per list and a precomputed next valid value (lookahead). hasNext checks the lookahead; next returns it and then advances to the following valid element: skip blocked ids and anything already in a seen set. For sorted merges use a heap of (value, list index, position). Each element is touched once, O(total) overall.

DRY-RUN THESE CASES

Empty lists, all values blocked, duplicates across lists, calling next after the end, a list that becomes empty mid-way.

LIKELY FOLLOW-UPS

Make the iterator re-iterable, support remove, or bound memory for the seen set when ids are unbounded (Bloom filter).

PITFALLS

Computing the lookahead lazily at the wrong moment so hasNext skips or repeats a value.

HOW TO THINK ABOUT IT

An iterator does its work lazily: hasNext() must be able to answer without consuming. So keep a lookahead, the next valid value already computed, and advance to the one after it when next() is called. Filtering and de-duplication live in the 'find the next valid value' step so both hasNext and next stay trivial.

Basis: LeetCode 281, Zigzag Iterator, with a blocked-id filter and no duplicate output. Assumed rules (confirm them): lists are read round robin; ids in the blocked set are skipped; each id is yielded at most once.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def merged_iterator(lists, blocked):
    seen = set()
    for x in round_robin(lists):
        if x in blocked or x in seen:
            continue
        seen.add(x)
        yield x
```

TIME	SPACE
O(total)	O(total)

WHY IT IS SLOW

It reads everything up front and keeps it all in memory, so it is not lazy and never works on a long or unbounded source. A lookahead value per iterator computes only the next valid item.

OPTIMAL SOLUTION · Lazy iterator with one pointer per list and a lookahead

TIME	SPACE
O(1) amortized per value	O(number of lists + distinct ids yielded)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The lists are sorted and the output must stay sorted.

Replace the round robin with a k-way merge on a heap (heapq.merge), and drop a value equal to the previous output instead of keeping a seen set: memory O(k).

FOLLOW-UP 2 Read the lists one after another, not interleaved.

Chain the lists with a generator; the same blocked/seen filtering applies. Generators give hasNext-style behaviour via a one-item lookahead.

FOLLOW-UP 3 The seen set grows without bound.

If the input is sorted or ids repeat only close together, remember only the last few ids; otherwise bound memory with a Bloom filter and accept rare false suppressions. **Cost:** O(1) per element with a Bloom filter using O(bits) memory, at the price of rare false suppressions.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement iterator merging lists with
# filtering
#
# This question evaluates iterator/generator design,
# sequence merging, stateful filtering and deduplication,
# checking a candidate's ability to maintain input
# ordering while excluding blocked IDs and preventing
# duplicate yields.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The lists are sorted and the output must stay sorted.
# 2. Read the lists one after another, not interleaved.
# 3. The seen set grows without bound.
#
# INTERVIEW NOTES
# An iterator does its work lazily: hasNext() must be able
# to answer without consuming. So keep a lookahead, the
# next valid value already computed, and advance to the
# one after it when next() is called. Filtering and de-
# duplication live in the 'find the next valid value' step
# so both hasNext and next stay trivial.
#
# BRUTE FORCE (say it first): Build the full merged,
# filtered list first
# Interleave the lists, drop blocked and repeated ids, and
# iterate over the resulting list. Correct and a good
# oracle, but it is not lazy: it reads everything up front
# and holds it all in memory. Time O(total). Space
# O(total).


import heapq
from itertools import chain


# BRUTE FORCE (say it first): Build the full merged,
# filtered list first | time O(total)
def merged_filtered_brute(lists, blocked):
    out, seen = [], set()
    for step in range(
        max((len(l) for l in lists), default=0)
    ):
        for l in lists:
            if (
                step < len(l)
                and l[step] not in blocked
                and l[step] not in seen
            ):
                seen.add(l[step])
                out.append(l[step])
    return out


# BASE QUESTION (optimal): Lazy iterator with one pointer
# per list and a lookahead
# Idea: Keep a pointer per list and a rotation index.
# _advance() scans round robin for the next value that is
# not blocked and not already yielded and stores it as the
# lookahead (or None when everything is exhausted).
class FilteredZigzag:
    """Take one item from each list in turn; skip blocked
    ids and ids already returned. Has hasNext / next."""

    def __init__(self, lists, blocked=()):
        self.lists = lists
        self.blocked = set(blocked)
        self.pos = [0] * len(lists)
        self.turn = 0
        self.seen = set()
        # one item of lookahead makes hasNext easy
        self.ahead = None
        self.advance()

    def advance(self):
        self.ahead = None
        misses = 0
        # stop when every list had nothing left in a full
        # round
        while misses < len(self.lists):
            i = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.pos[i] >= len(self.lists[i]):
                misses += 1
                continue
            value = self.lists[i][self.pos[i]]
            self.pos[i] += 1
            misses = 0
            blocked = value in self.blocked
            skip = blocked or value in self.seen
            if not skip:
                self.ahead = value
                return

    def has_next(self):
        return self.ahead is not None

    def next(self):
        value = self.ahead
        self.seen.add(value)
        self.advance()
        return value


# FOLLOW-UP 1: The lists are sorted and the output must
# stay sorted.
# Answer: Replace the round robin with a k-way merge on a
# heap (heapq.merge), and drop a value equal to the
# previous output instead of keeping a seen set: memory
# O(k).
```

```
def merge_sorted(lists, blocked=()):
    # sorted lists, sorted output: a k-way merge; equal
    # values
    # come out together, so remember only the previous one
    blocked, last = set(blocked), object()
    for value in heapq.merge(*lists):
        if value in blocked or value == last:
            continue
        last = value
        yield value


# FOLLOW-UP 2: Read the lists one after another, not
# interleaved.
# Answer: Chain the lists with a generator; the same
# blocked/seen filtering applies. Generators give hasNext-
# style behaviour via a one-item lookahead.
def read_in_turn(lists, blocked=()):
    # one list after another, with the same filtering
    blocked, seen = set(blocked), set()
    for value in chain.from_iterable(lists):
        if value not in blocked and value not in seen:
            seen.add(value)
            yield value
```

```
if __name__ == "__main__":
    lists = [[1, 2, 3], [2, 4], [5]]
    it = FilteredZigzag(lists, blocked=[4])
    out = []
    while it.has_next():
        out.append(it.next())
    # 1, 2, 5, then 2 (seen), 3, 4 (blocked)
    assert out == [1, 2, 5, 3]
    empty = FilteredZigzag([[], []])
    assert not empty.has_next()
    assert list(merge_sorted([[1, 3, 5], [1, 2, 5]],
                             blocked=[3])) == [1, 2, 5]
    chained = read_in_turn([[1, 2], [2, 3]], [1])
    assert list(chained) == [2, 3]
```

FUNCTION AND TEST CASES

Sample call

```
solution([3, 1, 2, 3], [2, 4, 1, 5], {4})
```

Test cases

Input	Expected
`([3, 1, 2, 3], [2, 4, 1, 5], {4})`	`[3, 1, 2, 5]`
`([], [1, 1], set())`	`[1]`

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | IN APP | LC 380 | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 11 of 16

Design set with O(1) random access

POSTED 2026-01-01 | 292 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 46.6 | 30 DAYS 0 | PRIORITY 74/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #380 Insert Delete GetRandom O(1)

LeetCode match: LC 380 Insert Delete GetRandom O(1)

Posted on PracHub: 2026-01-01 (updated 2026-06-01)

Original: https://prachub.com/coding-questions/design-set-with-o-1-random-access

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **`x`** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **`x`** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).
- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.

QUESTION

- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints

1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

EXAMPLES

```
Input: (['add','add','getRandom','delete','getRandom'
    ,'add','getRandom'], [[1],[2],[],[1],[],[2],[]],
    7)
Expected Output: [True, True, 1, True, 2, False, 2]
```

Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

```
Input: (['getRandom','add','add','delete','delete','g
    etRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True,
    None]
```

Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

APPROACH

Array of values + hash map value -> index; remove by swapping with the last element and popping.

HOW TO THINK ABOUT IT

A hash set gives O(1) add and delete but cannot pick a random element in O(1); an array can pick a random element but deleting from the middle is O(n). Combine them: keep the values in an array and a dict from value to array index. To delete, swap the element with the last one, pop the end, and update the moved element's index.

Basis: LeetCode 380, Insert Delete GetRandom O(1) (FancySet). Choice made: getRandom on an empty set raises an error (a defined, loud failure).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
items = []
def add(x):
    if x in items:
        return False
    items.append(x)
    return True
def delete(x):
    if x not in items:
        return False
    items.remove(x)
    return True
def get_random():
    return random.choice(items)
```

TIME	SPACE
O(n) for add and delete, O(1) getRandom	**O(n)**

WHY IT IS SLOW

Membership checks and deletion are O(n). An array plus a dictionary from value to index, deleting by swapping with the last element, makes all three operations O(1).

OPTIMAL SOLUTION · Array of values plus a dict from value to index

TIME	SPACE
O(1) average per operation	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make it thread-safe: one lock or finer-grained locking?

One lock around every operation is simplest and correct (delete touches the array, the dict and a second element's index together, so they must change as one unit). Finer locks do not help much because every operation touches shared structure; use sharding by value hash if contention is the problem.

FOLLOW-UP 2 What should getRandom do on an empty set?

Raise a clear error (as here) or return a sentinel such as None; whichever you pick, document it and make the caller handle it. Returning an arbitrary value hides bugs.

FOLLOW-UP 3 Duplicates allowed (a multiset, LeetCode 381).

Map each value to a set of indices in the array; delete swaps with the last element and fixes both index sets. getRandom stays a uniform pick over array positions, so values are chosen in proportion to their counts.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design set with O(1) random access
#
# Design a data structure ("FancySet") that stores unique
# integers and supports the following operations, each in
# average O(1) time: - add(x): Insert value x if it is not
# already present. Return whether the insert happened. -
# delete(x): Remove value x if it exists. Return whether a
# deletion happened. - getRandom(): Return a uniformly
# random element currently in the set. Requirements /
# follow-ups: - All operations should be average O(1). -
# Handle corner cases such as calling getRandom() on an
# empty set (define and justify behavior). - Discuss how
# you would make this data structure thread-safe. Compare
# using a single lock for the whole set vs finer-grained
# locking (e.g., element-level locks), and explain trade-
# offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make it thread-safe: one lock or finer-grained
# locking?
# 2. What should getRandom do on an empty set?
# 3. Duplicates allowed (a multiset, LeetCode 381).
#
# INTERVIEW NOTES
# A hash set gives O(1) add and delete but cannot pick a
# random element in O(1); an array can pick a random
# element but deleting from the middle is O(n). Combine
# them: keep the values in an array and a dict from value
# to array index. To delete, swap the element with the
# last one, pop the end, and update the moved element's
# index.
#
# BRUTE FORCE (say it first): A plain list
# Store values in a list; add checks for membership with a
# scan and delete removes by search. Correct and the
# oracle; membership and deletion are linear. Time O(n)
# for add and delete, O(1) getRandom. Space O(n).


import random
import threading


# BRUTE FORCE (say it first): A plain list | time O(n) for
# add and delete, O(1) getRandom
```

```
class FancySetBrute:
    def __init__(self):
        self.a = []
```

```
    def add(self, x):
        if x in self.a:
            return False
        self.a.append(x)
        return True

    def delete(self, x):
        if x not in self.a:
            return False
        self.a.remove(x)
        return True

    def getRandom(self):
        if not self.a:
            raise IndexError("empty set")
        return random.choice(self.a)


# BASE QUESTION (optimal): Array of values plus a dict
# from value to index
# Idea: add appends to the array and records the index.
# delete finds the index from the dict, moves the last
# element into that slot, updates its index in the dict,
# pops the array and removes the key.
class FancySet:
    """add, delete and getRandom, each O(1) on average."""

    def __init__(self, rng=None):
        # the values in a list (for random access) and a
        # map
        # from value to its index in that list
        self.items, self.pos = [], {}
        self.rng = rng or random

    def add(self, x):
        if x in self.pos:
            return False
        self.pos[x] = len(self.items)
        self.items.append(x)
        return True
```

```
    def delete(self, x):
        if x not in self.pos:
            return False
        i, last = self.pos[x], self.items[-1]
        # move the last item into the hole, then drop the
        # tail
        self.items[i] = last
        self.pos[last] = i
        self.items.pop()
        del self.pos[x]
        return True

    def get_random(self):
        if not self.items:
            # say what happens on an empty set
            raise IndexError("empty set")
        i = self.rng.randrange(len(self.items))
        return self.items[i]


# FOLLOW-UP 1: Make it thread-safe: one lock or finer-
# grained locking?
# Answer: One lock around every operation is simplest and
# correct (delete touches the array, the dict and a second
# element's index together, so they must change as one
# unit).
class SafeSet(FancySet):
    """One lock: delete changes three things at once."""

    def __init__(self, rng=None):
        super().__init__(rng)
```

```
        self.lock = threading.Lock()

    def add(self, x):
        with self.lock:
            return super().add(x)

    def delete(self, x):
        with self.lock:
            return super().delete(x)

    def get_random(self):
        with self.lock:
            return super().get_random()
```

```
# FOLLOW-UP 3: Duplicates allowed (a multiset, LeetCode
# 381).
# Answer: Map each value to a set of indices in the array;
# delete swaps with the last element and fixes both index
# sets. getRandom stays a uniform pick over array
# positions, so values are chosen in proportion to their
# counts.
class RandomizedCollection:
    """Duplicates allowed (LeetCode 381)."""

    def __init__(self):
        self.items = []
        # value -> set of indices where it sits in the
        # list
        self.where = {}

    def insert(self, x):
        first = x not in self.where or not self.where[x]
        spots = self.where.setdefault(x, set())
        spots.add(len(self.items))
        self.items.append(x)
        return first

    def remove(self, x):
        if not self.where.get(x):
            return False
        i = self.where[x].pop()
        end = len(self.items) - 1
        if i != end:
            # move the last item into the hole and fix its
            # index set
            last = self.items[end]
            self.items[i] = last
            self.where[last].discard(end)
            self.where[last].add(i)
        self.items.pop()
        return True

    def get_random(self):
        return random.choice(self.items)


if __name__ == "__main__":
    s = FancySet(random.Random(1))
```

```
    assert s.add(1) and s.add(2) and not s.add(1)
    assert s.delete(1) and not s.delete(1)
    assert s.get_random() == 2
    s.delete(2)
    try:
        s.get_random()
        raise AssertionError("empty set should raise")
    except IndexError:
        pass
    # every value is picked about equally often
    big = FancySet(random.Random(5))
    for v in range(4):
        big.add(v)
```

```
    picks = [big.get_random() for _ in range(4000)]
    for v in range(4):
        assert 850 < picks.count(v) < 1150
    # deleting from the middle keeps the map consistent
    for v in (0, 2):
        big.delete(v)
    assert sorted(big.items) == [1, 3]
    for v, i in big.pos.items():
        assert big.items[i] == v
    safe = SafeSet()
    assert safe.add(7) and safe.get_random() == 7
    rc = RandomizedCollection()
    assert rc.insert(5)
    assert not rc.insert(5)
    assert rc.insert(6)
    assert rc.remove(5) and rc.remove(5)
    assert not rc.remove(5)
    assert rc.items == [6]
```

FUNCTION AND TEST CASES

Signature

```
def solution(operations: list[str], values: list[list[int]], seed: int) -> str:
```

Sample call

```
solution(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
```

Test cases

Input	Expected
`(['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)`	`[True, True, 1, True, 2, False, 2]`
`(['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)`	`[None, True, False, False, True, None]`

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | PracHub | Technical Screen | 12 of 16

Match people to questions using tags and priorities

POSTED 2025-12-27 | 297 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 172/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

LeetCode match: LC 1820 Maximum Number of Accepted Invitations (bipartite matching) (inferred from overview)

Posted on PracHub: 2025-12-27 (updated 2026-06-04)

Original: https://prachub.com/coding-questions/match-people-to-questions-using-tags-and-priorities

QUESTION

Overview: This question evaluates competencies in bipartite matching and combinatorial optimization, along with practical data-structure design for scalable tag-based assignment and priority-aware selection.

APPROACH

Model as bipartite matching: edges where tags overlap, weights by priority; use Kuhn's/Hopcroft-Karp for max matching, or greedy by priority with an inverted index tag -> candidates.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether matching is one-to-one or a question can take many people, what priority means (higher wins, or a numeric weight to maximize), whether a person needs only one shared tag, and whether it must be maximum cardinality or maximum total priority.

BRUTE FORCE TO SAY OUT LOUD

Try every assignment: exponential. State it and move on.

OPTIMAL IDEA

Build an inverted index tag to candidate questions. If only the number of matches matters it is bipartite matching: Kuhn's augmenting-path algorithm in O(V * E) is enough, Hopcroft-Karp for scale. If priorities are weights use min-cost flow or the Hungarian algorithm. If priority is a strict order, process people by priority and augment, because matchings form a matroid and greedy by priority is optimal.

DRY-RUN THESE CASES

A person with no tag overlap. Two people competing for one question. A chain where the first greedy choice blocks a larger matching (shows why plain greedy fails without augmenting).

LIKELY FOLLOW-UPS

Capacities per question (turns into max flow), online arrivals, or fairness constraints.

PITFALLS

Assuming plain greedy gives a maximum matching. Not building the index and doing all-pairs tag checks.

HOW TO THINK ABOUT IT

'Assign each person at most one question, each question at most one person, allowed pairs given by tags' is maximum bipartite matching. The tag index only builds the edges faster. For maximum count use augmenting paths: try to give a person a question, and if taken ask its current owner to move. If priorities decide who is served, process people in priority order with the same augmenting step.

Basis: LeetCode 1820, Maximum Number of Accepted Invitations (bipartite matching). Assumed rules (confirm them): a person and a question can be matched if they share a tag; each person gets at most one question and each question at most one person; priority decides who is served first when not everyone can be matched.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = 0
def go(person, used, matches):
    global best
    if person == len(people):
        best = max(best, matches)
        return
    go(person + 1, used, matches)  # skip
    for q in range(len(questions)):
        if q in used:
            continue
        if not shares_tag(person, q):
            continue
        go(person + 1, used | {q},
           matches + 1)
```

TIME	SPACE
O((Q + 1)^P)	O(P)

WHY IT IS SLOW

Each person branches over all questions: exponential. Augmenting paths (move an earlier person to another question when needed) solve it in O(people * edges).

OPTIMAL SOLUTION · Augmenting paths (Kuhn's algorithm)

TIME	SPACE
O(P * E)	O(P + Q)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Priorities: higher-priority people must be served first.

Process people in decreasing priority and run the same augmenting step. Matchings of this kind form a matroid, so this greedy choice gives a matched set with the maximum total priority (checked against exhaustive search).

FOLLOW-UP 2 Edges have weights (how good a match is).

Then it is maximum-weight bipartite matching: the Hungarian algorithm in O(n^3), or min-cost max-flow. Kuhn's algorithm only maximises the count.

FOLLOW-UP 3 A question may take several people (capacity c).

Replicate the question c times as separate right-hand nodes, or run max flow with capacities on the question-to-sink edges.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Match people to questions using tags and
# priorities
#
# This question evaluates competencies in bipartite
# matching and combinatorial optimization, along with
# practical data-structure design for scalable tag-based
# assignment and priority-aware selection.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Priorities: higher-priority people must be served
# first.
# 2. Edges have weights (how good a match is).
# 3. A question may take several people (capacity c).
#
# INTERVIEW NOTES
# 'Assign each person at most one question, each question
# at most one person, allowed pairs given by tags' is
# maximum bipartite matching. The tag index only builds
# the edges faster. For maximum count use augmenting
# paths: try to give a person a question, and if taken ask
# its current owner to move. If priorities decide who is
# served, process people in priority order with the same
# augmenting step.
#
# BRUTE FORCE (say it first): Try every assignment
# recursively
# For each person either skip them or give them any unused
# question they share a tag with, and keep the largest
# number of matches. Exact, so it checks the fast
# algorithm, but exponential. Time O((Q + 1)^P). Space
# O(P).


# BRUTE FORCE (say it first): Try every assignment
# recursively | time O((Q + 1)^P)
def max_matching_brute(adj):
    best = 0

    def go(i, used, count):
        nonlocal best
        if i == len(adj):
            best = max(best, count)
            return
        go(i + 1, used, count)
        for q in adj[i]:
            if q not in used:
                go(i + 1, used | {q}, count + 1)

    go(0, frozenset(), 0)
    return best


# BUILD the person-question graph
def build_graph(people, questions):
    # a person can take a question if they share a tag
    # people: [(name, priority, tags)], questions: [(id,
    # tags, cap)]
    adj = {}
    for name, _, tags in people:
        adj[name] = [q for q, qtags, _ in questions
                     if set(tags) & set(qtags)]
    return adj


# BASE QUESTION (optimal): Augmenting paths (Kuhn's
# algorithm)
# Idea: For each person run a DFS that tries each
# acceptable question: if it is free take it, otherwise
# try to move its current owner to another question
# (recursively), using a visited set so no question is
# tried twice in one attempt.
def max_matching(adj):
    # the plain version: adj[p] = list of questions for
    # person p
    owner = {}

    def try_assign(p, seen):
        for q in adj[p]:
            if q in seen:
                continue
            seen.add(q)
            free = q not in owner
            if free or try_assign(owner[q], seen):
                owner[q] = p
                return True
        return False

    people = range(len(adj))
    return sum(try_assign(p, set()) for p in people)


# FOLLOW-UP 1: Priorities: higher-priority people must be
# served first.
# Answer: Process people in decreasing priority and run
# the same augmenting step. Matchings of this kind form a
# matroid, so this greedy choice gives a matched set with
# the maximum total priority (checked against exhaustive
# search).
def assign(people, questions):
    # Kuhn's augmenting paths. People are tried by
    # priority
    # (highest first): such matchings form a matroid, so
    # this
    # greedy order keeps the best possible set of people.
    adj = build_graph(people, questions)
    cap = {q: c for q, _, c in questions}
    # question -> the people currently holding its places
    held = {q: [] for q in cap}

    def try_assign(person, seen):
        for q in adj[person]:
            if q in seen:
```

```
                continue
            seen.add(q)
            # a free place, or someone who can move
            # elsewhere
            if len(held[q]) < cap[q]:
                held[q].append(person)
                return True
            for other in list(held[q]):
                if try_assign(other, seen):
                    held[q].remove(other)
                    held[q].append(person)
                    return True
        return False

    order = sorted(people, key=lambda p: -p[1])
    for name, _, _ in order:
        try_assign(name, set())
    return {p: q for q, ps in held.items() for p in ps}


if __name__ == "__main__":
    assert max_matching([[0, 1], [0], [1, 2]]) == 3
    assert max_matching([[0], [0], [0]]) == 1
```

```
    people = [("ann", 1, {"dp"}),
              ("bob", 5, {"dp", "graph"}),
              ("cy", 3, {"graph"})]
    questions = [("q1", {"dp"}, 1), ("q2", {"graph"}, 1)]
    got = assign(people, questions)
    # bob (priority 5) is served first, and the matching
    # is
    # still the largest possible: ann -> q1, bob or cy ->
    # q2
    assert len(got) == 2
    assert got["bob"] in ("q1", "q2")
    # a question with two places takes two people
    q = [("q1", {"dp"}, 2)]
    both = assign(people[:2], q)
    assert set(both) == {"ann", "bob"}
    # nobody shares a tag: nobody is matched
    nobody = assign([("x", 1, {"a"})], [("q", {"b"}, 1)])
    assert nobody == {}
```

FUNCTION AND TEST CASES

Signature

```
def solution(people: list[list[str]], question_tags: list[
list[str]], priorities: list[int]) -> list[int]:
```

Sample call

```
solution([['python', 'sql'], ['java'], ['ml']], [['sql'],
['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])
```

Test cases

Input	Expected
`([['python', 'sql'], ['java'], ['ml']], [['sql'], ['java', 'python'], ['ml'], ['go']], [5, 7, 4, 10])`	`(3, 16)`
`([['a'], ['a']], [['a'], ['a'], ['b']], [1, 10, 100])`	`(2, 11)`

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 13 of 16

Design server allocation for multi-type nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 220/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match; related LC 1912 Design Movie Rental System

Posted on PracHub: 2025-09-06 (updated 2026-08-05)

Original: https://prachub.com/interview-questions/design-server-allocation-for-multi-type-nodes

QUESTION

Design a service to manage a large pool (about 100, 000) of server nodes. Each node can satisfy multiple instance types; clients call getServer(reqNumOfServer, instanceTypes[]) to request servers that collectively meet the requested types, and returnServer(ids[]) to release them. Propose data structures and indexes for fast matching, availability tracking, fragmentation minimization, and scaling. Address concurrency, starvation, and failure handling.

APPROACH

Map instance type -> ordered set of free nodes; allocating a node removes it from every type's set (lazy deletion with an allocated flag); release re-adds it. Pick by the rule in the prompt.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what getServer returns when it cannot be satisfied, whether it must be atomic, what collectively meet the types means (every requested type is covered by at least one chosen server), and the scale (about 100,000 nodes).

BRUTE FORCE TO SAY OUT LOUD

Scan all nodes on every request and test their types. O(N) per call.

OPTIMAL IDEA

Keep, for each instance type, a heap or sorted set of free node ids ordered by how few types the node supports (least flexible first). To serve, pick for each requested type the least flexible free node not already chosen, then fill the remaining count with least flexible nodes that match any requested type. Mark nodes allocated (lazy deletion in the other types' structures). Roll back if the request cannot be completed. Release re-adds the node to each of its types.

DRY-RUN THESE CASES

Request larger than the free pool, a node shared by two requested types, return then re-request, releasing an id that is not allocated.

LIKELY FOLLOW-UPS

Concurrency (per-type locks or a single allocator thread), starvation (FIFO queue), fragmentation, failure handling (leases and timeouts).

PITFALLS

Allocating a node and forgetting to remove it from the other type indexes. Greedy choices that are not optimal set cover; say so.

HOW TO THINK ABOUT IT

A request mentions a few types, so index the free servers by type and avoid scanning all 100,000. To keep versatile servers available for harder requests, always prefer the server that supports the fewest types. Allocation touches every type index of a server, so use lazy deletion: mark it allocated and let the stale heap entries be skipped when they surface.

Basis: Written design. Assumed contract (confirm it): getServer(count, types) returns `count` free server ids such that every requested type is supported by at least one returned server, or None and no change; releaseServer(ids) frees them. To limit fragmentation the least flexible servers (supporting the fewest types) are used first.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def get_server(count, types):
    chosen = []
    for t in types:
        pool = [s for s in free if t in s.types
                and s not in chosen]
        chosen.append(min(pool, key=flexibility))
    rest = [s for s in free if s not in chosen
            and s.types & set(types)]
    chosen += rest[:count - len(chosen)]
    if len(chosen) < count:
        return None  # not enough
    return chosen
```

TIME	SPACE
O(N * T)	O(N)
per request (N servers)	

WHY IT IS SLOW

BRUTE FORCE

Every request scans all N servers, O(N * types). Per-type heaps ordered by flexibility with lazy deletion make a request O((count + types) log N).

OPTIMAL SOLUTION · Per-type heaps ordered by flexibility, with lazy deletion

TIME	SPACE
O((count + types) log N) amortized per request	**O(total type memberships)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Many clients call at the same time.

Make allocation atomic: one lock around the allocate step, or shard the pool by type or by id range and route requests to shards. The roll-back-on-failure design means a failed request leaves no trace.

FOLLOW-UP 2 Clients keep asking for rare types and starve.

Use a FIFO queue per request class and hold back versatile servers for waiting rare-type requests, or age requests so an old one outranks new ones. Fragmentation control (least flexible first) already reduces the pressure.

FOLLOW-UP 3 Servers fail while allocated.

Track a lease or heartbeat per allocation; when a server stops responding mark it unavailable, remove it from the heaps (lazy deletion handles stale entries), and tell the client so it can request a replacement.

FOLLOW-UP 4 A client dies without releasing its servers.

Give every allocation a lease (expiry time). A periodic sweep returns servers whose lease has expired to the free pool (the lazy-deletion indexes make a release cheap), and clients renew the lease while they are alive.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design server allocation for multi-type
# nodes
#
# Design a service to manage a large pool (about 100, 000)
# of server nodes. Each node can satisfy multiple instance
# types; clients call getServer(reqNumOfServer,
# instanceTypes[]) to request servers that collectively
# meet the requested types, and returnServer(ids[]) to
# release them. Propose data structures and indexes for
# fast matching, availability tracking, fragmentation
# minimization, and scaling. Address concurrency,
# starvation, and failure handling. Overview: This
# question evaluates understanding of scalable resource-
# allocation and matching systems, encompassing data-
# structure and index design, availability tracking,
# fragmentation minimization, concurrency control, and
# failure handling for large node pools.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Many clients call at the same time.
# 2. Clients keep asking for rare types and starve.
# 3. Servers fail while allocated.
# 4. A client dies without releasing its servers.
#
# INTERVIEW NOTES
# A request mentions a few types, so index the free
# servers by type and avoid scanning all 100,000. To keep
# versatile servers available for harder requests, always
# prefer the server that supports the fewest types.
# Allocation touches every type index of a server, so use
# lazy deletion: mark it allocated and let the stale heap
# entries be skipped when they surface.
#
# BRUTE FORCE (say it first): Scan every free server on
# each request
# For each requested type find the best free server by
# scanning all servers, then fill the remaining count the
# same way. Simple and a good oracle; every request
# touches every server. Time O(N * T) per request (N
# servers). Space O(N).


import heapq
from collections import defaultdict
```

```
# BRUTE FORCE (say it first): Scan every free server on
# each request | time O(N * T) per request (N servers)
class ServersBrute:
    def __init__(self, servers):
        self.types = dict(servers)
        self.free = set(servers)

    def getServer(self, count, types):
        chosen = []
        for t in types:
            if any(t in self.types[c] for c in chosen):
                continue
            cands = [
                s
                for s in self.free
                if s not in chosen
                and t in self.types[s]
            ]
            if not cands:
                return None
            chosen.append(
                min(
                    cands,
                    key=lambda s: (
                        len(self.types[s]),
                        s,
                    ),
                )
            )
        extra = sorted(
            (
                s
                for s in self.free
                if s not in chosen
                and self.types[s] & set(types)
            ),
```

```
            key=lambda s: (len(self.types[s]), s),
        )
        if (
            len(chosen) > count
            or len(chosen) + len(extra) < count
        ):
            return None
        chosen += extra[: count - len(chosen)]
```

```
        self.free -= set(chosen)
        return chosen

    def releaseServer(self, ids):
        self.free |= set(ids)


# BASE QUESTION (optimal): Per-type heaps ordered by
# flexibility, with lazy deletion
# Idea: For each type keep a heap of (number of supported
# types, id) for the servers that support it.
# FOLLOW-UP 4: A client dies without releasing its
# servers.
# Answer: Give every allocation a lease (expiry time). A
# periodic sweep returns servers whose lease has expired
# to the free pool (the lazy-deletion indexes make a
# release cheap), and clients renew the lease while they
# are alive.
class ServerPool:
    """Servers with several instance types. Each type has
    a heap of (number of types, id): the LEAST flexible
    server goes first, which keeps versatile ones free."""

    def __init__(self, servers):
        # servers: id -> set of instance types
        self.types = {s: frozenset(t)
                      for s, t in servers.items()}
        self.allocated = set()
        self.heaps = defaultdict(list)
        # FOLLOW-UP 4: a dead client's lease expires
        # lease expiry time of every allocated server
        self.lease = {}
        for s, ts in self.types.items():
            for t in ts:
                heap = self.heaps[t]
                heapq.heappush(heap, (len(ts), s))

    def best_for(self, t, skip):
        # the best free server of type t that is not in
        # `skip`;
        # allocated entries are dropped lazily
        heap, held = self.heaps[t], []
        while heap:
            flex, s = heap[0]
            if s in self.allocated:
```

```
                heapq.heappop(heap)
            elif s in skip:
                held.append(heapq.heappop(heap))
            else:
                break
        best = heap[0] if heap else None
        # put back the entries we only looked past
        for item in held:
            heapq.heappush(heap, item)
        return best

    def get_server(self, count, types):
        # pick servers first, commit only if the whole
        # request works
        chosen = []
        for t in types:
            # a server already chosen may cover this type
            if any(t in self.types[c] for c in chosen):
```

```
                continue
            top = self.best_for(t, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        # fill up to `count` with the least flexible free
        # servers
        while len(chosen) < count:
            skip = set(chosen)
            found = [self.best_for(t, skip)
                     for t in types]
            options = [o for o in found if o]
            if not options:
                return None
            chosen.append(min(options)[1])
        # more servers than allowed were needed to cover
        # the types
        if len(chosen) > count:
            return None
        self.allocated |= set(chosen)
        return chosen

    def release(self, ids):
        for s in ids:
            if s in self.allocated:
                self.allocated.discard(s)
                self.lease.pop(s, None)
```

```
                flex = len(self.types[s])
                for t in self.types[s]:
                    heap = self.heaps[t]
                    heapq.heappush(heap, (flex, s))

    def rent(self, count, types, now, ttl):
        # a client that dies must not hold servers forever
        got = self.get_server(count, types)
        for s in got or []:
            self.lease[s] = now + ttl
        return got

    def expire(self, now):
        # return servers whose lease ran out to the free
        # pool
        late = [s for s in self.lease
                if self.lease[s] <= now]
        self.release(late)
        return late


if __name__ == "__main__":
    pool = ServerPool({1: {"gpu"}, 2: {"gpu", "big"},
                       3: {"big"}, 4: {"cpu"}})
    # the least flexible server for each type is used
    # first
    assert pool.get_server(2, ["gpu", "big"]) == [1, 3]
    # the versatile server 2 is still there for a combined
    # need
    assert pool.get_server(1, ["gpu", "big"]) == [2]
    # nothing is left for a gpu request
    assert pool.get_server(1, ["gpu"]) is None
    # a request that cannot fit in `count` servers changes
    # nothing
    assert pool.get_server(1, ["cpu", "gpu"]) is None
    assert pool.get_server(1, ["cpu"]) == [4]
    pool.release([2])
    assert pool.get_server(1, ["gpu"]) == [2]
    # leases: a dead client's servers come back
    p2 = ServerPool({1: {"a"}})
    assert p2.rent(1, ["a"], now=0, ttl=10) == [1]
    assert p2.get_server(1, ["a"]) is None
    assert p2.expire(now=5) == []
    assert p2.expire(now=10) == [1]
    assert p2.get_server(1, ["a"]) == [1]
```

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Data structure design

MEDIUM | IN APP LC 362 | DESIGN-STYLE (skippable) | HackTheRounds | Phone Screen | 14 of 16

Hit Counter

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 115/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Data structure design

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #362 Design Hit Counter

LeetCode match: LC 362

QUESTION

Design a hit counter that counts the number of hits received in the past 5 minutes (300 seconds). hit(timestamp) records a hit; getHits(timestamp) returns the number of hits in the past 300 seconds.

EXAMPLES

Example 1	
Input	`hit(1), hit(2), hit(3), getHits(4), hit(300), getHits(300), getHits(301)`
Output	`[null, null, null, 3, null, 4, 3]`
Why	`At 301, hit at timestamp 1 expires.`

CONSTRAINTS

- 1 <= timestamp <= 2 * 10^9
- Timestamps are monotonically increasing

APPROACH

Queue/deque of timestamps; evict entries older than 300 seconds

HOW TO THINK ABOUT IT

Only the last 300 seconds matter, so anything older is dead weight: evict from the front of a queue of timestamps. If hits arrive at very high rates, store counts per second in a circular array of 300 buckets so memory does not depend on the number of hits.

Basis: LeetCode 362, Design Hit Counter.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
hits = []
def hit(t):
    hits.append(t)
def get_hits(t):
    return sum(1 for x in hits if x > t - 300)
```

BRUTE FORCE

TIME	SPACE
O(n) per getHits	O(n)

WHY IT IS SLOW

Memory grows with every hit forever, and each query scans them all, O(n). A queue that drops timestamps older than 300 seconds (or 300 per-second buckets) keeps both bounded.

OPTIMAL SOLUTION · Queue of timestamps with eviction

TIME	SPACE
O(1) amortized	O(hits in the window)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Millions of hits per second.

Use 300 buckets (one per second) instead of one entry per hit: hit is O(1), getHits is O(300) = O(1), and memory is constant. A bucket whose stored second is older than the window is reset on reuse.

FOLLOW-UP 2 Timestamps can arrive out of order.

A deque no longer works. Keep a sorted structure (a sorted list with bisect or a balanced tree) and count with two binary searches, or use the buckets, which are order-insensitive within the window.

FOLLOW-UP 3 Thread safety.

Guard the queue with a lock (or use per-thread counters merged on read). The bucket version needs the compare-then-reset of a slot to be atomic. **Cost:** one lock per call, O(1) extra; contention rather than complexity is the concern.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Hit Counter
#
# Design a hit counter that counts the number of hits
# received in the past 5 minutes (300 seconds).
# hit(timestamp) records a hit; getHits(timestamp) returns
# the number of hits in the past 300 seconds.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Millions of hits per second.
# 2. Timestamps can arrive out of order.
# 3. Thread safety.
```

```
#
# INTERVIEW NOTES
# Only the last 300 seconds matter, so anything older is
# dead weight: evict from the front of a queue of
# timestamps. If hits arrive at very high rates, store
# counts per second in a circular array of 300 buckets so
# memory does not depend on the number of hits.
#
# BRUTE FORCE (say it first): Store every hit and count in
# the window
# Append every timestamp to a list and count those with
# timestamp > now - 300. Correct, but memory grows forever
# and each query scans everything. Time O(n) per getHits.
# Space O(n).


import threading
from bisect import bisect_right, insort
from collections import deque


# HELPERS for the brute force
WINDOW = 300   # seconds


def brute(hits, ts):
    return sum(1 for t in hits if ts - WINDOW < t <= ts)


# BASE QUESTION (optimal): Queue of timestamps with
# eviction
# Idea: Keep a deque of timestamps in arrival order.
# getHits pops from the front while the oldest is outside
# the window, and the answer is the deque length. Each
```

```
# timestamp is pushed and popped once.
class QueueCounter:
    """One entry per hit; old entries are dropped when
    asked. Timestamps must not go backwards."""

    def __init__(self):
        self.hits = deque()

    def hit(self, ts):
        self.hits.append(ts)

    def get_hits(self, ts):
        # keep only the hits in (ts - 300, ts]
        while self.hits and self.hits[0] <= ts - WINDOW:
            self.hits.popleft()
        return len(self.hits)


# FOLLOW-UP 1: Millions of hits per second.
# Answer: Use 300 buckets (one per second) instead of one
# entry per hit: hit is O(1), getHits is O(300) = O(1),
# and memory is constant. A bucket whose stored second is
# older than the window is reset on reuse.
class BucketCounter:
    """Follow-up: millions of hits per second. One bucket
    per second of the window: constant memory and time."""

    def __init__(self):
        self.stamp = [0] * WINDOW
        self.count = [0] * WINDOW

    def hit(self, ts):
        slot = ts % WINDOW
        if self.stamp[slot] != ts:
            # the slot still holds an old second: reuse it
            self.stamp[slot], self.count[slot] = ts, 0
        self.count[slot] += 1
```

```
    def get_hits(self, ts):
        slots = zip(self.stamp, self.count)
        return sum(c for t, c in slots
                   if 0 <= ts - t < WINDOW)


# FOLLOW-UP 2: Timestamps can arrive out of order.
```

```
# Answer: A deque no longer works. Keep a sorted structure
# (a sorted list with bisect or a balanced tree) and count
# with two binary searches, or use the buckets, which are
# order-insensitive within the window.
class SortedCounter:
    """Follow-up: timestamps can arrive out of order.
    A sorted list and two binary searches count the
    window."""

    def __init__(self):
        self.hits = []

    def hit(self, ts):
        insort(self.hits, ts)

    def get_hits(self, ts):
        newest = bisect_right(self.hits, ts)
        oldest = bisect_right(self.hits, ts - WINDOW)
        return newest - oldest


# FOLLOW-UP 3: Thread safety.
# Answer: Guard the queue with a lock (or use per-thread
# counters merged on read). The bucket version needs the
# compare-then-reset of a slot to be atomic. Cost: one
# lock per call, O(1) extra; contention rather than
# complexity is the concern.
class Locked:
    """Follow-up: thread safety. One lock around every
    call keeps check-then-update steps atomic."""

    def __init__(self, counter):
        self.counter = counter
        self.lock = threading.Lock()

    def hit(self, ts):
        with self.lock:
            self.counter.hit(ts)

    def get_hits(self, ts):
        with self.lock:
            return self.counter.get_hits(ts)


if __name__ == "__main__":
```

```
    kinds = (QueueCounter, BucketCounter, SortedCounter)
    for make in kinds:
        c = make()
        for ts in (1, 2, 3, 300):
            c.hit(ts)
        assert c.get_hits(300) == 4
        # the hit at second 1 is 300 seconds old: it is
        # out
        assert c.get_hits(301) == 3
        assert c.get_hits(599) == 1
        assert c.get_hits(600) == 0
        assert c.get_hits(1000) == 0
    import random
    for make in kinds:
        c, log, now = make(), [], 0
        for _ in range(500):
            now += random.randint(0, 40)
            if random.random() < 0.6:
                c.hit(now)
```

```
            log.append(now)
        else:
            assert c.get_hits(now) == brute(log, now)
# out of order input only works with the sorted
# version
late = SortedCounter()
for ts in (50, 10, 400, 30):
    late.hit(ts)
assert late.get_hits(400) == 1
assert late.get_hits(310) == 2
shared = Locked(BucketCounter())
def spam():
    for _ in range(1000):
        shared.hit(7)

workers = [threading.Thread(target=spam)
           for _ in range(4)]
for w in workers:
    w.start()
for w in workers:
    w.join()
assert shared.get_hits(7) == 4000
```

ALSO REPORTED · 1 more

Design a rolling hit counter API | PracHub | Technical Screen

https://prachub.com/coding-questions/design-a-rolling-hit-counter-api

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Streaming and scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 15 of 16

Find top/bottom-k words in list or stream

POSTED 2026-01-11 | 282 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 32.6 | 30 DAYS 0 | PRIORITY 117/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Streaming and scale

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #692 Top K Frequent Words

LeetCode match: LC 692 Top K Frequent Words (+ stream variant)

Posted on PracHub: 2026-01-11 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/find-top-bottom-k-words-in-list-or-stream

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.
Constraints
0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated

QUESTION

EXAMPLES

```
Input: (['i', 'love', 'leetcode', 'i', 'love',
    'coding'], 2)
Expected Output: [['i', 'love'], ['coding',
    'leetcode']]
Explanation: `i` and `love` both appear twice, so
    lexicographical order gives `['i', 'love']`.
    `coding` and `leetcode` both appear once, so
    lexicographical order gives `['coding',
    'leetcode']`.
Input: (['apple', 'banana', 'apple', 'cherry',
    'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'],
    ['cherry', 'date', 'apple']]
```

Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.

Hints
Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).
Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.
Constraints
1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested
Examples
Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]
Expected Output: [['apple', 'banana'], ['banana', 'apple']]
Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.

Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]
Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]
Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.
Hints
Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

APPROACH

List: counts + heaps (min-heap size k for top, max-heap size k for bottom). Stream: keep counts in a map and maintain heaps with lazy updates, or approximate with Space-Saving / Count-Min when memory is bounded.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how ties break, whether top and bottom are over distinct words, what topK(k) returns for k larger than the number of words, and whether memory is bounded for the stream.

BRUTE FORCE TO SAY OUT LOUD

Count words, sort all distinct words by (count, word), take the first and last K. O(d log d).

OPTIMAL IDEA

List version: Counter, then heapq.nlargest and nsmallest with the key (count, word) in the right directions, O(n + d log K). Stream version: ingest increments a dict in O(1); topK and bottomK compute on demand with a heap in O(d log K). If queries are frequent, keep words grouped by count in buckets (a linked list of count buckets like an LFU cache) so topK walks from the highest bucket in O(K). If memory is bounded, the Space-Saving or Count-Min sketch approximates the top K; bottom-K cannot be approximated well, so say so.

DRY-RUN THESE CASES

Fewer words than K, all counts equal (ties decide everything), one word repeated, an empty stream.

LIKELY FOLLOW-UPS

Windowed counts (expire old words), or distributed counting with a merge step.

PITFALLS

Mixing up the direction for the bottom-K heap. Missing a deterministic tie-break.

HOW TO THINK ABOUT IT

Count first; then you only need the K best and K worst, which a heap does without sorting everything. For a stream, ingesting is just a counter increment, and the answer is computed from the counts on demand. If memory is bounded, only heavy hitters (top-K) can be approximated; the rarest words cannot.

Basis: LeetCode 692 (Top K Frequent Words) and its bottom-K counterpart, plus a streaming interface. Ties break alphabetically.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(words, k):
    counts = Counter(words)
    def rank(w):
        return (-counts[w], w)
    ranked = sorted(counts, key=rank)
    most = ranked[:k]
    least = ranked[::-1][:k]
    return most, least
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct words, O(d log d). Two heaps of size K are O(d log k); a stream only needs the counts, and a Space-Saving sketch bounds memory for the top K.

OPTIMAL SOLUTION · Counter plus two heaps of size K

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The stream is too big to count every distinct word.

Use the Space-Saving (or Misra-Gries) summary with m counters: a word that appears more than n / m times is guaranteed to be tracked, and its stored count is an upper bound. This works for the top K only; the least frequent words cannot be found this way.

FOLLOW-UP 2 topK must be O(k), not O(d log k), because it is called all the time.

Keep words in buckets by count (a linked list of count buckets, like the LFU cache structure); the most frequent words are in the last bucket. An increment moves a word one bucket up in O(1).

FOLLOW-UP 3 Case and punctuation.

Normalise before counting (lowercase, strip punctuation) and say so; the counting and ranking code is unchanged.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find top/bottom-k words in list or stream
#
# You are given words (strings) either as a finite list or
# as an unbounded stream. 1) **List version**: Given an
```

```
# array words and an integer k, return: - the k most
# frequent distinct words, and - the k least frequent
# distinct words. Specify how you break ties (e.g.,
# lexicographically, or any order). 2) **Stream version
# (online)**: Design a data structure that supports: -
# ingest(word): process the next word in the stream -
# topK(k): return the k most frequent distinct words seen
# so far - bottomK(k): return the k least frequent
# distinct words seen so far Discuss time/space complexity
# and how you would implement these operations
# efficiently. 3) **Variant (discuss/design)**: Extend the
# stream design to support returning the **first k**
# and/or **last k** words that are currently **non-
# repeating** (i.e., words with frequency exactly 1)
# according to stream arrival order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The stream is too big to count every distinct word.
# 2. topK must be O(k), not O(d log k), because it is
# called all the time.
# 3. Case and punctuation.
#
# INTERVIEW NOTES
# Count first; then you only need the K best and K worst,
# which a heap does without sorting everything. For a
# stream, ingesting is just a counter increment, and the
# answer is computed from the counts on demand. If memory
# is bounded, only heavy hitters (top-K) can be
# approximated; the rarest words cannot.
#
# BRUTE FORCE (say it first): Sort all distinct words by
# (count, word)
# Count, sort the distinct words by count and then
# alphabetically, and take the first or last K. Correct
# and the oracle. Time O(d log d). Space O(d).


import heapq
from collections import Counter, defaultdict
```

```
# BRUTE FORCE (say it first): Sort all distinct words by
# (count, word) | time O(d log d)
def top_bottom_sort(words, k):
    c = Counter(words)
    ordered = sorted(c, key=lambda w: (-c[w], w))
    least = sorted(c, key=lambda w: (c[w], w))
    return ordered[:k], least[:k]


# BASE QUESTION (optimal): Counter plus two heaps of size
# K
# Idea: One pass counts the words. heapq.nsmallest with
# key (-count, word) gives the K most frequent and with
# key (count, word) the K least frequent, each in O(d log
# k) without a full sort.
def top_bottom(words, k):
    # ties are broken alphabetically
    c = Counter(words)
    top = heapq.nsmallest(k, c, key=lambda w: (-c[w], w))
    low = heapq.nsmallest(k, c, key=lambda w: (c[w], w))
    bottom = low
    return top, bottom


# FOLLOW-UP 1: The stream is too big to count every
# distinct word.
# Answer: Use the Space-Saving (or Misra-Gries) summary
# with m counters: a word that appears more than n / m
# times is guaranteed to be tracked, and its stored count
# is an upper bound.
class SpaceSaving:
    """Too many distinct words: m counters. A word seen
```

```
    over n / m times is always tracked."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, word):
        if word in self.counts:
            self.counts[word] += 1
        elif len(self.counts) < self.m:
            self.counts[word] = 1
        else:
            # replace the smallest counter; the new word
            # inherits its count + 1 (an upper bound)
```

```
            low = min(self.counts, key=self.counts.get)
            self.counts[word] = self.counts.pop(low) + 1

    def top_k(self, k):
        return sorted(self.counts, key=lambda w: (
            -self.counts[w], w))[:k]


# FOLLOW-UP 2: topK must be O(k), not O(d log k), because
# it is called all the time.
# Answer: Keep words in buckets by count (a linked list of
# count buckets, like the LFU cache structure); the most
# frequent words are in the last bucket. An increment
# moves a word one bucket up in O(1).
class FreqBuckets:
    """Stream: add(word) is O(1); top_k reads only the
    highest count buckets."""

    def __init__(self):
        self.count = defaultdict(int)
        # count -> words with exactly that count
        self.bucket = defaultdict(dict)
        self.high = 0

    def add(self, word):
        c = self.count[word]
        if c:
            # move the word up one bucket
            del self.bucket[c][word]
            if not self.bucket[c]:
                del self.bucket[c]
        self.count[word] = c + 1
        self.bucket[c + 1][word] = True
        self.high = max(self.high, c + 1)

    def top_k(self, k):
        out = []
        level = self.high
        while level > 0 and len(out) < k:
            # words inside one bucket: alphabetical
            out += sorted(self.bucket.get(level, ()))
            level -= 1
        return out[:k]

    def bottom_k(self, k):
```

```
        out = []
        for level in sorted(self.bucket):
            out += sorted(self.bucket[level])
            if len(out) >= k:
                break
        return out[:k]


# FOLLOW-UP 3: Case and punctuation.
# Answer: Normalise before counting (lowercase, strip
# punctuation) and say so; the counting and ranking code
# is unchanged.
def normalise(text):
```

```
    # case and punctuation: clean before counting
    letters = (ch for ch in text.lower() if ch.isalnum())
    return "".join(letters)


if __name__ == "__main__":
    words = "a b a c b a d d".split()
    top, bottom = top_bottom(words, 2)
    assert top == ["a", "b"] and bottom == ["c", "b"]
    f = FreqBuckets()
    for w in words:
        f.add(w)
    assert f.top_k(2) == ["a", "b"]
    assert f.bottom_k(2) == ["c", "b"]
    assert f.top_k(10) == ["a", "b", "d", "c"]
    s = SpaceSaving(2)
    for w in "x x x y z x x":
        s.add(w) if w.strip() else None
    assert s.top_k(1) == ["x"]
    assert normalise("Hello, World!") == "helloworld"
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], k: int) -> list[list[str]]:
```

Sample call

```
solution(['i', 'love', 'leetcode', 'i', 'love', 'coding'],
 2)
```

Test cases

Input	Expected
`(['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)`	`[['i', 'love'], ['coding', 'leetcode']]`
`(['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)`	`[['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]`

STUDY HERE: topic 5, Design, Streaming and Scale | main topic: Streaming and scale

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 16 of 16

Design an elevator control system

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 221/228

ABOUT THIS QUESTION

Topic: Design, Streaming and Scale: Streaming and scale

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (OOP / low-level design)

Posted on PracHub: 2025-09-06 (updated 2026-10-07)

Original: https://prachub.com/interview-questions/design-an-elevator-control-system-2

QUESTION

Design the software for a multi-elevator control system in a high-rise building. Specify classes, scheduling algorithms for car assignments, handling peak traffic, fairness, fault tolerance, and safety constraints. Consider real-time constraints and a simulation and testing strategy.

APPROACH

Model Elevator, Request and Controller classes; scheduling by nearest-car or SCAN/LOOK direction-based assignment; queue targets per elevator, handle peak traffic, fairness, fault tolerance and safety; discuss simulation and testing strategy.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how many elevators and floors, whether to design only the control logic or also the hardware, request types (hall call with direction, destination inside), and what to optimize (wait time, fairness, energy).

BRUTE FORCE TO SAY OUT LOUD

A single queue: serve requests first come first served with the nearest idle car. Easy but wasteful.

OPTIMAL IDEA

Classes: Request, Elevator (floor, direction, state, a set of stops), Controller (assigns requests). Use SCAN/LOOK inside each car: keep moving in the current direction serving stops, then reverse. Assign a hall call to the car with the lowest cost, such as distance plus a penalty if it is moving away or is full. Add a step method that advances time so it can be simulated and tested.

DRY-RUN THESE CASES

One request while idle, a request in the opposite direction, two cars equally close, a car out of service with pending stops, the top and bottom floors.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Peak traffic modes, weight limits, fault tolerance (reassign stops on failure), and safety (doors, overload).

PITFALLS

Putting all logic in one class. Not saying how you will test it.

HOW TO THINK ABOUT IT

Separate three things: the data (Request), one car's behaviour (Elevator: floor, direction, stops, door/state) and the policy that decides which car takes a call (Controller). Say the scheduling rule out loud: a car keeps going in its direction serving stops, then reverses (SCAN/LOOK), which avoids the back-and-forth of first-come-first-served. Add a step() so the whole thing can be simulated and tested.

Basis: Written object-oriented design (the classic low-level design question). A SCAN (elevator) scheduling policy per car with a cost-based assignment of hall calls.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
requests = deque()  # arrival order
def run():
    while requests:
        req = requests.popleft()
        car = nearest_idle_car(req.floor)
        car.go_to(req.floor)
        car.go_to(req.destination)
```

TIME	SPACE
O(requests * cars)	O(requests)

WHY IT IS SLOW

Cars zig-zag across the building and requests wait behind far-away ones, so total travel and waiting are high. Serving stops in sweep order (SCAN) and assigning by cost cuts both.

OPTIMAL SOLUTION · Elevator (SCAN) per car and cost-based assignment of calls

TIME	SPACE
O(cars) per call, O(1) per car per step	O(stops)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Peak traffic (morning lobby rush, evening exodus).

Add modes: during the morning rush park idle cars in the lobby and let them go up with a full load; in the evening do the opposite. Zone cars (low floors, high floors) or use destination dispatch (passengers enter their floor first) to group riders by destination.

FOLLOW-UP 2 Fairness and starvation.

SCAN already guarantees every stop is reached within one sweep. For hall calls add an aging term to the cost so a long-waiting call wins over a closer new one.

FOLLOW-UP 3 Safety and fault tolerance.

Keep door and overload as explicit states; a car that fails is marked out of service and its stops are reassigned (the controller does this), and the controller should be restartable from persistent state of cars and pending calls.

FOLLOW-UP 4 Morning rush: idle cars should wait at the lobby.

When a car has no stops, give it a stop at the lobby (or a configured parking floor per car). Peak modes are just policies layered on the same step() and cost-based assignment; the elevator algorithm does not change.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design an elevator control system
#
# Design the software for a multi-elevator control system
# in a high-rise building. Specify classes, scheduling
# algorithms for car assignments, handling peak traffic,
# fairness, fault tolerance, and safety constraints.
# Consider real-time constraints and a simulation and
# testing strategy.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Peak traffic (morning lobby rush, evening exodus).
# 2. Fairness and starvation.
# 3. Safety and fault tolerance.
# 4. Morning rush: idle cars should wait at the lobby.
#
# INTERVIEW NOTES
# Separate three things: the data (Request), one car's
# behaviour (Elevator: floor, direction, stops,
# door/state) and the policy that decides which car takes
# a call (Controller). Say the scheduling rule out loud: a
# car keeps going in its direction serving stops, then
# reverses (SCAN/LOOK), which avoids the back-and-forth of
# first-come-first-served. Add a step() so the whole thing
# can be simulated and tested.
#
# BRUTE FORCE (say it first): First-come-first-served on
# the nearest idle car

# Serve requests in arrival order, each by whichever car
# is closest at that moment, visiting stops in the order
# they were requested. Easy to explain and fair, but cars
# zig-zag across the building. Time O(requests * cars).
# Space O(requests).


# BRUTE FORCE (say it first): First-come-first-served on
# the nearest idle car | time O(requests * cars)
def fcfs_distance(stops, start=0):
    pos, total = start, 0
    for s in stops:
        total += abs(s - pos)
        pos = s
    return total


# HELPERS for the base question
```

```
INF = float("inf")


class Request:
    def __init__(self, floor, direction):
        # direction: +1 up, -1 down
        self.floor, self.direction = floor, direction


# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
class Elevator:
    def __init__(self, car_id, floor=0):
        self.id, self.floor = car_id, floor
        # 0 = idle, +1 = going up, -1 = going down
        self.direction = 0
        self.stops = set()
        self.in_service = True

    def cost(self, req):
        # how bad is it to give this car the request?
        if not self.in_service:
            return INF
        dist = abs(self.floor - req.floor)
        if self.direction == 0:
            return dist
        # on the way: the car is moving toward the floor
        # and
        # the call goes the same way
        gap = req.floor - self.floor
        ahead = gap * self.direction >= 0
        if ahead and req.direction == self.direction:
            return dist
        # otherwise it finishes its sweep first
        span = max(self.stops) - min(self.stops) + 1
        return dist + 2 * span + 10

    def step(self):
        # one tick: serve a stop here, then move one floor
        served = None
        if self.floor in self.stops:
```

```
            self.stops.discard(self.floor)
            served = self.floor
        if not self.stops:
            self.direction = 0
            return served
        up = [s for s in self.stops if s > self.floor]
        down = [s for s in self.stops if s < self.floor]
        # keep the direction while there are stops ahead
```

```
        # (SCAN)
        if (self.direction >= 0 and up) or (
                self.direction < 0 and not down):
            self.direction = 1
        else:
            self.direction = -1
        self.floor += self.direction
        return served


# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
# FOLLOW-UP 1: Peak traffic (morning lobby rush, evening
# exodus).
# Answer: Add modes: during the morning rush park idle
# cars in the lobby and let them go up with a full load;
# in the evening do the opposite.
class Controller:
    def __init__(self, cars, lobby=0):
        self.cars, self.lobby = cars, lobby

    def assign(self, req):
        # the cheapest in-service car takes the call
        car = min(self.cars, key=lambda c: c.cost(req))
        if car.cost(req) == INF:
            return None
        car.stops.add(req.floor)
        return car.id

    def tick(self):
        return {c.id: c.step() for c in self.cars}

    # FOLLOW-UP 3: a failed car hands its stops over
```

```
    def fail(self, car_id):
        # a broken car: take it out and hand its stops to
        # others
        car = next(c for c in self.cars if c.id == car_id)
        car.in_service = False
        orphans, car.stops = car.stops, set()
        for floor in orphans:
            self.assign(Request(floor, 0))

    # FOLLOW-UP 1, 4: morning rush: idle cars wait in the
    # lobby
    def park_idle(self):
        # morning rush: idle cars wait in the lobby
        for car in self.cars:
            if car.in_service and not car.stops:
                car.stops.add(self.lobby)


if __name__ == "__main__":
    a, b = Elevator("A", 0), Elevator("B", 10)
    ctl = Controller([a, b])
    # the closest idle car takes the call
    assert ctl.assign(Request(2, 1)) == "A"
    assert ctl.assign(Request(9, -1)) == "B"
    for _ in range(3):
        served = ctl.tick()
    assert a.floor == 2 and served["A"] in (None, 2)
    assert 2 not in a.stops
    # a failed car hands its stops to the other car
    b.stops = {8}
    ctl.fail("B")
    assert not b.in_service and 8 in a.stops
    assert ctl.assign(Request(1, 1)) == "A"
    # nobody in service: the request cannot be taken
    a.in_service = False
```

```
    assert ctl.assign(Request(1, 1)) is None
    # idle cars go back to the lobby
    c = Elevator("C", 5)
    Controller([c]).park_idle()
    assert c.stops == {0}
```

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 6 OF 10 | CORE TOPIC

Dynamic Programming

15 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

12	3
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Dynamic Programming | 15 questions | tap a title to jump to it

M	1. Bounded Knapsack Variant	1Point3Acres	2026-10-16
M	2. Longest Non-Decreasing Subarray with One Replacement	1Point3Acres	2026-10-04
M	3. Grid Path Count with Diagonal Moves and Checkpoints	1Point3Acres	2026-09-29
M	4. Minimum-Cost Tree Disconnection	1Point3Acres	2026-09-09
M	5. Target Expressions from Ordered Digits	1Point3Acres	2026-08-13
M	6. Count Word Decompositions into Chemical Element Symbols	1Point3Acres	2026-07-14
M	7. Fence Painting Minimum Operations (Gemini Team)	1Point3Acres	2026-04-27
M	8. Maximize sum without choosing adjacent elements	PracHub	2026-01-06
M	9. Jump Game with Profit	1Point3Acres	no date
M	10. Count Same-Color Squares in Matrix	1Point3Acres	no date
M	11. Russian Doll Envelopes	1Point3Acres	no date
M	12. Reverse Coin Change	HackTheRounds	no date
H	13. Count Perfect Wake Numbers up to N	1Point3Acres	2026-05-07
H	14. Tree Distance Sum (Re-root DP)	1Point3Acres	2026-01-19
H	15. Determine winner in stack-merging game	PracHub	2026-01-06

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: greedy, dp, quickselect, sorting, medium

MEDIUM | 1Point3Acres | tech-screen | 1 of 15

Bounded Knapsack Variant

ASKED 2026-10-16 | 4 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 180/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1710 Maximum Units on a Truck

Frequency: Single report | Last asked: 2026-10-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/unit-weight-bounded-knapsack

QUESTION

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n).

Input: for each item type, its value and the number of copies available. Every item has weight 1.
Input: a maximum total weight W.
Output: the maximum total value achievable without exceeding W.

APPROACH

Write the sort-based greedy and argue its optimality in two sentences (unit weights mean every slot is interchangeable). Implement quickselect that partitions item types by value and tracks the total count on each side to locate where W runs out. Compare the O(n × W), O(n log n), and O(n) versions out loud with their complexities.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether every item really weighs 1, whether values can be zero or negative (assume non-negative), whether W can exceed the total number of items, and how large the counts and W can get.

BRUTE FORCE TO SAY OUT LOUD

A knapsack table over the capacity, one pass per copy. O(total copies * W), correct for any weights.

OPTIMAL IDEA

With unit weights every slot is interchangeable, so sort the item types by value, highest first, and take min(copies, slots left) of each until the bag is full. O(n log n). To reach O(n), run quickselect on the value: split the types into higher, equal and lower, recurse into the side that contains the W-th slot, and keep a running total of what was taken.

DRY-RUN THESE CASES

W = 0, W larger than all copies together, one type, equal values, a type with a huge count, all values equal.

LIKELY FOLLOW-UPS

Different weights (bounded knapsack with binary splitting), a huge W (greedy still works), or negative values (never take them).

PITFALLS

Reaching for DP because the word knapsack appears. Forgetting that a type may only be partly taken. Letting the remaining capacity go negative.

HOW TO THINK ABOUT IT

The word knapsack points at DP, but the statement says every item weighs 1. With equal weights every slot in the bag is interchangeable, so an item that is worth more is never worse than one that is worth less: swapping a cheaper item for a dearer one always helps. Take item types by value, highest first, until W slots are used. You do not even need the full sort: quickselect finds where the W-th slot falls in O(n).

Basis: LeetCode 1710, Maximum Units on a Truck: every item weighs 1, so the knapsack collapses to 'take the most valuable items first'.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def knapsack(items, W):
    best = [0] * (W + 1)
    for value, count in items:
        for _ in range(count):
            for w in range(W, 0, -1):
                take = best[w - 1] + value
                best[w] = max(best[w], take)
    return best[W]
```

TIME	SPACE
O(total copies * W)	O(W)

WHY IT IS SLOW

Every copy is its own pass over the table, O(total copies * W). With unit weights the most valuable items are always the right ones, so sorting by value and taking greedily is O(n log n).

OPTIMAL SOLUTION · Sort by value and take greedily

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is greedy correct here but not for a general knapsack?

With unit weights each chosen item uses exactly one slot, so the only thing that matters is value per slot, which is just the value. If an optimal bag held a cheaper item while a dearer one was available, swapping them keeps the weight and raises the value, so the optimal bag can always be rearranged into the greedy one. With different weights a cheap item can leave room for several others, so value per weight and the leftover space interact and greedy fails.

FOLLOW-UP 2 What if the weights are not all 1?

Then it is a real bounded knapsack and greedy no longer works; use DP over the capacity. Splitting each count into 1, 2, 4, ... copies keeps the number of 0/1 items at about log(count) per type, so the cost is O(n * log(max count) * W) instead of O(total copies * W).

FOLLOW-UP 3 W is up to 10^9. Which version still works?

Only the greedy ones: sorting is O(n log n) and quickselect O(n), and neither depends on W, while any table over the capacity needs O(W) memory. The loop simply stops when the remaining capacity reaches zero.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Bounded Knapsack Variant
#
# Each item type has a value and an available count; every
# item weighs 1. Given capacity W, maximize total value.
# DP or DFS + memo works at O(n·W), greedy by value after
# sorting is O(n log n), and the best answer uses
# quickselect for O(n). Input: for each item type, its
# value and the number of copies available. Every item has
# weight 1. Input: a maximum total weight W. Output: the
# maximum total value achievable without exceeding W.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is greedy correct here but not for a general
# knapsack?
# 2. What if the weights are not all 1?
# 3. W is up to 10^9. Which version still works?
#
# INTERVIEW NOTES
# The word knapsack points at DP, but the statement says
# every item weighs 1. With equal weights every slot in
# the bag is interchangeable, so an item that is worth
# more is never worse than one that is worth less:
# swapping a cheaper item for a dearer one always helps.
# Take item types by value, highest first, until W slots
# are used. You do not even need the full sort:
# quickselect finds where the W-th slot falls in O(n).
#
# BRUTE FORCE (say it first): DP over the capacity
# Treat every copy as its own 0/1 item and run the classic
# 1D knapsack table. It is correct for any weights, so it
# is the oracle, but it is far too slow when the counts or
# W are large. Time O(total copies * W). Space O(W).


import random


# BRUTE FORCE (say it first, it also checks the answers)
def knapsack_dp(items, W):
    # the oracle: every copy is its own 0/1 item
    best = [0] * (W + 1)
    for value, count in items:
        for _ in range(count):
            for w in range(W, 0, -1):
                take = best[w - 1] + value
                best[w] = max(best[w], take)
    return best[W]


# BASE QUESTION (optimal): Sort by value and take greedily
# Idea: Sort the item types by value, highest first. For
# each type take min(copies, slots left) and add value
# times that. Stop when the bag is full.
def max_value_sorted(items, W):
    """Unit weights: every slot is worth the same, so the
    best value always goes in first."""
    total = 0
    for value, count in sorted(items, reverse=True):
        # take as many copies as still fit in the bag
        take = min(count, W)
        total += value * take
        W -= take
        if W == 0:
            break
    return total


# SAME answer in O(n): quickselect
def max_value_select(items, W):
    """Same answer in O(n) expected: quickselect finds
    where the W-th slot falls without a full sort."""
    total, pool = 0, list(items)
    while pool and W > 0:
        pivot = random.choice(pool)[0]
        high = [it for it in pool if it[0] > pivot]
        mid = [it for it in pool if it[0] == pivot]
        low = [it for it in pool if it[0] < pivot]
        high_count = sum(c for _, c in high)
        if high_count >= W:
            # the bag fills up inside the dearer types
            pool = high
            continue
        # every dearer copy fits, then fill from the pivot
        total += sum(v * c for v, c in high)
        W -= high_count
        take = min(W, sum(c for _, c in mid))
        total += pivot * take
        W -= take
        pool = low
    return total
```

```
# FOLLOW-UP 2: What if the weights are not all 1?
# Answer: Then it is a real bounded knapsack and greedy no
# longer works; use DP over the capacity. Splitting each
# count into 1, 2, 4, ...
def knapsack_bounded(items, W):
    """Weights differ: greedy fails, so use DP. Splitting
    each count into 1, 2, 4, ... copies keeps it small."""
    best = [0] * (W + 1)
    for value, weight, count in items:
        chunk = 1
        while count > 0:
            take = min(chunk, count)
            count -= take
            cost, gain = take * weight, take * value
            for w in range(W, cost - 1, -1):
                best[w] = max(best[w],
                              best[w - cost] + gain)
            chunk *= 2
    return best[W]


if __name__ == "__main__":
    demo = [(3, 2), (5, 1), (4, 4)]
    assert max_value_sorted(demo, 4) == 17
    assert max_value_select(demo, 4) == 17
    assert max_value_sorted([], 5) == 0
    assert max_value_sorted([(7, 3)], 0) == 0
    assert max_value_sorted([(7, 3)], 10) == 21
    mixed = [(10, 5, 1), (6, 3, 2)]
    assert knapsack_bounded(mixed, 6) == 12
    # greedy is only valid when every weight is 1
    unit = [(3, 1, 2), (5, 1, 1), (4, 1, 4)]
    assert knapsack_bounded(unit, 4) == 17
    for _ in range(300):
        n = random.randint(0, 6)
        rnd = random.randint
        items = [(rnd(1, 9), rnd(1, 4)) for _ in range(n)]
        W = random.randint(0, 12)
        want = knapsack_dp(items, W)
        assert max_value_sorted(items, W) == want
        assert max_value_select(items, W) == want
```

NOTES FROM CANDIDATES

Pitfalls: Recognizing "knapsack" and reaching for DP is the trap; with unit weights the greedy argument is simpler and faster.

PREPARATION

Write the sort-based greedy and argue its optimality in two sentences (unit weights mean every slot is interchangeable). Implement quickselect that partitions item types by value and tracks the total count on each side to locate where W runs out. Compare the O(n × W), O(n log n), and O(n) versions out loud with their complexities.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: array, dp, two-pointer, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 2 of 15

Longest Non-Decreasing Subarray with One Replacement

ASKED 2026-10-04 | 16 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 209/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-non-decreasing-subarray-one-replacement

QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

Input: an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

Variant: other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

EXAMPLES

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of
    length 5.
```

APPROACH

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

HOW TO THINK ABOUT IT

Precompute left[i], the length of the non-decreasing run ending at i, and right[i], the length of the run starting at i. If you change element i to any value, it can join the run on its left and the run on its right when a[i-1] <= a[i+1] (a value between them exists); otherwise it can join only the longer of the two. Take the best over all i.

Basis: Written for this question (the one-change version of LeetCode 674, Longest Continuous Increasing Subsequence).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    best = longest_non_decreasing(a)
    for i in range(len(a)):
        for v in candidate_values(a):
            b = a[:i] + [v] + a[i + 1:]
            run = longest_non_decreasing(b)
            best = max(best, run)
    return best
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^3). The run lengths on both sides of each position, computed once, answer every change in O(1).

OPTIMAL SOLUTION · Left and right run lengths

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The run must be strictly increasing.

Use < in the run lengths, and when the replaced value must fit between the neighbours check a[i+1] - a[i-1] >= 2 (an integer strictly between them exists) instead of a[i-1] <= a[i+1]. Clarify strict versus non-strict before coding.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Longest Non-Decreasing Subarray with One
# Replacement
#
# Find the maximum length of a contiguous non-decreasing
# subarray if at most one element may be changed to any
# value. Some rounds start from the plain longest-
# increasing-subarray question and add the one-replacement
# follow-up; those versions ask for an increasing run, so
# clarify strict vs non-strict. Input: an integer array
# nums. You may change at most one element to any value.
# Return the maximum length of a contiguous subarray that
# is non-decreasing (nums[i] <= nums[i + 1] for every
# adjacent pair inside it) after the change. Variant:
# other rounds ask for a strictly increasing run and open
# with the no-replacement version (longest increasing
# subarray) before adding the one-replacement follow-up —
# clarify strict vs non-strict before coding. Expect to
# discuss and compare the complexity of several
# approaches; a brute-force first pass is usually followed
# by a request to optimize.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The run must be strictly increasing.
#
# INTERVIEW NOTES
# Precompute left[i], the length of the non-decreasing run
# ending at i, and right[i], the length of the run
# starting at i. If you change element i to any value, it
# can join the run on its left and the run on its right
# when a[i-1] <= a[i+1] (a value between them exists);
# otherwise it can join only the longer of the two. Take
# the best over all i.
#
# BRUTE FORCE (say it first): Try every change and re-scan
# For every position and every candidate value, rewrite
# that element and find the longest non-decreasing run.
# Correct and a good oracle; cubic. Time O(n^3). Space
# O(n).


# BRUTE FORCE (say it first): Try every change and re-scan
# | time O(n^3)
def one_change_brute(a):
    best = 0
    cands = set(a) | {0}
    for i in range(len(a)):
        for v in cands | {a[i]}:
            b = a[:i] + [v] + a[i + 1 :]
            run = 0
            for k in range(len(b)):
                run = (
                    run + 1
                    if k and b[k - 1] <= b[k]
                    else 1
                )
                best = max(best, run)
    return best


# HELPERS for the base question
def run_lengths(a, ok):
    n = len(a)
    # left[i]: run ending at i, right[i]: starting at i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if ok(a[i - 1], a[i]):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ok(a[i], a[i + 1]):
            right[i] = right[i + 1] + 1
    return left, right


# BASE QUESTION (optimal): Left and right run lengths
# Idea: left[i] and right[i] are the non-decreasing run
# lengths ending and starting at i. Changing a[i] gives
# left[i-1] + 1 + right[i+1] if a[i-1] <= a[i+1],
# otherwise max(left[i-1], right[i+1]) + 1.
# FOLLOW-UP 1: The run must be strictly increasing.
# Answer: Use < in the run lengths, and when the replaced
# value must fit between the neighbours check a[i+1] -
# a[i-1] >= 2 (an integer strictly between them exists)
# instead of a[i-1] <= a[i+1]. Clarify strict versus non-
# strict before coding.
def longest_with_one_change(a, strict=False):
    n = len(a)
    # nothing to fix in arrays of length 0, 1 or 2
    if n <= 2:
        return n
    # FOLLOW-UP 1: strict=True means strictly increasing
    # non-decreasing uses <=, strictly increasing uses <
    def ok(x, y):
        return x < y if strict else x <= y

    # can a value fit between the two neighbours?
    gap = 2 if strict else 0
    left, right = run_lengths(a, ok)
    best = 1
    # try changing each element a[i]
    for i in range(n):
        L = left[i - 1] if i else 0
        R = right[i + 1] if i + 1 < n else 0
        # at an end, or when a value fits between the
        # neighbours, the change joins both runs
        if not L or not R or a[i + 1] - a[i - 1] >= gap:
            best = max(best, L + 1 + R)
        # otherwise it joins only the longer run
        else:
            best = max(best, max(L, R) + 1)
    return best


# WARM-UP: no replacement allowed
def longest_without_change(a):
    # the plain longest increasing subarray warm-up
    best = run = 0
    for i, x in enumerate(a):
        run = run + 1 if i and a[i - 1] < x else 1
        best = max(best, run)
    return best


if __name__ == "__main__":
    # change 0 to 1: [1, 1, 3, 4, 5] has length 5
    nums = [1, 0, 3, 4, 5, 2, 3]
    assert longest_with_one_change(nums) == 5
    # tiny arrays
    assert longest_with_one_change([]) == 0
```

```
assert longest_with_one_change([7]) == 1
# strict version needs room between neighbours
strict = longest_with_one_change
assert strict([1, 2, 2, 3, 4], True) == 4
assert strict([1, 1, 1, 1], True) == 2
# the warm-up without any change
assert longest_without_change([1, 3, 5, 4, 7]) == 3
```

NOTES FROM CANDIDATES

Asked as an onsite coding round in an early-career L3 loop, and as the first technical round of new-grad VOs.
Candidates who wrote a slow version first were asked to optimize, leaving little time for anything else — the problem is short, so interviewers expect the linear approach to arrive quickly.
Edge cases worth confirming: arrays of length 0–2, where the replaced element is the first or last of the run, and whether the replaced value must fit between both neighbors or only one.

PREPARATION

Write the plain longest-run scan, then extend it with left-run / right-run arrays (or a two-state DP: "no change used" / "change used") so you can explain the jump from one to the other.
Test the bridge case explicitly: joining the run ending at i - 1 with the run starting at i + 1 is only valid when nums[i - 1] <= nums[i + 1]; in the strict variant there must be room for a value strictly between the neighbors, i.e. nums[i + 1] - nums[i - 1] >= 2 when the replacement must be an integer (plain < only if any real value is allowed). Otherwise the change only extends one side by one.
Repeat the exercise with the strict-increasing variant to make sure your comparisons flip correctly.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: dp, grid, medium

MEDIUM | 1Point3Acres | phone-screen | 3 of 15

Grid Path Count with Diagonal Moves and Checkpoints

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | **ALL TIME 48.9** | **30 DAYS 33** | **PRIORITY 35/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #62 Unique Paths

LeetCode match: LC 62 Unique Paths (related)

Frequency: Single report | Last asked: 2026-09-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/grid-path-count-three-moves-checkpoints

QUESTION

Count paths in an n x m matrix from (n-1, 0) to (n-1, m-1) moving up-right, right, or down-right. Follow-ups: count paths visiting all checkpoints; then checkpoints must be visited in order, with the observation that each move changes column, and optimize space.

Matrix of size n x m. Start at (n-1, 0), end at (n-1, m-1). Allowed moves from (r, c): up-right (r-1, c+1), right (r, c+1), down-right (r+1, c+1); stay inside the matrix.
Return the total number of distinct paths from start to end.
Follow-up 1: the matrix contains checkpoints; count paths that visit all checkpoints.
Follow-up 2: the checkpoints must be visited in order. Since every move changes the column, each column can contain at most one visited checkpoint. Count the paths and optimize space.

APPROACH

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

HOW TO THINK ABOUT IT

Every move goes exactly one column to the right, so a path visits each column exactly once and the state is just 'which row am I in at this column'. That is a DP over columns with a single rolling array. A checkpoint (r, c) simply says the path must be in row r at column c, so you zero every other row of that column.

Basis: LeetCode 62, Unique Paths, with up-right / right / down-right moves, and checkpoints the path must visit.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def count(r, c, seen):
    if c == m - 1:
        done = r == n - 1 and checkpoints <= seen
        return 1 if done else 0
    total = 0
    for dr in (-1, 0, 1):
        nr = r + dr
        if 0 <= nr < n:
            nxt = seen | {(nr, c + 1)}
            total += count(nr, c + 1, nxt)
    return total
answer = count(n - 1, 0, {(n - 1, 0)})
```

TIME	SPACE
O(3^m)	O(m)

WHY IT IS SLOW

Up to 3^m paths. Every move advances one column, so a dp over (column, row) with a rolling array counts them in O(n * m).

OPTIMAL SOLUTION · Column DP with a rolling array and checkpoint filters

TIME	SPACE
O(n * m)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Two checkpoints share a column.

No path can visit both, because a path is in exactly one row per column, so the answer is 0 (the code returns it before the DP). Checkpoints at the start or end cell are fine: they only filter rows of the first or last column.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Grid Path Count with Diagonal Moves and
# Checkpoints
#
```

```python
# Count paths in an n x m matrix from (n-1, 0) to (n-1,
# m-1) moving up-right, right, or down-right. Follow-ups:
# count paths visiting all checkpoints; then checkpoints
# must be visited in order, with the observation that each
# move changes column, and optimize space. Matrix of size
# n x m. Start at (n-1, 0), end at (n-1, m-1). Allowed
# moves from (r, c): up-right (r-1, c+1), right (r, c+1),
# down-right (r+1, c+1); stay inside the matrix. Return
# the total number of distinct paths from start to end.
# Follow-up 1: the matrix contains checkpoints; count
# paths that visit all checkpoints. Follow-up 2: the
# checkpoints must be visited in order. Since every move
# changes the column, each column can contain at most one
# visited checkpoint. Count the paths and optimize space.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Two checkpoints share a column.
#
# INTERVIEW NOTES
# Every move goes exactly one column to the right, so a
# path visits each column exactly once and the state is
# just 'which row am I in at this column'. That is a DP
# over columns with a single rolling array. A checkpoint
# (r, c) simply says the path must be in row r at column
# c, so you zero every other row of that column.
#
# BRUTE FORCE (say it first): Enumerate every path
# Depth-first search over all move sequences, counting
# those that end at the bottom-right with the checkpoints
# visited. Exponential in the number of columns. Time
# O(3^m). Space O(m).


# BRUTE FORCE (say it first): Enumerate every path | time
# O(3^m)
def count_paths_brute(
    n, m, checkpoints=(), ordered=False
):
    cps = [tuple(c) for c in checkpoints]
    total = 0

    def go(r, c, seen):
        nonlocal total
        if (r, c) in cps:
            seen = seen + [(r, c)]
        if c == m - 1:
            if r == n - 1 and (
                seen == cps
                if ordered
                else sorted(seen) == sorted(cps)
            ):
                total += 1
            return
        for dr in (-1, 0, 1):
            if 0 <= r + dr < n:
                go(r + dr, c + 1, seen)

    go(n - 1, 0, [])
    return total


# HELPERS for the base question
def step(ways):
    # next column from the current column only
    n = len(ways)
    return [(ways[r - 1] if r else 0) + ways[r]
            + (ways[r + 1] if r + 1 < n else 0)
            for r in range(n)]


def keep_row(ways, row):
    # a checkpoint forces the path through one row
    return [x if r == row else 0
            for r, x in enumerate(ways)]


# FOLLOW-UP 1: Two checkpoints share a column.
# Answer: No path can visit both, because a path is in
# exactly one row per column, so the answer is 0 (the code
# returns it before the DP). Checkpoints at the start or
# end cell are fine: they only filter rows of the first or
# last column.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def checkpoint_rows(checkpoints, ordered):
    # column -> required row; None if impossible
    rows = {}
    for r, c in checkpoints:
        # two different checkpoints in one column: no path
        if rows.setdefault(c, r) != r:
            return None
    if len(rows) != len(checkpoints):
        return None
    cols = [c for _, c in checkpoints]
    # ordered: they must be listed left to right
    if ordered and cols != sorted(cols):
        return None
    return rows


# BASE QUESTION (optimal): Column DP with a rolling array
# and checkpoint filters
# Idea: ways[r] is the number of paths that are in row r
# at the current column. The next column is ways'[r] =
# ways[r - 1] + ways[r] + ways[r + 1]. At a checkpoint
# column keep only the checkpoint's row.
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
    # one rolling column is enough: O(n) space
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
    for c in range(1, m):
        ways = step(ways)
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]


# HELPERS for the checks below
def count_paths_table(n, m):
    # full table: ways[c][r] paths reach row r, column c
    ways = [[0] * n for _ in range(m)]
    ways[0][n - 1] = 1
    for c in range(1, m):
        for r in range(n):
            # arrive from up-right, right or down-right
            for pr in (r - 1, r, r + 1):
                if 0 <= pr < n:
                    ways[c][r] += ways[c - 1][pr]
    return ways[m - 1][n - 1]


if __name__ == "__main__":
    assert count_paths(3, 3) == 2
    assert count_paths_table(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths_table(4, 6) == count_paths(4, 6)
    # force the middle column through row 1
```

```
assert count_paths(3, 3, [(1, 1)]) == 1
# two checkpoints in one column: impossible
assert count_paths(3, 3, [(0, 1), (1, 1)]) == 0
# ordered checkpoints must go left to right
assert count_paths(3, 4, [(1, 1), (1, 2)], True) == 1
assert count_paths(3, 4, [(1, 2), (1, 1)], True) == 0
```

NOTES FROM CANDIDATES

Asked in an L4 phone interview.
Every move advances exactly one column, so the path visits each column exactly once — this is the key structural fact the follow-ups build on.
Clarify for follow-up 1 what happens if two checkpoints share a column (no path can visit both) and whether checkpoints are given as a list or marked in the grid.

PREPARATION

Write the column-by-column DP where dp[c][r] depends only on dp[c-1][r-1..r+1], then reduce it to a single rolling column.
For checkpoints, practice the "segment" approach: sort by column, and between consecutive checkpoints only allow paths that pass through them (zero out other rows in a checkpoint's column).
Test: n = 1 (only straight right), m = 1, a checkpoint at the start or end, and two checkpoints in the same column.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: tree, dp

MEDIUM | 1Point3Acres | phone-screen | 4 of 15

Minimum-Cost Tree Disconnection

ASKED 2026-09-09 | 41 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 197/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-root-leaf-minimum-cut

QUESTION

EXAMPLES

Negative-weight follow-up

```
A --(-1)--> B --(-1)--> C
Minimum total cut cost: -2
```

Cutting both edges costs -2, even though cutting just the first edge already separates the root from the original leaf.

APPROACH

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what must be disconnected (root from every leaf), whether the tree is rooted and directed, whether weights can be negative, and what to return for an empty or single-node tree.

BRUTE FORCE TO SAY OUT LOUD

Try every subset of edges and keep those that separate the root from all leaves. Exponential.

OPTIMAL IDEA

Define best(v) as the cheapest set of cuts in v's subtree that disconnects v from every leaf below it. For each child c with edge weight w, either cut the edge, paying w, or keep it and require best(c) (not allowed if c is a leaf). Sum over children. With non-negative weights that is the answer. With negative weights, once you cut an edge the whole subtree below becomes free, and you may still cut any negative edge there, so the cut option costs w plus the sum of min(0, edge) over that subtree.

DRY-RUN THESE CASES

A single edge, the negative chain A to B to C (answer -2), a leaf at different depths, a star.

LIKELY FOLLOW-UPS

Negative weights, many roots, or recursion depth for a chain tree.

PITFALLS

Treating every negative edge as a mandatory cut. Recursing deeply on a chain.

INTERVIEW-DAY PLAN

A single edge, the negative chain A to B to C (answer -2), a leaf at different depths, a star.

LIKELY FOLLOW-UPS

Negative weights, many roots, or recursion depth for a chain tree.

PITFALLS

Treating every negative edge as a mandatory cut. Recursing deeply on a chain.

HOW TO THINK ABOUT IT

On a tree, every decision about a subtree depends only on the edge above it, so define a function of the node that returns the best cost inside its subtree. For each child you either cut the edge above it (pay its weight) or keep it and make the child itself disconnected from the leaves below. With negative weights, a cut edge frees the whole subtree, so you may cut every negative edge in it for free gain.

Basis: Written. Assumed setup (confirm it): a rooted tree with edge weights (cuts cost the weight, weights may be negative); cut edges so the root is disconnected from every leaf; a leaf is a non-root node with no children; with no leaves the cost is 0.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, edges):
    best = INF
    for k in range(len(edges) + 1):
        for cut in combinations(edges, k):
            tree = remove_edges(edges, cut)
            if not reaches_leaf(tree, root):
                cost = sum(w for _, _, w in cut)
                best = min(best, cost)
    return best
```

TIME	SPACE
O(2^E * n)	O(n)

WHY IT IS SLOW

2^E subsets, each checked with a search: O(2^E * n). A tree DP (cut the edge above a child, or require the child to be disconnected from its leaves) is O(n).

OPTIMAL SOLUTION · Tree DP: best(v) = cheapest cuts inside v's subtree that disconnect v from its leaves

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Weights are non-negative.

Then free(c) is always 0 and the recurrence collapses to best(v) = sum over children of (w if the child is a leaf, else min(w, best(child))).

FOLLOW-UP 2 The tree is a long chain and recursion overflows.

Compute nodes in reverse BFS order (children before parents) and store best and free in arrays, no recursion needed. State the space as O(n).

FOLLOW-UP 3 Return which edges to cut.

Remember at each child which option was cheaper, then walk down from the root collecting the cut edges. Cut edges also include every negative edge below any cut edge.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimum-Cost Tree Disconnection
#
# Examples: Negative-weight follow-up: A --(-1)--> B
# --(-1)--> C Minimum total cut cost: -2 Cutting both
# edges costs -2, even though cutting just the first edge
# already separates the root from the original leaf.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Weights are non-negative.
# 2. The tree is a long chain and recursion overflows.
# 3. Return which edges to cut.
#
# INTERVIEW NOTES
# On a tree, every decision about a subtree depends only
# on the edge above it, so define a function of the node
# that returns the best cost inside its subtree. For each
# child you either cut the edge above it (pay its weight)
# or keep it and make the child itself disconnected from
# the leaves below. With negative weights, a cut edge
# frees the whole subtree, so you may cut every negative
# edge in it for free gain.
#
# BRUTE FORCE (say it first): Try every subset of edges
# Enumerate each subset of edges to cut, check that the
# root can reach no leaf, and keep the cheapest.
# Exponential, but exact, so it checks the DP. Time O(2^E
# * n). Space O(n).


from collections import defaultdict


# BRUTE FORCE (say it first): Try every subset of edges |
# time O(2^E * n)
def disconnect_brute(n, edges):
    children = defaultdict(list)
    for u, v, w in edges:
        children[u].append(v)
    leaves = [
        v
        for v in range(n)
        if v != 0 and not children[v]
    ]
    best = None
    for mask in range(1 << len(edges)):
        kept = [
            (u, v)
            for i, (u, v, w) in enumerate(edges)
            if not mask >> i & 1
        ]
        adj = defaultdict(list)
        for u, v in kept:
            adj[u].append(v)
        seen, stack = {0}, [0]
        while stack:
            x = stack.pop()
            for y in adj[x]:
                if y not in seen:
                    seen.add(y)
                    stack.append(y)
        if not any(leaf in seen for leaf in leaves):
            cost = sum(
                w
                for i, (u, v, w) in enumerate(edges)
                if mask >> i & 1
            )
            best = (
                cost
                if best is None
                else min(best, cost)
            )
    return best if best is not None else 0


# HELPERS for the base question
INF = float("inf")


def tree_costs(n, edges):
    # edges are (parent, child, cost); node 0 is the root
    kids = defaultdict(list)
    for u, v, w in edges:
        kids[u].append((v, w))
    # FOLLOW-UP 2: a loop, not recursion: safe on a long
    # chain
    # breadth-first order: parents come before children
    order = [0]
    for u in order:
        for v, _ in kids[u]:
            order.append(v)
    # best[v]: cheapest way to cut v off from its leaves
    # free[v]: sum of every negative cost below v
    best, free = [0] * n, [0] * n
    for v in reversed(order):
        for c, w in kids[v]:
            # 1) cut this edge and the negatives below
            cut = w + free[c]
            # 2) keep it and cut deeper (a leaf cannot)
            keep = best[c] if kids[c] else INF
            best[v] += min(cut, keep)
            free[v] += min(0, w) + free[c]
    return kids, best, free


# BASE QUESTION (optimal): Tree DP: best(v) = cheapest
# cuts inside v's subtree that disconnect v from its
# leaves
# Idea: For a child c with edge weight w there are two
# options: cut the edge, paying w plus whatever extra
# negative-weight edges you can still cut below c (free(c)
# = sum of min(0, weight) over its subtree), or keep the
# edge and require best(c), which is impossible when c is
# a leaf.
```

```
# FOLLOW-UP 1: Weights are non-negative.
# Answer: Then free(c) is always 0 and the recurrence
# collapses to best(v) = sum over children of (w if the
# child is a leaf, else min(w, best(child))).
# FOLLOW-UP 2: The tree is a long chain and recursion
# overflows.
# Answer: Compute nodes in reverse BFS order (children
# before parents) and store best and free in arrays, no
# recursion needed. State the space as O(n).
def disconnect_cost(n, edges):
    # works for negative costs and needs no recursion
    return tree_costs(n, edges)[1][0]


# FOLLOW-UP 3: Return which edges to cut.
# Answer: Remember at each child which option was cheaper,
# then walk down from the root collecting the cut edges.
# Cut edges also include every negative edge below any cut
# edge.
def edges_to_cut(n, edges):
    kids, best, free = tree_costs(n, edges)
    cuts, stack = [], [0]
```

```
    while stack:
        v = stack.pop()
        for c, w in kids[v]:
            keep = best[c] if kids[c] else INF
            if w + free[c] <= keep:
                # cut here, plus negative edges below c
                cuts.append((v, c))
                below = [c]
                while below:
                    x = below.pop()
                    for y, wy in kids[x]:
                        if wy < 0:
                            cuts.append((x, y))
                        below.append(y)
            else:
                stack.append(c)
    return cuts


if __name__ == "__main__":
    # the question's chain A -(-1)-> B -(-1)-> C
    neg = [(0, 1, -1), (1, 2, -1)]
    assert disconnect_cost(3, neg) == -2
    cuts = sorted(edges_to_cut(3, neg))
    assert cuts == [(0, 1), (1, 2)]
    # non-negative costs: cut the cheaper edge
    chain = [(0, 1, 5), (1, 2, 3)]
    assert disconnect_cost(3, chain) == 3
    assert edges_to_cut(3, chain) == [(1, 2)]
    # a single node needs no cut
    assert disconnect_cost(1, []) == 0
    # a long chain does not overflow the stack
    big = [(i, i + 1, 1) for i in range(5000)]
    assert disconnect_cost(5001, big) == 1
```

NOTES FROM CANDIDATES

Clarify the required result for an empty tree and a single-node tree. No final return convention is specified for these cases.
Include recursion-stack memory in space analysis. The tree can degenerate into a chain; balanced-tree depth is not a worst-case bound.
Coding took place directly in Google Docs with no supplied framework; the candidate also had to define the tree node structure.
The interviewer asked the candidate to think aloud while developing the approach.
For nonnegative weights, use postorder DP. Let F(v) be the minimum cost to disconnect node v from every original leaf below it. An original leaf has F(v) = infinity; an internal node has F(v) = sum(min(weight(v,c), F(c))) over its children. Each branch must be blocked independently. Use a safe infinity representation rather than adding to a fixed-width maximum integer.
For negative weights, cutting an ancestor edge does not remove the opportunity to cut negative edges below it. Track A(v), the minimum optional cut cost when the subtree is already disconnected: A(v) = sum(min(0, weight(v,c)) + A(c)), with A(leaf) = 0. Then use F(v) = sum(min(weight(v,c) + A(c), F(c))), retaining F(leaf) = infinity. This includes beneficial redundant negative cuts

without double counting edges.
Both recurrences process each edge once: O(n) time and O(h) recursive auxiliary space, where h can equal n. A single original root-leaf cannot be disconnected from itself by cutting edges; agree on its result convention before coding.

PREPARATION

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: array, backtracking, dp, math, medium

MEDIUM | 1Point3Acres | onsite-coding | 5 of 15

Target Expressions from Ordered Digits

ASKED 2026-08-13 | 68 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 36.5** | **30 DAYS 0** | **PRIORITY 103/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 282 Expression Add Operators

Frequency: Single report | Last asked: 2026-08-13 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/target-expressions-ordered-digits

QUESTION

EXAMPLES

```
For digits [2, 3, 4] and target 14, valid outputs
    include:
2 * (3 + 4)
2 + 3 * 4
```

APPROACH

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether digits may be concatenated into multi-digit numbers, which operators are allowed, whether parentheses are free, whether the digit order is fixed, and whether to return all expressions or a boolean.

BRUTE FORCE TO SAY OUT LOUD

Enumerate every operator placement and every parenthesization for the three or four digits and evaluate each. That is fine for a handful of digits and is what the first stage asks for.

OPTIMAL IDEA

Interval DP: for each interval of digits compute a map from value to the expressions that achieve it by splitting at every point and combining left and right with plus, minus and times. Memoize by interval. If only reachability is needed store just the set of values per interval, which is much smaller than the strings.

DRY-RUN THESE CASES

[2, 3, 4] with target 14 gives 2*(3+4) and 2+3*4. A target that is unreachable. A single digit. Zeros.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

N digits (the DP above), division with fractions, or concatenation.

PITFALLS

Missing parentheses in the output string. Evaluating with floats.

HOW TO THINK ABOUT IT

Any expression over a run of digits is two smaller expressions joined by an operator at some split point, so the set of values for an interval is built from the sets of its two halves. Compute values bottom-up per interval and only build text for the target. Be explicit about parentheses: they come from the split structure, so every split is a candidate.

Basis: Related to LeetCode 282, Expression Add Operators, but with parentheses allowed and digits not concatenated.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def values(digits):
    if len(digits) == 1:
        return {digits[0]}
    result = set()
    for cut in range(1, len(digits)):
        for a in values(digits[:cut]):
            for b in values(digits[cut:]):
                result |= {a + b, a - b, a * b}
    return result
reachable = target in values(all_digits)
```

TIME	SPACE
exponential (Catalan(n) *...	**O(n)**
exponential (Catalan(n) * 3^(n-1) expressions)	recursion plus the value set

WHY IT IS SLOW

The same sub-interval of digits is recomputed from scratch for every split above it, giving Catalan-number growth. Remembering the values per interval (memoisation) makes it polynomial in the number of distinct values.

OPTIMAL SOLUTION · Interval DP: all expressions for digits[l:r]

TIME	SPACE
exponential in the number...	**O(distinct values per interval)**
exponential in the number of results; polynomial in the number of distinct values	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you handle N digits where listing every expression is too much?

Agree on the contract: if you only need to know whether a target is reachable, store just the set of values per interval (a boolean DP over (l, r, value)), which is far smaller than the set of strings. If you need one witness, store one expression string per value. Values are bounded by the digits, so the table stays practical for moderate N.

FOLLOW-UP 2 What if concatenating digits (23) or division is allowed?

Concatenation adds base cases: for every l and r the number int(digits[l:r]) is also an atom of the interval. Division adds a fourth operator with a zero check and, if exactness matters, Fraction values instead of floats.

FOLLOW-UP 3 Only a yes/no answer is needed for many digits.

Store just the set of reachable values per interval (as target_reachable does) instead of the expression strings; the strings blow up long before the value sets do. Build the expression only for the winning target by remembering one split per value.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Target Expressions from Ordered Digits
#
# Examples: For digits [2, 3, 4] and target 14, valid
# outputs include: 2 * (3 + 4) 2 + 3 * 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you handle N digits where listing every
# expression is too much?
# 2. What if concatenating digits (23) or division is
# allowed?
# 3. Only a yes/no answer is needed for many digits.
#
# INTERVIEW NOTES
# Any expression over a run of digits is two smaller
# expressions joined by an operator at some split point,
# so the set of values for an interval is built from the
# sets of its two halves. Compute values bottom-up per
# interval and only build text for the target. Be explicit
# about parentheses: they come from the split structure,
# so every split is a candidate.
#
# BRUTE FORCE (say it first): Recursively try every split
# and operator, without remembering sub-results
# Split the digits at every point, combine all value pairs
# of the two parts with +, - and *, and see whether the
# target appears. Correct, but the same sub-interval is
# recomputed many times; memoising by interval is the fix.
# Time exponential (Catalan(n) * 3^(n-1) expressions).
# Space O(n) recursion plus the value set.
```

```
from collections import defaultdict


# BRUTE FORCE (say it first): Recursively try every split
# and operator, without remembering sub-results | time
# exponential (Catalan(n) * 3^(n-1) expressions)
def _all_values(digits):
    if len(digits) == 1:
        return {digits[0]}
    out = set()
    for i in range(1, len(digits)):
        for a in _all_values(digits[:i]):
            for b in _all_values(digits[i:]):
                out |= {a + b, a - b, a * b}
    return out


def target_reachable_brute(digits, target):
    return target in _all_values(digits)


# HELPERS for the base question
def build(digits):
    # table[(l, r)] = {value: {(text, precedence)}}
    table = {}

    def solve(l, r):
        if (l, r) in table:
            return table[(l, r)]
        res = defaultdict(set)
        if r - l == 1:
            # a single digit is an atom (precedence 3)
            res[digits[l]].add((str(digits[l]), 3))
        # try every split point
        for m in range(l + 1, r):
            for lv, ls in solve(l, m).items():
                for rv, rs in solve(m, r).items():
                    combine(res, lv, ls, rv, rs)
        table[(l, r)] = res
        return res

    return solve(0, len(digits))


def combine(res, lv, ls, rv, rs):
    ops = (("+", 1, lv + rv), ("-", 1, lv - rv),
           ("*", 2, lv * rv))
    for lt, lp in ls:
        for rt, rp in rs:
            for op, prec, val in ops:
                # brackets only where precedence needs
                left = f"({lt})" if lp < prec else lt
                need = rp < prec
                if rp == prec and op == "-":
                    need = True
                right = f"({rt})" if need else rt
                res[val].add((left + op + right, prec))
```

```
# BASE QUESTION (optimal): Interval DP: all expressions
# for digits[l:r]
# Idea: Every expression over digits[l:r] is two sub-
# expressions over a split point joined by +, - or *.
# Memoize, for each interval, a map from numeric value to
# the set of expression strings that produce it.
def expressions_for_target(digits, target):
    found = build(digits).get(target, ())
    return sorted({text for text, _ in found})


# FOLLOW-UP 3: Only a yes/no answer is needed for many
# digits.
```

```python
# Answer: Store just the set of reachable values per
# interval (as target_reachable does) instead of the
# expression strings; the strings blow up long before the
# value sets do.
def target_reachable(digits, target):
    # yes/no only: keep just the set of values per range
    values = {}

    def solve(l, r):
        if (l, r) in values:
            return values[(l, r)]
        out = {digits[l]} if r - l == 1 else set()
        for m in range(l + 1, r):
            for a in solve(l, m):
                for b in solve(m, r):
                    out |= {a + b, a - b, a * b}
        values[(l, r)] = out
        return out

    return target in solve(0, len(digits))


if __name__ == "__main__":
    ans = expressions_for_target([2, 3, 4], 14)
    assert "2*(3+4)" in ans and "2+3*4" in ans
    assert target_reachable([2, 3, 4], 14)
    assert not target_reachable([2, 3, 4], 100)
    # subtraction needs brackets on the right
    sub = expressions_for_target([5, 3, 1], 3)
    assert "5-(3-1)" in sub
    assert target_reachable([1] * 8, 8)
```

NOTES FROM CANDIDATES

The base case is small enough for direct enumeration, but a complicated ad hoc implementation becomes difficult to extend. Keep expression construction separate from numeric evaluation while negotiating the N-digit contract.
If the agreed contract uses every digit exactly once in order, memoize each interval [l, r) as a map from attainable numeric values to the expression strings producing them. For every split point, combine left and right entries with + and *, wrapping composite children as needed; the full-interval target bucket is the answer. Because every distinct expression must be retained, both time and storage are output-sensitive and can grow exponentially even though interval states are memoized.

PREPARATION

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: string, dp, medium

MEDIUM | 1Point3Acres | phone-screen | 6 of 15

Count Word Decompositions into Chemical Element Symbols

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 43.5** | **30 DAYS 0** | **PRIORITY 81/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: LC 139 Word Break (related)

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/periodic-table-word-decomposition

QUESTION

EXAMPLES

Physics has four stated decompositions

P - H - Y - Si - C - S P - H - Y - Si - Cs P - H - Y - S - I - C - S P - H - Y - S - I - Cs The returned count is 4.

APPROACH

Implement a suffix-count or prefix-count formulation and state what each state represents.

Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether matching is case-insensitive, whether to count decompositions or just test for one, and for the dictionary (the periodic table symbols of one and two letters).

BRUTE FORCE TO SAY OUT LOUD

Recursive search that tries both a one-letter and a two-letter symbol at each position. Exponential in the worst case.

OPTIMAL IDEA

Suffix-count DP: ways[i] is the number of decompositions of word[i:], with ways[n] = 1. ways[i] adds ways[i + 1] if word[i] is a symbol and ways[i + 2] if word[i:i + 2] is a symbol. O(n). Physics gives 4.

DRY-RUN THESE CASES

Physics (4), a word with no decomposition (hello), one with exactly one, the empty string, overlapping one- and two-letter symbols such as Si versus S + I.

LIKELY FOLLOW-UPS

List the decompositions, handle a larger dictionary with a trie, or count modulo a prime.

INTERVIEW-DAY PLAN

PITFALLS

Forgetting to lowercase before the lookup. Starting the DP with ways[n] = 0.

HOW TO THINK ABOUT IT

Counting the ways to cut a string into dictionary pieces is Word Break with a count instead of a yes/no. Define ways[i] as the number of decompositions of the suffix starting at i; it depends only on ways[i + 1] and ways[i + 2], because symbols have one or two letters. Compute from the end so every value you need already exists.

Basis: A counting version of LeetCode 139, Word Break, with the periodic-table symbols as the dictionary.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def ways(i):
    if i == len(word):
        return 1
    total = 0
    if word[i] in SYMBOLS:
        total += ways(i + 1)
    if word[i:i + 2] in SYMBOLS:
        total += ways(i + 2)
    return total
```

TIME	SPACE
exponential (Fibonacci gr... exponential (Fibonacci growth)	**O(n)**

WHY IT IS SLOW

The same suffix is reached by many different splits, so the calls grow like the Fibonacci numbers. Computing ways from the end of the word once per position is O(n).

OPTIMAL SOLUTION · Suffix-count DP

TIME	SPACE
O(n)	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you list the decompositions, not just count them?

Keep the same suffix recursion but return the list of splits instead of a number, memoized by start index. The count version is O(n); the list version is output-sensitive because there can be exponentially many splits, so agree on whether the interviewer wants a count, one witness, or all of them.

FOLLOW-UP 2 What if the dictionary is large and symbols can be long?

Put the dictionary in a trie (or a set plus the maximum word length L) and only look at substrings up to length L from each index. That makes the DP O(n * L) instead of O(n^2), which is the standard Word Break optimization.

FOLLOW-UP 3 List the decompositions instead of counting them.

Same recursion, but carry the parts chosen so far and record them at the end. The number of results can be exponential, so ask whether a count or a list is wanted.

FOLLOW-UP 4 A larger dictionary with long entries.

Store the dictionary in a set plus the maximum word length L (only look at substrings up to L), or in a trie, so each position costs O(L) lookups instead of O(n).

FOLLOW-UP 5 Count modulo a prime, or only test whether any decomposition exists.

Replace addition by addition modulo the prime, or by logical or for a yes/no answer. The DP structure is the same.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Word Decompositions into Chemical
# Element Symbols
#
# Examples: Physics has four stated decompositions: P - H
# - Y - Si - C - S P - H - Y - Si - Cs P - H - Y - S - I -
# C - S P - H - Y - S - I - Cs The returned count is 4.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you list the decompositions, not just count
# them?
# 2. What if the dictionary is large and symbols can be
# long?
# 3. List the decompositions instead of counting them.
# 4. A larger dictionary with long entries.
# 5. Count modulo a prime, or only test whether any
# decomposition exists.
#
# INTERVIEW NOTES
# Counting the ways to cut a string into dictionary pieces
# is Word Break with a count instead of a yes/no. Define
# ways[i] as the number of decompositions of the suffix
# starting at i; it depends only on ways[i + 1] and ways[i
# + 2], because symbols have one or two letters. Compute
# from the end so every value you need already exists.
#
# BRUTE FORCE (say it first): Recursion that tries a one-
# letter and a two-letter symbol at every position
# At each position take either the next letter or the next
# two letters if they are symbols, and add up the ways to
# finish. Correct and the oracle; the same suffix is
# solved many times. Time exponential (Fibonacci growth).
# Space O(n).


# HELPERS for the brute force
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())


# BRUTE FORCE (say it first): Recursion that tries a one-
# letter and a two-letter symbol at every position | time
# exponential (Fibonacci growth)
def decompositions_recursive(word, symbols=ELEMENTS):
    w = word.lower()

    def go(i):
        if i == len(w):
            return 1
        total = 0
        if w[i] in symbols:
            total += go(i + 1)
        if i + 2 <= len(w) and w[i : i + 2] in symbols:
            total += go(i + 2)
        return total

    return go(0)


# BASE QUESTION (optimal): Suffix-count DP
# Idea: ways[i] is the number of decompositions of
# word[i:]. A symbol has one or two letters, so ways[i] =
# ways[i + 1] (if word[i] is a symbol) + ways[i + 2] (if
# word[i:i + 2] is a symbol).
def count_decompositions(word, symbols=ELEMENTS,
                         mod=None):
    w, n = word.lower(), len(word)
    # ways[i]: ways to split the suffix starting at i
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        # a one-letter symbol, then a two-letter symbol
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] %= mod
    return ways[0]


# FOLLOW-UP 3: List the decompositions instead of counting
# them.
# Answer: Same recursion, but carry the parts chosen so
# far and record them at the end. The number of results
# can be exponential, so ask whether a count or a list is
# wanted.
def list_decompositions(word, symbols=ELEMENTS):
    w, n, memo = word.lower(), len(word), {}

    def splits(i):
        # every way to split the suffix starting at i
        if i == n:
            return [[]]
        if i in memo:
            return memo[i]
        out = []
        for size in (1, 2):
```

```
            part = w[i:i + size]
            if len(part) == size and part in symbols:
                for rest in splits(i + size):
                    out.append([part] + rest)
        memo[i] = out
        return out

    return splits(0)


# FOLLOW-UP 4: A larger dictionary with long entries.
# Answer: Store the dictionary in a set plus the maximum
# word length L (only look at substrings up to L), or in a
# trie, so each position costs O(L) lookups instead of
# O(n).
def count_with_max_len(word, symbols):
    # big dictionary: only look at pieces up to its
    # longest
    longest = max(map(len, symbols))
    w, n = word.lower(), len(word)
    ways = [0] * (n + 1)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        for size in range(1, min(longest, n - i) + 1):
            if w[i:i + size] in symbols:
                ways[i] += ways[i + size]
    return ways[0]


# FOLLOW-UP 5: Count modulo a prime, or only test whether
# any decomposition exists.
# Answer: Replace addition by addition modulo the prime,
# or by logical or for a yes/no answer. The DP structure
```

```
# is the same.
def exists_decomposition(word, symbols=ELEMENTS):
    # yes / no only: the same DP with or instead of +
    return count_decompositions(word, symbols) > 0


if __name__ == "__main__":
    assert count_decompositions("Physics") == 4
    parts = list_decompositions("Physics")
    assert len(parts) == 4
    assert ["p", "h", "y", "si", "c", "s"] in parts
    assert exists_decomposition("Physics")
    # no element starts with x then q
    assert not exists_decomposition("xq")
    assert count_decompositions("Physics", mod=3) == 1
    assert count_with_max_len("Physics", ELEMENTS) == 4
```

NOTES FROM CANDIDATES

Confirm whether matching is case-sensitive and whether the caller supplies the symbol set or the implementation may treat the periodic table as fixed.
The result counts decompositions, not merely whether at least one decomposition exists.

PREPARATION

Implement a suffix-count or prefix-count formulation and state what each state represents.
Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: dp, recursion, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 7 of 15

Fence Painting Minimum Operations (Gemini Team)

ASKED 2026-04-27 | 176 DAYS AGO | LAST 6 MONTHS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 200/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-04-27 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/fence-painting-min-ops

QUESTION

EXAMPLES

```
n = 5, a = [2, 2, 1, 2, 1]
Optimal:
  1) Horizontal stroke at h=1 across [1..5]
      (covers row 1 across whole fence)
  2) Horizontal stroke at h=2 across [1..2]
  3) Vertical stroke at plank 4 (height 2 still
      missing row 2)
Optimal answer: 3.
```

APPROACH

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether vertical strokes paint one plank fully, whether horizontal strokes must cover consecutive planks at one height, and the sizes involved.

BRUTE FORCE TO SAY OUT LOUD

Try all combinations of strokes. Exponential.

OPTIMAL IDEA

Divide and conquer on the minimum height: for a range with base height h, painting every plank vertically costs the number of planks; otherwise paint one horizontal band up to the minimum height m (cost 1) and recurse on the segments above m, split where planks are exactly m. The answer is the minimum of the two. O(n^2) worst case, O(n log n) with a sparse table or a min-stack for range minimum queries.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

The example giving 3. All equal heights (1). Strictly increasing heights. A single plank.

LIKELY FOLLOW-UPS

A brush of width W, or return the strokes.

PITFALLS

Forgetting the vertical option, so a tall single plank is overcounted. Off-by-one on the band height.

HOW TO THINK ABOUT IT

The shortest plank in a range decides the shape: a horizontal band along the bottom, up to that plank's height, is free of regret because every plank in the range needs it. That splits the range at the shortest planks, so recurse on the pieces above the band. The other choice for a range is to paint every plank vertically. Take whichever is cheaper.

Basis: Codeforces 448C, Painting Fence. Strokes: a vertical stroke paints one whole plank; a horizontal stroke paints one row across consecutive planks that all reach that height.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def paint(l, r, base):
    if l > r:
        return 0
    m = min(height[l:r + 1])
    vertical = r - l + 1
    horizontal = m - base
    for part in split_at_height(l, r, m):
        horizontal += paint(part.l, part.r, m)
    return min(vertical, horizontal)
```

TIME	SPACE
O(n^2)	O(n)
worst case	recursion

WHY IT IS SLOW

Finding the minimum by scanning makes a sorted or constant fence quadratic, O(n^2). A sparse table for range-minimum gives O(n log n).

OPTIMAL SOLUTION · Same recursion with a sparse table for range minimum

TIME	SPACE
O(n log n)	O(n log n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The horizontal brush is only W planks wide.

A band over a range needs ceil(width / W) strokes per row instead of one, so the band cost becomes (m - base) * ceil(len / W). The vertical option and the splitting are unchanged.

FOLLOW-UP 2 Return the strokes, not just the count.

Record which option won in each range and rebuild the list by walking the recursion again: horizontal bands as (row, l, r), vertical strokes as (plank).

FOLLOW-UP 3 Is it the same as largest rectangle in a histogram?

It shares the idea of splitting at the minimum (a Cartesian tree on the heights), but the objective differs: here you minimise strokes, there you maximise area. **Cost:** the fence recursion is O(n log n) with a sparse table (O(n^2) worst case with a plain scan); the histogram problem has an O(n) stack solution.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Fence Painting Minimum Operations (Gemini
# Team)
#
# Examples: n = 5, a = [2, 2, 1, 2, 1] Optimal: 1)
# Horizontal stroke at h=1 across [1..5] (covers row 1
# across whole fence) 2) Horizontal stroke at h=2 across
# [1..2] 3) Vertical stroke at plank 4 (height 2 still
# missing row 2) Optimal answer: 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The horizontal brush is only W planks wide.
# 2. Return the strokes, not just the count.
# 3. Is it the same as largest rectangle in a histogram?
#
# INTERVIEW NOTES
# The shortest plank in a range decides the shape: a
# horizontal band along the bottom, up to that plank's
# height, is free of regret because every plank in the
# range needs it. That splits the range at the shortest
# planks, so recurse on the pieces above the band. The
# other choice for a range is to paint every plank
# vertically. Take whichever is cheaper.
#
# BRUTE FORCE (say it first): Divide and conquer, scanning
# for the minimum each time
# For a range with base height b: either paint every plank
# vertically (cost = number of planks) or paint a
# horizontal band up to the range minimum (cost = min - b)
# and recurse on the pieces split at the minimum planks.
# Finding the minimum by scanning makes a sorted or
# constant fence quadratic. Time O(n^2) worst case. Space
# O(n) recursion.


# BRUTE FORCE (say it first): Divide and conquer, scanning
# for the minimum each time | time O(n^2) worst case
def paint_brute(a):
    def solve(l, r, base):
        if l > r:
            return 0
        m = min(a[l : r + 1])
        cost = m - base
        i = l
        while i <= r:
            if a[i] == m:
                i += 1
                continue
            j = i
            while j <= r and a[j] != m:
                j += 1
            cost += solve(i, j - 1, m)
            i = j
        return min(r - l + 1, cost)

    return solve(0, len(a) - 1, 0)


# HELPERS for the base question
def sparse_table(a):
    # table[j][i] = index of the smallest plank in a[i : i
    # + 2^j]
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        prev, row = table[-1], []
        for i in range(len(a) - (1 << j) + 1):
            x, y = prev[i], prev[i + (1 << (j - 1))]
            row.append(x if a[x] <= a[y] else y)
        table.append(row)
        j += 1
    return table


# FOLLOW-UP 2: Return the strokes, not just the count.
# Answer: Record which option won in each range and
# rebuild the list by walking the recursion again:
# horizontal bands as (row, l, r), vertical strokes as
# (plank).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def paint_strokes(a, width=None):
    # returns (number of strokes, the list of strokes)
    n = len(a)
    if n == 0:
        return 0, []
    table = sparse_table(a)

    def argmin(l, r):
        # smallest plank in a[l..r] in O(1)
        k = (r - l + 1).bit_length() - 1
        x, y = table[k][l], table[k][r - (1 << k) + 1]
        return x if a[x] <= a[y] else y

    def solve(l, r, base):
        # paint planks l..r whose bottom `base` rows are
        # done
        if l > r:
            return 0, []
        m = argmin(l, r)
        size = r - l + 1
        # a band of rows over the whole range needs one
        # stroke per row (or per `width` planks)
        per_row = 1
        if width is not None:
            per_row = -(-size // width)
        band = (a[m] - base) * per_row
        lc, ls = solve(l, m - 1, a[m])
        rc, rs = solve(m + 1, r, a[m])
        horizontal = band + lc + rc
        # or paint every plank by itself, top to bottom
```

```
        if size <= horizontal:
            planks = range(l, r + 1)
            return size, [("vertical", i) for i in planks]
        rows = [("horizontal", h, l, r)
                for h in range(base + 1, a[m] + 1)]
        return horizontal, rows + ls + rs

    return solve(0, n - 1, 0)


# BASE QUESTION (optimal): Same recursion with a sparse
# table for range minimum
# Idea: The recursion is correct; only the minimum lookup
# is slow. A sparse table answers range-minimum-index
# queries in O(1), and each plank becomes the split point
# at most once, so the recursion visits n ranges.
def paint(a, width=None):
    # FOLLOW-UP 1: width= limits the horizontal brush
    return paint_strokes(a, width)[0]


if __name__ == "__main__":
    # the example: 3 strokes
    assert paint([2, 2, 1, 2, 1]) == 3
    assert paint([]) == 0
```

```
    assert paint([3]) == 1
    # a flat fence is one stroke per row
    assert paint([4, 4, 4, 4]) == 4
    # tall thin planks are cheaper to paint upright
    assert paint([5, 0, 5]) == 2
    count, strokes = paint_strokes([2, 2, 1, 2, 1])
    assert count == len(strokes) == 3
    # a narrow brush needs several strokes per row
    assert paint([2, 2, 2, 2], width=2) == 4
    assert paint([2, 2, 2, 2], width=4) == 2
```

NOTES FROM CANDIDATES

Classic recursive divide-and-conquer: for range [l..r] with base height h:
Option A: paint every plank vertically -> cost r - l + 1.
Option B: paint a horizontal band of height min - h, then recurse on the sub-ranges above the band split by planks at height min.
Take the minimum.
Equivalent to Cartesian-tree DP; runs in O(N log N) with min-stack or sparse table, or O(N²) with the recursive form.
Verbalize the recursion's correctness: the band optimally covers all planks >= min in the range, then each "taller column block" is solved independently.

PREPARATION

Drill The canonical largest-rectangle-in-histogram problem — the geometric setup is the same.
Practice the recursive painter pattern on small inputs by hand; mistakes around the band height (min - base) are common.
Be ready for the follow-up: "the brush has width W instead of 1" — extends to bounded horizontal width.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

MEDIUM | IN APP LC 198 | PracHub | Technical Screen | 8 of 15

★ TOP 75: rank 61 in the Medium and Hard priority list

★ Maximize sum without choosing adjacent elements

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.2 | 30 DAYS 0 | PRIORITY 50/228

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #198 House Robber

LeetCode match: LC 198 House Robber (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/maximize-sum-without-choosing-adjacent-elements

QUESTION

Overview: This question evaluates a candidate's understanding of array-based optimization and dynamic programming concepts for computing a maximum sum under non-adjacent selection constraints.

APPROACH

dp: take = skip_prev + x, skip = max(take_prev, skip_prev); O(n) time, O(1) space.

HOW TO THINK ABOUT IT

At each element you either take it (then the previous one is off limits) or skip it. Only two numbers matter from the past: the best total if you took the previous element and the best if you did not. Update them together and the whole array is one pass with no table.

Basis: LeetCode 198, House Robber: pick elements with no two adjacent to maximise the sum.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

BRUTE FORCE

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION · Two running values (take, skip)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Maximize sum without choosing adjacent
# elements
#
# This question evaluates a candidate's understanding of
# array-based optimization and dynamic programming
# concepts for computing a maximum sum under non-adjacent
# selection constraints.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The houses form a circle (LeetCode 213).
# 2. Return which houses to rob.
# 3. Robbing a house also alerts the next k houses.
#
# INTERVIEW NOTES
# At each element you either take it (then the previous
# one is off limits) or skip it. Only two numbers matter
# from the past: the best total if you took the previous
# element and the best if you did not. Update them
# together and the whole array is one pass with no table.
#
# BRUTE FORCE (say it first): Recursion over take or skip
# At index i either skip it (go to i + 1) or take it and
# jump to i + 2, and keep the larger total. Exact and the
# oracle; the same suffixes are solved over and over. Time
# O(2^n). Space O(n).
```

```python
# BRUTE FORCE (say it first): Recursion over take or skip
# | time O(2^n)
def _house_brute(nums):
    def go(i):
        if i >= len(nums):
            return 0
        return max(go(i + 1), nums[i] + go(i + 2))

    return go(0)


# BASE QUESTION (optimal): Two running values (take, skip)
# Idea: take is the best total that includes the current
# element, skip the best that excludes it. For the next
# element: new take = skip + x (the previous element must
# have been skipped), new skip = max(take, skip).
def rob_line(nums):
    # take: best when the current element is taken,
```

```python
    # skip: best when it is not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)


# FOLLOW-UP 1: The houses form a circle (LeetCode 213).
# Answer: The first and last are adjacent, so you cannot
# take both: the answer is the better of 'ignore the first
# house' and 'ignore the last house', each solved with the
# linear version.
def rob_circle(nums):
    # the first and last are neighbours: ignore one of
    # them
    if len(nums) == 1:
        return nums[0]
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))


# FOLLOW-UP 2: Return which houses to rob.
# Answer: Keep the table dp[i] = best from i onward
# (filled from the right) and walk forward: take house i
# when nums[i] + dp[i + 2] equals dp[i], then jump two;
# otherwise move one.
def rob_with_choice(nums):
    # dp[i] = best from house i onward, filled from the
    # right
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    # walk forward: take house i when it explains dp[i]
    chosen, i = [], 0
    while i < n:
        if nums[i] + dp[i + 2] == dp[i] and nums[i] > 0:
            chosen.append(i)
            i += 2
        else:
            i += 1
    return dp[0], chosen


# FOLLOW-UP 3: Robbing a house also alerts the next k
# houses.
```

```python
# Answer: The recurrence becomes dp[i] = max(dp[i + 1],
# nums[i] + dp[i + k + 1]); the O(1) space trick needs a
# window of k + 1 values (a deque), or just keep the O(n)
# table.
def rob_gap(nums, k):
    # taking a house blocks the next k houses (k = 1 is
    # the
    # classic problem): dp[i] = max(dp[i + 1], nums[i] +
    # dp[i + k + 1])
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]


if __name__ == "__main__":
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0
    assert rob_line([5]) == 5
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([4]) == 4
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]
    values = [2, 7, 9, 3, 1]
    assert sum(values[i] for i in houses) == total
    assert rob_gap([2, 7, 9, 3, 1], 1) == 12
    # blocking two houses after each pick
    assert rob_gap([5, 1, 1, 5, 1, 1, 5], 2) == 15
```

FUNCTION AND TEST CASES

Signature

```python
def solution(nums: list[int]) -> int:
```

Sample call

```python
solution([2, 7, 9, 3, 1])
```

Test cases

Input	Expected
`([2, 7, 9, 3, 1],)`	`12`
`([1, 2, 3, 1],)`	`4`

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: array, dp, medium

MEDIUM | 1Point3Acres | onsite-coding | 9 of 15

Jump Game with Profit

STUDY: TYPE | **ALL TIME 56** | **30 DAYS 0** | **PRIORITY 58/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #55 Jump Game

LeetCode match: LC 55 Jump Game (related)

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/jump-game-with-profit

QUESTION

EXAMPLES

Variant-dependent; the principal recurrence is the one above.

APPROACH

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.
Practice clarifying "can I skip without consuming a turn" before coding.

INTERVIEW-DAY PLAN

CLARIFY FIRST

This is underspecified, so ask: is arr[i] the maximum jump (a cap) or an exact jump, is profit collected on landing or when leaving, can you skip a cell for free, are revisits allowed, and what is the goal (reach the end, or maximize profit)?

BRUTE FORCE TO SAY OUT LOUD

Recursively try every jump from every cell. Exponential.

OPTIMAL IDEA

Pick a reading and say it. Cap reading: best[i] is the best profit standing on i; best[i] = profit[i] + max best[j] over j that can reach i; O(n * max jump), or O(n log n) with a heap and lazy deletion. Exact jump with optional skipping: dp[i] = max(dp[i + 1], profit[i] + dp[i + arr[i]]) from the right, O(n). Plain reachability (no profit) is a greedy farthest-reach scan in O(n).

DRY-RUN THESE CASES

A cell with jump 0, an unreachable end, negative profits, a single cell.

LIKELY FOLLOW-UPS

Minimum jumps (LeetCode 45), profits that can be negative, or a limit on the number of jumps.

PITFALLS

INTERVIEW-DAY PLAN

Treating the jump as a cap when it is exact, or the reverse. Not defining what skipping costs.

HOW TO THINK ABOUT IT

This statement is underspecified, so pin it down first (cap or exact jump, when profit is collected, can you skip, what is the goal). The reading with a maximum jump and profit on landing is a DP over positions: the best profit at i is its own profit plus the best of all positions that can reach it. The reading with an exact jump and a free skip is a one-line DP from the right.

Basis: Variant of LeetCode 55 (Jump Game) and LeetCode 45 (Jump Game II). The prompt is underspecified, so two readings are given; confirm which one with the interviewer.

BRUTE FORCE · Reading A: recursion over every jump length

BRUTE FORCE · PSEUDO-CODE

```
best = -INF
def go(i, total):
    global best
    if i == n - 1:
        best = max(best, total)
        return
    for step in range(1, arr[i] + 1):
        if i + step < n:
            gain = profit[i + step]
            go(i + step, total + gain)
```

TIME	SPACE
O(max jump ^ n)	O(n)

WHY IT IS SLOW

Every cell branches into up to max-jump choices, so the paths multiply (max jump to the power n). A table of the best profit per cell, filled left to right, is O(n * max jump).

BRUTE FORCE · Reading B: recursion over 'jump or skip' choices

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= n:
        return 0
    skip = best(i + 1)
    jump = 0
    if arr[i] > 0:
        jump = profit[i] + best(i + arr[i])
    return max(skip, jump)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

BRUTE FORCE · Reading B: recursion over 'jump or skip' choices

Two branches at every cell and the same suffix is solved repeatedly: O(2^n). Computing best(i) once per index from the right is O(n).

OPTIMAL SOLUTION · Reading A: arr[i] is a maximum jump, profit[i] is gained on landing, maximize total profit to the last index

TIME	SPACE
O(n * maxJump)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Reading B: arr[i] is an exact jump; at each index you may jump (gaining profit[i]) or skip one cell for free

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Speed up reading A to near-linear time.

The inner loop asks for the max of best[j] over all j that can still reach i. Maintain a max-heap of (best[j], j + arr[j]) and pop entries whose reach is below i (lazy deletion): O(n log n). If the jump limit is the same for all cells, a monotonic deque over a sliding window makes it O(n).

FOLLOW-UP 2 What if you want to know whether the end is reachable at all (plain Jump Game)?

Greedy: scan left to right keeping the farthest reachable index; if i exceeds it you are stuck. O(n) time, O(1) space. Use the DP when profits matter, and the greedy when only reachability matters.

FOLLOW-UP 3 Only reachability, no profit (LeetCode 55).

Greedy: keep the farthest index reachable so far; if the current index is beyond it you are stuck. O(n), O(1).

FOLLOW-UP 4 Make the profit DP faster.

best[i] needs the maximum of best[j] over j that can reach i. A max-heap with lazy deletion of positions that can no longer reach i makes it O(n log n); with a constant maximum jump a monotonic deque gives O(n).

FOLLOW-UP 5 Fewest jumps (LeetCode 45).

BFS by levels or a greedy that tracks the end of the current jump range and the farthest reach, O(n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Jump Game with Profit
#
# Examples: Variant-dependent; the principal recurrence is
# the one above.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Speed up reading A to near-linear time.
# 2. What if you want to know whether the end is reachable
# at all (plain Jump Game)?
# 3. Only reachability, no profit (LeetCode 55).
# 4. Make the profit DP faster.
# 5. Fewest jumps (LeetCode 45).
#
# INTERVIEW NOTES
# This statement is underspecified, so pin it down first
# (cap or exact jump, when profit is collected, can you
# skip, what is the goal). The reading with a maximum jump
# and profit on landing is a DP over positions: the best
# profit at i is its own profit plus the best of all
# positions that can reach it. The reading with an exact
# jump and a free skip is a one-line DP from the right.
#
# BRUTE FORCE (say it first): Reading A: recursion over
# every jump length
# From each cell try every allowed jump length and add the
# profit of the landing cell; take the best total that
# reaches the last index. Exact and the oracle, and
# exponential. Time O(max jump ^ n). Space O(n).


import heapq


# HELPERS for the brute force
NEG = float("-inf")


# SLOWER VERSION (say it first)
def max_profit_slow(arr, profit):
    # best[i]: most profit when landing on i (arr[j] is
    # the
    # farthest jump from j); O(n^2)
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
```

```
    for i in range(1, n):
        for j in range(i):
            # can j reach i? then landing on i is possible
            if best[j] != NEG and j + arr[j] >= i:
                total = best[j] + profit[i]
                best[i] = max(best[i], total)
    return -1 if best[-1] == NEG else best[-1]


# BASE QUESTION (optimal): Reading A: arr[i] is a maximum
# jump, profit[i] is gained on landing, maximize total
# profit to the last index
# Idea: Let best[i] be the largest profit achievable when
# standing on i. Landing on i from any j < i with j +
```

```python
# arr[j] >= i gives best[i] = profit[i] + max(best[j]).
# Unreachable cells stay at minus infinity.
def max_profit_fast(arr, profit):
    # a max-heap of (-best, reach) with lazy deletion
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    heap = [(-best[0], arr[0])]
    for i in range(1, n):
        # forget cells that can no longer reach i
        while heap and heap[0][1] < i:
            heapq.heappop(heap)
        if heap:
            best[i] = -heap[0][0] + profit[i]
            heapq.heappush(heap, (-best[i], i + arr[i]))
    return -1 if best[-1] == NEG else best[-1]


# VARIANT: reachability only (LeetCode 55)
# FOLLOW-UP 3: Only reachability, no profit (LeetCode 55).
# Answer: Greedy: keep the farthest index reachable so
# far; if the current index is beyond it you are stuck.
# O(n), O(1).
def can_reach_end(arr):
    # LeetCode 55: only reachability, greedy O(n)
    farthest = 0
    for i, jump in enumerate(arr):
        if i > farthest:
            return False
        farthest = max(farthest, i + jump)
    return True
```

```python
# VARIANT: fewest jumps (LeetCode 45)
# FOLLOW-UP 5: Fewest jumps (LeetCode 45).
# Answer: BFS by levels or a greedy that tracks the end of
# the current jump range and the farthest reach, O(n).
def fewest_jumps(arr):
    # LeetCode 45: levels of a BFS, greedy O(n)
    jumps = end = farthest = 0
    for i in range(len(arr) - 1):
        farthest = max(farthest, i + arr[i])
        if i == end:
            # we must jump now: extend to the farthest
            # reach
            jumps += 1
            end = farthest
    return jumps


if __name__ == "__main__":
    arr, profit = [2, 3, 1, 1, 4], [1, 5, 2, 3, 4]
    # best path: 0 -> 1 -> 2 -> 3 -> 4 = 15
    assert max_profit_slow(arr, profit) == 15
    assert max_profit_fast(arr, profit) == 15
    # the end cannot be reached
    assert max_profit_slow([1, 0, 1], [1, 1, 1]) == -1
    assert max_profit_fast([1, 0, 1], [1, 1, 1]) == -1
    assert can_reach_end(arr)
    assert not can_reach_end([1, 0, 1])
    assert fewest_jumps(arr) == 2
    # the fast version agrees with the slow one
    import itertools
    for jumps in itertools.product(range(4), repeat=5):
        pr = [1, 2, 3, 2, 1]
        assert max_profit_slow(jumps, pr) == (
            max_profit_fast(jumps, pr))
```

NOTES FROM CANDIDATES

Standard intern coding round, single source — clarification with interviewer is key (does "skip" advance by 1 or 0? are revisits allowed?).
Common bug: treating arr[i] as range cap rather than exact jump.

PREPARATION

Drill the reachability-jump, min-jumps, and coin-change family together so the DP shape is automatic.
Practice clarifying "can I skip without consuming a turn" before coding.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: grid, dp, medium

MEDIUM | 1Point3Acres | onsite-coding | 10 of 15

Count Same-Color Squares in Matrix

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 0** | **PRIORITY 68/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #221 Maximal Square

LeetCode match: LC 221 Maximal Square (related)

Frequency: Low | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/count-same-color-squares

QUESTION

LeetCode 221 variant — count maximal same-color square sub-grids. DP on each cell as the bottom-right corner.

Input: m × n grid of colored cells.
Output: count of square sub-grids whose cells are all the same color (or maximum side length, depending on prompt).
DP: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if grid[i][j] == grid[i-1][j] == grid[i][j-1] == grid[i-1][j-1] else 1.
Sum dp[i][j] across the grid for the count variant; max for the side variant.

EXAMPLES

The plain "count all-1 squares" DP generalizes to same-color directly.

APPROACH

Write the all-1-squares DP from memory in under 8 min. Generalize to multi-color in <2 min — only the equality check changes.

HOW TO THINK ABOUT IT

Let dp[i][j] be the side of the largest same-colour square whose bottom-right corner is (i, j). That square exists only if the three neighbours above, left and above-left have the same colour as the cell and can themselves carry a square, so dp is 1 plus the smallest of their dp values. Summing dp over all cells counts every square once (each cell contributes one square per side length up to dp).

Basis: LeetCode 221, Maximal Square, and LeetCode 1277, Count Square Submatrices with All Ones, generalised from 1s to any one colour.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for i, j in all_cells(grid):
    for k in range(1, min(m - i, n - j) + 1):
        cells = square(grid, i, j, k)
        if all(c == grid[i][j] for c in cells):
            count += 1
```

TIME	SPACE
O(m * n * min(m, n)^3)	O(1)

WHY IT IS SLOW

Re-checks the same cells again and again, O(m * n * min(m, n)^3). A dp on the bottom-right corner reuses the neighbours' results in O(m * n).

OPTIMAL SOLUTION · dp on the bottom-right corner

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only the largest side instead of the count.

Keep the same dp and take the maximum instead of adding. The common bug is checking only the diagonal neighbour: the up and left neighbours must also match, otherwise a different colour can sit inside the square.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Same-Color Squares in Matrix
#
# LeetCode 221 variant — count maximal same-color square
# sub-grids. DP on each cell as the bottom-right corner.
# Input: m × n grid of colored cells. Output: count of
# square sub-grids whose cells are all the same color (or
# maximum side length, depending on prompt). DP: dp[i][j]
# = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if
# grid[i][j] == grid[i-1][j] == grid[i][j-1] ==
# grid[i-1][j-1] else 1. Sum dp[i][j] across the grid for
# the count variant; max for the side variant. Examples:
# The plain "count all-1 squares" DP generalizes to same-
# color directly.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only the largest side instead of the count.
#
# INTERVIEW NOTES
# Let dp[i][j] be the side of the largest same-colour
# square whose bottom-right corner is (i, j). That square
# exists only if the three neighbours above, left and
# above-left have the same colour as the cell and can
```

```
# themselves carry a square, so dp is 1 plus the smallest
# of their dp values. Summing dp over all cells counts
# every square once (each cell contributes one square per
# side length up to dp).
#
# BRUTE FORCE (say it first): Check every square directly
# For every corner and every side length look at all cells
# in the square. Correct; far too slow for a large grid.
# Time O(m * n * min(m, n)^3). Space O(1).


# BRUTE FORCE (say it first): Check every square directly
# | time O(m * n * min(m, n)^3)
def count_squares_brute(g):
    m, n = len(g), len(g[0]) if g else 0
    total = 0
    for i in range(m):
        for j in range(n):
            for k in range(1, min(m - i, n - j) + 1):
                if all(
                    g[a][b] == g[i][j]
                    for a in range(i, i + k)
                    for b in range(j, j + k)
                ):
                    total += 1
    return total


# HELPERS for the base question
def square_sides(g):
    # side[i][j]: the biggest same-colour square whose
    # bottom-right corner is (i, j); one row at a time
    if not g:
        return []
    prev, out = [0] * len(g[0]), []
    for i, row in enumerate(g):
        cur = [1] * len(row)
        for j, c in enumerate(row):
            # all three neighbours must match this cell
            if i and j and g[i - 1][j] == c \
                    and g[i][j - 1] == c \
                    and g[i - 1][j - 1] == c:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        out.append(cur)
        prev = cur
    return out


# BASE QUESTION (optimal): dp on the bottom-right corner
# Idea: dp[j] holds the best square side ending in the
# current row at column j.
def count_same_color_squares(g):
    # every cell contributes one square per side up to its
    # side
    return sum(sum(row) for row in square_sides(g))


# FOLLOW-UP 1: Return only the largest side instead of the
# count.
# Answer: Keep the same dp and take the maximum instead of
# adding. The common bug is checking only the diagonal
# neighbour: the up and left neighbours must also match,
# otherwise a different colour can sit inside the square.
def largest_side(g):
    # the variant that asks for the biggest square only
    return max((max(row) for row in square_sides(g)),
               default=0)
```

```
if __name__ == "__main__":
    # a 3 x 3 block of one colour: 9 + 4 + 1 squares
    block = ["RRR", "RRR", "RRR"]
    assert count_same_color_squares(block) == 14
    assert largest_side(block) == 3
    # a different colour inside blocks the bigger squares
    holes = ["RRR", "RGR", "RRR"]
    assert largest_side(holes) == 1
    assert count_same_color_squares(holes) == 9
    assert count_same_color_squares([]) == 0
    assert largest_side([]) == 0
```

NOTES FROM CANDIDATES

Variant reported in both intern VO and an additional NG round. Common bug: assuming all four corners equal is sufficient — need all three neighbors (top, left, top-left) to match the current cell.

PREPARATION

Write the all-1-squares DP from memory in under 8 min. Generalize to multi-color in <2 min — only the equality check changes.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: array, dp, binary-search, sorting, medium

MEDIUM | IN APP LC 354 | 1Point3Acres | phone-screen | 11 of 15

Russian Doll Envelopes

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 33 | PRIORITY 28/228

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: your app's interview approach (LeetCode #354)

LeetCode match: LC 354 Russian Doll Envelopes

Frequency: Single report | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

QUESTION

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction.

Input: list of (width, height) envelopes.
Output: longest chain where each envelope strictly fits in the next.
Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights.
LIS with patience-sort gives O(n log n).

EXAMPLES

```
[[5,4],[6,4],[6,7],[2,3]] -> 3.
```

APPROACH

Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

HOW TO THINK ABOUT IT

Sort by width ascending so that only the heights matter, then find the longest strictly increasing subsequence of heights. Envelopes with equal width cannot nest, so sort those by height descending: then a strictly increasing run of heights can never use two envelopes of the same width.

Basis: LeetCode 354, Russian Doll Envelopes.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each envelope is compared with all earlier ones, O(n^2). After sorting, a longest increasing subsequence of the heights with binary search is O(n log n).

OPTIMAL SOLUTION · Sort, then LIS with binary search

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why sort heights descending when widths tie?

If two envelopes have the same width they cannot nest, but an ascending height order would let the LIS pick both. Descending order makes a strictly increasing height run use at most one envelope per width.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Russian Doll Envelopes
#
# LeetCode 354. Maximum chain of envelopes that strictly
# fit inside each other. Sort + LIS reduction. Input: list
# of (width, height) envelopes. Output: longest chain
# where each envelope strictly fits in the next. Trick:
# sort by width ascending, height descending (when widths
# tie); run LIS (Longest Increasing Subsequence) on
# heights. LIS with patience-sort gives O(n log n).
# Examples: [[5,4],[6,4],[6,7],[2,3]] -> 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why sort heights descending when widths tie?
#
# INTERVIEW NOTES
# Sort by width ascending so that only the heights matter,
# then find the longest strictly increasing subsequence of
# heights. Envelopes with equal width cannot nest, so sort
# those by height descending: then a strictly increasing
# run of heights can never use two envelopes of the same
# width.
#
```

```python
# BRUTE FORCE (say it first): O(n^2) DP over sorted
# envelopes
# Sort by width and for each envelope take the best chain
# of an earlier envelope that is strictly smaller in both
# dimensions. Correct; quadratic is too slow for 10^5
# envelopes. Time O(n^2). Space O(n).


from bisect import bisect_left


# CHECKER: the O(n^2) version
def max_envelopes_slow(envelopes):
    # O(n^2) check used for testing
    ordered = sorted(envelopes)
    best = [1] * len(ordered)
    for i in range(len(ordered)):
        for j in range(i):
            wj, hj = ordered[j]
            wi, hi = ordered[i]
            if wj < wi and hj < hi:
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)
```

```python
# BRUTE FORCE (say it first): O(n^2) DP over sorted
# envelopes | time O(n^2)
def doll_brute(envs):
    envs = sorted(envs)
    best = [1] * len(envs)
    for i in range(len(envs)):
        for j in range(i):
            if (
                envs[j][0] < envs[i][0]
                and envs[j][1] < envs[i][1]
            ):
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)


# BASE QUESTION (optimal): Sort, then LIS with binary
# search
# Idea: Sort by (width ascending, height descending).
def max_envelopes(envelopes):
    # sort by width; for equal widths put TALLER first so
    # a
    # strictly increasing height run can use only one of
    # them
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    # tails[k]: smallest ending height of a chain of k + 1
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)


if __name__ == "__main__":
    envs = [[5, 4], [6, 4], [6, 7], [2, 3]]
    assert max_envelopes(envs) == 3
    assert max_envelopes([]) == 0
    # equal widths cannot nest, however the heights go
    assert max_envelopes([[1, 1], [1, 2], [1, 3]]) == 1
    assert max_envelopes([[4, 5], [4, 6], [6, 7], [2, 3],
```

```python
                          [1, 1]]) == 4
    import itertools
    pts = [(w, h) for w in range(1, 4)
           for h in range(1, 4)]
    for combo in itertools.combinations(pts, 4):
        assert max_envelopes(list(combo)) == (
            max_envelopes_slow(list(combo)))
```

NOTES FROM CANDIDATES

Sorting heights descending on width-ties prevents same-width envelopes from chaining — easy to miss.
LIS via bisect_left on a tails array is the standard O(n log n) implementation.

PREPARATION

Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

MEDIUM | HackTheRounds | Onsite | 12 of 15

Reverse Coin Change

STUDY: TALK | ALL TIME 37.9 | 30 DAYS 0 | PRIORITY 100/228

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #518 Coin Change II

LeetCode match: Inverse of LC 518

QUESTION

In standard coin change, given a coins array, we build dp where dp[i] = number of ways to make amount i. Reverse problem: given a dp array, find the original coins array that produced it.

EXAMPLES

Example 1	
Input	`dp = [1, 1, 2, 2, 3]`
Output	`[1, 2]`
Why	`Coins [1, 2] produce this dp array.`

Example 2	
Input	`dp = [1, 1, 1, 2, 2, 2, 3]`
Output	`[1, 3]`

CONSTRAINTS

- 1 <= dp.length <= 1000
- dp[0] = 1

APPROACH

Rebuild coins left to right: the first amount whose dp exceeds the dp built from coins so far is a new coin

HOW TO THINK ABOUT IT

Think of building the ways table forward: adding a coin c changes dp[c] by exactly one (the single coin c). So scanning amounts upward, the first amount where the target has more ways than your table can only be explained by a new coin of exactly that value. Add it, rebuild, and continue; at the end compare with the target to detect an impossible input.

Basis: The inverse of LeetCode 518, Coin Change II (number of ways to make each amount with unlimited coins).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(dp):
    n = len(dp) - 1
    for k in range(n + 1):
        values = range(1, n + 1)
        for coins in combinations(values, k):
            if ways_table(coins, n) == dp:
                return list(coins)
    return None
```

TIME	SPACE
O(2^n * n * coins)	O(n)

WHY IT IS SLOW

2^n candidate sets, each with an O(n * coins) table. Scanning amounts upward and adding a coin whenever the target has one more way than the table is O(n^2).

OPTIMAL SOLUTION · Scan upward and add a coin whenever the table falls short

TIME	SPACE
O(n * coins) = O(n^2) worst case	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The array might not come from any set of coins.

Return None (or raise) when dp[0] != 1, when an amount needs more than one extra way, or when the rebuilt table differs from the input at the end; the code does all three checks.

FOLLOW-UP 2 Order of the coins matters (the dp counts permutations, not combinations).

That is a different recurrence: dp[a] = sum of dp[a - c] over coins. Inverting it is a linear-algebra style problem: at each amount the new coins explain the excess; ask which counting is used.

FOLLOW-UP 3 Each coin may be used at most once.

The forward recurrence iterates amounts downward. The inverse still scans upward adding a coin when the target exceeds the table, but the table update must use the 0/1 form.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Reverse Coin Change
#
# In standard coin change, given a coins array, we build
# dp where dp[i] = number of ways to make amount i.
# Reverse problem: given a dp array, find the original
# coins array that produced it.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The array might not come from any set of coins.
# 2. Order of the coins matters (the dp counts
# permutations, not combinations).
# 3. Each coin may be used at most once.
#
# INTERVIEW NOTES
# Think of building the ways table forward: adding a coin
# c changes dp[c] by exactly one (the single coin c). So
# scanning amounts upward, the first amount where the
# target has more ways than your table can only be
# explained by a new coin of exactly that value. Add it,
# rebuild, and continue; at the end compare with the
# target to detect an impossible input.
#
# BRUTE FORCE (say it first): Try every subset of coin
# values 1..n
# Enumerate each subset of {1..n} as a candidate coin set,
# build its ways table and compare with the target. Exact,
# and the oracle for the fast method. Time O(2^n * n *
# coins). Space O(n).


# HELPERS for the brute force
def ways_table(coins, n):
    # forward: dp[a] = number of combinations making a
    dp = [1] + [0] * n
    for c in coins:
        for a in range(c, n + 1):
            dp[a] += dp[a - c]
    return dp


# BRUTE FORCE (say it first): Try every subset of coin
# values 1..n | time O(2^n * n * coins)
def coins_from_dp_brute(dp):
    n = len(dp) - 1
    for mask in range(1 << n):
        coins = [
            i + 1 for i in range(n) if mask >> i & 1
        ]
        if ways_table(coins, n) == list(dp):
            return coins
    return None


# BASE QUESTION (optimal): Scan upward and add a coin
# whenever the table falls short
# Idea: Keep the ways table for the coins found so far.
# FOLLOW-UP 1: The array might not come from any set of
# coins.
# Answer: Return None (or raise) when dp[0] != 1, when an
# amount needs more than one extra way, or when the
# rebuilt table differs from the input at the end; the
# code does all three checks.
def coins_from_dp(dp):
    # backward: rebuild the table with the coins found so
    # far. The first amount where the real dp is exactly
    # one
    # bigger must be a new coin (it adds the single way
    # "just that coin"). Anything else is impossible.
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    cur, coins = [1] + [0] * n, []
    for a in range(1, n + 1):
        if dp[a] == cur[a]:
            continue
        if dp[a] != cur[a] + 1:
            return None
        coins.append(a)
        for x in range(a, n + 1):
            cur[x] += cur[x - a]
    # the table rebuilt from the coins must equal the
    # input
    return coins if cur == list(dp) else None


# FOLLOW-UP 2: Order of the coins matters (the dp counts
# permutations, not combinations).
# Answer: That is a different recurrence: dp[a] = sum of
# dp[a - c] over coins. Inverting it is a linear-algebra
# style problem: at each amount the new coins explain the
# excess; ask which counting is used.
def ordered_table(coins, n):
    # follow-up: the order of coins matters
    # (permutations):
    # dp[a] = sum of dp[a - c] over the coins
    dp = [1] + [0] * n
    for a in range(1, n + 1):
        dp[a] = sum(dp[a - c] for c in coins if c <= a)
    return dp


# FOLLOW-UP 2: Order of the coins matters (the dp counts
# permutations, not combinations).
# Answer: That is a different recurrence: dp[a] = sum of
# dp[a - c] over coins. Inverting it is a linear-algebra
# style problem: at each amount the new coins explain the
# excess; ask which counting is used.
def coins_from_ordered(dp):
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    coins = []
    for a in range(1, n + 1):
        # what the known coins already explain at amount a
        known = sum(dp[a - c] for c in coins)
        if dp[a] == known + 1:
            coins.append(a)
        elif dp[a] != known:
            return None
    again = ordered_table(coins, n)
    return coins if again == list(dp) else None


# FOLLOW-UP 3: Each coin may be used at most once.
# Answer: The forward recurrence iterates amounts
# downward. The inverse still scans upward adding a coin
# when the target exceeds the table, but the table update
# must use the 0/1 form.
def once_table(coins, n):
    # follow-up: each coin used at most once; amounts are
    # visited downward in the forward step
    dp = [1] + [0] * n
    for c in coins:
        for a in range(n, c - 1, -1):
            dp[a] += dp[a - c]
```

```
    return dp


# FOLLOW-UP 3: Each coin may be used at most once.
# Answer: The forward recurrence iterates amounts
# downward. The inverse still scans upward adding a coin
# when the target exceeds the table, but the table update
# must use the 0/1 form.
def coins_from_once(dp):
    n = len(dp) - 1
    if not dp or dp[0] != 1:
        return None
    cur, coins = [1] + [0] * n, []
    for a in range(1, n + 1):
        if dp[a] == cur[a]:
            continue
        if dp[a] != cur[a] + 1:
            return None
        coins.append(a)
        for x in range(n, a - 1, -1):
            cur[x] += cur[x - a]
    return coins if cur == list(dp) else None


if __name__ == "__main__":
    assert coins_from_dp([1, 1, 2, 2, 3]) == [1, 2]
    assert coins_from_dp([1, 1, 1, 2, 2, 2, 3]) == [1, 3]
    assert coins_from_dp([1]) == []
    assert coins_from_dp([]) is None
    # not a table: dp[0] must be 1
    assert coins_from_dp([2, 1]) is None
    # an amount that jumps by more than one new way
    assert coins_from_dp([1, 3]) is None
    # looks fine early, but the whole table does not match
    assert coins_from_dp([1, 1, 2, 2, 5]) is None
    table = ordered_table([1, 2], 6)
    assert coins_from_ordered(table) == [1, 2]
    assert coins_from_ordered([1, 1, 2, 3, 5]) == [1, 2]
    table = once_table([1, 2, 3], 6)
    assert coins_from_once(table) == [1, 2, 3]
    import random
    for _ in range(300):
        size = random.randint(0, 4)
        coins = sorted(random.sample(range(1, 10), size))
        n = random.randint(0, 14)
```

```
        small = [c for c in coins if c <= n]
        table = ways_table(coins, n)
        assert coins_from_dp(table) == small
        table = ordered_table(coins, n)
        assert coins_from_ordered(table) == small
        table = once_table(coins, n)
        assert coins_from_once(table) == small
        junk = [1]
        junk += [random.randint(0, 3) for _ in range(n)]
        found = coins_from_dp(junk)
        if found is not None:
            assert ways_table(found, n) == junk
```

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: dp, bit-manipulation, hard

HARD | 1Point3Acres | phone-screen, onsite-coding | 13 of 15

Count Perfect Wake Numbers up to N

ASKED 2026-05-07 | 166 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 187/228**

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-05-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/perfect-wake-number-count

QUESTION

EXAMPLES

```
196   -> perfect (digits 1,9,6 all distinct, no 0, 9
    is not a valley)
23    -> perfect
12463 -> perfect (1<2, 2<4, 4>6? No, 4<6, 6>3 — 6 is
    not a valley because its right neighbor 3 is
    smaller)
1546  -> not perfect (4 is adjacent to 5 and 6, both
    strictly larger)
320   -> not perfect (contains 0)
34321 -> not perfect (digit 3 repeats)
```

APPROACH

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate." Practice bitmask-over-digits + tight-flag state machines. Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact definition: no zero digit, all digits distinct, no interior digit smaller than both neighbors. Ask for the range of N and whether to count 1..N inclusive.

BRUTE FORCE TO SAY OUT LOUD

Test every number from 1 to N with a digit check. O(N * d), fine for small N.

OPTIMAL IDEA

Digit DP over positions with state (used-digit mask, last digit, whether the digits have started falling, tight flag). Numbers with fewer digits than N are counted without a tight constraint, and zero is never allowed, so there are no leading-zero cases. Digits must rise then fall, which is exactly the no-valley rule. There is also a closed form of C(9, k) * 2^(k - 1) for k-digit numbers.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

196, 23, 12463 perfect; 1546, 320, 34321 not perfect. N = 9, N = 10, N a power of ten.

LIKELY FOLLOW-UPS

A range [A, B] (inclusion-exclusion), or different digit rules.

PITFALLS

Forgetting the zero rule. Letting a digit repeat across the tight and free branches.

HOW TO THINK ABOUT IT

Distinct digits with no zero and no valley means the digits go up and then down (they can only change direction once). That structure is both a closed form for each length and a state for a digit DP: position, which digits are used, the last digit, and whether the digits have started falling. The tight flag handles the bound N.

Basis: Digit DP, like LeetCode 1012 (Numbers With Repeated Digits) and 233. Definition from the report: no zero digit, all digits distinct, and no interior digit is a valley (strictly smaller than both neighbours).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_perfect(x):
    d = list(map(int, str(x)))
    if 0 in d or len(set(d)) < len(d):
        return False
    for i in range(1, len(d) - 1):
        if d[i] < d[i - 1] and d[i] < d[i + 1]:
            return False  # a valley
    return True
count = 0
for x in range(1, N + 1):
    count += is_perfect(x)
```

TIME	SPACE
O(N * d)	O(1)

WHY IT IS SLOW

O(N * digits): hopeless for N around 10^9. A digit DP counts by position, used digits, last digit and direction in time independent of N's size.

OPTIMAL SOLUTION · Part 1: test a single number

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Part 2: digit DP with tight flag, used-digit mask, last digit and direction

TIME	SPACE
O(d * 2^9 * 9 * 9)	O(2^9 * 9)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the DP only need a 'falling' flag instead of the full history?

A valley needs a digit that is lower than the digit before it and then higher than the one after it. Once any digit has decreased, a later increase would create exactly that valley, so the only information to carry is whether we have already gone down. The used mask and last digit cover the other two rules.

FOLLOW-UP 2 How do you count the numbers in a range [A, B]?

Use inclusion-exclusion: count(B) - count(A - 1). The DP is a function of the upper bound only, and it is cheap enough (a few thousand states) to call twice.

FOLLOW-UP 3 What is the closed form without a DP?

Since digits are distinct and unimodal, pick which k digits (from 1..9) appear and the peak is the largest; each of the other k - 1 digits goes either left or right of the peak: 2^(k-1) arrangements per set. So the count of perfect numbers with k digits is C(9, k) * 2^(k-1). Only the 'at most N' bound needs the DP.

FOLLOW-UP 4 Is there a closed form for k-digit numbers?

Pick the k distinct digits from 1..9 (C(9, k) ways); the largest must be the peak, and each of the other k - 1 digits goes either left or right of it in sorted order, so 2^(k-1) arrangements: C(9, k) * 2^(k-1). Only the bound N needs the DP.

FOLLOW-UP 5 Count perfect numbers in a range [A, B].

Compute count(B) - count(A - 1); the DP depends on the upper bound only.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Perfect Wake Numbers up to N
#
# Examples: 196 -> perfect (digits 1,9,6 all distinct, no
# 0, 9 is not a valley) 23 -> perfect 12463 -> perfect (1<
2,
# 2<4, 4>6? No, 4<6, 6>3 — 6 is not a valley because its
# right neighbor 3 is smaller) 1546 -> not perfect (4 is
# adjacent to 5 and 6, both strictly larger) 320 -> not
# perfect (contains 0) 34321 -> not perfect (digit 3
# repeats)
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the DP only need a 'falling' flag instead of
# the full history?
# 2. How do you count the numbers in a range [A, B]?
# 3. What is the closed form without a DP?
# 4. Is there a closed form for k-digit numbers?
# 5. Count perfect numbers in a range [A, B].
#
# INTERVIEW NOTES
# Distinct digits with no zero and no valley means the
# digits go up and then down (they can only change
# direction once). That structure is both a closed form
# for each length and a state for a digit DP: position,
# which digits are used, the last digit, and whether the
# digits have started falling. The tight flag handles the
# bound N.
#
# BRUTE FORCE (say it first): Test every number from 1 to
# N
# Apply the three rules to each number. Correct and the
# oracle; it is hopeless for N around 10^9. Time O(N * d).
# Space O(1).


from functools import lru_cache
from math import comb


# BRUTE FORCE (say it first): Test every number from 1 to
# N | time O(N * d)
def count_perfect_brute(N):
    return sum(
        1 for x in range(1, N + 1) if is_perfect(x)
    )
```

```
# HELPERS for the base question
# FOLLOW-UP 4: Is there a closed form for k-digit numbers?
# Answer: Pick the k distinct digits from 1..9 (C(9, k)
# ways); the largest must be the peak, and each of the
# other k - 1 digits goes either left or right of it in
# sorted order, so 2^(k-1) arrangements: C(9, k) *
# 2^(k-1).
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def with_k_digits(k):
    # choose the k digits; the largest is the peak and
    # each
    # other digit goes left or right of it
    return comb(9, k) * 2 ** (k - 1) if k >= 1 else 0


# BASE QUESTION (optimal): test a single number
# Idea: Check the three rules directly on the digit list:
# no '0', all digits distinct, and for each interior digit
# not (left > digit < right). The first and last digits
# are never valleys.
def is_perfect(n):
    # distinct digits, no zero, and no valley: no digit is
    # smaller than both of its neighbours
    d = list(map(int, str(n)))
    if 0 in d or len(set(d)) != len(d):
        return False
    return all(not (d[i - 1] > d[i] < d[i + 1])
               for i in range(1, len(d) - 1))
```

```
# PART 2: count the perfect numbers up to N
# FOLLOW-UP 2: How do you count the numbers in a range [A,
# B]?
# Answer: Use inclusion-exclusion: count(B) - count(A -
# 1). The DP is a function of the upper bound only, and it
# is cheap enough (a few thousand states) to call twice.
def count_up_to(n):
    if n < 1:
        return 0
    digits = list(map(int, str(n)))
    size = len(digits)
    # all shorter numbers: closed form
    total = sum(with_k_digits(k) for k in range(1, size))
```

```
    @lru_cache(None)
    def go(pos, mask, last, falling, tight):
        # count completions of a number of full length
        if pos == size:
            return 1
        out = 0
        limit = digits[pos] if tight else 9
        for d in range(1, limit + 1):
            if mask >> d & 1:
                continue
            fall = falling
            if pos:
                if d > last and falling:
                    # going up after going down: a valley
                    continue
                if d < last:
                    fall = True
            out += go(pos + 1, mask | 1 << d, d, fall,
                      tight and d == digits[pos])
        return out

    # numbers with the same length as n, not above n
    return total + go(0, 0, 0, False, True)


# FOLLOW-UP 5: Count perfect numbers in a range [A, B].
# Answer: Compute count(B) - count(A - 1); the DP depends
# on the upper bound only.
def count_between(a, b):
    # inclusion-exclusion over the upper bounds
    return count_up_to(b) - count_up_to(a - 1)


if __name__ == "__main__":
    for ok in (196, 23, 12463):
        assert is_perfect(ok)
    for bad in (1546, 320, 34321):
        assert not is_perfect(bad)
    # the DP agrees with checking every number
    for n in (0, 9, 10, 99, 100, 999, 5000, 20000):
        every = range(1, n + 1)
        brute = sum(is_perfect(i) for i in every)
        assert count_up_to(n) == brute
    assert count_between(10, 99) == (
```

```
        count_up_to(99) - count_up_to(9))
    # 1-digit and 2-digit counts from the closed form
    assert with_k_digits(1) == 9
    assert with_k_digits(2) == 72
```

NOTES FROM CANDIDATES

Part 1 is a linear scan; Part 2 is digit DP with state (position, mask of used digits, last digit, is_tight against n). Track whether the previous step went up or down to detect a future valley. Multiple candidates report not finishing Part 2 in the 45-minute window. The interviewer is generally patient and dispenses hints ("think DP" / "think digit DP") when you stall.
Watch out: the "no valley" rule is about interior digits, so the first and last digit are never themselves the valley; they can still create one for their neighbor.
The full prompt is short and recent — expect the follow-up to vary (e.g. count exactly-K-digit Perfect Wake Numbers).

PREPARATION

Drill digit DP templates: classic digit-DP problems — "count digit one", "non-negative integers without consecutive ones", "numbers at most N given digit set", "numbers with repeated digits" — cover "count integers <= n satisfying digit predicate." Practice bitmask-over-digits + tight-flag state machines. Memorize the standard skeleton: recurse over positions, accept any digit <= n's digit when tight, else any digit 1..9.

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

Site tags: tree, dp, dfs, hard

HARD | IN APP LC 834 | 1Point3Acres | onsite-coding | 14 of 15

Tree Distance Sum (Re-root DP)

ASKED 2026-01-19 | 274 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 124/228

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: your app's interview approach (LeetCode #834)

LeetCode match: LC 834 Sum of Distances in Tree

Frequency: Single report | Last asked: 2026-01-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tree-distance-sum-reroot

QUESTION

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Interviewer expects baseline O(n²) DFS followed by the O(n) re-root DP.

Input: undirected tree with N nodes and N-1 edges.
Output: array ans where ans[u] is the sum of shortest-path distances from u to every other node.
Walk through baseline (BFS/DFS from every node, O(n²)) before optimizing.
Final solution should be O(n) via two-pass re-root DP.

EXAMPLES

```
Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]]
ans:   [8, 12, 6, 10, 10, 10]
```

APPROACH

Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes. Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.
Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

HOW TO THINK ABOUT IT

Running a BFS from every node costs O(n^2). Notice that moving the root from a parent p to its child v changes every distance by exactly one: the nodes inside v's subtree get closer by 1, all the others get farther by 1. So compute the answer for one root, then derive each child's answer from its parent's in O(1) using the subtree sizes.

Basis: LeetCode 834, Sum of Distances in Tree.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, adj):
    ans = [0] * n
    for s in range(n):
        dist = bfs(adj, s)
        ans[s] = sum(dist)
    return ans
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

n BFS runs of O(n) each, O(n^2). Moving the root across one edge changes every distance by exactly 1, so two passes over the tree give all answers in O(n).

OPTIMAL SOLUTION · Two passes: subtree sizes, then re-root

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the re-root formula correct?

Compare the distance from every node w to p and to its child v. If w is inside v's subtree the path to p goes through v, so dist(w, p) = dist(w, v) + 1. If w is outside it, dist(w, v) = dist(w, p) + 1. There are size[v] nodes of the first kind and n - size[v] of the second, so ans[v] = ans[p] - size[v] + (n - size[v]).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Tree Distance Sum (Re-root DP)
#
# Onsite coding round 1: given an undirected tree on N
# nodes, return for every node the sum of distances to all
# other nodes. Equivalent to LC 834. Interviewer expects
# baseline O(n²) DFS followed by the O(n) re-root DP.
# Input: undirected tree with N nodes and N-1 edges.
# Output: array ans where ans[u] is the sum of shortest-
# path distances from u to every other node. Walk through
# baseline (BFS/DFS from every node, O(n²)) before
# optimizing. Final solution should be O(n) via two-pass
# re-root DP. Examples: Edges:
# [[0,1],[0,2],[2,3],[2,4],[2,5]] ans: [8, 12, 6, 10, 10,
# 10]
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the re-root formula correct?
#
# INTERVIEW NOTES
# Running a BFS from every node costs O(n^2). Notice that
# moving the root from a parent p to its child v changes
```

```python
# every distance by exactly one: the nodes inside v's
# subtree get closer by 1, all the others get farther by
# 1. So compute the answer for one root, then derive each
# child's answer from its parent's in O(1) using the
# subtree sizes.
#
# BRUTE FORCE (say it first): BFS from every node
# Run a BFS from each node and add up the distances.
# Correct and the right baseline to state in the first
# minutes; quadratic is too slow for 10^5 nodes. Time
# O(n^2). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): BFS from every node | time
# O(n^2)
def tree_sums_brute(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    out = []
```

```python
    for s in range(n):
        dist, q = {s: 0}, deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if v not in dist:
                    dist[v] = dist[u] + 1
                    q.append(v)
        out.append(sum(dist.values()))
    return out


# HELPERS for the base question
def build(n, edges):
    adj = [[] for _ in range(n)]
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    return adj


# BASELINE (say it first): BFS from every node
def distance_sums_bfs(n, edges):
    # the baseline to say first: a BFS from every node,
    # O(n^2)
    adj, out = build(n, edges), []
    for s in range(n):
        dist, queue = {s: 0}, deque([s])
        while queue:
            u = queue.popleft()
            for v in adj[u]:
                if v not in dist:
                    dist[v] = dist[u] + 1
                    queue.append(v)
        out.append(sum(dist.values()))
    return out


# BASE QUESTION (optimal): Two passes: subtree sizes, then
# re-root
# Idea: Root the tree at 0 and get a BFS order. In reverse
# order compute size[u] and the sum of distances inside
# u's subtree. For the root that sum is already the
# answer.
def tree_distance_sums(n, edges):
```

```python
    if n == 0:
        return []
    adj = build(n, edges)
    # order: parents before children (no recursion)
    parent, order = [-1] * n, [0]
    for u in order:
        for v in adj[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    # size[u]: nodes in u's subtree
    # inside[u]: distances from u to its own subtree
    size, inside = [1] * n, [0] * n
    for u in reversed(order):
        p = parent[u]
        if p >= 0:
            size[p] += size[u]
            inside[p] += inside[u] + size[u]
    # the root's answer is already complete
    ans = [0] * n
    ans[0] = inside[0]
    # moving the root from p to child u: the size[u] nodes
    # below get 1 closer, the other n - size[u] get 1
    # farther
    for u in order[1:]:
        ans[u] = ans[parent[u]] + n - 2 * size[u]
    return ans


if __name__ == "__main__":
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    want = [8, 12, 6, 10, 10, 10]
    assert tree_distance_sums(6, edges) == want
    assert distance_sums_bfs(6, edges) == want
    assert tree_distance_sums(1, []) == [0]
    assert tree_distance_sums(0, []) == []
    # a chain of 100000 nodes: no recursion trouble
    n = 100000
    chain = [(i, i - 1) for i in range(1, n)]
    first = tree_distance_sums(n, chain)[0]
    assert first == n * (n - 1) // 2
```

NOTES FROM CANDIDATES

The interviewer explicitly liked the "baseline first, optimize later" structure — name the O(n²) and move on within the first ~10 minutes.
Two-pass technique to memorize:

Post-order DFS rooted at any node: compute subSize[u] and subSum[u] (sum of distances inside u's subtree). Recurrence: subSum[u] += subSum[v] + subSize[v].
Pre-order DFS doing re-root: when moving root from parent p to child v, ans[v] = ans[p] + (n - subSize[v]) - subSize[v].

Verbalize the geometric intuition for the re-root formula: moving root toward v shortens distance to the subSize[v] nodes inside v's subtree by 1 and lengthens the other n - subSize[v] by 1.

PREPARATION

Drill the canonical sum-of-distances-in-tree problem until you can derive the re-root formula on a whiteboard without notes.
Practice both an iterative and a recursive version; deep recursion can blow the stack on chain-like trees.
Be prepared to explain why re-rooting is correct (proof by induction on the subtree boundary).

STUDY HERE: topic 6, Dynamic Programming | main topic: Dynamic programming

HARD | PracHub | Technical Screen | 15 of 15

Determine winner in stack-merging game

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 31.2 | 30 DAYS 0 | PRIORITY 121/228

ABOUT THIS QUESTION

Topic: Dynamic Programming

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #877 Stone Game

LeetCode match: Related: LC 292 Nim Game, LC 464 Can I Win, LC 877 Stone Game (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-14)

Original: https://prachub.com/coding-questions/determine-winner-in-stack-merging-game

QUESTION

Overview: This question evaluates understanding of combinatorial game theory, impartial game analysis, state-space encoding, and efficient state-space search techniques.

APPROACH

Impartial game: encode the state (tuple of stack sizes), memoize win/lose with DFS over all legal merges, or use Sprague-Grundy values. Prune with symmetry (sorted state).

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy of this question has only the overview, so before anything ask for the exact rules: what a move is (for example merge two stacks, possibly adjacent ones, with a size limit), who moves first, whether the player who cannot move loses (normal play) or wins, and the number and size of stacks.

BRUTE FORCE TO SAY OUT LOUD

Recursive search over every legal move with no memory. Exponential.

OPTIMAL IDEA

Treat it as an impartial game. Encode the state as the multiset of stack sizes (a sorted tuple, which removes symmetric duplicates). A state is winning if some move leads to a losing state, and a state with no moves is losing for the player to move under normal play. Memoize on the sorted tuple. If the game splits into independent parts, use Sprague-Grundy numbers (mex of reachable values) and XOR them instead of searching the product.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

No legal moves, one stack, two equal stacks, a state reachable by two move orders (checks memoization).

LIKELY FOLLOW-UPS

Larger states (find a pattern from small cases with a brute-force table, then prove it), misere play, or counting winning first moves.

PITFALLS

Starting to code before the rules are confirmed. Memoizing on an unsorted state and blowing up the state space.

HOW TO THINK ABOUT IT

Two-player game with perfect play and no draws: define win(state) = some move leads to a state where the opponent loses. A state with no moves is a loss for whoever must move. If a move does not care about stack order, sort the state so equal positions share one cache entry. If the game splits into independent parts, use Grundy numbers and XOR them.

Basis: Written. The statement is not available in my copy, so this uses an example rule (confirm the real one): any two stacks may be merged if their combined size is at most a cap; a player who cannot move loses. The method (memoized win/lose search, Sprague-Grundy for independent games) is the part that carries over.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wins(state):
    for nxt in all_merges(state):
        if not wins(nxt):
            return True  # opponent loses
    return False  # no move wins
```

TIME	SPACE
exponential in the number... exponential in the number of stacks	**O(depth)**

WHY IT IS SLOW

The same position is reached through many different move orders, so the work is exponential. Remembering the result per position (sorted state as key) solves each position once.

OPTIMAL SOLUTION · Memoize on the sorted state

TIME	SPACE
O(number of distinct sorted states * k^2)	**O(number of states)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The game is a sum of independent games (several separate boards or piles).

Compute each part's Grundy number (the mex of the Grundy numbers of its moves) and XOR them. The first player wins exactly when the XOR is not zero. This turns an exponential search over the product of states into a sum of small searches.

FOLLOW-UP 2 Misere play (the player who cannot move wins).

Flip the base case in the memoized search: no moves means the player to move wins. Grundy numbers no longer compose by XOR in general, so search the whole state or use the known special-case theory.

FOLLOW-UP 3 Count the number of winning first moves.

Keep the same memoized win/lose function and count the moves whose resulting state is a loss for the opponent.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine winner in stack-merging game
#
# This question evaluates understanding of combinatorial
# game theory, impartial game analysis, state-space
# encoding, and efficient state-space search techniques.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The game is a sum of independent games (several
# separate boards or piles).
# 2. Misere play (the player who cannot move wins).
# 3. Count the number of winning first moves.
#
# INTERVIEW NOTES
# Two-player game with perfect play and no draws: define
# win(state) = some move leads to a state where the
# opponent loses. A state with no moves is a loss for
# whoever must move. If a move does not care about stack
# order, sort the state so equal positions share one cache
# entry. If the game splits into independent parts, use
# Grundy numbers and XOR them.
#
# BRUTE FORCE (say it first): Plain recursion over every
# move
# Try every merge, and the current player wins if some
# merge leaves the opponent in a losing position. Correct,
# but the same positions are explored again and again
# through different move orders. Time exponential in the
# number of stacks. Space O(depth).


from functools import lru_cache


# BRUTE FORCE (say it first): Plain recursion over every
# move | time exponential in the number of stacks
def merge_game_brute(stacks, cap):
    def wins(state):
        for i in range(len(state)):
            for j in range(i + 1, len(state)):
                if state[i] + state[j] <= cap:
                    nxt = (
                        state[:i]
                        + state[i + 1 : j]
                        + state[j + 1 :]
                        + [state[i] + state[j]]
                    )
                    if not wins(nxt):
                        return True
        return False

    return wins(list(stacks))


# HELPERS for the base question
def moves(state, cap):
    # merge two stacks if the new stack fits under the cap
    seen = set()
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] <= cap:
                rest = [x for k, x in enumerate(state)
                        if k not in (i, j)]
                merged = state[i] + state[j]
                new = tuple(sorted(rest + [merged]))
                # equal results are the same move
                if new not in seen:
                    seen.add(new)
                    yield new


# BASE QUESTION (optimal): Memoize on the sorted state
# Idea: Merging does not depend on order, so a sorted
# tuple identifies the position. Each state is solved once
# and then looked up. This is the standard fix for 'many
# move orders reach the same position'.
def first_player_wins(stacks, cap, misere=False):
    # the player who cannot move loses (misere: wins)
    @lru_cache(None)
    def wins(state):
        options = list(moves(state, cap))
        if not options:
            return misere
        # a winning move leaves the opponent in a losing
        # state
        return any(not wins(nxt) for nxt in options)

    return wins(tuple(sorted(stacks)))


# FOLLOW-UP 1: The game is a sum of independent games
# (several separate boards or piles).
# Answer: Compute each part's Grundy number (the mex of
# the Grundy numbers of its moves) and XOR them. The first
# player wins exactly when the XOR is not zero.
# FOLLOW-UP 2: Misere play (the player who cannot move
# wins).
# Answer: Flip the base case in the memoized search: no
# moves means the player to move wins. Grundy numbers no
# longer compose by XOR in general, so search the whole
# state or use the known special-case theory.
def grundy(stacks, cap):
    # Sprague-Grundy number of one game (mex of its moves)
    @lru_cache(None)
    def g(state):
        reach = {g(nxt) for nxt in moves(state, cap)}
        number = 0
        while number in reach:
            number += 1
        return number
```

```
    return g(tuple(sorted(stacks)))


# FOLLOW-UP 1: The game is a sum of independent games
# (several separate boards or piles).
# Answer: Compute each part's Grundy number (the mex of
# the Grundy numbers of its moves) and XOR them. The first
# player wins exactly when the XOR is not zero.
# FOLLOW-UP 2: Misere play (the player who cannot move
# wins).
# Answer: Flip the base case in the memoized search: no
# moves means the player to move wins. Grundy numbers no
# longer compose by XOR in general, so search the whole
# state or use the known special-case theory.
def sum_of_games_wins(boards, cap):
    # independent boards: XOR their Grundy numbers
    total = 0
    for board in boards:
        total ^= grundy(board, cap)
    return total != 0


# FOLLOW-UP 3: Count the number of winning first moves.
# Answer: Keep the same memoized win/lose function and
# count the moves whose resulting state is a loss for the
```

```
# opponent.
def winning_first_moves(stacks, cap):
    # how many different first moves win?
    state = tuple(sorted(stacks))
    return sum(1 for nxt in moves(state, cap)
               if not first_player_wins(nxt, cap))


if __name__ == "__main__":
    # one stack: no move, so the first player loses
    assert not first_player_wins([3], 5)
    # two stacks that can merge: one move ends it, so we
    # win
    assert first_player_wins([1, 1], 5)
    # three stacks: after one merge the other two may
    # merge too
    assert first_player_wins([1, 1, 1], 2) is True
    # in misere play the result flips for a single stack
    assert first_player_wins([3], 5, misere=True)
    assert winning_first_moves([1, 1, 1], 2) >= 1
    # normal play: a position wins exactly when Grundy !=
    # 0
    for state in ([1, 2, 3], [2, 2, 2, 2], [1, 1, 4]):
        win = first_player_wins(state, 6)
        assert win == (grundy(state, 6) != 0)
    # two separate boards: XOR of their numbers
    both = sum_of_games_wins([[1, 1], [1, 1]], 5)
    assert both is False
```

FUNCTION AND TEST CASES

Signature

```
def solution(colors: list[str]) -> bool:
SAMPLE INPUT
['Y']
```

Test cases

Input	Expected
['Y']	False
['Y', 'W']	True

ALSO REPORTED · 1 more

Babylon Tile-Stack Merge Game | 1Point3Acres | onsite-coding | asked 2025-12-29 | Single report

https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

This is closer to an implementation/simulation interview than a clever-algorithm problem — the interviewer wants a clean game state class, legal-move enumeration, and a play(move) API. Represent state as a List[Stack] where Stack = (height, topColor, fullSequence?). Whether you keep the full color sequence depends on whether the interviewer asks about "undo" or "re-color from top".
Add a minimax / negamax for "who wins from this state with optimal play" as the natural follow-up — branching factor is small (at most C(12, 2) = 66 initially, drops fast).
Reported as "too hard to finish coding in 45 minutes"; the interviewer gives hints throughout. Pacing matters.

Preparation: Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 7 OF 10 | COMMON TOPIC

Binary Search and Sorting

13 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	9	3
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Binary Search and Sorting | 13 questions | tap a title to jump to it

BINARY SEARCH (7)

E	1. Check an Interval Against Existing Intervals for Conflicts	PracHub	2026-08-08
M	2. Cutting Ribbons (LC 1891)	1Point3Acres	2026-08-16
M	3. First Bad Version with Parallel Search Follow-up	1Point3Acres	2026-06-15
M	4. Find median of merged RLE arrays	PracHub	2026-01-06
M	5. Solve Banana Speed and Interval Merge	PracHub	2025-01-27
H	6. Find Maximum Envelope Nesting	PracHub	2026-03-18
H	7. Count of Smaller Numbers After Self	HackTheRounds	no date

SORTING (6)

M	8. Progressive Array Partitioning	1Point3Acres	2026-08-11
M	9. Streaming Points: Emit K Within Distance	1Point3Acres	2026-06-23
M	10. Find Containing Range	PracHub	2026-06-04
M	11. Check if all substrings are anagrams of words	PracHub	2026-03-11
M	12. Find shared objects across two log files	PracHub	2025-11-05
H	13. Find safe travel intervals between planet influences	PracHub	2025-11-22

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

EASY | IN APP LC 729 | PracHub | Technical Screen | 1 of 13

Check an Interval Against Existing Intervals for Conflicts

POSTED 2026-08-08 | 73 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 26.5 30 DAYS 0 PRIORITY 132/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #729 My Calendar I

LeetCode match: LC 729 My Calendar I (inferred from overview)

Posted on PracHub: 2026-08-08 (updated 2026-08-29)

Original: https://prachub.com/coding-questions/check-an-interval-against-existing-intervals-for-conflicts

QUESTION

Overview: Check whether one half-open candidate interval conflicts with any interval in a large unsorted collection. Respect strict start and end semantics so ranges that only touch at an endpoint do not conflict.

APPROACH

Half-open intervals [s,e) conflict iff new.start < e and s < new.end (touching endpoints do not conflict). One-off check on an unsorted collection is a single O(n) scan; for repeated queries keep a sorted structure (TreeMap/balanced BST or sorted list + bisect) and check only the neighbors. O(log n) per query.

HOW TO THINK ABOUT IT

Two half-open intervals [s1, e1) and [s2, e2) overlap exactly when s1 < e2 and s2 < e1. If the booked intervals are kept sorted and disjoint, only the neighbours at the insertion point can conflict, so a binary search replaces the scan.

Basis: LeetCode 729, My Calendar I (booking succeeds only when the new half-open interval overlaps nothing), and 731 for double booking.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
booked = []
def book(s, e):
    for s2, e2 in booked:
        if s < e2 and s2 < e:
            return False  # overlap
    booked.append((s, e))
    return True
```

BRUTE FORCE

TIME	SPACE
O(n) per booking	O(n)

WHY IT IS SLOW

Each booking scans all earlier bookings, O(n). With the intervals sorted, binary search finds the only two neighbours that could conflict.

OPTIMAL SOLUTION · Sorted starts with bisect: check only two neighbours

TIME	SPACE
O(log n) search, O(n) list insert	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Allow double booking but never triple booking (LeetCode 731).

Keep a difference map (+1 at start, -1 at end) and sweep it in sorted order; if the running count ever exceeds 2, undo the change and reject. O(n log n) per booking, or use a segment tree for faster.

FOLLOW-UP 2 Report the maximum number of overlapping bookings after each booking (LeetCode 732).

The same difference map, taking the maximum running count in the sweep. A segment tree with lazy range increments gives O(log n) per booking.

FOLLOW-UP 3 Closed intervals instead of half-open.

Then touching endpoints conflict, so the tests become <= instead of <. Say it, and keep the choice in one comparison.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check an Interval Against Existing
# Intervals for Conflicts
#
# Check whether one half-open candidate interval conflicts
# with any interval in a large unsorted collection.
# Respect strict start and end semantics so ranges that
# only touch at an endpoint do not conflict.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Allow double booking but never triple booking
# (LeetCode 731).
# 2. Report the maximum number of overlapping bookings
# after each booking (LeetCode 732).
```

```
# 3. Closed intervals instead of half-open.
#
# INTERVIEW NOTES
# Two half-open intervals [s1, e1) and [s2, e2) overlap
# exactly when s1 < e2 and s2 < e1. If the booked
# intervals are kept sorted and disjoint, only the
# neighbours at the insertion point can conflict, so a
# binary search replaces the scan.
#
# BRUTE FORCE (say it first): Scan every booked interval
# Compare the new interval with all existing ones using
# the overlap test. Correct and simple, but each booking
# costs time proportional to the number of bookings. Time
# O(n) per booking. Space O(n).


from bisect import bisect_right
from collections import defaultdict


# BRUTE FORCE (say it first): Scan every booked interval |
# time O(n) per booking
class CalendarBrute:
    def __init__(self):
        self.booked = []

    def book(self, start, end):
        if any(
            start < e and s < end
            for s, e in self.booked
        ):
            return False
```

```
        self.booked.append((start, end))
        return True


# BASE QUESTION (optimal): Sorted starts with bisect:
# check only two neighbours
# Idea: Keep the booked intervals sorted by start.
class Calendar:
    """A sorted calendar: O(log n) per check."""

    def __init__(self):
        self.starts, self.ends = [], []

    def book(self, start, end):
        i = bisect_right(self.starts, start)
        # the booking before must end by `start`
        if i > 0 and self.ends[i - 1] > start:
            return False
        # the booking after must start at or after `end`
        if i < len(self.starts) and self.starts[i] < end:
            return False
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True


# FOLLOW-UP 1: Allow double booking but never triple
# booking (LeetCode 731).
# Answer: Keep a difference map (+1 at start, -1 at end)
# and sweep it in sorted order; if the running count ever
# exceeds 2, undo the change and reject. O(n log n) per
# booking, or use a segment tree for faster.
class DoubleBooking:
    """LeetCode 731: two overlaps are fine, three are not.
    LeetCode 732: also report the largest overlap."""

    def __init__(self, limit=2):
        self.delta = defaultdict(int)
        self.limit = limit
```

```
    def peak(self):
        # sweep the +1 / -1 changes in time order
        run = best = 0
        for t in sorted(self.delta):
            run += self.delta[t]
```

```
            best = max(best, run)
        return best

    def book(self, start, end):
        self.delta[start] += 1
        self.delta[end] -= 1
        if self.peak() > self.limit:
            # undo the change and reject the booking
            self.delta[start] -= 1
            self.delta[end] += 1
            return False
        return True


# HELPERS for the checks below
def conflicts(candidate, intervals):
    # half-open [start, end): touching ends do NOT
    # conflict
    s, e = candidate
    return any(a < e and s < b for a, b in intervals)


if __name__ == "__main__":
    taken = [(10, 20), (30, 40)]
    assert conflicts((15, 25), taken)
    assert not conflicts((20, 30), taken)
    assert conflicts((19, 31), taken)
    # sorted calendar
    c = Calendar()
    assert c.book(10, 20) and c.book(20, 30)
    assert not c.book(15, 25) and c.book(5, 10)
    # double booking is allowed, triple booking is not
    d = DoubleBooking()
    assert d.book(10, 20) and d.book(15, 25)
    assert not d.book(17, 18)
    assert d.peak() == 2
    # limit 1 is the plain no-overlap calendar
    one = DoubleBooking(limit=1)
    assert one.book(1, 5) and not one.book(4, 6)
```

FUNCTION AND TEST CASES

Signature

```
def hasIntervalConflict(candidate: list[int], intervals: l
ist[list[int]]) -> bool:
```

Sample call

```
hasIntervalConflict([4,7], [[1,3],[7,9]])
```

Test cases

Input	Expected
([4,7],[[1,3],[7,9]])	False
([4,7],[[1,5],[8,9]])	True

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

Site tags: array, binary-search, complexity-analysis, medium

MEDIUM | 1Point3Acres | onsite-coding | 2 of 13

Cutting Ribbons (LC 1891)

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **ALL TIME 19.6** | **30 DAYS 0** | **PRIORITY 147/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1891 Cutting Ribbons

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/cutting-ribbons-max-equal-length

QUESTION

LC 1891 Cutting Ribbons: given wood lengths and K, maximize the equal piece length that yields at least K pieces. Follow-ups cover the search bound, floating-point lengths, and why complexity depends on the value range rather than the number of elements.SWEarraybinary-searchcomplexity-analysismedium FrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether piece lengths must be integers, what to return if even length 1 fails (0), and the sizes of the array and k.

BRUTE FORCE TO SAY OUT LOUD

Try every length from the maximum down to 1 and count pieces. O(n * max).

OPTIMAL IDEA

Feasibility is monotone: if you can cut k pieces of length L you can for any smaller length. Binary search L over [1, max]: count = sum(r // L), move right if count >= k. O(n log max). A tighter upper bound is min(max, sum // k).

DRY-RUN THESE CASES

k larger than the total length (0), one ribbon, all equal ribbons, k = 1 (answer is the max ribbon).

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Real-valued lengths (fixed number of halvings, not an epsilon loop), why complexity depends on the value range, or an unlimited number of cuts.

PITFALLS

Overflow when summing. Searching from 0 and dividing by zero.

HOW TO THINK ABOUT IT

'Largest length such that at least K pieces fit' is a maximisation with a yes/no check. Larger lengths yield fewer pieces, so the check is monotone and you can binary search the answer. The check itself is one pass: sum of ribbon // length.

Basis: LeetCode 1891, Cutting Ribbons.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        pieces = sum(r // L for r in ribbons)
        if pieces >= k:
            return L
    return 0
```

TIME	SPACE
O(n * max)	O(1)

WHY IT IS SLOW

Tries up to max(ribbon) lengths, each costing O(n); with ribbons up to 10^9 that is a billion probes. The check is monotone, so binary search needs only log2(max) probes.

OPTIMAL SOLUTION · Binary search the answer (integer lengths)

TIME	SPACE
O(n log(max length))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

FOLLOW-UP 2 What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

FOLLOW-UP 3 What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. **Cost:** each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

FOLLOW-UP 4 Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Cutting Ribbons (LC 1891)
#
# LC 1891 Cutting Ribbons: given wood lengths and K,
# maximize the equal piece length that yields at least K
# pieces. Follow-ups cover the search bound, floating-
# point lengths, and why complexity depends on the value
# range rather than the number of elements.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the running time depend on the value range
# and not just on the number of ribbons?
# 2. What if the candidate length is given to you?
# 3. What is a good upper bound for the search?
# 4. Give the search a tighter upper bound.
#
# INTERVIEW NOTES
# 'Largest length such that at least K pieces fit' is a
# maximisation with a yes/no check. Larger lengths yield
# fewer pieces, so the check is monotone and you can
# binary search the answer. The check itself is one pass:
# sum of ribbon // length.
#
# BRUTE FORCE (say it first): Try every length from the
# longest ribbon down
# For each length L from the maximum ribbon down to 1
# count the pieces and return the first L that gives at
# least K. Correct, but with ribbons up to 10^9 it would
# try a billion lengths. Time O(n * max). Space O(1).


# BRUTE FORCE (say it first): Try every length from the
# longest ribbon down | time O(n * max)
def max_length_brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        if sum(r // L for r in ribbons) >= k:
            return L
    return 0
```

```
# HELPERS for the base question
def pieces(ribbons, length):
    # how many pieces of this length can we cut?
    return sum(r // length for r in ribbons)
```

```
# FOLLOW-UP 4: Give the search a tighter upper bound.
# Answer: No piece can be longer than the longest ribbon,
# and K pieces of length L need K * L total length, so L
# <= sum // K. Starting hi at min(max(ribbons), sum // K)
# saves a few probes and also returns 0 immediately when
# sum < K.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and
    # k pieces of length L need k * L total length
    return min(max(ribbons), sum(ribbons) // k)


# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
# Idea: If you can cut k pieces of length L you can cut k
# pieces of any shorter length, so feasibility is
# monotone.
def max_length(ribbons, k):
    # the best whole-number length (0 if impossible)
    lo, hi, best = 1, upper_bound(ribbons, k), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if pieces(ribbons, mid) >= k:
            # mid works: remember it and try longer
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best


# VARIANT: fractional lengths
def max_length_float(ribbons, k, rounds=50):
    # lengths may be fractions: fixed rounds of halving
    lo, hi = 0.0, float(max(ribbons))
    for _ in range(rounds):
        mid = (lo + hi) / 2
        # enough pieces: the answer is at least mid
        if sum(int(r // mid) for r in ribbons) >= k:
            lo = mid
        else:
            hi = mid
    return lo
```

```
if __name__ == "__main__":
    assert max_length([9, 7, 5], 3) == 5
    assert max_length([7, 5, 9], 4) == 4
    # not enough total length: no pieces at all
    assert max_length([1, 2], 10) == 0
    # the check alone, when the length is given
    assert pieces([9, 7, 5], 5) == 3
    assert upper_bound([9, 7, 5], 3) == 7
    # fractional lengths: 3 pieces from one ribbon of 1
    assert abs(max_length_float([1], 3) - 1 / 3) < 1e-9
    assert max_length_float([1], 0 + 2) > 0.49
```

NOTES FROM CANDIDATES

The interviewer allowed the candidate to abandon an initial greedy/sorting direction, then unlocked the problem with one question: what changes if a candidate length is supplied?
Expect the follow-ups, not the base implementation, to consume most of the round.
For the integer version, initialize the best answer to 0, search only positive candidate lengths, and return 0 when even length 1 cannot produce K pieces. The feasibility predicate is monotone: once a length is infeasible, every larger length is also infeasible.

PREPARATION

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

Site tags: binary-search, parallelism, concurrency, medium

MEDIUM | 1Point3Acres | phone-screen | 3 of 13

★ TOP 75: rank 72 in the Medium and Hard priority list

★ First Bad Version with Parallel Search Follow-up

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 41.1** | **30 DAYS 33** | **PRIORITY 47/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #278 First Bad Version

LeetCode match: LC 278 First Bad Version

Frequency: Single report | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/first-bad-version-parallel-search

QUESTION

Phone-screen R1 coding round. The base problem is the standard first-bad-version monotonic search — given an **`isBadVersion(v)`** oracle over versions **`1..n`**, find the first bad version with a single binary search. The graded follow-up requires parallelizing the search: partition the range into buckets and search them concurrently (bucket + binary search), with working code expected.SWEbinary-searchparallelismconcurrencymediumFrequencySingle reportLast asked2026-06-15Stagephone-screen

APPROACH

Write the single-threaded binary search first and verify it on small n.
Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the oracle is (isBadVersion is monotone), what concurrency primitive to use (ThreadPoolExecutor, Java's ExecutorService), and which metric matters: wall-clock time or total calls.

BRUTE FORCE TO SAY OUT LOUD

Check every version from 1. O(n) calls.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Base: binary search, O(log n) calls. Parallel: each round fires up to k probes concurrently at evenly spaced points, finds the first probe that is bad, and narrows the interval to the gap between it and the previous probe. Each round shrinks the interval by k + 1, so there are about log base (k + 1) of n sequential rounds. Use a thread pool and map the probes in one call.

DRY-RUN THESE CASES

n = 1, the first version bad, the last version bad, n smaller than the number of workers.

LIKELY FOLLOW-UPS

Rate limiting, retries when a probe fails, and caching results.

PITFALLS

Mutating shared state from threads. Assuming more threads always means fewer total calls (they increase it).

HOW TO THINK ABOUT IT

'First bad version' is a boundary in a sorted yes/no sequence, which is binary search. The parallel part is a different trade: instead of one probe per round, send several probes in the same round and keep the gap that contains the boundary. Fewer rounds, more total calls.

Basis: LeetCode 278, First Bad Version, plus a concurrent k-ary search follow-up.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    for v in range(1, n + 1):
        if is_bad_version(v):
            return v
```

TIME	SPACE
O(n)	**O(1)**
calls	

WHY IT IS SLOW

One call per version, O(n) calls; with a billion versions and a slow check that is far too many. The check is monotone, so binary search needs only log2(n) calls, and probing several points in parallel needs fewer rounds.

OPTIMAL SOLUTION · Base problem: binary search

TIME	SPACE
O(log n)	**O(1)**
calls	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Parallel: probe k points at once, then recurse into the bucket that flips

TIME	SPACE
O(log_{k+1} n) rounds	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does parallelism buy you, and what does it cost?

It cuts the number of sequential rounds from log2(n) to log_{k+1}(n), so wall-clock time drops when each probe has high latency. It increases total probes (about k per round instead of 1), so it costs more calls and money. Say which metric you are optimizing before choosing k.

FOLLOW-UP 2 What if the oracle is rate limited or can fail?

Cap the pool size to the rate limit, retry transient failures with backoff (probes are idempotent), and never advance the interval from a probe that errored. Cache results by version so a retry or a later round never repeats a call.

FOLLOW-UP 3 Count how many rounds each approach needs.

Single binary search: log2(n) rounds of one probe each. k workers: about log base (k + 1) of n rounds with k probes each. For n = 10^9, one probe per round needs 30 rounds, while 7 probes per round need 10.

FOLLOW-UP 4 Show how many rounds and calls the parallel search uses.

With k workers each round shrinks the interval by about k + 1, so the number of sequential rounds is about log base (k + 1) of n and the total calls at most k times that. For n = 10^6 and 7 workers that is about 7 rounds and 49 calls, against 20 rounds and 20 calls for the sequential search: fewer rounds, more calls.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: First Bad Version with Parallel Search
# Follow-up
#
# Phone-screen R1 coding round. The base problem is the
# standard first-bad-version monotonic search — given an
# isBadVersion(v) oracle over versions 1..n, find the
# first bad version with a single binary search. The
# graded follow-up requires parallelizing the search:
# partition the range into buckets and search them
# concurrently (bucket + binary search), with working code
# expected.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does parallelism buy you, and what does it cost?
# 2. What if the oracle is rate limited or can fail?
# 3. Count how many rounds each approach needs.
# 4. Show how many rounds and calls the parallel search
# uses.
#
# INTERVIEW NOTES
# 'First bad version' is a boundary in a sorted yes/no
# sequence, which is binary search. The parallel part is a
# different trade: instead of one probe per round, send
# several probes in the same round and keep the gap that
# contains the boundary. Fewer rounds, more total calls.
#
# BRUTE FORCE (say it first): Check versions one by one
# Call isBadVersion for 1, 2, 3, ... and stop at the first
# bad one. Obviously correct and the oracle; with a
# billion versions and a slow check it is far too many
# calls. Time O(n) calls. Space O(1).


from concurrent.futures import ThreadPoolExecutor
from threading import Lock


# BRUTE FORCE (say it first): Check versions one by one |
# time O(n) calls
def first_bad_linear(n, is_bad):
    return next(v for v in range(1, n + 1) if is_bad(v))


# BASE QUESTION (optimal): problem: binary search
# Idea: isBad is monotone (all good, then all bad), so
# keep the invariant 'the first bad version is in [lo,
# hi]' and halve the interval. Use lo + (hi - lo) // 2 in
# languages with fixed-width integers.
def first_bad_version(n, is_bad):
    # the base question: binary search on versions 1..n
    lo, hi = 1, n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            # the first bad version is mid or earlier
            hi = mid
        else:
            lo = mid + 1
    return lo


# PARALLEL SEARCH helper
def probe_points(lo, hi, workers):
    # evenly spaced versions inside [lo, hi - 1]
    step = max(1, (hi - lo + 1) // (workers + 1))
    pts = {min(hi - 1, lo + step * (i + 1) - 1)
           for i in range(workers)}
    return sorted(pts)


# FOLLOW-UP 1: What does parallelism buy you, and what
# does it cost?
# Answer: It cuts the number of sequential rounds from
# log2(n) to log_{k+1}(n), so wall-clock time drops when
# each probe has high latency. It increases total probes
# (about k per round instead of 1), so it costs more calls
# and money.
# FOLLOW-UP 3: Count how many rounds each approach needs.
# Answer: Single binary search: log2(n) rounds of one
# probe each. k workers: about log base (k + 1) of n
# rounds with k probes each. For n = 10^9, one probe per
```

```
# round needs 30 rounds, while 7 probes per round need 10.
# FOLLOW-UP 4: Show how many rounds and calls the parallel
# search uses.
# Answer: With k workers each round shrinks the interval
# by about k + 1, so the number of sequential rounds is
# about log base (k + 1) of n and the total calls at most
# k times that.
def first_bad_parallel(n, is_bad, workers=4):
    # invariant: the answer is in [lo, hi]
```

```
    lo, hi, cache, rounds = 1, n, {}, 0
    with ThreadPoolExecutor(workers) as pool:
        while lo < hi:
            points = probe_points(lo, hi, workers)
            # ask the oracle about new points at the same
            # time
            fresh = [p for p in points if p not in cache]
            answers = pool.map(is_bad, fresh)
            for p, bad in zip(fresh, answers):
                cache[p] = bad
            rounds += 1
            bad_points = [p for p in points if cache[p]]
            if not bad_points:
                # every probe was good: answer is after
                # them
                lo = points[-1] + 1
            else:
                first = bad_points[0]
                # the answer sits between the last good
                # probe before it and the first bad one
                before = [p for p in points if p < first]
                if before:
                    lo = before[-1] + 1
                hi = first
    return lo, rounds, len(cache)


if __name__ == "__main__":
    calls, lock = [], Lock()

    def oracle(first_bad):
        def is_bad(v):
            with lock:
                calls.append(v)
            return v >= first_bad
        return is_bad

    cases = ((1, 1), (10, 1), (10, 10), (1000, 637))
    for n, bad in cases:
        assert first_bad_version(n, oracle(bad)) == bad
        ans, rounds, used = first_bad_parallel(
            n, oracle(bad), workers=4)
        assert ans == bad
    # fewer rounds than the 20 of a plain binary search
    ans, rounds, used = first_bad_parallel(
```

```
        10 ** 6, oracle(123456), workers=7)
    assert ans == 123456 and rounds < 20
    assert used <= 7 * rounds
```

NOTES FROM CANDIDATES

The base is trivial; budget your time for the parallel follow-up and aim to get a complete, compiling version through even if it has minor bugs — finishing the parallel version is what passes this round.

Be explicit about what the parallel design optimizes: wall-clock latency (fire many isBadVersion probes at once) versus total number of calls. Splitting into k buckets and probing each boundary concurrently localizes the transition in one parallel round, then a binary search finishes it.

Common bugs: off-by-one when mapping a bucket index back to an absolute version range; the transition sitting exactly on a bucket boundary; integer overflow in mid if you write (lo + hi) / 2 instead of lo + (hi - lo) // 2.

PREPARATION

Write the single-threaded binary search first and verify it on small n.

Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

MEDIUM | PracHub | Technical Screen | 4 of 13

★ TOP 75: rank 2 in the Medium and Hard priority list

★ Find median of merged RLE arrays

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 5/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #4 Median of Two Sorted Arrays

LeetCode match: LC 4 Median of Two Sorted Arrays (RLE variant) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-median-of-merged-rle-arrays

QUESTION

Overview: The question evaluates understanding of run-length encoding, order-statistics (median) computation, and efficient merging of frequency-annotated sequences, emphasizing handling large frequencies without explicit decompression.

APPROACH

Total length = sum of run counts; walk both run lists merged by value accumulating counts until the median index(es) are covered. Or binary search on value with counts.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether each array is sorted by value, how large counts can be (so decompression is impossible), and what to return for an even total (average of the two middle values).

BRUTE FORCE TO SAY OUT LOUD

Decompress both arrays, merge, take the middle. O(total count) memory, which counts can make huge.

OPTIMAL IDEA

Let N be the sum of all counts and the median positions be (N - 1) // 2 and N // 2. Merge the two run lists by value with two pointers (like the merge step of merge sort), accumulating counts until the running total passes each median position, and read the value there. O(r1 + r2) with no decompression. Alternative: binary search on the value, counting how many elements are at most that value in each array.

DRY-RUN THESE CASES

A single run, an even total, the median falling exactly on a run boundary, both arrays with the same values, an empty array.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

K arrays (heap merge), or arrays not sorted by value.

PITFALLS

Treating counts as lengths of different arrays instead of merging by value. Off-by-one at a run boundary.

HOW TO THINK ABOUT IT

Counts can be huge, so never expand a run. You only need the elements at ranks (N - 1) // 2 and N // 2 in the merged order. Merge the two run lists by value, counting how many elements you have passed, and read the value when the count reaches each rank.

Basis: Written; LeetCode 4 with run-length encoded inputs. Assumed input: two lists of (value, count) runs, each sorted by value; the median of the merged multiset (average of the two middle values for an even total).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(runs_a, runs_b):
    dense = []
    for value, count in runs_a + runs_b:
        dense += [value] * count
    dense.sort()
    n = len(dense)
    if n % 2:
        return dense[n // 2]
    mid = n // 2
    return (dense[mid - 1] + dense[mid]) / 2
```

TIME	SPACE
O(total count)	O(total count)

WHY IT IS SLOW

Needs time and memory proportional to the total count, which can be 10^9 or more. Walking the two run lists in value order and counting up to the middle rank costs only the number of runs.

OPTIMAL SOLUTION · Merge the runs and walk to the two middle ranks

TIME	SPACE
O(r1 + r2)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 K arrays instead of two.

Merge all runs by value (heap of K heads, or sort the combined run list if memory allows) and walk to the two middle ranks. O(R log K) with a heap, R the total number of runs.

FOLLOW-UP 2 The arrays are huge and sorted, and you want O(log) time.

Binary search on the value: for a candidate value count how many elements are at most it in each array (binary search on prefix sums of counts) and find the smallest value with rank above the middle. O(log(range) * log(runs)).

FOLLOW-UP 3 Runs are not sorted by value.

Sort the runs by value first, O(R log R), then use the same walk.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find median of merged RLE arrays
#
# The question evaluates understanding of run-length
# encoding, order-statistics (median) computation, and
# efficient merging of frequency-annotated sequences,
# emphasizing handling large frequencies without explicit
# decompression.
#
# FOLLOW-UPS (answered below, in this order):
# 1. K arrays instead of two.
# 2. The arrays are huge and sorted, and you want O(log)
# time.
# 3. Runs are not sorted by value.
#
# INTERVIEW NOTES
# Counts can be huge, so never expand a run. You only need
# the elements at ranks (N - 1) // 2 and N // 2 in the
# merged order. Merge the two run lists by value, counting
# how many elements you have passed, and read the value
# when the count reaches each rank.
#
# BRUTE FORCE (say it first): Expand both arrays, merge
# and take the middle
# Decompress each run into repeated values, merge, and
# read the middle. Correct, and the oracle; counts like
# 10^9 make the expansion impossible. Time O(total count).
# Space O(total count).


import heapq
from bisect import bisect_right
from itertools import accumulate


# BRUTE FORCE (say it first): Expand both arrays, merge
# and take the middle | time O(total count)
def median_rle_expand(a, b):
    data = sorted(
        [v for v, c in a for _ in range(c)]
        + [v for v, c in b for _ in range(c)]
    )
    n = len(data)
    return (
        data[n // 2]
        if n % 2
        else (data[n // 2 - 1] + data[n // 2]) / 2
    )
```

```
# HELPERS for the base question
def kth_smallest(runs_a, runs_b, k):
    # zero-based rank in the merged order, walking the
    # runs
    # of two sorted run-length lists; nothing is expanded
    i = j = seen = 0
    while True:
        take_a = j >= len(runs_b)
        if not take_a and i < len(runs_a):
            take_a = runs_a[i][0] <= runs_b[j][0]
        value, count = runs_a[i] if take_a else runs_b[j]
        seen += count
        if seen > k:
            return value
        if take_a:
            i += 1
        else:
            j += 1


# BASE QUESTION (optimal): Merge the runs and walk to the
# two middle ranks
# Idea: Let N be the sum of all counts.
def median_rle(a, b):
    # a and b: [(value, count)] sorted by value
    total = sum(c for _, c in a) + sum(c for _, c in b)
    if total % 2:
        return kth_smallest(a, b, total // 2)
    low = kth_smallest(a, b, total // 2 - 1)
    high = kth_smallest(a, b, total // 2)
    return (low + high) / 2


# FOLLOW-UP 1: K arrays instead of two.
# Answer: Merge all runs by value (heap of K heads, or
# sort the combined run list if memory allows) and walk to
# the two middle ranks. O(R log K) with a heap, R the
# total number of runs.
def median_k_arrays(arrays):
    # K arrays: merge the run lists by value, then walk to
    # the
    # middle ranks
    runs = list(heapq.merge(*arrays))
    total = sum(c for _, c in runs)

    def kth(k):
        seen = 0
        for value, count in runs:
            seen += count
            if seen > k:
                return value

    if total % 2:
        return kth(total // 2)
    return (kth(total // 2 - 1) + kth(total // 2)) / 2


# FOLLOW-UP 2: The arrays are huge and sorted, and you
# want O(log) time.
# Answer: Binary search on the value: for a candidate
# value count how many elements are at most it in each
# array (binary search on prefix sums of counts) and find
# the smallest value with rank above the middle.
# O(log(range) * log(runs)).
def median_by_value(arrays):
    # sorted arrays, whole-number values: binary search
    # the
    # value; count_at_most uses prefix sums of the counts
    prefixes = []
    for runs in arrays:
        values = [v for v, _ in runs]
```

```
        sums = list(accumulate(c for _, c in runs))
        prefixes.append((values, sums))

    def count_at_most(x):
        total = 0
        for values, sums in prefixes:
            i = bisect_right(values, x)
            total += sums[i - 1] if i else 0
        return total

    n = count_at_most(10 ** 18)

    def kth(k):
        lo, hi = -(10 ** 18), 10 ** 18
        while lo < hi:
```

```
            mid = (lo + hi) // 2
            # smallest value with more than k elements <=
            # it
            if count_at_most(mid) > k:
                hi = mid
            else:
                lo = mid + 1
        return lo

    if n % 2:
        return kth(n // 2)
    return (kth(n // 2 - 1) + kth(n // 2)) / 2


if __name__ == "__main__":
    a = [(1, 3), (4, 1)]
    b = [(2, 2), (5, 2)]
    # merged: 1 1 1 2 2 4 5 5 -> middle two are 2 and 2
    assert median_rle(a, b) == 2
    assert median_k_arrays([a, b]) == 2
    assert median_by_value([a, b]) == 2
    # an odd total: 1, 7, 7, 7, 7, 7 plus one more 7
    assert median_rle([(7, 6)], [(1, 1)]) == 7
    # huge counts are never expanded
    big = [(1, 10 ** 12)]
    assert median_rle(big, [(3, 10 ** 12)]) == 2
    c = [(0, 1), (10, 1)]
    three = [a, b, c]
    by_merge = median_k_arrays(three)
    assert by_merge == median_by_value(three)
```

FUNCTION AND TEST CASES

Sample call

```
solution([(1, 2), (5, 1)], [(2, 3)])
```

Test cases

Input	Expected
`([(1, 2), (5, 1)], [(2, 3)])`	`2.0`
`([(1, 1)], [(2, 1)])`	`1.5`

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

MEDIUM | PracHub | Technical Screen | 5 of 13

★ TOP 75: rank 12 in the Medium and Hard priority list

★ Solve Banana Speed and Interval Merge

POSTED 2025-01-27 | 631 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 10/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #875 Koko Eating Bananas, LeetCode #56 Merge Intervals

LeetCode match: LC 875 Koko Eating Bananas + LC 56 Merge Intervals

Posted on PracHub: 2025-01-27 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/solve-banana-speed-and-interval-merge

QUESTION

During two technical phone screens for a new graduate software engineering role, the candidate was asked to solve two coding problems: minimum eating speed and merging overlapping intervals.

Constraints & Assumptions

- Provide clear algorithms, correctness reasoning, complexity, and edge cases.
- For minimum eating speed, each hour can be spent on only one pile.
- For interval merging, intervals overlap when the next start is less than or equal to the current merged end.
- Use sorted or binary-search approaches where appropriate.

Clarifying Questions

to Ask - Are **`piles`** and **h** always positive?

- Is **h** at least the number of piles?
- Are interval endpoints inclusive?
- Can intervals be empty or unsorted?

Part 1

- Minimum Eating Speed Given **`piles`** and **h**, return the minimum integer speed **k** such that all piles can be eaten within **h** hours.

What This Part Should Cover

- Binary search on answer from 1 to **`max(piles)`**.
- Feasibility function using sum of **`ceil(pile / k)`**.
- Why feasibility is monotonic.
- Complexity and edge cases.

Part 2

QUESTION

- Merge Overlapping Intervals Given intervals **`[start, end]`**, merge all overlapping intervals and return non-overlapping intervals sorted by start.

What This Part Should Cover

- Sort by start.
- Maintain current merged interval.
- Extend on overlap, otherwise append and start a new interval.
- Complexity and edge cases.

What a Strong

Answer Covers - Recognizes monotonic binary search for eating speed.

- Uses integer-safe ceiling division.
- Correctly defines interval overlap.
- Gives clean code and complexity for both problems.

Follow-up Questions

- How would you avoid floating-point division in the banana problem?
- What if **h** is smaller than the number of piles?
- How do you merge touching intervals like **`[1,3]`** and **`[3,5]`**?
- How would you handle streaming intervals?Minimum Eating Speed (Koko Eating Bananas)Koko loves to eat bananas. There are **n** piles of bananas; the i-th pile has **`piles[i]`** bananas. The guards will return in **h** hours. Koko can decide her constant bananas-per-hour eating speed **k**. Each hour she chooses one pile and eats **k** bananas from it. If that pile has fewer than **k** bananas, she eats all of them and does not eat more during that hour. An hour is spent on only one pile. Return the minimum integer speed **k** such that she can finish all the bananas within **h** hours. Example:
- **`piles = [3, 6, 7, 11]`**, **`h = 8`** -> **`4`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 5`** -> **`30`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 6`** -> **`23`**
  Constraints: **`1 <= piles.length <= h`**, **`1 <= piles[i] <= 10^9`**.

Constraints

1 <= piles.length <= h1 <= piles[i] <= 10^9h can be as large as 10^9

EXAMPLES

```
Input: ([3, 6, 7, 11], 8)
Expected Output: 4
Explanation: At k=4:
    ceil(3/4)+ceil(6/4)+ceil(7/4)+ceil(11/4) =
    1+2+2+3 = 8 hours. k=3 needs 1+2+3+4 = 10 > 8, so
    4 is minimal.
Input: ([30, 11, 23, 4, 20], 5)
Expected Output: 30
```

Explanation: With exactly 5 piles and 5 hours, each pile must be eaten in one hour, so k must be at least max(piles) = 30.

QUESTION

Hints

If speed k finishes in time, any larger speed also finishes in time. This monotonicity lets you binary search on the answer.The answer lies in [1, max(piles)]: speed 1 is the slowest, and max(piles) finishes every pile in at most one hour.Hours needed at speed k is sum of ceil(pile / k). Use integer-safe ceiling: (pile + k - 1) // k to avoid floating point.Merge Overlapping IntervalsGiven an array of intervals where **`intervals[i] = [start_i, end_i]`**, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start. Two intervals overlap when the next start is less than or equal to the current merged end

APPROACH

Binary search the speed with ceil(pile/speed) sum <= h; sort intervals by start and merge overlaps in one pass.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Bananas: piles and h are positive, h is at least the number of piles (otherwise impossible), one pile per hour, and an hour finishes at most one pile. Intervals: overlap includes touching (next start at most the current end).

BRUTE FORCE TO SAY OUT LOUD

Bananas: try every speed from 1 up. Intervals: compare every pair.

OPTIMAL IDEA

Bananas (LeetCode 875): the time function is monotone in the speed, so binary search the smallest speed in [1, max(piles)] with sum(ceil(p / s)) <= h. O(n log max). Intervals: sort by start and sweep, merging while the next start is at most the current end. O(n log n).

DRY-RUN THESE CASES

Bananas: h equal to the pile count (speed is the maximum), a single pile, large piles (overflow). Intervals: touching, nested, unsorted, one interval.

LIKELY FOLLOW-UPS

Bananas with a variable h or a floating-point speed. Intervals: insert one interval or count the merged result.

PITFALLS

Using floor instead of ceil. Merging with strict less-than.

HOW TO THINK ABOUT IT

Bananas: the time needed is non-increasing in the speed, so the smallest speed that finishes in h hours can be found by binary search over speeds, with a simple sum of ceil(pile / speed) as the check. Intervals: sort by start and sweep with one 'current' interval; the sort guarantees nothing earlier can still overlap.

HOW TO THINK ABOUT IT

Basis: LeetCode 875 (Koko Eating Bananas; one pile per hour, h at least the number of piles) and LeetCode 56 (Merge Intervals; overlapping when the next start is at most the current end).

BRUTE FORCE · Bananas: try every speed upward

BRUTE FORCE · PSEUDO-CODE

```
def brute(piles, h):
    for speed in range(1, max(piles) + 1):
        hours = 0
        for p in piles:
            hours += ceil(p / speed)
        if hours <= h:
            return speed
```

TIME	SPACE
O(max * n)	O(1)

WHY IT IS SLOW

Up to max(piles) speeds, each costing O(n); piles can be 10^9. Hours are non-increasing in the speed, so binary search over speeds is O(n log max).

BRUTE FORCE · Intervals: merge any overlapping pair until none is left

BRUTE FORCE · PSEUDO-CODE

```
def merge(intervals):
    intervals = list(intervals)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(intervals, 2):
            if overlaps(a, b):
                intervals.remove(a)
                intervals.remove(b)
                intervals.append(union(a, b))
                merged = True
                break
    return sorted(intervals)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Bananas: binary search the speed

TIME	SPACE
O(n log max)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Intervals: sort and sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Bananas: h varies per query, or the piles change.

The check is independent of the search; for many queries keep the piles sorted with prefix sums so the hours needed at speed s can be computed in O(log n) per distinct pile size.

FOLLOW-UP 2 Intervals: insert one new interval into an already merged list (LeetCode 57).

Copy intervals ending before the new one, merge the ones that overlap it, then copy the rest. O(n), no sort needed.

FOLLOW-UP 3 Bananas: the guards come back in fewer hours than piles.

Then no speed works (an hour can finish at most one pile): check h >= len(piles) first and return -1 otherwise.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve Banana Speed and Interval Merge
#
# During two technical phone screens for a new graduate
# software engineering role, the candidate was asked to
# solve two coding problems: minimum eating speed and
# merging overlapping intervals. ### Constraints &
# Assumptions - Provide clear algorithms, correctness
# reasoning, complexity, and edge cases. - For minimum
# eating speed, each hour can be spent on only one pile. -
# For interval merging, intervals overlap when the next
# start is less than or equal to the current merged end. -
# Use sorted or binary-search approaches where
# appropriate. ### Clarifying Questions to Ask - Are piles
# and h always positive? - Is h at least the number of
# piles? - Are interval endpoints inclusive? - Can
# intervals be empty or unsorted? ### Part 1 - Minimum
# Eating Speed Given piles and h, return the minimum
# integer speed k such that all piles can be eaten within
# h hours.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Bananas: h varies per query, or the piles change.
# 2. Intervals: insert one new interval into an already
# merged list (LeetCode 57).
# 3. Bananas: the guards come back in fewer hours than
# piles.
#
# INTERVIEW NOTES
# Bananas: the time needed is non-increasing in the speed,
# so the smallest speed that finishes in h hours can be
# found by binary search over speeds, with a simple sum of
# ceil(pile / speed) as the check. Intervals: sort by
# start and sweep with one 'current' interval; the sort
# guarantees nothing earlier can still overlap.
#
# BRUTE FORCE (say it first): Bananas: try every speed
# upward
# For speed 1, 2, 3, ... compute the hours needed and
# return the first speed that fits in h. Correct and the
# oracle; the speed can be 10^9. Time O(max * n). Space
# O(1).


# BRUTE FORCE (say it first): Bananas: try every speed
# upward | time O(max * n)
```

```
def banana_speed_linear(piles, h):
    for speed in range(1, max(piles) + 1):
        if sum(-(-p // speed) for p in piles) <= h:
            return speed


# HELPERS for the base question
def hours_needed(piles, speed):
    # each hour she works on ONE pile, so every pile
    # rounds up
    return sum(-(-p // speed) for p in piles)


# BASE QUESTION (optimal): Bananas: binary search the
# speed
# Idea: Hours needed are non-increasing in the speed, so
# keep lo = 1 and hi = max(piles) and halve: if speed mid
# finishes within h, try slower, otherwise faster. Use
# ceiling division (an hour finishes at most one pile).
def min_eating_speed(piles, h):
    # fewer hours than piles can never work
    if h < len(piles):
        return -1
    lo, hi = 1, max(piles)
    # more speed never takes more hours: binary search
    while lo < hi:
        mid = (lo + hi) // 2
        if hours_needed(piles, mid) <= h:
            hi = mid
        else:
            lo = mid + 1
    return lo


# PART 2: merge intervals
def merge_intervals(intervals):
    out = []
    for s, e in sorted(intervals):
        # overlap (sharing a point counts): extend the
        # last one
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

```
# FOLLOW-UP 2: Intervals: insert one new interval into an
# already merged list (LeetCode 57).
# Answer: Copy intervals ending before the new one, merge
# the ones that overlap it, then copy the rest. O(n), no
# sort needed.
def insert_interval(merged, new):
    # LeetCode 57: the list is already merged and sorted:
    # O(n)
    out, i, n = [], 0, len(merged)
    # 1) intervals that end before the new one starts
```

```
    while i < n and merged[i][1] < new[0]:
        out.append(merged[i])
        i += 1
    # 2) intervals that overlap it are absorbed
    s, e = new
    while i < n and merged[i][0] <= e:
        s, e = min(s, merged[i][0]), max(e, merged[i][1])
        i += 1
    out.append([s, e])
    # 3) the rest comes after
    out += merged[i:]
    return out


if __name__ == "__main__":
    assert min_eating_speed([3, 6, 7, 11], 8) == 4
    assert min_eating_speed([30, 11, 23, 4, 20], 5) == 30
    assert min_eating_speed([30, 11, 23, 4, 20], 6) == 23
    # fewer hours than piles: impossible
    assert min_eating_speed([1, 1, 1], 2) == -1
    assert hours_needed([3, 6, 7, 11], 4) == 8
    spans = [[1, 3], [2, 6], [8, 10], [15, 18]]
    merged = merge_intervals(spans)
    assert merged == [[1, 6], [8, 10], [15, 18]]
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []
    got = insert_interval([[1, 3], [6, 9]], [2, 5])
    assert got == [[1, 5], [6, 9]]
    assert insert_interval([], [4, 8]) == [[4, 8]]
    assert insert_interval([[1, 2], [5, 6]], [3, 4]) == [
        [1, 2], [3, 4], [5, 6]]
```

FUNCTION AND TEST CASES

Signature

```
def solution(piles: list[int], h: int) -> int:
```

Sample call

```
solution([3, 6, 7, 11], 8)
```

Test cases

Input	Expected
([3, 6, 7, 11], 8)	4
([30, 11, 23, 4, 20], 5)	30

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

HARD | IN APP LC 354 | PracHub | Technical Screen | 6 of 13

★ TOP 75: rank 14 in the Medium and Hard priority list

★ Find Maximum Envelope Nesting

POSTED 2026-03-18 | 216 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 52.8 | 30 DAYS 33 | PRIORITY 27/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #354 Russian Doll Envelopes

LeetCode match: LC 354 Russian Doll Envelopes

Posted on PracHub: 2026-03-18 (updated 2026-05-06)

Original: https://prachub.com/coding-questions/find-maximum-envelope-nesting

QUESTION

Overview: This question evaluates algorithmic problem-solving skills, particularly reasoning about multi-dimensional ordering, sequence optimization, and time-complexity analysis.

APPROACH

Sort by width asc and height desc on ties, then find the strictly increasing subsequence on heights with binary-search LIS (patience sorting). O(n log n).

HOW TO THINK ABOUT IT

Two dimensions are hard, one is easy. Sort by width so that width order is settled, then the question becomes a longest strictly increasing subsequence of heights. The trap is envelopes with equal width: they cannot nest, so sort equal widths by height descending, which prevents two of them from forming an increasing pair. The LIS is then done with patience sorting (binary search over tails).

Basis: LeetCode 354, Russian Doll Envelopes (strictly larger in both width and height).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
envelopes.sort()  # by width
best = [1] * len(envelopes)
for i in range(len(envelopes)):
    for j in range(i):
        w1, h1 = envelopes[j]
        w2, h2 = envelopes[i]
        if w1 < w2 and h1 < h2:
            best[i] = max(best[i], best[j] + 1)
answer = max(best)
```

BRUTE FORCE

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every envelope compares with all earlier ones, O(n^2). Sorting by width ascending and height descending, then a longest increasing subsequence on heights with binary search, is O(n log n).

OPTIMAL SOLUTION · Sort by width ascending and height descending, then LIS by binary search

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the actual chain of envelopes.

Remember, for each element, the index of the tail it extended (parent pointers), and rebuild from the last tail of the longest length. The sorted order and binary search are unchanged.

FOLLOW-UP 2 Three dimensions (boxes).

Sorting plus LIS no longer reduces the problem; use an O(n^2) DP, or CDQ divide and conquer with a Fenwick tree for O(n log^2 n).

FOLLOW-UP 3 Non-strict nesting (equal sizes allowed).

Sort by width ascending and height ascending, and use bisect_right (longest non-decreasing subsequence) instead of bisect_left.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find Maximum Envelope Nesting
#
# This question evaluates algorithmic problem-solving
# skills, particularly reasoning about multi-dimensional
# ordering, sequence optimization, and time-complexity
# analysis.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the actual chain of envelopes.
# 2. Three dimensions (boxes).
# 3. Non-strict nesting (equal sizes allowed).
#
# INTERVIEW NOTES
# Two dimensions are hard, one is easy. Sort by width so
# that width order is settled, then the question becomes a
# longest strictly increasing subsequence of heights. The
# trap is envelopes with equal width: they cannot nest, so
```

```
# sort equal widths by height descending, which prevents
# two of them from forming an increasing pair. The LIS is
# then done with patience sorting (binary search over
# tails).
#
# BRUTE FORCE (say it first): DP over pairs: longest chain
# ending at each envelope
# Sort by width, then best[i] is 1 plus the best best[j]
# over earlier envelopes that are strictly smaller in both
# dimensions. Correct and the oracle; quadratic. Time
# O(n^2). Space O(n).


from bisect import bisect_left, bisect_right


# BRUTE FORCE (say it first): DP over pairs: longest chain
# ending at each envelope | time O(n^2)
def nesting_dp(envelopes):
    env = sorted(envelopes)
    best = [1] * len(env)
    for i in range(len(env)):
        for j in range(i):
            if (
                env[j][0] < env[i][0]
                and env[j][1] < env[i][1]
            ):
                best[i] = max(best[i], best[j] + 1)
```

```
    return max(best, default=0)


# BASE QUESTION (optimal): Sort by width ascending and
# height descending, then LIS by binary search
# Idea: Sorting equal widths by decreasing height means
# two envelopes of the same width can never both appear in
# a strictly increasing height sequence.
def max_envelopes(envelopes):
    # sort by width, taller first on ties, then find the
    # longest
    # strictly increasing run of heights
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails = []
    for _, h in ordered:
        i = bisect_left(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)


# FOLLOW-UP 1: Return the actual chain of envelopes.
# Answer: Remember, for each element, the index of the
# tail it extended (parent pointers), and rebuild from the
# last tail of the longest length. The sorted order and
# binary search are unchanged.
def envelope_chain(envelopes):
    # also return the chain: remember what each element
    # extended
    ordered = sorted(envelopes,
                     key=lambda e: (e[0], -e[1]))
    tails, tail_idx = [], []
    prev = [-1] * len(ordered)
    for i, (_, h) in enumerate(ordered):
        k = bisect_left(tails, h)
        if k == len(tails):
            tails.append(h)
            tail_idx.append(i)
        else:
            tails[k] = h
            tail_idx[k] = i
```

```
        # the element that ended a chain one shorter
```

```
        prev[i] = tail_idx[k - 1] if k else -1
    chain = []
    i = tail_idx[-1] if tail_idx else -1
    while i != -1:
        chain.append(ordered[i])
        i = prev[i]
    return chain[::-1]


# FOLLOW-UP 2: Three dimensions (boxes).
# Answer: Sorting plus LIS no longer reduces the problem;
# use an O(n^2) DP, or CDQ divide and conquer with a
# Fenwick tree for O(n log^2 n).
def max_boxes_3d(boxes):
    # three sizes: the reduction no longer works, use
    # O(n^2)
    ordered = sorted(boxes)
    best = [1] * len(ordered)
    for i in range(len(ordered)):
        for j in range(i):
            a, b = ordered[j], ordered[i]
            if all(x < y for x, y in zip(a, b)):
                best[i] = max(best[i], best[j] + 1)
    return max(best, default=0)


# FOLLOW-UP 3: Non-strict nesting (equal sizes allowed).
# Answer: Sort by width ascending and height ascending,
# and use bisect_right (longest non-decreasing
# subsequence) instead of bisect_left.
def max_nested_non_strict(envelopes):
    # equal sizes may nest: sort both ways ascending and
    # use
    # bisect_right (a longest non-decreasing subsequence)
    tails = []
    for _, h in sorted(envelopes):
        i = bisect_right(tails, h)
        if i == len(tails):
            tails.append(h)
        else:
            tails[i] = h
    return len(tails)


if __name__ == "__main__":
```

```
    envs = [[5, 4], [6, 4], [6, 7], [2, 3]]
    assert max_envelopes(envs) == 3
    chain = envelope_chain(envs)
    assert len(chain) == 3
    # every envelope fits strictly inside the next
    for a, b in zip(chain, chain[1:]):
        assert a[0] < b[0] and a[1] < b[1]
    assert envelope_chain([]) == []
    # equal sizes: only the non-strict version nests them
    twins = [[3, 3], [3, 3], [4, 4]]
    assert max_envelopes(twins) == 2
    assert max_nested_non_strict(twins) == 3
    boxes = [(1, 1, 1), (2, 2, 2), (3, 1, 4), (3, 3, 3)]
    assert max_boxes_3d(boxes) == 3
```

FUNCTION AND TEST CASES

Signature

```
def solution(envelopes: list[list[int]]) -> int:
SAMPLE INPUT
[[5, 4], [6, 4], [6, 7], [2, 3]]
```

Test cases

Input	Expected
`[[5, 4], [6, 4], [6, 7], [2, 3]]`	`3`
`[]`	`0`

ALSO REPORTED · 1 more

Russian Doll Envelopes | 1Point3Acres | phone-screen | Single report

https://www.1point3acres.com/interview/problems/company/google/russian-doll-envelopes

Sorting heights descending on width-ties prevents same-width envelopes from chaining — easy to miss.
LIS via bisect_left on a tails array is the standard O(n log n) implementation.

Preparation: Write LIS O(n log n) from memory in under 8 min.
Justify the descending-height-on-tie trick out loud.
Plain LIS and the mountain-LIS variant make good drill partners.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Binary search

HARD | IN APP LC 315 | HackTheRounds | Phone Screen | 7 of 13

Count of Smaller Numbers After Self

STUDY: TALK | ALL TIME 33.2 | 30 DAYS 0 | PRIORITY 114/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Binary search

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #315 Count of Smaller Numbers After Self

LeetCode match: LC 315

QUESTION

Given an integer array nums, return an array counts where counts[i] is the number of smaller elements to the right of nums[i].

EXAMPLES

Example 1	
Input	nums = [5, 2, 6, 1]
Output	[2, 1, 1, 0]
Why	To the right of 5 there are 2 smaller: [2, 1]. To the right of 2 there is 1 smaller: [1]. To the right of 6 there is 1 smaller: [1]. To the right of 1 there are 0 smaller.

Example 2	
Input	nums = [1, 2, 3, 4]
Output	[0, 0, 0, 0]

Example 3	
Input	nums = [4, 3, 2, 1]
Output	[3, 2, 1, 0]

CONSTRAINTS

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4

APPROACH

Merge sort, binary indexed tree, or segment tree

HOW TO THINK ABOUT IT

For each element you want the count of smaller elements to its right: an order-statistics query on the elements seen so far if you scan right to left. A Fenwick tree over compressed values gives rank counts in O(log n). The same count falls out of merge sort: when a left element is placed after right elements that are smaller, those right elements are exactly the smaller numbers after it.

Basis: LeetCode 315, Count of Smaller Numbers After Self.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    counts = []
    for i in range(len(nums)):
        smaller = 0
        for j in range(i + 1, len(nums)):
            if nums[j] < nums[i]:
                smaller += 1
        counts.append(smaller)
    return counts
```

TIME	SPACE
O(n^2)	O(1) extra

WHY IT IS SLOW

Two nested loops, O(n^2): too slow at n = 10^5. A Fenwick tree over ranks scanned right to left (or merge sort) is O(n log n).

OPTIMAL SOLUTION · Fenwick tree over compressed values, scanning right to left

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it with merge sort instead of a Fenwick tree.

Sort indices by value with merge sort; whenever you place an element from the left half, add the number of right-half elements already placed (they are smaller). Same O(n log n), and it avoids coordinate compression.

FOLLOW-UP 2 Count larger numbers after self, or smaller numbers before self.

Larger after: query the suffix instead of the prefix of the Fenwick tree. Smaller before: scan left to right instead of right to left. **Cost:** still O(n log n) with the Fenwick tree.

FOLLOW-UP 3 Count inversions in an array.

The sum of the answers above is the number of inversions; merge sort counts them directly in O(n log n).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count of Smaller Numbers After Self
#
# Given an integer array nums, return an array counts
# where counts[i] is the number of smaller elements to the
# right of nums[i].
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do it with merge sort instead of a Fenwick tree.
# 2. Count larger numbers after self, or smaller numbers
# before self.
# 3. Count inversions in an array.
#
# INTERVIEW NOTES
# For each element you want the count of smaller elements
# to its right: an order-statistics query on the elements
# seen so far if you scan right to left. A Fenwick tree
# over compressed values gives rank counts in O(log n).
# The same count falls out of merge sort: when a left
# element is placed after right elements that are smaller,
# those right elements are exactly the smaller numbers
# after it.
#
# BRUTE FORCE (say it first): For each element count
# smaller elements to its right
# Two nested loops. Correct, and the oracle; too slow for
# n = 10^5. Time O(n^2). Space O(1) extra.


def brute(nums):
    return [sum(y < x for y in nums[i + 1:])
            for i, x in enumerate(nums)]


# BRUTE FORCE (say it first): For each element count
# smaller elements to its right | time O(n^2)
def count_smaller_brute(nums):
    return [
        sum(1 for y in nums[i + 1 :] if y < x)
        for i, x in enumerate(nums)
    ]


# HELPERS for the base question
class Fenwick:
    """Counts of seen values; prefix(i) = how many seen
    values have a rank below i."""

    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i):
        i += 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total


def ranks(nums):
    # coordinate compression: value -> 0, 1, 2 ... by size
    return {v: i for i, v in enumerate(sorted(set(nums)))}


# BASE QUESTION (optimal): Fenwick tree over compressed
# values, scanning right to left
# Idea: Compress values to ranks 1..m. Scan from the
# right: the answer for x is the number of already
# inserted values with rank less than rank(x) (a prefix
# sum query), then insert rank(x). Both operations are
# O(log m).
def count_smaller_after(nums):
    # go from the right; before inserting x ask how many
    # already-seen values are strictly smaller than x
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    out = []
    for x in reversed(nums):
        out.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    return out[::-1]


# FOLLOW-UP 1: Do it with merge sort instead of a Fenwick
# tree.
# Answer: Sort indices by value with merge sort; whenever
# you place an element from the left half, add the number
# of right-half elements already placed (they are
# smaller). Same O(n log n), and it avoids coordinate
# compression.
def count_smaller_merge(nums):
    # follow-up: merge sort on indexes. When a left item
    # is
    # placed, every right item already placed is smaller.
    counts = [0] * len(nums)
    order = list(range(len(nums)))

    def sort(lo, hi):
        if hi - lo <= 1:
            return
        mid = (lo + hi) // 2
        sort(lo, mid)
        sort(mid, hi)
        merged, i, j = [], lo, mid
        while i < mid or j < hi:
            left_first = False
            if i < mid and j < hi:
                a, b = nums[order[i]], nums[order[j]]
                left_first = a <= b
            if j >= hi or left_first:
                counts[order[i]] += j - mid
                merged.append(order[i])
                i += 1
            else:
                merged.append(order[j])
                j += 1
        order[lo:hi] = merged

    sort(0, len(nums))
    return counts


# FOLLOW-UP 2: Count larger numbers after self, or smaller
# numbers before self.
# Answer: Larger after: query the suffix instead of the
# prefix of the Fenwick tree. Smaller before: scan left to
# right instead of right to left. Cost: still O(n log n)
# with the Fenwick tree.
def count_larger_after(nums):
    # follow-up: the suffix of the tree instead of the
    # prefix
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    out, seen = [], 0
    for x in reversed(nums):
        out.append(seen - tree.prefix(rank[x] + 1))
        tree.add(rank[x])
        seen += 1
```

```
    return out[::-1]


# FOLLOW-UP 2: Count larger numbers after self, or smaller
# numbers before self.
# Answer: Larger after: query the suffix instead of the
# prefix of the Fenwick tree. Smaller before: scan left to
# right instead of right to left. Cost: still O(n log n)
# with the Fenwick tree.
def count_smaller_before(nums):
    # follow-up: scan from the left instead
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    out = []
    for x in nums:
        out.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    return out


# FOLLOW-UP 3: Count inversions in an array.
# Answer: The sum of the answers above is the number of
# inversions; merge sort counts them directly in O(n log
# n).
def inversions(nums):
    # follow-up: the answers added up are the inversions
    return sum(count_smaller_merge(nums))


if __name__ == "__main__":
    sample = [5, 2, 6, 1]
    assert count_smaller_after(sample) == [2, 1, 1, 0]
    assert count_smaller_after([-1]) == [0]
    assert count_smaller_after([-1, -1]) == [0, 0]
    assert count_smaller_after([]) == []
    assert count_larger_after(sample) == [1, 1, 0, 0]
```

```
    assert count_smaller_before(sample) == [0, 0, 2, 0]
    assert inversions([2, 4, 1, 3, 5]) == 3
    assert inversions([3, 2, 1]) == 3
    import random
    for _ in range(300):
        nums = [random.randint(-5, 5)
                for _ in range(random.randint(0, 12))]
        want = brute(nums)
        assert count_smaller_after(nums) == want
        assert count_smaller_merge(nums) == want
        assert inversions(nums) == sum(want)
        bigger = [sum(y > x for y in nums[i + 1:])
                  for i, x in enumerate(nums)]
        assert count_larger_after(nums) == bigger
        before = [sum(y < x for y in nums[:i])
                  for i, x in enumerate(nums)]
        assert count_smaller_before(nums) == before
```

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

Site tags: array, hashmap, sorting, medium

MEDIUM | 1Point3Acres | phone-screen | 8 of 13

Progressive Array Partitioning

ASKED 2026-08-11 | 70 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 198/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

EXAMPLES

Stage 1

```
[1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the
    four value-groups have different sizes.
[1, 2, 3] -> false because every value-group has size
    one.
```

Stage 2

```
[1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true.
[4, 5, 6, 7, 7, 8, 9] -> false.
```

APPROACH

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the groups are and the rule for each stage. From the examples, the first stage looks like: true when the counts of all distinct values are different from each other (1:1, 2:2, 3:3, 4:4 is true; all counts 1 is false). The second stage is not clear from the examples alone, so ask for its exact rule and test it on both examples before coding.

BRUTE FORCE TO SAY OUT LOUD

Try every way to group the elements and test the rule. Exponential.

OPTIMAL IDEA

For stage one: count each value with a hash map, then check that the multiset of counts has no repeats (a set of counts has the same size as the number of values). O(n). Keep the data model (a counter) easy to adapt, because the second stage arrives only after the first is written.

DRY-RUN THESE CASES

The four examples in the statement, an empty array, a single value, many values with equal counts.

LIKELY FOLLOW-UPS

Stage two, which changes the grouping rule; keep helper functions small so it is a local change.

PITFALLS

Coding stage two from a guess. Not reconciling the rule with both examples.

HOW TO THINK ABOUT IT

The statement arrives in stages, so keep the data model easy to adapt: count the values first, because any rule about 'groups of equal values' is a rule about the counts. Stage 1 is then 'are all counts different', which is a set-size comparison. Before stage 2, reconcile the new rule with every example; here a natural guess (LeetCode 659) contradicts the second example, which is exactly the situation where you ask.

Basis: Written. Stage 1 is inferred from the examples: true when every distinct value occurs a different number of times. The Stage 2 rule is NOT recoverable from the two examples, so only a candidate reading is shown; ask the interviewer for the exact rule and test it against both examples before coding.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    counts = Counter(a)
    for x, y in combinations(counts, 2):
        if counts[x] == counts[y]:
            return False
    return True
```

TIME	SPACE
O(d^2) where d is the number of distinct values	**O(d)**

WHY IT IS SLOW

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION · Counter, then compare the set of counts with the number of values

TIME	SPACE
O(n)	**O(d)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

FOLLOW-UP 2 Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Progressive Array Partitioning
#
# Examples: Stage 1 [1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true
# because the four value-groups have different sizes. [1,
# 2, 3] -> false because every value-group has size one.
# Stage 2 [1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true. [4, 5, 6
,
# 7, 7, 8, 9] -> false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Stage 2 arrives. What do you do?
# 2. Keep the code adaptable between stages.
#
# INTERVIEW NOTES
# The statement arrives in stages, so keep the data model
# easy to adapt: count the values first, because any rule
# about 'groups of equal values' is a rule about the
# counts. Stage 1 is then 'are all counts different',
# which is a set-size comparison. Before stage 2,
# reconcile the new rule with every example; here a
# natural guess (LeetCode 659) contradicts the second
# example, which is exactly the situation where you ask.
#
# BRUTE FORCE (say it first): Compare every pair of counts
# Count each value, then check every pair of counts for
# equality. Fine for small d and a clear statement of the
# rule. Time O(d^2) where d is the number of distinct
# values. Space O(d).


from collections import Counter


# BRUTE FORCE (say it first): Compare every pair of counts
# | time O(d^2) where d is the number of distinct values
def distinct_counts_brute(nums):
    counts = list(Counter(nums).values())
    return all(
        counts[i] != counts[j]
        for i in range(len(counts))
        for j in range(i + 1, len(counts))
    )


# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of values
```

```
# Idea: All counts are different exactly when the set of
# counts has as many elements as there are distinct
# values. One pass to count, one pass over the counts.
def stage1(counts):
    # true when every value occurs a different number of
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)


# FOLLOW-UP 1: Stage 2 arrives. What do you do?
# Answer: Do not guess. Write down what the second example
# pair implies, and check any candidate rule against BOTH.
def stage2_candidate(nums, min_len=3):
    # ONE reading of stage 2 (LeetCode 659): split the
    # array into runs of consecutive values, each at
    # least min_len long. Check it against BOTH examples.
    left, ends = Counter(nums), Counter()
    for x in nums:
        if left[x] == 0:
            continue
        left[x] -= 1
        if ends[x - 1] > 0:
            # extend a run that ended at x - 1
            ends[x - 1] -= 1
            ends[x] += 1
        elif left[x + 1] > 0 and left[x + 2] > 0:
            # start a new run x, x + 1, x + 2
            left[x + 1] -= 1
            left[x + 2] -= 1
            ends[x + 2] += 1
        else:
            return False
    return True


# HELPERS for the checks below
def counts_of(nums):
    # every rule about groups of equal values is a
    # rule about these counts, so compute them once
    return Counter(nums)


# each stage is one small function, so a new rule
# replaces one entry instead of the whole pipeline
RULES = {1: lambda nums: stage1(counts_of(nums)),
```

```
         2: stage2_candidate}


def check(stage, nums):
    return RULES[stage](nums)


if __name__ == "__main__":
    # stage 1 examples
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])
    # stage 2 example that the candidate gets right
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
    # the second stage-2 example says False, but this
    # reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True
```

NOTES FROM CANDIDATES

The interviewer presents Stage 2 only after the first implementation, so keep the initial data model easy to adapt.
Before coding the straight variant, reconcile the length rule with both examples; otherwise a superficially valid decomposition can contradict the expected result.
Evaluation emphasized the efficiency of the chosen data structures, not only whether the sample cases passed.
Solution skeleton
Stage 1: count each distinct value, then verify that the frequency values are themselves unique. A hash map plus a set gives O(n) time and O(u) space for u distinct values.
Stage 2: first pin down a group size k. For the fixed-size interpretation, reject when n % k != 0, process keys in ascending order, and whenever the smallest remaining key x has multiplicity c, remove c copies from every key in [x, x + k). Any missing count makes the partition impossible. Sorting the u keys gives O(n + u log u) time and O(u) space.
Do not let length-one groups make every input trivially valid unless the interviewer explicitly allows them; the negative example requires a stronger group-size contract.

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

Site tags: data-structure, sorting, sliding-window, medium, ambiguous-spec

MEDIUM | 1Point3Acres | onsite-coding | 9 of 13

Streaming Points: Emit K Within Distance

ASKED 2026-06-23 | 119 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 199/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-points-k-within-distance

QUESTION

Process a stream of float positions. Whenever three points satisfy the distance condition, output and remove those three points; a follow-up generalizes from 3 points to K points. The important ambiguity is whether the condition means a span of length **d** or pairwise distance within **d**.SWEdata-structuresorting sliding-windowmediumambiguous-specFrequencySingle reportLast asked2026-06-23Stageonsite-coding

APPROACH

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the condition means all K points fit in a span of length d or every pair is within d (on a line these are equivalent), whether to emit as soon as a group exists, and how to choose among overlapping groups.

BRUTE FORCE TO SAY OUT LOUD

Keep a list, and after each point sort it and scan for a group. O(n log n) per point.

OPTIMAL IDEA

Keep points in a sorted list (bisect.insort). After inserting x only windows containing x can be new, so look at the K-wide windows of sorted neighbors around its index; if the largest minus the smallest in a window is at most d, emit and remove those K points. O(log n) to locate plus the window size.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

Exactly d apart, duplicates, points arriving out of order, two overlapping candidate groups, K = 1.

LIKELY FOLLOW-UPS

General K (the same code), 2D points, or a time-based expiration.

PITFALLS

Using epsilon-free float comparison blindly. Checking only the K points directly before x.

HOW TO THINK ABOUT IT

On a line, K points are all within distance d exactly when the largest minus the smallest is at most d, so only K consecutive points in sorted order can form a group. Keep the points sorted and look only where something changed: a window that contains the new point, or the seam left by removed points.

Basis: Written. Assumed rules (confirm them): after each new point, while some K consecutive points (in sorted order) span at most d, emit the lowest such group and remove it; on a line 'all within d' and 'span at most d' are the same condition.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
points = []
def on_point(x):
    points.append(x)
    points.sort()
    emitted = []
    while True:
        group = find_group()
        if group is None:
            return emitted
        emitted.append(group)
        remove(points, group)
def find_group():
    for i in range(len(points) - k + 1):
        if points[i + k - 1] - points[i] <= d:
            return points[i:i + k]
    return None
```

TIME	SPACE
O(n log n) per point	**O(n)**

WHY IT IS SLOW

Every arrival re-sorts and rescans the whole list, O(n log n). A sorted list with binary search checks only the windows around the new point.

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

TIME	SPACE
O(log n + K) per check, plus list shifting	**O(n)**

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if 'within d' means every pair, and points are in 2D?

On a line pairwise distance equals the span, so nothing changes. In 2D it becomes a clique problem: candidates must be neighbours in a spatial grid of cell size d, and you test the K-subset of nearby points; that is much harder, so ask whether the 1D reading is intended.

FOLLOW-UP 2 Floating-point coordinates.

Compare with the exact subtraction hi - lo <= d (not by building a range around the target), and agree on an epsilon policy only if the interviewer wants one. Do not compare floats for equality.

FOLLOW-UP 3 Several groups qualify at once.

Agree on a rule (lowest start, as coded) so output is deterministic, and note that emitting one group can create or destroy others.

FOLLOW-UP 4 The condition is 'every pair within d', not 'span at most d'.

On a line they are the same: the largest pairwise distance is max - min, so checking the span is enough (verified against checking all pairs). In 2D they differ and become a clique problem.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Points: Emit K Within Distance
#
# Process a stream of float positions. Whenever three
# points satisfy the distance condition, output and remove
# those three points; a follow-up generalizes from 3
# points to K points. The important ambiguity is whether
# the condition means a span of length d or pairwise
# distance within d.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if 'within d' means every pair, and points are
# in 2D?
# 2. Floating-point coordinates.
# 3. Several groups qualify at once.
# 4. The condition is 'every pair within d', not 'span at
# most d'.
#
# INTERVIEW NOTES
# On a line, K points are all within distance d exactly
# when the largest minus the smallest is at most d, so
# only K consecutive points in sorted order can form a
# group. Keep the points sorted and look only where
# something changed: a window that contains the new point,
# or the seam left by removed points.
#
# BRUTE FORCE (say it first): Re-sort and scan every
# window after each point
# Sort all points, slide a window of K, and emit the
# lowest group that fits, repeating until none does.
# Obviously correct; the whole list is re-examined for
# every new point. Time O(n log n) per point. Space O(n).


from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Re-sort and scan every
# window after each point | time O(n log n) per point
class PointStreamBrute:
    def __init__(self, k, d):
        self.k, self.d, self.pts = k, d, []

    def add(self, x):
        self.pts.append(x)
        out = []
        while True:
            self.pts.sort()
            for i in range(len(self.pts) - self.k + 1):
                if (
                    self.pts[i + self.k - 1]
                    - self.pts[i]
                    <= self.d
                ):
                    out.append(self.pts[i : i + self.k])
                    del self.pts[i : i + self.k]
                    break
            else:
                return out


# BASE QUESTION (optimal): Sorted list, check only windows
# touching the change
# Idea: Insert with bisect. Because no window existed
# before, a new window must contain the new point, so test
# the K windows that include its position.
class PointStream:
    def __init__(self, k, d):
        # emit when k points fit in a span of d
        self.k, self.d = k, d
        # all points still waiting, kept sorted
        self.points = []

    def window_start(self, pos):
        # first window of k neighbours that includes pos
        lo = max(0, pos - self.k + 1)
        hi = min(pos, len(self.points) - self.k)
        for i in range(lo, hi + 1):
            last = self.points[i + self.k - 1]
            span = last - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        # insert in order, then emit every window made by
        # x
        insort(self.points, x)
        out, pos = [], bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
```

```
            if i is None:
                return out
            # remove the group so it is emitted only once
            out.append(self.points[i:i + self.k])
            del self.points[i:i + self.k]
            pos = i


# FOLLOW-UP 4: The condition is 'every pair within d', not
# 'span at most d'.
# Answer: On a line they are the same: the largest
# pairwise distance is max - min, so checking the span is
# enough (verified against checking all pairs). In 2D they
# differ and become a clique problem.
def all_pairs_within(group, d):
    # "every pair within d": on a line this equals the
    # span
    pairs = [(a, b) for a in group for b in group]
    return all(abs(a - b) <= d for a, b in pairs)


if __name__ == "__main__":
    s = PointStream(3, 1.0)
    assert s.add(0.5) == []
    assert s.add(5.0) == []
    assert s.add(1.0) == []
    # 0.5, 1.0, 1.4 span 0.9 <= 1.0
    out = s.add(1.4)
    assert out == [[0.5, 1.0, 1.4]]
    # the far point is still waiting
    assert s.points == [5.0]
    # on a line, span and pairwise checks agree
    for group in out:
        assert all_pairs_within(group, 1.0)
        assert max(group) - min(group) <= 1.0
    # k = 2 groups closest neighbours as they appear
    t = PointStream(2, 0.5)
    assert t.add(1.0) == []
    assert t.add(1.4) == [[1.0, 1.4]]
```

NOTES FROM CANDIDATES

A balanced ordered set / TreeSet-style structure is a natural fit because each new point only needs nearby neighbors.
Be careful with floating-point comparison and epsilon policy.
A wrong clarification can lead to checking subset(target, target + d) when the interviewer expects a wider candidate range around the target. Confirm the predicate with a tiny numeric example before coding.

PREPARATION

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

MEDIUM | PracHub | Technical Screen | 10 of 13

★ TOP 75: rank 16 in the Medium and Hard priority list

★ Find Containing Range

POSTED 2026-06-04 | 138 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 65.2** | **30 DAYS 41.1** | **PRIORITY 12/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #56 Merge Intervals

LeetCode match: No direct match; related LC 56 Merge Intervals + binary search (inferred from overview)

Posted on PracHub: 2026-06-04 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/find-containing-range

QUESTION

Overview: This question evaluates the ability to design appropriate data structures and apply algorithmic reasoning for efficient interval containment queries, emphasizing preprocessing, correctness, and time-space trade-offs.

APPROACH

Sort/merge ranges once, then answer each query with binary search on starts and check the end. O(n log n) preprocessing, O(log n) per query.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the given ranges can overlap, what to return when a point lies in several ranges (the original range, the merged range, or all of them), and what to return when none contains it.

BRUTE FORCE TO SAY OUT LOUD

For each query scan all ranges. O(n) per query.

OPTIMAL IDEA

Sort the ranges by start and merge overlaps once, O(n log n). For each query binary search for the last merged range whose start is at most the point, then check that the point is at most its end. O(log n) per query. If the original (unmerged) range is required and ranges may overlap, use an interval tree or keep the maximum end seen in a prefix array.

DRY-RUN THESE CASES

A point before all ranges, after all ranges, in a gap, exactly on an endpoint, inside two overlapping ranges.

LIKELY FOLLOW-UPS

Insert and delete ranges (a balanced tree), or many point queries at once (sort and sweep).

PITFALLS

INTERVIEW-DAY PLAN

Binary searching without merging when ranges overlap. Mishandling the endpoint rule.

HOW TO THINK ABOUT IT

Merge the ranges once so they are disjoint and sorted; then 'which range contains x' is 'the last range whose start is at most x, and does its end reach x'. That is one binary search per query. Be clear whether the answer is the original range or the merged one, since they differ when ranges overlap.

Basis: Written (compare LeetCode 56 and a binary search for the range). Assumed rule (confirm it): given possibly overlapping ranges, answer queries 'which merged range contains x?', or None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def find_containing(ranges, x):
    for start, end in ranges:
        if start <= x <= end:
            return (start, end)
    return None
```

TIME	SPACE
O(n) per query	**O(n)**

WHY IT IS SLOW

Every query walks all n ranges. Merging once (O(n log n)) and binary searching the sorted starts makes each query O(log n).

OPTIMAL SOLUTION · Merge once, then binary search the starts

TIME	SPACE
O(n log n) to build, O(log n) per query	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return every original range that contains x, not the merged one.

Sort by start and consider only ranges starting at or before x (one bisect); among those, keep the ones whose end is at least x. With many overlaps use an interval tree for O(log n + k) per query.

FOLLOW-UP 2 Ranges are inserted and deleted over time.

Keep the disjoint ranges in a sorted container (balanced tree); an insert merges neighbours, and a query is the same predecessor search. O(log n) per operation.

FOLLOW-UP 3 Queries are given in bulk.

Sort the queries and sweep once over the merged ranges with two pointers: O((n + q) log) overall instead of a binary search per query.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find Containing Range
#
# This question evaluates the ability to design
# appropriate data structures and apply algorithmic
# reasoning for efficient interval containment queries,
# emphasizing preprocessing, correctness, and time-space
# trade-offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return every original range that contains x, not the
# merged one.
# 2. Ranges are inserted and deleted over time.
# 3. Queries are given in bulk.
#
# INTERVIEW NOTES
# Merge the ranges once so they are disjoint and sorted;
# then 'which range contains x' is 'the last range whose
# start is at most x, and does its end reach x'. That is
# one binary search per query. Be clear whether the answer
# is the original range or the merged one, since they
# differ when ranges overlap.
#
# BRUTE FORCE (say it first): Scan all ranges for each
# query
# Merge the ranges (or not) and for each query test every
# range for containment. Correct and the oracle; each
# query walks the whole list. Time O(n) per query. Space
# O(n).


from bisect import bisect_left, bisect_right


# BRUTE FORCE (say it first): Scan all ranges for each
# query | time O(n) per query
def containing_range_scan(ranges, x):
    merged = []
    for s, e in sorted(ranges):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    for s, e in merged:
        if s <= x <= e:
            return (s, e)
    return None


# BASE QUESTION (optimal): Merge once, then binary search
# the starts
# Idea: After sorting and merging, the starts are
# increasing, so bisect_right(starts, x) - 1 is the only
# candidate; it contains x if its end is at least x.
# Preprocessing is paid once.
# FOLLOW-UP 1: Return every original range that contains
# x, not the merged one.
# Answer: Sort by start and consider only ranges starting
# at or before x (one bisect); among those, keep the ones
# whose end is at least x. With many overlaps use an
# interval tree for O(log n + k) per query.
class RangeIndex:
    """Merged, sorted, disjoint closed ranges."""

    def __init__(self, ranges=()):
        self.starts, self.ends = [], []
        self.original = sorted(ranges)
        for s, e in self.original:
            self.insert(s, e)

    # FOLLOW-UP 2: ranges can be inserted over time
    def insert(self, s, e):
        # first range ending at or after s ...
        i = bisect_left(self.ends, s)
        # ... up to the last range starting at or before e
        j = bisect_right(self.starts, e)
        if i < j:
            s = min(s, self.starts[i])
            e = max(e, self.ends[j - 1])
        # replace everything the new range touches
        self.starts[i:j] = [s]
        self.ends[i:j] = [e]

    def find(self, x):
        # the merged range containing x, or None
        i = bisect_right(self.starts, x) - 1
        if i >= 0 and self.ends[i] >= x:
            return self.starts[i], self.ends[i]
        return None

    # FOLLOW-UP 1: every ORIGINAL range containing x
    def find_original(self, x):
        # every ORIGINAL range that contains x
        k = bisect_right([s for s, _ in self.original], x)
        return [r for r in self.original[:k] if r[1] >= x]

    # FOLLOW-UP 3: queries given in bulk
    def find_many(self, queries):
        # bulk: sort the queries and sweep once
        order = sorted(range(len(queries)),
                       key=lambda i: queries[i])
        out, j = [None] * len(queries), 0
        for i in order:
            x = queries[i]
            # skip ranges that end before x
            while j < len(self.ends) and self.ends[j] < x:
                j += 1
            inside = j < len(self.starts)
            if inside and self.starts[j] <= x:
                out[i] = (self.starts[j], self.ends[j])
        return out


if __name__ == "__main__":
    idx = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    assert idx.find(4) == (1, 6)
    assert idx.find(7) is None
    assert idx.find(8) == (8, 10)
    assert idx.find(0) is None and idx.find(99) is None
    # the original ranges that cover 2
    assert idx.find_original(2) == [(1, 3), (2, 6)]
    # touching ranges merge: (6, 8) joins the first two
    # blocks
    idx.insert(6, 8)
    assert idx.find(7) == (1, 10)
    assert idx.find(12) is None
    qs = [4, 20, 16, 9]
    assert idx.find_many(qs) == [idx.find(q) for q in qs]
```

FUNCTION AND TEST CASES

Signature

```
def solution(ranges: list[list[int]], queries: list[int])
-> list[list[int]]:
```

Sample call

```
solution([[1, 3], [50, 100]], [2, 50, 10])
```

Test cases

Input	Expected
`([[1, 3], [50, 100]], [2, 50, 10])`	`[[1, 3], [50, 100], None]`
`([[5, 8], [-10, -5], [0, 0]], [-7, 0, 8, 9, -11])`	`[[-10, -5], [0, 0], [5, 8], None, None]`

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

MEDIUM | PracHub | Technical Screen | 11 of 13

★ TOP 75: rank 22 in the Medium and Hard priority list

★ Check if all substrings are anagrams of words

POSTED 2026-03-11 | 223 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 51.4 | PRIORITY 16/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #49 Group Anagrams

LeetCode match: No direct match; related LC 49 Group Anagrams

Posted on PracHub: 2026-03-11 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/check-if-all-substrings-are-anagrams-of-words

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **`t = s[i..j]`** with **`|t| >= 3`**, there exists some word **`w in dict`** such that **`w`** is a permutation of **`t`** (same multiset of characters). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**. - "Can be rearranged into a word" means the substring and the dictionary word have exactly the same character counts. Example:
- If **`t = "act"`**, it passes if the dictionary contains any of **`{ "act", "cat", "tac" }`**.

APPROACH

Precompute sorted-letter (or count-vector) signatures of dictionary words into a set; for each substring of length >= 3 compute its signature and look it up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether every substring of length at least 3 must be rearrangeable into some dictionary word, the alphabet (letters only), and the dictionary size and word lengths.

BRUTE FORCE TO SAY OUT LOUD

For every substring, compare against every dictionary word by sorted letters. Very slow.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Preprocess the dictionary into a set of signatures, where a signature is the sorted letters or a 26-count tuple, and also record the set of word lengths present. For each start i extend j one letter at a time, updating a count vector incrementally; once the length is at least 3, look up its signature (skip lengths that no word has) and return false at the first miss. O(n^2 * 26) time.

DRY-RUN THESE CASES

A string shorter than 3, a failing length that has no dictionary words, a long repeated letter, all substrings valid.

LIKELY FOLLOW-UPS

Return the failing substring, or a larger alphabet (use a hash of counts).

PITFALLS

Sorting each substring from scratch, which costs an extra log factor. Forgetting that a signature must match the length too (a count tuple already does).

HOW TO THINK ABOUT IT

Anagrams share a signature: the multiset of letters (a sorted string or a 26-count tuple). So turn the dictionary into a set of signatures once, and test each substring by its signature. Extending a substring by one letter changes its count vector in O(1), so you do not rebuild the signature from scratch.

Basis: Written; every substring of length >= 3 must be a permutation of some dictionary word (compare Group Anagrams, LeetCode 49).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s, words):
    for i in range(len(s)):
        for j in range(i + 3, len(s) + 1):
            key = sorted(s[i:j])
            match = [w for w in words
                     if key == sorted(w)]
            if not match:
                return False
    return True
```

TIME	SPACE
O(n^2 * W * L log L)	O(1) extra

WHY IT IS SLOW

Each substring is compared with every word and sorted again: O(n^2 * W * L log L). A set of word signatures, with the substring's count vector updated incrementally, makes each check O(alphabet).

OPTIMAL SOLUTION · Set of signatures, incremental count vectors

TIME	SPACE
O(n^2 * A) for alphabet size A	O(dictionary size)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A bigger alphabet or Unicode.

Replace the fixed array by a Counter and use a frozen tuple of sorted (letter, count) pairs, or a hash of the counts, as the signature. **Cost:** O(n^2 * distinct letters in a substring) with a Counter signature instead of a fixed array.

FOLLOW-UP 2 Only substrings of lengths that exist in the dictionary need checking.

Record the set of word lengths; a substring whose length is not in that set can never match, so you can return False the first time such a length is reached (assuming all substrings of length >= 3 up to n must match).

FOLLOW-UP 3 Subsets of the letters (any sub-multiset of a word) instead of a full permutation.

Then the test is containment of count vectors, which is a subset-of-multiset query; use a sorted signature trie or precompute all sub-multisets for short words.

FOLLOW-UP 4 Report the first failing substring instead of True or False.

Test substrings by length (shortest first) so the report is a smallest counterexample; the signature set makes each test one lookup after sorting the substring's letters.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check if all substrings are anagrams of
# words
#
# You are given a string s (letters only) and access to an
# English dictionary dict (a set of valid words). Return
# true if **every contiguous substring** of s with length
# **>= 3** can be **rearranged (anagrammed)** into a valid
# dictionary word. Formally, for every substring t =
# s[i..j] with |t| >= 3, there exists some word w in dict
# such that w is a permutation of t (same multiset of
# characters). Otherwise return false. Clarifications: -
# Substring means **contiguous**. - “Can be rearranged
# into a word” means the substring and the dictionary word
# have exactly the same character counts. Example: - If t
# = "act", it passes if the dictionary contains any of {
# "act", "cat", "tac" }.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A bigger alphabet or Unicode.
# 2. Only substrings of lengths that exist in the
# dictionary need checking.
# 3. Subsets of the letters (any sub-multiset of a word)
# instead of a full permutation.
# 4. Report the first failing substring instead of True or
# False.
#
# INTERVIEW NOTES
# Anagrams share a signature: the multiset of letters (a
# sorted string or a 26-count tuple). So turn the
# dictionary into a set of signatures once, and test each
# substring by its signature. Extending a substring by one
# letter changes its count vector in O(1), so you do not
# rebuild the signature from scratch.
#
# BRUTE FORCE (say it first): Compare each substring with
# every word's sorted letters
# For each substring, sort its letters and compare with
# the sorted letters of every dictionary word. Correct,
# and the dictionary is rescanned for each substring. Time
# O(n^2 * W * L log L). Space O(1) extra.


from collections import Counter
```

```
# BRUTE FORCE (say it first): Compare each substring with
# every word's sorted letters | time O(n^2 * W * L log L)
def _sig(w):
    return tuple(sorted(w))


def all_anagram_brute(s, words):
    sigs = [_sig(w) for w in words]
    return all(
        _sig(s[i:j]) in sigs
        for i in range(len(s))
        for j in range(i + 3, len(s) + 1)
    )


# HELPERS for the base question
ALPHABET = "abcdefghijklmnopqrstuvwxyz"


# HELPERS for follow-up 4
def signature(text):
    # the letters in sorted order: equal for anagrams
    return "".join(sorted(text))


# BASE QUESTION (optimal): Set of signatures, incremental
# count vectors
# Idea: Precompute the set of count-vector signatures of
# the dictionary words. For each start extend one letter
# at a time, update the count vector in O(1), and look up
# its tuple (once the length is at least 3).
def all_anagram_words(s, words, min_len=3):
    # every substring needs a dictionary word with the
    # same
    # letters. Count vectors are updated as the window
    # grows.
    index = {c: i for i, c in enumerate(ALPHABET)}
    sigs = set()
    for w in words:
        v = [0] * len(ALPHABET)
        for ch in w:
            v[index[ch]] += 1
        sigs.add(tuple(v))
    for i in range(len(s)):
```

```
        v = [0] * len(ALPHABET)
```

```
        for j in range(i, len(s)):
            v[index[s[j]]] += 1
            long_enough = j - i + 1 >= min_len
            if long_enough and tuple(v) not in sigs:
                return False
    return True


# FOLLOW-UP 1: A bigger alphabet or Unicode.
# Answer: Replace the fixed array by a Counter and use a
# frozen tuple of sorted (letter, count) pairs, or a hash
# of the counts, as the signature.
def all_anagram_words_any_alphabet(s, words, min_len=3):
    # Unicode or a big alphabet: a frozen Counter as
    # signature
    sigs = {frozenset(Counter(w).items()) for w in words}
    for i in range(len(s)):
        count = Counter()
        for j in range(i, len(s)):
            count[s[j]] += 1
            key = frozenset(count.items())
            if j - i + 1 >= min_len and key not in sigs:
                return False
    return True


# FOLLOW-UP 4: Report the first failing substring instead
# of True or False.
# Answer: Test substrings by length (shortest first) so
# the report is a smallest counterexample; the signature
# set makes each test one lookup after sorting the
# substring's letters.
def first_failing(s, words, min_len=3):
    # smallest counterexample: lengths from short to long
    sigs = {signature(w) for w in words}
    lengths = {len(w) for w in words}
    for length in range(min_len, len(s) + 1):
        # no word has this length: the first window fails
        if length not in lengths:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if signature(piece) not in sigs:
                return piece
    return None
```

```
if __name__ == "__main__":
    words = {"cat", "act", "tac", "tact", "cats"}
    assert all_anagram_words("cat", words)
    assert all_anagram_words("tca", words)
    assert not all_anagram_words("tcaz", words)
    assert all_anagram_words_any_alphabet("cat", words)
    assert first_failing("cat", words) is None
    # "catz" fails at length 3? "atz" is not an anagram
    assert first_failing("catz", words) == "atz"
    assert all_anagram_words("ab", set())
    assert first_failing("aaaa", {"aaa"}) == "aaaa"
```

FUNCTION AND TEST CASES

Signature

```
def solution(s: str, words: list[str]) -> bool:
```

Sample call

```
solution("act", ["cat", "dog"])
```

Test cases

Input	Expected
("act", ["cat", "dog"])	True
("abca", ["cab", "abca", "zzz"])	True

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

MEDIUM | PracHub | Technical Screen | 12 of 13

★ TOP 75: rank 35 in the Medium and Hard priority list

★ Find shared objects across two log files

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 50.4 | 30 DAYS 46.9 | PRIORITY 23/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: the interview site's own solution

LeetCode match: LC 349 Intersection of Two Arrays

Posted on PracHub: 2025-11-05 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-shared-objects-across-two-log-files

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.

- You may still sequentially scan each file and you may perform external sorting.
- Describe how you would modify your approach to work under this memory constraint.
- Analyze the time and space complexity of this memory-efficient solution.

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields

QUESTION

EXAMPLES

```
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']],
    [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
```

Hints

Track which days each object appears in and its set of clients.

APPROACH

Load the smaller file's obj_ids into a hash set, stream the other and collect matches; if both are huge use external sort/merge or hash partitioning to disk (or a Bloom filter prefilter).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether to report distinct ids or every occurrence, how large each file is, whether order matters, and what to output (ids only or full lines).

BRUTE FORCE TO SAY OUT LOUD

For each id in file A scan file B. O(n * m), unusable at scale.

OPTIMAL IDEA

Load the ids of the smaller file into a hash set, stream the larger file and collect hits (O(n + m) time). If both do not fit in memory, hash-partition both files by id into buckets on disk so each bucket pair fits, then join bucket by bucket; or external sort both and merge. A Bloom filter of one file prefilters the other.

DRY-RUN THESE CASES

Empty file, duplicate ids, identical files, no overlap, ids that differ only in case or whitespace.

LIKELY FOLLOW-UPS

Distributed version (shuffle by key), top N shared ids by frequency, restrict to a time window.

PITFALLS

Loading both files into memory. Not deduplicating output.

HOW TO THINK ABOUT IT

A common key in two files is a join. If one file fits in memory, load its keys into a set and stream the other. If neither fits, split both files by hash of the key into buckets small enough for memory; equal keys always land in the same bucket number, so you only compare bucket i of one file with bucket i of the other (Grace hash join).

Basis: Written; a join of two large relations (external hash join). Line format assumed: 'timestamp obj_id other'.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(file_a, file_b):
    shared = set()
    for a in file_a:
        for b in file_b:
            if a.obj_id == b.obj_id:
                shared.add(a.obj_id)
    return shared
```

TIME	SPACE
O(n * m)	O(1) extra

WHY IT IS SLOW

O(n * m) comparisons. Loading the smaller file's ids into a hash set and streaming the other file is O(n + m); if neither fits, partition both by a hash of the id.

OPTIMAL SOLUTION · Hash set of the smaller file's ids

TIME	SPACE
O(n + m)	O(distinct ids of the smaller file)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Both files are too big for memory.

Hash-partition both files into B buckets on disk by hash of the object id, then process bucket pairs one at a time with the in-memory set join. Choose B so that one bucket of the smaller file fits in memory. External sort and merge also works.

FOLLOW-UP 2 Want objects seen on both days within an hour of each other.

Keep a list of timestamps per id in the set join and compare timestamps (sorted lists with two pointers), instead of just membership.

FOLLOW-UP 3 A tiny memory budget and an approximate answer is fine.

Build a Bloom filter from one file and test the other against it, accepting a small false-positive rate; verify the few hits exactly in a second pass if needed. **Cost:** O(n + m) time with O(bits) memory for the Bloom filter and a small false-positive rate.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find shared objects across two log files
#
# You are given two large log files representing activity
# on two different days. Each line of each log has three
# fields: - timestamp: a time value (you may treat it as
# an opaque string or integer) - obj_id: identifier of an
# object (string or integer) - client_id: identifier of a
# client (string or integer) The logs are not necessarily
# sorted by any field. Define an object as **interesting**
# if: 1. Its obj_id appears at least once in day 1's log
# **and** at least once in day 2's log; and 2. Across all
# of its appearances in both days combined, it is
# associated with at least **two distinct** client_id
# values. ### Tasks 1. Design and implement a function
# that, given file handles (or streaming iterators) for
# the two log files, returns the set of all such
# interesting obj_ids. 2.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Both files are too big for memory.
# 2. Want objects seen on both days within an hour of each
# other.
# 3. A tiny memory budget and an approximate answer is
# fine.
#
# INTERVIEW NOTES
# A common key in two files is a join. If one file fits in
# memory, load its keys into a set and stream the other.
# If neither fits, split both files by hash of the key
# into buckets small enough for memory; equal keys always
# land in the same bucket number, so you only compare
# bucket i of one file with bucket i of the other (Grace
# hash join).
#
# BRUTE FORCE (say it first): Compare every pair of lines
# For each line in A scan all of B for the same object id.
# Correct, but quadratic in the file sizes. Time O(n * m).
# Space O(1) extra.


import zlib
from collections import defaultdict


# BRUTE FORCE (say it first): Compare every pair of lines
```

```
# | time O(n * m)
def shared_brute(a_lines, b_lines):
    out = set()
    for x in a_lines:
        for y in b_lines:
            if x.split()[1] == y.split()[1]:
                out.add(x.split()[1])
    return out


# HELPERS for follow-up 1
def bucket_of(obj, buckets):
    # a stable hash (Python's own hash changes between
    # runs)
    return zlib.crc32(str(obj).encode()) % buckets


# BASE QUESTION (optimal): Hash set of the smaller file's
# ids
# Idea: Load the object ids of the smaller file into a
```

```python
# set, then stream the other file and keep ids that are in
# the set. Each line is read once; memory depends only on
# the smaller file.
def shared_objects(day1, day2):
    # rows are [timestamp, obj_id, client_id]. Remember,
    # per
    # object, which days it appeared on and its clients.
    days, clients = defaultdict(set), defaultdict(set)
    for number, rows in ((1, day1), (2, day2)):
        for _, obj, client in rows:
            days[obj].add(number)
            clients[obj].add(client)
    return sorted(o for o in days
                  if len(days[o]) == 2
                  and len(clients[o]) >= 2)


# OTHER WAY: external sort of both days
def shared_objects_sorted(day1, day2):
    # the other way: external sort both days by object,
    # then a
    # merge that holds only one object's clients at a time
    a = sorted(day1, key=lambda r: r[1])
    b = sorted(day2, key=lambda r: r[1])
    i = j = 0
```

```python
    out = []
    while i < len(a) and j < len(b):
        if a[i][1] < b[j][1]:
            i += 1
        elif a[i][1] > b[j][1]:
            j += 1
        else:
            obj, clients = a[i][1], set()
            while i < len(a) and a[i][1] == obj:
                clients.add(a[i][2])
                i += 1
            while j < len(b) and b[j][1] == obj:
                clients.add(b[j][2])
                j += 1
            if len(clients) >= 2:
                out.append(obj)
    return out


# FOLLOW-UP 1: Both files are too big for memory.
# Answer: Hash-partition both files into B buckets on disk
# by hash of the object id, then process bucket pairs one
# at a time with the in-memory set join. Choose B so that
# one bucket of the smaller file fits in memory.
def shared_objects_partitioned(day1, day2, buckets=4):
    # memory limit: write each row to a bucket "file"
    # chosen by
    # the hash of its object, then solve one bucket at a
    # time.
    # All rows of an object land in the same bucket.
    parts1 = [[] for _ in range(buckets)]
    parts2 = [[] for _ in range(buckets)]
    for rows, parts in ((day1, parts1), (day2, parts2)):
        for row in rows:
            parts[bucket_of(row[1], buckets)].append(row)
    out = []
    for b in range(buckets):
        out += shared_objects(parts1[b], parts2[b])
    return sorted(out)


# FOLLOW-UP 3: A tiny memory budget and an approximate
# answer is fine.
# Answer: Build a Bloom filter from one file and test the
# other against it, accepting a small false-positive rate;
```

```python
# verify the few hits exactly in a second pass if needed.
class Bloom:
    """Tiny memory, approximate: may say 'yes' for a few
    objects that were never added."""

    def __init__(self, bits=1 << 16, hashes=3):
        self.bits, self.hashes = bytearray(bits), hashes

    def spots(self, key):
        for seed in range(self.hashes):
            text = f"{seed}:{key}".encode()
            yield zlib.crc32(text) % len(self.bits)

    def add(self, key):
        for s in self.spots(key):
            self.bits[s] = 1

    def maybe(self, key):
        return all(self.bits[s] for s in self.spots(key))


if __name__ == "__main__":
    d1 = [["t1", "o1", "c1"], ["t2", "o2", "c1"]]
    d2 = [["t3", "o1", "c2"], ["t4", "o2", "c1"]]
    assert shared_objects(d1, d2) == ["o1"]
    assert shared_objects([], [["t", "o", "c"]]) == []
    assert shared_objects_partitioned(d1, d2, 3) == ["o1"]
    assert shared_objects_sorted(d1, d2) == ["o1"]
    many1 = [[i, f"o{i % 7}", f"c{i % 3}"]
             for i in range(40)]
    many2 = [[i, f"o{i % 5}", f"c{i % 2}"]
             for i in range(40)]
    want = shared_objects(many1, many2)
    parts = shared_objects_partitioned(many1, many2, 5)
    assert parts == want
    assert shared_objects_sorted(many1, many2) == want
    bloom = Bloom()
    for row in d1:
        bloom.add(row[1])
    assert bloom.maybe("o1") and not bloom.maybe("o999")
```

ALSO REPORTED · 1 more

Two-Day Log Intersection under Memory Limit | 1Point3Acres | onsite-coding | asked 2025-11-27 | Single report

https://www.1point3acres.com/interview/problems/company/google/two-logs-intersection-memory-limit

In-memory solution: build a hashmap obj_id -> set(client_id) while scanning each file; intersect keys across the two hashmaps at the end.

Memory-limited solution: external sort each file by obj_id, then run a merge / two-pointer scan over both sorted files. For each matching obj_id group, count distinct client_id on the fly (HyperLogLog if duplicates are large).

The interviewer explicitly preferred sort + two-pointer over a streaming hashmap — be prepared to defend the choice (sort is O(n log n) on disk vs O(n) RAM-bound for the hashmap).

For distributed scale, hash-partition by obj_id and run the same algorithm per partition.

ALSO REPORTED · 1 more

Preparation: Practice external-sort / merge-step problems (classic in-memory merge / k-way-merge / partition problems — and discuss disk-based variants verbally).
Drill the streaming hashmap vs external sort trade-off conversation.
Have ready: HyperLogLog (or Count-Min Sketch) one-liner for approximate distinct-count under tight memory.

STUDY HERE: topic 7, Binary Search and Sorting | main topic: Sorting

HARD | PracHub | Technical Screen | 13 of 13

★ TOP 75: rank 10 in the Medium and Hard priority list

★ Find safe travel intervals between planet influences

POSTED 2025-11-22 | 332 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 65.2 | 30 DAYS 41.1 | PRIORITY 9/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting: Sorting

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #56 Merge Intervals

LeetCode match: LC 56 Merge Intervals (then take gaps)

Posted on PracHub: 2025-11-22 (updated 2026-05-23)

Original: https://prachub.com/coding-questions/find-safe-travel-intervals-between-planet-influences

QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **c** is the coordinate of the planet's center on the x-axis.
- **r** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [|x - c_i| \le r_i \quad \text{for some planet } i] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

QUESTION

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

Constraints
0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9

EXAMPLES

```
Input: []
Expected Output: [(None, None)]
```

Explanation: With no planets, no point is unsafe, so the entire real line is safe.

```
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8,
    12]. Their complement is (-infinity, 2), (4, 8),
    and (12, +infinity).
```

Hints

First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

APPROACH

Turn each planet into [c-r, c+r], sort, merge overlaps, and the safe intervals are the open gaps between merged intervals plus the two infinite ends. Watch endpoint inclusivity (unsafe intervals are closed).

HOW TO THINK ABOUT IT

The safe points are the complement of the union of the unsafe intervals. Merge the unsafe intervals first (sort by start, extend while they overlap or touch), and the safe intervals are exactly the gaps between consecutive merged intervals plus the two unbounded ends. Endpoints themselves are unsafe because the influence is closed (<=).

Basis: Interval complement (LeetCode 56, Merge Intervals, then take the gaps). Each planet (c, r) makes the closed interval [c - r, c + r] unsafe; safe intervals are the open gaps, possibly unbounded.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(planets):
    pts = set()
    for c, r in planets:
        pts |= {c - r, c + r}
    bounds = [-INF] + sorted(pts) + [INF]
    safe = []
    for a, b in zip(bounds, bounds[1:]):
        mid = midpoint(a, b)
        if not any(abs(mid - c) < r
                   for c, r in planets):
            safe.append((a, b))
    return safe
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each midpoint is tested against every planet, O(n^2). Merging the unsafe intervals after sorting and reading the gaps is O(n log n).

OPTIMAL SOLUTION · Merge the unsafe intervals, then read off the gaps

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the part between two given positions matters (a route from A to B).

Intersect each safe interval with [A, B] and keep the non-empty ones. Or clip the planets first; the merge-and-gap logic is the same.

FOLLOW-UP 2 Are the endpoints safe?

The statement uses <=, so the endpoints of an unsafe interval are unsafe and the safe intervals are open. If the influence were strict (<), touching intervals would leave a single safe point between them and the merge test would use < instead of <=.

FOLLOW-UP 3 Planets arrive over time.

Keep the merged unsafe intervals in a sorted structure; an insert merges neighbours (like the stream-of-intervals problem) and the safe intervals are always the gaps.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find safe travel intervals between planet
# influences
#
# You are planning a space route along a one-dimensional
# line (the x-axis). You are given a list of planets. Each
# planet is represented by an integer pair \((c, r)\): - c
# is the coordinate of the planet's center on the x-axis.
# - r is a non-negative integer radius of its
# gravitational influence. A point x on the x-axis is
# **unsafe** for a space traveler if it lies within the
# gravitational influence of at least one planet: \[ |x -
# c_i| \le r_i \quad \text{for some planet } i \]
# Equivalently, each planet makes the interval \([c_i -
# r_i,\; c_i + r_i]\) unsafe. Task: - Consider the entire
# real line as the possible travel path. - Compute all
# **maximal continuous intervals** on this line where the
# traveler is **safe**, i.e., points not covered by any
# planet's unsafe interval. - Return these safe intervals
# as a sorted list of disjoint intervals.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the part between two given positions matters (a
# route from A to B).
# 2. Are the endpoints safe?
# 3. Planets arrive over time.
#
# INTERVIEW NOTES
# The safe points are the complement of the union of the
# unsafe intervals. Merge the unsafe intervals first (sort
# by start, extend while they overlap or touch), and the
# safe intervals are exactly the gaps between consecutive
# merged intervals plus the two unbounded ends. Endpoints
# themselves are unsafe because the influence is closed
# (<=).
#
# BRUTE FORCE (say it first): Test the midpoint of every
# segment between critical points
# Collect all interval endpoints, and for every
# consecutive pair (and the two outer rays) test whether
# the midpoint is covered by some planet. Adjacent safe
# segments are separate because their shared endpoint is
# unsafe. Correct and the oracle; every midpoint is tested
# against every planet. Time O(n^2). Space O(n).
```

```
# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Test the midpoint of every
# segment between critical points | time O(n^2)
def safe_intervals_brute(planets):
    iv = [(c - r, c + r) for c, r in planets]
    pts = sorted({x for a, b in iv for x in (a, b)})
    if not pts:
        return [(-INF, INF)]

    def safe(x):
        return not any(a <= x <= b for a, b in iv)

    out = []
    if safe(pts[0] - 1):
        out.append((-INF, pts[0]))
    for p, q in zip(pts, pts[1:]):
        if safe((p + q) / 2):
            out.append((p, q))
```

```
    if safe(pts[-1] + 1):
        out.append((pts[-1], INF))
    return out


# HELPERS for the base question
def unsafe_blocks(planets):
    # planet (c, r) makes [c - r, c + r] unsafe; merge
    # overlaps
    merged = []
    spans = sorted((c - r, c + r) for c, r in planets)
    for a, b in spans:
        # sharing even one point joins two blocks
        if merged and a <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], b)
        else:
            merged.append([a, b])
    return merged


# HELPERS for follow-up 3
def safe_intervals_from(blocks):
    if not blocks:
        return [(-INF, INF)]
```

```
    out = [(-INF, blocks[0][0])]
    for prev, nxt in zip(blocks, blocks[1:]):
        out.append((prev[1], nxt[0]))
    out.append((blocks[-1][1], INF))
    return out


# BASE QUESTION (optimal): Merge the unsafe intervals,
# then read off the gaps
# Idea: Sort the unsafe intervals by start and merge any
# that overlap or touch (they share a point, which is
# unsafe).
def safe_intervals(planets):
    # the gaps between unsafe blocks (open intervals)
    merged = unsafe_blocks(planets)
    if not merged:
        return [(-INF, INF)]
    out = [(-INF, merged[0][0])]
    for prev, nxt in zip(merged, merged[1:]):
        out.append((prev[1], nxt[0]))
    out.append((merged[-1][1], INF))
    return out


# FOLLOW-UP 1: Only the part between two given positions
# matters (a route from A to B).
# Answer: Intersect each safe interval with [A, B] and
# keep the non-empty ones. Or clip the planets first; the
# merge-and-gap logic is the same.
def safe_between(planets, a, b):
    # a route from a to b: clip every safe interval to [a,
    # b]
    out = []
    for lo, hi in safe_intervals(planets):
        lo, hi = max(lo, a), min(hi, b)
        if lo < hi:
            out.append((lo, hi))
    return out


# FOLLOW-UP 3: Planets arrive over time.
# Answer: Keep the merged unsafe intervals in a sorted
# structure; an insert merges neighbours (like the stream-
# of-intervals problem) and the safe intervals are always
# the gaps.
```

```
class LiveRoute:
    """Planets arrive over time: keep blocks merged."""
    def __init__(self):
        self.blocks = []

    def add(self, c, r):
        a, b = c - r, c + r
        keep = []
        for s, e in self.blocks:
            if e < a or s > b:
                keep.append([s, e])
            else:
                # overlaps the new block: absorb it
                a, b = min(a, s), max(b, e)
        keep.append([a, b])
        self.blocks = sorted(keep)

    def safe(self):
        return safe_intervals_from(self.blocks)


if __name__ == "__main__":
    planets = [(0, 2), (10, 1), (3, 1)]
    # unsafe: [-2, 4] and [9, 11]
    assert safe_intervals(planets) == [(-INF, -2), (4, 9),
                                       (11, INF)]
    assert safe_intervals([]) == [(-INF, INF)]
    # touching blocks merge: [0, 2] and [2, 4]
    assert safe_intervals([(1, 1), (3, 1)]) == [
        (-INF, 0), (4, INF)]
    # only the part of the route between 5 and 20 matters
    route = safe_between(planets, 5, 20)
    assert route == [(5, 9), (11, 20)]
    live = LiveRoute()
    for c, r in planets:
        live.add(c, r)
    assert live.safe() == safe_intervals(planets)
    # a big planet at 6 swallows both blocks
    live.add(6, 3)
    assert live.safe() == [(-INF, -2), (11, INF)]
```

FUNCTION AND TEST CASES

Signature

```
def solution(planets: list[list[int]]) -> list[list[int]]:
SAMPLE INPUT
[]
```

Test cases

Input	Expected
`[]`	`[(None, None)]`
`[(3, 1), (10, 2)]`	`[(None, 2), (4, 8), (12, None)]`

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 8 OF 10 | COMMON TOPIC

Strings, Parsing and Recursion

12 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	7	4
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Strings, Parsing and Recursion | 12 questions | tap a title to jump to it

STRINGS (8)

M	1. Snake Case to Camel Case with Repository-Wide Refactor	1Point3Acres	2026-07-14
M	2. Check Whether Digits Are Monotone Increasing	PracHub	2026-07-04
M	3. Can board states be transformed?	PracHub	2026-04-01
M	4. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)	1Point3Acres	2026-02-01
M	5. Decide string transform with directional tokens and walls	PracHub	2025-07-15
H	6. Find the Longest Straight Run of One Color	PracHub	2026-08-23
H	7. Convert Valid Snake Case to Lower Camel Case	PracHub	2026-04-16
H	8. Find Dictionary Matches for a Jammed String	PracHub	2026-04-09

RECURSION AND BACKTRACKING (3)

M	9. Mahjong Winning Hand Detection (14 Tiles)	1Point3Acres	2026-10-09
M	10. Count Prefix-Matching Simple Paths in a Labeled Graph	PracHub	2026-08-09
H	11. Matrix Flower Placement with House Adjacency	1Point3Acres	2026-05-31

PARSING (1)

E	12. Implement string formatting and request tracking tasks	PracHub	2026-02-11

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Strings

Site tags: string, refactoring, medium

MEDIUM | 1Point3Acres | tech-screen | 1 of 12

Snake Case to Camel Case with Repository-Wide Refactor

ASKED 2026-07-14 | 98 DAYS AGO | LAST 6 MONTHS

STUDY: READ | **NO LEETCODE FREQUENCY** | **PRIORITY 213/228**

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

APPROACH

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

HOW TO THINK ABOUT IT

Converting foo_bar to fooBar is easy. The risk is in the migration: a plain text replace also changes strings, comments, third-party names and public interfaces. Work on tokens (or the syntax tree) instead, rename only symbols defined in your own code, keep an explicit allow-list of public names, and prove nothing broke by running the tests before and after.

Basis: Written for this question; the conversion is a one-liner and the real work is a safe, token-based migration (not plain text replace).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for path in source_files(repo):
    text = read(path)
    for word in find_snake_words(text):
        if not is_third_party(word):
            new = to_camel(word)
            text = text.replace(word, new)
    write(path, text)
```

TIME	SPACE
O(size of the repository)	O(1)

WHY IT IS SLOW

Also rewrites strings, comments, third-party calls and public interface names. Renaming only the identifiers the code defines, found with the syntax tree and tokens, keeps those intact.

OPTIMAL SOLUTION · Convert names, rename only defined identifiers via tokens

TIME	SPACE
O(size of the source)	O(names)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you prove the migration is safe?

Run the full test suite before and after (green both times), compare the public API surface (exported names) before and after, and migrate in small reviewable commits with a codemod that only touches owned symbols. Anything reachable by reflection, serialisation, config files or other repositories needs an explicit alias or a deprecation period rather than a silent rename.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Snake Case to Camel Case with Repository-
# Wide Refactor
#
# Convert a valid snake_case identifier to camelCase, then
# design a safe repository-wide migration for variables
# and function names without changing third-party symbols,
# public interfaces, or the tests' ability to detect
# regressions. Implement a conversion from a valid
# snake_case identifier to camelCase. The base input is
# guaranteed valid: no leading underscore, trailing
# underscore, or doubled separator. Example: foo_bar ->
# fooBar. Extend the discussion to a repository too large
# for manual review: replace snake-case variables and
# functions across the codebase. Explain how to verify
# that the repository still works after the automated
# rewrite, including when test-package identifiers are
# rewritten too. Preserve third-party functions and public
# interfaces; explain how the migration distinguishes
# symbols that may change from contracts that must remain
# stable. Examples: foo_bar -> fooBar
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you prove the migration is safe?
#
# INTERVIEW NOTES
# Converting foo_bar to fooBar is easy. The risk is in the
# migration: a plain text replace also changes strings,
# comments, third-party names and public interfaces. Work
# on tokens (or the syntax tree) instead, rename only
# symbols defined in your own code, keep an explicit
# allow-list of public names, and prove nothing broke by
# running the tests before and after.
#
# BRUTE FORCE (say it first): Plain text replace across
# the files
# Regex-replace every snake_case word in every file. Fast
# and wrong: it rewrites strings, comments, third-party
# calls and public interface names. Time O(size of the
# repository). Space O(1).


import re
import ast
import io
import tokenize
```

```
# BRUTE FORCE (say it first): Plain text replace across
# the files | time O(size of the repository)
def snake_rewrite_brute(source):
    return re.sub(
        r"\b[a-z]+(?:_[a-z0-9]+)+\b",
        lambda m: snake_to_camel(m.group()),
        source,
    )


# BASE QUESTION (optimal): Convert names, rename only
# defined identifiers via tokens
# Idea: snake_to_camel capitalises each part after the
# first. owned_names collects the functions, arguments and
# variables the code defines (using the syntax tree),
# minus an allow-list of names that must stay.
def snake_to_camel(name):
    # foo_bar -> fooBar (input is a valid snake_case name)
    head, *rest = name.split("_")
    caps = [p[:1].upper() + p[1:] for p in rest]
    return head + "".join(caps)


# FOLLOW-UP 1: How do you prove the migration is safe?
# Answer: Run the full test suite before and after (green
# both times), compare the public API surface (exported
```

```
# names) before and after, and migrate in small reviewable
# commits with a codemod that only touches owned symbols.
def owned_names(source, keep=()):
    # names this code defines itself: safe to rename
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.FunctionDef):
            names.add(node.name)
            args = node.args.args + node.args.kwonlyargs
            names.update(a.arg for a in args)
        elif isinstance(node, ast.Name):
            if isinstance(node.ctx, ast.Store):
                names.add(node.id)
    # only real snake_case, never dunder or private names
    return {n for n in names if "_" in n.strip("_")
            and not n.startswith("_") and n not in keep}
```

```
# FOLLOW-UP 1: How do you prove the migration is safe?
# Answer: Run the full test suite before and after (green
# both times), compare the public API surface (exported
# names) before and after, and migrate in small reviewable
# commits with a codemod that only touches owned symbols.
def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
    out, prev = [], None
    # work on tokens: strings and comments are untouched
    for tok in tokenize.generate_tokens(
            io.StringIO(source).readline):
        # skip attribute names (after a dot): they may be
        # third-party, so they stay as written
        dot = prev is not None and prev.string == "."
        is_name = tok.type == tokenize.NAME
        if is_name and tok.string in owned and not dot:
            new = snake_to_camel(tok.string)
            tok = tok._replace(string=new)
        out.append(tok)
        prev = tok
    return tokenize.untokenize(out)


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    src = (
        "import os\n"
        "def load_data(file_path):\n"
        "    full_path = os.path.join(file_path, 'x_y')\n"
        "    return full_path  # load_data is fine\n"
        "result_value = load_data('a')\n")
    new = rename_identifiers(src, keep={"file_path"})
    assert "def loadData(file_path)" in new
    assert "fullPath = os.path.join" in new
    # strings, comments and third-party calls are intact
    assert "'x_y'" in new and "# load_data is fine" in new
    assert "resultValue = loadData('a')" in new
    # the migrated code still runs
    scope = {}
    exec(compile(new, "x", "exec"), scope)
    assert scope["resultValue"] == "a/x_y"
```

NOTES FROM CANDIDATES

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

PREPARATION

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Strings

MEDIUM | PracHub | Technical Screen | 2 of 12

Check Whether Digits Are Monotone Increasing

POSTED 2026-07-04 | 108 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 193/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy digit scan) (inferred from overview)

Posted on PracHub: 2026-07-04 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/check-whether-digits-are-monotone-increasing

QUESTION

Overview: Check whether a non-negative integer's decimal digits are non-decreasing from left to right. Cover zero, single-digit values, repeated digits, and an O(d) implementation using either arithmetic digit extraction or string traversal.

APPROACH

Compare each digit with the previous one using % 10 and // 10 from the right (or string scan); handle 0 and single digits. O(d).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether equal neighbors are allowed (non-decreasing), whether the input can be negative, and whether to use the integer or string form.

BRUTE FORCE TO SAY OUT LOUD

Convert to a string and compare each pair. Fine and O(d).

OPTIMAL IDEA

Without strings: peel digits from the right with % 10 and // 10, keeping the previous digit; fail if the current digit is greater than the previous one seen to its right. O(d), O(1) space.

DRY-RUN THESE CASES

0, a single digit, 1234, 1224, 1243, 100.

LIKELY FOLLOW-UPS

Strictly increasing, or find the next monotone number.

PITFALLS

Comparing in the wrong direction when scanning right to left.

HOW TO THINK ABOUT IT

Compare neighbouring digits. You can read the digits from the right with % 10 and // 10, remembering the previous (right-hand) digit: the number is monotone if no digit is larger than the one to its right. Say first whether equal neighbours are allowed.

Basis: The check behind LeetCode 738 (non-decreasing digits).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_monotone(x):
    s = str(x)
    return s == "".join(sorted(s))
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorting costs O(d log d) and copies the digits. Comparing each digit with its neighbour while peeling digits with % 10 is O(d) with O(1) space.

OPTIMAL SOLUTION · Peel digits from the right and compare with the previous one

TIME	SPACE
O(d)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Strictly increasing digits.

Use >= instead of > when comparing with the previous digit, and start prev at 10 so a single digit passes.

FOLLOW-UP 2 Negative numbers or numbers in another base.

Decide what monotone means for the sign (usually apply it to the absolute value). For base b replace 10 with b in % and //.

FOLLOW-UP 3 Find the next monotone number after n.

That is the left-to-right fix from the previous question: find the first descent and copy that digit across the rest.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check Whether Digits Are Monotone
# Increasing
#
```

```
# Check whether a non-negative integer's decimal digits
# are non-decreasing from left to right. Cover zero,
# single-digit values, repeated digits, and an O(d)
# implementation using either arithmetic digit extraction
# or string traversal.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Strictly increasing digits.
# 2. Negative numbers or numbers in another base.
# 3. Find the next monotone number after n.
#
# INTERVIEW NOTES
# Compare neighbouring digits. You can read the digits
# from the right with % 10 and // 10, remembering the
# previous (right-hand) digit: the number is monotone if
# no digit is larger than the one to its right. Say first
# whether equal neighbours are allowed.
#
# BRUTE FORCE (say it first): String comparison with its
# sorted version
# Convert to a string and compare it with the sorted
# string: a number has non-decreasing digits exactly when
# its digits are already in sorted order. Clear, with an
# unneeded sort. Time O(d log d). Space O(d).


# BRUTE FORCE (say it first): String comparison with its
# sorted version | time O(d log d)
def digits_monotone_sorted(n):
    s = str(n)
    return s == "".join(sorted(s))


# BASE QUESTION (optimal): Peel digits from the right and
# compare with the previous one
# Idea: Take the last digit with n % 10, then the next
# one, and so on. Because we move right to left, the digit
# just read must not exceed the one read before it. A
# single digit and zero are monotone.
def is_monotone_digits(n):
    # non-decreasing digits, read from the right with n %
    # 10
    prev = 9
    while n:
        d = n % 10
        # a digit bigger than the one to its right: a
        # descent
        if d > prev:
            return False
        prev = d
        n //= 10
    return True


# SAME TEST on the text of the number
def is_monotone_string(n):
    # the same test on the text of the number
    s = str(n)
    return all(a <= b for a, b in zip(s, s[1:]))


# FOLLOW-UP 1: Strictly increasing digits.
# Answer: Use >= instead of > when comparing with the
# previous digit, and start prev at 10 so a single digit
# passes.
def is_strictly_monotone(n):
    # strictly increasing: start prev at 10, compare with
    # >=
    prev = 10
    while n:
        d = n % 10
        if d >= prev:
            return False
        prev = d
        n //= 10
    return True


# FOLLOW-UP 2: Negative numbers or numbers in another
# base.
# Answer: Decide what monotone means for the sign (usually
# apply it to the absolute value). For base b replace 10
# with b in % and //.
def is_monotone_base(n, base=10):
    # any base: use the base for % and //
    prev = base - 1
    while n:
        d = n % base
        if d > prev:
            return False
        prev = d
        n //= base
    return True


# FOLLOW-UP 3: Find the next monotone number after n.
# Answer: That is the left-to-right fix from the previous
# question: find the first descent and copy that digit
# across the rest.
def next_monotone(n):
    # smallest monotone number AFTER n: fix the first
    # descent
    d = list(str(n + 1))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            d[i + 1:] = d[i] * (len(d) - i - 1)
            break
    return int("".join(d))


if __name__ == "__main__":
    assert is_monotone_digits(0) and is_monotone_digits(7)
    assert is_monotone_digits(1223)
    assert is_monotone_digits(999)
    assert not is_monotone_digits(332)
    assert not is_monotone_digits(10)
    assert is_strictly_monotone(1234)
    assert not is_strictly_monotone(1224)
    assert is_strictly_monotone(5)
    # binary 11 -> 3 is monotone; binary 10 -> 2 is not
    assert is_monotone_base(3, 2)
    assert not is_monotone_base(2, 2)
    assert next_monotone(332) == 333
    assert next_monotone(9) == 11
    for n in range(2000):
        by_text = is_monotone_string(n)
        assert is_monotone_digits(n) == by_text
```

FUNCTION AND TEST CASES

Signature

```
def has_monotone_increasing_digits(n: int) -> bool:
SAMPLE INPUT
0
```

Test cases

Input	Expected
0	True
7	True

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Strings

MEDIUM | PracHub | Technical Screen | 3 of 12

Can board states be transformed?

POSTED 2026-04-01 | 202 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 93/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (inferred from overview)

Posted on PracHub: 2026-04-01 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/can-board-states-be-transformed

QUESTION

Overview: This question evaluates reasoning about constrained piece movement, string transformation, and invariant-based correctness in the Coding & Algorithms domain, testing skills in string manipulation and combinatorial reasoning while examining both conceptual properties and practical algorithmic implementation.

APPROACH

Invariants: the sequence of non-empty pieces must match; each L can only move left and each R only right (compare positions with two pointers).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the pieces are, which direction each may move, whether pieces may jump over each other, and what the empty cell is called.

BRUTE FORCE TO SAY OUT LOUD

BFS over all reachable board strings. Exponential but clear.

OPTIMAL IDEA

Invariants: pieces never cross, so the order of the non-empty pieces must be identical in both strings; an L can only move left, so in the start its index must be at least its index in the target; an R can only move right, so its start index must be at most its target index. Walk two pointers over the non-empty cells and check those conditions. O(n).

DRY-RUN THESE CASES

Equal strings. A different piece order. An L that would have to move right. Same pieces but different counts.

LIKELY FOLLOW-UPS

Add walls that cannot be passed or crossed, or ask for the minimum number of moves.

PITFALLS

INTERVIEW-DAY PLAN

Comparing counts only and not order. Skipping the empty cells incorrectly in the two-pointer loop.

HOW TO THINK ABOUT IT

Think invariants, not search. Pieces cannot jump each other, so the sequence of L and R letters (ignoring blanks) cannot change. Within that, an L can only go left and an R only right, so compare the position of each piece in the start and in the target. That reduces exponential BFS to one pass with two pointers.

Basis: LeetCode 777, Swap Adjacent in LR String: 'X' is an empty cell, 'L' only moves left, 'R' only moves right, pieces never pass each other.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in legal_moves(board):
            if nxt not in seen:  # XL->LX, RX->XR
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential in the length	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially. Pieces never cross, so compare the sequence of L and R pieces and check that no L moved right and no R moved left, in O(n).

OPTIMAL SOLUTION · Two pointers over the pieces

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the minimum number of moves.

Because pieces never interact beyond keeping their order, each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences when the transformation is possible.

FOLLOW-UP 2 Add walls that cannot be crossed (next question).

A wall is just another piece that never moves: it stays in the sequence of non-blank symbols and must keep the same index in both strings. The same pointer walk handles it.

FOLLOW-UP 3 Pieces can move one step per round and several move at once.

The count of rounds is the maximum over pieces of their distance, if pieces never block each other; with blocking you simulate rounds or use the per-piece arrival times with the ordering constraint.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Can board states be transformed?
#
# This question evaluates reasoning about constrained
# piece movement, string transformation, and invariant-
# based correctness in the Coding & Algorithms domain,
# testing skills in string manipulation and combinatorial
# reasoning while examining both conceptual properties and
# practical algorithmic implementation.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the minimum number of moves.
# 2. Add walls that cannot be crossed (next question).
# 3. Pieces can move one step per round and several move
# at once.
#
# INTERVIEW NOTES
# Think invariants, not search. Pieces cannot jump each
# other, so the sequence of L and R letters (ignoring
# blanks) cannot change. Within that, an L can only go
# left and an R only right, so compare the position of
# each piece in the start and in the target. That reduces
# exponential BFS to one pass with two pointers.
#
# BRUTE FORCE (say it first): BFS over every reachable
# board
# From the start string, repeatedly apply the legal moves
# (XL becomes LX, RX becomes XR) and see whether the
# target appears. Correct, and the oracle for the
# invariant-based check, but the number of boards
# explodes. Time exponential in the length. Space
# exponential.


from collections import deque


# BRUTE FORCE (say it first): BFS over every reachable
# board | time exponential in the length
def can_transform_bfs(start, end):
    seen, queue = {start}, deque([start])
    while queue:
        s = queue.popleft()
        if s == end:
            return True
        for i in range(len(s) - 1):
            nxt = None
            if s[i : i + 2] == "XL":
                nxt = s[:i] + "LX" + s[i + 2 :]
            elif s[i : i + 2] == "RX":
                nxt = s[:i] + "XR" + s[i + 2 :]
            if nxt and nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    return False


# HELPERS for the base question
def pieces(board):
    # the non-blank symbols with their positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "X"]


# BASE QUESTION (optimal): Two pointers over the pieces
# Idea: Skip blanks in both strings and compare the next
# piece in each: they must be the same letter.
def can_transform(start, end):
    # L moves left, R moves right, X is blank. Pieces
    # never pass
    # each other, so the order of the non-X symbols is
    # fixed;
    # a '#' is a wall: it never moves and no piece crosses
    # it.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        # an L may only end up further left, an R further
        # right
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
        # a wall must stay exactly where it is
        if c1 == "#" and i != j:
            return False
    return True


# FOLLOW-UP 1: Return the minimum number of moves.
# Answer: Because pieces never interact beyond keeping
# their order, each piece moves independently from its
# start index to its target index, so the minimum is the
# sum of the absolute index differences when the
# transformation is possible.
def min_moves(start, end):
    # every piece moves on its own, one cell per move
    if not can_transform(start, end):
        return None
    a, b = pieces(start), pieces(end)
    return sum(abs(i - j) for (_, i), (_, j) in zip(a, b))


# FOLLOW-UP 3: Pieces can move one step per round and
# several move at once.
# Answer: The count of rounds is the maximum over pieces
# of their distance, if pieces never block each other;
# with blocking you simulate rounds or use the per-piece
# arrival times with the ordering constraint.
def rounds_needed(start, end):
    # if all pieces move at once and never block each
    # other,
    # the number of rounds is the longest single trip
    if not can_transform(start, end):
```

```
        return None
    a, b = pieces(start), pieces(end)
    trips = [abs(i - j) for (_, i), (_, j) in zip(a, b)]
    return max(trips, default=0)


if __name__ == "__main__":
    assert can_transform("RXXLRXRXL", "XRLXXRRLX")
    assert not can_transform("X", "L")
    assert can_transform("XXXL", "XXLX")
    # R can not move left, L can not move right
    assert not can_transform("XR", "RX")
    assert not can_transform("LX", "XL")
    # pieces keep their order: RL can not become LR
    assert not can_transform("RL", "LR")
    assert min_moves("RXXLRXRXL", "XRLXXRRLX") == 4
    assert min_moves("RXX", "XXR") == 2
```

```
    assert min_moves("RX", "XR") == 1
    assert min_moves("LL", "RR") is None
    # a wall splits the board: R can not jump over it
    assert can_transform("R#X", "R#X")
    assert not can_transform("R#X", "X#R")
    assert rounds_needed("RXX", "XXR") == 2
    assert rounds_needed("LL", "RR") is None
```

FUNCTION AND TEST CASES

Signature

```
def solution(start: str, target: str) -> bool:
```

Sample call

```
solution("R_L|__L", "_RL|L__")
```

Test cases

Input	Expected
("R_L|__L", "_RL|L__")	True
("L_|R", "L|_R")	False

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Strings

Site tags: string, simulation, medium

MEDIUM | 1Point3Acres | onsite-coding | 4 of 12

Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)

ASKED 2026-02-01 | 261 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 202/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-01 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/document-line-wrap

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"             (5 chars; "world" + space
    would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"            (7 chars; "baz" -> "Foo
    bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

APPROACH

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the width counts characters in a monospace font, how to treat explicit newlines, multiple spaces and leading spaces, what to do with a word longer than the width, and whether to return the number of lines or the lines themselves.

BRUTE FORCE TO SAY OUT LOUD

Build the text line by line by trying to add each word and measuring. That is already the greedy algorithm.

OPTIMAL IDEA

Tokenize into words and newline tokens. Keep the current line length; for each word, if it fits (with a space unless the line is empty) add it, otherwise start a new line. A newline token always ends the current line. Count lines (empty lines count). Greedy is expected; mention Knuth's minimum-raggedness DP only if asked. O(n).

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

The example giving 4. width = 1. Consecutive newlines. A word exactly the width. A word longer than the width.

LIKELY FOLLOW-UPS

Return the lines, justify the text, or minimize raggedness.

PITFALLS

Dropping empty lines. Adding a trailing space at the end of a line.

HOW TO THINK ABOUT IT

Greedy is optimal for the question 'fewest lines': put as many words on a line as fit. The decision per word is only 'fits with a space', 'fits as the first word on the line', or 'start a new line', so track the current line length, not the line text. Treat newline as a hard break and decide up front what empty lines and long words do.

Basis: Written; greedy word wrap (LeetCode 68, Text Justification, without the justification step). Assumed rules (confirm them): monospace width in characters; words are separated by whitespace; '\n' always ends a line and an empty line counts; a word longer than the width gets its own line.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wrap(text, width):
    count = 0
    for paragraph in text.split("\n"):
        line = ""
        for word in paragraph.split():
            joined = len(line) + 1 + len(word)
            if line and joined > width:
                count += 1  # finish the line
                line = word
            else:
                line = line + " " + word
                line = line.strip()
        count += 1  # last line
    return count
```

TIME	SPACE
O(total characters * words per line)	O(width)

WHY IT IS SLOW

Re-measuring a joined string for every word costs extra copying. Keeping a running length makes the whole thing O(total characters).

OPTIMAL SOLUTION · Greedy with a running line length

TIME	SPACE
O(n)	O(1) extra beyond the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the lines, not only the count.

Keep the same loop but append the finished line to a list (the brute force above already does it). Build each line from a list of words and join once, so each word is copied once.

FOLLOW-UP 2 A word longer than the width must be broken.

Before placing a word, while it is longer than the width flush the current line and emit width-sized chunks of the word, then continue with the remainder as a normal word.

FOLLOW-UP 3 Minimise raggedness instead of lines.

That is Knuth's word-wrap DP: dp[i] is the best cost of laying out words i.. with cost (spaces left)^2 on each line except the last; O(n * words per line). Greedy is not optimal for this objective.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Google Doc Line Wrap (Word Wrap with
# Whitespace and Newlines)
#
# Examples: text = "Hello world\nFoo bar baz qux" width =
# 10 Line 1: "Hello" (5 chars; "world" + space would be 11
# -> break) Line 2: "world" Line 3: "Foo bar" (7 chars;
# "baz" -> "Foo bar baz" = 11 -> break) Line 4: "baz qux"
# Output: 4
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the lines, not only the count.
# 2. A word longer than the width must be broken.
# 3. Minimise raggedness instead of lines.
#
# INTERVIEW NOTES
# Greedy is optimal for the question 'fewest lines': put
# as many words on a line as fit. The decision per word is
# only 'fits with a space', 'fits as the first word on the
# line', or 'start a new line', so track the current line
# length, not the line text. Treat newline as a hard break
# and decide up front what empty lines and long words do.
#
# BRUTE FORCE (say it first): Build each line as a string
# and measure it
# Try appending each word to the current line string and
# check its length, starting a new line when it exceeds
# the width. Correct, but joining strings repeatedly costs
# more than the length counter in the optimal version.
# Time O(total characters * words per line). Space
# O(width).


# BRUTE FORCE (say it first): Build each line as a string
# and measure it | time O(total characters * words per
# line)
def wrap_lines_brute(text, width):
    lines = []
    for paragraph in text.split("\n"):
        words = paragraph.split()
        cur = ""
        for w in words:
            trial = (cur + " " + w) if cur else w
            if len(trial) <= width or not cur:
                cur = trial
            else:
                lines.append(cur)
                cur = w
        lines.append(cur)
    return lines


# HELPERS for the base question
# FOLLOW-UP 1: Return the lines, not only the count.
# Answer: Keep the same loop but append the finished line
# to a list (the brute force above already does it). Build
# each line from a list of words and join once, so each
# word is copied once.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def wrap_lines(text, width, break_long=False):
    lines = []
    # a newline always starts a new paragraph (and a new
    # line)
    for paragraph in text.split("\n"):
        cur = ""
        words = paragraph.split()
        if break_long:
            pieces = [split_long(w, width) for w in words]
            words = [p for chunk in pieces for p in chunk]
        for w in words:
            if not cur:
                cur = w
            elif len(cur) + 1 + len(w) <= width:
                # the word and one space still fit
                cur += " " + w
            else:
                lines.append(cur)
                cur = w
        # the last (or only, or empty) line of the
        # paragraph
        lines.append(cur)
    return lines


# FOLLOW-UP 2: A word longer than the width must be
# broken.
# Answer: Before placing a word, while it is longer than
# the width flush the current line and emit width-sized
# chunks of the word, then continue with the remainder as
# a normal word.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def split_long(word, width):
    # a word longer than the line is cut into pieces
    starts = range(0, len(word), width)
    return [word[i:i + width] for i in starts]


# BASE QUESTION (optimal): Greedy with a running line
# length
# Idea: Split on newlines first (each paragraph ends its
# line), then for each word compare cur_len + 1 +
# len(word) with the width. Only the length is tracked,
# never the joined line.
def wrap_count(text, width):
    return len(wrap_lines(text, width))


# FOLLOW-UP 3: Minimise raggedness instead of lines.
# Answer: That is Knuth's word-wrap DP: dp[i] is the best
# cost of laying out words i.. with cost (spaces left)^2
# on each line except the last; O(n * words per line).
# Greedy is not optimal for this objective.
```

```python
def min_raggedness(words, width):
    # Knuth: minimise the squared spare space of every
    # line
    # except the last; greedy is NOT optimal for this goal
    n = len(words)
    INF = float("inf")
    # cost[i]: best total cost for words[i:]
    cost, nxt = [INF] * (n + 1), [0] * (n + 1)
    cost[n] = 0
    for i in range(n - 1, -1, -1):
        length = -1
        for j in range(i, n):
            length += len(words[j]) + 1
            if length > width:
                break
            spare = width - length
            # the last line costs nothing
            extra = 0 if j == n - 1 else spare ** 2
            if extra + cost[j + 1] < cost[i]:
                cost[i] = extra + cost[j + 1]
                nxt[i] = j + 1
    lines, i = [], 0
```

```python
    while i < n:
        lines.append(" ".join(words[i:nxt[i]]))
        i = nxt[i]
    return cost[0], lines


if __name__ == "__main__":
    text = "Hello world\nFoo bar baz qux"
    assert wrap_lines(text, 10) == [
        "Hello", "world", "Foo bar", "baz qux"]
    assert wrap_count(text, 10) == 4
    # blank lines count as lines
    assert wrap_count("a\n\nb", 5) == 3
    # a long word is split when asked
    cut = wrap_lines("abcdefgh", 3, True)
    assert cut == ["abc", "def", "gh"]
    assert wrap_lines("abcdefgh", 3) == ["abcdefgh"]
    # the best layout can beat the greedy one
    cost, lines = min_raggedness(
        "aaa bb cc ddddd".split(), 6)
    assert lines == ["aaa", "bb cc", "ddddd"]
    assert cost == 9 + 1
```

NOTES FROM CANDIDATES

Single-pass tokenizer + accumulator: track curLen; on each token, decide "fits with leading space", "fits without space" (start of line), or "flush and start new line".
Watch out for \n right after another \n (empty line counts), \n at the end of input, leading whitespace, multiple consecutive spaces inside a line.
Don't try to handle font / kerning / hyphenation — the prompt assumes monospace width counted in characters.
Greedy packing is the expected answer (LC 68 style). Mention DP word-wrap (Knuth's algorithm) only if the interviewer pushes on 'minimum raggedness'; greedy is optimal-enough and what graders look for. Watch edge cases: a single word longer than W, an explicit newline forcing a break mid-pack, trailing whitespace handling.

PREPARATION

Implement a clean tokenizer that treats \n as a special token and yields plain words otherwise.
Test cases: width = 1, text = "a\n\n\nb", text = " leading", text with words exactly width chars long.
Be ready to extend to "return the rendered lines" (string list), not just the count, as a likely follow-up.

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Strings

MEDIUM | PracHub | Technical Screen | 5 of 12

Decide string transform with directional tokens and walls

POSTED 2025-07-15 | 462 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 40.3 | 30 DAYS 0 | PRIORITY 94/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: LC 777 Swap Adjacent in LR String (with X walls)

Posted on PracHub: 2025-07-15 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/decide-string-transform-with-directional-tokens-and-walls

QUESTION

You are given two equal-length strings start and target of length n, each consisting only of the characters 'L', 'R', '.', and 'X'. A dot '.' represents an empty slot; 'X' represents an immovable wall; 'L' and 'R' are tokens that may move across empty slots subject to these rules: (1) An 'L' token may move any number of positions left into '.', but may never move right, pass through another token, or pass through an 'X'. (2) An 'R' token may move any number of positions right into '.', but may never move left, pass through another token, or pass through an 'X'. (3) Only one token can occupy a position at any time. Determine whether target can be obtained from start by a sequence of valid moves. If yes, return true; otherwise, return false. Design an algorithm that runs in O(n) time and O(1) extra space beyond a few pointers/counters. Explain your invariants, prove correctness informally, analyze time and space complexity, and walk through a few tricky edge cases (e.g., mismatched wall layouts, blocks separated by walls, consecutive tokens with no gaps).

APPROACH

Split both strings at 'X' walls (must align), then within each segment apply the L/R two-pointer rule: L only moves left, R only right, order of pieces preserved.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what each symbol means (L moves left, R moves right, dot empty, X wall that never moves and cannot be crossed), whether a token can jump over another (no), and whether both strings have the same length.

BRUTE FORCE TO SAY OUT LOUD

BFS over all board states by moving tokens one step. Exponential.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Compare the sequences of non-dot characters: they must be identical, which also forces every X to appear in the same order. Then with two pointers over those tokens check that each L has start index at least target index, each R has start index at most target index, and each X has the same index in both. Walls cannot be crossed because order is preserved. O(n).

DRY-RUN THESE CASES

Identical strings, an L that would move right, an R blocked by a wall, a different number of tokens, a wall at a different index.

LIKELY FOLLOW-UPS

Return the minimum number of moves (sum of index differences), or the move sequence.

PITFALLS

Letting a token cross a wall. Checking only counts of L and R.

HOW TO THINK ABOUT IT

Walls are pieces that never move. Ignore the dots and compare the sequences of L, R and X: they must match in order and in count, and each X must be at the same index in both strings. For the L and R pieces keep the same rule: L never moves right, R never moves left. Order is preserved, so no piece can get across a wall.

Basis: LeetCode 777 with walls: 'L' moves left, 'R' moves right, '.' is empty, 'X' is an immovable wall; no piece crosses another piece or a wall.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def can_transform(start, target):
    seen, queue = {start}, deque([start])
    while queue:
        board = queue.popleft()
        if board == target:
            return True
        for nxt in one_step(board):
            if nxt not in seen:
                seen.add(nxt)
                queue.append(nxt)
    return False
```

TIME	SPACE
exponential	exponential

WHY IT IS SLOW

The number of reachable boards grows exponentially with the length. Comparing the non-dot sequence (walls and pieces in order) with position rules (L never moves right, R never moves left, walls stay) is O(n).

OPTIMAL SOLUTION · Compare the non-dot sequences with position rules

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Segments between walls are independent.

You can also split both strings at the walls (they must be at identical positions) and apply the L/R rule to each segment separately. The single pass above is equivalent.

FOLLOW-UP 2 Return the moves or the minimum number of steps.

Per piece, the steps are the index difference; list moves by letting each piece walk one cell at a time in an order that never blocks (move L pieces left to right and R pieces right to left).

FOLLOW-UP 3 The board is 2D.

Then pieces can pass each other by going around, and invariants like order no longer hold. Ask for the exact movement rules; most likely you would model it as a graph search with a heuristic.

FOLLOW-UP 4 Return the minimum number of moves.

When the transformation is possible each piece moves independently from its start index to its target index, so the minimum is the sum of the absolute index differences (checked against BFS on small strings).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Decide string transform with directional
# tokens and walls
#
# You are given two equal-length strings start and target
# of length n, each consisting only of the characters 'L',
# 'R', '.', and 'X'. A dot '.' represents an empty slot;
# 'X' represents an immovable wall; 'L' and 'R' are tokens
# that may move across empty slots subject to these rules:
# ( 1) An 'L' token may move any number of positions left
# into '.', but may never move right, pass through another
# token, or pass through an 'X'. ( 2) An 'R' token may
# move any number of positions right into '.', but may
# never move left, pass through another token, or pass
# through an 'X'. ( 3) Only one token can occupy a
# position at any time. Determine whether target can be
# obtained from start by a sequence of valid moves. If
# yes, return true; otherwise, return false. Design an
# algorithm that runs in O(n) time and O( 1) extra space
# beyond a few pointers/counters.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Segments between walls are independent.
# 2. Return the moves or the minimum number of steps.
# 3. The board is 2D.
# 4. Return the minimum number of moves.
#
# INTERVIEW NOTES
# Walls are pieces that never move. Ignore the dots and
# compare the sequences of L, R and X: they must match in
# order and in count, and each X must be at the same index
# in both strings. For the L and R pieces keep the same
# rule: L never moves right, R never moves left. Order is
# preserved, so no piece can get across a wall.
#
# BRUTE FORCE (say it first): BFS over boards with walls
# Apply the legal single-step moves ('.L' becomes 'L.',
# 'R.' becomes '.R'), walls never move, and search for the
# target. Exact, so it is the oracle for the pointer
# check, but it explodes with the length. Time
# exponential. Space exponential.


from collections import deque
```

```
# BRUTE FORCE over real moves (checker)
def bfs_moves(start, end):
    # brute force over real moves (the check used by the
    # demo)
    seen, queue = {start: 0}, deque([start])
    while queue:
        s = queue.popleft()
        if s == end:
            return seen[s]
        for i, ch in enumerate(s):
            for step, piece in ((-1, "L"), (1, "R")):
                j = i + step
                inside = 0 <= j < len(s)
                if ch == piece and inside and s[j] == ".":
                    t = list(s)
                    t[i], t[j] = ".", ch
                    t = "".join(t)
                    if t not in seen:
                        seen[t] = seen[s] + 1
                        queue.append(t)
    return None


# HELPERS for the base question
def pieces(board):
    # everything except the empty '.' slots, with
    # positions
    return [(ch, i) for i, ch in enumerate(board)
            if ch != "."]


# BASE QUESTION (optimal): Compare the non-dot sequences
# with position rules
# Idea: Walk both strings skipping dots. The next symbol
# must be equal; a wall must be at the same index; an L
# must satisfy start index >= target index; an R must
# satisfy start index <= target index.
def can_transform(start, end):
    # 'L' moves left, 'R' moves right, 'X' is an immovable
    # wall. Pieces never pass each other, so their order
    # is
    # fixed; each one only has a direction it may move.
    if len(start) != len(end):
        return False
    a, b = pieces(start), pieces(end)
```

```
    if len(a) != len(b):
        return False
    for (c1, i), (c2, j) in zip(a, b):
        if c1 != c2:
            return False
        if c1 == "X" and i != j:
            return False
        if c1 == "L" and i < j:
            return False
        if c1 == "R" and i > j:
            return False
    return True


# MIN MOVES (pieces move independently)
# FOLLOW-UP 4: Return the minimum number of moves.
# Answer: When the transformation is possible each piece
# moves independently from its start index to its target
# index, so the minimum is the sum of the absolute index
# differences (checked against BFS on small strings).
def min_moves(start, end):
    # pieces move independently: the sum of index
    # differences
    if not can_transform(start, end):
        return None
    a, b = pieces(start), pieces(end)
    return sum(abs(i - j) for (_, i), (_, j) in zip(a, b))


if __name__ == "__main__":
    assert can_transform("R.L", ".RL") is True
    assert not can_transform("RL.", "LR.")
    # a wall cannot move and splits the board
    assert can_transform("R.X.L", ".RX.L") is True
    assert not can_transform("R.X", "X.R")
    assert min_moves("R..L", ".RL.") == 2
    assert min_moves("L.", ".L") is None
    import itertools
    boards = []
    for n in range(1, 5):
        for p in itertools.product("LR.X", repeat=n):
            boards.append("".join(p))
    for s in boards:
        for t in boards:
            if len(s) == len(t) and len(s) <= 4:
```

```
                fast = min_moves(s, t)
                assert fast == bfs_moves(s, t), (s, t)
```

FUNCTION AND TEST CASES

Signature

```
def canTransform(start: str, target: str) -> bool:
```

Sample call

```
canTransform('R...L', 'RL...')
```

Test cases

Input	Expected
`('R...L', 'RL...')`	`True`
`('R.L', 'R..L')`	`False`

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Strings

HARD | PracHub | Technical Screen | 6 of 12

Find the Longest Straight Run of One Color

POSTED 2026-08-23 | 58 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 145/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix

Posted on PracHub: 2026-08-23 (updated 2026-09-10)

Original: https://prachub.com/coding-questions/find-the-longest-straight-run-of-one-color

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **R** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

QUESTION

The down-left diagonal from the top-right cell contains three **G** cells. No horizontal or vertical sequence has that length.

APPROACH

Per-cell DP over four directions (horizontal, vertical, diagonal, anti-diagonal): run[d][i][j] = run[d][prev]+1 if same color else 1. O(m*n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether diagonals count in both directions (yes, anti-diagonals too), whether cells must share an exact color value, and what to return for an empty grid. Confirm that a run cannot bend or skip a cell.

BRUTE FORCE TO SAY OUT LOUD

For every cell and each of the four directions, walk forward while the color matches. That is O(m * n * min(m, n)) and is a fine first answer to say out loud.

OPTIMAL IDEA

Let run[d][i][j] be the length of the same-color run that ends at (i, j) going in direction d. It is run[d][prev] + 1 if the previous cell in that direction has the same color, else 1. Four directions need only the previous row, so memory can be O(n). Answer is the maximum over everything. Time O(m * n).

DRY-RUN THESE CASES

A single cell (answer 1). A 1 x n row of one color. A grid where the longest run is on the anti-diagonal. A grid where every neighbor differs (answer 1).

LIKELY FOLLOW-UPS

Return the cells of the run, not just the length. Allow one wildcard cell. Make the run length at least K and count how many such runs exist.

PITFALLS

Forgetting the anti-diagonal needs the up-right previous cell, so it must be filled in the right scan order. Counting a bent path as a run.

HOW TO THINK ABOUT IT

A run in a fixed direction depends only on the cell before it in that direction, so each of the four directions is a one-line recurrence (like longest common suffix). Only four directions are needed because a line read backwards is the same line. Rolling only the previous row keeps memory at O(n).

Basis: Written; the same recurrence as LeetCode 562, Longest Line of Consecutive One in Matrix, with any color instead of 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = 1
for r, c in all_cells(grid):
    for dr, dc in DIRECTIONS:  # 4 directions
        n = 1
        while same(grid, r, c,
                   r + n * dr, c + n * dc):
            n += 1
        best = max(best, n)
```

TIME	SPACE
O(m * n * min(m, n))	O(1)

WHY IT IS SLOW

A long run is re-walked from every cell inside it, O(m * n * min(m, n)). A table of the run length ending at each cell per direction is O(m * n).

OPTIMAL SOLUTION · DP per direction with a rolling row

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where the run is, not only its length.

Only start a walk at cells that begin a run (the previous cell in that direction has a different color); that keeps the total work linear, and you record the start cell and direction of the best run. Equivalent: keep the DP table and read the end cell of the maximum, then step back by its length.

FOLLOW-UP 2 Require the run to be at least K and count how many such runs exist.

Count a run once, at its start, when its length is at least K (use the begins-a-run test above). Counting at every cell would count a long run many times.

FOLLOW-UP 3 The grid is huge and arrives row by row.

The rolling-row version already needs only the previous row, so it works on a stream of rows with O(n) memory.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the Longest Straight Run of One
# Color
#
# Given a rectangular grid of colors, return the length of
# the longest consecutive sequence of cells with the same
# color along a horizontal, vertical, or diagonal straight
# line. A sequence cannot change direction or skip a cell.
# Both diagonal orientations count. The length is the
# number of cells in the sequence. ### Input - grid: an
# array of strings of equal length. Each character
# represents one cell's color. ### Output Return the
# maximum length as an integer. ### Constraints and Edge
# Cases - For this practice version, the grid contains at
# least one row and one column, with at most 1000000 cells
# in total. - Colors are uppercase English letters; equal
# letters mean equal colors. - A one-cell grid has answer
# 1. - Diagonal lines may run down-right or down-left.
# Reading the same line in reverse does not change its
# length. - A connected region that bends is not
# necessarily one valid sequence.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return where the run is, not only its length.
# 2. Require the run to be at least K and count how many
# such runs exist.
# 3. The grid is huge and arrives row by row.
#
# INTERVIEW NOTES
# A run in a fixed direction depends only on the cell
# before it in that direction, so each of the four
# directions is a one-line recurrence (like longest common
# suffix). Only four directions are needed because a line
# read backwards is the same line. Rolling only the
# previous row keeps memory at O(n).
#
# BRUTE FORCE (say it first): From every cell, walk in
# each of four directions
# For each start cell and direction, step while the color
# matches. Correct and short, but cells in a long run are
# re-walked from every start. Time O(m * n * min(m, n)).
# Space O(1).


# BRUTE FORCE (say it first): From every cell, walk in
# each of four directions | time O(m * n * min(m, n))
```

```
DIRS4 = [(0, 1), (1, 0), (1, 1), (1, -1)]


def longest_run_brute(grid):
    best = 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            for dr, dc in DIRS4:
                k = 0
                while (
                    0 <= r + dr * k < len(grid)
                    and 0 <= c + dc * k < len(grid[0])
                    and grid[r + dr * k][c + dc * k]
                    == grid[r][c]
                ):
                    k += 1
                best = max(best, k)
    return best


# HELPERS for the base question
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


def runs(grid):
    # yield (length, start, direction) for every maximal
    # run;
    # a run is walked only from its first cell, so the
    # total
    # work is linear in the number of cells
    rows, cols = len(grid), len(grid[0])
```

```
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                pr, pc = r - dr, c - dc
                before = 0 <= pr < rows and 0 <= pc < cols
                if before and grid[pr][pc] == grid[r][c]:
                    # not the first cell of its run
                    continue
                n, a, b = 0, r, c
                while 0 <= a < rows and 0 <= b < cols \
                        and grid[a][b] == grid[r][c]:
                    n += 1
                    a, b = a + dr, b + dc
                yield n, (r, c), (dr, dc)
```

```
# BASE QUESTION (optimal): DP per direction with a rolling
# row
# Idea: run[d][i][j] is the length of the same-color run
# ending at (i, j) in direction d. It is the previous
# cell's run plus one when the colors match, else 1.
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    return max(n for n, _, _ in runs(grid))


# FOLLOW-UP 1: Return where the run is, not only its
# length.
# Answer: Only start a walk at cells that begin a run (the
# previous cell in that direction has a different color);
# that keeps the total work linear, and you record the
# start cell and direction of the best run.
def longest_run_where(grid):
    # the length, the start cell and the direction
    if not grid or not grid[0]:
        return 0, None, None
    return max(runs(grid), key=lambda x: x[0])


# FOLLOW-UP 2: Require the run to be at least K and count
# how many such runs exist.
# Answer: Count a run once, at its start, when its length
# is at least K (use the begins-a-run test above).
# Counting at every cell would count a long run many
# times.
def count_runs_at_least(grid, k):
    # each run is counted once, at its start
    if not grid or not grid[0]:
        return 0
    return sum(1 for n, _, _ in runs(grid) if n >= k)


# FOLLOW-UP 3: The grid is huge and arrives row by row.
# Answer: The rolling-row version already needs only the
# previous row, so it works on a stream of rows with O(n)
# memory.
def longest_run_rows(row_stream):
    # rows arrive one by one: keep only the previous row's
```

```
    # runs
    best, prev = 0, None
    h = v = d = a = None
    for row in row_stream:
        n = len(row)
        nh, nv = [1] * n, [1] * n
        nd, na = [1] * n, [1] * n
        for c in range(n):
            if c and row[c] == row[c - 1]:
                nh[c] = nh[c - 1] + 1
            if prev is not None:
                if row[c] == prev[c]:
                    nv[c] = v[c] + 1
                if c and row[c] == prev[c - 1]:
                    nd[c] = d[c - 1] + 1
                if c + 1 < n and row[c] == prev[c + 1]:
                    na[c] = a[c + 1] + 1
            best = max(best, nh[c], nv[c], nd[c], na[c])
        prev, h, v, d, a = row, nh, nv, nd, na
    return best


if __name__ == "__main__":
    g1 = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_run(g1) == 4
    assert longest_run(["RBG", "BGR", "GRB"]) == 3
    assert longest_run(["R"]) == 1
    assert longest_run([]) == 0
    length, start, direction = longest_run_where(g1)
    assert length == 4
    # runs of length at least 3 are counted once each
    assert count_runs_at_least(g1, 3) == 4
    assert longest_run_rows(iter(g1)) == 4
```

FUNCTION AND TEST CASES

Signature

```
def longestColorRun(grid: list[str]) -> int:
```

Sample call

```
longestColorRun(['RRRG', 'RRBB', 'GRRR', 'BGBR'])
```

Test cases

Input	Expected
(['RRRG', 'RRBB', 'GRRR', 'BGBR'],)	4
(['RBG', 'BGR', 'GRB'],)	3

ALSO REPORTED · 1 more

Longest Same-Color Grid Sequence | 1Point3Acres | phone-screen | asked 2026-09-08 | Single report

https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.

Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.

Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

ALSO REPORTED · 1 more

Preparation: Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Strings

HARD | PracHub | Technical Screen | 7 of 12

Convert Valid Snake Case to Lower Camel Case

POSTED 2026-04-16 | 187 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 184/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: No LC match (easy string manipulation)

Posted on PracHub: 2026-04-16 (updated 2026-07-18)

Original: https://prachub.com/coding-questions/convert-valid-snake-case-to-lower-camel-case

QUESTION

Convert Valid Snake Case to

Lower Camel Case Given a valid snake-case identifier, convert it to lower camel case. The input consists of one or more lowercase ASCII words separated by single underscores. There is no leading underscore, trailing underscore, or pair of adjacent underscores. The output keeps the first word lowercase, removes each underscore, and capitalizes the first letter of every later word. Implement:

```
def solve(name: str) -> str: ...
```

Constraints

- `1 <= len(name) <= 100_000`
- `name` matches `[a-z]+(?:_[a-z]+)*`

Examples

```
"foo_bar" -> "fooBar" "already" -> "already" "http_api_name" -> "httpApiName"
```

APPROACH

Split on '_', keep the first word lowercase, capitalize the first letter of the rest, join. O(n). The Hard tag on PracHub looks like a labeling quirk.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether input can have digits, uppercase, leading or trailing underscores, or double underscores. The problem states it is valid, so say you will rely on that and mention what you would add if not.

BRUTE FORCE TO SAY OUT LOUD

Loop over characters and capitalize the character after each underscore. Linear and simple.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Split on '_', keep the first word as is, capitalize the first letter of each other word and join. O(n). The Hard label on PracHub looks like a labeling quirk; do not over-engineer it.

DRY-RUN THESE CASES

A single word, two words, many words, a one-letter word.

LIKELY FOLLOW-UPS

Convert back from camel to snake, or handle acronyms.

PITFALLS

Capitalizing the first word. Using replace on every underscore without capitalizing.

HOW TO THINK ABOUT IT

The statement says the input is valid (single underscores, no edge underscores), so rely on that and say what you would add if it were not. The pattern is 'first word unchanged, capitalize the first letter of each later word'.

Basis: Written. The PracHub Hard label looks like a labeling quirk; this is an easy string task.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def to_camel(s):
    out, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            out += ch.upper() if up else ch
            up = False
    return out
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Already O(n); it is just longer than needed. Splitting on '_' and capitalising each word after the first is the short form.

OPTIMAL SOLUTION · Split on underscores and capitalize

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The input may have leading, trailing or repeated underscores, or mixed case.

Drop empty parts after the split, lowercase the first word, capitalize the rest. Say what you do with digits and non-letters, and keep the rule in one place.

FOLLOW-UP 2 Convert back from camel case to snake case.

Insert an underscore before each uppercase letter and lowercase it. Acronyms such as HTTPServer need a smarter rule (split between a lowercase-to-uppercase edge and between an acronym and the next word).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Convert Valid Snake Case to Lower Camel
# Case
#
# # Convert Valid Snake Case to Lower Camel Case Given a
# valid snake-case identifier, convert it to lower camel
# case. The input consists of one or more lowercase ASCII
# words separated by single underscores. There is no
# leading underscore, trailing underscore, or pair of
# adjacent underscores. The output keeps the first word
# lowercase, removes each underscore, and capitalizes the
# first letter of every later word. Implement: python def
# solve(name: str) -> str: ... ## Constraints - 1 <=
# len(name) <= 100_000 - name matches [a-z]+(?:_[a-z]+)*
# ## Examples text "foo_bar" -> "fooBar" "already" ->
# "already" "http_api_name" -> "httpApiName"
#
# FOLLOW-UPS (answered below, in this order):
# 1. The input may have leading, trailing or repeated
# underscores, or mixed case.
# 2. Convert back from camel case to snake case.
#
# INTERVIEW NOTES
# The statement says the input is valid (single
# underscores, no edge underscores), so rely on that and
# say what you would add if it were not. The pattern is
# 'first word unchanged, capitalize the first letter of
# each later word'.
#
# BRUTE FORCE (say it first): Walk the characters and
# capitalize after each underscore
# Keep a flag that is set after an underscore; when set,
# append the uppercase of the next letter. Linear and
# fine, just longer than the split-based answer. Time
# O(n). Space O(n).


import re


# BRUTE FORCE (say it first): Walk the characters and
# capitalize after each underscore | time O(n)
def snake_to_camel_brute(s):
    out, up = [], False
    for ch in s:
        if ch == "_":
```

```
            up = True
        else:
            out.append(ch.upper() if up else ch)
            up = False
    return "".join(out)


# BASE QUESTION (optimal): Split on underscores and
# capitalize
# Idea: Split on '_', keep the first word, and capitalize
# the first letter of each other word with a join. For
# valid input there are no empty parts.
def snake_to_camel(s):
    # foo_bar_baz -> fooBarBaz
    first, *rest = s.split("_")
    caps = [w[:1].upper() + w[1:] for w in rest]
    return first + "".join(caps)


# FOLLOW-UP 1: The input may have leading, trailing or
# repeated underscores, or mixed case.
# Answer: Drop empty parts after the split, lowercase the
# first word, capitalize the rest. Say what you do with
# digits and non-letters, and keep the rule in one place.
def snake_to_camel_tolerant(s):
    # leading, trailing or repeated underscores and mixed
    # case: drop empty parts, lowercase the first word
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    first = words[0].lower()
    rest = [w.capitalize() for w in words[1:]]
    return first + "".join(rest)


# FOLLOW-UP 2: Convert back from camel case to snake case.
# Answer: Insert an underscore before each uppercase
# letter and lowercase it. Acronyms such as HTTPServer
# need a smarter rule (split between a lowercase-to-
# uppercase edge and between an acronym and the next
# word).
def camel_to_snake(s):
    # fooBarBaz -> foo_bar_baz, and HTTPServer ->
    # http_server
    # 1) split an acronym from the next word: HTTPServer
```

```
    # -> HTTP_Server
    s = re.sub(r"([A-Z]+)([A-Z][a-z])", r"\1_\2", s)
    # 2) split a lower-case letter or digit from an upper-
    # case one
    s = re.sub(r"([a-z0-9])([A-Z])", r"\1_\2", s)
    return s.lower()


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    got = snake_to_camel("get_http_response")
    assert got == "getHttpResponse"
    # messy input
    messy = snake_to_camel_tolerant("_Foo__BAR_")
    assert messy == "fooBar"
    assert snake_to_camel_tolerant("___") == ""
    # and back again
    assert camel_to_snake("fooBarBaz") == "foo_bar_baz"
    assert camel_to_snake("HTTPServer") == "http_server"
    assert camel_to_snake("x") == "x"
    camel = snake_to_camel("my_var_2")
    assert camel_to_snake(camel) == "my_var2"
```

FUNCTION AND TEST CASES

Signature

```
def solve(name) -> str:
```

Sample call

```
solve('a')
```

Test cases

Input	Expected
`('a',)`	`'a'`
`('already',)`	`'already'`

ALSO REPORTED · 2 more

Define the scope and requirements for a snake_case to camelCase converter function. | Exponent

https://www.aced.io/questions/6328/define-scope-snake-case-camel-case-converter

Snake Case to Camel Case with Repository-Wide Refactor | 1Point3Acres | tech-screen | asked 2026-07-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

Preparation: Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Strings

HARD | PracHub | Technical Screen | 8 of 12

Find Dictionary Matches for a Jammed String

POSTED 2026-04-09 | 194 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 134/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Strings

Optimal answer from: the interview site's own solution

LeetCode match: LC 809 Expressive Words (inferred from overview)

Posted on PracHub: 2026-04-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/find-dictionary-matches-for-a-jammed-string

QUESTION

Overview: This question evaluates string-processing and pattern-matching skills, specifically run-length grouping of consecutive characters and efficient dictionary lookup, within the Coding & Algorithms domain.

APPROACH

Run-length encode the query and every dictionary word; a word matches if run characters align and each run length is compatible. Pre-group dictionary by compressed signature in a hash map for fast lookup.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what jammed means exactly: a letter repeated more than it should be, and whether the repeat count must be at least the word's count or at least some threshold. Confirm case, empty strings and whether a match requires the same letter order and same number of runs.

BRUTE FORCE TO SAY OUT LOUD

For each dictionary word, compare it with the query run by run. O(W * L) per query.

OPTIMAL IDEA

Compress each word into runs of (letter, count). Use the letter sequence (without counts) as a hash key so only words with the same run letters are candidates, then compare the counts under the agreed rule. For repeated queries a trie over run letters keeps lookups proportional to the number of runs.

DRY-RUN THESE CASES

Query 'heeelllo' against 'hello' and 'helo'. A query with fewer runs than the word. A single-letter word. A word whose runs are longer than the query's.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Stretch only allowed for runs of three or more, return all matches in input order, handle a huge dictionary with a trie.

PITFALLS

Treating 'ee' to 'e' as a match without checking the rule. Comparing total length instead of per-run counts.

HOW TO THINK ABOUT IT

'Same letters, stretched' means compare run-length encodings, not characters. The letter order of the runs has to match exactly, so the sequence of distinct run letters is a hash key that cuts the dictionary down before any counts are compared.

Basis: LeetCode 809, Expressive Words. Assumed rule (confirm it): a query run may be longer than the word's run only when the query run has length at least 3; otherwise the lengths must be equal.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def matches(query, words):
    runs_q = runs(query)  # [(letter, count)]
    result = []
    for word in words:
        runs_w = runs(word)
        if compatible(runs_w, runs_q):
            result.append(word)
    return result
```

TIME	SPACE
O(W * L) per query	O(L)

WHY IT IS SLOW

Every query touches every word, O(W * L), even words whose letters cannot match. Grouping words by their run letters in a hash map checks only the right bucket.

OPTIMAL SOLUTION · Index the dictionary by its run letters

TIME	SPACE
O(L) per query plus the size of the matching bucket	O(total dictionary length)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A different stretch rule: any run may be longer, no minimum.

Only the comparison changes. The index (grouping by run letters) is the same, so make the minimum stretch a parameter, as the code does with min_stretch.

FOLLOW-UP 2 Millions of dictionary words and many queries.

Keep the group index in memory and put a trie over the run-letter strings if the keys share long prefixes. For counts compare only the first words in a bucket that survive a cheap check such as total length. **Cost:** O(L) per query to build the run key plus the size of the bucket scanned; building the index is O(total dictionary length) once.

FOLLOW-UP 3 The jammed string may also be missing letters.

That becomes an edit-distance style problem on run encodings, usually solved with DP over (query run, word run). Ask for the exact rule before coding it.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find Dictionary Matches for a Jammed
# String
#
# This question evaluates string-processing and pattern-
# matching skills, specifically run-length grouping of
# consecutive characters and efficient dictionary lookup,
# within the Coding & Algorithms domain.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A different stretch rule: any run may be longer, no
# minimum.
# 2. Millions of dictionary words and many queries.
# 3. The jammed string may also be missing letters.
#
# INTERVIEW NOTES
# 'Same letters, stretched' means compare run-length
# encodings, not characters. The letter order of the runs
# has to match exactly, so the sequence of distinct run
# letters is a hash key that cuts the dictionary down
# before any counts are compared.
#
# BRUTE FORCE (say it first): Compare the query with every
# dictionary word
# Encode the query once and compare it with each word's
# runs. It is correct; the cost is that every query
# touches every word, even words whose letters cannot
# match. Time O(W * L) per query. Space O(L).


from itertools import groupby


# HELPERS for the brute force
def _runs(s):
    out = []
    for ch in s:
        if out and out[-1][0] == ch:
            out[-1][1] += 1
        else:
            out.append([ch, 1])
    return out


# BRUTE FORCE (say it first): Compare the query with every
# dictionary word | time O(W * L) per query
def _stretchy(query, word, min_stretch=3):
    a, b = _runs(query), _runs(word)
    if len(a) != len(b):
        return False
    return all(
        qa == wa
        and (
            qc == wc or (qc > wc and qc >= min_stretch)
        )
        for (qa, qc), (wa, wc) in zip(a, b)
    )


def jammed_matches_brute(words, query):
    return [w for w in words if _stretchy(query, w)]


# HELPERS for the base question
def runs(word):
    # "heello" -> [("h", 1), ("e", 2), ("l", 2), ("o", 1)]
    return [(ch, len(list(g))) for ch, g in groupby(word)]


# BASE QUESTION (optimal): Index the dictionary by its run
# letters
# Idea: Group words by the string of their distinct run
# letters ('hello' -> 'helo'). A query can only match
# words in its own group, so compare run counts only
# there. Build the index once and reuse it for many
# queries.
# FOLLOW-UP 1: A different stretch rule: any run may be
# longer, no minimum.
# Answer: Only the comparison changes. The index (grouping
# by run letters) is the same, so make the minimum stretch
# a parameter, as the code does with min_stretch.
class JammedIndex:
    """A query run may be longer than the word's run
    when it is at least min_stretch long."""

    def __init__(self, words):
        # words grouped by their run letters ("hello" ->
        # "helo")
        self.groups = {}
        for w in words:
            r = runs(w)
            letters = "".join(ch for ch, _ in r)
            self.groups.setdefault(letters, []).append(
                (w, [n for _, n in r]))

    # FOLLOW-UP 1: min_stretch=1: any run may be longer
    def matches(self, query, min_stretch=3):
        r = runs(query)
        letters = "".join(ch for ch, _ in r)
        counts = [n for _, n in r]
        found = []
        # only words with the same run letters can match
        group = self.groups.get(letters, [])
        for word, word_counts in group:
            if all(self.fits(q, w, min_stretch)
                   for q, w in zip(counts, word_counts)):
                found.append(word)
        return sorted(found)

    @staticmethod
    def fits(q, w, min_stretch):
        # the same length, or a stretch of at least
        # min_stretch
        return q == w or (q > w and q >= min_stretch)


if __name__ == "__main__":
    index = JammedIndex(["hello", "help", "hi", "hip"])
    # e stretched to 4 and o stretched to 3
```

```
    assert index.matches("heeeelloooo") == ["hello"]
    assert index.matches("hello") == ["hello"]
    # a stretch of only 2 is not enough
    assert index.matches("heello") == []
    assert index.matches("hiiii") == ["hi"]
    assert index.matches("zzz") == []
    # a smaller minimum accepts the short stretch
    short = index.matches("heello", min_stretch=2)
    assert short == ["hello"]
    # no minimum: any longer run is fine
    assert index.matches("hiip", min_stretch=1) == ["hip"]
    want = [("a", 2), ("b", 1), ("c", 3)]
    assert runs("aabccc") == want
```

FUNCTION AND TEST CASES

Signature

```
def solution(dictionary_words: list[str], jammed_string: s
tr) -> list[str]:
```

Sample call

```
solution(['help', 'hello', 'shell', 'hellp'], 'hellp')
```

Test cases

Input	Expected
(['help', 'hello', 'shell', 'hellp'], 'hellp')	['help', 'hellp']
(['hello', 'halo', 'heeello'], 'hheeellllo')	['hello', 'heeello']

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Recursion and backtracking

Site tags: recursion, simulation, medium

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 9 of 12

Mahjong Winning Hand Detection (14 Tiles)

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 210/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Recursion and backtracking

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

APPROACH

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

HOW TO THINK ABOUT IT

A winning hand is one pair plus four melds (a triplet of identical tiles or a run of three consecutive ranks in one suit). Try every tile as the pair, then repeatedly take the smallest remaining tile: it can only be covered by a triplet of itself or by a run that starts at it, so there are at most two choices at each step and the search is tiny for 14 tiles.

HOW TO THINK ABOUT IT

Basis: Written for this question; the pair-plus-melds check is the same backtracking as LeetCode 846, Hand of Straights, plus a triplet option.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(tiles):
    for t in set(tiles):
        if tiles.count(t) >= 2:
            rest = remove(tiles, [t, t])
            if all_melds(rest):
                return True
    return False
def all_melds(tiles):
    if not tiles:
        return True
    first = min(tiles)
    for meld in (triple(first), run(first)):
        if contains(tiles, meld):
            if all_melds(remove(tiles, meld)):
                return True
    return False
```

TIME	SPACE
roughly C(12,3) * C(9,3) ... roughly C(12,3) * C(9,3) * C(6,3) per pair	**O(1)**

WHY IT IS SLOW

Tries every set of three tiles at every step. The smallest remaining tile can only be in a triplet of itself or a run starting at it, so at most two choices remain.

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

TIME	SPACE
O(1) for 14 tiles (about 2^4 branches per pair)	**O(distinct tiles)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A variant needs only two melds plus a pair (8 tiles of 14).

Ask what happens to the other six tiles before coding: if they are free, check whether any 8 tiles form a pair and two melds with the same search over subsets of the 14. The rule set is intentionally vague, so confirm melds, honours and flowers first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Mahjong Winning Hand Detection (14 Tiles)
#
# Mountain View onsite coding round 1: given a hand of 14
# tiles, decide whether it is a winning mahjong hand. The
# candidate must clarify the rule set (typically: 4 melds
# of three identical tiles or three consecutive ranks + 1
# pair). Input: array of 14 integers representing tile
# values (clarify suit / range during the interview). A
# winning hand consists of: One pair (two identical
# tiles), and Four melds, each being either: A triplet (3
# identical tiles), or A run (3 consecutive ranks in the
# same suit). Minority variant: A newer version instead
# specifies one pair plus two melds while retaining a
# 14-tile input. Because that accounts for only eight
# tiles, clarify the intended meld count and how the
# remaining six tiles should be handled before coding.
# Output: boolean.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A variant needs only two melds plus a pair (8 tiles
# of 14).
#
# INTERVIEW NOTES
# A winning hand is one pair plus four melds (a triplet of
# identical tiles or a run of three consecutive ranks in
# one suit). Try every tile as the pair, then repeatedly
# take the smallest remaining tile: it can only be covered
# by a triplet of itself or by a run that starts at it, so
# there are at most two choices at each step and the
# search is tiny for 14 tiles.
#
# BRUTE FORCE (say it first): Choose any three tiles for
# every meld
# For each pair, try every set of three remaining tiles
# that forms a meld and recurse. Correct and the oracle;
# it ignores that the smallest tile must be used first.
# Time roughly C(12,3) * C(9,3) * C(6,3) per pair. Space
# O(1).


import itertools
from collections import Counter


# BRUTE FORCE (say it first): Choose any three tiles for
# every meld | time roughly C(12,3) * C(9,3) * C(6,3) per
# pair
def mahjong_brute(hand):
    def is_meld(t):
        t = sorted(t)
        same = t[0] == t[1] == t[2]
        run = (
            t[0] // 10 == t[1] // 10 == t[2] // 10 < 3
            and t[1] == t[0] + 1
            and t[2] == t[0] + 2
        )
        return same or run

    def melds(tiles):
        if not tiles:
            return True
        for idx in itertools.combinations(
            range(len(tiles)), 3
        ):
            if is_meld(
                [tiles[i] for i in idx]
            ) and melds(
                [
                    t
                    for i, t in enumerate(tiles)
                    if i not in idx
                ]
            ):
                return True
        return False

    for i, j in itertools.combinations(
        range(len(hand)), 2
    ):
        if hand[i] == hand[j] and melds(
            [
                t
                for k, t in enumerate(hand)
                if k not in (i, j)
            ]
        ):
            return True
    return False
```

```
# HELPERS for the base question
def is_run_start(tile, count):
    # numbered suits are 0..2 with ranks 1..9; the tile
    # id is suit * 10 + rank. Honours (30+) never run.
    ok = tile < 30 and tile % 10 <= 7
    nxt, nxt2 = count.get(tile + 1), count.get(tile + 2)
    return ok and nxt and nxt2


def melds_left(count, need):
    # can the tiles in `count` form exactly `need` melds?
    live = [t for t in sorted(count) if count[t]]
    tile = live[0] if live else None
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3
        if ok:
            return True
    # ... or starts a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1
        return ok
    return False


# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# Idea: Tiles are encoded as suit * 10 + rank for the
# numbered suits (ranks 1 to 9, suits 0 to 2) and 30 and
# above for honours, which never form runs.
# FOLLOW-UP 1: A variant needs only two melds plus a pair
# (8 tiles of 14).
# Answer: Ask what happens to the other six tiles before
# coding: if they are free, check whether any 8 tiles form
# a pair and two melds with the same search over subsets
# of the 14.
def is_winning_hand(hand, melds=4):
```

```
    # FOLLOW-UP 1: melds=2 handles the 8-tile variant
    # one pair plus `melds` melds uses every tile
    if len(hand) != 2 + 3 * melds:
        return False
    count = Counter(hand)
    # try every tile value as the pair
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False


if __name__ == "__main__":
    win = [1, 1, 1, 2, 3, 4, 11, 12, 13,
           21, 22, 23, 31, 31]
    assert is_winning_hand(win)
    # 2, 3, 5 is not a run
    lose = [1, 1, 1, 2, 3, 5, 11, 12, 13,
            21, 22, 23, 31, 31]
    assert not is_winning_hand(lose)
    # honours only make triplets or the pair
    honours = [30, 30, 31, 31, 31, 32, 32, 32, 33, 33, 33,
               34, 34, 34]
    assert is_winning_hand(honours)
    # the smaller variant: a pair and two melds (8 tiles)
    assert is_winning_hand([5, 5, 1, 2, 3, 7, 7, 7], 2)
    # wrong tile count is never a win
    assert not is_winning_hand(win[:13])
```

NOTES FROM CANDIDATES

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:

Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.

For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

PREPARATION

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Recursion and backtracking

MEDIUM | PracHub | Technical Screen | 10 of 12

Count Prefix-Matching Simple Paths in a Labeled Graph

POSTED 2026-08-09 | 72 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 42.5 | 30 DAYS 0 | PRIORITY 83/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Recursion and backtracking

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #79 Word Search

LeetCode match: No direct match; related LC 79 Word Search (inferred from overview)

Posted on PracHub: 2026-08-09 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/count-prefix-matching-simple-paths-in-a-labeled-graph

QUESTION

Overview: Count distinct simple graph paths whose labels match every nonempty prefix of a query word. Paths may start anywhere but cannot revisit a vertex, so the task rewards subset-state search and careful counting over a small labeled graph.

APPROACH

DFS/backtracking from every start vertex, extending a path only while the labels match the next query character and the vertex is unvisited (visited bitmask); count paths for every prefix length.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the graph is directed, whether a path is counted as a vertex sequence, whether paths of every prefix length are counted (the statement says every nonempty prefix), and the size (small, since the state is a visited subset).

BRUTE FORCE TO SAY OUT LOUD

Enumerate all simple paths and then test labels. Exponential and wasteful.

OPTIMAL IDEA

DFS with backtracking from every start vertex. Extend a path only if the next vertex is unvisited (a bitmask of visited vertices) and its label equals the next character of the query word. Every time the path length is L, count one for prefix length L. The pruning on labels keeps the search small. Time depends on the number of matching paths, bounded by n * d^(L-1).

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

A query of length 1 (count of vertices with that label), a graph with a cycle (simple paths may not revisit), a label that appears nowhere, repeated letters in the query.

LIKELY FOLLOW-UPS

Longer queries (memoize on (vertex, mask, position) only helps if counting, not listing), or undirected edges.

PITFALLS

Counting a path that revisits a vertex. Counting an undirected path twice (once in each direction) without agreeing on it.

HOW TO THINK ABOUT IT

A path is only worth extending while its labels still match the query, so prune on the label before going deeper. Distinctness is tracked with a visited set (a bitmask for small graphs). Count at every depth, since each prefix length has its own answer.

Basis: Written; backtracking on a small labelled graph. Assumed rules (confirm them): directed edges; a path is a sequence of distinct vertices; for each L from 1 to the length of the query, count the paths of L vertices whose labels spell the first L letters of the query.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
def dfs(path):
    global count
    if len(path) <= len(query):
        if matches_prefix(path):
            count += 1
    for nxt in graph[path[-1]]:
        if nxt not in path:
            dfs(path + [nxt])
for v in vertices:
    dfs([v])
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

It explores paths whose first label already disagrees with the query. Extending a path only when the next label matches the next query letter prunes those branches.

OPTIMAL SOLUTION · DFS with label pruning and a visited bitmask

TIME	SPACE
O(number of matching paths * degree)	O(path)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges are undirected.

Add every edge in both directions, as above. A path is then counted once per direction unless you divide the counts for length >= 2 by 2 (a path and its reverse are different sequences only if the query is not a palindrome, so say how you count).

FOLLOW-UP 2 Count only the paths that spell the whole query.

Return counts[-1] instead of the list. For long queries on bigger graphs, memoise on (vertex, depth) if repeated vertices are allowed (walks, not simple paths); for simple paths the visited set prevents memoisation.

FOLLOW-UP 3 The graph has up to 100,000 vertices.

Simple paths are exponential, so ask whether walks are allowed. For walks a DP over (depth, vertex) counts them in O(L * E), the standard way to handle large graphs.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count Prefix-Matching Simple Paths in a
# Labeled Graph
#
# Count distinct simple graph paths whose labels match
# every nonempty prefix of a query word. Paths may start
# anywhere but cannot revisit a vertex, so the task
# rewards subset-state search and careful counting over a
# small labeled graph.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges are undirected.
# 2. Count only the paths that spell the whole query.
# 3. The graph has up to 100,000 vertices.
#
# INTERVIEW NOTES
# A path is only worth extending while its labels still
# match the query, so prune on the label before going
# deeper. Distinctness is tracked with a visited set (a
# bitmask for small graphs). Count at every depth, since
# each prefix length has its own answer.
#
# BRUTE FORCE (say it first): Enumerate all simple paths,
# then test the labels
# List every simple path of every length and compare its
# labels with each prefix of the query. Correct and the
# oracle; it explores paths whose first label already
# disagrees. Time exponential. Space O(path).


from collections import defaultdict


# BRUTE FORCE (say it first): Enumerate all simple paths,
# then test the labels | time exponential
def prefix_paths_brute(labels, edges, query):
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    counts = [0] * len(query)

    def go(path):
        word = "".join(labels[v] for v in path)
        if (
            len(path) <= len(query)
            and word == query[: len(path)]
        ):
            counts[len(path) - 1] += 1
        if len(path) < len(query):
            for w in adj[path[-1]]:
                if w not in path:
                    go(path + [w])

    for s in range(len(labels)):
        go([s])
    return counts


# BASE QUESTION (optimal): DFS with label pruning and a
# visited bitmask
# Idea: Start from every vertex whose label equals the
# first query letter. Extend to an unvisited neighbour
# only if its label equals the next query letter, adding
# one to the count for the current length at each step.
def prefix_paths(labels, edges, query, undirected=False):
    # counts[d]: simple paths whose first d + 1 labels
    # spell
    # the first d + 1 letters of the query
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        # FOLLOW-UP 1: undirected=True adds both
        # directions
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)

    def go(u, depth, mask):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            # a simple path never revisits a vertex
            if mask >> w & 1:
                continue
            if labels[w] == query[depth + 1]:
                go(w, depth + 1, mask | 1 << w)

    # a path may start at any vertex with the first letter
    for s in range(len(labels)):
        if query and labels[s] == query[0]:
            go(s, 0, 1 << s)
    return counts


# VARIANT: walks may revisit vertices
def prefix_walks(labels, edges, query):
    # walks may revisit vertices: a DP over (depth,
    # vertex)
    # costs O(L * E) and works for very large graphs
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    ways = {v: 1 for v in range(len(labels))
            if query and labels[v] == query[0]}
    counts = [sum(ways.values())] if query else []
    for ch in query[1:]:
        nxt = defaultdict(int)
        for u, n in ways.items():
            for w in adj[u]:
```

```
                if labels[w] == ch:
                    nxt[w] += n
        ways = nxt
        counts.append(sum(ways.values()))
    return counts


# FOLLOW-UP 2: Count only the paths that spell the whole
# query.
# Answer: Return counts[-1] instead of the list. For long
# queries on bigger graphs, memoise on (vertex, depth) if
# repeated vertices are allowed (walks, not simple paths);
# for simple paths the visited set prevents memoisation.
def whole_query_paths(labels, edges, query,
                      undirected=False):
    # only the paths that spell the entire query
    both = undirected
    counts = prefix_paths(labels, edges, query, both)
    return counts[-1] if counts else 0


if __name__ == "__main__":
    labels = ["a", "b", "a", "b"]
    edges = [(0, 1), (1, 2), (2, 3), (3, 0)]
    # "aba": directed square a>b>a>b>...
    assert prefix_paths(labels, edges, "aba") == [2, 2, 2]
```

```
    assert whole_query_paths(labels, edges, "aba") == 2
    # undirected: every edge works both ways
    und = prefix_paths(labels, edges, "ab", True)
    assert und == [2, 4]
    # a walk may reuse vertices, a simple path may not
    walks = prefix_walks(labels, edges, "abab")
    assert walks == [2, 2, 2, 2]
    five = prefix_paths(labels, edges, "ababa")
    assert five == [2, 2, 2, 2, 0]
    assert prefix_paths(labels, edges, "") == []
```

FUNCTION AND TEST CASES

Signature

```
def count_prefix_paths(labels: str, edges: list[list[int]]
, word: str) -> list[int]:
```

Sample call

```
count_prefix_paths('roteoreote', [[0, 1], [0, 2], [0, 3],
[1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')
```

Test cases

Input	Expected
`('roteoreote', [[0, 1], [0, 2], [0, 3], [1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')`	`[2, 2, 1, 1, 0]`
`('a', [], 'aa')`	`[1, 0]`

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Recursion and backtracking

Site tags: grid, backtracking, hard

HARD | 1Point3Acres | onsite-coding | 11 of 12

Matrix Flower Placement with House Adjacency

ASKED 2026-05-31 | 142 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 190/228**

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Recursion and backtracking

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-05-31 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/matrix-flower-placement-with-house-adjacency

QUESTION

EXAMPLES

```
H 0 0 0
H 0 0 0
0 0 0 H
0 0 0 0
```

The task is to fill some 0 cells with F while satisfying both the row/column constraint and each house's adjacency requirement.

APPROACH

Practice N-Queens with row/column sets and a recursive placement order.
Drill grid-neighbor helpers and constraint pruning before writing the full DFS.
Prepare the boolean version first, then generalize to collect all solutions.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether each row and column holds at most one or exactly one flower, whether a house needs at least one or exactly one adjacent flower, whether adjacency is four-way, and whether to return one solution, a boolean, or all.

BRUTE FORCE TO SAY OUT LOUD

Try every subset of empty cells. Exponential.

OPTIMAL IDEA

Row-by-row backtracking like N-Queens: place at most one flower per row, track used columns, and check each house as soon as the row below it is decided (its neighbors are in rows h - 1 to h + 1). Prune at the first failed house. The boolean version stops at the first solution; the all-solutions version continues.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

No houses, a house with no empty neighbor (impossible), two houses sharing one neighbor, a 1 x 1 grid.

LIKELY FOLLOW-UPS

Exactly-one rule, all solutions, or a faster exact-cover search.

PITFALLS

Checking a house before the next row is decided. Not restoring the cell on backtrack.

HOW TO THINK ABOUT IT

At most one flower per row and column is the N-Queens pattern: decide row by row which column (if any) gets the flower and remember the used columns. The extra rule about houses is local, so check a house as soon as the last row that can affect it is decided, and prune the branch if it fails.

Basis: N-Queens style backtracking (LeetCode 51/52) with extra local constraints. Assumptions stated explicitly: every house must have at least one flower in a 4-neighbour cell; rows and columns hold at most one flower (or exactly one if requested).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    empty = cells_with(grid, 0)
    count = 0
    for k in range(len(empty) + 1):
        for flowers in combinations(empty, k):
            if not one_per_row_and_col(flowers):
                continue
            if houses_covered(grid, flowers):
                count += 1
    return count
```

TIME	SPACE
O(2^cells)	O(cells)

WHY IT IS SLOW

2^cells subsets: exponential. Backtracking row by row with a used-columns set and early house checks prunes almost all of them (still exponential in the worst case).

OPTIMAL SOLUTION · Row-by-row backtracking with used-column set and delayed house checks

TIME	SPACE
exponential worst case, h... exponential worst case, heavily pruned	O(rows + cols)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What changes between 'at most one' and 'exactly one' flower per row and column?

With 'exactly one' the number of flowers is forced (rows must equal columns) and a row cannot be skipped, which prunes much harder, like N-Queens without the diagonal rule. With 'at most one' you also branch on skipping a row. State which reading you are using before writing code.

FOLLOW-UP 2 How would you speed up the search?

Prune earlier by checking each house as soon as its last neighbour row is decided (done above), order houses by how few candidate cells they have (most-constrained first), and keep bitmasks for used columns. For large grids this becomes a constraint satisfaction or exact-cover problem, so a SAT solver or DLX is the next step. **Cost:** the worst case stays exponential; most-constrained-first ordering and early house checks cut the explored states by orders of magnitude on typical grids.

FOLLOW-UP 3 Exactly one flower per row and column.

Add a flag that forbids skipping a row and requires as many columns as rows (otherwise no solution). The pruning is stronger, since every row must find a column.

FOLLOW-UP 4 Count the solutions instead of listing them.

Keep the same recursion but add one to a counter at the leaf instead of copying the grid; with a bitmask of used columns and memoisation on (row, mask, pending-house state) it can become a DP.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Matrix Flower Placement with House
# Adjacency
#
# Examples: H 0 0 0 H 0 0 0 0 0 0 H 0 0 0 0 The task is to
# fill some 0 cells with F while satisfying both the
# row/column constraint and each house's adjacency
# requirement.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What changes between 'at most one' and 'exactly one'
# flower per row and column?
# 2. How would you speed up the search?
# 3. Exactly one flower per row and column.
# 4. Count the solutions instead of listing them.
#
# INTERVIEW NOTES
# At most one flower per row and column is the N-Queens
# pattern: decide row by row which column (if any) gets
# the flower and remember the used columns. The extra rule
# about houses is local, so check a house as soon as the
# last row that can affect it is decided, and prune the
# branch if it fails.
#
# BRUTE FORCE (say it first): Choose any subset of empty
# cells and check all the rules
# Enumerate every subset of empty cells, keep those with
# at most one flower per row and column whose every house
# has a flower neighbour, and count them. Exact, the
# oracle, and exponential in the number of cells. Time
# O(2^cells). Space O(cells).


import itertools


# BRUTE FORCE (say it first): Choose any subset of empty
# cells and check all the rules | time O(2^cells)
def count_placements_brute(grid, exactly_one=False):
    R, C = len(grid), len(grid[0])
    empties = [
        (r, c)
        for r in range(R)
        for c in range(C)
        if grid[r][c] == "0"
    ]
    total = 0
    for k in range(len(empties) + 1):
        for chosen in itertools.combinations(
            empties, k
        ):
            rows, cols = [r for r, _ in chosen], [
                c for _, c in chosen
            ]
            if len(set(rows)) != len(rows) or len(
                set(cols)
            ) != len(cols):
                continue
            if exactly_one and not (
                len(rows) == R == C
            ):
                continue
            fl = set(chosen)
            if all(
                any(
                    (r + dr, c + dc) in fl
                    for dr, dc in (
                        (1, 0),
                        (-1, 0),
                        (0, 1),
                        (0, -1),
                    )
                )
                for r in range(R)
                for c in range(C)
                if grid[r][c] == "H"
            ):
                total += 1
    return total


# HELPERS for the base question
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# BASE QUESTION (optimal): Row-by-row backtracking with
# used-column set and delayed house checks
# Idea: Place at most one flower per row, tracking used
# columns, and only on empty cells ('0').
def place_flowers(grid, exactly_one=False,
                  find_all=False):
    # 'H' house, '0' empty. At most one F per row and
    # column; every house needs an F on a side.
    R, C = len(grid), len(grid[0])
```

```
    cells = [list(row) for row in grid]
    houses = [(r, c) for r in range(R) for c in range(C)
              if cells[r][c] == "H"]
    found = []

    def house_ok(r, c):
        # is some neighbour of this house a flower?
        for dr, dc in STEPS:
            a, b = r + dr, c + dc
            if 0 <= a < R and 0 <= b < C \
                    and cells[a][b] == "F":
                return True
        return False

    def row_houses_ok(r):
        return all(house_ok(hr, hc)
                   for hr, hc in houses if hr == r)

    def solve(r, used):
        # FOLLOW-UP 4: find_all=True lists every solution
        if found and not find_all:
            return
        # rows up to r-1 are decided, so row r-2 is final
        if r >= 2 and not row_houses_ok(r - 2):
            return
        if r == R:
            # FOLLOW-UP 3: exactly_one=True: a flower in
            # every column
            full = len(used) == C
            if row_houses_ok(R - 1) and (full or
                                         not exactly_one):
                found.append(["".join(x) for x in cells])
            return
        # try a flower in every free column of this row
        for c in range(C):
            if c not in used and cells[r][c] == "0":
                cells[r][c] = "F"
                solve(r + 1, used | {c})
                cells[r][c] = "0"
        # or leave the row empty (not allowed for exactly
        # one)
        if not exactly_one:
```

```
            solve(r + 1, used)

    solve(0, frozenset())
    if find_all:
        return found
    return found[0] if found else None


# FOLLOW-UP 4: Count the solutions instead of listing
# them.
# Answer: Keep the same recursion but add one to a counter
# at the leaf instead of copying the grid; with a bitmask
# of used columns and memoisation on (row, mask, pending-
# house state) it can become a DP.
def count_solutions(grid, exactly_one=False):
    return len(place_flowers(grid, exactly_one, True))


if __name__ == "__main__":
    grid = ["H00", "0H0", "00H"]
    sol = place_flowers(grid)
    assert sol is not None
    # every house has a flower beside it
    for r in range(3):
        for c in range(3):
            if grid[r][c] == "H":
                near = []
                for a, b in STEPS:
                    x, y = r + a, c + b
                    if 0 <= x < 3 and 0 <= y < 3:
```

```
                        near.append(sol[x][y])
                assert "F" in near
    # one flower per row and column
    assert all(row.count("F") <= 1 for row in sol)
    for c in range(3):
        assert [row[c] for row in sol].count("F") <= 1
    # a lone house in a 1 x 1 grid cannot be served
    assert place_flowers(["H"]) is None
    assert count_solutions(["00", "00"], True) == 2
    assert place_flowers(["000", "000"], True) is None
```

NOTES FROM CANDIDATES

This is an N-Queens-style DFS / backtracking problem with extra local constraints around houses.
Track occupied rows and columns, then validate the four-neighbor requirement either incrementally or at the leaf.
The two variants differ on at most one vs exactly one flower per row/column; state the assumption before coding.

PREPARATION

Practice N-Queens with row/column sets and a recursive placement order.
Drill grid-neighbor helpers and constraint pruning before writing the full DFS.
Prepare the boolean version first, then generalize to collect all solutions.

STUDY HERE: topic 8, Strings, Parsing and Recursion | main topic: Parsing

EASY | PracHub | Technical Screen | 12 of 12

Implement string formatting and request tracking tasks

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 16.5 | 30 DAYS 0 | PRIORITY 159/228

ABOUT THIS QUESTION

Topic: Strings, Parsing and Recursion: Parsing

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

LeetCode match: LC 246 Strobogrammatic Number (+ formatting and tracking subtasks) (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-05-21)

Original: https://prachub.com/interview-questions/implement-string-formatting-and-request-tracking-tasks

QUESTION

Overview: This multi-part question evaluates string manipulation, numeric formatting, character-mapping validation under 180° rotation, and event-tracking/order management competencies, assessing parsing, edge-case handling, mapping validation, and design of data structures for ordered output.

APPROACH

Multi-part: numeric/string formatting, validating a number that reads the same after a 180 degree rotation (digit map 0-0, 1-1, 6-9, 8-8, 9-6, two-pointer check), and an ordered request tracker (dict + insertion order). Solve each part independently.

INTERVIEW-DAY PLAN

CLARIFY FIRST

This is four small tasks, so confirm each contract. For the rotation task ask what the rotation map contains and whether leading zeros are allowed. For the tracker ask what 'print in order' means when a later request can still have an earlier end time.

BRUTE FORCE TO SAY OUT LOUD

For each part write the direct version first: split and join, loop with commas, check both readings, and print immediately.

OPTIMAL IDEA

Snake to camel: split on underscores, drop empty parts, capitalize all but the first. Thousands separators: format the absolute value in groups of three from the right and restore the sign. OCR check: validate as-is and after reversing and mapping each character, reject unmapped characters. Tracker: a map for the first timestamp, a min-heap of completed (end, id) and a drain with a watermark.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

snake: leading, trailing and repeated underscores. Commas: 0, negative, 64-bit minimum. OCR: unmapped letter, value 201, value 0. Tracker: end before begin, equal end times.

LIKELY FOLLOW-UPS

Locale-aware formatting, rotation maps given at runtime, and bounded memory in the tracker.

PITFALLS

Negating the minimum 64-bit integer. Printing a request before it is safe to order.

HOW TO THINK ABOUT IT

Four independent tasks, so solve and test each separately and say the contract for each before coding: what an empty or negative input returns, what 'valid' means for the rotated number, and for the tracker when it is safe to print in order. For the tracker, completion is the second event of a pair, and order printing is a heap that is drained only below a watermark.

Basis: Four small tasks: snake_case to camelCase, thousands separators, a rotated-OCR check (LeetCode 246 / 1056 flavor), and a request begin/end tracker that prints in end-time order.

BRUTE FORCE · Task A by walking characters

BRUTE FORCE · PSEUDO-CODE

```
def to_camel(s):
    result, up = "", False
    for ch in s:
        if ch == "_":
            up = True
        else:
            result += ch.upper() if up else ch
            up = False
    return result
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Linear and fine; the split-based version is shorter. The only care is leading and repeated underscores.

BRUTE FORCE · Task B with the language's format specifier

BRUTE FORCE · PSEUDO-CODE

```
def format_number(n):
    return format(n, ",")  # built-in separator
```

TIME	SPACE
O(d)	O(d)

WHY IT IS SLOW

BRUTE FORCE · Task B with the language's format specifier

Correct and O(d), but it hides the algorithm; interviews usually ask for the grouping by hand: peel three digits at a time from the right.

BRUTE FORCE · Task C by checking all three readings with a straightforward helper

BRUTE FORCE · PSEUDO-CODE

```
def is_valid_rotation(text):
    if in_range(text):  # a number 1..200
        return True
    if not all(ch in ROTATE for ch in text):
        return False
    turned = [ROTATE[c] for c in reversed(text)]
    return in_range("".join(turned))
```

TIME	SPACE
O(L)	O(L)

WHY IT IS SLOW

It is already linear; there is little to speed up. The risk is correctness: unmapped characters make the rotation invalid, and leading zeros are not valid numbers.

BRUTE FORCE · Task D by storing all events and sorting when asked

BRUTE FORCE · PSEUDO-CODE

```
begin, end = {}, {}
def on_begin(rid, t):
    begin[rid] = t
def on_end(rid, t):
    end[rid] = t
def print_finished():
    done = [r for r in end if r in begin]
    done.sort(key=lambda r: (end[r], r))
    for r in done:
        print(r, begin[r], end[r])
        del begin[r], end[r]
```

TIME	SPACE
O(n log n) per print	O(n)

WHY IT IS SLOW

Each print sorts everything pending, O(n log n); a min-heap of completed requests prints in order with O(log n) per event, releasing only entries below a safe watermark.

OPTIMAL SOLUTION · Task A: snake_case to camelCase

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task B: thousands separators

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task C: valid as-is or after a 180 degree rotation

TIME	SPACE
O(L)	O(L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task D: request tracker that prints completed requests in end-time order

TIME	SPACE
O(log n) per event	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why not print as soon as a request completes in the follow-up?

Because another request that completes later can still have an earlier end time. An ordered printer needs a contract for when it is safe to emit: either a watermark (everything with end <= W is final because ends arrive within some known lateness), or a flush at the end of input. State that contract, then the heap pops only below the watermark.

FOLLOW-UP 2 How do you handle bad input (duplicate begin, end before begin, never-completed requests)?

Duplicate begins overwrite or are rejected by policy; an end timestamp earlier than its begin is flagged as invalid rather than printed. Requests that stay in the pending map are the leak to watch: evict by age with a timer or a second heap keyed on first-seen time, and report them as incomplete.

FOLLOW-UP 3 Rotation map provided at runtime with extra pairs.

Nothing changes: the map is a parameter. Validate that it is consistent (rot of rot is the original) if you control it, and treat characters that are not in it as invalid.

FOLLOW-UP 4 The tracker must bound its memory.

Evict requests whose begin or end has been waiting longer than a retention window (a second heap keyed by first-seen time) and report them as incomplete. **Cost:** O(log n) per event for the eviction heap; memory O(requests inside the retention window).

FOLLOW-UP 5 Another locale: a different separator or grouping (Indian style 12,34,567).

Parameterise the separator and the group size: the lowest group may differ from the others (3 digits, then groups of 2 in the Indian system). The loop peels a group from the right each time.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement string formatting and request
# tracking tasks
#
# This multi-part question evaluates string manipulation,
# numeric formatting, character-mapping validation under
# 180° rotation, and event-tracking/order management
# competencies, assessing parsing, edge-case handling,
# mapping validation, and design of data structures for
# ordered output.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why not print as soon as a request completes in the
# follow-up?
# 2. How do you handle bad input (duplicate begin, end
# before begin, never-completed requests)?
# 3. Rotation map provided at runtime with extra pairs.
# 4. The tracker must bound its memory.
# 5. Another locale: a different separator or grouping
# (Indian style 12,34,567).
#
# INTERVIEW NOTES
# Four independent tasks, so solve and test each
# separately and say the contract for each before coding:
# what an empty or negative input returns, what 'valid'
# means for the rotated number, and for the tracker when
# it is safe to print in order. For the tracker,
# completion is the second event of a pair, and order
# printing is a heap that is drained only below a
# watermark.
#
# BRUTE FORCE (say it first): Task A by walking characters
# Walk the identifier, set a flag after each underscore,
# and uppercase the next letter. It ignores leading
# underscores by construction when the flag is applied to
# the first output letter, so handle that case explicitly.
# Time O(n). Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Task A by walking characters
# | time O(n)
def snake_to_camel_walk(s):
    out, up, started = [], False, False
    for ch in s:
        if ch == "_":
            up = started
        else:
            out.append(ch.upper() if up else ch)
            up, started = False, True
    return "".join(out)


# HELPERS for the base question
ROTATE = {"0": "0", "1": "1", "6": "9",
          "8": "8", "9": "6"}


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def snake_to_camel(s):
    # foo_bar -> fooBar (empty parts are ignored)
    words = [w for w in s.split("_") if w]
    if not words:
        return ""
    rest = [w.capitalize() for w in words[1:]]
    return words[0] + "".join(rest)


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
# FOLLOW-UP 5: Another locale: a different separator or
# grouping (Indian style 12,34,567).
# Answer: Parameterise the separator and the group size:
# the lowest group may differ from the others (3 digits,
# then groups of 2 in the Indian system). The loop peels a
# group from the right each time.
def format_number(n, sep=",", sizes=(3,)):
    # FOLLOW-UP 5: sep= and sizes=(3, 2) for other locales
    # group the digits from the right: 1234567 ->
    # 1,234,567.
    # sizes = (3, 2) is the Indian style 12,34,567: the
    # first
    # group has 3 digits, every later one has 2.
    sign = "-" if n < 0 else ""
    digits = str(abs(n))
    parts, i = [], 0
    while len(digits) > sizes[min(i, len(sizes) - 1)]:
        size = sizes[min(i, len(sizes) - 1)]
        # peel one group off the right end
        parts.append(digits[-size:])
        digits = digits[:-size]
        i += 1
    parts.append(digits)
    return sign + sep.join(reversed(parts))


# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
def is_rotation_valid(s, mapping=ROTATE):
    # does the number read the same upside down (180
    # degrees)?
    # a character not in the map makes it invalid
    if any(ch not in mapping for ch in s):
        return False
    # rotate every digit and read the result backwards
    turned = "".join(mapping[ch] for ch in reversed(s))
    return turned == s
```

```python
# BASE QUESTION (optimal): snake_case to camelCase
# Idea: Split on '_' and drop empty parts, which handles
# leading, trailing and repeated underscores in one move.
# The first word stays lowercase and every later word is
# capitalized.
class RequestTracker:
    """begin(id, t) and end(id, t). Finished requests are
    printed in the order they BEGAN: one is printed only
    when everything that began before it is done."""

    def __init__(self):
        self.order = deque()
        self.begin_at, self.end_at = {}, {}

    def begin(self, rid, t):
        if rid in self.begin_at:
            raise ValueError("duplicate begin")
        self.begin_at[rid] = t
        self.order.append(rid)

    def end(self, rid, t):
        if rid not in self.begin_at:
            raise ValueError("end without begin")
        if t < self.begin_at[rid]:
            raise ValueError("ends before it begins")
        self.end_at[rid] = t
        # print from the front while the oldest are
        # finished
        out = []
        while self.order and self.order[0] in self.end_at:
            first = self.order.popleft()
            out.append((first, self.begin_at[first],
                        self.end_at[first]))
        return out


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("__a__b_") == "aB"
    assert snake_to_camel("") == ""
    assert format_number(1234567) == "1,234,567"
    assert format_number(-1000) == "-1,000"
    assert format_number(999) == "999"
    assert format_number(0) == "0"
    indian = format_number(1234567, sizes=(3, 2))
    assert indian == "12,34,567"
    assert format_number(1234, ".") == "1.234"
    assert is_rotation_valid("69")
    assert is_rotation_valid("818")
    assert not is_rotation_valid("962")
    assert not is_rotation_valid("12")
    assert is_rotation_valid("") is True
    t = RequestTracker()
    t.begin("a", 1)
    t.begin("b", 2)
    # b finished first, but a has not: nothing can be
    # printed
    assert t.end("b", 5) == []
    # a finishes: both are printed, in begin order
    assert t.end("a", 7) == [("a", 1, 7), ("b", 2, 5)]
    again = lambda: t.begin("a", 9)
    orphan = lambda: t.end("zz", 1)
    for call in (again, orphan):
        try:
            call()
            raise AssertionError("should fail")
        except ValueError:
            pass
```

FULL QUESTION TEXT FROM THE SITE

You are given several small coding tasks. Implement each as described.

Task A — Convert snake_case to camelCase

Write a function that converts an identifier from snake_case to camelCase.

Input: a string s containing lowercase letters a-z and underscores _ .
Output: the camelCase version.
Rules/edge cases:
Consecutive underscores should be treated as a single word separator (ignore empty parts).
If s starts/ends with underscores, ignore those underscores.
The first word remains lowercase; each subsequent word is capitalized.

Example:

"hello_world" -> "helloWorld"
"__my__var__name_" -> "myVarName"
Task B — Format an integer with thousands separators

Write a function that formats a (possibly negative) integer with commas as thousands separators.

Input: an integer n (may be large; assume it fits in 64-bit signed integer).
Output: a string representation using commas every 3 digits from the right.

Examples:

123 -> "123"
1234 -> "1,234"
-12345678 -> "-12,345,678"
Task C — Validate an OCR string under 180° rotation

An OCR system outputs a string t that may contain digits and a few letters. You need to decide whether it can represent an integer between 1 and 200 inclusive under either normal reading or after a 180-degree rotation.

Define a rotation mapping rot(c) for characters; rotation is valid only if every character in the string has a mapping. Rotating a whole string means:

reverse the string, and
replace each character c with rot(c) .

Return true if either:

t as-is is a valid base-10 integer in [1, 200] , OR
rotate180(t) is a valid base-10 integer in [1, 200] .

Assume the rotation map is provided as a dictionary (examples from the interview note included):

'6' <-> '9'
plus any additional pairs given at runtime (e.g., 'h' -> '4' , 'L' -> '7'). Characters not in the map are invalid for rotation.

Examples (assuming map includes 6<->9, 0->0, 1->1, 8->8):

FULL QUESTION TEXT FROM THE SITE

"69" rotates to "69" and is valid (69 in range) -> true
"201" is out of range, rotate may or may not help -> depends on mapping -> compute
Task D — Track request begin/end times and print in a required order

You are implementing a Server class that receives events for requests. Each request has a unique requestId.

You must implement two methods:

begin(requestId, timestamp)
end(requestId, timestamp)
Part 1

Whenever both timestamps for a request are known, print/log a line:

requestId begin=<beginTs> end=<endTs>

Assumptions:

Events may arrive out of order (an end can arrive before begin). There may be multiple in-flight requests.
Follow-up

Instead of printing immediately when a pair is complete, the server must always print completed requests in increasing order of end time (earliest end printed first). If multiple have the same end time, break ties by requestId.

Design the data structures and logic to support this ordering efficiently.

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 9 OF 10 | LESS COMMON TOPIC

Greedy and Math

10 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

4	6
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Greedy and Math | 10 questions | tap a title to jump to it

MATH AND BITS (9)

M	1. Find the Largest Monotone Increasing Number	PracHub	2026-07-04
M	2. Choose optimal guesses for green-only Wordle	PracHub	2025-10-14
M	3. Trip Expense Settlement	HackTheRounds	no date
H	4. Check Digits and Combine Ranges	PracHub	2026-03-29
H	5. Find secret word with match-count feedback	PracHub	2026-02-11
H	6. Wordle-Style 5-Letter Minimum-Guess Strategy	1Point3Acres	2025-12-15
H	7. Shuffle array using random integer API	PracHub	2025-12-08
H	8. Find maximum value in huge text file	PracHub	2025-12-08
H	9. Find All Bad Pairs Using runTest Oracle	1Point3Acres	2025-11-27

GREEDY (1)

M	10. Min boards to cover roof holes	PracHub	2026-02-12

STUDY HERE: topic 9, Greedy and Math | main topic: Math and bits

MEDIUM | PracHub | Technical Screen | 1 of 10

Find the Largest Monotone Increasing Number

POSTED 2026-07-04 | 108 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 5** **30 DAYS 0** **PRIORITY 171/228**

ABOUT THIS QUESTION

Topic: Greedy and Math: Math and bits

Optimal answer from: the interview site's own solution

LeetCode match: LC 738 Monotone Increasing Digits (inferred from overview)

Posted on PracHub: 2026-07-04 (updated 2026-07-15)

Original: https://prachub.com/coding-questions/find-the-largest-monotone-increasing-number

QUESTION

Overview: Find the largest number no greater than n whose decimal digits are monotone increasing. Develop a linear-time greedy correction that revisits earlier decreasing boundaries and fills the remaining suffix optimally without introducing leading zeros.

APPROACH

Scan digits right to left; when digit[i-1] > digit[i], decrement digit[i-1] and mark position i as the point to fill with 9s. O(d).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether monotone means non-decreasing (equal digits allowed) or strictly increasing, whether n can be 0, and whether n is given as an integer or a very long string.

BRUTE FORCE TO SAY OUT LOUD

Decrement n until its digits are monotone. Correct but up to O(n * d).

OPTIMAL IDEA

Scan the digits from right to left. When digit[i - 1] > digit[i], decrease digit[i - 1] by one and remember i as the first position to fill with 9s. After the scan set every digit from that marker to the end to 9. O(d) time.

DRY-RUN THESE CASES

10 gives 9, 332 gives 299, 1234 stays 1234, 100 gives 99, 0 gives 0, 1000 gives 999.

LIKELY FOLLOW-UPS

Strictly increasing digits variant, or return the largest number below n when n itself is already valid.

PITFALLS

INTERVIEW-DAY PLAN

Scanning left to right, which misses the cascading decrement. Setting 9s from the wrong index.

HOW TO THINK ABOUT IT

If a digit is smaller than the one before it, the earlier digit has to drop by one and everything after that point can become 9 (as large as possible). Dropping a digit can break the pair before it, so scan from the right: each repair is checked against the digit to its left. That is why one right-to-left pass is enough.

Basis: LeetCode 738, Monotone Increasing Digits (non-decreasing digits).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    while not is_non_decreasing(str(n)):
        n -= 1
    return n
```

TIME	SPACE
O(n * d)	O(1)

WHY IT IS SLOW

n can be 10^9, so it may take a billion steps. A right-to-left scan that lowers a digit where order breaks and fills the rest with 9s is O(digits).

OPTIMAL SOLUTION · Right-to-left scan, then fill with nines

TIME	SPACE
O(d)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The number is given as a string with 100,000 digits.

The scan works directly on the digit list, so keep it as a list of characters and never convert to an integer until the end (or not at all). The algorithm is O(d).

FOLLOW-UP 2 Return the smallest monotone number that is at least n instead.

Scan left to right for the first digit that is larger than its right neighbour and copy that digit over the whole suffix (332 becomes 333). The prefix before it is already monotone.

FOLLOW-UP 3 Digits must be strictly increasing.

Same idea, but the suffix to fill is the largest strictly increasing run (for example ...6789), and you must check that the prefix leaves enough room. Say the extra care and write the check as a brute-force comparison first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find the Largest Monotone Increasing
# Number
#
# Find the largest number no greater than n whose decimal
# digits are monotone increasing. Develop a linear-time
# greedy correction that revisits earlier decreasing
# boundaries and fills the remaining suffix optimally
# without introducing leading zeros.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The number is given as a string with 100,000 digits.
# 2. Return the smallest monotone number that is at least
# n instead.
# 3. Digits must be strictly increasing.
#
# INTERVIEW NOTES
# If a digit is smaller than the one before it, the
# earlier digit has to drop by one and everything after
# that point can become 9 (as large as possible). Dropping
# a digit can break the pair before it, so scan from the
# right: each repair is checked against the digit to its
# left. That is why one right-to-left pass is enough.
#
# BRUTE FORCE (say it first): Decrement n until its digits
# are monotone
# Starting at n, subtract one until every digit is at
# least the one before it. Obviously correct, but n can be
# 10^9, so it can take a billion steps. Time O(n * d).
# Space O(1).


# BRUTE FORCE (say it first): Decrement n until its digits
# are monotone | time O(n * d)
def _is_monotone(n):
    s = str(n)
    return all(a <= b for a, b in zip(s, s[1:]))


def monotone_brute(n):
    while not _is_monotone(n):
        n -= 1
    return n


# BASE QUESTION (optimal): Right-to-left scan, then fill
# with nines
# Idea: Walk i from the last digit to the first. When
# digit[i - 1] > digit[i], decrease digit[i - 1] by one
# and remember i as the start of the nines.
def largest_monotone(n):
    # largest number <= n whose digits never decrease
    d = list(str(n))
    mark = len(d)
    # scan from the right; a drop forces the left digit
    # down by
    # one and everything after it up to 9
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = str(int(d[i - 1]) - 1)
            mark = i
    for i in range(mark, len(d)):
        d[i] = "9"
    return int("".join(d))


# FOLLOW-UP 1: The number is given as a string with
# 100,000 digits.
# Answer: The scan works directly on the digit list, so
# keep it as a list of characters and never convert to an
# integer until the end (or not at all). The algorithm is
# O(d).
def largest_monotone_string(digits):
    # the same on a string of any length (no big integers)
    d = list(digits)
    mark = len(d)
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = chr(ord(d[i - 1]) - 1)
            mark = i
    d[mark:] = "9" * (len(d) - mark)
    # a leading zero (from 10 -> 09) is dropped
    return "".join(d).lstrip("0") or "0"


# FOLLOW-UP 2: Return the smallest monotone number that is
# at least n instead.
# Answer: Scan left to right for the first digit that is
# larger than its right neighbour and copy that digit over
# the whole suffix (332 becomes 333). The prefix before it
# is already monotone.
def smallest_monotone_at_least(n):
    # follow-up: smallest monotone number >= n
    d = list(str(n))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            # copy the bigger digit over the rest: 332 ->
            # 333
            d[i + 1:] = d[i] * (len(d) - i - 1)
            break
    return int("".join(d))


# HELPERS for the checks below
def is_monotone(n):
    s = str(n)
    return all(a <= b for a, b in zip(s, s[1:]))


if __name__ == "__main__":
    assert largest_monotone(10) == 9
    assert largest_monotone(1234) == 1234
    assert largest_monotone(332) == 299
    assert largest_monotone(0) == 0
    assert largest_monotone_string("332") == "299"
    assert largest_monotone_string("10") == "9"
    assert smallest_monotone_at_least(332) == 333
    assert smallest_monotone_at_least(120) == 122
    # check against brute force for many numbers
    for n in range(0, 3000):
        every = range(n + 1)
        want = max(x for x in every if is_monotone(x))
        assert largest_monotone(n) == want
        text = largest_monotone_string(str(n))
        assert text == str(want)
        later = range(n, 99999)
        up = next(x for x in later if is_monotone(x))
        assert smallest_monotone_at_least(n) == up
    # a 100,000 digit input is fine as a string
```

```
    big = "9" * 50000 + "1" * 50000
    assert largest_monotone_string(big)[:3] == "899"
```

FUNCTION AND TEST CASES

Sample call

```
largest_monotone_increasing(0)
```

Test cases

Input	Expected
(0,)	0
(9,)	9

STUDY HERE: topic 9, Greedy and Math | main topic: Math and bits

MEDIUM | PracHub | Technical Screen | 2 of 10

Choose optimal guesses for green-only Wordle

POSTED 2025-10-14 | 371 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 42.2 | 30 DAYS 0 | PRIORITY 86/228

ABOUT THIS QUESTION

Topic: Greedy and Math: Math and bits

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #843 Guess the Word

LeetCode match: No direct match; related LC 843 Guess the Word (inferred from overview)

Posted on PracHub: 2025-10-14 (updated 2026-03-29)

Original: https://prachub.com/interview-questions/choose-optimal-guesses-for-green-only-wordle

QUESTION

Overview: This question evaluates probabilistic reasoning, expected-value optimization, information partitioning, and algorithmic efficiency in selecting guesses under partial (green-only) feedback; it belongs to the Coding & Algorithms domain, specifically search and probabilistic modeling, and emphasizes practical algorithm design and implementation rather than purely theoretical concepts. It is commonly asked to assess a candidate's ability to model uncertainty with weighted distributions, minimize the expected remaining search space, and manage time/space trade-offs for inputs up to several thousand words.

APPROACH

Filter candidates by green-position constraints, then pick the guess that maximizes expected information (partition sizes) over the remaining words.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what feedback exists (only green: letter in the right position), whether answers are equally likely or weighted, whether guesses must come from the candidate list, and the objective (minimize the expected size of the remaining set).

BRUTE FORCE TO SAY OUT LOUD

For each candidate guess and each possible answer compute the feedback and count the survivors. O(W^2 * L).

OPTIMAL IDEA

A guess g splits the candidates by its pattern against each answer, a bitmask of which positions are green. If groups have sizes s_1 ... s_k, the expected number of survivors is the sum of s_i^2 divided by W (weights replace sizes with weight sums). Pick the guess minimizing it. To speed up, store for each (position, letter) a bitset of candidate words so groups come from bit operations instead of pairwise comparisons.

DRY-RUN THESE CASES

Two candidates (any guess), words that differ in one position, a guess that is itself the answer (pattern all green), a single candidate.

LIKELY FOLLOW-UPS

Full Wordle feedback with yellow, multiple rounds, or an adaptive strategy.

PITFALLS

Minimizing the number of groups instead of the expected survivors. Recomputing the filter from scratch each round.

HOW TO THINK ABOUT IT

A guess splits the candidates into groups by which positions turned green. If the groups have sizes s_1 ... s_k out of W candidates, the expected size of what remains is the sum of s_i squared divided by W (a candidate falls in a group of size s with probability s / W, and then s remain). Pick the guess with the smallest value. To make it fast, store for each (position, letter) the set of words with that letter there as a bitset, so the groups come from a few AND operations.

Basis: Written. Assumed rules (confirm them): words of equal length, the answer is uniformly random among the candidates, a guess must be a candidate, feedback marks only the positions where the guess letter equals the answer letter (green). Goal: minimise the expected number of candidates left.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(guesses, answers):
    best, best_score = None, INF
    for g in guesses:
        groups = Counter()
        for a in answers:
            groups[green_pattern(g, a)] += 1
        sizes = groups.values()
        score = sum(c * c for c in sizes)
        score /= len(answers)
        if score < best_score:
            best, best_score = g, score
    return best
```

TIME	SPACE
O(W^2 * L)	O(W)

WHY IT IS SLOW

BRUTE FORCE

Compares every pair of words, O(W^2 * L). Bitsets per (position, letter) split the candidates into pattern groups with a few AND operations.

OPTIMAL SOLUTION · Bitsets per (position, letter), splitting groups with AND

TIME	SPACE
O(W * L * groups) with bit-parallel work per group	**O(W * L)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Play a whole game, using the feedback after each guess.

Keep the candidates that would have produced exactly the observed green pattern (filter_candidates), then choose the best guess among the remaining ones. Each round at least removes the guess itself, so the game ends.

FOLLOW-UP 2 Answers are not equally likely (weights).

Replace group sizes by group weights: score = sum(weight(group)^2) / total weight. The grouping step is unchanged; use sums of weights instead of popcounts.

FOLLOW-UP 3 Real Wordle also gives yellow feedback.

The pattern becomes three-valued per position (green, yellow, grey) with the usual duplicate-letter rules, so the pattern space is 3^L; bitsets still work if you precompute patterns, but the computation is O(W^2) pattern evaluations for a full table.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Choose optimal guesses for green-only
# Wordle
#
# This question evaluates probabilistic reasoning,
# expected-value optimization, information partitioning,
# and algorithmic efficiency in selecting guesses under
# partial (green-only) feedback; it belongs to the Coding
# & Algorithms domain, specifically search and
# probabilistic modeling, and emphasizes practical
# algorithm design and implementation rather than purely
# theoretical concepts. It is commonly asked to assess a
# candidate's ability to model uncertainty with weighted
# distributions, minimize the expected remaining search
# space, and manage time/space trade-offs for inputs up to
# several thousand words.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Play a whole game, using the feedback after each
# guess.
# 2. Answers are not equally likely (weights).
# 3. Real Wordle also gives yellow feedback.
#
# INTERVIEW NOTES
# A guess splits the candidates into groups by which
# positions turned green. If the groups have sizes s_1 ...
# s_k out of W candidates, the expected size of what
# remains is the sum of s_i squared divided by W (a
# candidate falls in a group of size s with probability s
# / W, and then s remain). Pick the guess with the
# smallest value. To make it fast, store for each
# (position, letter) the set of words with that letter
# there as a bitset, so the groups come from a few AND
# operations.
#
# BRUTE FORCE (say it first): Compute the feedback of
# every guess against every answer
# For each guess compute its green pattern against each
# candidate answer, count the sizes of the pattern groups,
# and evaluate sum(s^2) / W. Correct and the oracle; it
# compares every pair of words. Time O(W^2 * L). Space
# O(W).


from collections import Counter
from collections import defaultdict
```

```
from fractions import Fraction


# HELPERS for the brute force
def green(guess, answer):
    # green-only feedback: which positions match exactly
    return tuple(g == a for g, a in zip(guess, answer))


def partition(guess, words):
    # group the possible answers by the feedback they
    # would give
    groups = defaultdict(list)
    for w in words:
        groups[green(guess, w)].append(w)
    return list(groups.values())


def score(guess, words, weight=None):
    # expected number of answers left after this guess:
    # sum(group^2) / total, with optional per-word weights
    weight = weight or (lambda w: 1)
    total = sum(weight(w) for w in words)
    spread = sum(sum(weight(w) for w in g) ** 2
                 for g in partition(guess, words))
    return Fraction(spread, total)


# BRUTE FORCE (say it first): Compute the feedback of
# every guess against every answer | time O(W^2 * L)
def _green_mask(guess, answer):
    return tuple(g == a for g, a in zip(guess, answer))


def best_guess_brute(words):
    best = None
    for g in words:
        groups = Counter(
            _green_mask(g, a) for a in words
        )
        score = Fraction(
            sum(s * s for s in groups.values()),
            len(words),
        )
        if best is None or (score, g) < best:
```

```
            best = (score, g)
    return best[1]


# BASE QUESTION (optimal): Bitsets per (position, letter),
# splitting groups with AND
# Idea: Index the words: bits[pos][letter] is an integer
# whose bit i is set when word i has that letter at pos.
def best_guess(words, weight=None):
    # the guess that leaves the fewest answers on average
    # (ties: alphabetical, so the result is deterministic)
    def rank(g):
        return (score(g, words, weight), g)
    return min(words, key=rank)


# FASTER: bitsets
def best_guess_bitsets(words):
    # faster: one bitmask per (position, letter); each
    # position
    # splits every group in two with two AND operations
    W = len(words)
    bits = [defaultdict(int)
            for _ in range(len(words[0]))]
    for i, w in enumerate(words):
        for pos, ch in enumerate(w):
            bits[pos][ch] |= 1 << i
    best = None
    for g in words:
        groups = [(1 << W) - 1]
        for pos, ch in enumerate(g):
            mask, nxt = bits[pos][ch], []
            for grp in groups:
                for part in (grp & mask, grp & ~mask):
                    if part:
                        nxt.append(part)
            groups = nxt
        sizes = [bin(g).count("1") for g in groups]
        spread = sum(s * s for s in sizes)
        cand = (Fraction(spread, W), g)
        if best is None or cand < best:
            best = cand
    return best[1]
```

```
# FOLLOW-UP 1: Play a whole game, using the feedback after
# each guess.
# Answer: Keep the candidates that would have produced
# exactly the observed green pattern (filter_candidates),
# then choose the best guess among the remaining ones.
# Each round at least removes the guess itself, so the
# game ends.
def filter_candidates(words, guess, pattern):
    # keep only answers that would give the observed
    # feedback
    return [w for w in words
            if green(guess, w) == pattern]


# HELPERS for the checks below
def play(words, secret):
    # a whole game: guess, read the feedback, narrow down
    candidates, guesses = list(words), []
    while True:
        guess = best_guess(candidates)
        guesses.append(guess)
        if guess == secret:
            return guesses
        pattern = green(guess, secret)
        candidates = filter_candidates(
            candidates, guess, pattern)


if __name__ == "__main__":
```

```
    words = ["crane", "crate", "grape", "grade", "trade",
             "slate", "plate"]
    assert best_guess(words) == best_guess_bitsets(words)
    # a guess can never leave more than all the words
    assert score("crane", words) <= len(words)
    for secret in words:
        path = play(words, secret)
        assert path[-1] == secret
        # each round removes at least the guess itself
        assert len(path) <= len(words)
    # weights: a very likely word makes its guess more
    # attractive
    heavy = lambda w: 100 if w == "slate" else 1
    slate = score("slate", words, heavy)
    crane = score("crane", words, heavy)
    assert slate < crane
```

```
    assert partition("crane", ["crane"]) == [["crane"]]
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are playing a simplified 5-letter word guessing game.

There is a hidden secret word of length 5 .
You have a dictionary of valid 5-letter words. Each word may also have an associated frequency/weight representing how likely it is to be the secret.
When you guess a 5-letter word, the game returns only “green” information (correct letter in the correct position). There is no information about letters that exist in the word but are in the wrong position (“yellow”), and no explicit info about letters that are absent.
Feedback definition

For a guess g and secret s, define the feedback as a 5-bit mask m where:

m[i] = 1 iff g[i] == s[i]
otherwise m[i] = 0

Example: g="crane", s="cared" -> mask is 1 0 0 0 0 (only position 0 matches).

Task

Given:

words : a list of N valid 5-letter words
weight[w] (optional): a positive number for each word (if omitted, assume uniform)
candidates : the current set of possible secret words consistent with feedback so far (initially all words)

Write an algorithm/function to choose the next guess that is optimal under the following objective:

Choose a guess word g (typically from words) that minimizes the expected size of the remaining candidate set after receiving the green-only mask, where the expectation is taken over the secret word drawn from candidates according to weight .

Formally, if C is the current candidate set and C_m(g) is the subset of candidates that would produce mask m for guess g, choose g that minimizes:

FULL QUESTION TEXT FROM THE SITE

E
[
|
C
m
a
s
k
(
g
)
|
]
=
sum
m
in
{
0
,
1
}
5
P
(

m
a
s
k
=
m
)
*
|
C
m
(
g
)
|
E[|C
mask

(g)|]=sum
min{0,1}
5

P(mask=m)*|C
m

(g)|

FULL QUESTION TEXT FROM THE SITE

where
P
(
m
a
s
k
=
m
)
P(mask=m) is induced by the weighted distribution over secrets in C.

Input/Output expectations
Input sizes: 1 <= N <= 5,000 (you may propose optimizations if needed), word length is always 5.
Output: the selected next guess word.
Follow-ups (optional)
How would you incorporate non-uniform word frequencies?
How would you scale if N is very large (e.g., 100k)?
How would you change the objective to minimize expected number of guesses instead of expected remaining candidates?

ALSO REPORTED · 1 more

Wordle-Style 5-Letter Minimum-Guess Strategy | 1Point3Acres | phone-screen, onsite-coding | asked 2025-12-15 | Single report

https://www.1point3acres.com/interview/problems/company/google/wordle-min-guess-strategy

Without the dictionary, query positions in order; per position, query letters in descending frequency (e, t, a, o, i, n, ...) and stop as soon as one returns true.
With the dictionary, maintain a candidate set; at each step pick the (position, letter) query whose true/false outcome splits the candidate set most evenly (entropy maximization).
Mention information theory: each query yields <= 1 bit, so the absolute lower bound on calls is ceil(log2(|candidates|)).
Don't waste minutes on a closed-form optimum — the interviewer mostly wants you to articulate the entropy heuristic and pick one reasonable scoring function.

Preparation: Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

STUDY HERE: topic 9, Greedy and Math | main topic: Math and bits

MEDIUM | IN APP LC 465 | HackTheRounds | Onsite | 3 of 10

Trip Expense Settlement

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 140/228

ABOUT THIS QUESTION

Topic: Greedy and Math: Math and bits

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #465 Optimal Account Balancing

LeetCode match: LC 465 (Optimal Account Balancing)

QUESTION

A group of friends go on a trip. Each friend may pay for others. Given payment records, compute minimal payments required to settle all debts.

EXAMPLES

Example 1

Input	`[{"payer": "alice", "amount": 4000, "payees": ["bob", "jess", "alice", "sam"]}, {"payer": "jess", "amount": 2000, "payees": ["jess", "alice"]}]`
Output	`[{"payer": "bob", "amount": 1000, "payee": "alice"}, {"payer": "sam", "amount": 1000, "payee": "alice"}]`
Why	`Calculate net balance for each person, then settle with minimum transactions.`

Example 2

Input	`expenses = [{"payer": "Alice", "amount": 40, "participants": ["Alice", "Bob"]}, {"payer": "Bob", "amount": 20, "participants": ["Alice", "Bob"]}]`
Output	`["Bob pays Alice 10.00"]`

CONSTRAINTS

- 1 <= payments.length <= 1000
- 1 <= amount <= 10^6
- 1 <= payees.length <= 100

APPROACH

Compute each person's net balance, then match debtors and creditors with heaps

HOW TO THINK ABOUT IT

First reduce the records to one number per person: net balance = paid - owed. Only that matters; who owed whom originally is irrelevant. Then settling is moving money between people with positive and negative balances. Matching largest creditor with largest debtor is quick and uses at most n - 1 payments, but only searching groups of people who sum to zero gives the true minimum.

Basis: LeetCode 465, Optimal Account Balancing. Each record is split equally among the listed payees (the payer may be one of them).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def settle(i):
    while i < n and net[i] == 0:
        i += 1  # already settled
    if i == n:
        return 0
    best = INF
    for j in range(i + 1, n):
        if net[j] * net[i] < 0:  # opposite sign
            net[j] += net[i]  # move balance i
            best = min(best, 1 + settle(i + 1))
            net[j] -= net[i]  # undo
    return best
```

TIME	SPACE
O(n!)	O(n)
worst case	

WHY IT IS SLOW

It tries every pairing, up to n! in the worst case. Matching the largest creditor with the largest debtor (heaps) is O(n log n) and uses at most n - 1 payments, though not always the minimum.

OPTIMAL SOLUTION · Heaps: match the largest debtor with the largest creditor

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do you need the true minimum number of payments?

That is NP-hard in general (it is the zero-sum partition problem). The greedy answer is at most n - 1 payments and is usually accepted; for exact answers use the bitmask DP above (about 20 people) or backtracking with pruning.

FOLLOW-UP 2 Amounts that do not divide evenly (cents).

Work in integer cents and decide where the remainder goes (to the payer, or round-robin among payees). Never use floats for money, and make the rule deterministic.

FOLLOW-UP 3 Different shares per person instead of equal split.

Replace the equal share with an explicit amount per payee in each record; the net balance per person is computed the same way.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Trip Expense Settlement
#
# A group of friends go on a trip. Each friend may pay for
# others. Given payment records, compute minimal payments
# required to settle all debts.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Do you need the true minimum number of payments?
# 2. Amounts that do not divide evenly (cents).
# 3. Different shares per person instead of equal split.
#
# INTERVIEW NOTES
# First reduce the records to one number per person: net
# balance = paid - owed. Only that matters; who owed whom
# originally is irrelevant. Then settling is moving money
# between people with positive and negative balances.
# Matching largest creditor with largest debtor is quick
# and uses at most n - 1 payments, but only searching
# groups of people who sum to zero gives the true minimum.
#
# BRUTE FORCE (say it first): Backtracking over who pays
# whom (exact minimum)
# Take the first person with a nonzero balance and try
# settling them with every later person of the opposite
# sign, recursing on the rest; keep the fewest payments.
# This gives the exact minimum, but the search explodes
# with the number of people. Time O(n!) worst case. Space
# O(n).


import heapq
from itertools import combinations


# HELPERS for the brute force
def balances(records):
    # net balance per person: what they paid minus what
    # they
    # owe. Positive: the group owes them. All sum to zero.
    net = {}
    for rec in records:
        payer = rec["payer"]
        net[payer] = net.get(payer, 0) + rec["amount"]
        for person, share in shares_of(rec).items():
            net[person] = net.get(person, 0) - share
    return {p: v for p, v in net.items() if v}


# BRUTE FORCE (say it first): Backtracking over who pays
# whom (exact minimum) | time O(n!) worst case
def settle_min_backtracking(records):
    debts = list(balances(records).values())

    def go(i):
        while i < len(debts) and debts[i] == 0:
            i += 1
        if i == len(debts):
            return 0
        best = float("inf")
        for j in range(i + 1, len(debts)):
            if debts[i] * debts[j] < 0:
                debts[j] += debts[i]
                best = min(best, 1 + go(i + 1))
                debts[j] -= debts[i]
        return best

    return go(0)


# BASE QUESTION (optimal): Heaps: match the largest debtor
# with the largest creditor
# Idea: Push creditors (positive net) in a max-heap and
# debtors (negative net) in another. Repeatedly pay
# min(debt, credit) from the biggest debtor to the biggest
# creditor, and push back whoever still has a remainder.
def settle_greedy(records):
    # the biggest debtor pays the biggest creditor, as
    # much
    # as one side can: at most n - 1 payments
    net = balances(records)
    owed = [(-v, p) for p, v in net.items() if v > 0]
    debt = [(v, p) for p, v in net.items() if v < 0]
    heapq.heapify(owed)
    heapq.heapify(debt)
    out = []
    while owed and debt:
        o_amt, creditor = heapq.heappop(owed)
        d_amt, debtor = heapq.heappop(debt)
        pay = min(-o_amt, -d_amt)
        out.append({"payer": debtor, "amount": pay,
                    "payee": creditor})
        if -o_amt > pay:
            heapq.heappush(owed, (o_amt + pay, creditor))
        if -d_amt > pay:
            heapq.heappush(debt, (d_amt + pay, debtor))
    return out


# FOLLOW-UP 1: Do you need the true minimum number of
# payments?
# Answer: That is NP-hard in general (it is the zero-sum
# partition problem). The greedy answer is at most n - 1
# payments and is usually accepted; for exact answers use
# the bitmask DP above (about 20 people) or backtracking
# with pruning.
def min_payments_exact(values):
    # follow-up: the true minimum. People split into
    # groups
    # that settle among themselves (sum zero); a group of
    # g
    # people needs g - 1 payments. So find the most
    # groups:
    # a DP over subsets of people.
    n = len(values)
    total = [0] * (1 << n)
    best = [0] * (1 << n)
    for mask in range(1, 1 << n):
        low = (mask & -mask).bit_length() - 1
        rest = mask & (mask - 1)
        total[mask] = total[rest] + values[low]
```

```
        top = max((best[mask ^ (1 << i)]
                   for i in range(n) if mask >> i & 1))
        best[mask] = top + (1 if total[mask] == 0 else 0)
    return n - best[(1 << n) - 1]


# FOLLOW-UP 1: Do you need the true minimum number of
# payments?
# Answer: That is NP-hard in general (it is the zero-sum
# partition problem). The greedy answer is at most n - 1
# payments and is usually accepted; for exact answers use
# the bitmask DP above (about 20 people) or backtracking
# with pruning.
def min_payments_search(values):
    # the same answer by backtracking: settle the first
```

```
    # open
    # debt against every opposite one
    values = [v for v in values if v]

    def go(start):
        while start < len(values) and values[start] == 0:
            start += 1
        if start == len(values):
            return 0
        best = float("inf")
        seen = set()
        for j in range(start + 1, len(values)):
            if values[j] * values[start] < 0 and (
                    values[j] not in seen):
                seen.add(values[j])
                values[j] += values[start]
                best = min(best, 1 + go(start + 1))
                values[j] -= values[start]
        return best

    return go(0)


# FOLLOW-UP 2: Amounts that do not divide evenly (cents).
# Answer: Work in integer cents and decide where the
# remainder goes (to the payer, or round-robin among
# payees). Never use floats for money, and make the rule
# deterministic.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def split_equally(amount, people):
    # follow-up: cents that do not divide evenly. Work in
    # whole cents; the leftover cents go to the first
    # people
    # in sorted order, so the rule is always the same.
    people = sorted(people)
    base, extra = divmod(amount, len(people))
    return {p: base + (1 if i < extra else 0)
            for i, p in enumerate(people)}


# FOLLOW-UP 3: Different shares per person instead of
# equal split.
# Answer: Replace the equal share with an explicit amount
# per payee in each record; the net balance per person is
```

```
# computed the same way.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def shares_of(record):
    # follow-up: explicit shares per person instead of
    # equal
    payees = record["payees"]
    if isinstance(payees, dict):
        if sum(payees.values()) != record["amount"]:
            raise ValueError("shares do not add up")
        return payees
    return split_equally(record["amount"], payees)


# HELPERS for the checks below
def apply(payments, net):
    # check: after the payments every balance is zero
    left = dict(net)
    for pay in payments:
        who, whom = pay["payer"], pay["payee"]
        left[who] = left.get(who, 0) + pay["amount"]
        left[whom] = left.get(whom, 0) - pay["amount"]
    return all(v == 0 for v in left.values())


if __name__ == "__main__":
    trip = [{"payer": "alice", "amount": 4000,
             "payees": ["bob", "jess", "alice", "sam"]},
            {"payer": "jess", "amount": 2000,
             "payees": ["jess", "alice"]}]
    got = settle_greedy(trip)
    want = [
        {"payer": "bob", "amount": 1000,
         "payee": "alice"},
        {"payer": "sam", "amount": 1000,
         "payee": "alice"}]
    assert sorted(got, key=lambda p: p["payer"]) == want
    assert balances(trip) == {"alice": 2000, "bob": -1000,
                              "sam": -1000}
    # 10.00 split three ways: 3.34 + 3.33 + 3.33
    assert split_equally(1000, ["c", "a", "b"]) == {
        "a": 334, "b": 333, "c": 333}
    custom = [{"payer": "x", "amount": 90,
               "payees": {"x": 10, "y": 30, "z": 50}}]
    owed = {"x": 80, "y": -30, "z": -50}
```

```
    assert balances(custom) == owed
    assert min_payments_exact([5, -5]) == 1
    assert min_payments_exact([3, -3, 2, -2]) == 2
    assert min_payments_search([3, -3, 2, -2]) == 2
    import random
    for _ in range(100):
        names = "abcdef"
        records = []
        for _ in range(random.randint(1, 5)):
            count = random.randint(1, 4)
            who = random.sample(names, count)
            cents = random.randint(1, 500)
            records.append({"payer": random.choice(names),
                            "amount": cents,
                            "payees": who})
        net = balances(records)
        assert sum(net.values()) == 0
        pays = settle_greedy(records)
        assert apply(pays, net)
        assert len(pays) <= max(len(net) - 1, 0)
        values = list(net.values())
        exact = min_payments_exact(values)
        assert exact == min_payments_search(values)
        assert exact <= len(pays)
```

ALSO REPORTED · 1 more

Compute minimal transfers to settle group expenses | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimal-transfers-to-settle-group-expenses

STUDY HERE: topic 9, Greedy and Math | main topic: Math and bits

HARD | PracHub | Technical Screen | 4 of 10

★ TOP 75: rank 4 in the Medium and Hard priority list

★ Check Digits and Combine Ranges

POSTED 2026-03-29 | 205 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 73.6 | 30 DAYS 63.4 | PRIORITY 2/228

ABOUT THIS QUESTION

Topic: Greedy and Math: Math and bits

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #9 Palindrome Number, LeetCode #56 Merge Intervals, LeetCode #228 Summary Ranges

LeetCode match: LC 9 Palindrome Number + LC 56 Merge Intervals / LC 228 Summary Ranges

Posted on PracHub: 2026-03-29 (updated 2026-05-19)

Original:
https://prachub.com/coding-questions/check-digits-and-combine-ranges

QUESTION

You are asked to solve two independent coding tasks.

Task 1:

Check Whether an Integer Reads the Same Backward Given an integer `x`, return `true` if its decimal representation reads the same from left to right and from right to left; otherwise return `false`. Requirements and edge cases:

- Negative integers should return `false` because of the leading minus sign.
- You may assume `x` fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

Task 2:

Combine Overlapping Ranges Given an array of intervals `intervals`, where each interval is represented as `[start, end]`, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.
- Two intervals overlap if they share at least one point. For example, `[1, 4]` and `[4, 5]` should be combined into `[1, 5]`.

QUESTION

- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

Part 1: Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.
Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion

EXAMPLES

```
Input: 121
Expected Output: True
```

Explanation: 121 reads the same forward and backward.

```
Input: -121
Expected Output: False
```

Explanation: Negative numbers are not palindromes.

Hints

A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.

Part 2: Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.
Constraints
0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted
Examples
Input: [[1, 3], [2, 6], [8, 10], [15, 18]]
Expected Output: [[1, 6], [8, 10], [15, 18]]
Explanation: The first two intervals overlap and merge into [1, 6].
Input: [[1, 4], [4, 5]]
Expected Output: [[1, 5]]
Explanation: The intervals share the point 4, so they overlap.
Hints

Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

APPROACH

Task 1: reverse half the digits and compare (no string, handle negatives and trailing zeros). Task 2: sort ranges and merge or combine consecutive ones in one pass.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two independent tasks. For the palindrome: negatives are false, no string conversion, 32-bit input. For the ranges: ask whether touching ranges such as [1, 4] and [4, 5] merge (the statement says they do) and whether adjacent integer ranges like [1, 2] and [3, 4] should also merge.

BRUTE FORCE TO SAY OUT LOUD

Palindrome: convert to a string and compare with its reverse. Ranges: compare every pair repeatedly until nothing merges, O(n^2).

OPTIMAL IDEA

Palindrome: return false for negatives and for any positive number ending in 0; then build a reversed number from the low digits while x > rev, and compare x with rev or rev // 10 (to skip the middle digit). Only half the digits are processed, so overflow cannot happen. Ranges: sort by start, then sweep, extending the last merged range when the next start is at most its end, else append. O(n log n).

DRY-RUN THESE CASES

121 true, -121 false, 10 false, 0 true, 1221, 12321. Ranges: [[1,3],[2,6],[8,10],[15,18]] gives [[1,6],[8,10],[15,18]]; [[1,4],[4,5]] gives [[1,5]]; unsorted input; one range; a range fully inside another.

LIKELY FOLLOW-UPS

Palindrome in another base, or palindrome linked list. Ranges: insert a new interval into a sorted list (LeetCode 57), summary ranges of sorted integers (LeetCode 228).

PITFALLS

Reversing the whole number and overflowing. Forgetting to sort before merging. Using strict less-than when touching ranges must merge.

HOW TO THINK ABOUT IT

Task 1: a number is a palindrome if its second half reversed equals its first half, so reverse only half the digits (stop when the reversed part catches up) and avoid overflow. Negatives and numbers ending in 0 are never palindromes. Task 2: after sorting by start, an overlap can only be with the last merged range, so one pass with a 'current' range is enough.

Basis: Two independent tasks: LeetCode 9 (Palindrome Number, no string conversion) and LeetCode 56 (Merge Intervals; ranges sharing a point merge).

BRUTE FORCE · Task 1: convert to a string and compare with its reverse

BRUTE FORCE · PSEUDO-CODE

```
def is_palindrome(x):
    s = str(x)
    return x >= 0 and s == s[::-1]
```

TIME	SPACE
O(d)	O(d)

WHY IT IS SLOW

O(d) time but it builds strings, which the question forbids. Reversing only the second half of the digits arithmetically (x % 10, x // 10) uses O(1) space.

BRUTE FORCE · Task 2: merge any two overlapping ranges until none overlap

BRUTE FORCE · PSEUDO-CODE

```
def combine(ranges):
    ranges = list(ranges)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(ranges, 2):
            if overlaps(a, b):
                ranges.remove(a)
                ranges.remove(b)
                ranges.append(union(a, b))
                merged = True
                break
    return sorted(ranges)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Task 1: reverse only the second half arithmetically

TIME	SPACE
O(d)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Task 2: sort by start, then one sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Adjacent integer ranges ([1, 2] and [3, 4]) should merge too.

Change the overlap test to s <= out[-1][1] + 1 (integers only). For real-valued ranges that is wrong, so say which you assume.

FOLLOW-UP 2 Summarise a sorted list of integers as ranges (LeetCode 228).

One pass keeping the current range and extending it while the next number is exactly one more; the same 'extend or start new' structure as the merge.

FOLLOW-UP 3 Palindrome in another base, or as a linked list.

Another base replaces 10 by the base in % and //. For a linked list find the middle with slow and fast pointers, reverse the second half in place and compare.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Check Digits and Combine Ranges
#
# You are asked to solve two independent coding tasks. ### Task 1: Check Whether an Integer Reads the Same Backward
# Given an integer x, return true if its decimal representation reads the same from left to right and from right to left; otherwise return false. Requirements and edge cases: - Negative integers should return false because of the leading minus sign. - You may assume x fits in a signed 32-bit integer. - Try to solve it without converting the integer to a string. Example: text Input: x = 121 Output: true text Input: x = -121 Output: false text Input: x = 10 Output: false ### Task 2: Combine Overlapping Ranges Given an array of intervals intervals, where each interval is represented as [start, end], combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Adjacent integer ranges ([1, 2] and [3, 4]) should merge too.
# 2. Summarise a sorted list of integers as ranges (LeetCode 228).
# 3. Palindrome in another base, or as a linked list.
#
# INTERVIEW NOTES
# Task 1: a number is a palindrome if its second half reversed equals its first half, so reverse only half the digits (stop when the reversed part catches up) and avoid overflow. Negatives and numbers ending in 0 are never palindromes. Task 2: after sorting by start, an overlap can only be with the last merged range, so one pass with a 'current' range is enough.
#
# BRUTE FORCE (say it first): Task 1: convert to a string and compare with its reverse
# str(x) == str(x)[::-1] after rejecting negatives.
# Correct and short; the question asks to avoid the string. Time O(d). Space O(d).
```

```
# BRUTE FORCE (say it first): Task 1: convert to a string and compare with its reverse | time O(d)
def palindrome_number_string(x):
    return x >= 0 and str(x) == str(x)[::-1]


# BASE QUESTION (optimal): reverse only the second half arithmetically
# Idea: Reject negatives and positive numbers ending in 0.
def is_palindrome(x, base=10):
    # no string: build the reversed second half
    if x < 0 or (x % base == 0 and x != 0):
        return False
    rev = 0
    # stop when the reversed half catches up with the rest
    while x > rev:
        rev = rev * base + x % base
        x //= base
    # an odd number of digits has a middle digit to drop
    return x == rev or x == rev // base


# TASK 2: combine ranges
def merge_ranges(intervals, adjacent=False):
    out = []
    # sort by start so overlaps are neighbours
    for s, e in sorted(intervals):
        # sharing one point counts as overlap: [1,4] + [4,5]
        gap = 1 if adjacent else 0
        if out and s <= out[-1][1] + gap:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out


# FOLLOW-UP 2: Summarise a sorted list of integers as ranges (LeetCode 228).
# Answer: One pass keeping the current range and extending it while the next number is exactly one more; the same 'extend or start new' structure as the merge.
def summarize_ranges(nums):
    # LeetCode 228: a sorted list of ints as ranges
    out = []
    for x in nums:
        if out and x == out[-1][1] + 1:
            # exactly one more: extend the current range
            out[-1][1] = x
        else:
            out.append([x, x])
    return out


if __name__ == "__main__":
    assert is_palindrome(121) and is_palindrome(0)
    assert not is_palindrome(-121)
    assert not is_palindrome(10)
    assert is_palindrome(1221) and not is_palindrome(123)
    # binary 101 is 5; binary 110 is 6
    assert is_palindrome(5, 2) and not is_palindrome(6, 2)
    ranges = [[1, 3], [2, 6], [8, 10], [15, 18]]
    merged = merge_ranges(ranges)
    assert merged == [[1, 6], [8, 10], [15, 18]]
    assert merge_ranges([[1, 4], [4, 5]]) == [[1, 5]]
    # any input order, and an empty list
    shuffled = merge_ranges([[8, 10], [1, 3]])
    assert shuffled == [[1, 3], [8, 10]]
    assert merge_ranges([]) == []
```

```
    # adjacent integer ranges merge only when asked
    touching = [[1, 2], [3, 4]]
    assert merge_ranges(touching) == touching
    assert merge_ranges(touching, True) == [[1, 4]]
    assert summarize_ranges([0, 1, 2, 4, 5, 7]) == [
        [0, 2], [4, 5], [7, 7]]
```

FUNCTION AND TEST CASES

Signature

```
def solution(x: int) -> bool:
SAMPLE INPUT
121
```

Test cases

Input	Expected
`121`	`True`
`-121`	`False`

STUDY HERE: topic 9, Greedy and Math | main topic: Math and bits

HARD | IN APP LC 843 | PracHub | Technical Screen | 5 of 10

Find secret word with match-count feedback

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 42.2 | 30 DAYS 0 | PRIORITY 84/228

ABOUT THIS QUESTION

Topic: Greedy and Math: Math and bits

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #843 Guess the Word

LeetCode match: LC 843 Guess the Word

Posted on PracHub: 2026-02-11 (updated 2026-04-30)

Original: https://prachub.com/coding-questions/find-secret-word-with-match-count-feedback

QUESTION

Problem

You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API:

- `matchCount(guess) -> int`: when you submit a word `guess` from the list, the API returns the number of positions `i` such that `guess[i] == secret[i]` (i.e., exact character match at the same index). You are allowed to call `matchCount`

at most 10 times.

Task

Design/implement a function (e.g., `findSecretWord(words)`) that uses the API to identify the secret word within the guess limit.

Notes / Constraints

- `words` contains between 1 and 100 words.
- All words have equal length `L` (commonly `L = 6`).
- Every guess must be a word from `words`.
- After each API response, you may adapt your next guess.

What to return

Depending on the interface, you may either: - return the identified secret word, or - stop once `matchCount(guess) == L` (found it). (Your interviewer may treat this as an interactive problem; focus on strategy + correctness under the 10-guess constraint.)

You are given a list of unique lowercase words, and every word has the same length `L`. Exactly one word in the list is the secret word. In the original interview version, you interact with a black-box API:

- `matchCount(guess) -> int`

QUESTION

- It returns how many positions `i` satisfy `guess[i] == secret[i]` You may call this API at most 10 times, and every guess must be a word from the given list. In this non-interactive version, implement `solution(words, secret)` and simulate the API inside your function. The parameter `secret` is provided only so the judge can verify your strategy. Your function should repeatedly make guesses, use the returned match counts to eliminate impossible candidates, and return the discovered secret word. If your strategy does not find the secret within 10 guesses, return an empty string `""`.

Constraints`1 <= len(words) <= 1001 <= len(words[i]) <= 10`All words are unique, lowercase, and have the same length `Lsecret` is guaranteed to be one of the words in `words`Your algorithm may use at most 10 simulated `matchCount` callsTest data is designed so that a correct elimination strategy can identify the secret within the limit

EXAMPLES

```
Input: (["acckzz", "ccbazz", "eiowzz", "abcczz"],
    "acckzz")
```

Expected Output: 'acckzz'Explanation: The secret is already one of the candidates. Using match-count feedback, the candidate set shrinks until `acckzz` is identified.

```
Input: (["secret"], "secret")
```

Expected Output: 'secret'Explanation: With only one word in the list, the first allowed guess must be the secret.

Hints

After a guess returns `k`, any valid remaining candidate must also have exactly `k` matching positions with that guess.A strong strategy is to choose the next guess that splits the remaining candidates as evenly as possible, minimizing the size of the largest feedback bucket.

APPROACH

Pick a guess, read matchCount, keep only candidates with the same match count against that guess. Choosing the guess that minimizes the worst-case remaining set (minimax) keeps queries low.

HOW TO THINK ABOUT IT

Each guess returns a number that every remaining candidate must agree with: a candidate can still be the secret only if it has the same match count with the guess as the secret does. So after each guess, filter the candidate list. To shrink fast, guess a word that splits the candidates evenly by match count (minimise the largest group); that is what keeps you inside 10 guesses.

Basis: LeetCode 843, Guess the Word (match count = positions with the same letter).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words):
    candidates = list(words)
    while True:
        guess = candidates[0]
        m = matchCount(guess)
        if m == len(guess):
            return guess
        candidates = [w for w in candidates
                      if matches(w, guess) == m]
```

TIME	SPACE
O(guesses * W * L)	O(W)

WHY IT IS SLOW

A poor guess may remove very few candidates, so there is no guarantee of finishing in 10 guesses. Guessing the word whose largest match-count group is smallest (minimax) keeps the list shrinking fast.

OPTIMAL SOLUTION · Guess the word that minimises the size of its largest match-count group

TIME	SPACE
O(guesses * W^2 * L)	O(W)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Show how the candidate list shrinks.

After a guess with answer m, only words with the same match count against the guess survive (the secret and the candidate agree on every comparison). Printing the sizes round by round shows why a balanced guess finishes in a few rounds.

FOLLOW-UP 2 Guess at random instead of the minimax choice.

Expected to finish in far fewer than W guesses, but not guaranteed within 10 on a hard list. The minimax choice is the safe answer; random with a 'zero-match' preference (guess words that share no letters with earlier guesses) is a cheap improvement.

FOLLOW-UP 3 The word list is much larger.

Computing all pairwise match counts is O(W^2 L). Sample candidates when choosing a guess, or precompute buckets by letter at each position to count matches faster.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find secret word with match-count
# feedback
#
# ## Problem You are given a list of **unique** lowercase
# words, all of the same length (e.g., length = 6). One of
# these words is a **secret** word. You can interact with
# a black-box API: - matchCount(guess) -> int: when you
# submit a word guess from the list, the API returns the
# number of positions i such that guess[i] == secret[i]
# (i.e., exact character match at the same index). You are
# allowed to call matchCount **at most 10 times**. ###
# Task Design/implement a function (e.g.,
# findSecretWord(words)) that uses the API to identify the
# secret word within the guess limit. ### Notes /
# Constraints - words contains between 1 and 100 words. -
# All words have equal length L (commonly L = 6). - Every
# guess must be a word from words. - After each API
# response, you may adapt your next guess.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Show how the candidate list shrinks.
# 2. Guess at random instead of the minimax choice.
# 3. The word list is much larger.
#
# INTERVIEW NOTES
# Each guess returns a number that every remaining
# candidate must agree with: a candidate can still be the
# secret only if it has the same match count with the
# guess as the secret does. So after each guess, filter
# the candidate list. To shrink fast, guess a word that
# splits the candidates evenly by match count (minimise
# the largest group); that is what keeps you inside 10
# guesses.
#
# BRUTE FORCE (say it first): Guess the first candidate
# each time, then filter
# Pick any remaining word, call the API, and keep only the
# words that have the same match count with it. Correct
# but there is no guarantee of finishing in 10 guesses on
# adversarial lists. Time O(guesses * W * L). Space O(W).


from collections import Counter
```

```
# BRUTE FORCE (say it first): Guess the first candidate
# each time, then filter | time O(guesses * W * L)
def _match(a, b):
    return sum(x == y for x, y in zip(a, b))


# HELPERS for the base question
def match(a, b):
    # number of positions holding the same letter
    return sum(x == y for x, y in zip(a, b))


def find_secret_first(words, secret):
    cand, guesses = list(words), 0
    while True:
        g = cand[0]
        guesses += 1
        m = _match(g, secret)
        if m == len(secret):
            return guesses
        cand = [w for w in cand if _match(g, w) == m]
```

```
# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
def best_guess(candidates):
    # minimax: the guess whose worst answer leaves the
    # fewest
    # candidates (ties: alphabetical)
    def worst(g):
        sizes = Counter(match(g, w) for w in candidates)
        return max(sizes.values()), g
    return min(candidates, key=worst)


# BASE QUESTION (optimal): Guess the word that minimises
# the size of its largest match-count group
# Idea: For each candidate guess count how many other
# candidates would fall into each match-count group, and
# pick the guess whose biggest group is smallest (a
# minimax choice).
```

```
def find_secret(words, match_count, limit=10):
    # match_count(guess) is the black-box API
    candidates, guesses = list(words), 0
    while candidates and guesses < limit:
        guess = best_guess(candidates)
        guesses += 1
        score = match_count(guess)
        if score == len(guess):
            return guess, guesses
        # only words with the same score against this
        # guess
        # can still be the secret
        candidates = [w for w in candidates
                      if match(guess, w) == score]
    return None, guesses


# FOLLOW-UP 1: Show how the candidate list shrinks.
# Answer: After a guess with answer m, only words with the
# same match count against the guess survive (the secret
# and the candidate agree on every comparison).
def shrink_report(words, secret):
    # candidate counts after each guess (shows why it
    # works)
    sizes, candidates = [len(words)], list(words)
    while len(candidates) > 1:
        guess = best_guess(candidates)
        score = match(guess, secret)
        if score == len(secret):
            break
        candidates = [w for w in candidates
                      if match(guess, w) == score]
        sizes.append(len(candidates))
    return sizes


if __name__ == "__main__":
    import itertools
    words = ["".join(p) for p in
             itertools.product("abc", repeat=4)]
    worst = 0
    for secret in words:
        calls = []

        def api(guess, secret=secret):
```

```
            calls.append(guess)
            return match(guess, secret)
        found, used = find_secret(words, api)
        assert found == secret and used == len(calls)
        worst = max(worst, used)
    # 81 words are always found inside the limit of 10
    assert worst <= 10
    single = find_secret(["only"], lambda g: 4)
    assert single == ("only", 1)
    sizes = shrink_report(words, "abca")
    assert sizes[0] == 81
    assert sizes == sorted(sizes, reverse=True)
```

FUNCTION AND TEST CASES

Signature

```
def solution(words: list[str], secret: str) -> str:
```

Sample call

```
solution(["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz
")
```

Test cases

Input	Expected
(["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")	'acckzz'
(["secret"], "secret")	'secret'

STUDY HERE: topic 9, Greedy and Math | main topic: Math and bits

Site tags: algorithm-design, greedy, hard

HARD | 1Point3Acres | phone-screen, onsite-coding | 6 of 10

Wordle-Style 5-Letter Minimum-Guess Strategy

ASKED 2025-12-15 | 309 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 42.2 | 30 DAYS 0 | PRIORITY 85/228

ABOUT THIS QUESTION

Topic: Greedy and Math: Math and bits

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #843 Guess the Word

LeetCode match: LC 843 Guess the Word (related)

Frequency: Single report | Last asked: 2025-12-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/wordle-min-guess-strategy

QUESTION

Phone screen coding: given a black-box oracle that tells you whether a specific letter is at a specific position in a hidden 5-letter word, design an algorithm to identify the word in as few oracle calls as possible. Follow-up assumes a dictionary of candidate words is available.

Hidden word: a 5-letter string from a..z.
Oracle: query(pos: int, letter: char) -> bool returns whether position pos (0-indexed) holds letter.
Return the hidden word.
Primary metric: minimize the number of query calls.

Brute-force baseline: 5 × 26 = 130 calls.
Follow-up

A dictionary of candidate 5-letter words is available; design the strategy to use bigram/letter-frequency information to cut the expected number of calls.

EXAMPLES

```
HiddenWord = "GRAPE"
query(0, 'A') -> false
query(0, 'G') -> true   // first letter found
...
```

APPROACH

Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

HOW TO THINK ABOUT IT

Each call returns one bit, so you can never beat ceil(log2(number of candidates)) calls. Without a dictionary the best you can do is order the questions by likelihood and stop at the first yes for each position (and skip the 26th letter, because it is implied). With a dictionary, keep the list of words still possible and ask the (position, letter) question whose yes/no answer splits that list most evenly.

Basis: Written for this question; related to LeetCode 843, Guess the Word, but the oracle answers one position and one letter per call.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
word = ""
for i in range(5):
    for c in "abcdefghijklmnopqrstuvwxyz":
        if query(i, c):
            word += c
            break
```

TIME	SPACE
130 calls	O(1)

WHY IT IS SLOW

Always 130 calls. Asking likely letters first, skipping the implied last letter, or splitting a dictionary of candidates in half needs far fewer.

OPTIMAL SOLUTION · Frequency order, or entropy split when a dictionary is known

TIME	SPACE
at most 5 x 25 calls with... at most 5 x 25 calls without a dictionary; about log2(words) with one	O(words)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What is the lower bound and when is it reached?

Every call returns at most one bit, so at least ceil(log2(candidates)) calls are needed. The entropy split reaches it only when a question can cut the candidates exactly in half; real dictionaries usually need a few more. A greedy scoring rule (closest to half) is enough to state; the interviewer mostly wants the entropy idea.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Wordle-Style 5-Letter Minimum-Guess
# Strategy
#
# Phone screen coding: given a black-box oracle that tells
# you whether a specific letter is at a specific position
# in a hidden 5-letter word, design an algorithm to
# identify the word in as few oracle calls as possible.
# Follow-up assumes a dictionary of candidate words is
# available. Hidden word: a 5-letter string from a..z.
# Oracle: query(pos: int, letter: char) -> bool returns
# whether position pos (0-indexed) holds letter. Return
# the hidden word. Primary metric: minimize the number of
# query calls. Brute-force baseline: 5 × 26 = 130 calls.
# Follow-up A dictionary of candidate 5-letter words is
# available; design the strategy to use bigram/letter-
# frequency information to cut the expected number of
# calls. Examples: HiddenWord = "GRAPE" query(0, 'A') ->
# false query(0, 'G') -> true // first letter found ...
#
# FOLLOW-UPS (answered below, in this order):
# 1. What is the lower bound and when is it reached?
#
# INTERVIEW NOTES
# Each call returns one bit, so you can never beat
# ceil(log2(number of candidates)) calls. Without a
# dictionary the best you can do is order the questions by
# likelihood and stop at the first yes for each position
# (and skip the 26th letter, because it is implied). With
# a dictionary, keep the list of words still possible and
# ask the (position, letter) question whose yes/no answer
# splits that list most evenly.
#
# BRUTE FORCE (say it first): Ask every position and
# letter
# Query all 5 x 26 pairs and read the word off the
# answers. Always 130 calls, no matter how lucky the word
# is. Time 130 calls. Space O(1).


import string


# BRUTE FORCE: 130 calls
def brute_force(query):
    # 130 calls at most: every position, every letter
    return "".join(
        next(c for c in string.ascii_lowercase
             if query(i, c))
        for i in range(5))


# HELPERS for the base question
# letters from most to least common in English words
BY_FREQUENCY = "etaoinshrdlcumwfgypbvkjxqz"


# FOLLOW-UP 1: What is the lower bound and when is it
# reached?
# Answer: Every call returns at most one bit, so at least
# ceil(log2(candidates)) calls are needed. The entropy
# split reaches it only when a question can cut the
# candidates exactly in half; real dictionaries usually
# need a few more.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def split_score(words, i, c):
    # how evenly does asking (position i, letter c) split?
    yes = sum(w[i] == c for w in words)
    return min(yes, len(words) - yes)


# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def by_frequency(query):
    # no dictionary: common letters first, stop at the
    # first
    # yes, and skip the 26th letter (it must be the one
    # left)
    calls, word = 0, []
    for i in range(5):
        for k, c in enumerate(BY_FREQUENCY):
            if k == 25:
                word.append(c)
                break
            calls += 1
            if query(i, c):
                word.append(c)
                break
    return "".join(word), calls


# BASE QUESTION (optimal): Frequency order, or entropy
# split when a dictionary is known
# Idea: Without a dictionary: for each position ask
# letters from most to least common and stop at the first
# yes; if 25 letters said no the 26th is implied and needs
# no call.
def with_dictionary(query, dictionary):
    # keep the words that still fit every answer so far
    words, calls = list(dictionary), 0
    while len(words) > 1:
        options = {(i, w[i]) for w in words
                   for i in range(5)}
        # ask the question that cuts the list closest to
        # half
        i, c = max(options, key=lambda o: split_score(
            words, o[0], o[1]))
        calls += 1
        hit = query(i, c)
        words = [w for w in words if (w[i] == c) == hit]
    return words[0], calls


if __name__ == "__main__":
    def oracle(word):
        return lambda i, c: word[i] == c

    secret = "grape"
    assert brute_force(oracle(secret)) == secret
    word, calls = by_frequency(oracle(secret))
    assert word == secret and calls <= 125
    words = ["grape", "grade", "trade", "crane", "plane",
             "slate", "stare", "share"]
    for target in words:
        ask = oracle(target)
        found, used = with_dictionary(ask, words)
        assert found == target
        # about log2(8) = 3 questions suffice; allow a few
        # more
        assert used <= 6
    # lower bound: one bit per call, so ceil(log2(8)) = 3
```

```
    # calls
    assert min(with_dictionary(oracle(w), words)[1]
               for w in words) >= 3
```

NOTES FROM CANDIDATES

Without the dictionary, query positions in order; per position, query letters in descending frequency (e, t, a, o, i, n, ...) and stop as soon as one returns true.
With the dictionary, maintain a candidate set; at each step pick the (position, letter) query whose true/false outcome splits the candidate set most evenly (entropy maximization).
Mention information theory: each query yields <= 1 bit, so the absolute lower bound on calls is ceil(log2(|candidates|)).
Don't waste minutes on a closed-form optimum — the interviewer mostly wants you to articulate the entropy heuristic and pick one reasonable scoring function.

PREPARATION

Drill information-theoretic puzzles (egg drop, twenty questions) to internalize the entropy-split heuristic.
Practice maintaining a candidate set under positive/negative oracle feedback (filter on each query result).
Be ready with a one-liner on Wordle's known optimal first guess (SOARE / CRANE) for intuition; do not hard-code it.

STUDY HERE: topic 9, Greedy and Math | main topic: Math and bits

HARD | PracHub | Technical Screen | 7 of 10

Shuffle array using random integer API

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 14.7 | 30 DAYS 0 | PRIORITY 162/228

ABOUT THIS QUESTION

Topic: Greedy and Math: Math and bits

Optimal answer from: the interview site's own solution

LeetCode match: LC 384 Shuffle an Array

Posted on PracHub: 2025-12-08 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/shuffle-array-using-random-integer-api

QUESTION

You are given an array **nums** of length **n** representing a deck of distinct cards. You **do not** have access to a built-in shuffle function, but you are given the following API:

```
rand_int(l, r) # returns a uniformly random integer in [l, r], inclusive
```

Task

Implement a function **`shuffle(nums)`** that randomly permutes the array **in place** such that **every one of the `n!` permutations is equally likely**.

Requirements

- You may only use **`rand_int(l, r)`** as your source of randomness.
- The algorithm should run in **`O(n)`** time and **`O(1)`** extra space.
- You may assume **`rand_int`** itself is perfectly uniform and independent between calls. Describe your algorithm, argue why the resulting permutation is uniform, and give the time and space complexity.

APPROACH

Fisher-Yates: for i from n-1 down to 1, j = rand_int(0, i), swap nums[i], nums[j]. Uniform because each permutation has probability 1/n!. Avoid the biased 'swap with random index in [0,n)' version.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether rand_int(l, r) is inclusive of r, whether to shuffle in place, and whether the result must be uniformly random over all n! permutations.

BRUTE FORCE TO SAY OUT LOUD

Assign each element a random key and sort. It is uniform but O(n log n).

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Fisher-Yates: for i from n - 1 down to 1, pick j = rand_int(0, i) and swap nums[i] with nums[j]. Each of the n! permutations has probability 1 / n! because position i is filled by one of i + 1 equally likely elements. O(n) time, O(1) extra space.

DRY-RUN THESE CASES

n = 0 and n = 1. n = 3 with many trials: all six permutations should appear about equally. Say why swapping with a random index in [0, n) is biased (n^n outcomes cannot divide evenly into n! permutations).

LIKELY FOLLOW-UPS

Shuffle only k elements (stop the loop early), uniform integer from a coin flip only (rejection sampling), or a weighted shuffle.

PITFALLS

Using j in [0, n) every time. An off-by-one on the inclusive bound.

HOW TO THINK ABOUT IT

'Shuffle' means every one of the n! orders must be equally likely. Count outcomes: a procedure with n^n equally likely random choices cannot give n! equal buckets, so it is biased. Fisher-Yates makes n choices with sizes n, n-1, ..., 1, which multiply to exactly n!.

Basis: LeetCode 384, Shuffle an Array (Fisher-Yates).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def shuffle(nums):
    keyed = []
    for x in nums:
        keyed.append((rand_int(0, 10 ** 9), x))
    keyed.sort()
    return [x for _, x in keyed]
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

O(n log n) time and O(n) extra space, and equal random keys make the result very slightly non-uniform. Fisher-Yates (swap position i with a random position in 0..i) is O(n), in place and exactly uniform.

OPTIMAL SOLUTION · Fisher-Yates

TIME	SPACE
O(n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return only k random elements.

Run Fisher-Yates for k steps from the front (swap position i with a random index in i..n-1) and return the first k. O(k) after copying, still uniform.

FOLLOW-UP 2 Only a coin-flip API exists, no rand_int.

Build a uniform integer in [0, m) by rejection sampling: draw ceil(log2 m) random bits and retry if the number is at least m. That keeps every value equally likely, unlike taking a remainder.

FOLLOW-UP 3 Why is the biased version biased?

It makes n choices of n options, n^n equally likely paths, and n! does not divide n^n for n >= 3, so some permutations must appear more often than others.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Shuffle array using random integer API
#
# You are given an array nums of length n representing a
# deck of distinct cards. You **do not** have access to a
# built-in shuffle function, but you are given the
# following API: python rand_int(l, r) # returns a
# uniformly random integer in [l, r], inclusive ### Task
# Implement a function shuffle(nums) that randomly
# permutes the array **in place** such that **every one of
# the n! permutations is equally likely**. ###
# Requirements - You may only use rand_int(l, r) as your
# source of randomness. - The algorithm should run in O(n)
# time and O(1) extra space. - You may assume rand_int
# itself is perfectly uniform and independent between
# calls. Describe your algorithm, argue why the resulting
# permutation is uniform, and give the time and space
# complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return only k random elements.
# 2. Only a coin-flip API exists, no rand_int.
# 3. Why is the biased version biased?
#
# INTERVIEW NOTES
# 'Shuffle' means every one of the n! orders must be
# equally likely. Count outcomes: a procedure with n^n
# equally likely random choices cannot give n! equal
# buckets, so it is biased. Fisher-Yates makes n choices
# with sizes n, n-1, ..., 1, which multiply to exactly n!.
#
# BRUTE FORCE (say it first): Give each element a random
# key and sort
# Pair each element with a random number and sort by it.
# With keys from a large range it is practically uniform,
# but it spends O(n log n) and needs extra space; if two
# keys collide the result is slightly biased. Time O(n log
# n). Space O(n).


import random


# BRUTE FORCE (say it first): Give each element a random
# key and sort | time O(n log n)
def rand_int(lo, hi):
    return random.randint(lo, hi)


# HELPERS for the base question
def shuffle_by_keys(nums):
    keyed = sorted(
        (
            (rand_int(0, 10**9), i)
            for i in range(len(nums))
        )
    )
    return [nums[i] for _, i in keyed]


# BASE QUESTION (optimal): Fisher-Yates
# Idea: Walk i from n - 1 down to 1, pick j uniformly from
# 0..i (inclusive) and swap nums[i] with nums[j].
def shuffle(nums, rand_int):
    # Fisher-Yates: from the end, swap each position with
    # a
    # random one at or before it. All n! orders are
    # equally
    # likely because each step picks uniformly among i +
    # 1.
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums


# FOLLOW-UP 1: Return only k random elements.
# Answer: Run Fisher-Yates for k steps from the front
# (swap position i with a random index in i..n-1) and
# return the first k. O(k) after copying, still uniform.
def sample_k(nums, k, rand_int):
    # k random elements: only k Fisher-Yates steps, from
    # the front
    a = list(nums)
    for i in range(min(k, len(a))):
        j = rand_int(i, len(a) - 1)
        a[i], a[j] = a[j], a[i]
    return a[:k]


# FOLLOW-UP 2: Only a coin-flip API exists, no rand_int.
# Answer: Build a uniform integer in [0, m) by rejection
# sampling: draw ceil(log2 m) random bits and retry if the
# number is at least m. That keeps every value equally
# likely, unlike taking a remainder.
def rand_below(m, flip):
    # only a coin flip exists: rejection sampling gives a
    # uniform number in [0, m) (a remainder would be
    # biased)
    bits = max(1, (m - 1).bit_length())
    while True:
        x = 0
        for _ in range(bits):
            x = x * 2 + flip()
        if x < m:
            return x


# FOLLOW-UP 3: Why is the biased version biased?
# Answer: It makes n choices of n options, n^n equally
# likely paths, and n! does not divide n^n for n >= 3, so
# some permutations must appear more often than others.
def biased_shuffle(nums, rand_int):
    # the common WRONG version: n^n paths cannot be spread
```

```
    # over
    # n! orders evenly (shown only to explain the bias)
    for i in range(len(nums)):
        j = rand_int(0, len(nums) - 1)
        nums[i], nums[j] = nums[j], nums[i]
    return nums


if __name__ == "__main__":
    rng = random.Random(7)
    rand_int = lambda lo, hi: rng.randint(lo, hi)
    seen = {}
    for _ in range(6000):
        key = tuple(shuffle([1, 2, 3], rand_int))
        seen[key] = seen.get(key, 0) + 1
    # all 6 orders appear, each about 1000 times
    assert len(seen) == 6
    assert all(850 < v < 1150 for v in seen.values())
    # nothing is lost or repeated
    deck = shuffle(list(range(50)), rand_int)
    assert sorted(deck) == list(range(50))
    # k random elements
```

```
    pick = sample_k(list(range(10)), 3, rand_int)
    assert len(set(pick)) == 3 and len(pick) == 3
    # a coin gives uniform numbers below 5
    coin = lambda: rng.randint(0, 1)
    counts = [0] * 5
    for _ in range(5000):
        counts[rand_below(5, coin)] += 1
    assert all(800 < c < 1200 for c in counts)
    assert shuffle([], rand_int) == []
```

FUNCTION AND TEST CASES

Signature

```
def solution(nums: list[int], random_indices: list[int]) -
> list[int]:
```

Sample call

```
solution([1, 2, 3, 4], [1, 1, 0])
```

Test cases

Input	Expected
([1, 2, 3, 4], [1, 1, 0])	[3, 1, 4, 2]
([], [])	[]

STUDY HERE: topic 9, Greedy and Math | main topic: Math and bits

HARD | PracHub | Technical Screen | 8 of 10

Find maximum value in huge text file

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 185/228

ABOUT THIS QUESTION

Topic: Greedy and Math: Math and bits

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

LeetCode match: No LC match (systems / scalability question; the Hard tag looks inflated)

Posted on PracHub: 2025-12-08 (updated 2026-06-27)

Original: https://prachub.com/interview-questions/find-maximum-value-in-huge-text-file

QUESTION

You are given the path to a **very large** text file on disk. The file has the following properties:

- Each line contains exactly one signed 64-bit integer (in decimal).
- The file size can be tens or hundreds of gigabytes, so it **cannot** be fully loaded into memory.
- You have standard file I/O APIs that allow you to read the file sequentially (e.g., line by line or in chunks).

Tasks

1.

Single-machine solution Design an algorithm to compute the **maximum integer value** in this file.

- Assume you have limited RAM (much smaller than the file size).
- Discuss how you would read the file and track the result.
- Give high-level pseudocode.
- Analyze time and space complexity. 2.

Performance improvement / parallelization Suppose you now have a multi-core machine or multiple machines available and want to speed up the computation.

- How would you modify your approach to take advantage of parallelism (e.g., processing the file in chunks)?
- Explain how you would combine partial results to get the global maximum.
- Mention any edge cases or failure scenarios you would consider (e.g., very large values, empty file). Assume you do not have a database; you are working directly with the file system and your programming language's standard I/O libraries.

APPROACH

Stream the file line by line (or in large buffered chunks) keeping a single running max initialized to None/-2^63; O(n) time, O(1) memory. Parallelize by splitting byte ranges aligned to line boundaries across cores/machines (map), then take the max of the partial maxima (reduce). Edge cases: empty file, trailing newline, malformed lines, negative-only values, retries when a worker fails.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the file size relative to RAM, whether a line can be malformed, whether the file can be empty, and whether there is one machine or a cluster.

BRUTE FORCE TO SAY OUT LOUD

Load the file and call max. Say why it fails (memory), then move on.

OPTIMAL IDEA

Stream line by line with one running maximum initialized to None (not zero, so negative-only files work). O(n) time, O(1) memory. To parallelize split the file into byte ranges aligned to line boundaries: each worker discards its partial first line (the previous worker reads it) and finishes the line that crosses its end. Reduce with max of the partial results; retry failed ranges since each task is idempotent.

DRY-RUN THESE CASES

Empty file, one line, all negative values, last line without newline, values at the 64-bit limits.

LIKELY FOLLOW-UPS

Top K values, the median of a huge file, or distributed execution with failures.

PITFALLS

Starting the max at zero. Splitting ranges mid-line.

HOW TO THINK ABOUT IT

The answer is a running maximum, which needs one variable, so memory should not depend on the file size. Read the file as a stream. To go faster across cores, give each worker a byte range that starts and ends on line boundaries; the global maximum is the maximum of the partial maxima.

Basis: Systems / streaming question, no LeetCode match: a streaming reduce and its map-reduce parallelization.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(path):
    numbers = []
    for line in open(path):
        numbers.append(int(line))  # in memory
    return max(numbers)
```

BRUTE FORCE

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

Needs memory for the whole file, which is exactly what is not available at tens of gigabytes. Streaming with one running maximum uses O(1) memory.

OPTIMAL SOLUTION · Single machine: stream the file, keep one running maximum

TIME	SPACE
O(n)	O(1) plus the read buffer

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Parallel: split byte ranges on line boundaries (map), max of the maxima (reduce)

TIME	SPACE
O(n / workers) plus I/O limits	O(workers)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Edge cases to mention.

Empty file (return None or raise, by contract); a final line without a newline; blank lines; malformed lines; values at the 64-bit extremes; an all-negative file (do not initialize the max to 0); and Windows line endings. Decide whether a bad line fails the job or is skipped and counted.

FOLLOW-UP 2 How do you scale to many machines and handle failures?

Assign byte ranges (map tasks) to workers through a coordinator, and make each task idempotent (it only returns its maximum), so a failed or slow worker is simply retried or speculatively re-executed elsewhere. The reduce step is a max over a tiny list. On a distributed filesystem, align ranges to its block size for locality. **Cost:** O(n / workers) wall-clock time plus O(workers) to combine; total work stays O(n).

FOLLOW-UP 3 Is it I/O-bound or CPU-bound, and why does that matter?

Parsing integers is cheap compared with disk throughput, so one sequential reader often saturates the disk and extra threads help only if the data sits on several disks or on fast SSD or network storage. Measure first; the parallel version pays off mainly across machines or storage devices. **Cost:** reading takes about n / disk throughput and parsing is O(n) as well, so extra threads help only beyond the disk's speed.

FOLLOW-UP 4 The file has billions of lines and one machine is too slow.

Split by byte offsets across machines (each skips to the next newline), compute local maxima, and take the maximum of those few numbers. Tasks are idempotent, so a failed worker can simply be retried. **Cost:** O(n / machines) per machine and O(machines) to combine; a retry re-runs only the failed byte range.

FOLLOW-UP 5 Now return the K largest values instead of only the maximum.

Keep a min-heap of size K while streaming: push a value if the heap is not full, or replace the root when the value is larger. Memory O(K), time O(n log K); merge the per-worker heaps for a parallel version.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find maximum value in huge text file
#
# You are given the path to a **very large** text file on
# disk. The file has the following properties: - Each line
# contains exactly one signed 64-bit integer (in decimal).
# - The file size can be tens or hundreds of gigabytes, so
# it **cannot** be fully loaded into memory. - You have
# standard file I/O APIs that allow you to read the file
# sequentially (e.g., line by line or in chunks). ###
# Tasks 1. **Single-machine solution** Design an algorithm
# to compute the **maximum integer value** in this file. -
# Assume you have limited RAM (much smaller than the file
# size). - Discuss how you would read the file and track
# the result. - Give high-level pseudocode. - Analyze time
# and space complexity. 2. **Performance improvement /
# parallelization** Suppose you now have a multi-core
# machine or multiple machines available and want to speed
# up the computation.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Edge cases to mention.
# 2. How do you scale to many machines and handle
# failures?
# 3. Is it I/O-bound or CPU-bound, and why does that
# matter?
# 4. The file has billions of lines and one machine is too
# slow.
# 5. Now return the K largest values instead of only the
```

```
# maximum.
#
# INTERVIEW NOTES
# The answer is a running maximum, which needs one
# variable, so memory should not depend on the file size.
# Read the file as a stream. To go faster across cores,
# give each worker a byte range that starts and ends on
# line boundaries; the global maximum is the maximum of
# the partial maxima.
#
# BRUTE FORCE (say it first): Load the whole file and take
# max
# Read every number into a list and call max. Correct for
# small files and the oracle; it needs memory proportional
# to the file, which is the thing the problem rules out.
# Time O(n). Space O(n).
```

```
import heapq
import os
import tempfile


# BRUTE FORCE (say it first): Load the whole file and take
# max | time O(n)
def max_in_file_loaded(path):
    with open(path) as f:
        nums = [int(x) for x in f.read().split()]
    return max(nums) if nums else None


# BASE QUESTION (optimal): Single machine: stream the
# file, keep one running maximum
# Idea: Never load the file. Read it through a buffered
# iterator and keep one variable, the maximum so far,
# started at None so that an all-negative file or a one-
# line file is handled correctly (do not start at 0).
def max_in_file(path):
    # one sequential pass, one number in memory
    best = None
    with open(path, "r", buffering=1 << 20) as f:
        for line in f:
            line = line.strip()
            # blank lines are ignored (state this
            # contract)
            if not line:
                continue
            value = int(line)
            # start from None, not 0: all numbers may be
            # negative
            if best is None or value > best:
                best = value
    return best


# ONE WORKER: its byte range
def max_in_range(path, start, end):
    # one worker: lines that START inside [start, end)
    best = None
    with open(path, "rb") as f:
        f.seek(start)
        if start:
```

```
            # we may be in the middle of a line: the
            # previous
            # worker owns it, so skip to the next line
            # start
            f.seek(start - 1)
            f.readline()
        while f.tell() < end:
            line = f.readline()
            if not line:
                break
            if line.strip():
                v = int(line)
                best = v if best is None else max(best, v)
    return best


# FOLLOW-UP 4: The file has billions of lines and one
# machine is too slow.
# Answer: Split by byte offsets across machines (each
# skips to the next newline), compute local maxima, and
# take the maximum of those few numbers. Tasks are
# idempotent, so a failed worker can simply be retried.
def parallel_max(path, workers=4):
    # split by byte offsets; combine the few local maxima
    size = os.path.getsize(path)
    cuts = [size * i // workers
            for i in range(workers + 1)]
    parts = [max_in_range(path, a, b)
             for a, b in zip(cuts, cuts[1:])]
    parts = [p for p in parts if p is not None]
    return max(parts) if parts else None


# FOLLOW-UP 5: Now return the K largest values instead of
# only the maximum.
# Answer: Keep a min-heap of size K while streaming: push
# a value if the heap is not full, or replace the root
# when the value is larger. Memory O(K), time O(n log K);
# merge the per-worker heaps for a parallel version.
def top_k_in_file(path, k):
    # the k largest values: a min-heap of size k, O(k)
    # memory
    heap = []
    with open(path) as f:
        for line in f:
```

```
            if not line.strip():
                continue
            value = int(line)
            if len(heap) < k:
                heapq.heappush(heap, value)
            elif value > heap[0]:
                heapq.heapreplace(heap, value)
    return sorted(heap, reverse=True)


if __name__ == "__main__":
    def write(lines):
        f = tempfile.NamedTemporaryFile("w", delete=False)
        f.write("".join(lines))
        f.close()
        return f.name

    p = write(["5\n", "-3\n", "\n", "99\n", "12"])
    assert max_in_file(p) == 99
    assert top_k_in_file(p, 2) == [99, 12]
    assert parallel_max(p, 3) == 99
    # all negative numbers, extreme 64-bit values
    q = write(["-7\n", "-2\n", "-9\n"])
    assert max_in_file(q) == -2
    big = write([f"{2 ** 63 - 1}\n", f"{-2 ** 63}\n"])
    assert max_in_file(big) == 2 ** 63 - 1
    # an empty file has no maximum
    e = write([])
    assert max_in_file(e) is None
    assert parallel_max(e) is None
    # every worker count agrees on a bigger file
    nums = [(i * 7919) % 10007 - 5000
            for i in range(2000)]
    r = write([f"{x}\n" for x in nums])
    for w in (1, 2, 5, 8):
        assert parallel_max(r, w) == max(nums)
    for name in (p, q, big, e, r):
```

```
os.remove(name)
```

STUDY HERE: topic 9, Greedy and Math | main topic: Math and bits

Site tags: algorithm-design, math-reasoning, hard

HARD | 1Point3Acres | onsite-coding | 9 of 10

Find All Bad Pairs Using runTest Oracle

ASKED 2025-11-27 | 327 DAYS AGO | OLDER

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 192/228**

ABOUT THIS QUESTION

Topic: Greedy and Math: Math and bits

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2025-11-27 | Roles: SWE, RS

Original: https://www.1point3acres.com/interview/problems/company/google/find-bad-pairs-with-oracle

QUESTION

PhD intern coding round 2: given a black-box oracle **`runTest(S)`** that returns **`true`** iff a subset **`S`** contains no bad pair, identify all bad pairs across N test items with as few oracle calls as possible.

Items t_1, ..., t_N are given.
Oracle: runTest(S: Set[Item]) -> bool returns true iff S contains zero bad pairs; otherwise false.
Output: every bad pair (t_i, t_j).
Minimize total oracle calls.

EXAMPLES

```
N=4, bad pairs = {(1,2)}.
runTest({1,2,3,4}) -> false -> at least one bad pair
    somewhere.
runTest({1,3,4})   -> true  -> 2 is in every bad pair
    containing items from this set.
... binary search to isolate.
```

APPROACH

Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

HOW TO THINK ABOUT IT

Testing every pair costs N(N-1)/2 calls. A single call on a whole set answers 'is there any bad pair inside', so you can discard good sets fast and binary-search for the culprit inside a bad one. Add the tests one at a time to groups known to be clean: if a group plus the new test fails, binary search the group for a partner; otherwise the new test joins it.

Basis: Adaptive group testing; the same method as the 'bad pairs with runTest' question in the PracHub set.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
bad = []
for i in range(n):
    for j in range(i + 1, n):
        if not runTest({i, j}):
            bad.append((i, j))
```

TIME	SPACE
O(N^2) calls	O(1)

WHY IT IS SLOW

Always N(N - 1) / 2 calls. Testing whole groups and binary searching a failing group finds each bad pair in about log N calls.

OPTIMAL SOLUTION · Group the tests and binary search for partners

TIME	SPACE
O(K log N + N * G) about calls (K bad pairs, G groups)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many calls are needed in the best case and the worst case?

With no bad pairs it takes N - 1 calls, one per test after the first. With K bad pairs each one costs about log N calls to locate, so roughly K log N plus the cost of building clean groups. The brute force is always N(N-1)/2, which is far larger as soon as bad pairs are rare.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find All Bad Pairs Using runTest Oracle
#
# PhD intern coding round 2: given a black-box oracle
# runTest(S) that returns true iff a subset S contains no
# bad pair, identify all bad pairs across N test items
# with as few oracle calls as possible. Items t_1, ...,
# t_N are given. Oracle: runTest(S: Set[Item]) -> bool
# returns true iff S contains zero bad pairs; otherwise
# false. Output: every bad pair (t_i, t_j). Minimize total
# oracle calls. Examples: N=4, bad pairs = {(1,2)}.
# runTest({1,2,3,4}) -> false -> at least one bad pair
# somewhere. runTest({1,3,4}) -> true -> 2 is in every bad
# pair containing items from this set. ... binary search
# to isolate.
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many calls are needed in the best case and the
# worst case?
```

```
#
# INTERVIEW NOTES
# Testing every pair costs N(N-1)/2 calls. A single call
# on a whole set answers 'is there any bad pair inside',
# so you can discard good sets fast and binary-search for
# the culprit inside a bad one. Add the tests one at a
# time to groups known to be clean: if a group plus the
# new test fails, binary search the group for a partner;
# otherwise the new test joins it.
#
# BRUTE FORCE (say it first): Call runTest on every pair
# Test every pair of items with runTest and keep the
# failing pairs. Always N(N-1)/2 calls. Time O(N^2) calls.
# Space O(1).


import itertools


# BRUTE FORCE: ask about every pair
def bad_pairs_brute(items, run_test):
    # every pair costs a call: N * (N - 1) / 2 calls
    pairs = itertools.combinations(items, 2)
    return {(a, b) for a, b in pairs
            if not run_test({a, b})}
```

```
# HELPERS for the base question
def first_partner(group, t, run_test):
    # binary search: the shortest prefix of the group that
    # fails together with t; its last member is a partner
    lo, hi = 1, len(group)
    while lo < hi:
        mid = (lo + hi) // 2
        if run_test(set(group[:mid]) | {t}):
            lo = mid + 1
        else:
            hi = mid
    return group[lo - 1]


def partners(group, t, run_test):
    # all members of the group that are bad with t
    rest, found = list(group), []
    while rest and not run_test(set(rest) | {t}):
        x = first_partner(rest, t, run_test)
        found.append(x)
        # take it out and test the others again
        rest.remove(x)
    return found


# BASE QUESTION (optimal): Group the tests and binary
# search for partners
# Idea: Keep groups of tests that pass together.
def find_bad_pairs(items, run_test):
    # groups of items that pass together (no bad pair
    # inside)
    groups, pairs = [], set()
    for t in items:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                pairs.add((min(x, t), max(x, t)))
            # t joins the first group with no partner for
            # it
            if not found and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs


if __name__ == "__main__":
    def make_oracle(bad):
        calls = [0]

        def run_test(subset):
            calls[0] += 1
            ordered = sorted(subset)
            pairs = itertools.combinations(ordered, 2)
            return not any(p in bad for p in pairs)
        return run_test, calls

    bad = {(3, 17), (5, 29)}
    run, calls = make_oracle(bad)
    assert find_bad_pairs(list(range(30)), run) == bad
    # far fewer than the 435 calls of the brute force
    assert calls[0] < 435 // 2
    # no bad pairs: about one call per item
    run, calls = make_oracle(set())
    assert find_bad_pairs(list(range(30)), run) == set()
    assert calls[0] <= 30
    run, _ = make_oracle({(1, 2)})
    assert bad_pairs_brute([1, 2, 3, 4], run) == {(1, 2)}
```

NOTES FROM CANDIDATES

Standard "group testing" / Dorfman testing problem. Walk through the analogy with COVID pooled tests for intuition. Algorithm sketch:

Test the full set. If true, no bad pairs.
Otherwise, binary-search to find one bad pair by partitioning the set and testing halves.
Remove the items that participate in the located bad pair; repeat.

Tighter bound: with up to K bad pairs, total queries are O(K log N) using adaptive group testing.
A naive O(N²) pairwise scheme works but the interviewer expects you to beat it with adaptive splitting.
Verbalize correctness: each binary-search step rules out half the candidates; you can always reduce the search space until exactly one bad pair is isolated.

PREPARATION

Read the Wikipedia article on group testing and Dorfman's two-stage scheme.
Drill recursive binary partitioning on a candidate set with positive/negative oracle feedback.
Have a back-of-envelope analysis ready: "Total calls O(K log N) for K bad pairs and N items."

STUDY HERE: topic 9, Greedy and Math | main topic: Greedy

MEDIUM | PracHub | Technical Screen | 10 of 10

Min boards to cover roof holes

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 150/228

ABOUT THIS QUESTION

Topic: Greedy and Math: Greedy

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #452 Minimum Number of Arrows to Burst Balloons

LeetCode match: No direct match; related LC 1326 Minimum Number of Taps / LC 452 (inferred from overview)

Posted on PracHub: 2026-02-12 (updated 2026-05-05)

Original:
https://prachub.com/coding-questions/min-boards-to-cover-roof-holes

QUESTION

Overview: This question evaluates competency in combinatorial optimization and problem modeling for coverage constraints, testing algorithmic reasoning, complexity analysis, and efficient representation of grid-based data.

APPROACH

Greedy interval cover: sort hole positions, place a board starting at the first uncovered hole, skip all holes within its length, repeat.

INTERVIEW-DAY PLAN

CLARIFY FIRST

My copy has only the overview, so ask: are holes points on a line or cells in a grid, what is the board length, can boards be placed in either orientation, and must each board be axis aligned. The 1D case is a classic interval cover.

BRUTE FORCE TO SAY OUT LOUD

Try every placement of boards. Exponential.

OPTIMAL IDEA

1D with boards of length L: sort the holes, place a board starting at the first uncovered hole so it covers [h, h + L - 1], skip every hole inside it, repeat. The count is the answer, and the exchange argument shows starting at the leftmost uncovered hole is optimal. O(n log n). For a grid with horizontal boards only, do this per row. If boards can be placed in either orientation covering rows or columns, it becomes a minimum vertex cover in a bipartite graph (Konig's theorem), solved with matching.

DRY-RUN THESE CASES

No holes (0), one hole, holes spread more than L apart, adjacent holes, holes at the same position.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Boards of different lengths, or a limit on the number of boards.

PITFALLS

Starting a board in the middle of an uncovered gap. Not sorting the holes first.

HOW TO THINK ABOUT IT

Cover the leftmost hole first: any board that covers it can be slid right until it starts exactly at that hole without uncovering anything to its right, so start a board at the leftmost uncovered hole and skip everything it covers. Repeating that is optimal because starting later never covers more.

Basis: Written; interval covering (compare LeetCode 452 and 1326). Assumed rule (confirm it): holes are integer positions on a line, a board of length L covers positions p .. p + L - 1, boards may be placed anywhere; minimise the number of boards.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (len(holes) + 1)
best[0] = 0
for i in range(1, len(holes) + 1):
    for j in range(i):
        span = holes[i - 1] - holes[j] + 1
        if span <= L:  # one board covers j..i-1
            best[i] = min(best[i], best[j] + 1)
answer = best[len(holes)]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every i tries every j, O(n^2). Starting a board at the leftmost uncovered hole and skipping everything it covers is optimal and O(n log n) including the sort.

OPTIMAL SOLUTION · Greedy: start a board at the leftmost uncovered hole

TIME	SPACE
O(n log n)	O(1) extra

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 You have only K boards: what is the smallest board length that covers all holes?

Binary search the length L and test with the greedy (is the number of boards at most K?); feasibility is monotone in L. O(n log range).

FOLLOW-UP 2 Boards of different lengths with a cost each.

The greedy exchange argument breaks. Use DP over the sorted holes: best[i] = min over board types of best[j] + cost, where j is the first hole not covered by that board starting at hole i.

FOLLOW-UP 3 Holes are in a grid and boards are 1 x L horizontally.

Apply the greedy independently on every row. If boards may also be vertical it becomes a covering problem on a bipartite graph (König's theorem).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Min boards to cover roof holes
#
# This question evaluates competency in combinatorial
# optimization and problem modeling for coverage
# constraints, testing algorithmic reasoning, complexity
# analysis, and efficient representation of grid-based
# data.
#
# FOLLOW-UPS (answered below, in this order):
# 1. You have only K boards: what is the smallest board
# length that covers all holes?
# 2. Boards of different lengths with a cost each.
# 3. Holes are in a grid and boards are 1 x L
# horizontally.
#
# INTERVIEW NOTES
# Cover the leftmost hole first: any board that covers it
# can be slid right until it starts exactly at that hole
# without uncovering anything to its right, so start a
# board at the leftmost uncovered hole and skip everything
# it covers. Repeating that is optimal because starting
# later never covers more.
#
# BRUTE FORCE (say it first): DP over the number of holes
# covered
# best[i] is the fewest boards that cover the first i
# holes: try every start j of the last board (it must
# cover holes j..i-1 within length L). Exact and the
# oracle; quadratic. Time O(n^2). Space O(n).


from bisect import bisect_left


# BRUTE FORCE (say it first): DP over the number of holes
# covered | time O(n^2)
def boards_dp(holes, L):
    h = sorted(set(holes))
    best = [0] + [float("inf")] * len(h)
    for i in range(1, len(h) + 1):
        for j in range(i):
            if h[i - 1] - h[j] < L:
                best[i] = min(best[i], best[j] + 1)
    return best[-1]
```

```
# BASE QUESTION (optimal): Greedy: start a board at the
# leftmost uncovered hole
# Idea: Sort the holes. Place a board starting at the
# first uncovered hole, which covers every hole below
# start + L; skip those and repeat.
def min_boards(holes, length):
    # greedy: the leftmost uncovered hole starts a board,
    # which
    # then covers `length` positions
    count, covered_to = 0, float("-inf")
    for p in sorted(set(holes)):
        if p > covered_to:
            count += 1
            covered_to = p + length - 1
    return count


# FOLLOW-UP 1: You have only K boards: what is the
# smallest board length that covers all holes?
# Answer: Binary search the length L and test with the
# greedy (is the number of boards at most K?); feasibility
# is monotone in L. O(n log range).
def smallest_length(holes, k):
    # only k boards: the shortest board length that still
    # covers everything (more length never hurts: binary
    # search)
    pts = sorted(set(holes))
    if not pts:
        return 0
    lo, hi = 1, pts[-1] - pts[0] + 1
    while lo < hi:
        mid = (lo + hi) // 2
        if min_boards(pts, mid) <= k:
            hi = mid
        else:
            lo = mid + 1
    return lo


# FOLLOW-UP 2: Boards of different lengths with a cost
# each.
# Answer: The greedy exchange argument breaks. Use DP over
# the sorted holes: best[i] = min over board types of
# best[j] + cost, where j is the first hole not covered by
```

```
# that board starting at hole i.
def min_cost(holes, boards):
    # boards = [(length, cost)], any number of each: a DP
    # over
    # the sorted holes (the greedy no longer works)
    pts = sorted(set(holes))
    INF = float("inf")
    best = [0] * (len(pts) + 1)
    for i in range(len(pts) - 1, -1, -1):
        best[i] = INF
        for length, cost in boards:
            # the first hole this board does not reach
            j = bisect_left(pts, pts[i] + length)
            best[i] = min(best[i], cost + best[j])
    return best[0]


# FOLLOW-UP 3: Holes are in a grid and boards are 1 x L
# horizontally.
# Answer: Apply the greedy independently on every row. If
# boards may also be vertical it becomes a covering
# problem on a bipartite graph (König's theorem).
def boards_per_row(grid, length):
    # holes in a grid ('#'), horizontal boards: greedy per
    # row
    total = 0
    for row in grid:
        holes = [c for c in range(len(row))
                 if row[c] == "#"]
        total += min_boards(holes, length)
```

```
    return total


if __name__ == "__main__":
    assert min_boards([1, 2, 4, 5, 9], 3) == 3
    assert min_boards([], 3) == 0
    assert min_boards([5, 5, 5], 1) == 1
    # with 2 boards the shortest length is 5 (covers 1..5,
    # 9..13)
    assert smallest_length([1, 2, 4, 5, 9], 2) == 5
    assert smallest_length([1, 2, 4, 5, 9], 1) == 9
    assert smallest_length([], 1) == 0
    # a long board costs 5, two short ones cost 3 each
    assert min_cost([1, 2, 3, 10], [(3, 5), (1, 3)]) == 8
    assert min_cost([1, 2, 3, 4], [(2, 3), (4, 5)]) == 5
```

```
    roof = ["#.#.", "..##", "...."]
    assert boards_per_row(roof, 2) == 3
```

FUNCTION AND TEST CASES

Signature

```
def solution(roof: list[list[int]]) -> int:
SAMPLE INPUT
[[1,0,1],[0,1,1]]
```

Test cases

Input	Expected
`[[1,0,1],[0,1,1]]`	`2`
`[[1,1],[1,1]]`	`0`

GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 10 OF 10 | LESS COMMON TOPIC

Grids and Simulation

6 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2	4
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Grids and Simulation | 6 questions | tap a title to jump to it

SIMULATION (3)

M	1. Tic-Tac-Toe System with K Players and 3-in-a-Row Win	1Point3Acres	2026-05-18
M	2. Gate Entry / Exit Time Resolution with Tie-Break Rules	1Point3Acres	2026-05-04
H	3. Babylon Tile-Stack Merge Game	1Point3Acres	2025-12-29

GRIDS (3)

H	4. Count Overlapping Rectangle Updates on a Grid	PracHub	2026-07-10
H	5. Simulate In-Place Cellular State Updates	PracHub	2026-05-04
H	6. Count same-color squares in a character grid	PracHub	2025-11-22

STUDY HERE: topic 10, Grids and Simulation | main topic: Simulation

Site tags: object-design, grid, simulation, medium

MEDIUM | 1Point3Acres | phone-screen, tech-screen | 1 of 6

Tic-Tac-Toe System with K Players and 3-in-a-Row Win

ASKED 2026-05-18 | 155 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | **ALL TIME 18.1** | **30 DAYS 0** | **PRIORITY 154/228**

ABOUT THIS QUESTION

Topic: Grids and Simulation: Simulation

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 348 Design Tic-Tac-Toe (related)

Frequency: Single report | Last asked: 2026-05-18 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/tic-tac-toe-k-players-three-in-a-row

QUESTION

Design a Tic-Tac-Toe system for **k** players on an **n x n** board. After each move, return the game status; any player wins immediately after forming 3 consecutive marks in a straight line.

Input parameters: k players and an n x n board.
Players take turns placing their marks on empty cells.
A player wins as soon as they form 3 consecutive marks in a straight line, regardless of how large n is.
After each move, return the correct game status, such as Game Over or <player> won.
Discuss implementation details and time complexity before coding.

APPROACH

Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

HOW TO THINK ABOUT IT

A move can only create a win through the cell just played, so after each move look only along the four lines through that cell (row, column, both diagonals) and count how far the same player's marks extend in both directions. With a target of 3 that is a handful of cells per move, instead of rescanning the whole board.

Basis: LeetCode 348, Design Tic-Tac-Toe, generalised to k players, an n x n board and a fixed run length of 3.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def status(player):
    for r, c in all_cells(board):
        for dr, dc in DIRECTIONS:
            cells = line_of_3(r, c, dr, dc)
            if all(board[x][y] == player
                   for x, y in cells):
                return "WIN"
    if is_full(board):
        return "DRAW"
    return "PLAYING"
```

TIME	SPACE
O(n^2) per move	O(n^2)

WHY IT IS SLOW

Scans O(n^2) cells per move. Only lines through the cell just played can have changed, so checking four directions around it is O(1).

OPTIMAL SOLUTION · Check the four lines through the new mark

TIME	SPACE
O(1) per move (O(target) when generalised)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What changes when the winning length is a parameter w?

Replace the constant 3 by w in the run check: the cost per move becomes O(w) because each of the four directions walks at most w - 1 cells each way. The board size does not matter any more, which is the point of checking locally.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Tic-Tac-Toe System with K Players and
# 3-in-a-Row Win
#
# Design a Tic-Tac-Toe system for k players on an n x n
# board. After each move, return the game status; any
# player wins immediately after forming 3 consecutive
# marks in a straight line. Input parameters: k players
# and an n x n board. Players take turns placing their
# marks on empty cells. A player wins as soon as they form
# 3 consecutive marks in a straight line, regardless of
# how large n is. After each move, return the correct game
# status, such as Game Over or <player> won. Discuss
# implementation details and time complexity before
```

```
# coding.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What changes when the winning length is a parameter
# w?
#
# INTERVIEW NOTES
# A move can only create a win through the cell just
# played, so after each move look only along the four
# lines through that cell (row, column, both diagonals)
# and count how far the same player's marks extend in both
# directions. With a target of 3 that is a handful of
# cells per move, instead of rescanning the whole board.
#
# BRUTE FORCE (say it first): Rescan the whole board after
# every move
# After each move scan every cell and every direction for
# a run of 3. Correct and a good oracle; wasteful, because
# only lines through the new mark can have changed. Time
# O(n^2) per move. Space O(n^2).


# HELPERS for the brute force
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


# BRUTE FORCE (say it first): rescan the whole board
class TicTacToeBrute:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        self.board = [[0] * n for _ in range(n)]
        self.moves = 0
```

```
        self.over = False

    def winner(self):
        # every cell, every direction: is there a run?
        n, t, b = self.n, self.target, self.board
        for i in range(n):
            for j in range(n):
                for di, dj in DIRECTIONS:
                    cells = [(i + di * s, j + dj * s)
                             for s in range(t)]
                    inside = all(0 <= x < n and 0 <= y < n
                                 for x, y in cells)
                    if inside and b[i][j] and all(
                            b[x][y] == b[i][j]
                            for x, y in cells):
                        return b[i][j]
        return 0

    def move(self, player, r, c):
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not (0 <= r < self.n and 0 <= c < self.n):
            raise ValueError("illegal cell")
        if self.board[r][c]:
            raise ValueError("illegal cell")
        self.board[r][c] = player
        self.moves += 1
        if self.winner():
            self.over = True
            return f"player {player} wins"
        if self.moves == self.n * self.n:
            self.over = True
            return "draw"
        return "continue"


# BASE QUESTION (optimal): Check the four lines through
# the new mark
# Idea: Place the mark, then for each of the four
# directions walk outward both ways while the cells hold
# the same player and add the two counts plus one. A total
# of at least 3 wins immediately.
# FOLLOW-UP 1: What changes when the winning length is a
# parameter w?
```

```
# Answer: Replace the constant 3 by w in the run check:
# the cost per move becomes O(w) because each of the four
# directions walks at most w - 1 cells each way. The board
# size does not matter any more, which is the point of
# checking locally.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n, self.k, self.target = n, k, target
        self.board = [[0] * n for _ in range(n)]
        self.moves = 0
        self.over = False

    def inside(self, r, c):
        return 0 <= r < self.n and 0 <= c < self.n

    def run_length(self, r, c, dr, dc):
        # marks of the same player in a line through (r,
        # c)
        player, run = self.board[r][c], 1
        for sign in (1, -1):
            a, b = r + sign * dr, c + sign * dc
            while self.inside(a, b) and \
                    self.board[a][b] == player:
                run += 1
                a, b = a + sign * dr, b + sign * dc
        return run

    def move(self, player, r, c):
        # whose turn is it? players go 1, 2, ..., k, 1,
        # ...
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not self.inside(r, c) or self.board[r][c]:
            raise ValueError("illegal cell")
        self.board[r][c] = player
        self.moves += 1
        # only lines through the new mark can have changed
        for dr, dc in DIRECTIONS:
            run = self.run_length(r, c, dr, dc)
            # FOLLOW-UP 1: win with `target` in a row
            if run >= self.target:
                self.over = True
                return f"player {player} wins"
        if self.moves == self.n * self.n:
            self.over = True
            return "draw"
        return "continue"


if __name__ == "__main__":
    g = TicTacToe(3, 2)
    plan = [(1, 0, 0), (2, 1, 0), (1, 0, 1),
            (2, 1, 1), (1, 0, 2)]
    results = [g.move(p, r, c) for p, r, c in plan]
    assert results[:-1] == ["continue"] * 4
    assert results[-1] == "player 1 wins"
    # three players on a 4 x 4 board: player 2 wins down
    h = TicTacToe(4, 3)
    seq = [(1, 0, 0), (2, 0, 3), (3, 3, 3), (1, 1, 0),
           (2, 1, 3), (3, 3, 0), (1, 2, 1), (2, 2, 3)]
    out = [h.move(p, r, c) for p, r, c in seq]
    assert out[-1] == "player 2 wins"
    # wrong turn and used cell are rejected
    for bad in ((2, 0, 0), (1, 5, 5)):
        try:
```

```
            TicTacToe(3, 2).move(*bad)
            raise AssertionError(bad)
        except ValueError:
            pass
    # the win length is a parameter: 4 in a row
    w = TicTacToe(5, 2, target=4)
    for p, r, c in [(1, 0, 0), (2, 1, 0), (1, 0, 1),
                    (2, 1, 1), (1, 0, 2), (2, 1, 2)]:
        assert w.move(p, r, c) == "continue"
    # the brute force agrees with the fast version
    import random
    for _ in range(100):
        fast, slow = TicTacToe(4, 3), TicTacToeBrute(4, 3)
        cells = [(r, c) for r in range(4)
                 for c in range(4)]
        random.shuffle(cells)
        for i, (r, c) in enumerate(cells):
            p = i % 3 + 1
            got = fast.move(p, r, c)
            assert got == slow.move(p, r, c)
            if got != "continue":
                break
```

NOTES FROM CANDIDATES

Check four directions through the new mark: horizontal, vertical, main diagonal, anti-diagonal. Count contiguous marks on both sides; win if total reaches 3.
Avoid scanning the whole board after every move unless n is tiny. The local directional check is O(1) for the fixed 3-in-a-row target, or O(target) if generalized.
Clarify invalid moves, turn order, draw state, and whether 3-in-a-row must be exactly 3 or at least 3.

PREPARATION

Implement the classic Tic-Tac-Toe design problem, then generalize from 2 players / 3x3 to k players / n x n.
Practice local directional win checks with boundary guards.
Write tests for corners, edges, duplicate moves, and simultaneous longer lines.

STUDY HERE: topic 10, Grids and Simulation | main topic: Simulation

Site tags: simulation, algorithm, medium

MEDIUM | 1Point3Acres | onsite-coding | 2 of 6

Gate Entry / Exit Time Resolution with Tie-Break Rules

ASKED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 27.4** **30 DAYS 0** **PRIORITY 131/228**

ABOUT THIS QUESTION

Topic: Grids and Simulation: Simulation

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 2534 Time Taken to Cross the Door

Frequency: Single report | Last asked: 2026-05-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/gate-entry-exit-scheduler

QUESTION

Onsite coding round 2: given an array indexed by person id, each containing **`(timestamp, action in {enter, exit})`**, return per-person the actual time they passed through the gate after applying tie-break rules between simultaneous enters and exits.SWEsimulationalgorithmmediumFrequencySingle reportLast asked2026-05-04Stageonsite-coding

APPROACH

Translate the four priority rules into an explicit state machine (prev_state in {enter, exit, none}) before writing any code.
Practice scheduling problems that pair a per-tick simulation with an event-driven optimization (CPU scheduler, traffic light, calendar conflict resolver).
Stress-test corner cases: same timestamp containing both an enter and an exit for the same person, all-enters or all-exits batches, and very large gaps between two events.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the exact tie-break rules in the four cases (previous second unused, entered, exited, and which wins), whether one person passes per second, and how ties between equal arrival times break (by id).

BRUTE FORCE TO SAY OUT LOUD

Simulate every second between the first and last timestamp. Works, but its cost grows with the size of the timestamps.

OPTIMAL IDEA

Sort people by (time, id) and keep two queues. At each step apply the previous-state rule to choose a queue. When both queues are empty jump to the next arrival time and reset the previous state to idle if at least one second was skipped. Linear after the sort.

DRY-RUN THESE CASES

Same-time enter and exit, an idle gap of exactly one second and of many seconds, all enters, all exits.

LIKELY FOLLOW-UPS

Several gates, priority classes, or answering online.

PITFALLS

Forgetting to reset the state across an idle gap. Letting the clock go backward.

HOW TO THINK ABOUT IT

Write the rule as a small state machine first: the state is what the gate did in the previous second (entered, exited or idle), and the rule picks who goes next from that state. Then replace the second-by-second loop with a loop over people: when nobody is waiting jump the clock forward, and remember that an idle gap resets the state to idle.

Basis: LeetCode 2534, Time Taken to Cross the Door. Assumption: one person passes per second; if an enter and an exit compete, the door's previous-second use decides (enter if it was an enter, otherwise exit, and exit if the door was idle).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arrivals):
    t, prev = 0, None
    enter_q, exit_q = deque(), deque()
    while pending(arrivals, enter_q, exit_q):
        add_arrivals(t, arrivals, enter_q,
                     exit_q)
        queue = choose(prev, enter_q, exit_q)
        if queue:
            person = queue.popleft()
            passed[person] = t
            prev = name_of(queue)
        t += 1
    return passed
```

TIME	SPACE
O(T + n)	**O(n)**
where T is the last timestamp	

WHY IT IS SLOW

It runs through every second between the first and last timestamp, so a timestamp of 10^9 means a billion iterations. Jumping to the next arrival when the queues are empty makes it O(n log n).

OPTIMAL SOLUTION · Event-driven simulation with two queues

TIME	SPACE
O(n log n) for the sort, O(n) simulation	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the optimization need care across an empty gap?

If the door sits idle for even one second the previous state becomes 'none', which favours exits again. Jumping time forward without resetting the state would wrongly keep 'enter' priority from before the gap. Compare t with last_used + 1 to know whether the gap really was idle.

FOLLOW-UP 2 What is the complexity, and what if events come online?

Offline it is O(n log n) for the sort, then O(n) for the simulation, independent of how large the timestamps are. Online, keep the two queues and the previous-state variable and advance the clock on every arrival or tick; the work per person stays O(1).

FOLLOW-UP 3 Timestamps go up to 10^9.

That is why the event-driven loop jumps to the next arrival: its running time is O(n log n) and does not depend on the size of the timestamps. Remember to reset the previous state to idle across a gap.

FOLLOW-UP 4 The tie-break when the gate was idle differs in this version of the question.

Keep the simulation and make the idle winner a parameter: only the line that picks the queue changes. With 'exit first' it equals the standard rule; with 'enter first' an enter and an exit arriving together swap places.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Gate Entry / Exit Time Resolution with
# Tie-Break Rules
#
# Onsite coding round 2: given an array indexed by person
# id, each containing (timestamp, action in {enter, exit})
,
# return per-person the actual time they passed through
# the gate after applying tie-break rules between
# simultaneous enters and exits.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the optimization need care across an empty
# gap?
# 2. What is the complexity, and what if events come
# online?
# 3. Timestamps go up to 10^9.
# 4. The tie-break when the gate was idle differs in this
# version of the question.
#
# INTERVIEW NOTES
# Write the rule as a small state machine first: the state
# is what the gate did in the previous second (entered,
# exited or idle), and the rule picks who goes next from
# that state. Then replace the second-by-second loop with
# a loop over people: when nobody is waiting jump the
# clock forward, and remember that an idle gap resets the
# state to idle.
#
# BRUTE FORCE (say it first): Simulate every second
# Advance time one second at a time, add arrivals to two
# queues, and pick a queue by the previous-state rule.
# Easy to reason about and the oracle for the event-driven
# version, but it spends time on every empty second. Time
# O(T + n) where T is the last timestamp. Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Simulate every second | time
# O(T + n) where T is the last timestamp
def gate_ticks(events):
    n = len(events)
    order = sorted(
        range(n), key=lambda i: (events[i][0], i)
    )
    res, ent, ext, prev, t, k = (
        [None] * n,
        deque(),
        deque(),
        None,
        0,
        0,
    )
    while any(r is None for r in res):
        while k < n and events[order[k]][0] <= t:
            (
                ent
                if events[order[k]][1] == "enter"
                else ext
            ).append(order[k])
            k += 1
        if ent and ext:
            q = ent if prev == "enter" else ext
        else:
            q = ent or ext or None
        if q is None:
            prev = None
        else:
            p = q.popleft()
            res[p], prev = t, events[p][1]
        t += 1
    return res


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort people by (arrival time, id) and keep one
# FIFO queue for waiting enters and one for exits. At each
# second pick from the queues using the previous-second
# state.
# FOLLOW-UP 4: The tie-break when the gate was idle
```

```
# differs in this version of the question.
# Answer: Keep the simulation and make the idle winner a
# parameter: only the line that picks the queue changes.
# With 'exit first' it equals the standard rule; with
# 'enter first' an enter and an exit arriving together
# swap places.
def gate_pass_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit")
    n = len(events)
```

```
    # people sorted by arrival, then by id
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    queues = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or queues["enter"] or queues["exit"]:
        if not queues["enter"] and not queues["exit"]:
            # nobody waits: jump the clock to the next
            # arrival
            t = max(t, events[order[i]][0])
            # a gap of idle time forgets the previous
            # action
            if t > last_used + 1:
                prev = None
        # everyone who arrived by now joins their line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            queues[events[person][1]].append(person)
            i += 1
        both = queues["enter"] and queues["exit"]
        if both:
            # FOLLOW-UP 4: idle tie-break set by
            # idle_first
            # tie: continue the direction used last second
            # (when idle, use idle_first)
            want = prev or idle_first
        else:
            want = "enter" if queues["enter"] else "exit"
        person = queues[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result


if __name__ == "__main__":
    # one enter and one exit arrive together: exit first
    ev = [(0, "enter"), (0, "exit")]
    assert gate_pass_times(ev) == [1, 0]
    # with "enter" winning the idle tie it swaps
    assert gate_pass_times(ev, "enter") == [0, 1]
    # the same direction continues while people keep
    # coming
    ev2 = [(0, "enter"), (0, "enter"), (1, "exit")]
```

```
    assert gate_pass_times(ev2) == [0, 1, 2]
    # a long idle gap resets the state
    ev3 = [(0, "enter"), (10, "enter"), (10, "exit")]
    assert gate_pass_times(ev3) == [0, 11, 10]
    # huge timestamps cost nothing extra
    far = [(10 ** 9, "enter"), (10 ** 9, "exit")]
    assert gate_pass_times(far) == [10 ** 9 + 1, 10 ** 9]
```

NOTES FROM CANDIDATES

The clean implementation uses two queues (pending enters, pending exits) plus a prev_state variable; at each tick peek both queues, apply the four rules, advance, and update prev_state.
A naive tick-by-tick simulation walks every integer time unit from min to max timestamp; the expected optimization is to jump directly to the next event timestamp in the input, dropping complexity to O(n).
The optimization has to carry the "prior moment" state across empty gaps correctly (a 100-tick gap still counts as prev_state = none).
Walk through one or two examples on the whiteboard before coding — interviewers in this family prefer candidates who fully verbalize the precedence rules before writing.

PREPARATION

Translate the four priority rules into an explicit state machine (prev_state in {enter, exit, none}) before writing any code.
Practice scheduling problems that pair a per-tick simulation with an event-driven optimization (CPU scheduler, traffic light, calendar conflict resolver).
Stress-test corner cases: same timestamp containing both an enter and an exit for the same person, all-enters or all-exits batches, and very large gaps between two events.

STUDY HERE: topic 10, Grids and Simulation | main topic: Simulation

Site tags: simulation, data-structure, hard

HARD | 1Point3Acres | onsite-coding | 3 of 6

Babylon Tile-Stack Merge Game

ASKED 2025-12-29 | 295 DAYS AGO | OLDER

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 191/228**

ABOUT THIS QUESTION

Topic: Grids and Simulation: Simulation

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2025-12-29 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/babylon-stack-merge-game

QUESTION

Onsite NG round 1 (Munich office): implement a two-player tile-stack merge game ("Babylon") given the rules. Start state is 12 height-1 stacks of 4 colors (Y / W / G / B), 3 of each; a move places one stack on top of another under a constraint; the player who cannot move loses.

12 tiles initially, partitioned into 12 height-1 stacks.
4 colors: Y / W / G / B; exactly 3 tiles per color.
Two players alternate moves.
A move = pick two stacks and place one entirely on top of the other (merge).

Legal iff at least one of: (a) the two stacks have the same height, OR (b) their top-most tiles share the same color.
After merge: new height = sum of the two heights, new top color = color of the stack placed on top.

The player who has no legal move on their turn loses.
Implement the full game so it can be played out (legal-move detection, state transition, terminal check).

EXAMPLES

Start: 12 stacks of height 1, with tops Y Y Y W W W G G G B B B. A legal opening move: stack the first Y on the second Y (same color top), producing one height-2 stack of color Y plus 10 singletons.

APPROACH

Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

HOW TO THINK ABOUT IT

Model a state as the list of stacks, each as (height, top colour). A legal move puts stack i on stack j when the heights are equal or the top colours are equal; the new stack has the summed height and the colour of the stack that was placed on top. The player with no legal move loses, so a state is winning when some move leads to a losing state. The order of stacks does not matter, so sort the list to share states in a cache.

Basis: Written for this question: a two-player impartial game (the player who cannot move loses), solved by memoised search over the multiset of stacks.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def wins(position):
    for i, j in legal_moves(position):
        nxt = merge(position, i, j)
        if not wins(nxt):
            return True  # opponent loses
    return False
```

TIME	SPACE
exponential	O(depth)

WHY IT IS SLOW

Explores the same positions many times, exponential. Sorting the stacks and caching each position's result visits every distinct position once.

OPTIMAL SOLUTION · Negamax with memoisation on the sorted stack tuple

TIME	SPACE
O(states * moves^2)	O(states)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How big is the real state space?

With 12 starting stacks the first position has at most 66 unordered pairs and the number of reachable multisets of (height, colour) stacks shrinks quickly because every merge removes a stack, so memoised search is feasible. In the interview build the clean state class, legal moves and play(move) first and add the solver only if there is time.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Babylon Tile-Stack Merge Game
#
```

```
# Onsite NG round 1 (Munich office): implement a two-
# player tile-stack merge game ("Babylon") given the
# rules. Start state is 12 height-1 stacks of 4 colors (Y
# / W / G / B), 3 of each; a move places one stack on top
# of another under a constraint; the player who cannot
# move loses. 12 tiles initially, partitioned into 12
# height-1 stacks. 4 colors: Y / W / G / B; exactly 3
# tiles per color. Two players alternate moves. A move =
# pick two stacks and place one entirely on top of the
# other (merge). Legal iff at least one of: (a) the two
# stacks have the same height, OR (b) their top-most tiles
# share the same color. After merge: new height = sum of
# the two heights, new top color = color of the stack
# placed on top. The player who has no legal move on their
# turn loses. Implement the full game so it can be played
# out (legal-move detection, state transition, terminal
# check).
#
# FOLLOW-UPS (answered below, in this order):
# 1. How big is the real state space?
#
# INTERVIEW NOTES
# Model a state as the list of stacks, each as (height,
# top colour). A legal move puts stack i on stack j when
# the heights are equal or the top colours are equal; the
# new stack has the summed height and the colour of the
# stack that was placed on top. The player with no legal
# move loses, so a state is winning when some move leads
# to a losing state. The order of stacks does not matter,
# so sort the list to share states in a cache.
#
# BRUTE FORCE (say it first): Plain recursion over every
# move
# Try every legal move recursively with no caching.
# Correct and fine for a few stacks; it recomputes the
# same positions over and over. Time exponential. Space
# O(depth).


import itertools


# BRUTE FORCE (say it first): Plain recursion over every
```

```
# move | time exponential
def babylon_brute(stacks):
    stacks = list(stacks)
    for i, j in itertools.permutations(
        range(len(stacks)), 2
    ):
        (h1, c1), (h2, c2) = stacks[i], stacks[j]
        if h1 == h2 or c1 == c2:
            rest = [
                s
                for k, s in enumerate(stacks)
                if k not in (i, j)
            ] + [(h1 + h2, c1)]
            if not babylon_brute(rest):
                return True
    return False


# HELPERS for the base question
class Babylon:
    # a stack is (height, colour of its top tile)
    def __init__(self, stacks=None):
        if stacks is None:
            # 12 tiles: Y, W, G, B three times each
            stacks = [(1, c) for c in "YWGB" * 3]
        self.stacks = list(stacks)

    def legal_moves(self):
        # put stack i on top of stack j
        moves = []
        for i, j in itertools.permutations(
                range(len(self.stacks)), 2):
            h1, c1 = self.stacks[i]
            h2, c2 = self.stacks[j]
            # same height, or the same colour on top
            if h1 == h2 or c1 == c2:
                moves.append((i, j))
        return moves

    def after(self, i, j):
        # the stacks after the move (the original is kept)
        (h1, c1), (h2, _) = self.stacks[i], self.stacks[j]
        rest = [s for k, s in enumerate(self.stacks)
                if k not in (i, j)]
        # heights add; the top colour is the moved stack's
        return rest + [(h1 + h2, c1)]

    def play(self, i, j):
        if (i, j) not in self.legal_moves():
            raise ValueError("illegal move")
        self.stacks = self.after(i, j)

    def is_over(self):
        # the player with no move loses
        return not self.legal_moves()


# BASE QUESTION (optimal): Negamax with memoisation on the
# sorted stack tuple
# Idea: legal_moves lists the ordered pairs that may
# merge, apply builds the next state, and wins(state) is
# true when some move reaches a state where the opponent
# loses.
def wins(stacks):
    # does the player to move win with best play?
    cache = {}

    def solve(state):
        if state in cache:
            return cache[state]
        game = Babylon(state)
        result = False
        for i, j in game.legal_moves():
            nxt = tuple(sorted(game.after(i, j)))
            # a move to a position the opponent loses wins
            if not solve(nxt):
                result = True
                break
        cache[state] = result
        return result

    # sorted stacks: equal positions share one cache entry
    return solve(tuple(sorted(stacks)))


# PICK the winning move itself
def best_move(game):
    # pick a move that leaves the opponent lost, if any
    for i, j in game.legal_moves():
        if not wins(game.after(i, j)):
            return i, j
    moves = game.legal_moves()
    return moves[0] if moves else None


if __name__ == "__main__":
    # one stack: no moves, so the player to move loses
    assert Babylon([(1, "Y")]).is_over()
    assert not wins([(1, "Y")])
    # two equal stacks: one move ends the game, so we win
```

```
    assert wins([(1, "Y"), (1, "W")])
    g = Babylon([(1, "Y"), (1, "Y"), (1, "W")])
    assert (0, 1) in g.legal_moves()
    # play a whole game with the solver; the loser is
    # stuck
    while not g.is_over():
        g.play(*best_move(g))
    assert len(g.stacks) >= 1
    try:
        Babylon([(1, "Y"), (2, "W")]).play(0, 1)
        raise AssertionError("move should be illegal")
    except ValueError:
        pass
    # the real start has 12 stacks and up to 132 ordered
    # moves
    assert len(Babylon().legal_moves()) > 0
```

NOTES FROM CANDIDATES

This is closer to an implementation/simulation interview than a clever-algorithm problem — the interviewer wants a clean game state class, legal-move enumeration, and a play(move) API.
Represent state as a List[Stack] where Stack = (height, topColor, fullSequence?). Whether you keep the full color sequence depends on whether the interviewer asks about "undo" or "re-color from top".
Add a minimax / negamax for "who wins from this state with optimal play" as the natural follow-up — branching factor is small (at most C(12, 2) = 66 initially, drops fast).
Reported as "too hard to finish coding in 45 minutes"; the interviewer gives hints throughout. Pacing matters.

PREPARATION

Drill clean game-state implementations: e.g. Connect 4, Othello move generators, in 30 minutes.
Practice writing a generic minimax framework so you can plug in this game's legalMoves(state) and terminal(state) quickly.
Watch the merge invariant carefully: "same height OR same top color" — fail mode is forgetting the alternative branch.

STUDY HERE: topic 10, Grids and Simulation | main topic: Grids

HARD | IN APP LC 2536 | PracHub | Technical Screen | 4 of 6

Count Overlapping Rectangle Updates on a Grid

POSTED 2026-07-10 | 102 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 18.1 | 30 DAYS 0 | PRIORITY 151/228

ABOUT THIS QUESTION

Topic: Grids and Simulation: Grids

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #2536 Increment Submatrices by One

LeetCode match: LC 2536 Increment Submatrices by One

Posted on PracHub: 2026-07-10 (updated 2026-07-18)

Original: https://prachub.com/coding-questions/count-overlapping-rectangle-updates-on-a-grid

QUESTION

Count Overlapping Rectangle Updates on a Grid Implement `range_add_counts(n, rectangles)`. Start with an `n x n` matrix of zeroes. Each rectangle is represented as `[row1, col1, row2, col2]` with inclusive, zero-based coordinates. For every rectangle, add one to each matrix cell inside that rectangle. Return the final matrix, where each cell therefore contains the number of input rectangles covering it.

Example

For `n = 3` and `rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]]`, return:

```
[ [1, 1, 0], [1, 2, 1], [0, 1, 1] ]
```

Constraints

- `1 <= n <= 500`
- `0 <= len(rectangles) <= 100_000`
- `0 <= row1 <= row2 < n`
- `0 <= col1 <= col2 < n`

APPROACH

2D difference array: for each rectangle do +1 at (r1,c1), -1 at (r1,c2+1) and (r2+1,c1), +1 at (r2+1,c2+1); then take 2D prefix sums once. O(n^2 + q) instead of O(n^2*q).

HOW TO THINK ABOUT IT

Adding 1 to every cell of up to 100,000 rectangles one cell at a time is too slow. Record only the corners of each rectangle (+1 at the top-left, -1 just beyond the right and bottom edges, +1 at the bottom-right corner beyond), and rebuild the counts with one pass of 2D prefix sums. Each rectangle then costs O(1) instead of its area.

Basis: LeetCode 2536, Increment Submatrices by One (2D difference array).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def range_add_counts(n, rectangles):
    grid = [[0] * n for _ in range(n)]
    for r1, c1, r2, c2 in rectangles:
        for r in range(r1, r2 + 1):
            for c in range(c1, c2 + 1):
                grid[r][c] += 1
    return grid
```

TIME	SPACE
O(R * area)	O(n^2)

WHY IT IS SLOW

O(R * area): 100,000 rectangles of area up to 250,000 is billions of cell updates. A 2D difference array touches four corners per rectangle and one prefix-sum pass rebuilds the counts, O(R + n^2).

OPTIMAL SOLUTION · 2D difference array and prefix sums

TIME	SPACE
O(R + n^2)	O(n^2)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How many cells are covered by at least k rectangles?

Build the count matrix with the difference array and count the cells with value at least k. O(R + n^2).

FOLLOW-UP 2 The grid is huge (10^9 x 10^9) but there are few rectangles.

Coordinate-compress the corner coordinates and apply the same difference array on the compressed grid (at most (2R)^2 cells), or use a sweep line with a segment tree over columns. **Cost:** O(R log R) to sort the corner coordinates plus up to O(R^2) compressed cells, instead of 10^18 cells.

FOLLOW-UP 3 Rectangles are added and the count of a cell is queried online.

Use a 2D Fenwick tree with range update and point query (the difference-array idea turned into a BIT): O(log^2 n) per operation.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Count Overlapping Rectangle Updates on a
# Grid
#
# # Count Overlapping Rectangle Updates on a Grid
# Implement range_add_counts(n, rectangles). Start with an
# n x n matrix of zeroes. Each rectangle is represented as
# [row1, col1, row2, col2] with inclusive, zero-based
# coordinates. For every rectangle, add one to each matrix
# cell inside that rectangle. Return the final matrix,
# where each cell therefore contains the number of input
# rectangles covering it. ## Example For n = 3 and
# rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]], return: text
# [ [1, 1, 0], [1, 2, 1], [0, 1, 1] ] ## Constraints - 1
# <= n <= 500 - 0 <= len(rectangles) <= 100_000 - 0 <=
# row1 <= row2 < n - 0 <= col1 <= col2 < n
#
# FOLLOW-UPS (answered below, in this order):
# 1. How many cells are covered by at least k rectangles?
# 2. The grid is huge (10^9 x 10^9) but there are few
# rectangles.
# 3. Rectangles are added and the count of a cell is
# queried online.
#
# INTERVIEW NOTES
# Adding 1 to every cell of up to 100,000 rectangles one
# cell at a time is too slow. Record only the corners of
# each rectangle (+1 at the top-left, -1 just beyond the
# right and bottom edges, +1 at the bottom-right corner
# beyond), and rebuild the counts with one pass of 2D
# prefix sums. Each rectangle then costs O(1) instead of
# its area.
#
# BRUTE FORCE (say it first): Add one to every cell of
# every rectangle
# Loop over each rectangle and increment all of its cells.
# Correct and the oracle; with 100,000 rectangles of area
# up to 250,000 it is far too slow. Time O(R * area).
# Space O(n^2).


# BRUTE FORCE (say it first): Add one to every cell of
# every rectangle | time O(R * area)
def range_add_brute(n, rects):
    g = [[0] * n for _ in range(n)]
    for r1, c1, r2, c2 in rects:
```

```python
        for r in range(r1, r2 + 1):
            for c in range(c1, c2 + 1):
                g[r][c] += 1
    return g


# BASE QUESTION (optimal): 2D difference array and prefix
# sums
# Idea: For each rectangle add +1 at (r1, c1), -1 at (r1,
# c2 + 1), -1 at (r2 + 1, c1) and +1 at (r2 + 1, c2 + 1)
# in an (n + 1) x (n + 1) array.
def range_add_counts(n, rects):
    # rects are [r1, c1, r2, c2], inclusive. Mark only the
    # 4
    # corners of each rectangle in a difference array ...
    d = [[0] * (n + 1) for _ in range(n + 1)]
    for r1, c1, r2, c2 in rects:
        d[r1][c1] += 1
        d[r1][c2 + 1] -= 1
        d[r2 + 1][c1] -= 1
        d[r2 + 1][c2 + 1] += 1
    # ... then two prefix sums spread them over the cells
    for r in range(n + 1):
        for c in range(1, n + 1):
            d[r][c] += d[r][c - 1]
    for c in range(n + 1):
        for r in range(1, n + 1):
            d[r][c] += d[r - 1][c]
```

```python
    return [row[:n] for row in d[:n]]


# FOLLOW-UP 1: How many cells are covered by at least k
# rectangles?
# Answer: Build the count matrix with the difference array
# and count the cells with value at least k. O(R + n^2).
def cells_at_least(n, rects, k):
    # how many cells are covered by k or more rectangles?
    counts = range_add_counts(n, rects)
    return sum(1 for row in counts for v in row if v >= k)


# FOLLOW-UP 2: The grid is huge (10^9 x 10^9) but there
# are few rectangles.
# Answer: Coordinate-compress the corner coordinates and
# apply the same difference array on the compressed grid
```

```python
# (at most (2R)^2 cells), or use a sweep line with a
# segment tree over columns.
def cells_at_least_compressed(rects, k):
    # a huge grid, few rectangles: compress the
    # coordinates
    rows = sorted({r for r1, _, r2, _ in rects
                   for r in (r1, r2 + 1)})
    cols = sorted({c for _, c1, _, c2 in rects
                   for c in (c1, c2 + 1)})
    ri = {r: i for i, r in enumerate(rows)}
    ci = {c: i for i, c in enumerate(cols)}
    shape = (len(rows) + 1, len(cols) + 1)
    d = [[0] * shape[1] for _ in range(shape[0])]
    for r1, c1, r2, c2 in rects:
        a, b = ri[r1], ci[c1]
        e, f = ri[r2 + 1], ci[c2 + 1]
        d[a][b] += 1
        d[a][f] -= 1
        d[e][b] -= 1
        d[e][f] += 1
    total = 0
    run_col = [0] * (len(cols) + 1)
    for i in range(len(rows) - 1):
        row, run = [0] * (len(cols) + 1), 0
        for j in range(len(cols) - 1):
            run_col[j] += d[i][j]
            run += run_col[j]
            # each compressed cell stands for a block of
            # cells
            if run >= k:
                total += (rows[i + 1] - rows[i]) * (
                    cols[j + 1] - cols[j])
    return total


# FOLLOW-UP 3: Rectangles are added and the count of a
# cell is queried online.
# Answer: Use a 2D Fenwick tree with range update and
# point query (the difference-array idea turned into a
# BIT): O(log^2 n) per operation.
class Fenwick2D:
    """Online updates and point reads: O(log^2 n)."""

    def __init__(self, n):
        self.n = n
```

```python
        self.t = [[0] * (n + 2) for _ in range(n + 2)]

    def _add(self, r, c, v):
        i = r + 1
        while i <= self.n + 1:
            j = c + 1
            while j <= self.n + 1:
                self.t[i][j] += v
                j += j & -j
```

```
            i += i & -i

    def add_rect(self, r1, c1, r2, c2):
        corners = ((r1, c1, 1), (r1, c2 + 1, -1),
                   (r2 + 1, c1, -1), (r2 + 1, c2 + 1, 1))
        for r, c, v in corners:
            self._add(r, c, v)

    def get(self, r, c):
        total, i = 0, r + 1
        while i > 0:
            j = c + 1
            while j > 0:
                total += self.t[i][j]
                j -= j & -j
            i -= i & -i
        return total


if __name__ == "__main__":
    rects = [[0, 0, 1, 1], [1, 1, 2, 2]]
    assert range_add_counts(3, rects) == [
        [1, 1, 0], [1, 2, 1], [0, 1, 1]]
    assert cells_at_least(3, rects, 2) == 1
    assert cells_at_least(3, rects, 1) == 7
    # the compressed version gives the same area
    assert cells_at_least_compressed(rects, 2) == 1
    assert cells_at_least_compressed(rects, 1) == 7
    big = [[0, 0, 10 ** 9, 10 ** 9], [5, 5, 10, 10]]
    assert cells_at_least_compressed(big, 2) == 36
    # online updates and point queries
    f = Fenwick2D(3)
    for r in rects:
        f.add_rect(*r)
    assert f.get(1, 1) == 2 and f.get(0, 2) == 0
```

FUNCTION AND TEST CASES

Signature

```
def range_add_counts(n: int, rectangles: list[list[int]])
-> list[list[int]]:
```

Sample call

```
range_add_counts(1, [])
```

Test cases

Input	Expected
`(1, [])`	`[[0]]`
`(1, [[0, 0, 0, 0]])`	`[[1]]`

STUDY HERE: topic 10, Grids and Simulation | main topic: Grids

HARD | IN APP LC 289 | PracHub | Technical Screen | 5 of 6

Simulate In-Place Cellular State Updates

POSTED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 23.4 | 30 DAYS 0 | PRIORITY 143/228

ABOUT THIS QUESTION

Topic: Grids and Simulation: Grids

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #289 Game of Life

LeetCode match: LC 289 Game of Life (inferred from overview)

Posted on PracHub: 2026-05-04 (updated 2026-05-07)

Original: https://prachub.com/coding-questions/simulate-in-place-cellular-state-updates

QUESTION

Overview: This question evaluates a candidate's ability to implement grid-based state transitions and manage in-place updates with strict space constraints, assessing skills in matrix manipulation, neighbor traversal, and correctness under simultaneous update rules.

APPROACH

Encode old and new state together in each cell (extra bits or special values like 2 and 3) so neighbors still read the old state, then finalize in a second pass. O(mn) time, O(1) extra space.

HOW TO THINK ABOUT IT

'Update every cell at once, in place' means a cell's new value must not disturb its neighbors' reads. The trick is to store both the old and the new state in one cell (two bits), then decode in a second pass. Whenever you need old and new values together with no extra grid, think 'pack two states into one number'.

Basis: LeetCode 289, Game of Life.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def step(board):
    old = [row[:] for row in board]
    for r, c in all_cells(old):
        live = live_neighbours(old, r, c)
        alive = old[r][c] == 1
        if alive and live in (2, 3):
            board[r][c] = 1
        elif not alive and live == 3:
            board[r][c] = 1
        else:
            board[r][c] = 0
```

BRUTE FORCE

TIME	SPACE
O(m * n)	O(m * n)

WHY IT IS SLOW

Time O(m * n) is already optimal; the cost is a second full board, O(m * n) extra space. Storing the next state in a spare bit of each cell removes the copy.

OPTIMAL SOLUTION · Two-bit encoding in the same board

TIME	SPACE
O(m * n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The board is infinite, or huge and mostly empty.

Store only the live cells in a set. A cell's next state depends on its live neighbor count, so count how many live neighbors every candidate cell has with a Counter and keep cells with 3, or with 2 if already live. Work is proportional to the number of live cells.

FOLLOW-UP 2 Why is bit 0 used when counting neighbors?

Neighbors that were already processed in this pass carry their next state in bit 1. Masking with & 1 always reads the original generation, which is the whole point of the encoding.

FOLLOW-UP 3 Cells wrap around the edges (a torus).

Replace the bounds check with modular indexing: (r + dr) % m and (c + dc) % n. Be careful on very small boards where the same neighbor can be reached through two offsets.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Simulate In-Place Cellular State Updates
#
# This question evaluates a candidate's ability to
# implement grid-based state transitions and manage in-
# place updates with strict space constraints, assessing
# skills in matrix manipulation, neighbor traversal, and
# correctness under simultaneous update rules.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The board is infinite, or huge and mostly empty.
# 2. Why is bit 0 used when counting neighbors?
# 3. Cells wrap around the edges (a torus).
#
# INTERVIEW NOTES
# 'Update every cell at once, in place' means a cell's new
```

```
# value must not disturb its neighbors' reads. The trick
# is to store both the old and the new state in one cell
# (two bits), then decode in a second pass. Whenever you
# need old and new values together with no extra grid,
# think 'pack two states into one number'.
#
# BRUTE FORCE (say it first): Copy the board, read from
# the copy
# Compute the next generation from an untouched copy of
# the board. It is the simplest correct answer, and the
# extra copy is exactly what the in-place version removes.
# Time O(m * n). Space O(m * n).


from collections import Counter


# BRUTE FORCE (say it first): Copy the board, read from
# the copy | time O(m * n)
def _life_neighbors(board, r, c, live):
    return sum(
        live(board[nr][nc])
        for nr in range(r - 1, r + 2)
        for nc in range(c - 1, c + 2)
        if (nr, nc) != (r, c)
        and 0 <= nr < len(board)
        and 0 <= nc < len(board[0])
    )
```

```
def life_brute(board):
    old = [row[:] for row in board]
    for r in range(len(board)):
        for c in range(len(board[0])):
            n = _life_neighbors(
                old, r, c, lambda v: v == 1
            )
            board[r][c] = (
                1
                if (old[r][c] == 1 and n in (2, 3))
                or (old[r][c] == 0 and n == 3)
                else 0
            )
    return board


# HELPERS for the base question
AROUND = [(dr, dc) for dr in (-1, 0, 1)
          for dc in (-1, 0, 1) if (dr, dc) != (0, 0)]


def live_neighbors(board, r, c, wrap=False):
    m, n = len(board), len(board[0])
    count = 0
    for dr, dc in AROUND:
        a, b = r + dr, c + dc
        if wrap:
            # a torus: the edges are glued together
            a, b = a % m, b % n
        elif not (0 <= a < m and 0 <= b < n):
            continue
        # bit 0 is the OLD state, even for updated cells
        count += board[a][b] & 1
    return count


# BASE QUESTION (optimal): Two-bit encoding in the same
# board
# Idea: Bit 0 holds the current state and bit 1 holds the
# next state. First pass: count neighbors using only bit 0
# (value & 1) and set bit 1 by the rules.
def step_in_place(board, wrap=False):
    m, n = len(board), len(board[0])
```

```
    for r in range(m):
        for c in range(n):
```

```
            live = live_neighbors(board, r, c, wrap)
            alive = board[r][c] & 1
            # bit 1 stores the NEW state
            if live == 3 or (alive and live == 2):
                board[r][c] |= 2
    for r in range(m):
        for c in range(n):
            # shift the new state into place
            board[r][c] >>= 1
    return board


# FOLLOW-UP 1: The board is infinite, or huge and mostly
# empty.
# Answer: Store only the live cells in a set. A cell's
# next state depends on its live neighbor count, so count
# how many live neighbors every candidate cell has with a
# Counter and keep cells with 3, or with 2 if already
# live.
def step_sparse(live):
    # infinite or huge board: only the live cells are
    # stored
    counts = Counter()
    for r, c in live:
        for dr, dc in AROUND:
            counts[(r + dr, c + dc)] += 1
    return {cell for cell, k in counts.items()
            if k == 3 or (k == 2 and cell in live)}


if __name__ == "__main__":
    board = [[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]]
    want = [[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
    assert step_in_place(board) == want
    # a blinker flips between a row and a column
    blinker = {(1, 0), (1, 1), (1, 2)}
    flipped = {(0, 1), (1, 1), (2, 1)}
    assert step_sparse(blinker) == flipped
    assert step_sparse(step_sparse(blinker)) == blinker
    # a block never changes
    block = {(0, 0), (0, 1), (1, 0), (1, 1)}
    assert step_sparse(block) == block
    # the sparse and dense versions agree on a glider
    glider = {(0, 1), (1, 2), (2, 0), (2, 1), (2, 2)}
    grid = [[0] * 6 for _ in range(6)]
```

```
    for r, c in glider:
        grid[r][c] = 1
    dense = step_in_place(grid)
    assert {(r, c) for r in range(6) for c in range(6)
            if dense[r][c]} == step_sparse(glider)
    # on a torus the left and right edges touch
    ring = [[1, 0, 1]]
    assert live_neighbors(ring, 0, 0, wrap=True) >= 1
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> list[list[int]]:
```

Sample call

```
solution([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]])
```

Test cases

Input	Expected
`([[0, 1, 0], [0, 0, 1], [1, 1, 1], [0, 0, 0]],)`	`[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]`
`([[1, 1], [1, 1]],)`	`[[1, 1], [1, 1]]`

STUDY HERE: topic 10, Grids and Simulation | main topic: Grids

HARD | PracHub | Technical Screen | 6 of 6

Count same-color squares in a character grid

POSTED 2025-11-22 | 332 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 51.2 | 30 DAYS 0 | PRIORITY 66/228

ABOUT THIS QUESTION

Topic: Grids and Simulation: Grids

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #1277 Count Square Submatrices with All Ones, LeetCode #221 Maximal Square

LeetCode match: LC 1277 Count Square Submatrices with All Ones / LC 221 Maximal Square

Posted on PracHub: 2025-11-22 (updated 2026-06-13)

Original: https://prachub.com/coding-questions/count-same-color-squares-in-a-character-grid

QUESTION

You are given a 2D grid (matrix) of characters. Each character represents a color: cells with the same character are considered the same color. Formally:

- The grid has **`m`** rows and **`n`** columns.
- **`grid[i][j]`** is an alphabetic character (e.g., **`'a'`**–**`'z'`** or **`'A'`**–**`'Z'`**). A **square** in this grid is defined as a contiguous **`k × k`** submatrix aligned with the grid axes, for some integer **`k >= 1`**. A square is **monochromatic** if **all** of its cells contain the **same** character (i.e., the same color). Task:
- Count and return the total number of monochromatic squares in the grid.
- Count all sizes of squares (1×1, 2×2, ..., up to the largest possible that fits in the grid). Example (just for clarity, not necessarily exhaustive): If the grid is:

```
a a b a a b b b
```

Then some monochromatic squares include:

- All 1×1 cells (each single cell is a monochromatic square).
- The 2×2 square in the top-left corner formed by **`'a'`**. Your function should return the total count of such monochromatic squares for a given input grid.

QUESTION

You are given a 2D grid of characters, where each character represents a color. A square is a contiguous k × k submatrix aligned with the grid axes, for some k >= 1. A square is monochromatic if all of its cells contain the same character. Return the total number of monochromatic squares of all possible sizes and positions in the grid. Every 1 × 1 cell is considered a monochromatic square.

Constraints

0 <= len(grid) <= 1000If grid is non-empty, 0 <= len(grid[0]) <= 1000All rows in grid have the same lengthgrid[i][j] is an alphabetic characterThe total number of cells is at most 1,000,000

EXAMPLES

```
Input: ([])
Expected Output: 0
```

Explanation: An empty grid contains no squares.

```
Input: (["aab", "aab", "bbb"])
Expected Output: 10
```

Explanation: There are 9 single-cell squares and one 2x2 monochromatic square of 'a' in the top-left corner.

Hints

Try defining dp[i][j] as the side length of the largest monochromatic square whose bottom-right corner is cell (i, j).A square larger than 1 can end at (i, j) only if the current cell matches its top, left, and top-left neighboring cells.

APPROACH

dp[i][j] = 1 + min(top, left, top-left) when those neighbors share the same char as (i,j), else 1. Sum all dp values = number of monochromatic squares. O(mn).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the grid size, whether every k x k square counts (k from 1 up), and whether to count squares of every size (the statement says all).

BRUTE FORCE TO SAY OUT LOUD

For every top-left corner and size, check all cells. O(mn * min(m, n)^3).

OPTIMAL IDEA

dp[i][j] is the size of the largest same-color square whose bottom-right corner is (i, j), and it equals the number of such squares ending there. If i and j are positive and the cells above, left and diagonal match the current character, dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]); otherwise it is 1. The answer is the sum of all dp values. O(mn) time, O(n) with a rolling row.

DRY-RUN THESE CASES

A 1 x 1 grid (1), a grid of one color (sum of squares counts), a checkerboard (only 1 x 1 squares), a rectangle grid.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

The largest square only, or squares allowing at most one different cell.

PITFALLS

Forgetting to compare the three neighbors to the current character, not just to each other.

HOW TO THINK ABOUT IT

Let dp[i][j] be the side of the largest same-colour square whose bottom-right corner is (i, j). That is also the number of squares ending at that corner (one per size from 1 up to it). A square of side k at (i, j) needs squares of side k - 1 at the three cells above, left and above-left, all of the same colour, hence the min of those three plus one.

Basis: LeetCode 1277 (Count Square Submatrices with All Ones) with 'same character' instead of 1. Counts squares of every size k >= 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for i, j in all_cells(grid):
    for k in range(1, min(m - i, n - j) + 1):
        cells = square(grid, i, j, k)
        if all(c == grid[i][j] for c in cells):
            count += 1
```

TIME	SPACE
O(m * n * min(m, n)^3)	O(1)

WHY IT IS SLOW

The same cells are rechecked for every size: O(m * n * min(m, n)^3). A table of the largest same-colour square ending at each cell, summed over all cells, is O(m * n).

OPTIMAL SOLUTION · DP: side of the largest square ending at each cell, summed

TIME	SPACE
O(m * n)	O(n) with a rolling row

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Only the largest square (LeetCode 221).

Keep the maximum of the same dp table instead of the sum.

FOLLOW-UP 2 Allow at most one different cell inside a square.

The simple min-of-three recurrence no longer works. Use 2D prefix counts per colour and test each (corner, size) with binary search on the size, or DP with a second layer 'one defect used'.

FOLLOW-UP 3 Count only squares of colour c, or only side >= 2.

Filter while summing: add dp only for cells of colour c, or add max(0, dp - 1) for sizes at least 2.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Count same-color squares in a character
# grid
#
# You are given a 2D grid (matrix) of characters. Each
# character represents a color: cells with the same
# character are considered the same color. Formally: - The
# grid has m rows and n columns. - grid[i][j] is an
# alphabetic character (e.g., 'a'–'z' or 'A'–'Z'). A
# **square** in this grid is defined as a contiguous k × k
# submatrix aligned with the grid axes, for some integer k
# >= 1. A square is **monochromatic** if **all** of its
# cells contain the **same** character (i.e., the same
# color). Task: - Count and return the total number of
# monochromatic squares in the grid. - Count all sizes of
# squares (1×1, 2×2, ..., up to the largest possible that
# fits in the grid). Example (just for clarity, not
# necessarily exhaustive): If the grid is: a a b a a b b b
# b Then some monochromatic squares include: - All 1×1
# cells (each single cell is a monochromatic square).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Only the largest square (LeetCode 221).
# 2. Allow at most one different cell inside a square.
# 3. Count only squares of colour c, or only side >= 2.
#
# INTERVIEW NOTES
# Let dp[i][j] be the side of the largest same-colour
# square whose bottom-right corner is (i, j). That is also
# the number of squares ending at that corner (one per
# size from 1 up to it). A square of side k at (i, j)
# needs squares of side k - 1 at the three cells above,
# left and above-left, all of the same colour, hence the
# min of those three plus one.
#
# BRUTE FORCE (say it first): Check every square from
# every corner
# For each top-left corner and each size k, verify that
# all k * k cells are equal. Correct and the oracle; the
# cost is huge because the same cells are rechecked for
# every size. Time O(m * n * min(m, n)^3). Space O(1).


# BRUTE FORCE (say it first): Check every square from
# every corner | time O(m * n * min(m, n)^3)
def same_color_squares_brute(grid):
```

```
    m, n, total = len(grid), len(grid[0]), 0
    for i in range(m):
```

```
        for j in range(n):
            for k in range(1, min(m - i, n - j) + 1):
                if all(
                    grid[x][y] == grid[i][j]
                    for x in range(i, i + k)
                    for y in range(j, j + k)
                ):
                    total += 1
                else:
                    break
    return total


# HELPERS for the base question
def square_table(grid):
    # side[i][j]: the biggest same-colour square with its
    # bottom-right corner at (i, j)
    prev, rows = [0] * len(grid[0]), []
    for i, row in enumerate(grid):
        cur = [1] * len(row)
        for j, ch in enumerate(row):
            # the up, left and up-left cells must match
            # too
            if i and j and ch == grid[i - 1][j] \
                    and ch == grid[i][j - 1] \
                    and ch == grid[i - 1][j - 1]:
                cur[j] = 1 + min(prev[j], cur[j - 1],
                                 prev[j - 1])
        rows.append(cur)
        prev = cur
    return rows


# BASE QUESTION (optimal): DP: side of the largest square
# ending at each cell, summed
# Idea: If the cell matches its top, left and top-left
# neighbours then dp = 1 + min(dp top, dp left, dp top-
# left); otherwise 1. Summing dp over all cells counts
# every square exactly once, at its bottom-right corner.
def count_same_color_squares(grid):
    # each cell adds one square for every size up to its
    # side
    return sum(sum(row) for row in square_table(grid))
```

```
# FOLLOW-UP 1: Only the largest square (LeetCode 221).
# Answer: Keep the maximum of the same dp table instead of
# the sum.
def largest_square(grid):
    # LeetCode 221 style: only the biggest side
    return max(max(row) for row in square_table(grid))


# FOLLOW-UP 3: Count only squares of colour c, or only
# side >= 2.
# Answer: Filter while summing: add dp only for cells of
# colour c, or add max(0, dp - 1) for sizes at least 2.
def count_of_color(grid, color, min_side=1):
    # squares of one colour, and/or of side at least
    # min_side
    total = 0
    for i, row in enumerate(square_table(grid)):
        for j, side in enumerate(row):
            if grid[i][j] == color:
                # sides min_side .. side are counted
                total += max(0, side - min_side + 1)
    return total


if __name__ == "__main__":
    g = ["aab", "aab", "bbb"]
    # singles 9, plus the 2 x 2 block of a's
    assert count_same_color_squares(g) == 10
    assert largest_square(g) == 2
    assert count_of_color(g, "a") == 5
    assert count_of_color(g, "b") == 5
    full = ["zzz", "zzz", "zzz"]
    # 9 + 4 + 1 squares
    assert count_same_color_squares(full) == 14
    assert count_of_color(full, "z", 2) == 5
    assert count_of_color(full, "q") == 0
    assert count_same_color_squares(["a"]) == 1
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[str]) -> int:
SAMPLE INPUT
([])
```

Test cases

Input	Expected
`([])`	`0`
`(["aab", "aab", "bbb"])`	`10`

ALSO REPORTED · 1 more

Count Same-Color Squares in Matrix | 1Point3Acres | onsite-coding | Low

https://www.1point3acres.com/interview/problems/company/google/count-same-color-squares

Variant reported in both intern VO and an additional NG round. Common bug: assuming all four corners equal is sufficient — need all three neighbors (top, left, top-left) to match the current cell.

Preparation: Write the all-1-squares DP from memory in under 8 min.
Generalize to multi-color in <2 min — only the equality check changes.